

エンタープライズ 金融モデル構築

Rによる金融データ分析・数理モデル・並列計算

Eiichi Horigome

Table of contents

はじめに	16
金融モデル構築からエンタープライズへ	17
第 I 部 金融工学 均衡モデル	18
第 I-1 章 R によるデータ分析の基礎	19
1.1 R の基礎	19
R の基本操作	19
ベクトル	21
行列、リスト、データフレーム	22
条件判定と独自関数	24
記述統計	25
確率分布	26
1.2 tidyverse によるデータ整形	29
プロジェクト内のデータを読み込む	29
dplyr によるデータ操作	30
stringr による文字列処理	35
tidyr によるデータ整形	36
ggplot2 による可視化	38
1.3 関数型プログラミングと並列計算	43
purrr による反復処理	43
nest によるグループ別データ管理	46
future と furrr による並列計算	48
1.4 データベースと大規模データ処理	51
dbplyr によるデータベース操作	51
Arrow による Parquet データ処理	51
Arrow Dataset	52
Spark との関係	54
分析用データを作る	54
企業別の要約統計量を計算する	55
累積リターンを計算する	55
purrr で企業別の統計量を再計算する	56
累積リターンを可視化する	56
まとめ	57

金融データ分析パイプライン	57
第 I-2 章財務データの整理と分析	59
2.1 分析環境	59
2.2 財務データの読み込みと整備	59
2.2.1 財務データの読み込み	59
2.2.2 データ構造の確認	60
2.2.3 データ型と並び順の整理	61
2.3 標本構成と売上高分布	62
2.3.1 年度別の企業数	62
2.3.2 産業別の企業数	64
2.3.3 売上高の分布	65
2.4 株主資本と ROE 分析	67
2.4.1 株主資本と ROE	67
2.4.2 産業別の ROE	70
2.4.3 企業の順位付け	72
2.5 上級デュボン・モデル	74
2.5.1 ROE のデュボン分解	74
2.5.2 PM と ATO の関係	76
2.6 財務指標の要約と中間データ	79
2.6.1 複数の財務指標の要約	79
2.6.2 中間データの保存	80
2.7 まとめ	81
第 I-3 章株式データと時系列分析	82
3.1 分析環境	82
3.2 株式データの読み込みと整備	82
3.2.1 株式データの読み込み	82
3.2.2 データ構造と欠損値	83
3.2.3 データ型とパネルキー	85
3.3 月次リターンと時価総額	87
3.3.1 配当と調整係数	87
3.3.2 時価総額	89
3.3.3 月次株式リターン	91
3.3.4 無リスク金利と超過リターン	93
3.4 月次リターンの分析	96
3.4.1 要約統計量	96
3.4.2 リターン分布	98
3.4.3 企業別の累積リターン	99
3.4.4 企業別の要約統計量	101
3.5 年次データと財務データの結合	102
3.5.1 月次リターンの年次集約	102
3.5.2 年次財務データの読み込み	103
3.5.3 年次データの結合	105

3.5.4 月次データへの財務情報の付加	105
3.5.5 売上高・利益・時価総額	106
3.6 統計分析	108
3.6.1 月次超過リターンの一標本 t 検定	108
3.6.2 簿価時価比率と年次超過リターン	109
3.6.3 年度別回帰	112
3.7 分析用データの保存	114
3.8 まとめ	116
第 I-4 章ファクターモデル分析	117
4.1 分析環境	117
4.2 入力データ	117
4.2.1 第 3 章の出力の読み込み	117
4.2.2 必須列とパネルキー	120
4.2.3 分析用補助関数	122
4.3 ポートフォリオ形成と市場リターン	122
4.3.1 前年度情報の準備	122
4.3.2 市場ポートフォリオ	124
4.3.3 ポートフォリオ形成ユニバース	128
4.4 単変量ポートフォリオ・ソート	130
4.4.1 企業規模 10 分位ポートフォリオ	130
4.4.2 簿価時価比率 10 分位ポートフォリオ	133
4.5 CAPM	135
4.5.1 モデルの推定	135
4.5.2 証券市場線	138
4.6 2×3 ポートフォリオと SMB・HML	140
4.6.1 Size-BE/ME ポートフォリオ	140
4.6.2 SMB と HML	143
4.6.3 ファクターの要約統計量	145
4.7 FF3 モデルと CAPM の比較	147
4.7.1 FF3 モデルの推定	147
4.7.2 アルファとモデル適合度の比較	150
4.8 ファクター・データの保存	153
4.9 まとめ	154
第 I-5 章資産価格モデルとポートフォリオ最適化	156
5.1 分析対象とデータの接続	156
5.1.1 共通関数と分析データ	156
5.1.2 推定対象企業の選定	157
5.1.3 月次株式リターンとファクターの結合	157
5.2 FF3 モデルによる期待リターンの推定	158
5.2.1 FF3 モデル	158
5.2.2 企業別ファクター・ローディング	159
5.2.3 ファクター・リスクプレミアム	159

5.2.4 株式資本コスト	160
5.3 期待リターンと配当成長率	161
5.3.1 期待リターンの企業間分布	161
5.3.2 企業規模と期待リターン	162
5.3.3 配当利回り	164
5.3.4 潜在配当成長率	165
5.4 ファクターモデルによるリスク推定	166
5.4.1 ファクターの分散共分散	166
5.4.2 投資ユニバース	166
5.4.3 企業固有リスク	167
5.4.4 株式リターンの分散共分散	168
5.4.5 期待リターン	169
5.5 平均分散最適化	170
5.5.1 最適化問題	170
5.5.2 目標期待リターンの設定	170
5.5.3 最適ポートフォリオ	170
5.5.4 最適化結果の検算	171
5.6 効率的フロンティア	172
5.6.1 目標期待リターンの範囲	172
5.6.2 複数ポートフォリオの推定	172
5.6.3 効率的フロンティアの描画	173
5.7 まとめ	173
第 I-6 章 イベントスタディ	175
6.1 イベントスタディの基本設計	175
6.1.1 マーケット・モデル	175
6.1.2 分析期間	176
6.1.3 分析環境	176
6.2 入力データの読み込みと検証	178
6.2.1 入力ファイル	178
6.2.2 データの読み込み	179
6.2.3 必須列の確認	179
6.2.4 データ型の整理	180
6.2.5 キーと欠損値の検証	181
6.2.6 入力データの概要	182
6.2.7 個別株リターンの分布	183
6.2.8 市場リターンの時系列	184
6.3 決算サプライズとイベント分類	185
6.3.1 決算サプライズ	185
6.3.2 イベント強度別の概要	186
6.3.3 決算サプライズの分布	187
6.3.4 イベント強度別の箱ひげ図	188
6.3.5 年度別イベント件数	189
6.4 イベント日とイベント・パネル	190

6.4.1 取引日カレンダー	190
6.4.2 イベント日 0	191
6.4.3 発表日からイベント日 0 までの日数	192
6.4.4 イベント・パネル	193
6.4.5 データ利用可能性	194
6.4.6 推定期間の利用可能日数	196
6.5 マーケット・モデルの推定	197
6.5.1 イベント別回帰	197
6.5.2 推定結果の概要	198
6.5.3 ベータの分布	199
6.5.4 アルファの分布	199
6.5.5 決定係数の分布	200
6.5.6 ベータと決定係数	201
6.6 異常リターンと累積異常リターン	202
6.6.1 AR と CAR	202
6.6.2 AAR と CAAR	203
6.6.3 イベント強度別 CAAR	206
6.6.4 イベント日前後の AAR	207
6.6.5 イベント日 0 の AAR	208
6.6.6 イベント日後の CAAR	209
6.6.7 イベント強度別 AR 分布	211
6.7 イベント・ウィンドウ分析	212
6.7.1 ウィンドウ定義	212
6.7.2 イベント別 CAR	213
6.7.3 イベント強度別の平均 CAR	214
6.7.4 短期 CAR の分布	215
6.7.5 ウィンドウ別平均 CAR	216
6.7.6 正の CAR となったイベントの割合	217
6.7.7 平均 CAR のヒートマップ	218
6.8 決算サプライズと短期 CAR	220
6.8.1 回帰分析	220
6.8.2 決算サプライズと短期 CAR	220
6.8.3 年度別の関係	221
6.9 分析結果の保存	223
6.9.1 イベント単位の結果	223
6.9.2 日次結果	224
6.9.3 Parquet 保存	224
6.9.4 保存結果の検証	225
6.10 まとめ	226
第 1-7 章機械学習によるファクター選定	228
7.1 機械学習によるファクター分析	228
7.2 分析データの作成	229
既存データの読み込み	229

目的変数	230
価格特徴量	230
市場環境特徴量	231
学習データ	231
7.3 tidymodels による学習	233
前処理	233
時系列分割	234
7.4 ファクター選定	235
7.5 期待リターンの推定	237
テスト期間の予測	237
全データによる最終モデル	238
7.6 ポートフォリオ候補の作成	238
7.7 第 8 章へのデータ保存	240
第 I-8 章制約付きポートフォリオ最適化とバックテスト	243
8.1 ポートフォリオ構築の全体像	243
8.2 投資ユニバースとリスク推定	244
第 7 章の出力	244
最初のリバランス年度	245
リターン行列	245
8.3 制約条件	246
完全投資	246
ロングオンリー	246
銘柄別ウェイト上限	247
VaR 上限	247
売買回転率	247
8.4 制約付きポートフォリオ最適化	247
計算関数	247
最適化条件	248
nloptr による最適化	249
8.5 バックテスト	255
8.6 パフォーマンス評価	258
8.7 結果の保存	260
第 II 部 金融工学 無裁定モデル	262
第 II-1 章確定キャッシュフロー債券評価	263
1.1 金利期間構造の確認	263
1.2 マーケットコンベンション	266
1.2.1 一般的な市場慣行	267
1.2.2 日本の債券市場における慣行	267
1.3 固定利付債計算	270
1.3.1 クリーン価格から最終利回り	272
1.3.2 最終利回りから価格	273

1.3.3 金利リスク	274
1.3.4 デュレーションとコンベクシティによる価格変化の近似	275
1.4 キャッシュフローによる検証	276
1.4.1 フラットな利回りによる価格評価	277
1.5 計算結果のまとめ	280
第 II-2 章金利カーブ構築と債券先物 CTD、BPV	282
2.1 金利カーブの基礎	282
2.1.1 ゼロ金利ノード	283
2.1.2 ゼロカーブ	284
2.1.3 カーブの表現	285
2.2 補間と参照日	289
2.2.1 補間方式とフォワード金利	289
2.2.2 参照日と評価日	293
2.3 ブートストラップ	294
2.3.1 預金と債券	294
2.3.2 預金とスワップ	296
2.4 カーブ操作と検証	299
2.4.1 Implied Term Structure	299
2.4.2 カーブの平行シフト	301
2.4.3 カーブの検証	304
2.5 国債先物の基礎	308
2.5.1 計算条件と市場慣行	309
2.5.2 受渡適格国債	310
2.5.3 現物債評価	311
2.5.4 キャッシュフロー検証	313
2.5.5 経過利子	314
2.6 CTD 分析と先物 BPV	316
2.6.1 受渡適格銘柄と変換係数	316
2.6.2 グロスベース	316
2.6.3 キャリーとネットベース	316
2.6.4 インプライド・レポ	317
2.6.5 CTD 調整後の先物 BPV	320
2.7 まとめ	321
第 II-3 章 OIS スワップとマルチカーブ評価	323
3.1 IBOR 改革と OIS の基礎	323
3.1.1 IBOR 改革と歴史的背景	323
3.1.2 OIS と翌日物金利	324
3.2 TONA カーブと短期 OIS 評価	325
3.2.1 TONA 市場レート	325
3.2.2 TONA OIS カーブの構築	327
3.2.3 JPY 短期 OIS の評価	328
3.2.4 フェアレートの手計算	329

3.3 日次複利と Fixing	330
3.3.1 OIS 変動レグの日次複利	330
3.3.2 Fixing の確定と再評価	332
3.4 SOFR とマルチカーブ評価	334
3.4.1 SOFR 市場レートと OIS カーブ	334
3.4.2 SOFR OIS の評価	335
3.4.3 マルチカーブ評価	336
3.4.4 シングルカーブ評価との比較	338
3.5 ケーススタディ 1：EONIA カーブと OIS 評価	340
3.5.1 評価日と EONIA 市場データ	340
3.5.2 EONIA カーブの構築	341
3.5.3 ベンチマーク値の確認	342
3.5.4 市場レートの再現性	343
3.5.5 5 年 OIS の構築	344
3.5.6 OIS の構築と計算結果の照合	344
3.5.7 市場固定金利とフェアレート	347
3.5.8 固定レグと翌日物レグ	347
3.6 ケーススタディ 2：市場レート変動と評価損益	348
3.6.1 市場レートシナリオ	349
3.6.2 シナリオ別 EONIA カーブ	350
3.6.3 同一 OIS の再評価	352
3.6.4 基準シナリオの照合	354
3.6.5 市場レート変動と評価損益	356
3.6.6 1 ペーシスポイント当たりの評価変化	356
3.7 まとめ	357
参考資料	358
第 II-4 章株価・為替と幾何ブラウン運動モデル	359
4.1 Black-Scholes-Merton と Black-76	359
4.1.1 Black-Scholes-Merton モデル	359
4.1.2 Black-76 モデル	360
4.1.3 Black-76 の入力条件	360
4.2 Black-76 による解析評価	362
4.2.1 Black 公式による価格と Greeks	362
4.2.2 Black 過程による European option 評価	363
4.3 Black-Scholes-Merton と European option	364
4.3.1 Black-Scholes-Merton モデルの入力条件	364
4.3.2 European put の解析解と二項 Tree	365
4.4 Monte Carlo 評価	367
4.4.1 擬似乱数 Monte Carlo 評価	367
4.4.2 Monte Carlo パスの可視化	369
4.4.3 Black 過程の 1 パス	371
4.5 American put の早期行使価値	373
4.6 準 Monte Carlo と Longstaff-Schwartz 法	375

4.6.1 Sobol 系列によるパス生成	375
4.6.2 Longstaff-Schwartz 法	377
4.7 まとめ	380
第 II-5 章金利モデル（期間構造なし）Normal モデル	382
5.1 Normal モデルの基礎	383
5.1.1 金利モデルと測度	383
5.1.2 Bachelier 価格公式	383
5.1.3 Normal volatility の単位	384
5.2 SOFR 先物オプション	385
5.2.1 先物価格から金利への変換	385
5.2.2 価格と Greeks	386
5.3 マイナス金利とボラティリティ分析	388
5.3.1 ゼロ金利とマイナス金利	388
5.3.2 インプライド Normal ボラティリティ	389
5.3.3 金利・ボラティリティ・時間シナリオ	390
5.4 Caplet と Cap	393
5.5 Swaption 評価	395
5.5.1 フォワードスワップレートとアニュイティ	395
5.5.2 Bachelier Swaption	397
5.6 Black volatility と Normal volatility	399
5.7 まとめ	400
第 II-6 章 SABR モデル	402
6.1 SABR モデルとパラメータ	402
6.1.1 SABR パラメータの役割	402
6.1.2 Alpha	403
6.1.3 Beta	403
6.1.4 Volatility of volatility	403
6.1.5 Rho	403
6.1.6 Hagan 近似とカリブレーション	403
6.2 Black SABR のカリブレーション	403
6.2.1 市場データ	403
6.2.2 パラメータ・カリブレーション	405
6.2.3 ボラティリティ・スマイル	408
6.3 SABR パラメータの感応度	410
6.4 Shifted SABR と負の金利	414
6.5 Shifted SABR Normal のカリブレーション	414
6.5.1 市場データ	414
6.5.2 パラメータ・カリブレーション	416
6.5.3 Normal ボラティリティ・スマイル	419
6.6 外挿とモデル比較	421
6.6.1 カリブレーション範囲外の Strike	421
6.6.2 Black SABR と Shifted SABR Normal	422

6.7 まとめ	423
第 II-7 章 金利期間構造モデル Early Termination 債評価	425
7.1 Hull-White モデルと債券条件	426
7.1.1 Hull-White 1-factor モデル	426
7.1.2 債券条件	426
7.1.3 Bullet Bond	428
7.1.4 初期カーブと Hull-White モデル	430
7.2 Callable Bond の構築と Tree 評価	431
7.2.1 コールスケジュール	431
7.2.2 Callable Bond の Tree 評価	432
7.3 Hull-White パラメータ感応度	433
7.4 Bermudan Receiver Swaption との比較	435
7.5 Vasicek モデルと Hull-White モデル	437
7.5.1 Vasicek モデルとの違い	437
7.5.2 Vasicek と Hull-White のゼロ金利曲線	438
7.6 カリブレーションと数値検証	441
7.6.1 Hull-White モデルのカリブレーション	441
7.6.2 1Y×5Y ATM Swap	442
7.6.3 One-helper calibration	444
7.6.4 European Swaption の Engine 比較	446
7.6.5 Tree step の収束	449
7.7 まとめ	451
第 II-8 章信用リスクと CDS	453
8.1 信用リスクと CDS の基礎	454
8.1.1 デフォルト時刻と生存確率	454
8.1.2 Recovery Rate と Loss Given Default	455
8.1.3 CDS Spread と Hazard Rate の近似	455
8.2 フラットカーブによる CDS 評価	455
8.2.1 フラットカーブによる 1 年物 CDS	455
8.2.2 Hazard Rate 近似と確率曲線	457
8.2.3 CDS Schedule とカーブ	459
8.2.4 MidPoint CDS Engine	461
8.2.5 CDS キャッシュフローと確率表	463
8.3 Premium Leg と Protection Leg	464
8.3.1 Premium Leg の手計算	464
8.3.2 Protection Leg の手計算	464
8.3.3 Risky PV01 と Fair Spread	465
8.4 Hazard Rate 推定と感応度	467
8.4.1 Zero-NPV CDS から Hazard Rate を推定する	467
8.4.2 Spread と Recovery Rate の感応度	469
8.5 標準 CDS2015 と ISDA カーブ	470
8.5.1 標準 CDS と CDS2015 満期	470

8.5.2 標準 CDS2015 Schedule	472
8.5.3 ISDA 用割引カーブ	473
8.5.4 ISDA 整合 Hazard Rate の推定	474
8.6 ISDA Engine と評価比較	476
8.6.1 ISDA CDS Engine	476
8.6.2 Accrual rebate と NPV 分解	477
8.6.3 Fair Spread と Fair Upfront	478
8.6.4 ISDA 用キャッシュフロー表	480
8.6.5 MidPoint 型近似と ISDA Engine	481
8.6.6 MidPoint Engine と ISDA Engine の比較	482
8.7 まとめ	484
第 II-9 章エキゾチック・デリバティブと PRDC	486
9.1 PRDC の商品構造と市場条件	487
9.1.1 商品条件	487
9.1.2 Coupon Schedule	489
9.1.3 Domestic、Foreign および Discount Curve	491
9.2 FX プロセスと Coupon Payoff	493
9.2.1 FX Process	493
9.2.2 Coupon Payoff	494
9.3 Monte Carlo による PRDC 評価	498
9.3.1 Coupon 日ベースの FX 時間グリッド	498
9.3.2 サンプル FX パス	499
9.3.3 PRDC の現在価値	502
9.3.4 期待 Coupon とキャッシュフロー	504
9.4 Coupon の期間構造と分布	505
9.4.1 期待 Coupon の期間構造	505
9.4.2 Coupon Cap / Floor 到達確率	507
9.4.3 Coupon 分布	507
9.5 FX 感応度と Monte Carlo 誤差	510
9.5.1 FX Spot 感応度	510
9.5.2 FX Volatility 感応度	512
9.5.3 Monte Carlo 標準誤差	514
9.6 モデルの限界と Part II 総括	516
9.6.1 PRDC 評価の限界	516
9.6.2 金利を決定論的とした	516
9.6.3 FX volatility を一定とした	516
9.6.4 相関を明示的に扱っていない	516
9.6.5 発行体コール権を含めていない	516
9.6.6 Part II のまとめ	516
9.7 まとめ	518

第 III 部 エンタープライズ金融計算 — 並列・分散処理から自動微分へ	520
第 III-1 章 クオント計算の進化 — 並列化から自動微分へ	521
金融工学とコンピュータの 60 年	521
2020 年代に変わった金融計算の前提	522
Hardware Acceleration だけでは残る問題	523
Automatic Differentiation が実用技術になった理由	524
第 III 部で扱う二つの計算技術	524
R における並列コンピューティング	525
小さな現在価値計算	525
逐次計算	526
Multi-core による並列計算	526
並列化による処理時間の短縮	529
一台の PC における並列化の限界	530
まとめ	530
第 III-2 章 非同期・並列計算 — future / furrr による Task 実行	531
分散計算の基本構造	531
Remote Function と Worker 実行	532
Job Queue と Scheduler	534
分散システムのデータ設計	535
まとめ	537
第 III-3 章 分散計算と Job / Task Control	538
PRDC 取引・市場・計算条件	538
Scenario Pricing Function	539
Scenario Grid	540
並列シナリオ計算	541
Risk Table	542
計算資源の拡張と残る問題	544
第 III-4 章 エンタープライズ・クオント計算基盤	545
金融リスク感応度と偏微分	545
有限差分と Bump & Revalue	546
自動微分は数値微分でも記号微分でもない	547
連鎖律	548
Computational Graph	549
Forward Mode Automatic Differentiation	550
Reverse Mode Automatic Differentiation	550
AAD と金融工学	551
R torch による最小の自動微分	552
二変数関数の Gradient	553
金融リスク計測への接続	554
まとめ	555
第 III-5 章 Option Greeks — Monte Carlo × Automatic Differentiation	557

計算条件	557
GaussQuant による基準価格と Greeks	558
Monte Carlo による Option Pricing	559
Common Random Numbers	560
微分可能な Monte Carlo Pricing	560
torch Tensor の設定	561
Monte Carlo NPV	562
Delta・Vega・Theta の自動微分	562
Gamma と Payoff の Kink	563
Smooth Payoff による Gamma	564
GaussQuant と Monte Carlo + torch の比較	567
Monte Carlo 標準誤差	568
Automatic Differentiation の意味	570
PRDC への接続	570
まとめ	571
第 III-6 章 金利リスク — DV01 から Curve Gradient へ	573
金利商品の現在価値	574
Fixed Leg	574
Floating Leg	575
Swap NPV	575
DV01	576
torch による Swap 評価	576
自動微分による DV01	579
有限差分との比較	580
フラット金利から Yield Curve へ	582
Curve Node を Tensor として持つ	583
GPS としての Point Sensitivity	585
Total DV01 と Bucketed Risk	586
Market Quote から Curve Risk へ	587
Multi-Curve への拡張	588
自動微分による金利リスク計測の意味	589
まとめ	590
第 III-7 章 PRDC — 多因子 Monte Carlo と Automatic Differentiation	592
PRDC と複数のリスクファクター	592
本章で用いる簡略 PRDC	593
FX の Risk-Neutral Dynamics	594
Monte Carlo Pricing	595
計算条件	596
torch による Monte Carlo Path 生成	597
微分可能な PRDC Pricing Function	598
PRDC NPV	601
一度の Reverse Mode で複数リスクを取得する	601

FX Delta	602
JPY Curve Risk	602
Foreign Curve Risk	604
FX Vega	605
PRDC Risk Summary	605
有限差分との検算	607
Floor・Cap と微分可能性	609
Monte Carlo と Automatic Differentiation	609
GPU への拡張	610
AAD として見る PRDC Risk	610
第 III 部を通したリスク計測の拡張	611
Option	611
Interest Rate	612
PRDC	612
まとめ	612

はじめに

“All models are wrong, but some are useful.”

「すべてのモデルは正しくない。しかし、その中には有用なものもある。」

— *George E. P. Box*

金融モデル構築からエンタープライズへ

2008 年のリーマン・ショックを契機として、パーゼル規制をはじめとする金融規制の枠組みは再整備され、金融機関には、より精緻な価格評価やリスク計測が求められるようになった。その過程で、金融工学の理論や数値計算手法は実務の中に広く定着し、現在では一定の成熟段階に達している。

一方、それらを支えるソフトウェア技術と計算環境は、この 20 年で劇的に進歩した。クラウドコンピューティング、オープンソースソフトウェア、並列処理および分散処理技術の発展により、かつて金融機関や研究機関のみが利用できたスーパーコンピュータやグリッドコンピューティング環境は、クラウドサービスや PC クラスタを利用することで、個人や小規模組織でも比較的 low コストに構築できるようになった。また、モンテカルロシミュレーションやリスク計測など、大量の計算資源を必要とする金融工学計算も、現実的な時間で実行できる時代となっている。

こうした時代の変化を背景として、

金融工学を学ぶ時代から、自ら金融工学システムを構築する時代へ。

という新しい潮流が生まれている。

なお、本書は三部構成となっている。

第 I 部では、均衡モデルを中心とした実証ファイナンスを扱い、財務データや市場データを用いた分析手法を学ぶ。第 II 部では、無裁定価格理論を中心として、債券、金利期間構造、デリバティブ、信用リスクなどの金融工学モデルを実装する。さらに第 III 部では、第 II 部で実装した無裁定価格モデルを基盤として、並列処理および分散処理環境へ展開し、大規模な金融工学計算を実装する方法を扱う。

また、その実装基盤として R および tidyverse を採用する。R は統計解析言語として広く利用されているが、本書では、優れたデータハンドリング機能、ベクトル演算、関数型プログラミングの特徴を備えた実装基盤として位置付けている。これらの特徴を活かすことで、金融データの取得・加工から価格評価、リスク計測、さらには並列・分散コンピューティング環境を利用した大規模金融工学計算までを、一貫した設計思想のもとで実装することができる。

本書の目的は、金融理論を理解することだけではなく、それらをソフトウェアとして実装し、現代の金融工学システムを構築するための基礎技術を体系的に習得することにある。

第Ⅰ部 金融工学 均衡モデル

第 I-1 章 R によるデータ分析の基礎

本章では、R によるデータ分析に必要な基本操作を、Base R から tidyverse へ段階的に学ぶ。前半では、オブジェクト、ベクトル、行列、リスト、データフレーム、関数、条件判定、記述統計および確率分布を扱う。後半では、dplyr、stringr、tidyr、purrr、ggplot2 を用いて、データの読み込み、整形、集計、反復処理、可視化を一つの流れとして組み立てる。さらに、nest() によるリスト列、future と furrr による並列計算、dbplyr と Arrow によるデータベース・大規模データ処理へ進む。

Base R の項目構成は、Posit が公開している [Base R Cheatsheet](#) を参考にしている。本章のコードでは、反復処理に明示的なループを使わず、ベクトル化または purrr::map() 系関数を用いる。また、パイプ演算子はすべて|>に統一する。

1.1 R の基礎

R の基本操作

オブジェクトと代入

R では、計算結果やデータをオブジェクトとして保存する。代入には<-を用いる。

```
risk_free_rate <- 0.01
future_cash_flow <- 100

present_value <- future_cash_flow / (1 + risk_free_rate)

present_value
#> [1] 99.0099
```

オブジェクト名には、内容が分かる英単語を用いる。本書では複数の単語をアンダースコアで結ぶ snake_case を基本とする。

四則演算と代表的な数学関数

```
2 + 3
#> [1] 5

10 - 4
#> [1] 6

6 * 7
#> [1] 42
```

```
100 / 4
#> [1] 25

2^5
#> [1] 32

sqrt(16)
#> [1] 4

log(exp(1))
#> [1] 1

abs(-3)
#> [1] 3

round(pi, digits = 3)
#> [1] 3.142

signif(pi, digits = 4)
#> [1] 3.142
```

オブジェクトの型と構造

```
numeric_value <- 10.5
integer_value <- 10L
character_value <- "Finance"
logical_value <- TRUE

class(numeric_value)
#> [1] "numeric"

class(integer_value)
#> [1] "integer"

class(character_value)
#> [1] "character"

class(logical_value)
#> [1] "logical"

typeof(numeric_value)
#> [1] "double"

str(numeric_value)
#> num 10.5
```

欠損値

欠損値は NA で表す。欠損値の有無は `is.na()` で確認する。

```
returns_with_na <- c(0.02, NA, -0.01, 0.03)

is.na(returns_with_na)
```



```
#> [1] FALSE TRUE FALSE FALSE
sum(is.na(returns_with_na))
#> [1] 1

mean(returns_with_na, na.rm = TRUE)
#> [1] 0.01333333
```

ベクトル

ベクトルの作成

```
interest_rates <- c(0.01, 0.015, 0.02)
year_sequence <- 1:5
fine_grid <- seq(from = 0, to = 1, by = 0.25)
repeated_values <- rep(100, times = 5)

interest_rates
#> [1] 0.010 0.015 0.020
year_sequence
#> [1] 1 2 3 4 5
fine_grid
#> [1] 0.00 0.25 0.50 0.75 1.00
repeated_values
#> [1] 100 100 100 100 100
```

ベクトルの要素

R の添字は 1 から始まる。

```
interest_rates[1]
#> [1] 0.01
interest_rates[2:3]
#> [1] 0.015 0.020
interest_rates[-1]
#> [1] 0.015 0.020

interest_rates[interest_rates >= 0.015]
#> [1] 0.015 0.020
```

名前付きベクトル

```
portfolio_weights <- c(stock = 0.6, bond = 0.3, cash = 0.1)

portfolio_weights
#> stock bond cash
```

```
#> 0.6 0.3 0.1
portfolio_weights["bond"]
#> bond
#> 0.3
sum(portfolio_weights)
#> [1] 1
```

ベクトル化

R では、多くの計算をベクトル全体に一度に適用できる。

```
discount_rates <- seq(0.01, 0.05, by = 0.01)
cash_flow <- 100

present_values <- cash_flow / (1 + discount_rates)

tibble(discount_rates, present_values)
#> # A tibble: 5 x 2
#>   discount_rates present_values
#>   <dbl>          <dbl>
#> 1      0.01      99.010
#> 2      0.02      98.039
#> 3      0.03      97.087
#> 4      0.04      96.154
#> 5      0.05      95.238
```

行列、リスト、データフレーム

行列

```
return_matrix <- matrix(
  c(0.01, 0.02, -0.01, 0.03, 0.00, 0.01),
  nrow = 3,
  ncol = 2,
  byrow = TRUE
)

colnames(return_matrix) <- c("stock_a", "stock_b")
rownames(return_matrix) <- c("month_1", "month_2", "month_3")

return_matrix
#>      stock_a stock_b
#> month_1    0.01    0.02
#> month_2   -0.01    0.03
#> month_3    0.00    0.01
```

```
return_matrix[, "stock_a"]
#> month_1 month_2 month_3
#>    0.01   -0.01    0.00
colMeans(return_matrix)
#> stock_a stock_b
#>    0.00    0.02
```

リスト

リストには、型や長さの異なるオブジェクトをまとめて保存できる。

```
analysis_result <- list(
  model_name = "simple return model",
  coefficients = c(alpha = 0.001, beta = 1.05),
  observations = 60L,
  converged = TRUE
)

analysis_result$model_name
#> [1] "simple return model"
analysis_result$coefficients
#> alpha  beta
#> 0.001 1.050
analysis_result[["observations"]]
#> [1] 60
```

データフレーム

```
firm_data_base <- data.frame(
  firm_id = 1:3,
  firm_name = c("Firm A", "Firm B", "Firm C"),
  roe = c(0.08, 0.12, 0.15)
)

firm_data_base
#>   firm_id firm_name  roe
#> 1       1   Firm A 0.08
#> 2       2   Firm B 0.12
#> 3       3   Firm C 0.15
str(firm_data_base)
#> 'data.frame':    3 obs. of  3 variables:
#>  $ firm_id  : int  1 2 3
#>  $ firm_name: chr   "Firm A" "Firm B" "Firm C"
#>  $ roe      : num  0.08 0.12 0.15
```

tidyverse では、データフレームを拡張した `tibble` を用いる。

```
firm_data <- tibble(
  firm_id = 1:3,
  firm_name = c("Firm A", "Firm B", "Firm C"),
  roe = c(0.08, 0.12, 0.15)
)
```

```
firm_data
#> # A tibble: 3 x 3
#>   firm_id firm_name   roe
#>   <int> <chr>     <dbl>
#> 1     1 Firm A      0.08
#> 2     2 Firm B      0.12
#> 3     3 Firm C      0.15
```

少量のデータを行単位で記述する場合は `tribble()` が便利である。

```
project_data <- tribble(
  ~project, ~initial_cost, ~cash_flow,
  "A",      100,      120,
  "B",      150,      175,
  "C",      200,      215
)
```

```
project_data
#> # A tibble: 3 x 3
#>   project initial_cost cash_flow
#>   <chr>         <dbl>     <dbl>
#> 1 A             100       120
#> 2 B             150       175
#> 3 C             200       215
```

条件判定と独自関数

条件判定

```
npv <- 5.2

investment_decision <- if (npv >= 0) {
  "投資を実行する"
} else {
  "投資を見送る"
}
```

```
investment_decision
```

```
#> [1] "投資を実行する"
```

ベクトルに対する複数の条件分岐には `case_when()` を用いる。

```
credit_scores <- tibble(score = c(85, 72, 58, 91))

credit_scores |>
  mutate(
    rating = case_when(
      score >= 80 ~ "A",
      score >= 65 ~ "B",
      TRUE ~ "C"
    )
  )
#> # A tibble: 4 x 2
#>   score rating
#>   <dbl> <chr>
#> 1     85 A
#> 2     72 B
#> 3     58 C
#> 4     91 A
```

独自関数

```
calculate_present_value <- function(cash_flow, discount_rate, years = 1) {
  cash_flow / (1 + discount_rate)^years
}

calculate_present_value(cash_flow = 100, discount_rate = 0.02, years = 3)
#> [1] 94.23223
```

関数はベクトルをそのまま受け取れるように設計すると再利用しやすい。

```
calculate_present_value(
  cash_flow = c(100, 100, 100),
  discount_rate = 0.02,
  years = 1:3
)
#> [1] 98.03922 96.11688 94.23223
```

記述統計

代表値と散らばり

```
sample_returns <- c(0.01, 0.03, -0.02, 0.00, 0.04, 0.02)
```

```
mean(sample_returns)
#> [1] 0.01333333
median(sample_returns)
#> [1] 0.015
min(sample_returns)
#> [1] -0.02
max(sample_returns)
#> [1] 0.04
range(sample_returns)
#> [1] -0.02 0.04
var(sample_returns)
#> [1] 0.0004666667
sd(sample_returns)
#> [1] 0.02160247
quantile(sample_returns)
#>      0%      25%      50%      75%     100%
#> -0.0200  0.0025  0.0150  0.0275  0.0400
```

共分散と相関

```
stock_a <- c(0.01, 0.03, -0.02, 0.02, 0.04)
stock_b <- c(0.00, 0.02, -0.01, 0.01, 0.03)

cov(stock_a, stock_b)
#> [1] 0.00035
cor(stock_a, stock_b)
#> [1] 0.9615239
```

順位

```
firm_returns <- c(0.08, 0.12, -0.03, 0.05)

rank(firm_returns)
#> [1] 3 4 1 2
rank(-firm_returns)
#> [1] 2 1 4 3
order(firm_returns, decreasing = TRUE)
#> [1] 2 1 4 3
```

確率分布

R の確率分布関数には共通した命名規則がある。

接頭辞	内容
r	乱数を生成する
d	確率密度または確率質量を求める
p	累積確率を求める
q	累積確率に対応する分位点を求める

一様分布

```
set_random_seed_LFL(1234)

uniform_random <- runif(n = 10, min = 0, max = 1)

uniform_random
#> [1] 0.113703411 0.622299405 0.609274733 0.623379442 0.860915384 0.640310605
#> [7] 0.009495756 0.232550506 0.666083758 0.514251141
dunif(0.5, min = 0, max = 1)
#> [1] 1
punif(0.5, min = 0, max = 1)
#> [1] 0.5
qunif(0.95, min = 0, max = 1)
#> [1] 0.95
```

正規分布

```
set_random_seed_LFL(1234)

normal_random <- rnorm(n = 10, mean = 0, sd = 1)

normal_random
#> [1] -1.2070657 0.2774292 1.0844412 -2.3456977 0.4291247 0.5060559
#> [7] -0.5747400 -0.5466319 -0.5644520 -0.8900378
dnorm(0, mean = 0, sd = 1)
#> [1] 0.3989423
pnorm(1.96, mean = 0, sd = 1)
#> [1] 0.9750021
qnorm(0.975, mean = 0, sd = 1)
#> [1] 1.959964
```

`dnorm()` は正規分布の密度、`pnorm()` は指定した値以下となる確率、`qnorm()` は指定した累積確率に対応する分位点を返す。

```
probability_below <- pnorm(-0.02, mean = 0.01, sd = 0.03)
value_at_risk_95 <- qnorm(0.05, mean = 0.01, sd = 0.03)
```

```
probability_below
#> [1] 0.1586553
value_at_risk_95
#> [1] -0.03934561
```

二項分布

```
set_random_seed_LFL(1234)

binomial_random <- rbinom(n = 10, size = 20, prob = 0.6)

binomial_random
#> [1] 15 11 11 11 10 11 17 14 11 12
dbinom(12, size = 20, prob = 0.6)
#> [1] 0.1797058
pbinom(12, size = 20, prob = 0.6)
#> [1] 0.5841071
qbinom(0.95, size = 20, prob = 0.6)
#> [1] 16
```

ポアソン分布

```
set_random_seed_LFL(1234)

poisson_random <- rpois(n = 10, lambda = 3)

poisson_random
#> [1] 1 3 3 3 5 3 0 2 4 3
dpois(2, lambda = 3)
#> [1] 0.2240418
ppois(2, lambda = 3)
#> [1] 0.4231901
qpois(0.95, lambda = 3)
#> [1] 6
```

分布関数の比較

```
distribution_functions <- tribble(
  ~distribution, ~random_function, ~density_function, ~cumulative_function, ~quantile_function,
  "一様分布",    "runif()",      "dunif()",      "punif()",      "qunif()",
  "正規分布",    "rnorm()",      "dnorm()",      "pnorm()",      "qnorm()",
  "二項分布",    "rbinom()",     "dbinom()",     "pbinom()",     "qbinom()",
  "ポアソン分布", "rpois()",      "dpois()",      "ppois()",      "qpois()"
```



```
)

distribution_functions
#> # A tibble: 4 x 5
#>   distribution random_function density_function cumulative_function
#>   <chr>         <chr>         <chr>         <chr>
#> 1 一様分布      runif()         dunif()        punif()
#> 2 正規分布      rnorm()         dnorm()        pnorm()
#> 3 二項分布      rbinom()        dbinom()        pbinom()
#> 4 ポアソン分布 rpois()         dpois()        ppois()
#> # i 1 more variable: quantile_function <chr>
```

1.2 tidyverse によるデータ整形

プロジェクト内のデータを読み込む

本書で使用するデータは、プロジェクトルートの `data` ディレクトリに置き、外部入力に近いデータを `raw`、分析によって作成したデータを `derived` に分ける。プロジェクトルートの探索やファイル検索は `R/utility_LFL.R` へ集約し、各章では分析対象となる相対パスだけを指定する。

```
tbl_daily_stock_return <-
  read_parquet_file_LFL(
    file.path("raw", "market", "daily_stock_returns.parquet")
  )

vec_required_columns <- c("date", "firm1", "firm2")
vec_missing_columns <- setdiff(vec_required_columns, names(tbl_daily_stock_return))

if (length(vec_missing_columns) > 0) {
  stop("必要な列が不足しています: ", paste(vec_missing_columns, collapse = ", "))
}

show_table_LFL(tbl_daily_stock_return)
#> # A tibble: 21 x 3
#>   date          firm1      firm2
#>   * <date>      <dbl>      <dbl>
#> 1 2020-04-01 -0.039482  0.076963
#> 2 2020-04-02  0.0059787 -0.0072488
#> 3 2020-04-03  0.055787  -0.017304
#> 4 2020-04-06  0.041935  0.0021690
#> 5 2020-04-07 -0.020193  0.075545
#> 6 2020-04-08 -0.0022947 -0.096212
#> 7 2020-04-09  0.081707  0.078843
#> 8 2020-04-10  0.034561  0.021536
```

```
#> 9 2020-04-13 0.032601 0.13520
#> 10 2020-04-14 -0.037663 0.037979
#> # i 11 more rows
```

データの構造を確認する

```
show_data_structure_LFL(tbl_daily_stock_return)
#> Rows: 21
#> Columns: 3
#> $ date <date> 2020-04-01, 2020-04-02, 2020-04-03, 2020-04-06, 2020-04-07, 202~
#> $ firm1 <dbl> -0.039482142, 0.005978689, 0.055786605, 0.041934669, -0.02019279~
#> $ firm2 <dbl> 0.0769625974, -0.0072488484, -0.0173040800, 0.0021689573, 0.0755~
summary(tbl_daily_stock_return)
#>      date      firm1      firm2
#> Min.   :2020-04-01   Min.   : -0.072071   Min.   : -0.0962117
#> 1st Qu.:2020-04-08   1st Qu.: -0.027029   1st Qu.: -0.0196147
#> Median :2020-04-15   Median : 0.005979   Median : -0.0003056
#> Mean   :2020-04-15   Mean    : 0.014161   Mean    : 0.0080253
#> 3rd Qu.:2020-04-22   3rd Qu.: 0.044812   3rd Qu.: 0.0379793
#> Max.   :2020-04-30   Max.    : 0.171297   Max.    : 0.1352008
nrow(tbl_daily_stock_return)
#> [1] 21
ncol(tbl_daily_stock_return)
#> [1] 3
names(tbl_daily_stock_return)
#> [1] "date" "firm1" "firm2"
```

dplyr によるデータ操作

列を選ぶ

```
tbl_daily_stock_return |>
  select(date, firm1)
#> # A tibble: 21 x 2
#>   date      firm1
#>   <date>      <dbl>
#> 1 2020-04-01 -0.039482
#> 2 2020-04-02 0.0059787
#> 3 2020-04-03 0.055787
#> 4 2020-04-06 0.041935
#> 5 2020-04-07 -0.020193
#> 6 2020-04-08 -0.0022947
#> 7 2020-04-09 0.081707
#> 8 2020-04-10 0.034561
```

```
#> 9 2020-04-13 0.032601
#> 10 2020-04-14 -0.037663
#> # i 11 more rows
```

行を抽出する

```
tbl_daily_stock_return |>
  filter(firm1 > 0)
#> # A tibble: 12 x 3
#>   date          firm1    firm2
#>   <date>        <dbl>    <dbl>
#> 1 2020-04-02 0.0059787 -0.0072488
#> 2 2020-04-03 0.055787  -0.017304
#> 3 2020-04-06 0.041935   0.0021690
#> 4 2020-04-09 0.081707   0.078843
#> 5 2020-04-10 0.034561   0.021536
#> 6 2020-04-13 0.032601   0.13520
#> 7 2020-04-15 0.0098798 0.011427
#> 8 2020-04-17 0.17130    0.039463
#> 9 2020-04-20 0.046325   0.037274
#> 10 2020-04-22 0.051543  -0.053818
#> 11 2020-04-27 0.0045375 -0.019615
#> 12 2020-04-30 0.044812  -0.013410
```

複数の条件を同時に指定できる。

```
tbl_daily_stock_return |>
  filter(firm1 > 0, firm2 > 0)
#> # A tibble: 7 x 3
#>   date          firm1    firm2
#>   <date>        <dbl>    <dbl>
#> 1 2020-04-06 0.041935   0.0021690
#> 2 2020-04-09 0.081707   0.078843
#> 3 2020-04-10 0.034561   0.021536
#> 4 2020-04-13 0.032601   0.13520
#> 5 2020-04-15 0.0098798 0.011427
#> 6 2020-04-17 0.17130    0.039463
#> 7 2020-04-20 0.046325   0.037274
```

列を追加する

```
return_data <- tbl_daily_stock_return |>
  mutate(
    spread = firm1 - firm2,
```

```

    firm1_gross_return = 1 + firm1,
    firm2_gross_return = 1 + firm2,
    market_direction = case_when(
      firm1 > 0 & firm2 > 0 ~ "both_positive",
      firm1 < 0 & firm2 < 0 ~ "both_negative",
      TRUE ~ "mixed"
    )
  )
)

return_data
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>          <dbl>
#> 1 2020-04-01 -0.039482  0.076963 -0.11644          0.96052
#> 2 2020-04-02  0.0059787 -0.0072488  0.013228          1.0060
#> 3 2020-04-03  0.055787  -0.017304  0.073091          1.0558
#> 4 2020-04-06  0.041935  0.0021690  0.039766          1.0419
#> 5 2020-04-07 -0.020193  0.075545 -0.095738          0.97981
#> 6 2020-04-08 -0.0022947 -0.096212  0.093917          0.99771
#> 7 2020-04-09  0.081707  0.078843  0.0028641          1.0817
#> 8 2020-04-10  0.034561  0.021536  0.013025          1.0346
#> 9 2020-04-13  0.032601  0.13520  -0.10260          1.0326
#> 10 2020-04-14 -0.037663  0.037979 -0.075642          0.96234
#> # i 11 more rows
#> # i 2 more variables: firm2_gross_return <dbl>, market_direction <chr>

```

並べ替える

```

return_data |>
  arrange(firm1)
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>          <dbl>
#> 1 2020-04-28 -0.072071  -0.046873  -0.025197          0.92793
#> 2 2020-04-24 -0.041378  -0.0041105  -0.037268          0.95862
#> 3 2020-04-01 -0.039482  0.076963  -0.11644          0.96052
#> 4 2020-04-14 -0.037663  0.037979  -0.075642          0.96234
#> 5 2020-04-23 -0.028875  -0.035484  0.0066092          0.97112
#> 6 2020-04-16 -0.027029  -0.053488  0.026460          0.97297
#> 7 2020-04-07 -0.020193  0.075545  -0.095738          0.97981
#> 8 2020-04-21 -0.014593  -0.00030564 -0.014288          0.98541
#> 9 2020-04-08 -0.0022947 -0.096212  0.093917          0.99771
#> 10 2020-04-27  0.0045375  -0.019615  0.024152          1.0045

```

```
#> # i 11 more rows
#> # i 2 more variables: firm2_gross_return <dbl>, market_direction <chr>

return_data |>
  arrange(desc(firm1))
#> # A tibble: 21 x 7
#>   date          firm1      firm2      spread firm1_gross_return
#>   <date>         <dbl>      <dbl>      <dbl>          <dbl>
#> 1 2020-04-17 0.17130    0.039463 0.13183        1.1713
#> 2 2020-04-09 0.081707   0.078843 0.0028641      1.0817
#> 3 2020-04-03 0.055787  -0.017304 0.073091      1.0558
#> 4 2020-04-22 0.051543  -0.053818 0.10536       1.0515
#> 5 2020-04-20 0.046325   0.037274 0.0090516     1.0463
#> 6 2020-04-30 0.044812  -0.013410 0.058222     1.0448
#> 7 2020-04-06 0.041935   0.0021690 0.039766     1.0419
#> 8 2020-04-10 0.034561   0.021536 0.013025     1.0346
#> 9 2020-04-13 0.032601   0.13520  -0.10260     1.0326
#> 10 2020-04-15 0.0098798 0.011427  -0.0015473    1.0099
#> # i 11 more rows
#> # i 2 more variables: firm2_gross_return <dbl>, market_direction <chr>
```

要約する

```
return_data |>
  summarise(
    observations = n(),
    mean_firm1 = mean(firm1),
    sd_firm1 = sd(firm1),
    mean_firm2 = mean(firm2),
    sd_firm2 = sd(firm2),
    correlation = cor(firm1, firm2)
  )
#> # A tibble: 1 x 6
#>   observations mean_firm1 sd_firm1 mean_firm2 sd_firm2 correlation
#>   <int>         <dbl>    <dbl>      <dbl>    <dbl>          <dbl>
#> 1      21      0.014161 0.053840 0.0080253 0.054390      0.23349
```

グループごとに集計する

```
return_data |>
  group_by(market_direction) |>
  summarise(
    observations = n(),
```

```

    mean_firm1 = mean(firm1),
    mean_firm2 = mean(firm2),
    .groups = "drop"
  ) |>
  arrange(desc(observations))
#> # A tibble: 3 x 4
#>   market_direction observations mean_firm1 mean_firm2
#>   <chr>                <int>      <dbl>      <dbl>
#> 1 mixed                  8  0.0081649  0.0098864
#> 2 both_positive          7  0.059758   0.046559
#> 3 both_negative          6 -0.031040 -0.039412

```

件数を数える

```

return_data |>
  count(market_direction, sort = TRUE)
#> # A tibble: 3 x 2
#>   market_direction      n
#>   <chr>              <int>
#> 1 mixed                8
#> 2 both_positive        7
#> 3 both_negative        6

```

データを結合する

```

firm_master <- tribble(
  ~firm, ~firm_name, ~industry,
  "firm1", "Firm A", "Machinery",
  "firm2", "Firm B", "Chemicals"
)

return_summary <- return_data |>
  summarise(
    firm1 = mean(firm1),
    firm2 = mean(firm2)
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = "firm",
    values_to = "mean_return"
  )

return_summary |>

```

```

left_join(firm_master, by = "firm")
#> # A tibble: 2 x 4
#>   firm mean_return firm_name industry
#>   <chr>      <dbl> <chr>      <chr>
#> 1 firm1    0.014161 Firm A      Machinery
#> 2 firm2    0.0080253 Firm B      Chemicals

```

stringr による文字列処理

文字列処理では、関数名が `str_` で始まる `stringr` を使用する。

```

company_names <- tibble(
  raw_name = c(
    " Alpha Holdings Co., Ltd. ",
    "Beta Financial Group",
    "Gamma Industries"
  )
)

company_names <- company_names |>
  mutate(
    clean_name = str_squish(raw_name),
    lower_name = str_to_lower(clean_name),
    name_length = str_length(clean_name),
    contains_group = str_detect(clean_name, regex("group", ignore_case = TRUE)),
    first_word = str_extract(clean_name, "[A-Za-z]+")
  )

company_names
#> # A tibble: 3 x 6
#>   raw_name          clean_name lower_name name_length contains_group first_word
#>   <chr>            <chr>      <chr>          <int> <lgl>          <chr>
#> 1 " Alpha Holdings~ Alpha Hol~ alpha hol~      24 FALSE        Alpha
#> 2 "Beta Financial G~ Beta Fina~ beta fina~      20 TRUE         Beta
#> 3 "Gamma Industries" Gamma Ind~ gamma ind~      16 FALSE        Gamma

```

置換と結合

```

company_names |>
  mutate(
    short_name = str_remove(clean_name, regex(" co\\. , ltd\\. $", ignore_case = TRUE)),
    display_name = str_c(first_word, " / ", name_length, " characters")
  ) |>
  select(clean_name, short_name, display_name)

```

```
#> # A tibble: 3 x 3
#>   clean_name      short_name      display_name
#>   <chr>          <chr>          <chr>
#> 1 Alpha Holdings Co., Ltd. Alpha Holdings      Alpha / 24 characters
#> 2 Beta Financial Group   Beta Financial Group Beta / 20 characters
#> 3 Gamma Industries      Gamma Industries      Gamma / 16 characters
```

tidyr によるデータ整形

wide 形式から long 形式へ変換する

元のデータでは、`firm1` と `firm2` が別々の列に格納されている。`pivot_longer()` を使うと、企業名とリターンをそれぞれ一つの列へまとめられる。

```
return_long <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  )
```

```
return_long
#> # A tibble: 42 x 3
#>   date      firm      return
#>   <date>   <chr>    <dbl>
#> 1 2020-04-01 firm1 -0.039482
#> 2 2020-04-01 firm2  0.076963
#> 3 2020-04-02 firm1  0.0059787
#> 4 2020-04-02 firm2 -0.0072488
#> 5 2020-04-03 firm1  0.055787
#> 6 2020-04-03 firm2 -0.017304
#> 7 2020-04-06 firm1  0.041935
#> 8 2020-04-06 firm2  0.0021690
#> 9 2020-04-07 firm1 -0.020193
#> 10 2020-04-07 firm2  0.075545
#> # i 32 more rows
```

long 形式から wide 形式へ戻す

```
return_long |>
  pivot_wider(
    names_from = firm,
    values_from = return
  )
#> # A tibble: 21 x 3
```



```
#>   date      firm1    firm2
#>   <date>    <dbl>    <dbl>
#> 1 2020-04-01 -0.039482  0.076963
#> 2 2020-04-02  0.0059787 -0.0072488
#> 3 2020-04-03  0.055787  -0.017304
#> 4 2020-04-06  0.041935  0.0021690
#> 5 2020-04-07 -0.020193  0.075545
#> 6 2020-04-08 -0.0022947 -0.096212
#> 7 2020-04-09  0.081707  0.078843
#> 8 2020-04-10  0.034561  0.021536
#> 9 2020-04-13  0.032601  0.13520
#> 10 2020-04-14 -0.037663  0.037979
#> # i 11 more rows
```

文字列を複数列へ分ける

```
security_codes <- tibble(
  security_id = c("JP-1001", "JP-1002", "US-2001")
)

security_codes |>
  separate_wider_delim(
    security_id,
    delim = "-",
    names = c("country", "code")
  )

#> # A tibble: 3 x 2
#>   country code
#>   <chr>   <chr>
#> 1 JP     1001
#> 2 JP     1002
#> 3 US     2001
```

欠損値を処理する

```
missing_data <- tibble(
  firm = c("A", "B", "C"),
  return = c(0.02, NA, -0.01)
)

missing_data |>
  replace_na(list(return = 0))

#> # A tibble: 3 x 2
```

```
#>   firm  return
#>   <chr> <dbl>
#> 1 A      0.02
#> 2 B      0
#> 3 C     -0.01

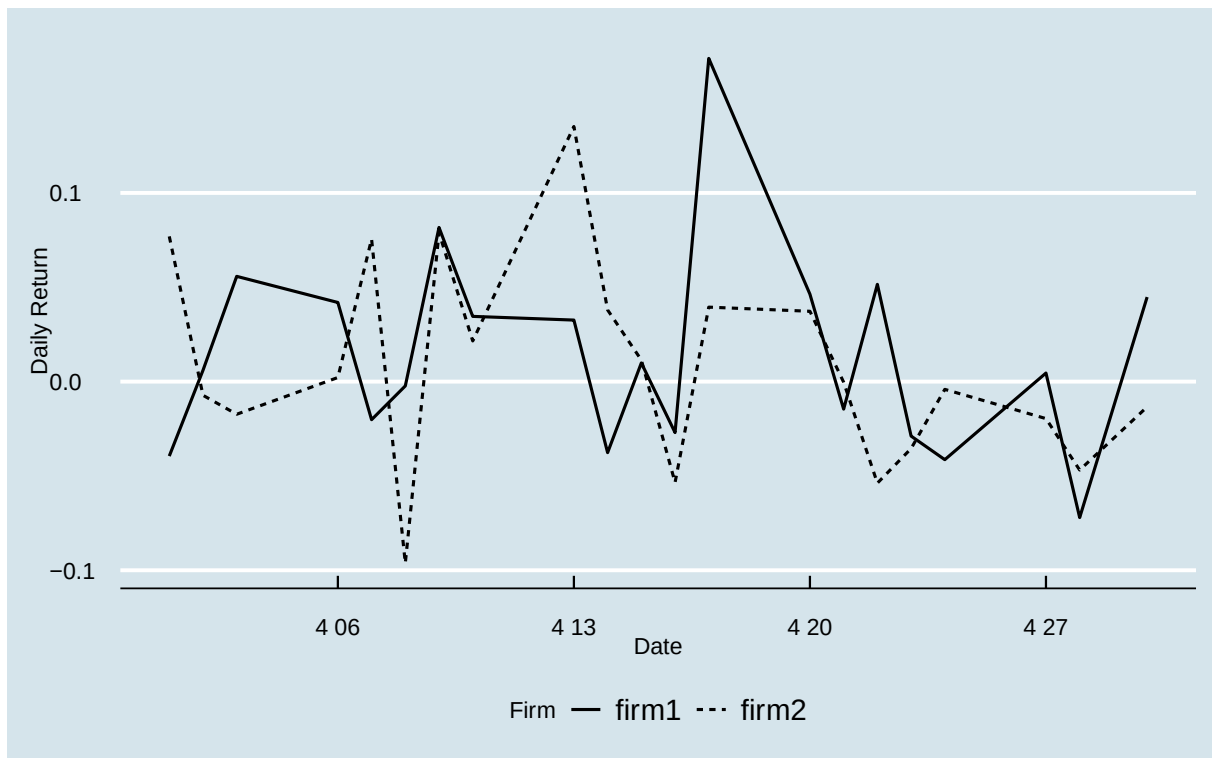
missing_data |>
  drop_na()
#> # A tibble: 2 x 2
#>   firm  return
#>   <chr> <dbl>
#> 1 A      0.02
#> 2 C     -0.01
```

ggplot2 による可視化

ggplot2 では、データを `ggplot()` へ渡し、`aes()` で変数と視覚要素の対応を指定し、`geom_*()` で図形を追加する。

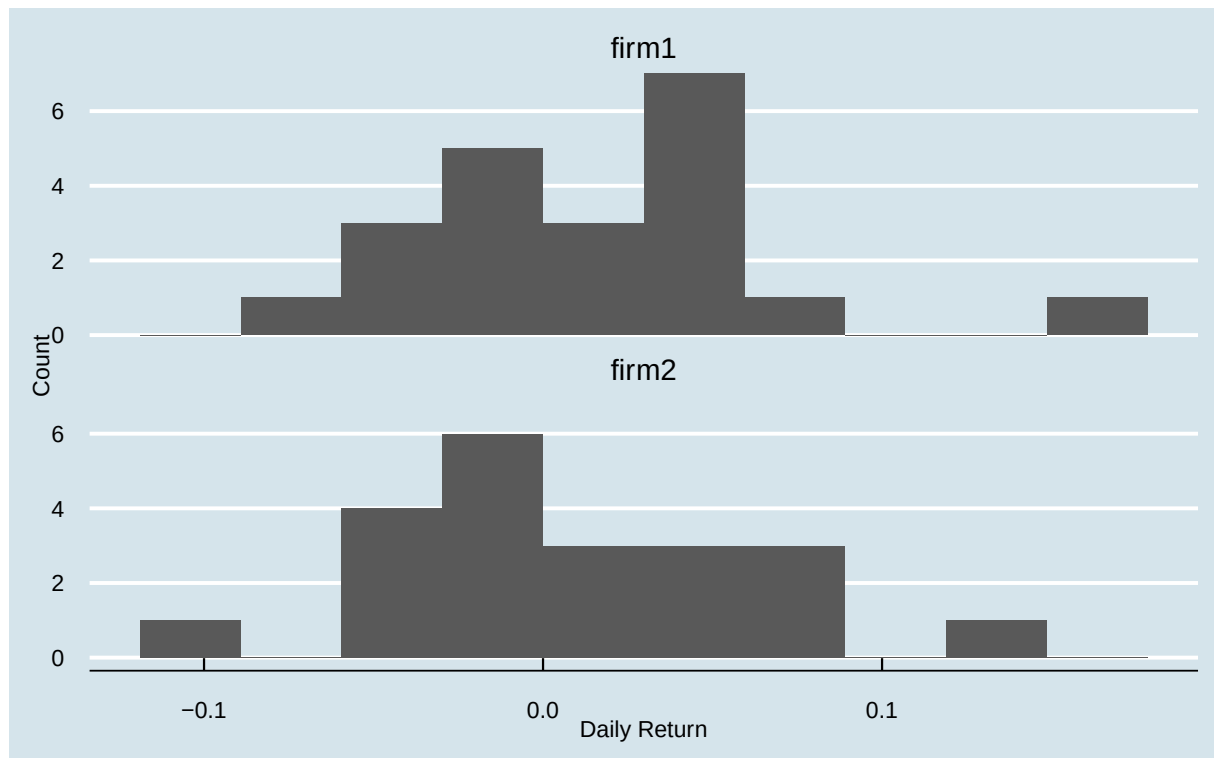
折れ線グラフ

```
(return_long |>
  ggplot(aes(x = date, y = return, linetype = firm)) +
  geom_line(linewidth = 0.7) +
  labs(
    x = "Date",
    y = "Daily Return",
    linetype = "Firm"
  )) |> add_lagrange_theme_LFL()
```



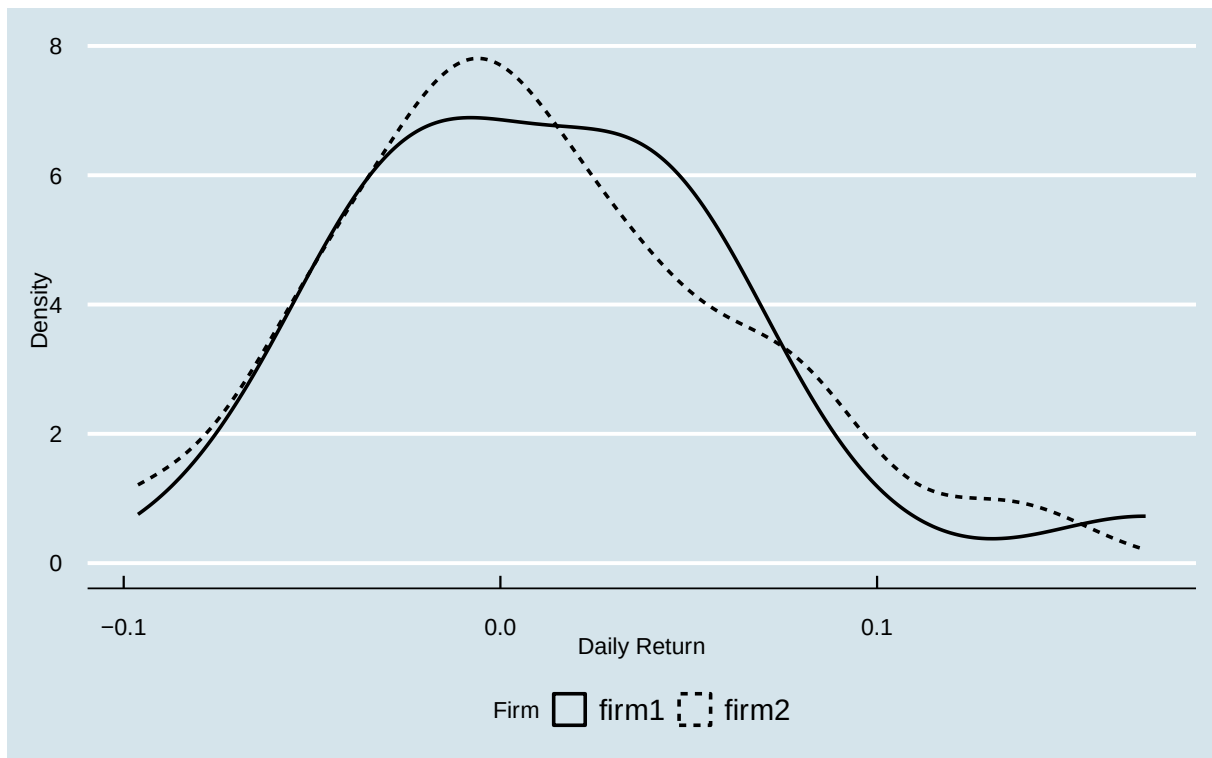
ヒストグラム

```
(return_long |>
  ggplot(aes(x = return)) +
  geom_histogram(bins = 10, boundary = 0) +
  facet_wrap(vars(firm), ncol = 1) +
  labs(
    x = "Daily Return",
    y = "Count"
  ) |>
  add_lagrange_theme_LFL())
```



密度関数

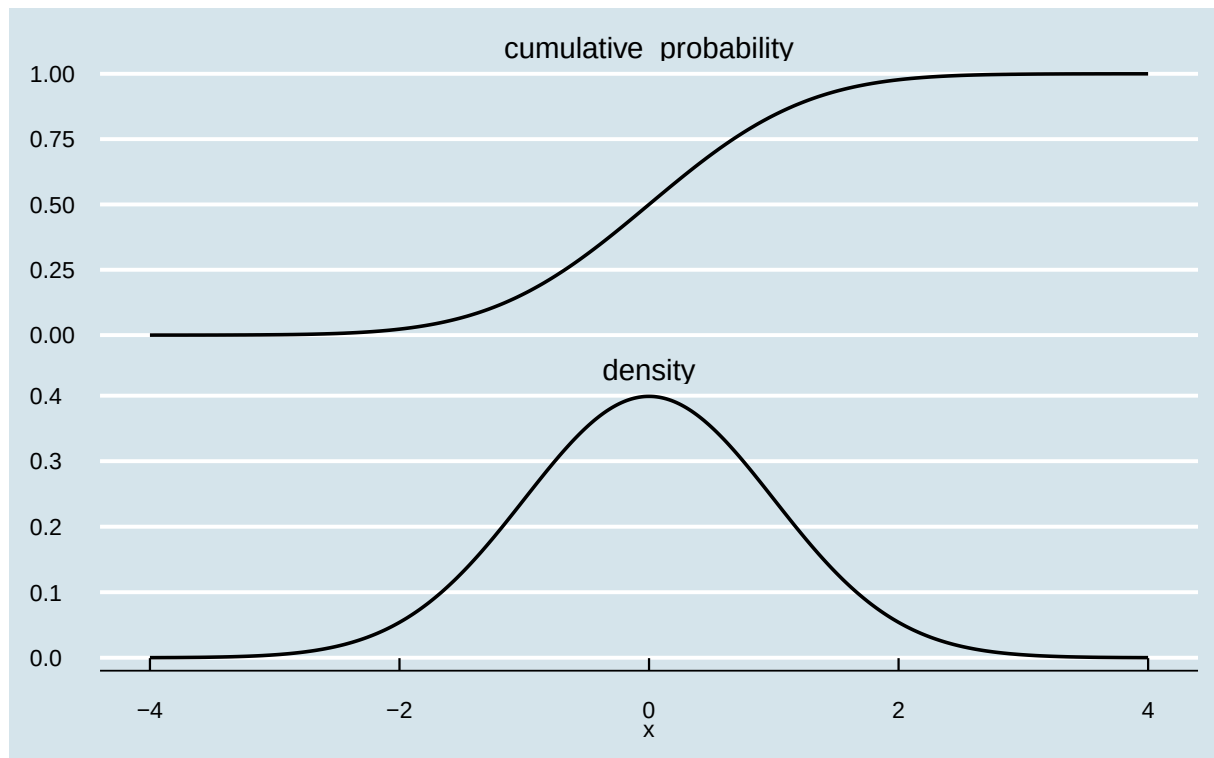
```
(return_long |>
  ggplot(aes(x = return, linetype = firm)) +
  geom_density(linewidth = 0.8) +
  labs(
    x = "Daily Return",
    y = "Density",
    linetype = "Firm"
  ) |>
  add_lagrange_theme_LFL())
```



正規分布の密度と累積分布

```
normal_curve <- tibble(
  x = seq(-4, 4, length.out = 401),
  density = dnorm(x),
  cumulative_probability = pnorm(x)
)

(normal_curve |>
  pivot_longer(
    cols = c(density, cumulative_probability),
    names_to = "function_type",
    values_to = "value"
  ) |>
  ggplot(aes(x = x, y = value)) +
  geom_line(linewidth = 0.8) +
  facet_wrap(vars(function_type), scales = "free_y", ncol = 1) +
  labs(
    x = "x",
    y = NULL
  ) |>
  add_lagrange_theme_LFL())
```

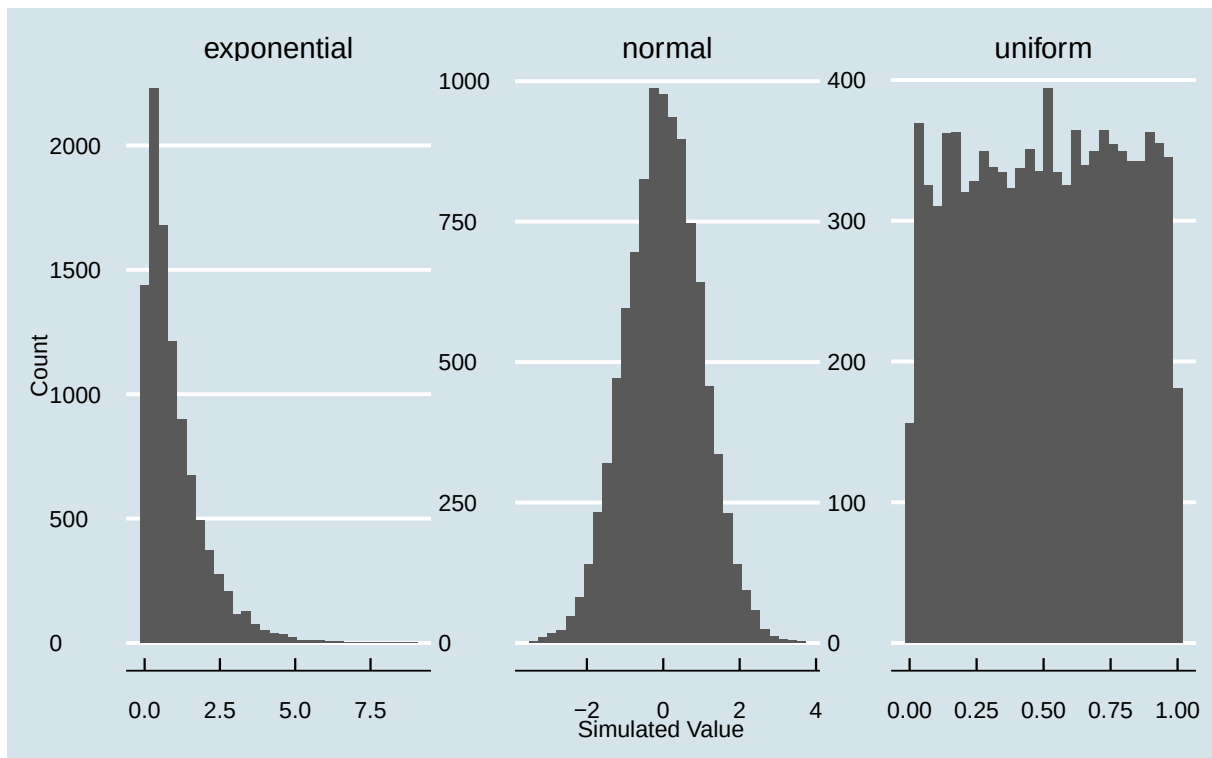


シミュレーション結果を比較する

```
simulated_distributions <- tibble(
  distribution = c("normal", "uniform", "exponential"),
  values = list(rnorm(10000), runif(10000), rexp(10000))
) |>
  tidyr::unnest(values)

p <- simulated_distributions |>
  ggplot(aes(x = values)) +
  geom_histogram(bins = 30) +
  facet_wrap(vars(distribution), scales = "free") +
  labs(
    x = "Simulated Value",
    y = "Count"
  )

add_lagrange_theme_LFL(p)
```



1.3 関数型プログラミングと並列計算

purrr による反復処理

map() と型を指定する map_* 関数

map() は結果をリストとして返す。数値ベクトルが必要な場合は map_dbl() を用いる。

```
discount_rates <- c(0.01, 0.02, 0.03, 0.04)

discount_rates |>
  map(\(rate) calculate_present_value(100, rate))
#> [[1]]
#> [1] 99.0099
#>
#> [[2]]
#> [1] 98.03922
#>
#> [[3]]
#> [1] 97.08738
#>
#> [[4]]
#> [1] 96.15385
```

```
present_value_table <- tibble(
  discount_rate = discount_rates,
  present_value = discount_rates |>
```

```

    map_dbl(\(rate) calculate_present_value(100, rate))
  )

present_value_table
#> # A tibble: 4 x 2
#>   discount_rate present_value
#>       <dbl>         <dbl>
#> 1         0.01         99.010
#> 2         0.02         98.039
#> 3         0.03         97.087
#> 4         0.04         96.154

```

map2() で二つのベクトルを同時に処理する

```

cash_flows <- c(100, 120, 150)
years <- c(1, 2, 3)

map2_dbl(
  cash_flows,
  years,
  \(cash_flow, year) calculate_present_value(cash_flow, 0.02, year)
)
#> [1] 98.03922 115.34025 141.34835

```

pmap() で複数の列を処理する

```

cash_flow_scenarios <- tribble(
  ~cash_flow, ~discount_rate, ~years,
    100,        0.01,        1,
    120,        0.02,        2,
    150,        0.03,        3
)

cash_flow_scenarios |>
  mutate(
    present_value = pmap_dbl(
      list(cash_flow, discount_rate, years),
      \(cash_flow, discount_rate, years) {
        calculate_present_value(cash_flow, discount_rate, years)
      }
    )
  )
#> # A tibble: 3 x 4

```



```
#>  cash_flow discount_rate years present_value
#>      <dbl>      <dbl> <dbl>      <dbl>
#> 1      100      0.01      1      99.010
#> 2      120      0.02      2     115.34
#> 3      150      0.03      3     137.27
```

複数列へ同じ集計を適用する

```
return_columns <- c("firm1", "firm2")

return_columns |>
  map_dfr(
    \(column_name) {
      values <- tbl_daily_stock_return[[column_name]]

      tibble(
        firm = column_name,
        observations = length(values),
        mean = mean(values),
        standard_deviation = sd(values),
        minimum = min(values),
        maximum = max(values)
      )
    }
  )
#> # A tibble: 2 x 6
#>   firm observations      mean standard_deviation  minimum maximum
#>   <chr>         <int>    <dbl>          <dbl>      <dbl>  <dbl>
#> 1 firm1           21 0.014161          0.053840 -0.072071 0.17130
#> 2 firm2           21 0.0080253         0.054390 -0.096212 0.13520
```

複数の確率分布を同じ形式で生成する

```
set_random_seed_LFL(1234)

distribution_specs <- tribble(
  ~distribution, ~generator,
  "uniform",    \(n) runif(n, min = -1, max = 1),
  "normal",     \(n) rnorm(n, mean = 0, sd = 1),
  "poisson",    \(n) rpois(n, lambda = 3)
)

simulated_distributions <- distribution_specs |>
```

```
mutate(values = map(generator, \(generator) generator(1000))) |>
select(-generator) |>
unnest_longer(values)

simulated_distributions
#> # A tibble: 3,000 x 2
#>   distribution values
#>   <chr>         <dbl>
#> 1 uniform      -0.77259
#> 2 uniform       0.24460
#> 3 uniform       0.21855
#> 4 uniform       0.24676
#> 5 uniform       0.72183
#> 6 uniform       0.28062
#> 7 uniform      -0.98101
#> 8 uniform      -0.53490
#> 9 uniform       0.33217
#> 10 uniform      0.028502
#> # i 2,990 more rows
```

walk() で副作用だけを実行する

walk() は、返り値を保存するのではなく、表示などの副作用もしくはファイル出力などに実行するときに使う。

```
c("firm1", "firm2") |>
  walk(\(firm) print(str_c("集計対象: ", firm)))
#> [1] "集計対象: firm1"
#> [1] "集計対象: firm2"
```

nest によるグループ別データ管理

tidyr::nest() は、グループごとのデータを一つのリスト列へ格納する。group_by() が主に集計単位を指定するのに対し、nest() は銘柄別、業種別、ポートフォリオ別などのデータそのものを保持する。これにより、各グループへ同じ回帰分析、リスク計算、バックテストを適用しやすくなる。

```
tbl_return_long <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  )

tbl_nested <- tbl_return_long |>
  nest(tbl_data = c(date, return), .by = firm)
```

```
tbl_nested
#> # A tibble: 2 x 2
#>   firm   tbl_data
#>   <chr> <list>
#> 1 firm1 <tibble [21 x 2]>
#> 2 firm2 <tibble [21 x 2]>
```

リスト列 `tbl_data` の各要素は `tibble` である。最初の企業のデータは次のように確認できる。

```
tbl_nested$tbl_data[[1]]
#> # A tibble: 21 x 2
#>   date          return
#>   <date>         <dbl>
#> 1 2020-04-01 -0.039482
#> 2 2020-04-02  0.0059787
#> 3 2020-04-03  0.055787
#> 4 2020-04-06  0.041935
#> 5 2020-04-07 -0.020193
#> 6 2020-04-08 -0.0022947
#> 7 2020-04-09  0.081707
#> 8 2020-04-10  0.034561
#> 9 2020-04-13  0.032601
#> 10 2020-04-14 -0.037663
#> # i 11 more rows
```

`nest()` と `map()` を組み合わせると、グループ別の処理を一つのパイプラインとして記述できる。

```
tbl_nested_summary <- tbl_nested |>
  mutate(
    tbl_summary = map(
      tbl_data,
      \(tbl_data) tibble(
        num_mean_return = mean(tbl_data$return),
        num_volatility = sd(tbl_data$return)
      )
    )
  ) |>
  select(firm, tbl_summary) |>
  unnest(tbl_summary)

tbl_nested_summary
#> # A tibble: 2 x 3
#>   firm   num_mean_return num_volatility
#>   <chr>         <dbl>         <dbl>
#> 1 firm1         0.014161         0.053840
```

```
#> 2 firm2      0.0080253      0.054390
```

future と furrr による並列計算

`purrr::map()` は要素を順番に処理する。互いに独立した計算が多数ある場合、`future` と `furrr` を使うと、`map()` に近い記法のまま複数の R セッションへ処理を分配できる。Windows では `plan(multisession)` を使用する。

ここでは、モンテカルロ法による円周率の推定を 1000 回繰り返し、逐次処理と並列処理の実行時間を比較する。各試行では、正方形内に発生させた点のうち単位円内に入った割合を 4 倍する。

```
compute_pi_path <- function(int_path_id, int_points = 100000L) {
  set_random_seed_LFL(100000L + int_path_id)

  vec_x <- runif(int_points, min = -1, max = 1)
  vec_y <- runif(int_points, min = -1, max = 1)

  tibble(
    int_path_id = int_path_id,
    num_pi = 4 * mean(vec_x^2 + vec_y^2 <= 1)
  )
}
```

1000 本の試行を、利用可能な CPU 数を上限として最大 8 個のチャンクへ分割する。

```
int_paths <- 1000L
int_workers <- min(8L, future::availableCores())

tbl_chunks <- tibble(int_path_id = seq_len(int_paths)) |>
  mutate(
    int_chunk_id = rep(seq_len(int_workers), length.out = int_paths)
  ) |>
  nest(tbl_path_ids = int_path_id, .by = int_chunk_id)

tbl_chunks
#> # A tibble: 8 x 2
#>   int_chunk_id tbl_path_ids
#>       <int> <list>
#> 1         1 1 <tibble [125 x 1]>
#> 2         2 2 <tibble [125 x 1]>
#> 3         3 3 <tibble [125 x 1]>
#> 4         4 4 <tibble [125 x 1]>
#> 5         5 5 <tibble [125 x 1]>
#> 6         6 6 <tibble [125 x 1]>
#> 7         7 7 <tibble [125 x 1]>
```

```
#> 8      8 <tibble [125 x 1]>
```

```
compute_pi_chunk <- function(tbl_path_ids, int_points = 100000L) {
  tbl_path_ids$int_path_id |>
    map_dfr(compute_pi_path, int_points = int_points)
}
```

まず `map()` による逐次処理を実行する。

```
future::plan(future::sequential)

num_time_map <- system.time({
  tbl_result_map <- tbl_chunks |>
    mutate(
      tbl_result = map(tbl_path_ids, compute_pi_chunk)
    ) |>
    select(int_chunk_id, tbl_result) |>
    unnest(tbl_result)
})[["elapsed"]]
```

次に `future_map()` による並列処理を実行する。

```
future::plan(future::multisession, workers = int_workers)

num_time_future_map <- system.time({
  tbl_result_future <- tbl_chunks |>
    mutate(
      tbl_result = future_map(
        tbl_path_ids,
        compute_pi_chunk,
        .options = furrr::furrr_options(seed = TRUE)
      )
    ) |>
    select(int_chunk_id, tbl_result) |>
    unnest(tbl_result)
})[["elapsed"]]

future::plan(future::sequential)
```

推定値と実行時間を比較する。

```
tbl_pi_benchmark <- tribble(
  ~chr_method, ~num_pi, ~num_elapsed_seconds,
  "map", mean(tbl_result_map$num_pi), num_time_map,
  str_c("future_map (", int_workers, " workers)"), mean(tbl_result_future$num_pi), num_time_future_map
) |>
  mutate(
```

```

    num_absolute_error = abs(num_pi - pi),
    num_speedup = first(num_elapsed_seconds) / num_elapsed_seconds
  )

tbl_pi_benchmark
#> # A tibble: 2 x 5
#>   chr_method      num_pi num_elapsed_seconds num_absolute_error num_speedup
#>   <chr>          <dbl>          <dbl>          <dbl>          <dbl>
#> 1 map            3.1414            4.33            0.00016693            1
#> 2 future_map (8 worke~ 3.1415            2.84            0.000094774            1.5246

p <- tbl_pi_benchmark |>
  ggplot(aes(x = reorder(chr_method, num_elapsed_seconds), y = num_elapsed_seconds)) +
  geom_col() +
  coord_flip() +
  labs(x = NULL, y = "Elapsed time (seconds)")
  add_lagrange_theme_LFL(p)

```

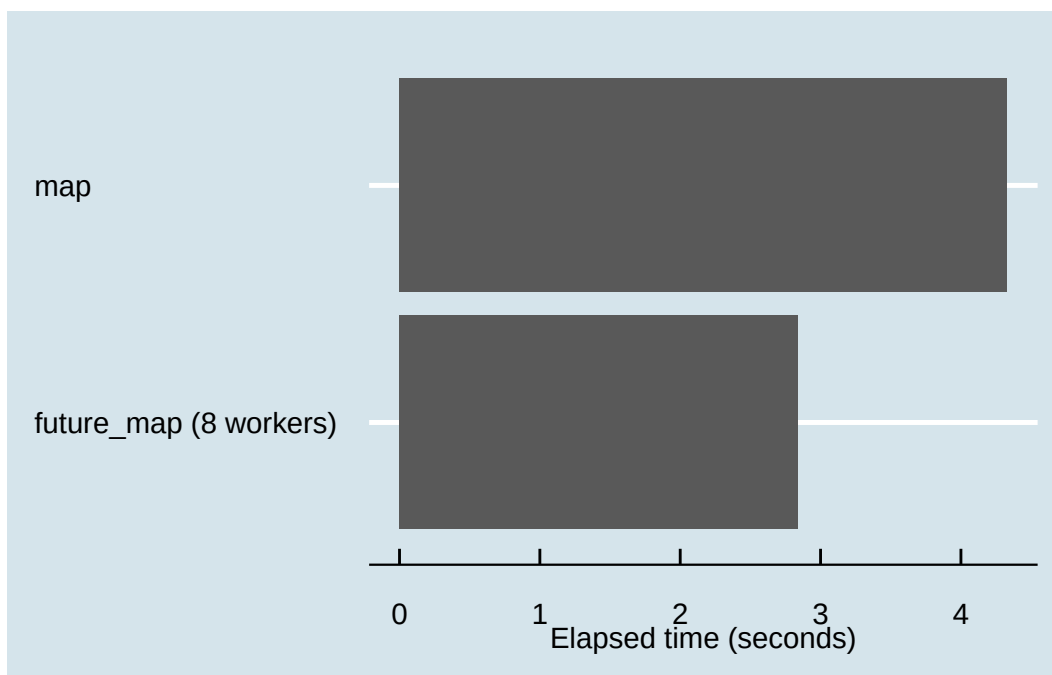


Figure1: map と future_map の実行時間比較

並列計算には、ワーカーの起動、データ転送、結果回収のオーバーヘッドがある。そのため、処理が軽い場合は `map()` の方が速いこともある。計算量が大きいモンテカルロ・シミュレーション、銘柄別モデル推定、シナリオ分析では、`future_map()` の効果が現れやすい。

1.4 データベースと大規模データ処理

金融データが大きくなると、すべての行と列を R のメモリへ読み込んでから処理する方法には限界がある。本節では、データベースを `dplyr` の文法で操作する `dbplyr` と、Parquet ファイルや Dataset を効率よく扱う Arrow を確認する。

dbplyr によるデータベース操作

`dbplyr` は、`dplyr` の処理を SQL へ変換する。データベース上で `filter()`、`select()`、`mutate()`、`summarise()` などを実行し、必要な結果だけを `collect()` で R へ取り込める。

ここでは、再現可能な例として一時的な SQLite データベースを使用する。

`dbplyr` では、処理は直ちに R 上で実行されず、SQL クエリとして組み立てられる。`show_query()` を使うと、生成された SQL を確認できる。

重要なのは、`collect()` を最後まで遅らせることである。先にデータベース側で行と列を絞り込めば、R へ転送するデータ量を小さくできる。

Arrow による Parquet データ処理

Apache Arrow は、列指向データを効率よく扱うための仕組みである。R では、Parquet ファイルの読書き、複数ファイルから成る Dataset の検索、Python や C++ など他言語との効率的なデータ交換に利用できる。

```
set_random_seed_LFL(20260726)

tbl_arrow_price <- crossing(
  date = seq.Date(as.Date("2025-01-01"), by = "day", length.out = 365),
  chr_symbol = str_c("STOCK_", str_pad(1:20, width = 2, pad = "0"))
) |>
  arrange(chr_symbol, date) |>
  mutate(
    num_return = rnorm(n(), mean = 0.0002, sd = 0.015),
    num_price = 100 * exp(cumsum(num_return)),
    .by = chr_symbol
  )

tbl_arrow_price
#> # A tibble: 7,300 x 4
#>   date      chr_symbol num_return num_price
#>   <date>    <chr>          <dbl>    <dbl>
#> 1 2025-01-01 STOCK_01    -0.014847    98.526
#> 2 2025-01-02 STOCK_01     0.00017318    98.543
#> 3 2025-01-03 STOCK_01     0.0027754    98.817
```

```
#> 4 2025-01-04 STOCK_01 0.010886 99.899
#> 5 2025-01-05 STOCK_01 0.016885 101.60
#> 6 2025-01-06 STOCK_01 0.017499 103.39
#> 7 2025-01-07 STOCK_01 0.046333 108.30
#> 8 2025-01-08 STOCK_01 -0.017287 106.44
#> 9 2025-01-09 STOCK_01 0.0033907 106.80
#> 10 2025-01-10 STOCK_01 -0.0078836 105.96
#> # i 7,290 more rows
```

```
path_parquet <- base::file.path(
  base::tempdir(),
  "financial_prices.parquet"
)

arrow::write_parquet(
  tbl_arrow_price,
  sink = path_parquet
)

base::basename(path_parquet)
#> [1] "financial_prices.parquet"
```

```
arrow::read_parquet(path_parquet) |>
  slice_head(n = 10)
#> # A tibble: 10 x 4
#>   date      chr_symbol num_return num_price
#>   <date>    <chr>         <dbl>    <dbl>
#> 1 2025-01-01 STOCK_01 -0.014847 98.526
#> 2 2025-01-02 STOCK_01 0.00017318 98.543
#> 3 2025-01-03 STOCK_01 0.0027754 98.817
#> 4 2025-01-04 STOCK_01 0.010886 99.899
#> 5 2025-01-05 STOCK_01 0.016885 101.60
#> 6 2025-01-06 STOCK_01 0.017499 103.39
#> 7 2025-01-07 STOCK_01 0.046333 108.30
#> 8 2025-01-08 STOCK_01 -0.017287 106.44
#> 9 2025-01-09 STOCK_01 0.0033907 106.80
#> 10 2025-01-10 STOCK_01 -0.0078836 105.96
```

Arrow Dataset

複数の Parquet ファイルをまとめて扱う場合は Dataset を使用する。ここでは銘柄をパーティションとして保存する。


```
path_dataset <- file.path(tempdir(), "financial_dataset")

arrow::write_dataset(
  tbl_arrow_price,
  path = path_dataset,
  format = "parquet",
  partitioning = "chr_symbol",
  existing_data_behavior = "overwrite"
)

dataset_finance <- arrow::open_dataset(path_dataset)

dataset_finance
#> FileSystemDataset with 20 Parquet files
#> 4 columns
#> date: date32[day]
#> num_return: double
#> num_price: double
#> chr_symbol: string
```

Dataset に対しても dplyr に近い文法を使用できる。必要な銘柄、期間、列を指定してから collect() する。

```
tbl_arrow_selected <- dataset_finance |>
  filter(
    chr_symbol == "STOCK_01",
    date >= as.Date("2025-07-01")
  ) |>
  select(date, chr_symbol, num_price, num_return) |>
  collect()

tbl_arrow_selected
#> # A tibble: 184 x 4
#>   date      chr_symbol num_price num_return
#>   <date>    <chr>      <dbl>    <dbl>
#> 1 2025-07-01 STOCK_01      92.811  0.0061451
#> 2 2025-07-02 STOCK_01      95.183  0.025240
#> 3 2025-07-03 STOCK_01      97.579  0.024861
#> 4 2025-07-04 STOCK_01      97.833  0.0025990
#> 5 2025-07-05 STOCK_01      97.612 -0.0022665
#> 6 2025-07-06 STOCK_01      98.638  0.010456
#> 7 2025-07-07 STOCK_01      98.926  0.0029172
#> 8 2025-07-08 STOCK_01      98.611 -0.0031869
#> 9 2025-07-09 STOCK_01     100.28   0.016826
#> 10 2025-07-10 STOCK_01      99.207 -0.010806
```

```
#> # i 174 more rows
```

Spark との関係

Arrow と Spark は役割が異なる。Arrow は、列指向形式、Parquet、Dataset、異なる言語間の高速なデータ交換を中心とし、1 台の PC でも利用しやすい。Spark は、複数のコンピュータから成るクラスターへデータと計算を分散するための処理基盤である。

データが 1 台の PC で扱える範囲では、Arrow と dbplyr が導入しやすい。データ量や計算量が 1 台のメモリと CPU を大きく超える場合には Spark が候補になる。本書では、実行環境を複雑にしないため、Arrow と dbplyr を実装対象とし、Spark は位置づけの説明にとどめる。## 総合演習

最後に、data/raw/market/daily_stock_returns.parquet から読み込んだ日次リターンを、読み込み、整形、集計、反復処理、可視化の順に分析する。

分析用データを作る

```
analysis_data <- tbl_daily_stock_return |>
  pivot_longer(
    cols = c(firm1, firm2),
    names_to = "firm",
    values_to = "return"
  ) |>
  mutate(
    gross_return = 1 + return,
    direction = case_when(
      return > 0 ~ "positive",
      return < 0 ~ "negative",
      TRUE ~ "zero"
    )
  ) |>
  arrange(firm, date)

analysis_data
#> # A tibble: 42 x 5
#>   date      firm      return gross_return direction
#>   <date>   <chr>    <dbl>      <dbl> <chr>
#> 1 2020-04-01 firm1 -0.039482    0.96052 negative
#> 2 2020-04-02 firm1  0.0059787    1.0060 positive
#> 3 2020-04-03 firm1  0.055787    1.0558 positive
#> 4 2020-04-06 firm1  0.041935    1.0419 positive
#> 5 2020-04-07 firm1 -0.020193    0.97981 negative
#> 6 2020-04-08 firm1 -0.0022947    0.99771 negative
#> 7 2020-04-09 firm1  0.081707    1.0817 positive
```

```
#> 8 2020-04-10 firm1 0.034561 1.0346 positive
#> 9 2020-04-13 firm1 0.032601 1.0326 positive
#> 10 2020-04-14 firm1 -0.037663 0.96234 negative
#> # i 32 more rows
```

企業別の要約統計量を計算する

```
firm_summary <- analysis_data |>
  group_by(firm) |>
  summarise(
    observations = n(),
    mean_return = mean(return),
    standard_deviation = sd(return),
    minimum_return = min(return),
    maximum_return = max(return),
    positive_ratio = mean(return > 0),
    .groups = "drop"
  )

firm_summary
#> # A tibble: 2 x 7
#>   firm observations mean_return standard_deviation minimum_return
#>   <chr>          <int>      <dbl>          <dbl>          <dbl>
#> 1 firm1             21  0.014161          0.053840       -0.072071
#> 2 firm2             21  0.0080253          0.054390       -0.096212
#> # i 2 more variables: maximum_return <dbl>, positive_ratio <dbl>
```

累積リターンを計算する

```
cumulative_return_data <- analysis_data |>
  group_by(firm) |>
  mutate(
    cumulative_gross_return = cumprod(gross_return),
    cumulative_return = cumulative_gross_return - 1
  ) |>
  ungroup()

cumulative_return_data
#> # A tibble: 42 x 7
#>   date      firm      return gross_return direction cumulative_gross_return
#>   <date>   <chr>    <dbl>      <dbl> <chr>          <dbl>
#> 1 2020-04-01 firm1 -0.039482  0.96052 negative      0.96052
```

```
#> 2 2020-04-02 firm1 0.0059787 1.0060 positive 0.96626
#> 3 2020-04-03 firm1 0.055787 1.0558 positive 1.0202
#> 4 2020-04-06 firm1 0.041935 1.0419 positive 1.0629
#> 5 2020-04-07 firm1 -0.020193 0.97981 negative 1.0415
#> 6 2020-04-08 firm1 -0.0022947 0.99771 negative 1.0391
#> 7 2020-04-09 firm1 0.081707 1.0817 positive 1.1240
#> 8 2020-04-10 firm1 0.034561 1.0346 positive 1.1628
#> 9 2020-04-13 firm1 0.032601 1.0326 positive 1.2007
#> 10 2020-04-14 firm1 -0.037663 0.96234 negative 1.1555
#> # i 32 more rows
#> # i 1 more variable: cumulative_return <dbl>
```

purrr で企業別の統計量を再計算する

```
firm_names <- unique(analysis_data$firm)

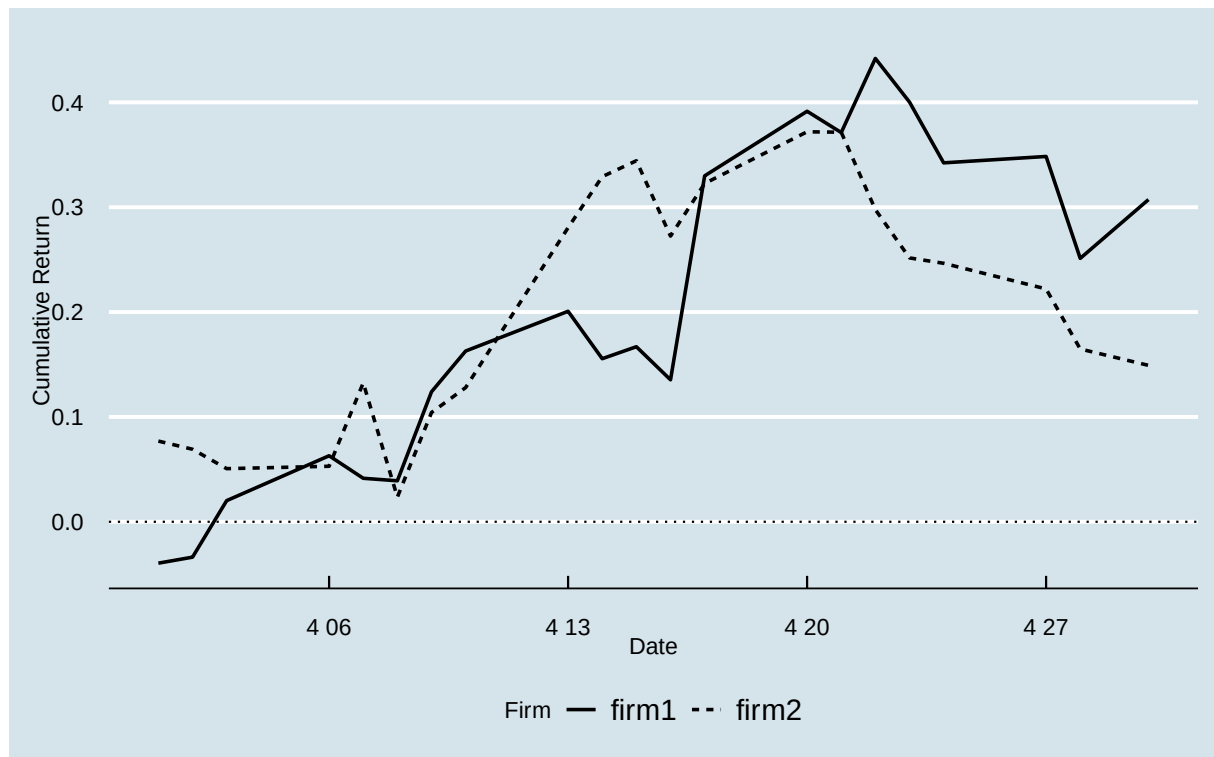
firm_names |>
  map_dfr(
    \(firm_name) {
      firm_returns <- analysis_data |>
        filter(firm == firm_name) |>
        pull(return)

      tibble(
        firm = firm_name,
        mean_return = mean(firm_returns),
        volatility = sd(firm_returns),
        lower_5_percent = quantile(firm_returns, 0.05),
        upper_95_percent = quantile(firm_returns, 0.95)
      )
    }
  )
#> # A tibble: 2 x 5
#>   firm mean_return volatility lower_5_percent upper_95_percent
#>   <chr>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1 firm1  0.014161  0.053840  -0.041378  0.081707
#> 2 firm2  0.0080253  0.054390  -0.053818  0.078843
```

累積リターンを可視化する

```
p <- cumulative_return_data |>
  ggplot(aes(x = date, y = cumulative_return, linetype = firm)) +
```

```
geom_line(linewidth = 0.8) +  
geom_hline(yintercept = 0, linetype = "dotted") +  
labs(  
  x = "Date",  
  y = "Cumulative Return",  
  linetype = "Firm"  
)  
p |> add_lagrange_theme_LFL()
```



まとめ

本章では、Base R の基本的なデータ構造と関数から出発し、記述統計、確率分布、tidyverse によるデータ分析までを一通り確認した。重要な点は次のとおりである。

- オブジェクト名には内容を表す snake_case を用いる。
- 同じ計算を複数の値へ適用するときは、まずベクトル化を検討する。
- 反復処理が必要な場合は purrr::map() 系関数を用いる。
- データ操作は dplyr、文字列処理は stringr、整形は tidyr を用いる。
- パイプ演算子は |> に統一する。
- グラフは ggplot2 でデータ、変数、図形を分けて組み立てる。
- データファイルは絶対パスではなく、プロジェクトルートを基準に読み込む。

金融データ分析パイプライン

本章で扱った技術は、次の流れとしてつながる。

CSV・データベース・Parquet



dbplyr・Arrow



filter・select



collect



tibble・stringr・tidyr



nest



map



future_map



集計・モデル推定・可視化

dbplyr と Arrow は、必要なデータを絞りと込む入口を担う。collect() 後は、tibble として文字列処理、整形、集計を行い、nest() で銘柄別・業種別・シナリオ別のデータを保持する。同じ分析は map() で反復し、計算量が多い場合は future_map() へ置き換える。可視化には ggplot2 を使い、本書では add_lagrange_theme_LFL() を標準テーマとする。

次章以降では、本章で学んだ操作を用いて、財務データと株式データの分析を進める。

第 1-2 章財務データの整理と分析

本章では、企業の財務データを読み込み、データ型、欠損値、年度構成および産業構成を確認する。

2.1 分析環境

2.2 財務データの読み込みと整備

2.2.1 財務データの読み込み

```
financial_data_raw <-
  read_parquet_file_LFL(
    file.path(
      "raw",
      "fundamentals",
      "financial_statements.parquet"
    )
  )

show_table_LFL(financial_data_raw, n = 10)
#> # A tibble: 7,920 x 11
#>   year fir_ID ind_ID sales    OX    NFE    X    OA    FA    OL
#>   * <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
#> 1  2015      1      1 5261.4 437.49 NA      286.64 13006.  3543.4 4373.0
#> 2  2016      1      1 5949.0 564.14 50.667  513.48 13866.  4642.2 4534.2
#> 3  2017      1      1 6505.1 691.18 29.543  661.64 13953.  7744.0 5111.2
#> 4  2018      1      1 6846.4 751.29 86.487  664.8  18818.  7284.7 5137.3
#> 5  2019      1      1 7572.2 958.53 298.05  660.48 18190   9735.1 5488.0
#> 6  2020      1      1 7537.6 778.37 -65.459 843.83 20463. 10274.  5371.4
#> 7  2015      2      1 3505.8  45.82  5.7511  40.07  2977.8  2258.3 1840.4
#> 8  2016      2      1 3491.3  51.25  1.8765  49.37  3184.8  1881.8 1769.2
#> 9  2017      2      1 3945.7  83.43  7.5279  75.9   3392.2  2425.2 1955.3
#> 10 2018      2      1 4139.3  93.4   6.8166  86.58  3569.3  2331.0 2022.1
#> # i 7,910 more rows
#> # i 1 more variable: FO <dbl>
```

2.2.2 データ構造の確認

```
show_data_structure_LFL(financial_data_raw)

#> Rows: 7,920
#> Columns: 11
#> $ year      <dbl> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ firm_ID    <dbl> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ industry_ID <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales      <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX         <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE        <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X          <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA         <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA         <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL         <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO         <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~

financial_data_overview <- financial_data_raw |>
  summarise(
    observations = n(),
    variables = ncol(financial_data_raw),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    firms = n_distinct(firm_ID),
    industries = n_distinct(industry_ID)
  )

show_table_LFL(financial_data_overview)

#> # A tibble: 1 x 6
#>   observations variables fir_year las_year firms industries
#>   <int>         <int>    <dbl>    <dbl> <int>         <int>
#> 1       7920         11    2015     2020  1515          10
```

列ごとの欠損数と欠損率を確認する。

```
missing_summary <- financial_data_raw |>
  summarise(
    across(
      everything(),
      list(
        missing = \ (x) sum(is.na(x)),
        missing_rate = \ (x) mean(is.na(x))
      )
    )
  )
```



```

    .names = "{.col}__{.fn}"
  )
) |>
pivot_longer(
  cols = everything(),
  names_to = c("variable", ".value"),
  names_sep = "__"
) |>
arrange(desc(missing_rate), variable)

show_table_LFL(missing_summary)
#> # A tibble: 11 x 3
#>   variable    missing  mis_rate
#>   <chr>         <int>    <dbl>
#> 1 NFE             1 0.00012626
#> 2 FA              0 0
#> 3 FO              0 0
#> 4 OA              0 0
#> 5 OL              0 0
#> 6 OX              0 0
#> 7 X               0 0
#> 8 firm_ID         0 0
#> 9 industry_ID     0 0
#> 10 sales           0 0
#> # i 1 more row

```

2.2.3 データ型と並び順の整理

企業 ID と産業 ID は分類変数であるためファクター型へ変換する。年度は整数型へ変換し、前年度の値を正しく取得できるように企業 ID と年度の順に並べる。

```

financial_data <- financial_data_raw |>
  mutate(
    firm_ID = as.factor(firm_ID),
    industry_ID = as.factor(industry_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year)

show_data_structure_LFL(financial_data)
#> Rows: 7,920
#> Columns: 11
#> $ year      <int> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ firm_ID    <fct> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~

```

```
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales      <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX         <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE        <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X          <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA         <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA         <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL         <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO         <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~
```

企業と年度の組合せが重複していないことを確認する。

```
panel_key_duplicates <- financial_data |>
  count(
    firm_ID,
    year,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(panel_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <fct>, year <int>, observations <int>

if (nrow(panel_key_duplicates) > 0) {
  stop(
    "firm_IDとyearの組合せに重複があります。",
    call. = FALSE
  )
}
```

2.3 標本構成と売上高分布

2.3.1 年度別の企業数

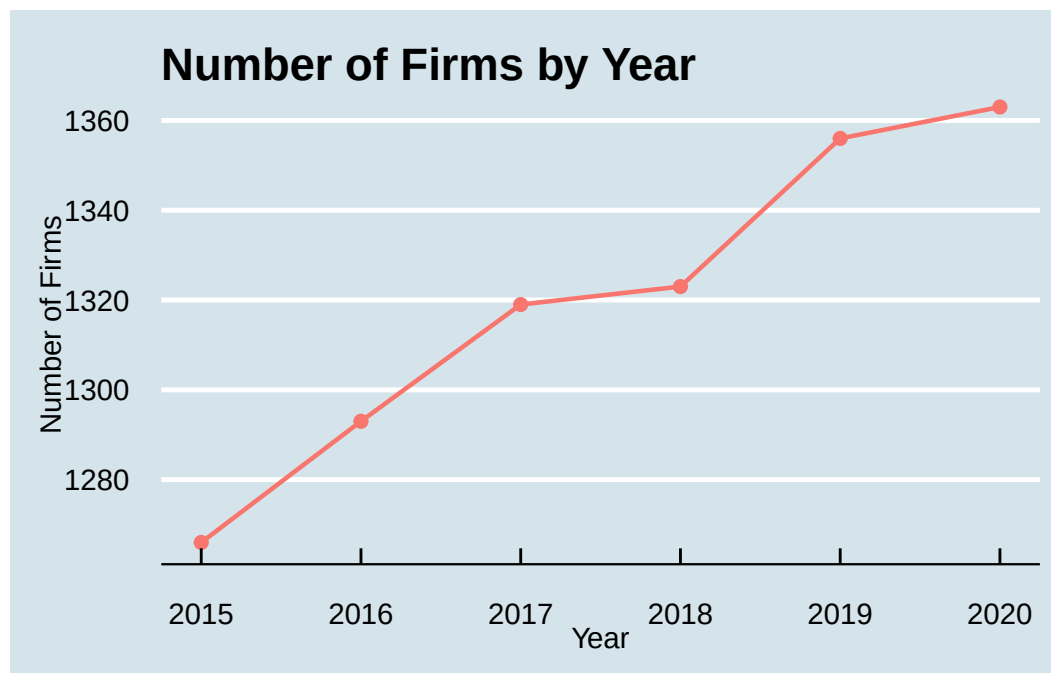
年度ごとの企業数と観測数を集計する。

```
firms_by_year <- financial_data |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = n(),
    .groups = "drop"
  ) |>
  arrange(year)
```

```
show_table_LFL(firms_by_year)
#> # A tibble: 6 x 3
#>   year firms observations
#>   <int> <int>         <int>
#> 1  2015  1266           1266
#> 2  2016  1293           1293
#> 3  2017  1319           1319
#> 4  2018  1323           1323
#> 5  2019  1356           1356
#> 6  2020  1363           1363
```

```
p_firms_by_year <- firms_by_year |>
  ggplot(aes(x = year, y = firms, colour = "Firms")) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = firms_by_year$year) +
  guides(colour = "none") +
  labs(
    x = "Year",
    y = "Number of Firms",
    title = "Number of Firms by Year"
  )

add_lagrange_theme_LFL(p_firms_by_year)
```



2.3.2 産業別の企業数

最新年度について、産業ごとの企業数を集計する。

```
latest_year <-  
  max(financial_data$year, na.rm = TRUE)  
  
firms_by_industry <- financial_data |>  
  filter(year == latest_year) |>  
  count(  
    industry_ID,  
    name = "firms",  
    sort = TRUE  
  )
```

```
show_table_LFL(firms_by_industry, n = 20)  
#> # A tibble: 10 x 2  
#>   ind_ID firms  
#>   <fct> <int>  
#> 1 3      309  
#> 2 10     291  
#> 3 7      233  
#> 4 1      192  
#> 5 9      123  
#> 6 8       71  
#> 7 5       59  
#> 8 2       37  
#> 9 4       28  
#> 10 6      20
```

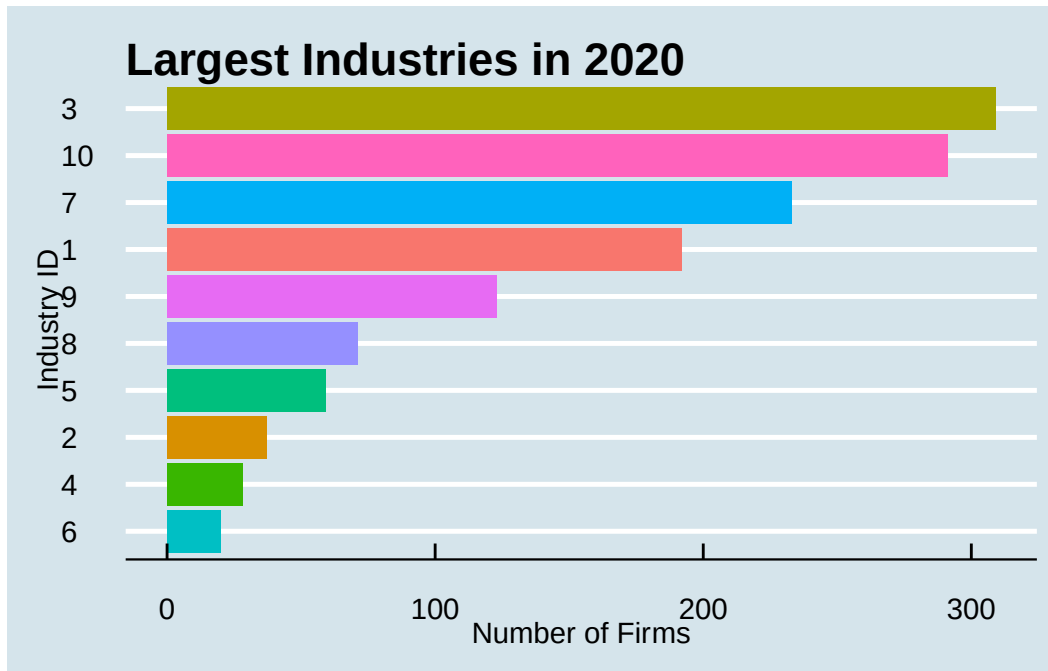
```
p_firms_by_industry <- firms_by_industry |>  
  slice_head(n = 15) |>  
  ggplot(  
    aes(  
      x = reorder(industry_ID, firms),  
      y = firms,  
      fill = industry_ID  
    )  
  ) +  
  geom_col(show.legend = FALSE) +  
  coord_flip() +  
  labs(  
    x = "Industry ID",  
    y = "Number of Firms",
```

```

    title = paste("Largest Industries in", latest_year)
  )

add_lagrange_theme_LFL(p_firms_by_industry)

```



2.3.3 売上高の分布

売上高は企業規模によって大きく異なるため、元の値と自然対数の両方を確認する。

```

sales_summary <- financial_data |>
  summarise(
    observations = sum(!is.na(sales)),
    minimum = min(sales, na.rm = TRUE),
    first_quartile = quantile(sales, 0.25, na.rm = TRUE),
    median = median(sales, na.rm = TRUE),
    mean = mean(sales, na.rm = TRUE),
    third_quartile = quantile(sales, 0.75, na.rm = TRUE),
    maximum = max(sales, na.rm = TRUE)
  )

show_table_LFL(sales_summary)
#> # A tibble: 1 x 7
#>   observations minimum fir_quart median    mean thi_quart maximum
#>   <int>      <dbl>    <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1      7920    205.34    16103. 40431. 166007. 118314. 3496433

p_sales_distribution <- financial_data |>
  filter(

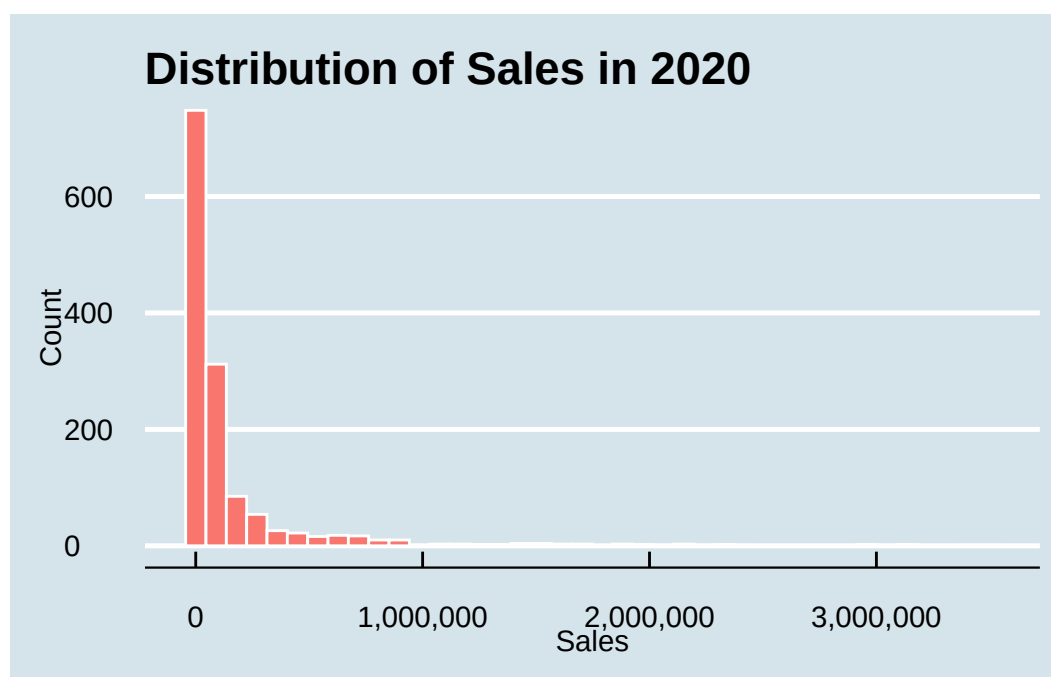
```

```

    year == latest_year,
    !is.na(sales),
    sales >= 0
  ) |>
  ggplot(aes(x = sales, fill = "Sales")) +
  geom_histogram(
    bins = 40,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(
    x = "Sales",
    y = "Count",
    title = paste("Distribution of Sales in", latest_year)
  )

add_lagrange_theme_LFL(p_sales_distribution)

```



```

p_log_sales_distribution <- financial_data |>
  filter(
    year == latest_year,
    !is.na(sales),
    sales > 0
  ) |>
  ggplot(aes(x = log(sales), fill = "Log Sales")) +
  geom_histogram(

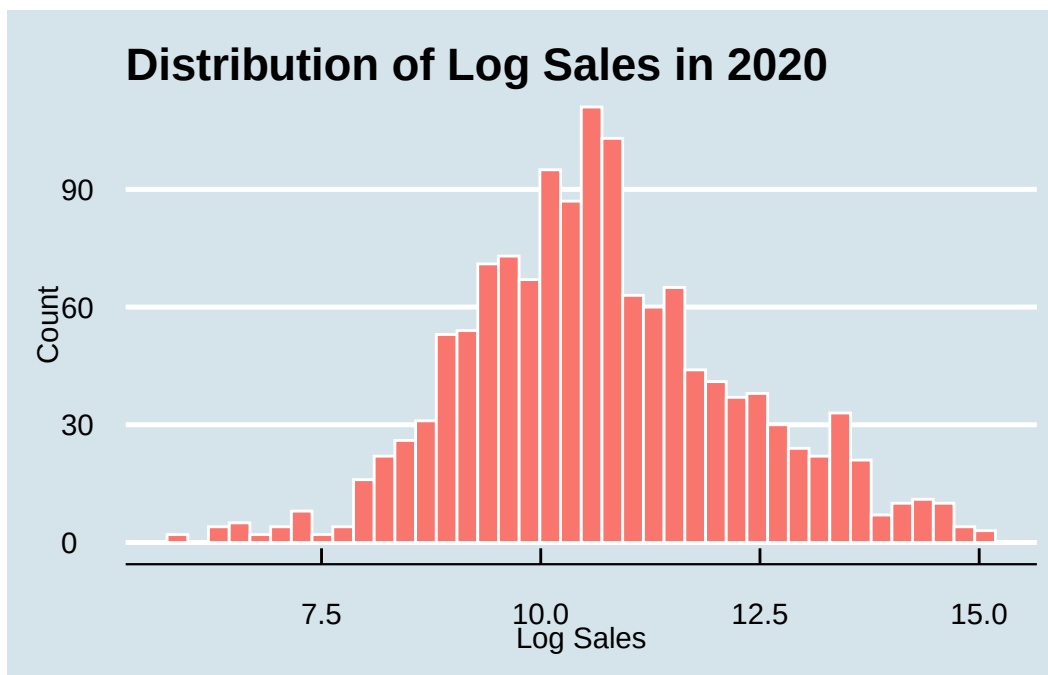
```

```

bins = 40,
colour = "white",
show.legend = FALSE
) +
labs(
  x = "Log Sales",
  y = "Count",
  title = paste("Distribution of Log Sales in", latest_year)
)

add_lagrange_theme_LFL(p_log_sales_distribution)

```



2.4 株主資本と ROE 分析

2.4.1 株主資本と ROE

本章では、営業資産から営業負債を控除した純営業資産と、金融負債から金融資産を控除した純金融負債の差として株主資本を計算する。

$$BE_t = (OA_t - OL_t) - (FO_t - FA_t)$$

ROE は当期純利益を期首株主資本で割って計算する。

$$ROE_t = \frac{X_t}{BE_{t-1}}$$

ゼロ除算や無限大は欠損値へ置き換える。

```
divide_safely_LFL <- function(numerator, denominator) {
  result <- numerator / denominator
  replace(result, !is.finite(result), NA_real_)
}
```

```
financial_data <- financial_data |>
  mutate(
    NOA = OA - OL,
    NFO = FO - FA,
    BE = NOA - NFO
  ) |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_BE = lag(BE),
    ROE = divide_safely_LFL(X, lagged_BE)
  ) |>
  ungroup()
```

```
financial_data |>
  select(
    firm_ID,
    industry_ID,
    year,
    X,
    BE,
    lagged_BE,
    ROE
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
```

```
#> # A tibble: 10 x 7
```

```
#>   fir_ID ind_ID year      X      BE lag_BE      ROE
#>   <fct> <fct> <int> <dbl> <dbl> <dbl> <dbl>
#> 1 1      1      2015 286.64 9695.3 NA      NA
#> 2 1      1      2016 513.48 10014. 9695.3 0.052962
#> 3 1      1      2017 661.64 10426. 10014. 0.066073
#> 4 1      1      2018 664.8 10842. 10426. 0.063762
#> 5 1      1      2019 660.48 11075. 10842. 0.060919
#> 6 1      1      2020 843.83 11594. 11075. 0.076193
#> 7 2      1      2015 40.07 1054.9 NA      NA
#> 8 2      1      2016 49.37 1081.6 1054.9 0.046800
#> 9 2      1      2017 75.9 1134.8 1081.6 0.070174
#> 10 2     1      2018 86.58 1183.7 1134.8 0.076295
```


ROE の要約統計量を確認する。

```
roe_summary <- financial_data |>
  summarise(
    observations = sum(!is.na(ROE)),
    mean = mean(ROE, na.rm = TRUE),
    median = median(ROE, na.rm = TRUE),
    standard_deviation = sd(ROE, na.rm = TRUE),
    first_percentile = quantile(ROE, 0.01, na.rm = TRUE),
    ninety_ninth_percentile = quantile(ROE, 0.99, na.rm = TRUE)
  )

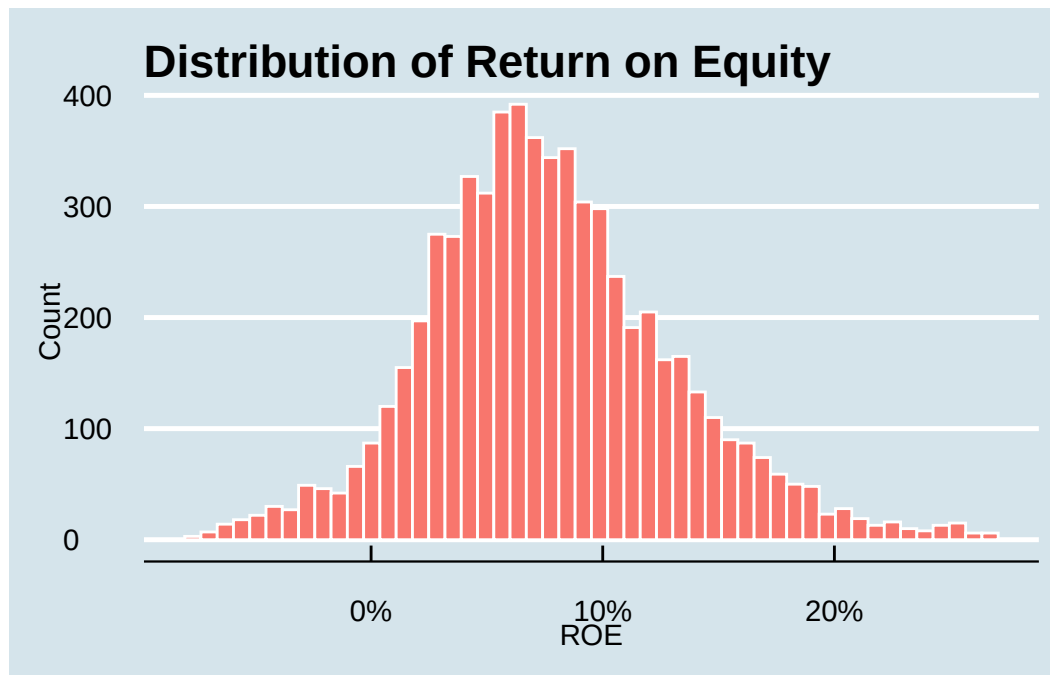
show_table_LFL(roe_summary)
#> # A tibble: 1 x 6
#>   observations      mean    median sta_devia fir_perce nin_perce
#>   <int>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1      6405 0.077713 0.072828 0.066274 -0.079541 0.26731
```

極端値の影響を避けるため、1 パーセンタイルから 99 パーセンタイルまでを表示する。

```
roe_limits <- quantile(
  financial_data$ROE,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_roe_distribution <- financial_data |>
  filter(
    !is.na(ROE),
    between(ROE, roe_limits[[1]], roe_limits[[2]])
  ) |>
  ggplot(aes(x = ROE, fill = "ROE")) +
  geom_histogram(
    bins = 50,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(labels = scales::label_percent()) +
  labs(
    x = "ROE",
    y = "Count",
    title = "Distribution of Return on Equity"
  )

add_lagrange_theme_LFL(p_roe_distribution)
```



2.4.2 産業別の ROE

```
industry_roe_summary <- financial_data |>
  group_by(industry_ID) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = sum(!is.na(ROE)),
    mean_ROE = mean(ROE, na.rm = TRUE),
    median_ROE = median(ROE, na.rm = TRUE),
    .groups = "drop"
  ) |>
  filter(observations > 0) |>
  arrange(desc(mean_ROE))

show_table_LFL(industry_roe_summary, n = 20)
```

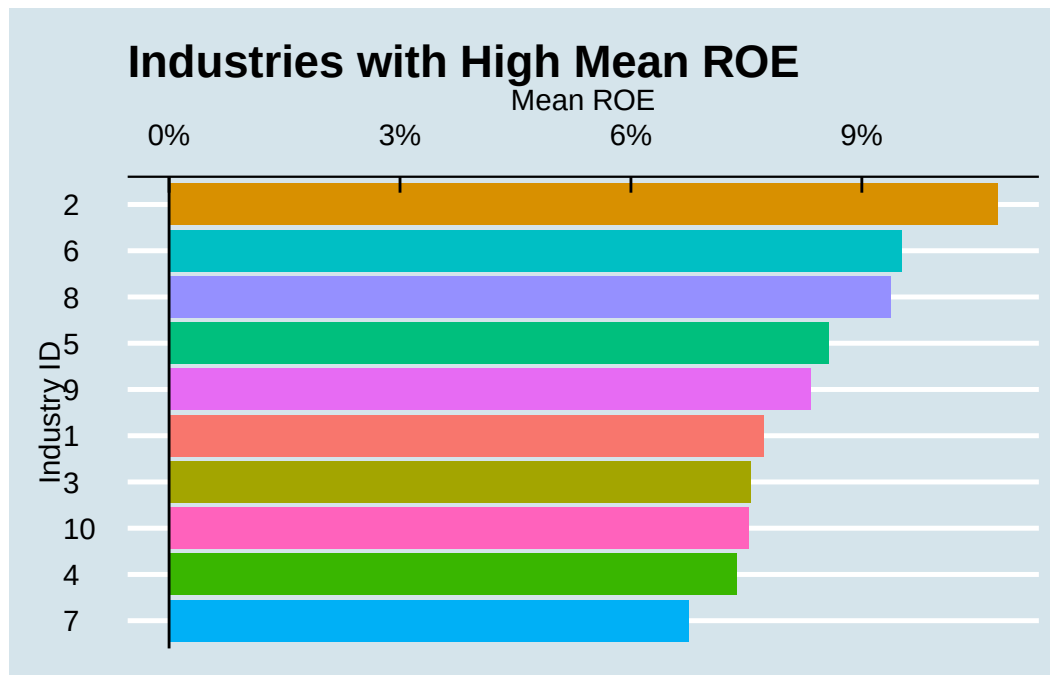
```
#> # A tibble: 10 x 5
#>   ind_ID firms observations  mea_ROE  med_ROE
#>   <fct> <int>      <int>    <dbl>    <dbl>
#> 1 2      40      186 0.10762  0.10283
#> 2 6      34      127 0.095142 0.091981
#> 3 8      80      349 0.093711 0.084671
#> 4 5      67      259 0.085703 0.084851
#> 5 9     136      531 0.083403 0.079794
#> 6 1     225      918 0.077285 0.075118
#> 7 3     327     1433 0.075489 0.070116
#> 8 10     314     1388 0.075273 0.070490
```

```
#> 9 4      34      138 0.073719 0.072702
#> 10 7     258     1076 0.067555 0.060409
```

産業ごとの標本数が小さい場合、平均値は一部企業の影響を強く受ける。ここでは ROE の観測数が 50 以上の産業だけを比較する。

```
p_industry_roe <- industry_roe_summary |>
  filter(observations >= 50) |>
  slice_max(
    order_by = mean_ROE,
    n = 15,
    with_ties = FALSE
  ) |>
  ggplot(
    aes(
      x = reorder(industry_ID, mean_ROE),
      y = mean_ROE,
      fill = industry_ID
    )
  ) +
  geom_col(show.legend = FALSE) +
  coord_flip() +
  geom_hline(yintercept = 0) +
  labs(
    x = "Industry ID",
    y = "Mean ROE",
    title = "Industries with High Mean ROE"
  )

add_lagrange_theme_LFL(p_industry_roe) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )
```



2.4.3 企業の順位付け

最新年度について、産業内で ROE が高い企業から順に順位を付ける。`dense_rank()` を使うと、同順位がある場合でも次の順位を飛ばさない。

```
latest_roe_ranking <- financial_data |>
  filter(
    year == latest_year,
    !is.na(ROE)
  ) |>
  group_by(industry_ID) |>
  mutate(
    ROE_rank = dense_rank(desc(ROE))
  ) |>
  ungroup() |>
  arrange(industry_ID, ROE_rank)

latest_roe_ranking |>
  select(
    industry_ID,
    ROE_rank,
    firm_ID,
    ROE
  ) |>
  slice_head(n = 20) |>
  show_table_LFL(n = 20)

#> # A tibble: 20 x 4
```

```
#>   ind_ID ROE_rank fir_ID    ROE
#>   <fct>    <int> <fct>    <dbl>
#> 1 1      1      1 8      0.38826
#> 2 1      2     90      0.33395
#> 3 1      3    225      0.30061
#> 4 1      4     81      0.23113
#> 5 1      5    171      0.21847
#> 6 1      6    138      0.20680
#> 7 1      7     66      0.20335
#> 8 1      8    153      0.19822
#> 9 1      9    181      0.19080
#> 10 1     10     65      0.18881
#> 11 1     11     92      0.17455
#> 12 1     12    195      0.15685
#> 13 1     13     13      0.15474
#> 14 1     14     23      0.15316
#> 15 1     15    148      0.14986
#> 16 1     16    214      0.14639
#> 17 1     17    115      0.14620
#> 18 1     18    201      0.14286
#> 19 1     19     50      0.14212
#> 20 1     20    143      0.14211
```

各産業で ROE が最も高い企業を抽出する。

```
top_firm_by_industry <- latest_roe_ranking |>
  group_by(industry_ID) |>
  slice_max(
    order_by = ROE,
    n = 1,
    with_ties = FALSE
  ) |>
  ungroup() |>
  select(
    industry_ID,
    firm_ID,
    ROE
  )

show_table_LFL(top_firm_by_industry, n = 20)
#> # A tibble: 10 x 3
#>   ind_ID fir_ID    ROE
#>   <fct> <fct>    <dbl>
#> 1 1      8      0.38826
```

```
#> 2 2      242    0.37500
#> 3 3      475    0.49754
#> 4 4      619    0.14913
#> 5 5      661    0.26731
#> 6 6      719    0.14220
#> 7 7      929    0.56418
#> 8 8     1042    0.25600
#> 9 9     1167    0.23462
#> 10 10     1380    0.24979
```

2.5 上級デュボン・モデル

2.5.1 ROE のデュボン分解

上級デュボン・モデルでは、ROE を純営業資産利益率と財務レバレッジの効果へ分解する。

$$ROE_t = RNOA_t + FLEV_{t-1}(RNOA_t - NBC_t)$$

各構成要素を次のように定義する。

$$RNOA_t = \frac{OX_t}{NOA_{t-1}}$$

$$PM_t = \frac{OX_t}{Sales_t}$$

$$ATO_t = \frac{Sales_t}{NOA_{t-1}}$$

$$FLEV_{t-1} = \frac{NFO_{t-1}}{BE_{t-1}}$$

$$NBC_t = \frac{NFE_t}{NFO_{t-1}}$$

```
financial_data_dupont <- financial_data |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_NOA = lag(NOA),
    lagged_NFO = lag(NFO),
    RNOA = divide_safely_LFL(OX, lagged_NOA),
    PM = divide_safely_LFL(OX, sales),
    ATO = divide_safely_LFL(sales, lagged_NOA),
```

```

    lagged_FLEV = divide_safely_LFL(lagged_NFO, lagged_BE),
    NBC = divide_safely_LFL(NFE, lagged_NFO),
    ROE_DuPont = RNOA + lagged_FLEV * (RNOA - NBC),
    DuPont_gap = ROE - ROE_DuPont
  ) |>
  ungroup()

financial_data_dupont |>
  select(
    firm_ID,
    year,
    ROE,
    RNOA,
    PM,
    ATO,
    lagged_FLEV,
    NBC,
    ROE_DuPont,
    DuPont_gap
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
#> # A tibble: 10 x 10
#>   fir_ID year    ROE    RNOA    PM    ATO lag_FLEV    NBC
#>   <fct> <int>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1 1      2015 NA      NA      0.083151 NA      NA      NA
#> 2 1      2016 0.052962 0.065350 0.094830 0.68913 -0.10961 -0.047678
#> 3 1      2017 0.066073 0.074071 0.10625 0.69712 -0.068152 -0.043289
#> 4 1      2018 0.063762 0.084974 0.10974 0.77436 -0.15202 -0.054567
#> 5 1      2019 0.060919 0.070062 0.12658 0.55348 0.26187 0.10498
#> 6 1      2020 0.076193 0.061279 0.10326 0.59342 0.14692 -0.040231
#> 7 2      2015 NA      NA      0.013070 NA      NA      NA
#> 8 2      2016 0.046800 0.045056 0.014679 3.0693 0.078263 0.022729
#> 9 2      2017 0.070174 0.058935 0.021145 2.7872 0.30884 0.022536
#> 10 2      2018 0.076295 0.064999 0.022564 2.8807 0.26625 0.022561
#> # i 2 more variables: ROE_DuPont <dbl>, DuPont_gap <dbl>

```

デュボン分解で再構成した ROE と、直接計算した ROE の差を確認する。

```

dupont_validation <- financial_data_dupont |>
  summarise(
    comparable_observations = sum(!is.na(DuPont_gap)),
    mean_absolute_gap = mean(abs(DuPont_gap), na.rm = TRUE),
    maximum_absolute_gap = max(abs(DuPont_gap), na.rm = TRUE)
  )

```

```
)

show_table_LFL(dupont_validation)
#> # A tibble: 1 x 3
#>   com_obser      mea_gap      max_gap
#>   <int>        <dbl>        <dbl>
#> 1     6405 0.00000036675 0.000020751
```

2.5.2 PM と ATO の関係

営業利益率 PM と純営業資産回転率 ATO は、企業の事業モデルを異なる側面から表す。産業別中央値を使って両者の関係を確認する。

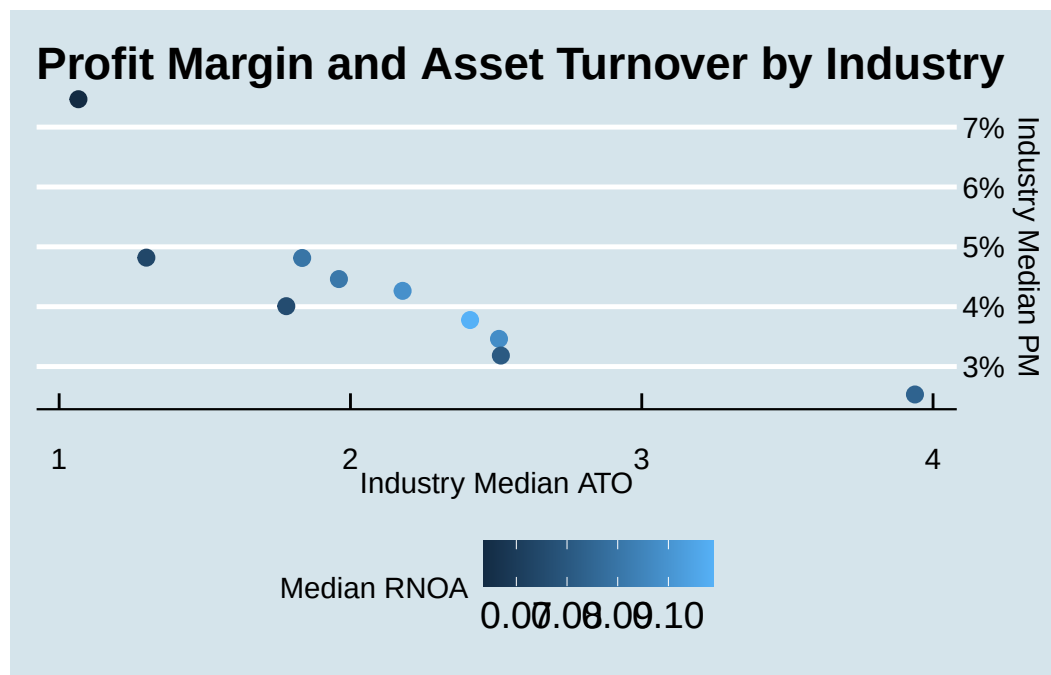
```
industry_pm_ato <- financial_data_dupont |>
  group_by(industry_ID) |>
  summarise(
    observations = sum(complete.cases(PM, ATO)),
    median_PM = median(PM, na.rm = TRUE),
    median_ATO = median(ATO, na.rm = TRUE),
    median_RNOA = median(RNOA, na.rm = TRUE),
    .groups = "drop"
  ) |>
  filter(
    observations >= 50,
    is.finite(median_PM),
    is.finite(median_ATO)
  )

show_table_LFL(industry_pm_ato, n = 20)
#> # A tibble: 10 x 5
#>   ind_ID observations      med_PM med_ATO med_RNOA
#>   <fct>         <int>        <dbl>   <dbl>   <dbl>
#> 1 1             918 0.025331  3.9378 0.083905
#> 2 2             186 0.037769  2.4106 0.10883
#> 3 3            1433 0.048140  1.8342 0.089713
#> 4 4             138 0.040086  1.7797 0.075919
#> 5 5             259 0.034620  2.5099 0.097564
#> 6 6             127 0.048194  1.2994 0.074109
#> 7 7            1076 0.031837  2.5163 0.080634
#> 8 8             349 0.074662  1.0662 0.063506
#> 9 9             531 0.042643  2.1790 0.098533
#> 10 10           1388 0.044615  1.9604 0.090718
```



```
p_pm_ato_tradeoff <- industry_pm_ato |>
  ggplot(
    aes(
      x = median_ATO,
      y = median_PM,
      colour = median_RNOA
    )
  ) +
  geom_point(size = 2.5) +
  labs(
    x = "Industry Median ATO",
    y = "Industry Median PM",
    colour = "Median RNOA",
    title = "Profit Margin and Asset Turnover by Industry"
  )

add_lagrange_theme_LFL(p_pm_ato_tradeoff) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )
```



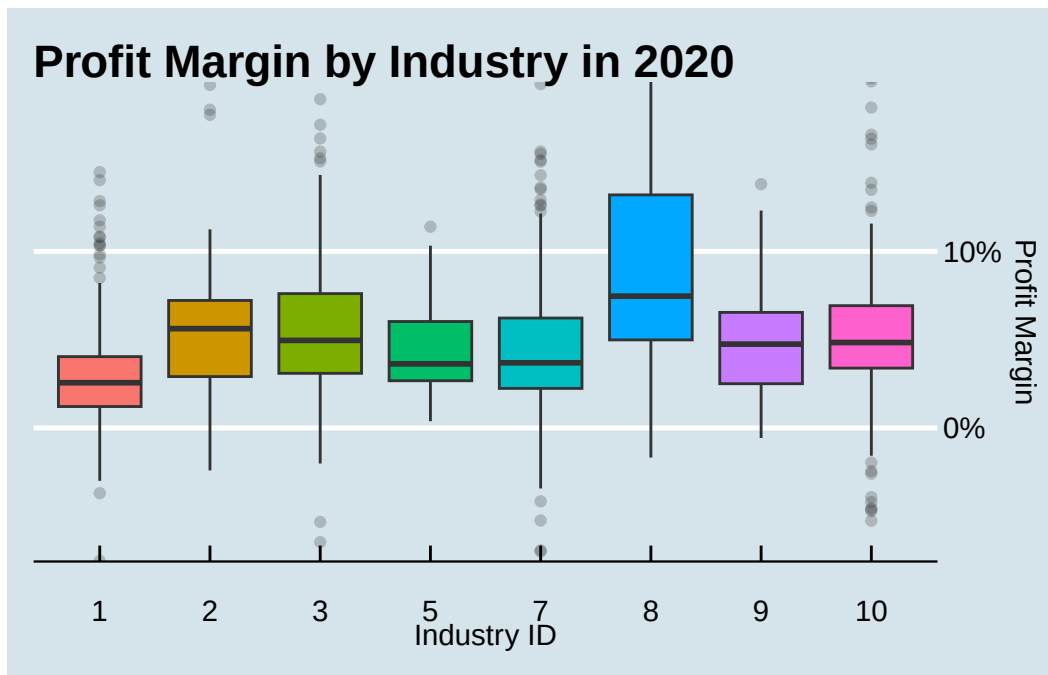
主要産業について PM の分布を箱ひげ図で比較する。

```
major_industries <- firms_by_industry |>
  slice_head(n = 8) |>
  pull(industry_ID)
```

```
pm_limits <- quantile(
  financial_data_dupont$PM,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_pm_by_industry <- financial_data_dupont |>
  filter(
    year == latest_year,
    industry_ID %in% major_industries,
    !is.na(PM)
  ) |>
  ggplot(
    aes(
      x = industry_ID,
      y = PM,
      fill = industry_ID
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25,
    show.legend = FALSE
  ) +
  coord_cartesian(ylim = pm_limits) +
  labs(
    x = "Industry ID",
    y = "Profit Margin",
    title = paste(
      "Profit Margin by Industry in",
      latest_year
    )
  )

add_lagrange_theme_LFL(p_pm_by_industry) +
  scale_y_continuous(
    position = "right",
    labels = scales::label_percent()
  )
```



2.6 財務指標の要約と中間データ

2.6.1 複数の財務指標の要約

複数列に同じ集計を適用する場合は、列名のベクトルと `purrr::map_dfr()` を組み合わせる。

```
financial_metrics <- c(
  "sales",
  "BE",
  "ROE",
  "RNOA",
  "PM",
  "ATO"
)

metric_summary <- financial_metrics |>
  map_dfr(
    \(metric_name) {
      values <- financial_data_dupont[[metric_name]]
      finite_values <- values[is.finite(values)]

      tibble(
        metric = metric_name,
        observations = length(finite_values),
        mean = mean(finite_values),
        median = median(finite_values),
        standard_deviation = sd(finite_values),

```

```

        minimum = min(finite_values),
        maximum = max(finite_values)
    )
}
)

show_table_LFL(metric_summary)
#> # A tibble: 6 x 7
#>   metric observations      mean      median sta_devia  minimum  maximum
#>   <chr>          <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 sales            7920 166007.    40431.    3.8198e+5 205.34    3.4964e+6
#> 2 BE              7920 111517.    26882.    4.2062e+5 136.93    2.5448e+7
#> 3 ROE             6405   0.077713   0.072828  6.6274e-2 -0.70192  1.4055e+0
#> 4 RNOA            6405   0.11141    0.085868  1.2622e-1 -0.82036  1.8283e+0
#> 5 PM              7920   0.045534   0.040555  4.2223e-2 -0.20079  3.8289e-1
#> 6 ATO             6405   2.7051     2.1024    1.8684e+0  0.25825  9.8170e+0

```

2.6.2 中間データの保存

第 3 章では、本章で計算した株主資本、前年度株主資本および ROE などを株式データと結合する。そのため、上級デュボン・モデルの計算結果を含む整理済みデータを `data/derived/fundamentals/financial_characteristics.parquet` へ保存する。

```

financial_output_path <-
  write_derived_parquet_file_LFL(
    financial_data_dupont,
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  )

tbl_financial_output <- tibble::tibble(
  output_file = relative_path_LFL(financial_output_path),
  size = base::file.info(financial_output_path)$size,
  modified = base::file.info(financial_output_path)$mtime
)

show_table_LFL(tbl_financial_output)
#> # A tibble: 1 x 3
#>   out_file          size modified
#>   <chr>          <dbl> <dtm>
#> 1 data/derived/fundamentals/financial_characteristi~ 1357543 2026-08-29 18:31:52

```

保存した Parquet ファイルを読み直し、行数と列名が一致することを確認する。

```
financial_output_check <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  )

stopifnot(
  nrow(financial_output_check) ==
    nrow(financial_data_dupont)
)

stopifnot(
  identical(
    names(financial_output_check),
    names(financial_data_dupont)
  )
)

output_summary <- tibble(
  output_file = relative_path_LFL(financial_output_path),
  rows = nrow(financial_output_check),
  columns = ncol(financial_output_check)
)

show_table_LFL(output_summary)
#> # A tibble: 1 x 3
#>   out_file                                rows columns
#>   <chr>                                <int>   <int>
#> 1 data/derived/fundamentals/financial_characteristics.parquet 7920     25
```

2.7 まとめ

本章では、財務データの構造、欠損値およびパネルデータのキーを確認した。次に、年度別・産業別の企業構成、売上高の分布、株主資本、ROE および産業内順位を計算した。最後に、ROE を RNOA、財務レバレッジおよび純借入コストへ分解し、第 3 章で利用する整理済みデータを `data/derived/fundamentals/financial_characteristics.parquet` へ保存した。

第 3 章では、株式データからリターンを計算し、本章で作成した財務データと結合する。

第 I-3 章 株式データと時系列分析

本章では、月次株式データから株式リターン、超過リターンおよび時価総額を作成し、第 2 章で整理した年次財務データと結合する。さらに、月次リターンの分布、累積リターン、年次リターンおよび簿価時価比率との関係を確認し、第 4 章以降で利用する分析用データを保存する。

3.1 分析環境

3.2 株式データの読み込みと整備

3.2.1 株式データの読み込み

```
stock_data_raw <-
read_parquet_file_LFL(
  file.path(
    "raw",
    "market",
    "stock_prices_and_returns.parquet"
  )
)

show_table_LFL(stock_data_raw, n = 10)
```

```
#> # A tibble: 95,040 x 9
#>   year month mon_ID fir_ID sto_price  DPS sha_outst adj_coeff      R_F
#>   * <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1  2015     1     1     1     954     0  2422000     1 0.00065068
#> 2  2015     2     2     1     960     0  2422000     1 0.00058341
#> 3  2015     3     3     1    1113     0  2422000     1 0.00061144
#> 4  2015     4     4     1    1081     0  2422000     1 0.00068482
#> 5  2015     5     5     1    1317     0  2422000     1 0.00073677
#> 6  2015     6     6     1    1366    29  2422000     1 0.00069540
#> 7  2015     7     7     1    1353     0  2422000     1 0.00049354
#> 8  2015     8     8     1    1209     0  2422000     1 0.00047106
#> 9  2015     9     9     1    1291     0  2422000     1 0.00055242
#> 10 2015    10    10     1    1407     0  2422000     1 0.00063803
#> # i 95,030 more rows
```

本章の計算に必要な列がすべて存在することを確認する。

```
required_stock_columns <- c(
  "year",
  "month",
  "month_ID",
  "firm_ID",
  "stock_price",
  "shares_outstanding",
  "DPS",
  "adjustment_coefficient",
  "R_F"
)

missing_stock_columns <-
  setdiff(required_stock_columns, names(stock_data_raw))

if (length(missing_stock_columns) > 0) {
  stop(
    "株式データに必要な列が不足しています: ",
    paste(missing_stock_columns, collapse = ", "),
    call. = FALSE
  )
}
```

3.2.2 データ構造と欠損値

```
show_data_structure_LFL(stock_data_raw)
#> Rows: 95,040
#> Columns: 9
#> $ year                <dbl> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month               <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID            <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID             <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price         <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS                 <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding  <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ R_F                 <dbl> 0.00065068255, 0.00058340985, 0.00061144232, 0.~

stock_data_overview <- stock_data_raw |>
  summarise(
    observations = n(),
    variables = ncol(stock_data_raw),
```

```

    firms = n_distinct(firm_ID),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    first_month_ID = min(month_ID, na.rm = TRUE),
    last_month_ID = max(month_ID, na.rm = TRUE)
  )

show_table_LFL(stock_data_overview)
#> # A tibble: 1 x 7
#>   observations variables firms fir_year las_year fir_ID las_ID
#>   <int>      <int> <int>    <dbl>    <dbl>  <dbl>  <dbl>
#> 1      95040         9 1515    2015    2020     1    72

stock_missing_summary <- stock_data_raw |>
  summarise(
    across(
      everything(),
      list(
        missing = \ (x) sum(is.na(x)),
        missing_rate = \ (x) mean(is.na(x))
      ),
      .names = "{.col}__{.fn}"
    )
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = c("variable", ".value"),
    names_sep = "__"
  ) |>
  arrange(desc(missing_rate), variable)

show_table_LFL(
  stock_missing_summary,
  n = nrow(stock_missing_summary)
)
#> # A tibble: 9 x 3
#>   variable      missing mis_rate
#>   <chr>          <int>    <dbl>
#> 1 DPS              0        0
#> 2 R_F              0        0
#> 3 adjustment_coefficient 0        0
#> 4 firm_ID          0        0
#> 5 month            0        0
#> 6 month_ID         0        0

```



```
#> 7 shares_outstanding      0      0
#> 8 stock_price             0      0
#> 9 year                   0      0
```

3.2.3 データ型とパネルキー

企業ごとの時系列計算を正しく行うため、企業 ID と月 ID の順に並べる。

```
stock_data <- stock_data_raw |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year),
    month = as.integer(month),
    month_ID = as.integer(month_ID),
    month_date = lubridate::make_date(year, month, 1)
  ) |>
  arrange(firm_ID, month_ID)

show_data_structure_LFL(stock_data)
#> Rows: 95,040
#> Columns: 10
#> $ year      <int> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month     <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID  <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID   <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS       <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ R_F       <dbl> 0.00065068255, 0.00058340985, 0.00061144232, 0.~
#> $ month_date <date> 2015-01-01, 2015-02-01, 2015-03-01, 2015-04-01~
```

firm_ID と month_ID の組合せが一意であることを確認する。

```
stock_key_duplicates <- stock_data |>
  count(
    firm_ID,
    month_ID,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(stock_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, mon_ID <int>, observations <int>
```

```

if (nrow(stock_key_duplicates) > 0) {
  stop(
    "firm_IDとmonth_IDの組合せに重複があります。",
    call. = FALSE
  )
}

```

同じ月 ID に複数の年月が対応していないことも確認する。

```

month_calendar_check <- stock_data |>
  distinct(
    month_ID,
    year,
    month,
    month_date
  ) |>
  count(
    month_ID,
    name = "calendar_rows"
  ) |>
  filter(calendar_rows > 1)

show_table_LFL(month_calendar_check)
#> # A tibble: 0 x 2
#> # i 2 variables: mon_ID <int>, cal_rows <int>

if (nrow(month_calendar_check) > 0) {
  stop(
    "同じmonth_IDに複数の年月が対応しています。",
    call. = FALSE
  )
}

```

年度別の企業数と観測数を確認する。

```

stock_observations_by_year <- stock_data |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    observations = n(),
    first_month_ID = min(month_ID),
    last_month_ID = max(month_ID),
    .groups = "drop"
  ) |>

```

```

arrange(year)

show_table_LFL(
  stock_observations_by_year,
  n = nrow(stock_observations_by_year)
)
#> # A tibble: 6 x 5
#>   year firms observations fir_ID las_ID
#>   <int> <int>         <int> <int> <int>
#> 1  2015  1266         15192     1    12
#> 2  2016  1293         15516    13    24
#> 3  2017  1319         15828    25    36
#> 4  2018  1323         15876    37    48
#> 5  2019  1356         16272    49    60
#> 6  2020  1363         16356    61    72

```

3.3 月次リターンと時価総額

3.3.1 配当と調整係数

配当が支払われた観測と、調整係数が 1 と異なる観測を確認する。

```

dividend_observations <- stock_data |>
  filter(
    !is.na(DPS),
    DPS != 0
  ) |>
  select(
    firm_ID,
    year,
    month,
    month_ID,
    stock_price,
    DPS,
    adjustment_coefficient
  ) |>
  arrange(firm_ID, month_ID)

adjustment_observations <- stock_data |>
  filter(
    !is.na(adjustment_coefficient),
    !near(adjustment_coefficient, 1)
  ) |>
  select(

```

```

    firm_ID,
    year,
    month,
    month_ID,
    stock_price,
    DPS,
    adjustment_coefficient
  ) |>
  arrange(firm_ID, month_ID)

corporate_action_summary <- stock_data |>
  summarise(
    observations = n(),
    dividend_observations =
      sum(!is.na(DPS) & DPS != 0),
    adjusted_observations =
      sum(
        !is.na(adjustment_coefficient) &
        !near(adjustment_coefficient, 1)
      )
  )

show_table_LFL(dividend_observations, n = 20)
#> # A tibble: 14,250 x 7
#>   fir_ID year month mon_ID sto_price  DPS adj_coeff
#>   <int> <int> <int>  <int>    <dbl> <dbl>    <dbl>
#> 1     1   2015     6     6    1366    29      1
#> 2     1   2015    12    12    1477    29      1
#> 3     1   2016     6    18    1696    40      1
#> 4     1   2016    12    24    2842    40      1
#> 5     1   2017     6    30    4317    43      1
#> 6     1   2017    12    36    3915    43      1
#> 7     1   2018     6    42    2986    43      1
#> 8     1   2018    12    48    2992    43      1
#> 9     1   2019     6    54    3608    53      1
#> 10    1   2019    12    60    4803    53      1
#> 11    1   2020     6    66    4698    56      1
#> 12    1   2020    12    72    3341    56      1
#> 13     2   2015     6     6    2167     3      1
#> 14     2   2015    12    12    2613     3      1
#> 15     2   2016     6    18    2395     7      1
#> 16     2   2016    12    24    3576     7      1
#> 17     2   2017     6    30    4966     7      1

```

```
#> 18      2 2017    12    36    5852     7     1
#> 19      2 2018     6    42    4415    12     1
#> 20      2 2018    12    48    4542    12     1
#> # i 14,230 more rows
show_table_LFL(adjustment_observations, n = 20)
#> # A tibble: 267 x 7
#>   fir_ID  year month mon_ID sto_price  DPS adj_coeff
#>   <int> <int> <int>  <int>    <dbl> <dbl>    <dbl>
#> 1     1    2017     4    28    4082     0     1.2
#> 2     9    2015     4     4    2655     0     1.2
#> 3    10    2015     2     2     757     0     1.2
#> 4    21    2019    12    60    3008    19     1.1
#> 5    27    2019     9    57   13750     0     0.1
#> 6    42    2016     5    17   47324     0     0.1
#> 7    48    2019     2    50   13232     0     0.1
#> 8    53    2019     4    52    1754     0     1.1
#> 9    54    2019     2    50    4015     0     1.1
#> 10   60    2019    12    60    1749     0     1.1
#> 11   66    2019    11    59   10140     0     0.1
#> 12   67    2018     1    37    2020     0     1.1
#> 13   72    2016     5    17    1876     0     1.1
#> 14   74    2017     6    30    1402    11     2
#> 15   80    2018     3    39    1393     0     1.2
#> 16   84    2017     4    28   10697     0     0.1
#> 17   90    2020     2    62  153396     0     0.1
#> 18   96    2015     4     4     805     0     1.2
#> 19  103    2020     6    66   44488   468     0.1
#> 20  104    2016     1    13    9191     0     0.1
#> # i 247 more rows
show_table_LFL(corporate_action_summary)
#> # A tibble: 1 x 3
#>   observations div_obser adj_obser
#>   <int>      <int>    <int>
#> 1   95040    14250     267
```

3.3.2 時価総額

企業 i の月 t における時価総額 $ME_{i,t}$ は、株価 $P_{i,t}$ と発行済株式数 $N_{i,t}$ の積である。

$$ME_{i,t} = P_{i,t} N_{i,t}$$

```
stock_data <- stock_data |>
  mutate(
```

```

ME = stock_price * shares_outstanding,
ME = if_else(
  is.finite(ME) & ME > 0,
  ME,
  NA_real_
)
)

stock_data |>
  select(
    firm_ID,
    month_date,
    stock_price,
    shares_outstanding,
    ME
  ) |>
  slice_head(n = 10) |>
  show_table_LFL(n = 10)
#> # A tibble: 10 x 5
#>   fir_ID mon_date  sto_price sha_outst      ME
#>   <int> <date>      <dbl>    <dbl>    <dbl>
#> 1     1  2015-01-01      954  2422000 2310588000
#> 2     2  2015-02-01      960  2422000 2325120000
#> 3     3  2015-03-01     1113  2422000 2695686000
#> 4     4  2015-04-01     1081  2422000 2618182000
#> 5     5  2015-05-01     1317  2422000 3189774000
#> 6     6  2015-06-01     1366  2422000 3308452000
#> 7     7  2015-07-01     1353  2422000 3276966000
#> 8     8  2015-08-01     1209  2422000 2928198000
#> 9     9  2015-09-01     1291  2422000 3126802000
#> 10    10  2015-10-01     1407  2422000 3407754000

```

```

p_market_equity <- stock_data |>
  filter(
    is.finite(ME),
    ME > 0
  ) |>
  ggplot(aes(x = ME, fill = "Market Equity")) +
  geom_histogram(
    bins = 50,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_log10(

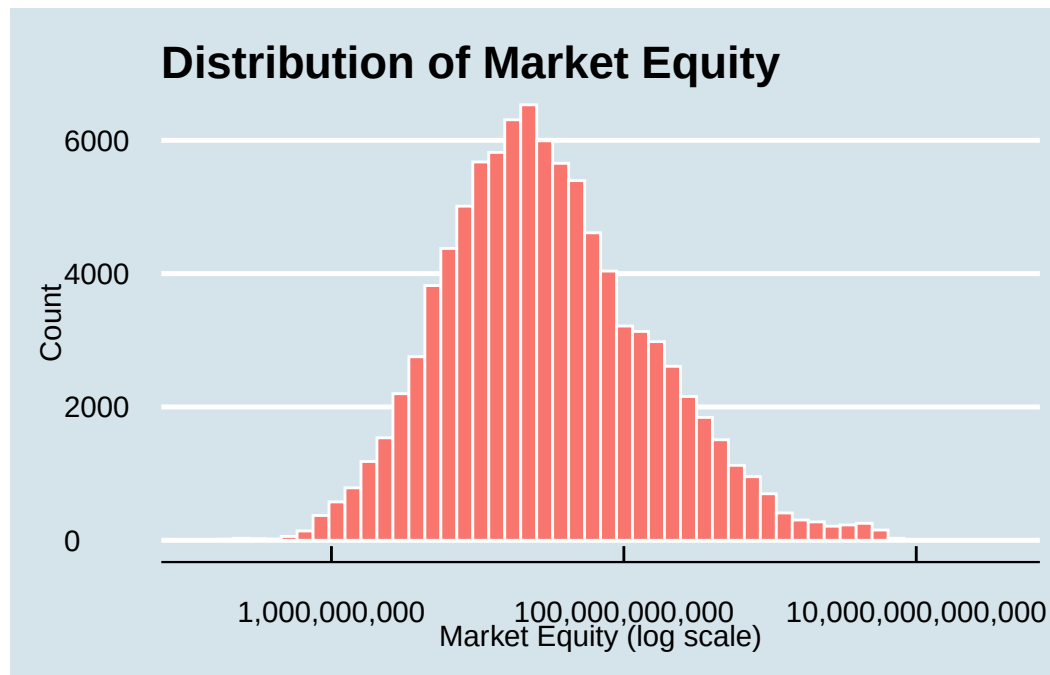
```

```

    labels = scales::label_comma()
  ) +
  labs(
    x = "Market Equity (log scale)",
    y = "Count",
    title = "Distribution of Market Equity"
  )

add_lagrange_theme_LFL(p_market_equity)

```



3.3.3 月次株式リターン

企業ごとに前月株価を作成し、月 ID が連続している場合だけ利用する。

```

stock_data <- stock_data |>
  group_by(firm_ID) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(
    lagged_stock_price = lag(stock_price),
    lagged_month_ID = lag(month_ID),
    consecutive_month =
      month_ID - lagged_month_ID == 1L,
    lagged_stock_price = if_else(
      consecutive_month,
      lagged_stock_price,
      NA_real_
    )
  )

```

```
) |>
ungroup()
```

月次株式リターン $R_{i,t}$ を、株価、配当および調整係数から計算する。

$$R_{i,t} = \frac{(P_{i,t} + D_{i,t})A_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

```
stock_data <- stock_data |>
  mutate(
    R = case_when(
      is.na(lagged_stock_price) ~ NA_real_,
      lagged_stock_price <= 0 ~ NA_real_,
      is.na(stock_price) ~ NA_real_,
      is.na(DPS) ~ NA_real_,
      is.na(adjointment_coefficient) ~ NA_real_,
      TRUE ~ (
        (stock_price + DPS) *
          adjointment_coefficient -
          lagged_stock_price
      ) / lagged_stock_price
    ),
    R = if_else(
      is.finite(R),
      R,
      NA_real_
    )
  )

stock_data |>
  select(
    firm_ID,
    month_date,
    lagged_stock_price,
    stock_price,
    DPS,
    adjointment_coefficient,
    R
  ) |>
  filter(!is.na(R)) |>
  slice_head(n = 15) |>
  show_table_LFL(n = 15)

#> # A tibble: 15 x 7
#>   fir_ID mon_date lag_price sto_price  DPS adj_coeff      R
#>   <dbl> <date>   <dbl>   <dbl>   <dbl> <dbl>   <dbl>
```


#>	<int>	<date>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	1	2015-02-01	954	960	0	1	0.0062893
#> 2	1	2015-03-01	960	1113	0	1	0.15937
#> 3	1	2015-04-01	1113	1081	0	1	-0.028751
#> 4	1	2015-05-01	1081	1317	0	1	0.21832
#> 5	1	2015-06-01	1317	1366	29	1	0.059226
#> 6	1	2015-07-01	1366	1353	0	1	-0.0095168
#> 7	1	2015-08-01	1353	1209	0	1	-0.10643
#> 8	1	2015-09-01	1209	1291	0	1	0.067825
#> 9	1	2015-10-01	1291	1407	0	1	0.089853
#> 10	1	2015-11-01	1407	1562	0	1	0.11016
#> 11	1	2015-12-01	1562	1477	29	1	-0.035851
#> 12	1	2016-01-01	1477	1561	0	1	0.056872
#> 13	1	2016-02-01	1561	1487	0	1	-0.047406
#> 14	1	2016-03-01	1487	1526	0	1	0.026227
#> 15	1	2016-04-01	1526	1542	0	1	0.010485

3.3.4 無リスク金利と超過リターン

同じ月の無リスク金利はすべての企業に共通であることを確認し、月ごとの中央値で統一する。

```
risk_free_rate_by_month <- stock_data |>
  group_by(
    month_ID,
    year,
    month,
    month_date
  ) |>
  summarise(
    observations = n(),
    valid_rates = sum(is.finite(R_F)),
    distinct_rates =
      n_distinct(R_F[is.finite(R_F)]),
    R_F_month = {
      values <- R_F[is.finite(R_F)]

      if (length(values) == 0) {
        NA_real_
      } else {
        median(values)
      }
    },
    .groups = "drop"
  ) |>
```

```

arrange(month_ID)

risk_free_rate_diagnostics <-
  risk_free_rate_by_month |>
  filter(distinct_rates > 1)

show_table_LFL(
  risk_free_rate_diagnostics,
  n = nrow(risk_free_rate_diagnostics)
)
#> # A tibble: 72 x 8
#>   mon_ID year month mon_date observations val_rates dis_rates R_month
#>   <int> <int> <int> <date>          <int>    <int>    <int>    <dbl>
#> 1     1     1  2015     1 2015-01-01        1266    1266     83 0.000082029
#> 2     2     2  2015     2 2015-02-01        1266    1266     83 0.000082029
#> 3     3     3  2015     3 2015-03-01        1266    1266     83 0.000082029
#> 4     4     4  2015     4 2015-04-01        1266    1266     83 0.000082029
#> 5     5     5  2015     5 2015-05-01        1266    1266     83 0.000082029
#> 6     6     6  2015     6 2015-06-01        1266    1266     83 0.000082029
#> 7     7     7  2015     7 2015-07-01        1266    1266     83 0.000082029
#> 8     8     8  2015     8 2015-08-01        1266    1266     83 0.000082029
#> 9     9     9  2015     9 2015-09-01        1266    1266     83 0.000082029
#> 10    10    10  2015    10 2015-10-01        1266    1266     83 0.000082029
#> 11    11    11  2015    11 2015-11-01        1266    1266     83 0.000082029
#> 12    12    12  2015    12 2015-12-01        1266    1266     83 0.000082029
#> 13    13    2016     1 2016-01-01        1293    1293     83 0.000082029
#> 14    14    2016     2 2016-02-01        1293    1293     83 0.000075959
#> 15    15    2016     3 2016-03-01        1293    1293     83 0.000075959
#> 16    16    2016     4 2016-04-01        1293    1293     83 0.000082029
#> 17    17    2016     5 2016-05-01        1293    1293     83 0.000082029
#> 18    18    2016     6 2016-06-01        1293    1293     83 0.000082029
#> 19    19    2016     7 2016-07-01        1293    1293     83 0.000082029
#> 20    20    2016     8 2016-08-01        1293    1293     83 0.000082029
#> 21    21    2016     9 2016-09-01        1293    1293     83 0.000082029
#> 22    22    2016    10 2016-10-01        1293    1293     83 0.000075959
#> 23    23    2016    11 2016-11-01        1293    1293     83 0.000075959
#> 24    24    2016    12 2016-12-01        1293    1293     83 0.000075959
#> 25    25    2017     1 2017-01-01        1319    1319     83 0.000075959
#> 26    26    2017     2 2017-02-01        1319    1319     83 0.000075959
#> 27    27    2017     3 2017-03-01        1319    1319     83 0.000075959
#> 28    28    2017     4 2017-04-01        1319    1319     83 0.000082029
#> 29    29    2017     5 2017-05-01        1319    1319     83 0.000075959
#> 30    30    2017     6 2017-06-01        1319    1319     83 0.000075959

```

#> 31	31	2017	7	2017-07-01	1319	1319	83	0.000075959
#> 32	32	2017	8	2017-08-01	1319	1319	83	0.000075959
#> 33	33	2017	9	2017-09-01	1319	1319	83	0.000075959
#> 34	34	2017	10	2017-10-01	1319	1319	83	0.000075959
#> 35	35	2017	11	2017-11-01	1319	1319	83	0.000075959
#> 36	36	2017	12	2017-12-01	1319	1319	83	0.000075959
#> 37	37	2018	1	2018-01-01	1323	1323	83	0.000075959
#> 38	38	2018	2	2018-02-01	1323	1323	83	0.000075959
#> 39	39	2018	3	2018-03-01	1323	1323	83	0.000082029
#> 40	40	2018	4	2018-04-01	1323	1323	83	0.000082029
#> 41	41	2018	5	2018-05-01	1323	1323	83	0.000082029
#> 42	42	2018	6	2018-06-01	1323	1323	83	0.000075959
#> 43	43	2018	7	2018-07-01	1323	1323	83	0.000075959
#> 44	44	2018	8	2018-08-01	1323	1323	83	0.000082029
#> 45	45	2018	9	2018-09-01	1323	1323	83	0.000082029
#> 46	46	2018	10	2018-10-01	1323	1323	83	0.000082029
#> 47	47	2018	11	2018-11-01	1323	1323	83	0.000082029
#> 48	48	2018	12	2018-12-01	1323	1323	83	0.000082029
#> 49	49	2019	1	2019-01-01	1356	1356	83	0.000075959
#> 50	50	2019	2	2019-02-01	1356	1356	83	0.000078994
#> 51	51	2019	3	2019-03-01	1356	1356	83	0.000082029
#> 52	52	2019	4	2019-04-01	1356	1356	83	0.000082029
#> 53	53	2019	5	2019-05-01	1356	1356	83	0.000082029
#> 54	54	2019	6	2019-06-01	1356	1356	83	0.000075959
#> 55	55	2019	7	2019-07-01	1356	1356	83	0.000082029
#> 56	56	2019	8	2019-08-01	1356	1356	83	0.000082029
#> 57	57	2019	9	2019-09-01	1356	1356	83	0.000082029
#> 58	58	2019	10	2019-10-01	1356	1356	83	0.000075959
#> 59	59	2019	11	2019-11-01	1356	1356	83	0.000075959
#> 60	60	2019	12	2019-12-01	1356	1356	83	0.000075959
#> 61	61	2020	1	2020-01-01	1363	1363	83	0.000082029
#> 62	62	2020	2	2020-02-01	1363	1363	83	0.000082029
#> 63	63	2020	3	2020-03-01	1363	1363	83	0.000082029
#> 64	64	2020	4	2020-04-01	1363	1363	83	0.000075959
#> 65	65	2020	5	2020-05-01	1363	1363	83	0.000075959
#> 66	66	2020	6	2020-06-01	1363	1363	83	0.000075959
#> 67	67	2020	7	2020-07-01	1363	1363	83	0.000075959
#> 68	68	2020	8	2020-08-01	1363	1363	83	0.000075959
#> 69	69	2020	9	2020-09-01	1363	1363	83	0.000075959
#> 70	70	2020	10	2020-10-01	1363	1363	83	0.000082029
#> 71	71	2020	11	2020-11-01	1363	1363	83	0.000082029
#> 72	72	2020	12	2020-12-01	1363	1363	83	0.000082029

```

stock_data <- stock_data |>
  select(-R_F) |>
  left_join(
    risk_free_rate_by_month |>
      select(
        month_ID,
        R_F = R_F_month
      ),
    by = "month_ID"
  )

missing_monthly_risk_free_rate <- stock_data |>
  distinct(month_ID, R_F) |>
  filter(!is.finite(R_F))

if (nrow(missing_monthly_risk_free_rate) > 0) {
  stop(
    "月次無リスク金利を作成できないmonth_IDがあります。",
    call. = FALSE
  )
}

```

月次超過リターン $R_{i,t}^e$ は、株式リターンから無リスク金利を差し引いて計算する。

$$R_{i,t}^e = R_{i,t} - R_{F,t}$$

```

stock_data <- stock_data |>
  mutate(
    Re = R - R_F,
    Re = if_else(
      is.finite(Re),
      Re,
      NA_real_
    )
  )

```

3.4 月次リターンの分析

3.4.1 要約統計量

```

calculate_skewness_LFL <- function(x) {
  values <- x[is.finite(x)]

```

```
if (
  length(values) < 3 ||
  near(sd(values), 0)
) {
  return(NA_real_)
}

mean(
  ((values - mean(values)) /
    sd(values))^3
)
}

calculate_kurtosis_LFL <- function(x) {
  values <- x[is.finite(x)]

  if (
    length(values) < 4 ||
    near(sd(values), 0)
  ) {
    return(NA_real_)
  }

  mean(
    ((values - mean(values)) /
      sd(values))^4
  )
}
```

```
monthly_return_summary <- stock_data |>
  select(R, Re) |>
  pivot_longer(
    cols = everything(),
    names_to = "return_type",
    values_to = "return"
  ) |>
  filter(is.finite(return)) |>
  group_by(return_type) |>
  summarise(
    observations = n(),
    mean = mean(return),
    median = median(return),
    standard_deviation = sd(return),
    minimum = min(return),
```

```

    first_quartile = quantile(return, 0.25),
    third_quartile = quantile(return, 0.75),
    maximum = max(return),
    skewness = calculate_skewness_LFL(return),
    kurtosis = calculate_kurtosis_LFL(return),
    .groups = "drop"
  )

show_table_LFL(monthly_return_summary)
#> # A tibble: 2 x 11
#>   ret_type observations      mean  median sta_devia  minimum fir_quart thi_quart
#>   <chr>          <int>    <dbl>   <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 R              93525 0.015859 0.010332 0.091131 -0.38028 -0.040573 0.064808
#> 2 Re             93525 0.015779 0.010252 0.091131 -0.38036 -0.040652 0.064730
#> # i 3 more variables: maximum <dbl>, skewness <dbl>, kurtosis <dbl>

```

3.4.2 リターン分布

極端値の影響を抑えるため、1 パーセントイルから 99 パーセントイルまでを表示する。

```

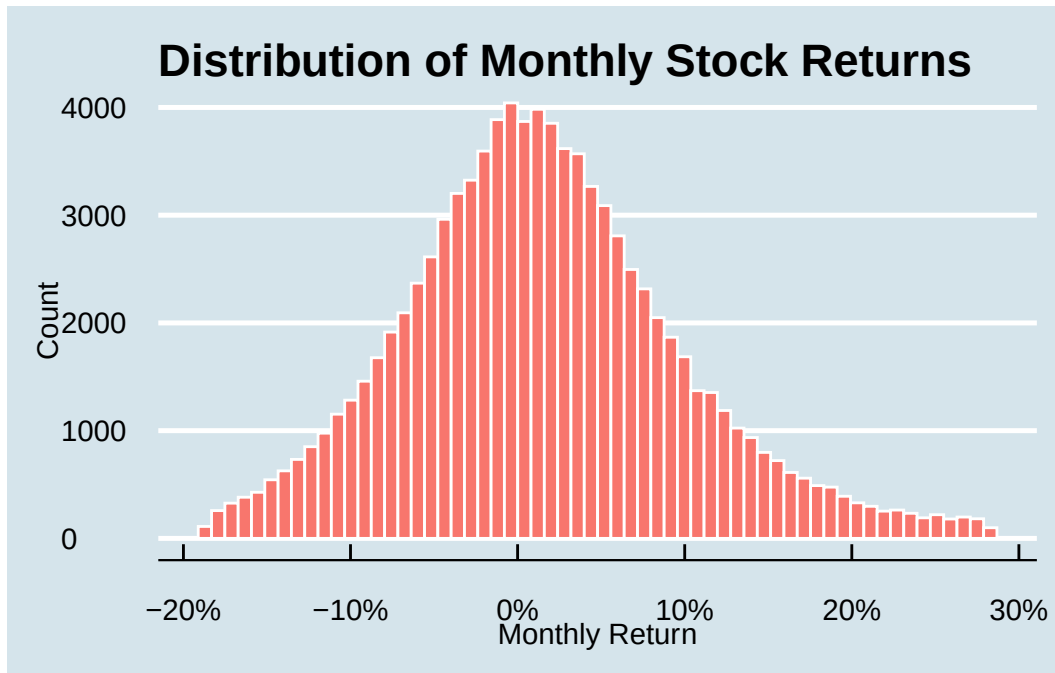
monthly_return_limits <- quantile(
  stock_data$R,
  probs = c(0.01, 0.99),
  na.rm = TRUE
)

p_monthly_return_distribution <- stock_data |>
  filter(
    is.finite(R),
    between(
      R,
      monthly_return_limits[[1]],
      monthly_return_limits[[2]]
    )
  ) |>
  ggplot(aes(x = R, fill = "Monthly Return")) +
  geom_histogram(
    bins = 60,
    colour = "white",
    show.legend = FALSE
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +

```

```
labs(
  x = "Monthly Return",
  y = "Count",
  title = "Distribution of Monthly Stock Returns"
)

add_lagrange_theme_LFL(
  p_monthly_return_distribution
)
```



3.4.3 企業別の累積リターン

観測数が多い企業から 6 社を選び、月次リターンを複利で累積する。

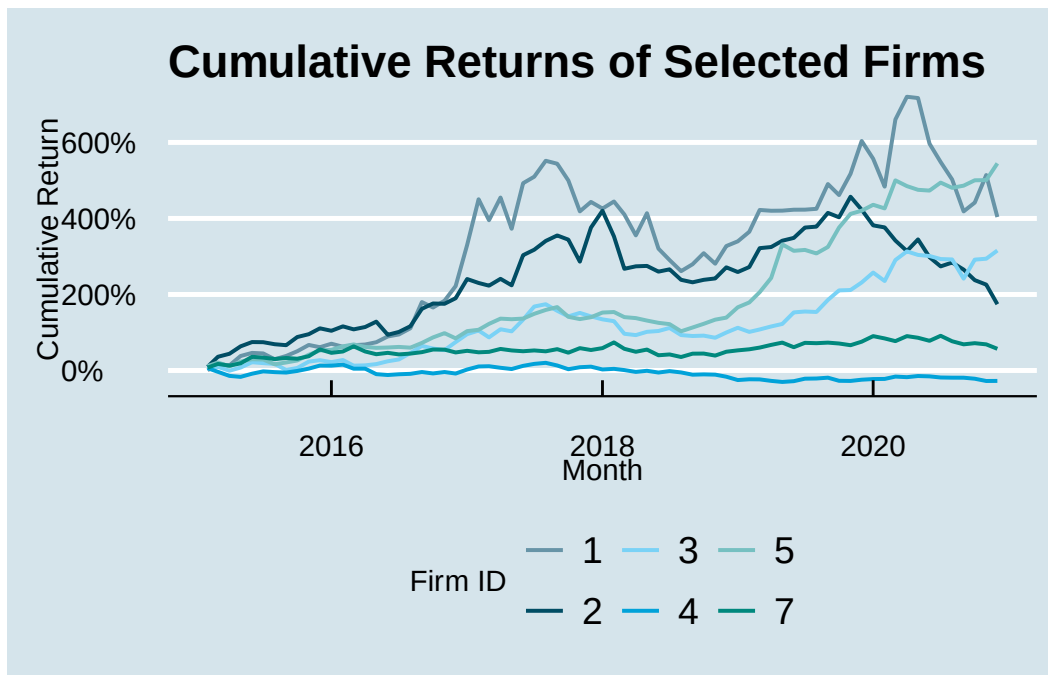
```
sample_firm_ids <- stock_data |>
  filter(is.finite(R)) |>
  count(
    firm_ID,
    sort = TRUE
  ) |>
  slice_head(n = 6) |>
  pull(firm_ID)

cumulative_return_data <- stock_data |>
  filter(
    firm_ID %in% sample_firm_ids,
    is.finite(R)
```

```
) |>
group_by(firm_ID) |>
arrange(month_ID, .by_group = TRUE) |>
mutate(
  cumulative_return = cumprod(1 + R) - 1
) |>
ungroup()

p_cumulative_return <- cumulative_return_data |>
ggplot(
  aes(
    x = month_date,
    y = cumulative_return,
    colour = factor(firm_ID)
  )
) +
geom_line(linewidth = 0.7) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Month",
  y = "Cumulative Return",
  colour = "Firm ID",
  title = "Cumulative Returns of Selected Firms"
)

add_lagrange_theme_LFL(p_cumulative_return) +
ggthemes::scale_colour_economist()
```

3.4.4 企業別の要約統計量

```

firm_return_summary <- sample_firm_ids |>
  map_dfr(
    \(firm_id) {
      firm_returns <- stock_data |>
        filter(
          firm_ID == firm_id,
          is.finite(R)
        ) |>
        pull(R)

      tibble(
        firm_ID = firm_id,
        observations = length(firm_returns),
        mean_return = mean(firm_returns),
        volatility = sd(firm_returns),
        minimum_return = min(firm_returns),
        maximum_return = max(firm_returns)
      )
    }
  )

show_table_LFL(firm_return_summary)
#> # A tibble: 6 x 6
#>   fir_ID observations  mea_retur volatility min_retur max_retur

```

#>	<int>	<int>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	1	71	0.029240	0.11544	-0.18157	0.33603
#> 2	2	71	0.018125	0.089135	-0.18825	0.24481
#> 3	3	71	0.023028	0.075369	-0.14460	0.16440
#> 4	4	71	-0.0030159	0.054766	-0.13816	0.11475
#> 5	5	71	0.028390	0.061602	-0.097372	0.25720
#> 6	7	71	0.0077876	0.053950	-0.096949	0.14549

3.5 年次データと財務データの結合

3.5.1 月次リターンの年次集約

企業 i の年 t における年次リターンは、月次リターンを複利で集約する。

$$R_{i,t}^{annual} = \prod_{m=1}^{M_{i,t}} (1 + R_{i,t,m}) - 1$$

```
annual_stock_data <- stock_data |>
  group_by(
    firm_ID,
    year
  ) |>
  summarise(
    months = sum(is.finite(R)),
    R = if (
      sum(is.finite(R)) == 0
    ) {
      NA_real_
    } else {
      prod(1 + R[is.finite(R)]) - 1
    },
    Re = if (
      sum(is.finite(Re)) == 0
    ) {
      NA_real_
    } else {
      prod(1 + Re[is.finite(Re)]) - 1
    },
    ME = {
      values <- ME[is.finite(ME)]

      if (length(values) == 0) {
        NA_real_
      } else {
```

```

      dplyr::last(values)
    }
  },
  .groups = "drop"
) |>
  arrange(firm_ID, year)

show_table_LFL(annual_stock_data, n = 20)
#> # A tibble: 7,920 x 6
#>   fir_ID year months      R      Re      ME
#>   <int> <int> <int>   <dbl>   <dbl>   <dbl>
#> 1     1     1  2015     11  0.61213  0.61073 3577294000
#> 2     1     1  2016     12  0.99727  0.99547 6883324000
#> 3     1     1  2017     12  0.68786  0.68637 11376990000
#> 4     1     1  2018     12 -0.21361 -0.21439 8694752000
#> 5     1     1  2019     12  0.64684  0.64533 13957518000
#> 6     1     1  2020     12 -0.28430 -0.28501 9708946000
#> 7     2     2  2015     11  1.1092   1.1074 4086732000
#> 8     2     2  2016     12  0.37523  0.37395 5592864000
#> 9     2     2  2017     12  0.64073  0.63928 9152528000
#> 10    2     2  2018     12 -0.21969 -0.22046 7103688000
#> 11    2     2  2019     12  0.40819  0.40688 9962680000
#> 12    2     2  2020     12 -0.47547 -0.47599 5194044000
#> 13    3     3  2015     11  0.27011  0.26898 70434234000
#> 14    3     3  2016     12  0.37526  0.37398 96064839000
#> 15    3     3  2017     12  0.38682  0.38558 132546843000
#> 16    3     3  2018     12 -0.17493 -0.17573 107515395000
#> 17    3     3  2019     12  0.65869  0.65717 174487833000
#> 18    3     3  2020     12  0.25382  0.25265 214032195000
#> 19    4     4  2015     11  0.12885  0.12784 14518674000
#> 20    4     4  2016     12 -0.18552 -0.18631 11592135000
#> # i 7,900 more rows

```

3.5.2 年次財務データの読み込み

第2章で作成した `data/derived/fundamentals/financial_characteristics.parquet` を読み込む。

```

financial_data <-
  read_derived_parquet_file_LFL(
    file.path(
      "fundamentals",
      "financial_characteristics.parquet"
    )
  ) |>

```

```
mutate(
  firm_ID = as.integer(firm_ID),
  year = as.integer(year)
)

show_table_LFL(financial_data, n = 10)
#> # A tibble: 7,920 x 25
#>   year fir_ID ind_ID sales    OX    NFE    X    OA    FA    OL
#>   <int> <int> <fct>   <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1  2015     1 1     5261.4 437.49 NA      286.64 13006. 3543.4 4373.0
#> 2  2016     1 1     5949.0 564.14 50.667  513.48 13866. 4642.2 4534.2
#> 3  2017     1 1     6505.1 691.18 29.543  661.64 13953. 7744.0 5111.2
#> 4  2018     1 1     6846.4 751.29 86.487  664.8  18818. 7284.7 5137.3
#> 5  2019     1 1     7572.2 958.53 298.05  660.48 18190  9735.1 5488.0
#> 6  2020     1 1     7537.6 778.37 -65.459 843.83 20463. 10274.  5371.4
#> 7  2015     2 1     3505.8  45.82  5.7511  40.07  2977.8 2258.3 1840.4
#> 8  2016     2 1     3491.3  51.25  1.8765  49.37  3184.8 1881.8 1769.2
#> 9  2017     2 1     3945.7  83.43  7.5279  75.9   3392.2 2425.2 1955.3
#> 10 2018     2 1     4139.3  93.4   6.8166  86.58  3569.3 2331.0 2022.1
#> # i 7,910 more rows
#> # i 15 more variables: FO <dbl>, NOA <dbl>, NFO <dbl>, BE <dbl>, lag_BE <dbl>,
#> #   ROE <dbl>, lag_NOA <dbl>, lag_NFO <dbl>, RNOA <dbl>, PM <dbl>, ATO <dbl>,
#> #   lag_FLEV <dbl>, NBC <dbl>, ROE_DuPon <dbl>, DuP_gap <dbl>
```

財務データの企業・年度キーが一意であることを確認する。

```
financial_key_duplicates <- financial_data |>
  count(
    firm_ID,
    year,
    name = "observations"
  ) |>
  filter(observations > 1)

show_table_LFL(financial_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, year <int>, observations <int>

if (nrow(financial_key_duplicates) > 0) {
  stop(
    "財務データのfirm_IDとyearに重複があります。",
    call. = FALSE
  )
}
```

3.5.3 年次データの結合

```
annual_data <- annual_stock_data |>
  left_join(
    financial_data,
    by = c("firm_ID", "year")
  ) |>
  arrange(firm_ID, year)

stopifnot(
  nrow(annual_data) ==
    nrow(annual_stock_data)
)

annual_data_overview <- annual_data |>
  summarise(
    observations = n(),
    firms = n_distinct(firm_ID),
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    financial_matches =
      sum(!is.na(BE))
  )

show_table_LFL(annual_data_overview)
#> # A tibble: 1 x 5
#>   observations firms fir_year las_year fin_match
#>   <int> <int>    <int>    <int>    <int>
#> 1      7920  1515      2015      2020      7920
```

3.5.4 月次データへの財務情報の付加

```
monthly_data <- stock_data |>
  left_join(
    financial_data,
    by = c("firm_ID", "year")
  ) |>
  arrange(firm_ID, month_ID)

stopifnot(
  nrow(monthly_data) ==
    nrow(stock_data)
```

```

)

row_count_check <- tibble(
  stock_rows = nrow(stock_data),
  monthly_rows = nrow(monthly_data),
  annual_rows = nrow(annual_data)
)

show_table_LFL(row_count_check)
#> # A tibble: 1 x 3
#>   sto_rows mon_rows ann_rows
#>   <int>    <int>    <int>
#> 1   95040   95040    7920

```

3.5.5 売上高・利益・時価総額

最新年度について、売上高、当期純利益および時価総額の関係を確認する。

```

latest_analysis_year <- annual_data |>
  filter(
    is.finite(sales),
    sales > 0,
    is.finite(X),
    X > 0,
    is.finite(ME),
    ME > 0
  ) |>
  summarise(
    latest_year = max(year)
  ) |>
  pull(latest_year)

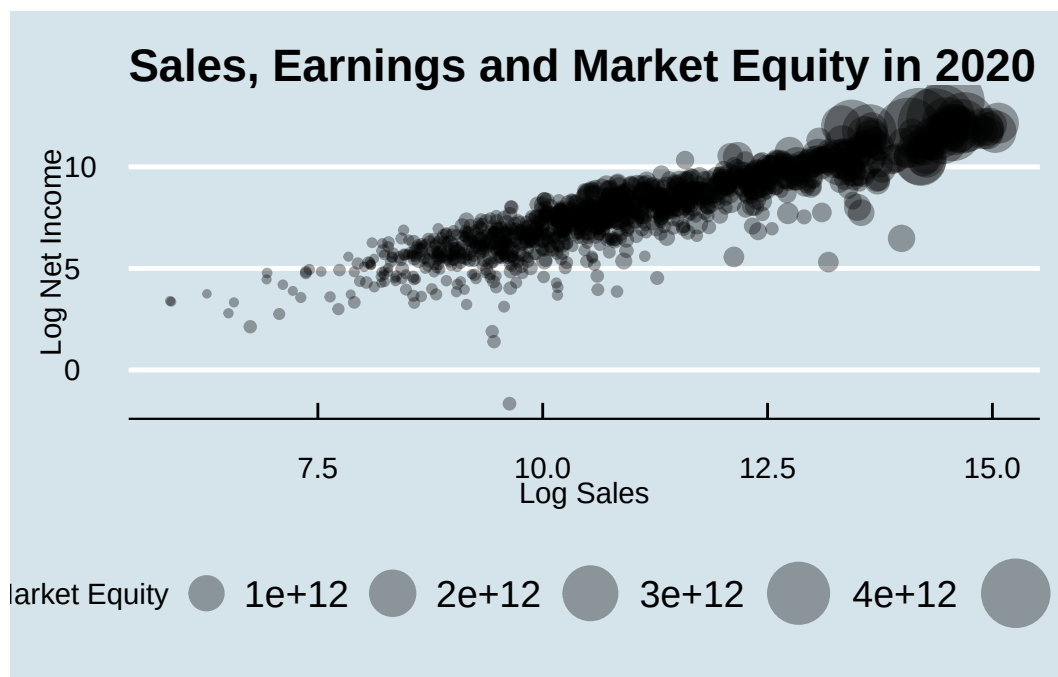
bubble_chart_data <- annual_data |>
  filter(
    year == latest_analysis_year,
    is.finite(sales),
    sales > 0,
    is.finite(X),
    X > 0,
    is.finite(ME),
    ME > 0
  )

```

```

p_sales_earnings_market_equity <- bubble_chart_data |>
  ggplot(
    aes(
      x = log(sales),
      y = log(X),
      size = ME
    )
  ) +
  geom_point(alpha = 0.35) +
  scale_size(
    range = c(1, 12),
    name = "Market Equity"
  ) +
  labs(
    x = "Log Sales",
    y = "Log Net Income",
    title = paste(
      "Sales, Earnings and Market Equity in",
      latest_analysis_year
    )
  )
add_lagrange_theme_LFL(
  p_sales_earnings_market_equity
)

```



3.6 統計分析

3.6.1 月次超過リターンの一標本 t 検定

観測数が最も多い企業を選び、月次超過リターンの平均が 0 と異なるかを検定する。

```
test_firm_id <- stock_data |>
  filter(is.finite(Re)) |>
  count(
    firm_ID,
    sort = TRUE
  ) |>
  slice_head(n = 1) |>
  pull(firm_ID)

test_firm_returns <- stock_data |>
  filter(
    firm_ID == test_firm_id,
    is.finite(Re)
  ) |>
  pull(Re)

manual_t_value <- mean(test_firm_returns) /
  (
    sd(test_firm_returns) /
    sqrt(length(test_firm_returns))
  )

t_test_result <- t.test(
  test_firm_returns,
  mu = 0
)

t_test_summary <- broom::tidy(t_test_result) |>
  mutate(
    firm_ID = test_firm_id,
    manual_t_value = manual_t_value,
    .before = 1
  )

show_table_LFL(t_test_summary)

#> # A tibble: 1 x 10
#>   fir_ID man_value estimate statistic p.value parameter  conf.low conf.high
```



```
#>      <int>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      1      2.1285 0.029161      2.1285 0.036817      70 0.0018367  0.056485
#> # i 2 more variables: method <chr>, alternative <chr>
```

3.6.2 簿価時価比率と年次超過リターン

前年度末の時価総額を作成し、期首株主資本を前年度末時価総額で割って簿価時価比率を計算する。

```
annual_data <- annual_data |>
  group_by(firm_ID) |>
  arrange(year, .by_group = TRUE) |>
  mutate(
    lagged_ME = lag(ME),
    lagged_BEME = lagged_BE / lagged_ME,
    lagged_BEME = if_else(
      is.finite(lagged_BEME) &
        lagged_BEME > 0,
      lagged_BEME,
      NA_real_
    )
  ) |>
  ungroup()
```

回帰に使用できる年度別観測数を確認する。

```
regression_observations_by_year <- annual_data |>
  filter(
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  count(
    year,
    name = "observations"
  ) |>
  arrange(year)

show_table_LFL(
  regression_observations_by_year,
  n = nrow(regression_observations_by_year)
)

#> # A tibble: 5 x 2
#>   year observations
#>   <int>      <int>
#> 1  2016      1241
#> 2  2017      1270
```

```
#> 3 2018      1268
#> 4 2019      1304
#> 5 2020      1322
```

```
eligible_regression_years <-
  regression_observations_by_year |>
  filter(observations >= 30)

if (nrow(eligible_regression_years) == 0) {
  stop(
    "回帰分析に必要な観測数を満たす年度がありません。",
    call. = FALSE
  )
}
```

```
regression_year <- eligible_regression_years |>
  summarise(year = max(year)) |>
  pull(year)
```

```
lm_sample_data <- annual_data |>
  filter(
    year == regression_year,
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  select(
    firm_ID,
    year,
    Re,
    lagged_BEME
  )
```

```
beme_plot_limit <- quantile(
  lm_sample_data$lagged_BEME,
  0.99,
  na.rm = TRUE
)
```

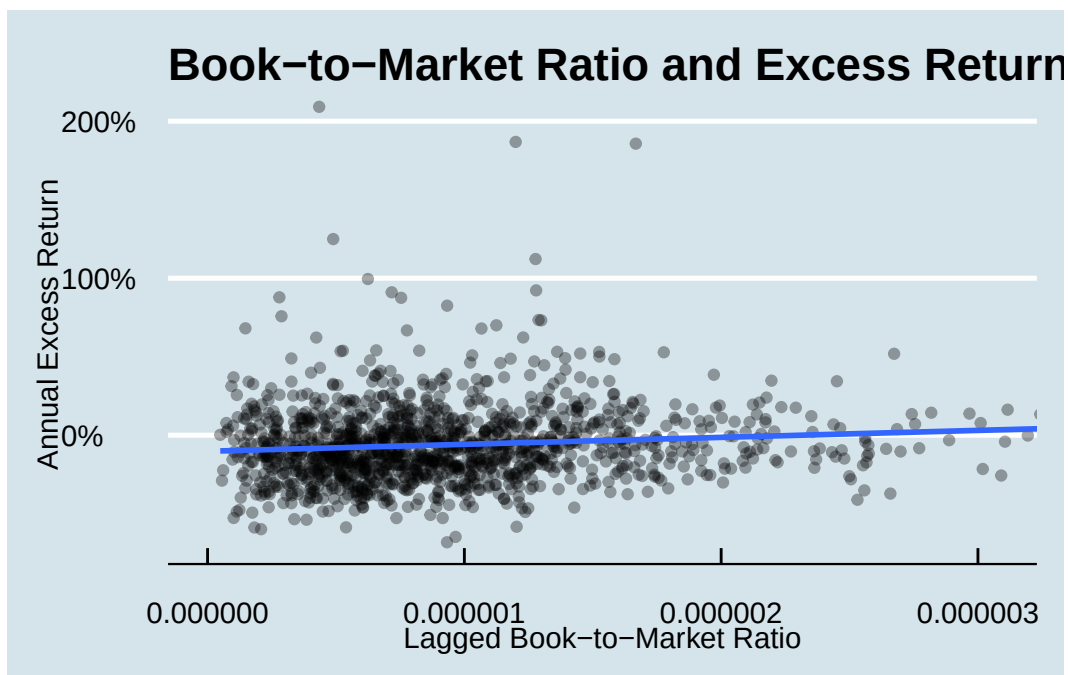
```
p_beme_return_regression <- lm_sample_data |>
  ggplot(
    aes(
      x = lagged_BEME,
      y = Re
    )
  )
```

```

) +
geom_point(alpha = 0.35) +
geom_smooth(
  method = "lm",
  se = FALSE
) +
coord_cartesian(
  xlim = c(0, beme_plot_limit)
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Lagged Book-to-Market Ratio",
  y = "Annual Excess Return",
  title = paste(
    "Book-to-Market Ratio and Excess Return in",
    regression_year
  )
)

add_lagrange_theme_LFL(
  p_beme_return_regression
)

```



```

lm_result <- lm(
  Re ~ lagged_BEME,

```

```

data = lm_sample_data
)

show_table_LFL(
  broom::tidy(lm_result)
)
#> # A tibble: 2 x 5
#>   term          estimate std.error statistic    p.value
#>   <chr>          <dbl>    <dbl>    <dbl>    <dbl>
#> 1 (Intercept)  -0.10237    0.012394  -8.2593 3.5232e-16
#> 2 lagged_BEME 44359.      10973.      4.0426 5.5911e- 5

show_table_LFL(
  broom::glance(lm_result)
)
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared sigma statistic    p.value    df logLik    AIC
#>   <dbl>      <dbl>    <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl>
#> 1  0.012230    0.011481 0.24523    16.343 0.000055911     1 -16.667 39.334
#> # i 4 more variables: BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>

```

3.6.3 年度別回帰

年度ごとの回帰を `nest()` と `purrr::map()` で実行する。

```

yearly_regression_models <- annual_data |>
  filter(
    is.finite(Re),
    is.finite(lagged_BEME)
  ) |>
  group_by(year) |>
  filter(n() >= 30) |>
  ungroup() |>
  nest(data = -year) |>
  mutate(
    model = map(
      data,
      \(data) lm(
        Re ~ lagged_BEME,
        data = data
      )
    ),
    coefficients = map(
      model,

```

```

    broom::tidy
  ),
  fit = map(
    model,
    broom::glance
  )
)

```

```

yearly_regression_coefficients <-
  yearly_regression_models |>
  select(year, coefficients) |>
  unnest(coefficients) |>
  arrange(year, term)

```

```

yearly_regression_fit <-
  yearly_regression_models |>
  select(year, fit) |>
  unnest(fit) |>
  select(
    year,
    r.squared,
    adj.r.squared,
    sigma,
    statistic,
    p.value,
    nobs
  ) |>
  arrange(year)

```

```

show_table_LFL(
  yearly_regression_coefficients,
  n = nrow(yearly_regression_coefficients)
)

```

```
#> # A tibble: 10 x 6
```

#>	year	term	estimate	std.error	statistic	p.value
#>	<int>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	2016	(Intercept)	0.056439	0.016563	3.4076	6.7644e- 4
#> 2	2016	lagged_BEME	72103.	12275.	5.8738	5.4661e- 9
#> 3	2017	(Intercept)	0.17989	0.015673	11.478	4.4094e-29
#> 4	2017	lagged_BEME	32467.	11890.	2.7306	6.4094e- 3
#> 5	2018	(Intercept)	-0.12679	0.015203	-8.3395	1.9259e-16
#> 6	2018	lagged_BEME	65283.	13067.	4.9959	6.6754e- 7
#> 7	2019	(Intercept)	0.28895	0.026406	10.942	1.0121e-26
#> 8	2019	lagged_BEME	100567.	19057.	5.2771	1.5363e- 7

```
#> 9 2020 (Intercept) -0.10237 0.012394 -8.2593 3.5232e-16
#> 10 2020 lagged_BEME 44359. 10973. 4.0426 5.5911e- 5

show_table_LFL(
  yearly_regression_fit,
  n = nrow(yearly_regression_fit)
)
#> # A tibble: 5 x 7
#>   year r.squared adj.r.squared sigma statistic p.value nobs
#>   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
#> 1 2016 0.027092 0.026306 0.28691 34.501 5.4661e-9 1241
#> 2 2017 0.0058459 0.0050619 0.28492 7.4562 6.4094e-3 1270
#> 3 2018 0.019334 0.018559 0.27721 24.959 6.6754e-7 1268
#> 4 2019 0.020940 0.020188 0.49819 27.847 1.5363e-7 1304
#> 5 2020 0.012230 0.011481 0.24523 16.343 5.5911e-5 1322
```

3.7 分析用データの保存

第4章以降で使用するため、月次データを `data/derived/returns/monthly_stock_returns.parquet`、年次データを `data/derived/fundamentals/annual_firm_characteristics.parquet` へ保存する。

```
output_data <- list(
  monthly = monthly_data,
  annual = annual_data
)

output_files <- c(
  monthly = file.path(
    "returns",
    "monthly_stock_returns.parquet"
  ),
  annual = file.path(
    "fundamentals",
    "annual_firm_characteristics.parquet"
  )
)

output_paths <- output_data |>
  imap_chr(
    \(data, data_name)
      write_derived_parquet_file_LFL(
        data,
        output_files[[data_name]]
      )
  )
```

```

    )
  )

tibble::tibble(
  output = base::names(output_paths),
  path = relative_path_LFL(base::unname(output_paths))
) |>
  show_table_LFL()
#> # A tibble: 2 x 2
#>   output path
#>   <chr>  <chr>
#> 1 monthly data/derived/returns/monthly_stock_returns.parquet
#> 2 annual  data/derived/fundamentals/annual_firm_characteristics.parquet

```

保存したファイルを読み直し、行数と列名を検証する。

```

output_verification <- output_data |>
  imap_dfr(
    \(expected_data, data_name) {
      output_check <-
        read_derived_parquet_file_LFL(
          output_files[[data_name]]
        )

      stopifnot(
        nrow(output_check) ==
          nrow(expected_data)
      )

      stopifnot(
        identical(
          names(output_check),
          names(expected_data)
        )
      )

      tibble(
        data = data_name,
        output_file = relative_path_LFL(output_paths[[data_name]]),
        rows = nrow(output_check),
        columns = ncol(output_check),
        size_bytes =
          file.info(
            output_paths[[data_name]]

```

```

    )$size
  )
}
)

show_table_LFL(output_verification)
#> # A tibble: 2 x 5
#>   data      out_file      rows columns siz_bytes
#>   <chr>    <chr>      <int>   <int>    <dbl>
#> 1 monthly data/derived/returns/monthly_stock_returns.pa~ 95040     39  4761273
#> 2 annual  data/derived/fundamentals/annual_firm_charact~  7920     31  1698508

```

3.8 まとめ

本章では、株価、配当および調整係数から月次リターンを作成し、無リスク金利を差し引いて超過リターンを計算した。さらに、月次リターンを年次リターンへ集約し、第 2 章で作成した財務データと結合した。最後に、簿価時価比率と年次超過リターンの関係を回帰分析で確認し、月次データを `data/derived/returns/monthly_stock_returns.parquet`、年次データを `data/derived/fundamentals/annual_firm_characteristics.parquet` として保存した。

第 4 章では、本章で作成した分析用データを利用して、ポートフォリオ分析へ進む。

第 1-4 章ファクターモデル分析

本章では、第 3 章で作成した月次株式データと年次財務データを利用し、市場ポートフォリオ、企業規模ポートフォリオ、簿価時価比率ポートフォリオ、SMB、HML を構築する。さらに、企業規模 10 分位ポートフォリオを対象として、CAPM と Fama-French の 3 ファクター・モデルを推定する。

本章の処理は、次の順序で進める。

1. 第 3 章の月次・年次データを読み込む
2. 前年度末の情報からポートフォリオを形成する
3. 市場ポートフォリオと市場超過リターンを計算する
4. 企業規模と簿価時価比率による単変量ソートを行う
5. 2×3 ポートフォリオから SMB と HML を構築する
6. CAPM と FF3 モデルを推定して比較する
7. 第 5 章以降で使用するファクター・データを保存する

4.1 分析環境

本章では、第 1 章から第 3 章と同じ共通ユーティリティを利用する。ファイル探索、Parquet 読み込み、表の表示、グラフのテーマ設定を各章で重複して定義しない。

4.2 入力データ

4.2.1 第 3 章の出力の読み込み

第 3 章で作成した次の 2 ファイルを読み込む。

```
data/derived/returns/monthly_stock_returns.parquet
data/derived/fundamentals/annual_firm_characteristics.parquet
```

monthly_stock_returns.parquet は月次データ、annual_firm_characteristics.parquet は年次データである。

```
tbl_monthly <-
  read_derived_parquet_file_LFL(
    file.path(
      "returns",
      "monthly_stock_returns.parquet"
    )
  )
```

```

) |>
mutate(
  firm_ID = as.integer(firm_ID),
  year = as.integer(year),
  month = as.integer(month),
  month_ID = as.integer(month_ID),
  month_date = as.Date(month_date)
) |>
arrange(firm_ID, month_ID)

tbl_annual <-
read_derived_parquet_file_LFL(
  file.path(
    "fundamentals",
    "annual_firm_characteristics.parquet"
  )
) |>
mutate(
  firm_ID = as.integer(firm_ID),
  year = as.integer(year)
) |>
arrange(firm_ID, year)

show_data_structure_LFL(tbl_monthly)
#> Rows: 95,040
#> Columns: 39
#> $ year                <int> 2015, 2015, 2015, 2015, 2015, 2015, 2015, 2015, ~
#> $ month                <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, ~
#> $ month_ID             <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, ~
#> $ firm_ID              <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ stock_price           <dbl> 954, 960, 1113, 1081, 1317, 1366, 1353, 1209, 1~
#> $ DPS                  <dbl> 0, 0, 0, 0, 0, 29, 0, 0, 0, 0, 0, 29, 0, 0, 0, ~
#> $ shares_outstanding    <dbl> 2422000, 2422000, 2422000, 2422000, 2422000, 24~
#> $ adjustment_coefficient <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
#> $ month_date            <date> 2015-01-01, 2015-02-01, 2015-03-01, 2015-04-01~
#> $ ME                   <dbl> 2310588000, 2325120000, 2695686000, 2618182000, ~
#> $ lagged_stock_price    <dbl> NA, 954, 960, 1113, 1081, 1317, 1366, 1353, 120~
#> $ lagged_month_ID       <int> NA, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
#> $ consecutive_month     <lgl> NA, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
#> $ R                    <dbl> NA, 0.006289308, 0.159375000, -0.028751123, 0.2~
#> $ R_F                  <dbl> 0.00008202927, 0.00008202927, 0.00008202927, 0.~
#> $ Re                   <dbl> NA, 0.006207279, 0.159292971, -0.028833152, 0.2~
#> $ industry_ID          <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~

```

```

#> $ sales      <dbl> 5261.40, 5261.40, 5261.40, 5261.40, 5261.40, 52~
#> $ OX          <dbl> 437.49, 437.49, 437.49, 437.49, 437.49, 437.49,~
#> $ NFE         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ X           <dbl> 286.64, 286.64, 286.64, 286.64, 286.64, 286.64,~
#> $ OA          <dbl> 13005.55, 13005.55, 13005.55, 13005.55, 13005.5~
#> $ FA          <dbl> 3543.43, 3543.43, 3543.43, 3543.43, 3543.43, 35~
#> $ OL          <dbl> 4372.96, 4372.96, 4372.96, 4372.96, 4372.96, 43~
#> $ FO          <dbl> 2480.72, 2480.72, 2480.72, 2480.72, 2480.72, 24~
#> $ NOA         <dbl> 8632.59, 8632.59, 8632.59, 8632.59, 8632.59, 86~
#> $ NFO         <dbl> -1062.71, -1062.71, -1062.71, -1062.71, -1062.7~
#> $ BE          <dbl> 9695.30, 9695.30, 9695.30, 9695.30, 9695.30, 96~
#> $ lagged_BE   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ ROE         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_NOA  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_NFO  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ RNOA        <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ PM          <dbl> 0.08315087, 0.08315087, 0.08315087, 0.08315087,~
#> $ ATO         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ lagged_FLEV <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ NBC         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ ROE_DuPont  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
#> $ DuPont_gap  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
show_data_structure_LFL(tbl_annual)
#> Rows: 7,920
#> Columns: 31
#> $ firm_ID      <int> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4~
#> $ year         <int> 2015, 2016, 2017, 2018, 2019, 2020, 2015, 2016, 2017, 2018~
#> $ months       <int> 11, 12, 12, 12, 12, 12, 11, 12, 12, 12, 12, 12, 11, 12, 12~
#> $ R            <dbl> 0.61213017, 0.99727265, 0.68786382, -0.21361287, 0.6468357~
#> $ Re           <dbl> 0.61073273, 0.99547052, 0.68636684, -0.21438527, 0.6453315~
#> $ ME           <dbl> 3577294000, 6883324000, 11376990000, 8694752000, 139575180~
#> $ industry_ID <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ sales        <dbl> 5261.40, 5948.96, 6505.06, 6846.38, 7572.24, 7537.63, 3505~
#> $ OX           <dbl> 437.49, 564.14, 691.18, 751.29, 958.53, 778.37, 45.82, 51.~
#> $ NFE          <dbl> NA, 50.667498, 29.543157, 86.486500, 298.049774, -65.45877~
#> $ X            <dbl> 286.64, 513.48, 661.64, 664.80, 660.48, 843.83, 40.07, 49.~
#> $ OA           <dbl> 13005.55, 13865.58, 13952.58, 18818.48, 18190.00, 20462.86~
#> $ FA           <dbl> 3543.43, 4642.16, 7743.99, 7284.72, 9735.13, 10274.25, 225~
#> $ OL           <dbl> 4372.96, 4534.22, 5111.22, 5137.28, 5487.96, 5371.38, 1840~
#> $ FO           <dbl> 2480.72, 3959.70, 6159.02, 10123.91, 11362.22, 13772.15, 2~
#> $ NOA          <dbl> 8632.59, 9331.36, 8841.36, 13681.20, 12702.04, 15091.48, 1~
#> $ NFO          <dbl> -1062.71, -682.46, -1584.97, 2839.19, 1627.09, 3497.90, 82~
#> $ BE          <dbl> 9695.30, 10013.82, 10426.33, 10842.01, 11074.95, 11593.58,~

```

```
#> $ lagged_BE      <dbl> NA, 9695.30, 10013.82, 10426.33, 10842.01, 11074.95, NA, 1~
#> $ ROE            <dbl> NA, 0.05296174, 0.06607269, 0.06376165, 0.06091859, 0.0761~
#> $ lagged_NOA     <dbl> NA, 8632.59, 9331.36, 8841.36, 13681.20, 12702.04, NA, 113~
#> $ lagged_NFO     <dbl> NA, -1062.71, -682.46, -1584.97, 2839.19, 1627.09, NA, 82.~
#> $ RNOA           <dbl> NA, 0.06535003, 0.07407066, 0.08497448, 0.07006184, 0.0612~
#> $ PM             <dbl> 0.083150872, 0.094830021, 0.106252671, 0.109735364, 0.1265~
#> $ ATO            <dbl> NA, 0.6891281, 0.6971181, 0.7743582, 0.5534778, 0.5934189,~
#> $ lagged_FLEV    <dbl> NA, -0.10961084, -0.06815181, -0.15201610, 0.26186934, 0.1~
#> $ NBC            <dbl> NA, -0.047677633, -0.043289214, -0.054566648, 0.104977044,~
#> $ ROE_DuPont     <dbl> NA, 0.05296097, 0.06607237, 0.06376199, 0.06091861, 0.0761~
#> $ DuPont_gap     <dbl> NA, 0.00000077332412, 0.00000031526680, -0.00000033567247,~
#> $ lagged_ME      <dbl> NA, 3577294000, 6883324000, 11376990000, 8694752000, 13957~
#> $ lagged_BEME    <dbl> NA, 0.0000027102329, 0.0000014547942, 0.0000009164401, 0.0~
```

```
tbl_input_overview <- tribble(
  ~data, ~observations, ~variables, ~firms, ~first_year, ~last_year,
  "monthly", nrow(tbl_monthly), ncol(tbl_monthly),
  n_distinct(tbl_monthly$firm_ID),
  min(tbl_monthly$year, na.rm = TRUE),
  max(tbl_monthly$year, na.rm = TRUE),
  "annual", nrow(tbl_annual), ncol(tbl_annual),
  n_distinct(tbl_annual$firm_ID),
  min(tbl_annual$year, na.rm = TRUE),
  max(tbl_annual$year, na.rm = TRUE)
)
```

```
show_table_LFL(tbl_input_overview)
```

```
#> # A tibble: 2 x 6
#>   data      observations variables firms fir_year las_year
#>   <chr>          <int>      <int> <int>    <int>    <int>
#> 1 monthly        95040         39  1515    2015    2020
#> 2 annual         7920         31  1515    2015    2020
```

4.2.2 必須列とパネルキー

分析を開始する前に、必要な列とパネルキーの一意性を確認する。

```
vec_required_monthly_columns <- c(
  "year", "month", "month_ID", "month_date",
  "firm_ID", "R", "R_F", "Re", "ME"
)
```

```
vec_required_annual_columns <- c(
  "year", "firm_ID", "ME", "BE", "lagged_BE"
```

```
)

vec_missing_monthly_columns <- setdiff(
  vec_required_monthly_columns,
  names(tbl_monthly)
)

vec_missing_annual_columns <- setdiff(
  vec_required_annual_columns,
  names(tbl_annual)
)

if (length(vec_missing_monthly_columns) > 0) {
  stop(
    "月次データに必要な列が不足しています: ",
    paste(vec_missing_monthly_columns, collapse = ", ")
  )
}

if (length(vec_missing_annual_columns) > 0) {
  stop(
    "年次データに必要な列が不足しています: ",
    paste(vec_missing_annual_columns, collapse = ", ")
  )
}

tbl_monthly_key_duplicates <- tbl_monthly |>
  count(firm_ID, month_ID, name = "observations") |>
  filter(observations > 1)

tbl_annual_key_duplicates <- tbl_annual |>
  count(firm_ID, year, name = "observations") |>
  filter(observations > 1)

if (nrow(tbl_monthly_key_duplicates) > 0) {
  stop("月次データのfirm_IDとmonth_IDの組合せに重複があります。")
}

if (nrow(tbl_annual_key_duplicates) > 0) {
  stop("年次データのfirm_IDとyearの組合せに重複があります。")
}

show_table_LFL(tbl_monthly_key_duplicates)
```

```
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, mon_ID <int>, observations <int>
show_table_LFL(tbl_annual_key_duplicates)
#> # A tibble: 0 x 3
#> # i 3 variables: fir_ID <int>, year <int>, observations <int>
```

4.2.3 分析用補助関数

次の関数は、欠損値や非有限値を除外して要約統計量と加重平均を計算する。

```
compute_median_safe <- function(x) {
  values <- x[is.finite(x)]
  if (length(values) == 0) return(NA_real_)
  median(values)
}

compute_mean_safe <- function(x) {
  values <- x[is.finite(x)]
  if (length(values) == 0) return(NA_real_)
  mean(values)
}

compute_weighted_mean_safe <- function(x, weight) {
  valid <- is.finite(x) & is.finite(weight) & weight > 0
  if (!any(valid)) return(NA_real_)
  sum(x[valid] * weight[valid]) / sum(weight[valid])
}

compute_significance_stars <- function(p_value) {
  case_when(
    is.na(p_value) ~ "",
    p_value < 0.01 ~ "***",
    p_value < 0.05 ~ "**",
    p_value < 0.10 ~ "*",
    TRUE ~ ""
  )
}
```

4.3 ポートフォリオ形成と市場リターン

4.3.1 前年度情報の準備

年度 t の月次リターンに対して、年度 $t-1$ 末に観測された時価総額と株主資本を使用する。これにより、将来情報をポートフォリオ形成に利用することを防ぐ。

```

if (!"lagged_ME" %in% names(tbl_annual)) {
  tbl_annual <- tbl_annual |>
    group_by(firm_ID) |>
    arrange(year, .by_group = TRUE) |>
    mutate(lagged_ME = lag(ME)) |>
    ungroup()
}

if (!"lagged_BEME" %in% names(tbl_annual)) {
  tbl_annual <- tbl_annual |>
    mutate(lagged_BEME = lagged_BE / lagged_ME)
}

tbl_annual <- tbl_annual |>
  mutate(
    lagged_ME = if_else(
      is.finite(lagged_ME) & lagged_ME > 0,
      lagged_ME,
      NA_real_
    ),
    lagged_BEME = if_else(
      is.finite(lagged_BEME) & lagged_BEME > 0,
      lagged_BEME,
      NA_real_
    )
  ) |>
  arrange(firm_ID, year)

tbl_lagged_variable_availability <- tbl_annual |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    valid_lagged_ME = sum(is.finite(lagged_ME)),
    valid_lagged_BEME = sum(is.finite(lagged_BEME)),
    valid_for_2x3 = sum(
      is.finite(lagged_ME) & is.finite(lagged_BEME)
    ),
    .groups = "drop"
  )

show_table_LFL(tbl_lagged_variable_availability)
#> # A tibble: 6 x 5
#>   year firms val_ME val_BEME val_2x3

```

```
#>   <int> <int> <int>   <int>   <int>
#> 1  2015 1266     0       0       0
#> 2  2016 1293 1241    1241    1241
#> 3  2017 1319 1270    1270    1270
#> 4  2018 1323 1268    1268    1268
#> 5  2019 1356 1304    1304    1304
#> 6  2020 1363 1322    1322    1322
```

4.3.2 市場ポートフォリオ

年度 t の企業 i の市場ポートフォリオ・ウェイトは、前年度末時価総額から計算する。

$$w_{i,t}^M = \frac{ME_{i,t-1}}{\sum_j ME_{j,t-1}}$$

```
tbl_market_membership <- tbl_annual |>
  filter(is.finite(lagged_ME), lagged_ME > 0) |>
  group_by(year) |>
  mutate(market_weight = lagged_ME / sum(lagged_ME)) |>
  ungroup() |>
  select(year, firm_ID, lagged_ME, market_weight)

tbl_market_weight_check <- tbl_market_membership |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    weight_sum = sum(market_weight),
    minimum_weight = min(market_weight),
    maximum_weight = max(market_weight),
    .groups = "drop"
  )

if (any(abs(tbl_market_weight_check$weight_sum - 1) > 1e-10)) {
  stop("市場ポートフォリオの年度別ウェイト合計が1ではありません。")
}

show_table_LFL(tbl_market_weight_check)
#> # A tibble: 5 x 5
#>   year firms wei_sum   min_weigh max_weigh
#>   <int> <int>   <dbl>     <dbl>   <dbl>
#> 1  2016 1241     1 0.0000019019  0.040134
#> 2  2017 1270     1 0.0000015083  0.032973
#> 3  2018 1268     1 0.0000012609  0.042639
#> 4  2019 1304     1 0.0000021887  0.039396
```



```
#> 5 2020 1322 1 0.00000098548 0.056868
```

市場ポートフォリオの月次リターンは、前年度末時価総額による加重平均である。

$$R_{M,t} = \sum_i w_{i,t}^M R_{i,t}$$

```
tbl_market_returns <- tbl_monthly |>
  inner_join(
    tbl_market_membership,
    by = c("year", "firm_ID")
  ) |>
  group_by(month_ID, year, month, month_date) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_M = compute_weighted_mean_safe(R, lagged_ME),
    eligible_firms = n_distinct(firm_ID),
    valid_return_firms = sum(
      is.finite(R) & is.finite(lagged_ME) & lagged_ME > 0
    ),
    .groups = "drop"
  ) |>
  mutate(
    R_Me = R_M - R_F,
    return_coverage = valid_return_firms / eligible_firms
  ) |>
  arrange(month_ID)

tbl_invalid_market_months <- tbl_market_returns |>
  filter(!is.finite(R_F) | !is.finite(R_M) | !is.finite(R_Me))

if (nrow(tbl_invalid_market_months) > 0) {
  stop("市場リターンを作成できない月があります。")
}

show_table_LFL(tbl_market_returns)
#> # A tibble: 60 x 10
#>   mon_ID year month mon_date      R_F      R_M eli_firms val_firms
#>   <int> <int> <int> <date>      <dbl>      <dbl>      <int>      <int>
#> 1     13 2016     1 2016-01-01 0.000082029 0.045835      1241      1241
#> 2     14 2016     2 2016-02-01 0.000075959 0.028423      1241      1241
#> 3     15 2016     3 2016-03-01 0.000075959 -0.057976      1241      1241
#> 4     16 2016     4 2016-04-01 0.000082029 -0.0010718      1241      1241
#> 5     17 2016     5 2016-05-01 0.000082029 -0.0064160      1241      1241
#> 6     18 2016     6 2016-06-01 0.000082029 -0.037075      1241      1241
```

```
#> 7      19 2016      7 2016-07-01 0.000082029 0.029853      1241      1241
#> 8      20 2016      8 2016-08-01 0.000082029 0.053757      1241      1241
#> 9      21 2016      9 2016-09-01 0.000082029 0.023650      1241      1241
#> 10     22 2016     10 2016-10-01 0.000075959 -0.012053      1241      1241
#> # i 50 more rows
#> # i 2 more variables: R_Me <dbl>, ret_cover <dbl>
```

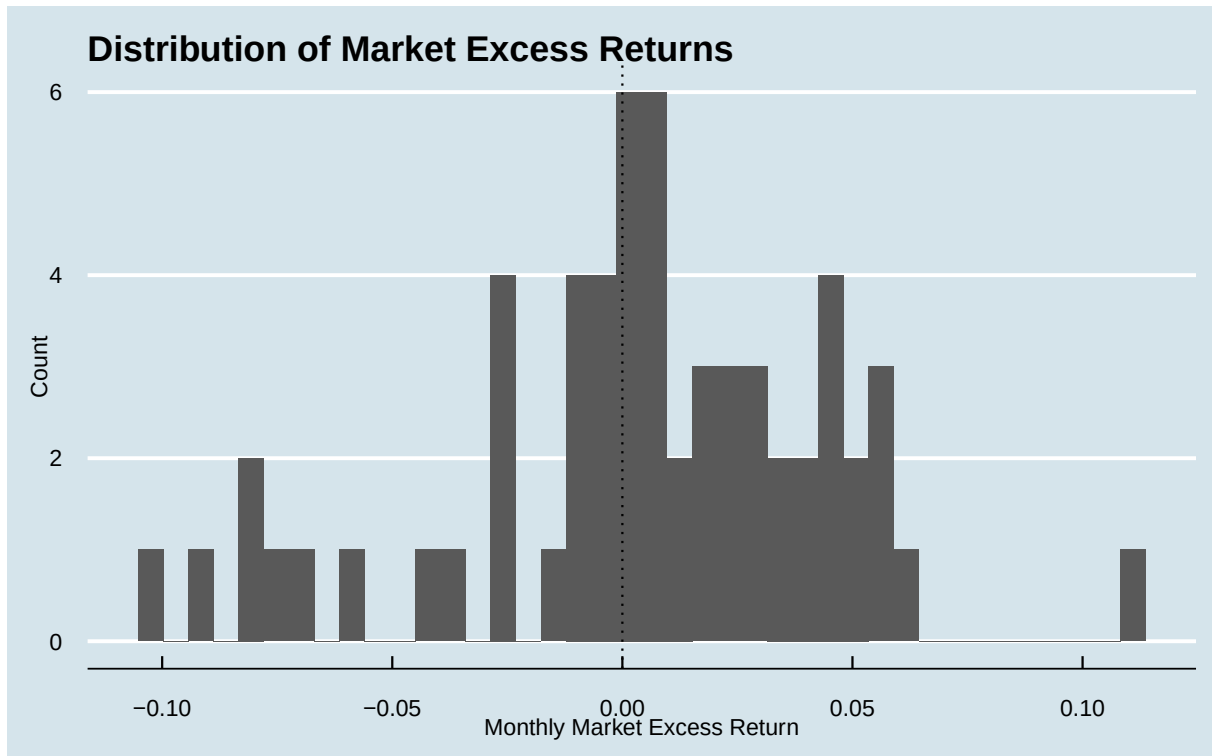
```
tbl_market_return_summary <- tbl_market_returns |>
  summarise(
    observations = n(),
    mean_market_return = mean(R_M),
    vec_mean_market_excess_return = mean(R_Me),
    market_volatility = sd(R_M),
    minimum_market_return = min(R_M),
    maximum_market_return = max(R_M),
    minimum_coverage = min(return_coverage)
  )
```

```
show_table_LFL(tbl_market_return_summary)
```

```
#> # A tibble: 1 x 7
#>   observations mea_retur vec_retur mar_volat min_retur max_retur min_cover
#>   <int>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1         60 0.0041003 0.0040215 0.041498 -0.10244 0.11104      1
```

```
plot_market_excess_return <- tbl_market_returns |>
  ggplot(aes(x = R_Me)) +
  geom_histogram(bins = 40) +
  geom_vline(xintercept = 0, linetype = "dotted") +
  labs(
    x = "Monthly Market Excess Return",
    y = "Count",
    title = "Distribution of Market Excess Returns"
  )
```

```
show_plot_LFL(add_lagrange_theme_LFL(plot_market_excess_return))
```

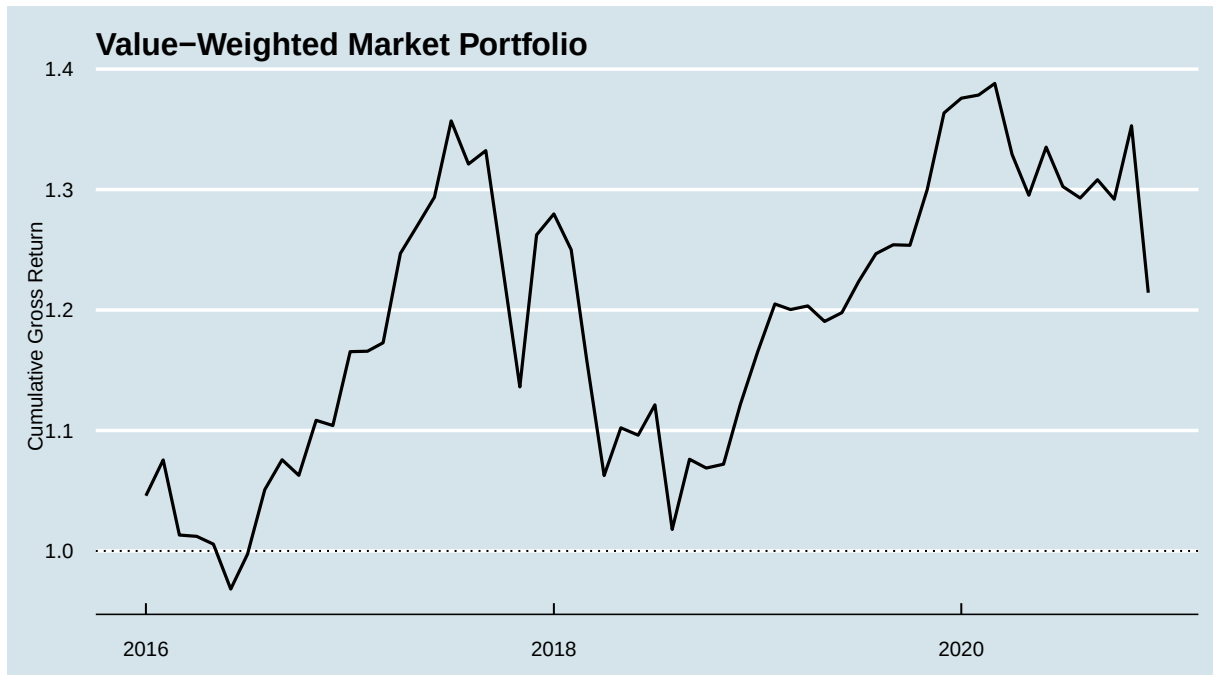


```

plot_cumulative_market_return <- tbl_market_returns |>
  arrange(month_ID) |>
  mutate(cumulative_gross_market_return = cumprod(1 + R_M)) |>
  ggplot(aes(x = month_date, y = cumulative_gross_market_return)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 1, linetype = "dotted") +
  labs(
    x = NULL,
    y = "Cumulative Gross Return",
    title = "Value-Weighted Market Portfolio"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_cumulative_market_return))

```



4.3.3 ポートフォリオ形成ユニバース

企業規模と簿価時価比率の両方を利用できる企業を共通の形成ユニバースとする。

```
tbl_formation_universe <- tbl_annual |>
  filter(
    is.finite(lagged_ME),
    lagged_ME > 0,
    is.finite(lagged_BEME),
    lagged_BEME > 0
  ) |>
  select(year, firm_ID, lagged_ME, lagged_BEME)

tbl_formation_breakpoints <- tbl_formation_universe |>
  group_by(year) |>
  summarise(
    firms = n_distinct(firm_ID),
    size_median = median(lagged_ME),
    BEME_30 = quantile(lagged_BEME, probs = 0.30, names = FALSE),
    BEME_70 = quantile(lagged_BEME, probs = 0.70, names = FALSE),
    .groups = "drop"
  )

tbl_formation <- tbl_formation_universe |>
  left_join(tbl_formation_breakpoints, by = "year") |>
  group_by(year) |>
  mutate(
```

```

    ME_rank10 = ntile(lagged_ME, 10),
    BEME_rank10 = ntile(lagged_BEME, 10)
  ) |>
  ungroup() |>
  mutate(
    size_group = if_else(lagged_ME <= size_median, "S", "B"),
    BEME_group = case_when(
      lagged_BEME <= BEME_30 ~ "L",
      lagged_BEME <= BEME_70 ~ "N",
      TRUE ~ "H"
    ),
    FF_portfolio_type = factor(
      paste0(size_group, BEME_group),
      levels = c("SL", "SN", "SH", "BL", "BN", "BH")
    )
  ) |>
  select(
    year, firm_ID, lagged_ME, lagged_BEME,
    ME_rank10, BEME_rank10,
    size_group, BEME_group, FF_portfolio_type
  )

tbl_portfolio_membership_counts <- tbl_formation |>
  count(year, FF_portfolio_type, name = "firms") |>
  complete(
    year,
    FF_portfolio_type,
    fill = list(firms = 0)
  ) |>
  arrange(year, FF_portfolio_type)

tbl_empty_portfolios <- tbl_portfolio_membership_counts |>
  filter(firms == 0)

if (nrow(tbl_empty_portfolios) > 0) {
  stop("企業が割り当てられていない2×3ポートフォリオがあります。")
}

show_table_LFL(tbl_formation_breakpoints)
#> # A tibble: 5 x 5
#>   year firms  siz_media      BEM_30      BEM_70
#>   <int> <int>      <dbl>      <dbl>      <dbl>
#> 1  2016  1241 24564654000 0.00000076171 0.0000014304

```

```
#> 2  2017  1270 25133528500 0.00000070949 0.0000013738
#> 3  2018  1268 28268839000 0.00000061518 0.0000012038
#> 4  2019  1304 23853596500 0.00000073364 0.0000014045
#> 5  2020  1322 31878381000 0.00000057440 0.0000011304
show_table_LFL(tbl_portfolio_membership_counts)
#> # A tibble: 30 x 3
#>   year FF_type firms
#>   <int> <fct>   <int>
#> 1  2016 SL      140
#> 2  2016 SN      210
#> 3  2016 SH      271
#> 4  2016 BL      233
#> 5  2016 BN      286
#> 6  2016 BH      101
#> 7  2017 SL      141
#> 8  2017 SN      224
#> 9  2017 SH      270
#> 10 2017 BL      240
#> # i 20 more rows
```

4.4 単変量ポートフォリオ・ソート

4.4.1 企業規模 10 分位ポートフォリオ

企業規模 10 分位ごとに、等加重リターンと前年度末時価総額加重リターンを計算する。

```
tbl_size_decile_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(year, firm_ID, lagged_ME, ME_rank10),
    by = c("year", "firm_ID")
  ) |>
  group_by(
    month_ID, year, month, month_date, ME_rank10
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_equal = compute_mean_safe(R),
    R_value = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(
```

```

    Re_equal = R_equal - R_F,
    Re_value = R_value - R_F
  ) |>
  arrange(month_ID, ME_rank10)

tbl_size_decile_coverage <- tbl_size_decile_returns |>
  group_by(month_ID) |>
  summarise(
    portfolios = n_distinct(ME_rank10),
    .groups = "drop"
  ) |>
  filter(portfolios != 10)

if (nrow(tbl_size_decile_coverage) > 0) {
  stop("10個の企業規模ポートフォリオがそろわない月があります。")
}

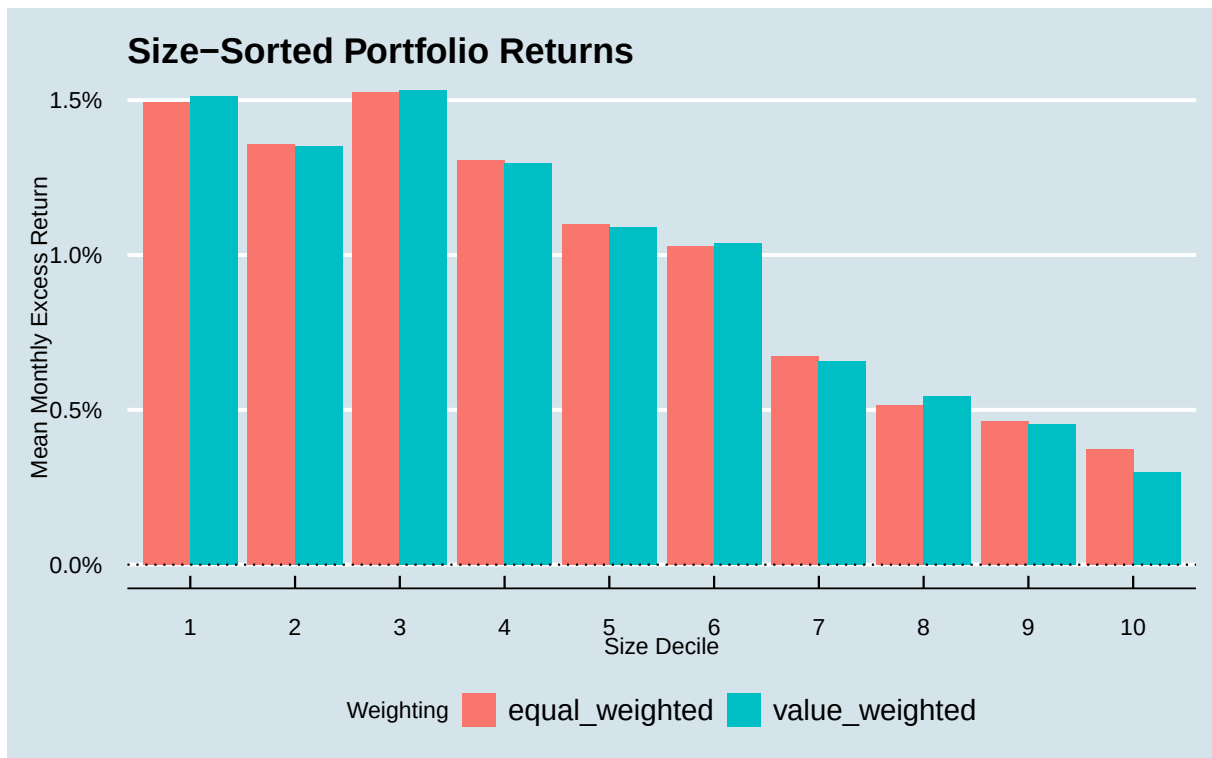
tbl_size_decile_summary <- tbl_size_decile_returns |>
  group_by(ME_rank10) |>
  summarise(
    months = n(),
    mean_equal_excess_return = mean(Re_equal, na.rm = TRUE),
    mean_value_excess_return = mean(Re_value, na.rm = TRUE),
    value_return_volatility = sd(Re_value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(ME_rank10)

show_table_LFL(tbl_size_decile_summary)
#> # A tibble: 10 x 5
#>   ME_rank1 months mea_retur mea_retur val_volat
#>   <int>   <int>     <dbl>     <dbl>     <dbl>
#> 1     1     60 0.014931  0.015119  0.042085
#> 2     2     60 0.013581  0.013512  0.040856
#> 3     3     60 0.015247  0.015316  0.040102
#> 4     4     60 0.013073  0.012958  0.041626
#> 5     5     60 0.010989  0.010910  0.041165
#> 6     6     60 0.010277  0.010388  0.040414
#> 7     7     60 0.0067439 0.0065837  0.041338
#> 8     8     60 0.0051647 0.0054430  0.041428
#> 9     9     60 0.0046434 0.0045278  0.044503
#> 10    10     60 0.0037215 0.0029811  0.042314

```

```
plot_size_decile <- tbl_size_decile_summary |>
  select(
    ME_rank10,
    equal_weighted = mean_equal_excess_return,
    value_weighted = mean_value_excess_return
  ) |>
  pivot_longer(
    cols = c(equal_weighted, value_weighted),
    names_to = "weighting",
    values_to = "mean_excess_return"
  ) |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = mean_excess_return,
      fill = weighting
    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "Mean Monthly Excess Return",
    fill = "Weighting",
    title = "Size-Sorted Portfolio Returns"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_size_decile))
```

4.4.2 簿価時価比率 10 分位ポートフォリオ

```
tbl_beme_decile_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(year, firm_ID, lagged_ME, BEME_rank10),
    by = c("year", "firm_ID")
  ) |>
  group_by(
    month_ID, year, month, month_date, BEME_rank10
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R_equal = compute_mean_safe(R),
    R_value = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(
    Re_equal = R_equal - R_F,
    Re_value = R_value - R_F
  ) |>
  arrange(month_ID, BEME_rank10)
```

```
tbl_beme_decile_summary <- tbl_beme_decile_returns |>
  group_by(BEME_rank10) |>
  summarise(
    months = n(),
    mean_equal_excess_return = mean(Re_equal, na.rm = TRUE),
    mean_value_excess_return = mean(Re_value, na.rm = TRUE),
    value_return_volatility = sd(Re_value, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(BEME_rank10)

show_table_LFL(tbl_beme_decile_summary)
#> # A tibble: 10 x 5
#>   BEM_rank1 months mea_retur  mea_retur val_volat
#>   <int>    <int>      <dbl>      <dbl>    <dbl>
#> 1     1      60 0.0069080 -0.0012722 0.052048
#> 2     2      60 0.0057059 0.0024925 0.039802
#> 3     3      60 0.0043601 0.0021706 0.045255
#> 4     4      60 0.0090970 0.0034693 0.045192
#> 5     5      60 0.0099654 0.0085384 0.047651
#> 6     6      60 0.010433 0.0055909 0.042627
#> 7     7      60 0.0086709 0.0085607 0.044756
#> 8     8      60 0.012979 0.0088343 0.044844
#> 9     9      60 0.016017 0.014657 0.049615
#> 10    10      60 0.014389 0.0097273 0.043590

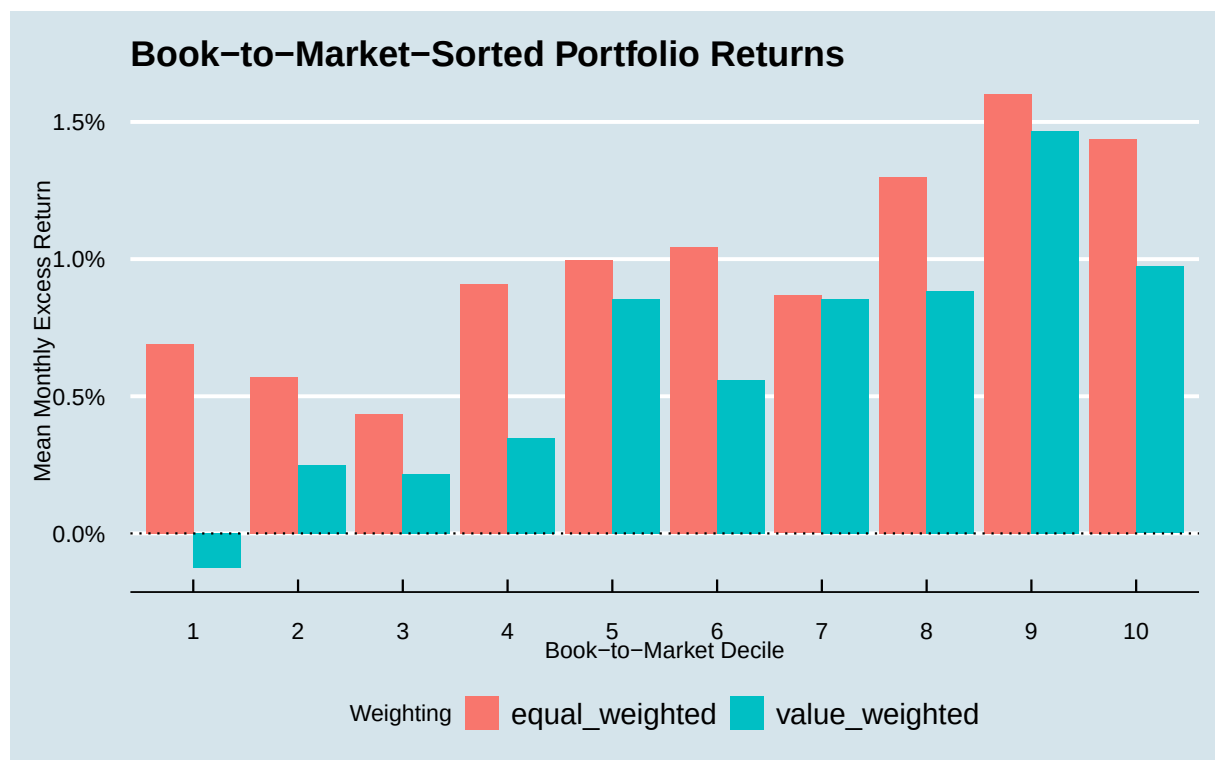
plot_beme_decile <- tbl_beme_decile_summary |>
  select(
    BEME_rank10,
    equal_weighted = mean_equal_excess_return,
    value_weighted = mean_value_excess_return
  ) |>
  pivot_longer(
    cols = c(equal_weighted, value_weighted),
    names_to = "weighting",
    values_to = "mean_excess_return"
  ) |>
  mutate(BEME_rank10 = factor(BEME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = BEME_rank10,
      y = mean_excess_return,
      fill = weighting
    )
  )
```

```

    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Book-to-Market Decile",
    y = "Mean Monthly Excess Return",
    fill = "Weighting",
    title = "Book-to-Market-Sorted Portfolio Returns"
  )
)

show_plot_LFL(add_lagrange_theme_LFL(plot_beme_decile))

```



4.5 CAPM

4.5.1 モデルの推定

CAPM では、ポートフォリオ p の超過リターンを市場超過リターンで説明する。

$$R_{p,t}^e = \alpha_p + \beta_p R_{M,t}^e + \varepsilon_{p,t}$$

```

tbl_capm_data <- tbl_size_decile_returns |>
  select(month_ID, ME_rank10, Re = Re_value) |>
  inner_join(

```

```
tbl_market_returns |>
  select(month_ID, R_Me),
  by = "month_ID"
) |>
filter(is.finite(Re), is.finite(R_Me)) |>
arrange(ME_rank10, month_ID)

tbl_capm_models <- tbl_capm_data |>
  group_by(ME_rank10) |>
  nest() |>
  mutate(
    model = map(data, \(data) lm(Re ~ R_Me, data = data)),
    coefficients = map(model, broom::tidy),
    fit = map(model, broom::glance)
  )

tbl_capm_results <- tbl_capm_models |>
  select(ME_rank10, coefficients) |>
  unnest(coefficients) |>
  mutate(significance = compute_significance_stars(p.value)) |>
  arrange(ME_rank10, term)

tbl_capm_fit <- tbl_capm_models |>
  select(ME_rank10, fit) |>
  unnest(fit) |>
  select(
    ME_rank10, r.squared, adj.r.squared,
    sigma, statistic, p.value, nobs
  ) |>
  arrange(ME_rank10)

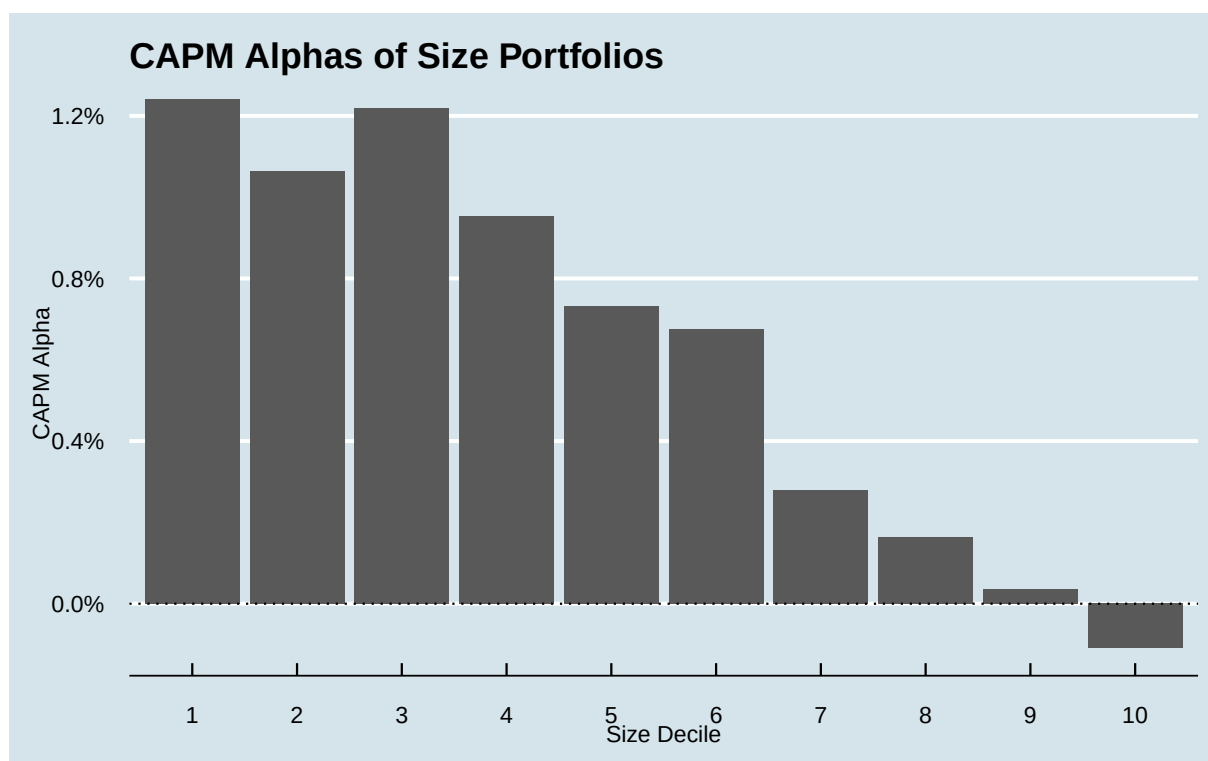
tbl_capm_alpha <- tbl_capm_results |>
  filter(term == "(Intercept)") |>
  transmute(
    ME_rank10,
    tbl_capm_alpha = estimate,
    standard_error = std.error,
    statistic,
    p_value = p.value,
    significance
  )

show_table_LFL(tbl_capm_results)
```

```
#> # A tibble: 20 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 term          estimate std.error statistic    p.value significance
#>   <int> <chr>          <dbl>    <dbl>    <dbl>    <dbl> <chr>
#> 1     1 1 (Intercept) 0.012412 0.0041175 3.0144 3.8151e- 3 ***
#> 2     2 1 R_Me      0.67325 0.099584 6.7607 7.3884e- 9 ***
#> 3     3 2 (Intercept) 0.010641 0.0036808 2.8910 5.3962e- 3 ***
#> 4     4 2 R_Me      0.71390 0.089021 8.0195 5.6813e-11 ***
#> 5     5 3 (Intercept) 0.012195 0.0031256 3.9016 2.5116e- 4 ***
#> 6     6 3 R_Me      0.77614 0.075595 10.267 1.1583e-14 ***
#> 7     7 4 (Intercept) 0.0095370 0.0028845 3.3062 1.6262e- 3 ***
#> 8     8 4 R_Me      0.85081 0.069764 12.196 1.2013e-17 ***
#> 9     9 5 (Intercept) 0.0073101 0.0023212 3.1493 2.5865e- 3 ***
#> 10    10 5 R_Me      0.89511 0.056138 15.945 7.2596e-23 ***
#> # i 10 more rows
show_table_LFL(tbl_capm_fit)
#> # A tibble: 10 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 r.squared adj.r.squared    sigma statistic    p.value  nobs
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl> <int>
#> 1     1 0.44073 0.43109 0.031743 45.706 7.3884e- 9 60
#> 2     2 0.52580 0.51763 0.028376 64.312 5.6813e-11 60
#> 3     3 0.64507 0.63895 0.024096 105.41 1.1583e-14 60
#> 4     4 0.71944 0.71461 0.022238 148.73 1.2013e-17 60
#> 5     5 0.81424 0.81104 0.017894 254.23 7.2596e-23 60
#> 6     6 0.86156 0.85918 0.015166 360.96 1.3994e-26 60
#> 7     7 0.89634 0.89456 0.013423 501.54 3.1166e-30 60
#> 8     8 0.90073 0.89902 0.013165 526.25 8.8761e-31 60
#> 9     9 0.93827 0.93721 0.011152 881.65 9.0177e-37 60
#> 10    10 0.98847 0.98827 0.0045827 4972.0 6.5289e-58 60
```

```
plot_capm_alpha <- tbl_capm_alpha |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(aes(x = ME_rank10, y = tbl_capm_alpha)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "CAPM Alpha",
    title = "CAPM Alphas of Size Portfolios"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_capm_alpha))
```



4.5.2 証券市場線

```
tbl_capm_beta <- tbl_capm_results |>
  filter(term == "R_Me") |>
  transmute(
    ME_rank10,
    tbl_capm_beta = estimate,
    beta_p_value = p.value
  )

tbl_security_market_line <- tbl_size_decile_summary |>
  select(
    ME_rank10,
    mean_excess_return = mean_value_excess_return
  ) |>
  left_join(tbl_capm_beta, by = "ME_rank10")

vec_mean_market_excess_return <- mean(
  tbl_market_returns$R_Me,
  na.rm = TRUE
)

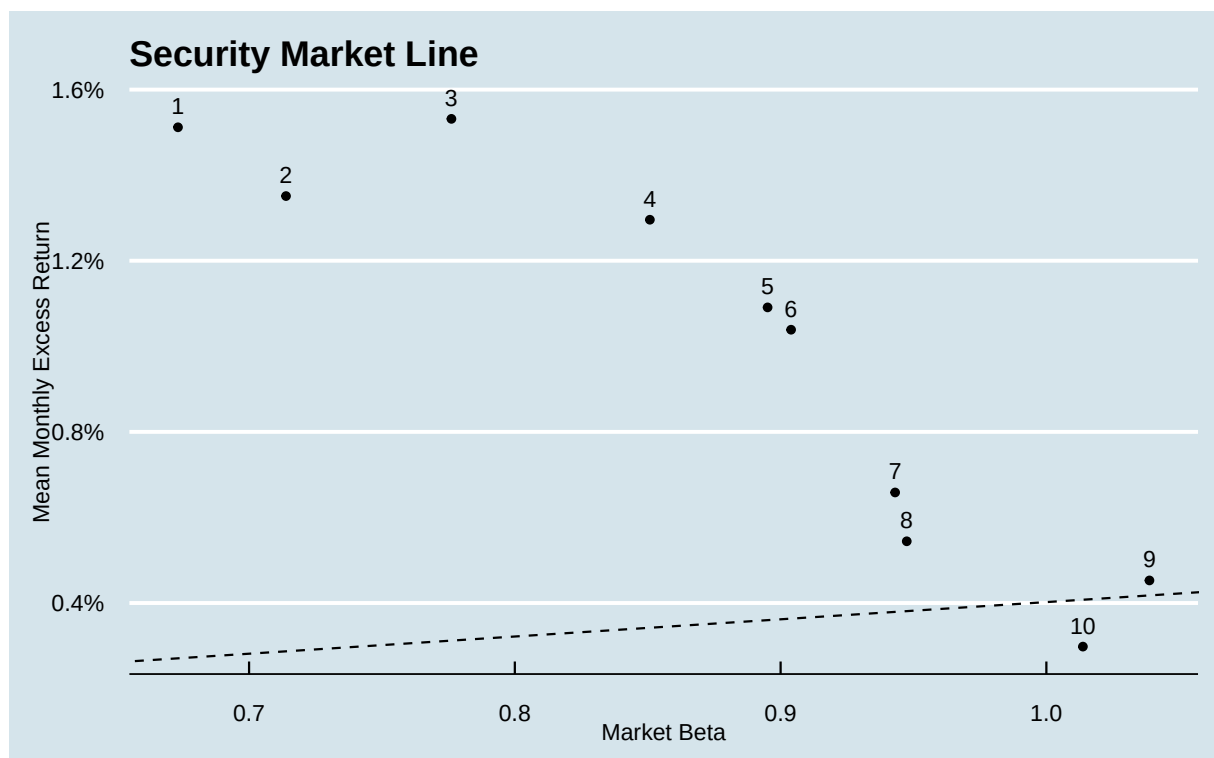
plot_security_market_line <- tbl_security_market_line |>
  ggplot(
```

```

aes(
  x = tbl_capm_beta,
  y = mean_excess_return
) +
geom_point() +
geom_text(aes(label = ME_rank10), nudge_y = 0.0005) +
geom_abline(
  intercept = 0,
  slope = vec_mean_market_excess_return,
  linetype = "dashed"
) +
scale_y_continuous(labels = scales::label_percent()) +
labs(
  x = "Market Beta",
  y = "Mean Monthly Excess Return",
  title = "Security Market Line"
)

show_plot_LFL(add_lagrange_theme_LFL(plot_security_market_line))

```



4.6 2×3 ポートフォリオと SMB・HML

4.6.1 Size-BE/ME ポートフォリオ

企業規模を Small と Big の 2 群、簿価時価比率を Low、Neutral、High の 3 群に分ける。

ポートフォリオ	企業規模	簿価時価比率
SL	Small	Low
SN	Small	Neutral
SH	Small	High
BL	Big	Low
BN	Big	Neutral
BH	Big	High

```
tbl_ff_portfolio_returns <- tbl_monthly |>
  inner_join(
    tbl_formation |>
      select(
        year, firm_ID, lagged_ME,
        size_group, BEME_group, FF_portfolio_type
      ),
    by = c("year", "firm_ID")
  ) |>
  filter(!is.na(FF_portfolio_type)) |>
  group_by(
    month_ID, year, month, month_date,
    FF_portfolio_type, size_group, BEME_group
  ) |>
  summarise(
    R_F = compute_median_safe(R_F),
    R = compute_weighted_mean_safe(R, lagged_ME),
    firms = n_distinct(firm_ID),
    valid_return_firms = sum(is.finite(R)),
    .groups = "drop"
  ) |>
  mutate(Re = R - R_F) |>
  arrange(month_ID, FF_portfolio_type)

tbl_ff_portfolio_coverage <- tbl_ff_portfolio_returns |>
  group_by(month_ID) |>
  summarise(
    portfolios = n_distinct(FF_portfolio_type),
```



```

    .groups = "drop"
  ) |>
  filter(portfolios != 6)

if (nrow(tbl_ff_portfolio_coverage) > 0) {
  stop("6個のSize--BE/MEポートフォリオがそろわない月があります。")
}

tbl_ff_portfolio_summary <- tbl_ff_portfolio_returns |>
  group_by(
    FF_portfolio_type,
    size_group,
    BEME_group
  ) |>
  summarise(
    months = n(),
    mean_return = mean(R),
    mean_excess_return = mean(Re),
    volatility = sd(R),
    .groups = "drop"
  ) |>
  arrange(FF_portfolio_type)

show_table_LFL(tbl_ff_portfolio_summary)
#> # A tibble: 6 x 7
#>   FF_type siz_group BEM_group months  mea_retur  mea_retur volatility
#>   <fct>   <chr>     <chr>    <int>    <dbl>      <dbl>      <dbl>
#> 1 SL      S        L        60 0.010056  0.0099773  0.044267
#> 2 SN      S        N        60 0.013474  0.013395   0.041252
#> 3 SH      S        H        60 0.014002  0.013923   0.040505
#> 4 BL      B        L        60 0.00081235 0.00073351 0.043049
#> 5 BN      B        N        60 0.0063080  0.0062291  0.042621
#> 6 BH      B        H        60 0.010766  0.010687   0.046846

plot_ff_portfolio <- tbl_ff_portfolio_summary |>
  ggplot(
    aes(
      x = BEME_group,
      y = mean_excess_return,
      fill = size_group
    )
  ) +
  geom_col(position = "dodge") +
  geom_text(

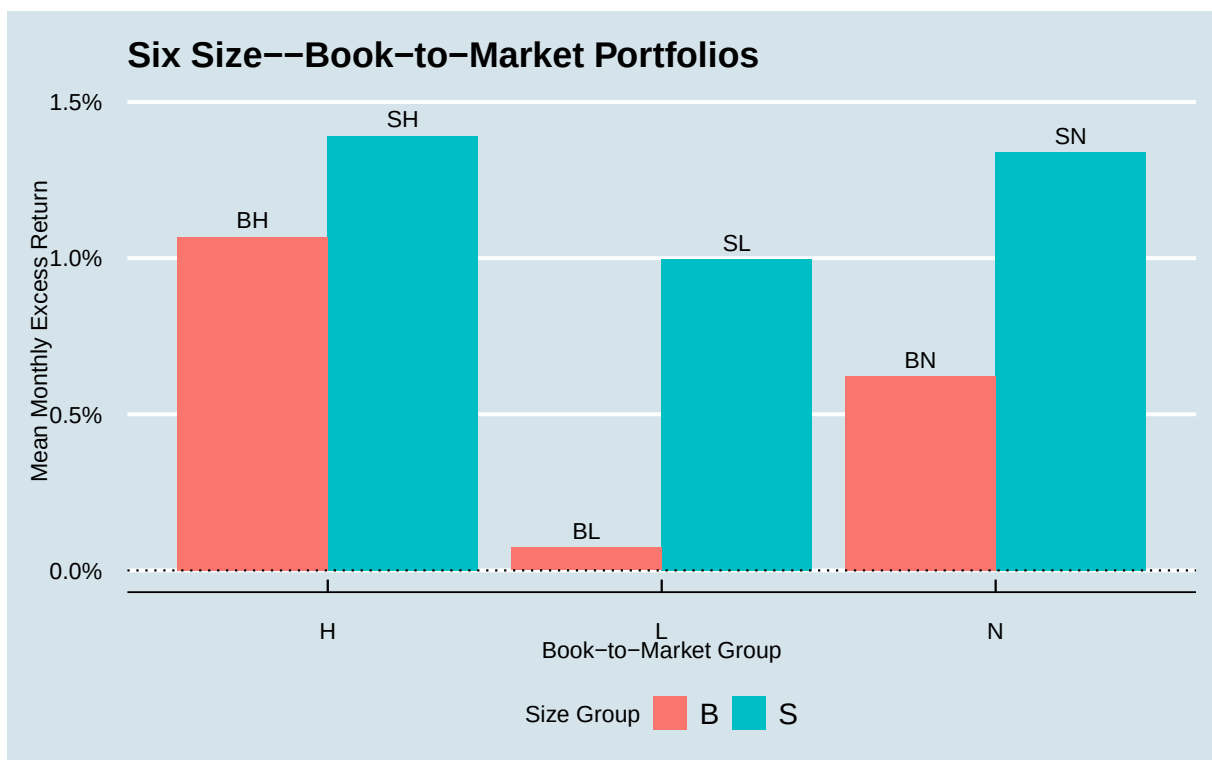
```

```

aes(label = FF_portfolio_type, group = size_group),
position = position_dodge(width = 0.9),
vjust = -0.5
) +
geom_hline(yintercept = 0, linetype = "dotted") +
scale_y_continuous(
  labels = scales::label_percent(),
  expand = expansion(mult = c(0.05, 0.15))
) +
labs(
  x = "Book-to-Market Group",
  y = "Mean Monthly Excess Return",
  fill = "Size Group",
  title = "Six Size--Book-to-Market Portfolios"
)

show_plot_LFL(add_lagrange_theme_LFL(plot_ff_portfolio))

```



```

plot_ff_cumulative <- tbl_ff_portfolio_returns |>
  group_by(FF_portfolio_type) |>
  arrange(month_ID, .by_group = TRUE) |>
  mutate(cumulative_gross_return = cumprod(1 + R)) |>
  ungroup() |>
  ggplot(
    aes(

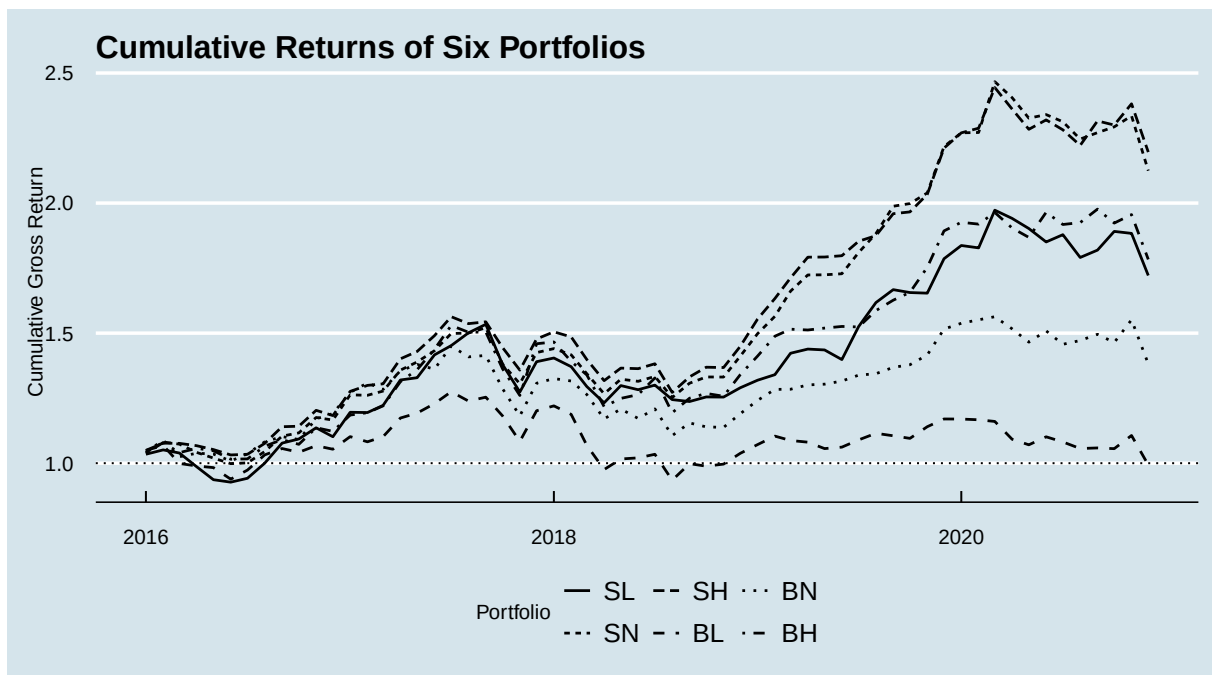
```

```

    x = month_date,
    y = cumulative_gross_return,
    linetype = FF_portfolio_type
  )
) +
geom_line(linewidth = 0.7) +
geom_hline(yintercept = 1, linetype = "dotted") +
labs(
  x = NULL,
  y = "Cumulative Gross Return",
  linetype = "Portfolio",
  title = "Cumulative Returns of Six Portfolios"
)

show_plot_LFL(add_lagrange_theme_LFL(plot_ff_cumulative))

```



4.6.2 SMB と HML

SMB は Small ポートフォリオの平均リターンから Big ポートフォリオの平均リターンを引いて計算する。

$$SMB_t = \frac{SL_t + SN_t + SH_t}{3} - \frac{BL_t + BN_t + BH_t}{3}$$

HML は High ポートフォリオの平均リターンから Low ポートフォリオの平均リターンを引いて計算する。

$$HML_t = \frac{SH_t + BH_t}{2} - \frac{SL_t + BL_t}{2}$$

```

tbl_ff_portfolio_wide <- tbl_ff_portfolio_returns |>
  select(
    month_ID, year, month, month_date,
    FF_portfolio_type, R
  ) |>
  pivot_wider(
    names_from = FF_portfolio_type,
    values_from = R
  ) |>
  arrange(month_ID)

tbl_smb_hml <- tbl_ff_portfolio_wide |>
  mutate(
    SMB = (SL + SN + SH) / 3 - (BL + BN + BH) / 3,
    HML = (SH + BH) / 2 - (SL + BL) / 2
  ) |>
  select(
    month_ID, year, month, month_date,
    SL, SN, SH, BL, BN, BH, SMB, HML
  )

tbl_factor <- tbl_market_returns |>
  select(
    month_ID, year, month, month_date,
    R_F, R_M, R_Me,
    market_return_coverage = return_coverage
  ) |>
  inner_join(
    tbl_smb_hml,
    by = c("month_ID", "year", "month", "month_date")
  ) |>
  arrange(month_ID)

tbl_invalid_factor_rows <- tbl_factor |>
  filter(
    !if_all(
      c(R_F, R_M, R_Me, SMB, HML),
      is.finite
    )
  )

if (nrow(tbl_invalid_factor_rows) > 0) {
  stop("ファクター・データに欠損値または非有限値があります。")
}

```

```

}

show_table_LFL(tbl_factor)
#> # A tibble: 60 x 16
#>   mon_ID year month mon_date      R_F      R_M      R_Me mar_cover
#>   <int> <int> <int> <date>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     13  2016     1 2016-01-01 0.000082029  0.045835  0.045753         1
#> 2     14  2016     2 2016-02-01 0.000075959  0.028423  0.028347         1
#> 3     15  2016     3 2016-03-01 0.000075959 -0.057976 -0.058052         1
#> 4     16  2016     4 2016-04-01 0.000082029 -0.0010718 -0.0011539         1
#> 5     17  2016     5 2016-05-01 0.000082029 -0.0064160 -0.0064980         1
#> 6     18  2016     6 2016-06-01 0.000082029 -0.037075 -0.037157         1
#> 7     19  2016     7 2016-07-01 0.000082029  0.029853  0.029771         1
#> 8     20  2016     8 2016-08-01 0.000082029  0.053757  0.053675         1
#> 9     21  2016     9 2016-09-01 0.000082029  0.023650  0.023568         1
#> 10    22  2016    10 2016-10-01 0.000075959 -0.012053 -0.012129         1
#> # i 50 more rows
#> # i 8 more variables: SL <dbl>, SN <dbl>, SH <dbl>, BL <dbl>, BN <dbl>,
#> #   BH <dbl>, SMB <dbl>, HML <dbl>

```

4.6.3 ファクターの要約統計量

```

vec_factor_names <- c("R_Me", "SMB", "HML")

tbl_factor_summary <- vec_factor_names |>
  map_dfr(
    \(factor_name) {
      values <- tbl_factor[[factor_name]]
      finite_values <- values[is.finite(values)]

      tibble(
        factor = factor_name,
        observations = length(finite_values),
        mean = mean(finite_values),
        standard_deviation = sd(finite_values),
        minimum = min(finite_values),
        median = median(finite_values),
        maximum = max(finite_values),
        t_value = mean(finite_values) /
          (sd(finite_values) / sqrt(length(finite_values)))
      )
    }
  )

```

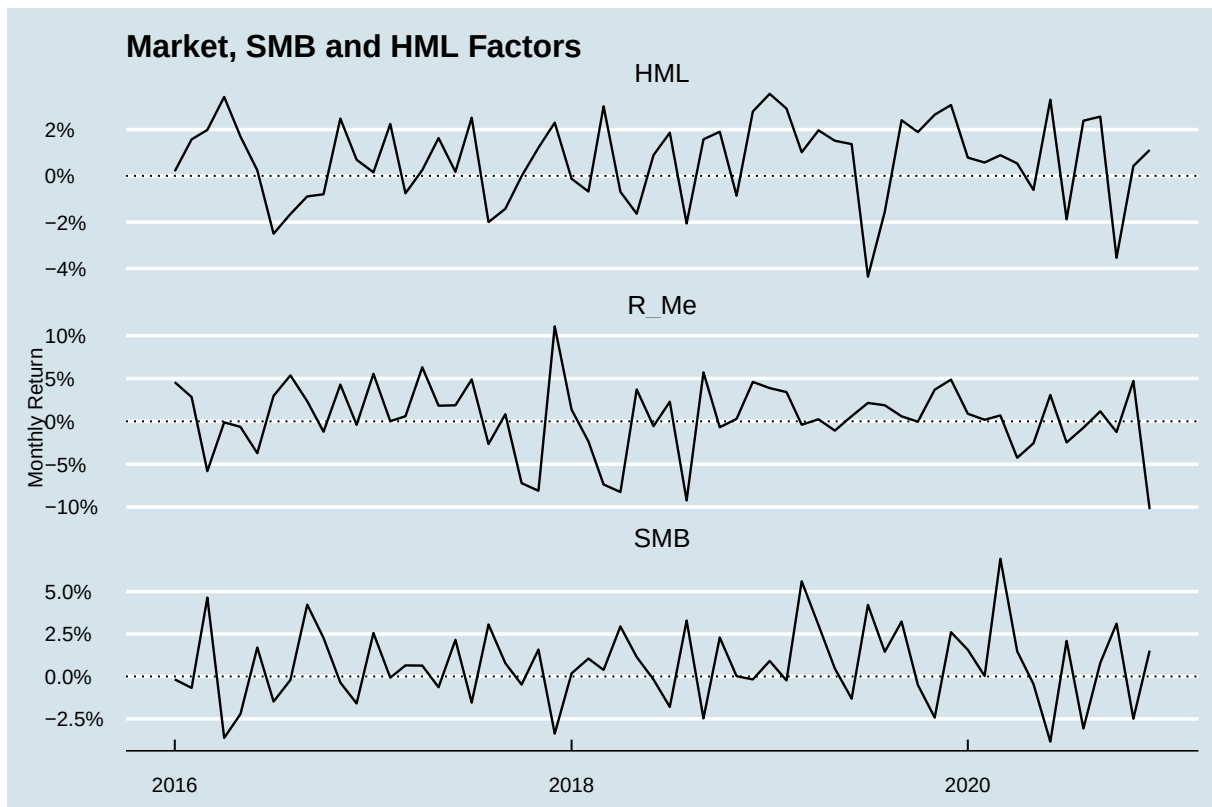
```
tbl_factor_correlation <- tbl_factor |>
  select(R_Me, SMB, HML) |>
  cor(use = "complete.obs")

show_table_LFL(tbl_factor_summary)
#> # A tibble: 3 x 8
#>   factor observations      mean sta_devia  minimum    median  maximum t_value
#>   <chr>          <int>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 R_Me              60 0.0040215  0.041498 -0.10252  0.0060055  0.11097  0.75064
#> 2 SMB                60 0.0065488  0.022857 -0.038447  0.0055499  0.069390  2.2193
#> 3 HML                60 0.0069496  0.018058 -0.043572  0.0089560  0.035442  2.9811

tbl_factor_correlation
#>           R_Me      SMB      HML
#> R_Me  1.0000000 -0.3431883  0.1702499
#> SMB  -0.3431883  1.0000000 -0.3806883
#> HML   0.1702499 -0.3806883  1.0000000
```

```
plot_factor_time_series <- tbl_factor |>
  select(month_date, R_Me, SMB, HML) |>
  pivot_longer(
    cols = c(R_Me, SMB, HML),
    names_to = "factor",
    values_to = "return"
  ) |>
  ggplot(aes(x = month_date, y = return)) +
  geom_line(linewidth = 0.6) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  facet_wrap(
    vars(factor),
    ncol = 1,
    scales = "free_y"
  ) +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = NULL,
    y = "Monthly Return",
    title = "Market, SMB and HML Factors"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_factor_time_series))
```



4.7 FF3 モデルと CAPM の比較

4.7.1 FF3 モデルの推定

Fama–French の 3 ファクター・モデルは、ポートフォリオの超過リターンを市場、SMB、HML で説明する。

$$R_{p,t}^e = \alpha_p + \beta_{M,p} R_{M,t}^e + \beta_{SMB,p} SMB_t + \beta_{HML,p} HML_t + \varepsilon_{p,t}$$

```
tbl_ff3_data <- tbl_size_decile_returns |>
  select(month_ID, ME_rank10, Re = Re_value) |>
  inner_join(
    tbl_factor |>
      select(month_ID, R_Me, SMB, HML),
    by = "month_ID"
  ) |>
  filter(
    if_all(
      c(Re, R_Me, SMB, HML),
      is.finite
    )
  ) |>
  arrange(ME_rank10, month_ID)

tbl_ff3_models <- tbl_ff3_data |>
```

```

group_by(ME_rank10) |>
nest() |>
mutate(
  model = map(
    data,
    \(data) lm(Re ~ R_Me + SMB + HML, data = data)
  ),
  coefficients = map(model, broom::tidy),
  fit = map(model, broom::glance)
)

tbl_ff3_results <- tbl_ff3_models |>
select(ME_rank10, coefficients) |>
unnest(coefficients) |>
mutate(significance = compute_significance_stars(p.value)) |>
arrange(ME_rank10, term)

tbl_ff3_fit <- tbl_ff3_models |>
select(ME_rank10, fit) |>
unnest(fit) |>
select(
  ME_rank10, r.squared, adj.r.squared,
  sigma, statistic, p.value, nobs
) |>
arrange(ME_rank10)

tbl_ff3_alpha <- tbl_ff3_results |>
filter(term == "(Intercept)") |>
transmute(
  ME_rank10,
  tbl_ff3_alpha = estimate,
  standard_error = std.error,
  statistic,
  p_value = p.value,
  significance
)

show_table_LFL(tbl_ff3_results)
#> # A tibble: 40 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 term          estimate std.error statistic    p.value significance
#>   <int> <chr>          <dbl>     <dbl>     <dbl>    <dbl> <chr>
#> 1      1 1 (Intercept) 0.0022160 0.0019246  1.1514 2.5446e- 1 ""

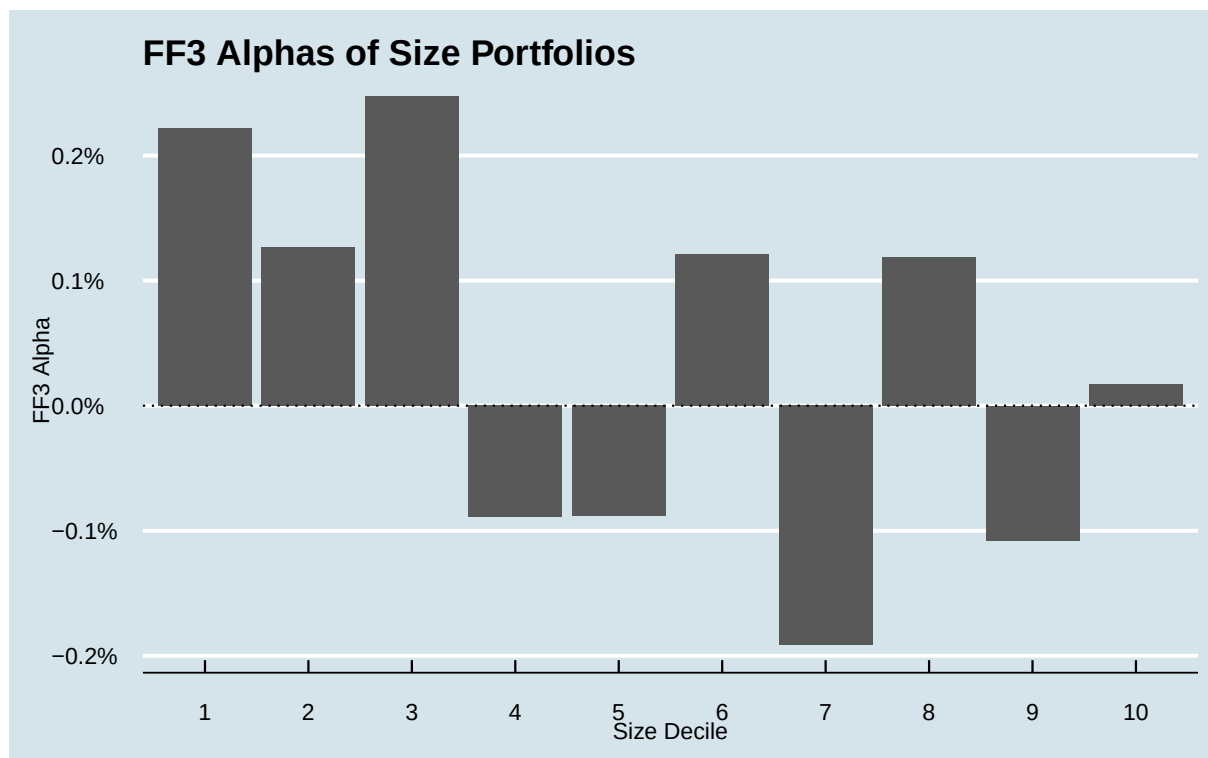
```



```
#> 2      1 HML      0.037314 0.098061    0.38052 7.0500e- 1 ""
#> 3      1 R_Me     0.92776  0.042009   22.085  2.5525e-29 "****"
#> 4      1 SMB      1.3610   0.081276   16.746  1.8226e-23 "****"
#> 5      2 (Intercept) 0.0012670 0.0015741    0.80490 4.2428e- 1 ""
#> 6      2 HML      0.048745 0.080200    0.60779 5.4578e- 1 ""
#> 7      2 R_Me     0.94434  0.034357   27.486  3.3917e-34 "****"
#> 8      2 SMB      1.2382   0.066472   18.627  1.1450e-25 "****"
#> 9      3 (Intercept) 0.0024754 0.0014256    1.7364  8.7992e- 2 "*"
#> 10     3 HML      0.26825  0.072634    3.6931  5.0419e- 4 "****"
#> # i 30 more rows
show_table_LFL(tbl_ff3_fit)
#> # A tibble: 10 x 7
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 r.squared adj.r.squared    sigma statistic    p.value  nobs
#>   <int>     <dbl>      <dbl>    <dbl>    <dbl>    <dbl> <int>
#> 1       1  0.91540    0.91087 0.012564    201.99 5.3609e-30    60
#> 2       2  0.93996    0.93674 0.010276    292.24 3.6748e-34    60
#> 3       3  0.94888    0.94615 0.0093063    346.51 4.0778e-36    60
#> 4       4  0.98279    0.98187 0.0056047   1066.2 2.3878e-49    60
#> 5       5  0.97989    0.97881 0.0059918    909.61 1.8726e-47    60
#> 6       6  0.95716    0.95487 0.0085857    417.09 2.9086e-38    60
#> 7       7  0.96821    0.96651 0.0075647    568.60 6.8782e-42    60
#> 8       8  0.95121    0.94860 0.0093927    363.94 1.1072e-36    60
#> 9       9  0.94869    0.94594 0.010347     345.12 4.5371e-36    60
#> 10     10 0.99594    0.99572 0.0027668   4581.1 6.5209e-67    60
```

```
plot_ff3_alpha <- tbl_ff3_alpha |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(aes(x = ME_rank10, y = tbl_ff3_alpha)) +
  geom_col() +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "FF3 Alpha",
    title = "FF3 Alphas of Size Portfolios"
  )

show_plot_LFL(add_lagrange_theme_LFL(plot_ff3_alpha))
```



4.7.2 アルファとモデル適合度の比較

```
tbl_model_alpha_comparison <- tbl_capm_alpha |>
  select(ME_rank10, tbl_capm_alpha) |>
  left_join(
    tbl_ff3_alpha |>
      select(ME_rank10, tbl_ff3_alpha),
    by = "ME_rank10"
  ) |>
  pivot_longer(
    cols = c(tbl_capm_alpha, tbl_ff3_alpha),
    names_to = "model",
    values_to = "alpha"
  )

tbl_model_fit_comparison <- tbl_capm_fit |>
  select(
    ME_rank10,
    CAPM_adjusted_R2 = adj.r.squared
  ) |>
  left_join(
    tbl_ff3_fit |>
      select(
        ME_rank10,
```

```

      FF3_adjusted_R2 = adj.r.squared
    ),
    by = "ME_rank10"
  )

show_table_LFL(tbl_model_alpha_comparison)
#> # A tibble: 20 x 3
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 model          alpha
#>   <int> <chr>          <dbl>
#> 1      1 tbl_capm_alpha 0.012412
#> 2      1 tbl_ff3_alpha 0.0022160
#> 3      2 tbl_capm_alpha 0.010641
#> 4      2 tbl_ff3_alpha 0.0012670
#> 5      3 tbl_capm_alpha 0.012195
#> 6      3 tbl_ff3_alpha 0.0024754
#> 7      4 tbl_capm_alpha 0.0095370
#> 8      4 tbl_ff3_alpha -0.00088852
#> 9      5 tbl_capm_alpha 0.0073101
#> 10     5 tbl_ff3_alpha -0.00088220
#> # i 10 more rows
show_table_LFL(tbl_model_fit_comparison)
#> # A tibble: 10 x 3
#> # Groups:   ME_rank1 [10]
#>   ME_rank1 CAP_R2 FF3_R2
#>   <int>   <dbl> <dbl>
#> 1      1 0.43109 0.91087
#> 2      2 0.51763 0.93674
#> 3      3 0.63895 0.94615
#> 4      4 0.71461 0.98187
#> 5      5 0.81104 0.97881
#> 6      6 0.85918 0.95487
#> 7      7 0.89456 0.96651
#> 8      8 0.89902 0.94860
#> 9      9 0.93721 0.94594
#> 10    10 0.98827 0.99572

plot_model_alpha <- tbl_model_alpha_comparison |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = alpha,
      fill = model
    )
  )

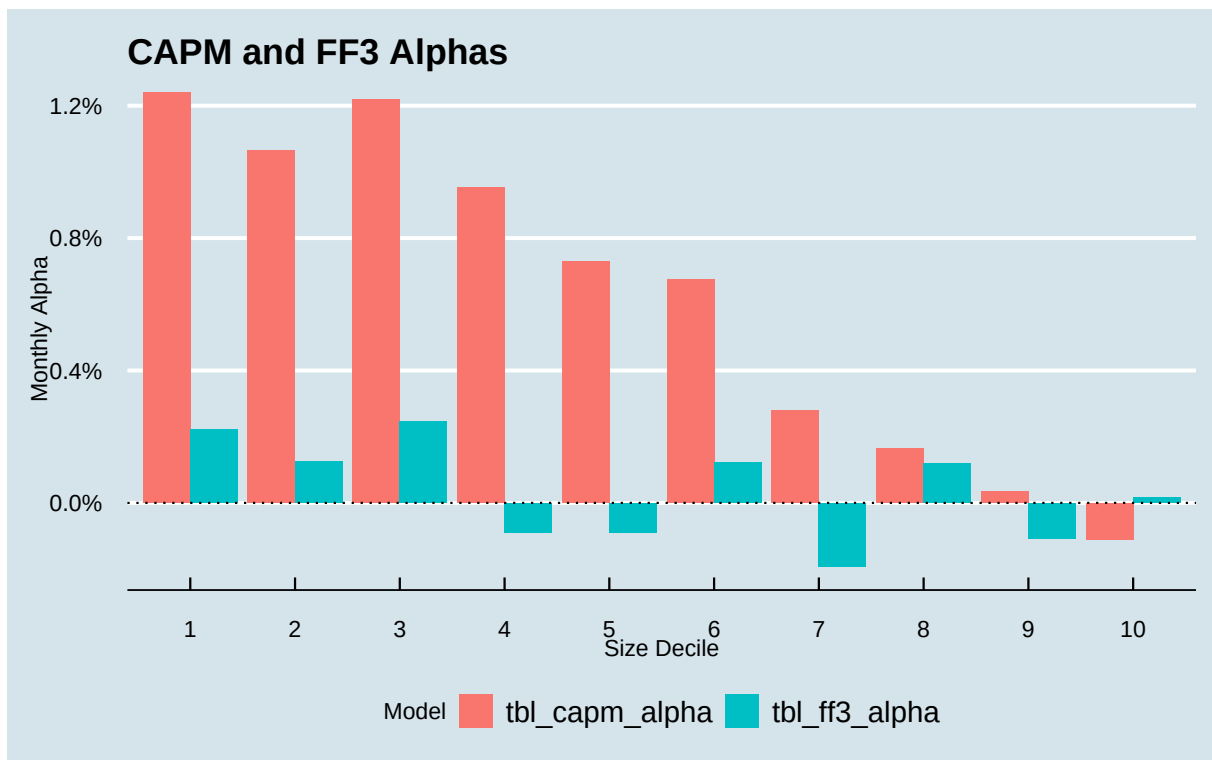
```

```

    )
  ) +
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = "dotted") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    x = "Size Decile",
    y = "Monthly Alpha",
    fill = "Model",
    title = "CAPM and FF3 Alphas"
  )
)

show_plot_LFL(add_lagrange_theme_LFL(plot_model_alpha))

```



```

plot_model_fit <- tbl_model_fit_comparison |>
  pivot_longer(
    cols = c(CAPM_adjusted_R2, FF3_adjusted_R2),
    names_to = "model",
    values_to = "adjusted_R2"
  ) |>
  mutate(ME_rank10 = factor(ME_rank10, levels = 1:10)) |>
  ggplot(
    aes(
      x = ME_rank10,
      y = adjusted_R2,

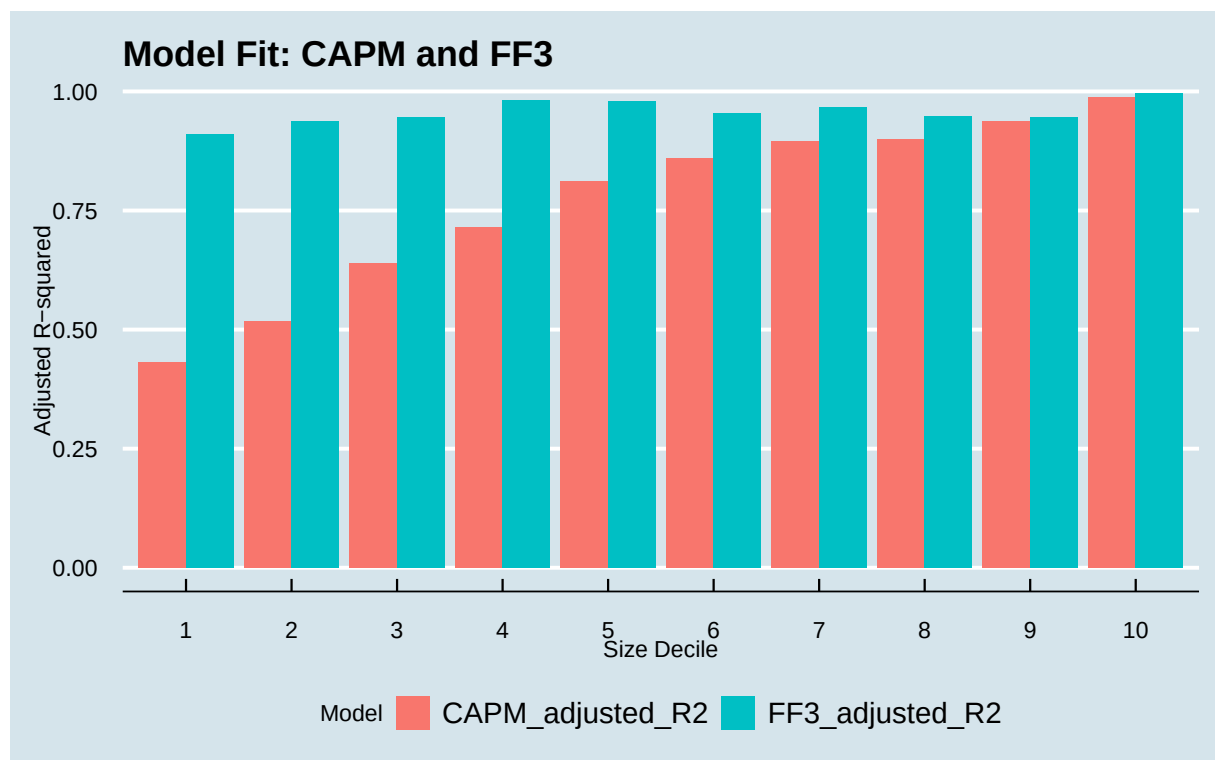
```

```

    fill = model
  )
) +
geom_col(position = "dodge") +
labs(
  x = "Size Decile",
  y = "Adjusted R-squared",
  fill = "Model",
  title = "Model Fit: CAPM and FF3"
)

show_plot_LFL(add_lagrange_theme_LFL(plot_model_fit))

```



4.8 ファクター・データの保存

第5章以降で使用するファクター・データを、`data/derived/factors/fama_french_factors.parquet` に保存する。

```

tbl_factor_output <- tbl_factor |>
  select(
    month_ID, year, month, month_date,
    R_F, R_M, R_Me, SMB, HML,
    SL, SN, SH, BL, BN, BH,
    market_return_coverage
  )

```

```

vec_factor_output_path <-
  write_derived_parquet_file_LFL(
    tbl_factor_output,
    file.path(
      "factors",
      "fama_french_factors.parquet"
    )
  )

tbl_factor_output_check <-
  read_derived_parquet_file_LFL(
    file.path(
      "factors",
      "fama_french_factors.parquet"
    )
  )

stopifnot(
  nrow(tbl_factor_output_check) == nrow(tbl_factor_output),
  identical(names(tbl_factor_output_check), names(tbl_factor_output))
)

tbl_factor_output_summary <- tibble(
  output_file = relative_path_LFL(vec_factor_output_path),
  rows = nrow(tbl_factor_output_check),
  columns = ncol(tbl_factor_output_check),
  first_month = min(tbl_factor_output_check$month_ID),
  last_month = max(tbl_factor_output_check$month_ID)
)

show_table_LFL(tbl_factor_output_summary)
#> # A tibble: 1 x 5
#>   out_file                                rows columns fir_month las_month
#>   <chr>                                <int>  <int>    <int>    <int>
#> 1 data/derived/factors/fama_french_factors.pa~    60    16      13      72

```

4.9 まとめ

本章では、前年度末時価総額を用いて市場ポートフォリオと企業規模ポートフォリオを構築した。また、企業規模と簿価時価比率による独立 2×3 ソートから 6 ポートフォリオを作成し、SMB と HML を計算した。

企業規模 10 分位ポートフォリオに対して CAPM と FF3 モデルを推定し、アルファと調整済み決定係数を比較した。回帰の反復処理には `nest()` と `purrr::map()` を使用しており、明示的なループは使用していない。

作成したファクター・データは次の場所に保存される。

```
data/derived/factors/fama_french_factors.parquet
```

第5章では、このファクター・データを利用して、個別企業の資本コスト、共分散構造、平均分散ポートフォリオを分析する。

第 I-5 章資産価格モデルとポートフォリオ最適化

本章では、Fama-French の 3 ファクター・モデルを用いて企業別の期待リターンを推定し、その結果を配当成長率の分析とポートフォリオ最適化へ接続する。

前章までに作成した月次株式データ、年次企業データ、ファクターデータを利用し、企業固有リスクと共通ファクターリスクから株式リターンの分散共分散行列を構築する。最後に、平均分散最適化を実行し、目標期待リターンの変化に応じた効率的フロンティアを確認する。

5.1 分析対象とデータの接続

5.1.1 共通関数と分析データ

本章では、プロジェクト共通の utility を読み込み、前章までに作成された派生データを利用する。パッケージの導入方法そのものは扱わず、分析に必要な処理から開始する。

```
tbl_monthly_raw <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "returns",  
      "monthly_stock_returns.parquet"  
    )  
  )  
tbl_annual_raw <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "fundamentals",  
      "annual_firm_characteristics.parquet"  
    )  
  )  
tbl_factor <-  
  read_derived_parquet_file_LFL(  
    file.path(  
      "factors",  
      "fama_french_factors.parquet"  
    )  
  )
```



```
)
```

5.1.2 推定対象企業の選定

企業別の時系列回帰には一定数の観測値が必要である。本章では、2020 年末まで上場が継続し、月次観測値が 36 か月以上存在する企業を推定対象とする。

```
tbl_monthly <- tbl_monthly_raw |>
  group_by(firm_ID) |>
  mutate(
    is_public_2020 = max(month_ID, na.rm = TRUE) == 72,
    n_observations = n()
  ) |>
  ungroup() |>
  filter(
    is_public_2020,
    n_observations >= 36
  ) |>
  select(-n_observations)
```

年次データについても、月次データに含まれる企業と年度の組合せに限定する。

```
tbl_annual <- tbl_monthly |>
  distinct(year, firm_ID) |>
  left_join(
    tbl_annual_raw,
    by = c("year", "firm_ID")
  )
```

5.1.3 月次株式リターンとファクターの結合

月次株式データとファクターデータを month_ID で結合する。無リスク金利 R_F が月次株式データにも含まれる場合は、重複を避けるため、ファクターデータ側の値を利用する。

```
tbl_monthly_factor <- tbl_monthly |>
  select(-any_of("R_F")) |>
  left_join(
    tbl_factor,
    by = "month_ID"
  )
```

5.2 FF3 モデルによる期待リターンの推定

5.2.1 FF3 モデル

企業 i の時点 t における超過リターンを、次の 3 ファクター・モデルで表す。

$$R_{i,t} - R_{F,t} = \beta_{i,M} R_{M,t}^e + \beta_{i,SMB} SMB_t + \beta_{i,HML} HML_t + \varepsilon_{i,t}$$

ここで、 $R_{M,t}^e$ は市場超過リターン、 SMB_t は企業規模ファクター、 HML_t は簿価時価比率ファクターである。本章では、入力データに含まれる超過リターン Re を被説明変数とし、定数項を置かずに企業別回帰を推定する。

```
tbl_ff3_models <- tbl_monthly_factor |>
  select(
    firm_ID,
    Re,
    R_Me,
    SMB,
    HML
  ) |>
  drop_na() |>
  group_by(firm_ID) |>
  nest(tbl_firm = -firm_ID) |>
  mutate(
    list_model = map(
      tbl_firm,
      \(tbl_data) {
        lm(
          Re ~ 0 + R_Me + SMB + HML,
          data = tbl_data
        )
      }
    ),
    tbl_coefficient = map(
      list_model,
      broom::tidy
    )
  ) |>
  ungroup()
```

5.2.2 企業別ファクター・ローディング

企業別回帰から得られた係数を横持ち形式に変換し、市場、企業規模、簿価時価比率に対する感応度として整理する。

```
tbl_ff3_loadings <- tbl_ff3_models |>
  select(
    firm_ID,
    tbl_coefficient
  ) |>
  unnest(tbl_coefficient) |>
  select(
    firm_ID,
    term,
    estimate
  ) |>
  pivot_wider(
    names_from = term,
    values_from = estimate
  ) |>
  rename(
    beta_market = R_Me,
    beta_size = SMB,
    beta_value = HML
  )

show_table_head_LFL(tbl_ff3_loadings)
#> # A tibble: 6 x 4
#>   firm_ID beta_market beta_size beta_value
#>   <int>      <dbl>      <dbl>      <dbl>
#> 1     1      1.6191      2.6384      0.48152
#> 2     2      1.2405      0.50565     -0.52252
#> 3     3      0.69623     1.1439      0.33640
#> 4     4      0.37560     0.50193     -0.43245
#> 5     5      0.58264     1.5325      1.1194
#> 6     7      0.38022     0.92052     -0.29961
```

5.2.3 ファクター・リスクプレミアム

月次ファクターリターンの標本平均を 12 倍し、年率換算したファクター・リスクプレミアムを求める。

```
vec_factor_names <- c("R_Me", "SMB", "HML")

tbl_factor_risk_premium <- tbl_factor |>
```

```

summarise(
  across(
    all_of(vec_factor_names),
    \(vec_return) {
      mean(vec_return, na.rm = TRUE) * 12
    }
  )
) |>
pivot_longer(
  cols = everything(),
  names_to = "factor",
  values_to = "annual_risk_premium"
)

vec_factor_risk_premium <- tbl_factor_risk_premium |>
  arrange(match(factor, vec_factor_names)) |>
  pull(annual_risk_premium)

names(vec_factor_risk_premium) <- vec_factor_names

show_table_LFL(tbl_factor_risk_premium)
#> # A tibble: 3 x 2
#>   factor ann_premi
#>   <chr>      <dbl>
#> 1 R_Me      0.048258
#> 2 SMB       0.078586
#> 3 HML       0.083395

```

5.2.4 株式資本コスト

FF3 モデルに基づく企業 i の期待超過リターンは、ファクター・ローディングとファクター・リスクプレミアムの積和として求められる。

$$E[R_i - R_F] = \beta_{i,M} E[R_M^e] + \beta_{i,SMB} E[SMB] + \beta_{i,HML} E[HML]$$

本章では、この期待超過リターンを企業の株式資本コストとして利用する。

```

tbl_cost_of_equity <- tbl_ff3_loadings |>
  mutate(
    cost_of_equity =
      beta_market * vec_factor_risk_premium[["R_Me"]] +
      beta_size * vec_factor_risk_premium[["SMB"]] +
      beta_value * vec_factor_risk_premium[["HML"]]
  )

```

```
show_table_head_LFL(tbl_cost_of_equity)
#> # A tibble: 6 x 5
#>   firm_ID beta_market beta_size beta_value cost_of_equity
#>   <int>      <dbl>    <dbl>    <dbl>      <dbl>
#> 1     1        1.6191    2.6384    0.48152    0.32563
#> 2     2        1.2405    0.50565  -0.52252    0.056027
#> 3     3        0.69623    1.1439    0.33640    0.15155
#> 4     4        0.37560    0.50193  -0.43245    0.021505
#> 5     5        0.58264    1.5325    1.1194     0.24190
#> 6     7        0.38022    0.92052  -0.29961    0.065702
```

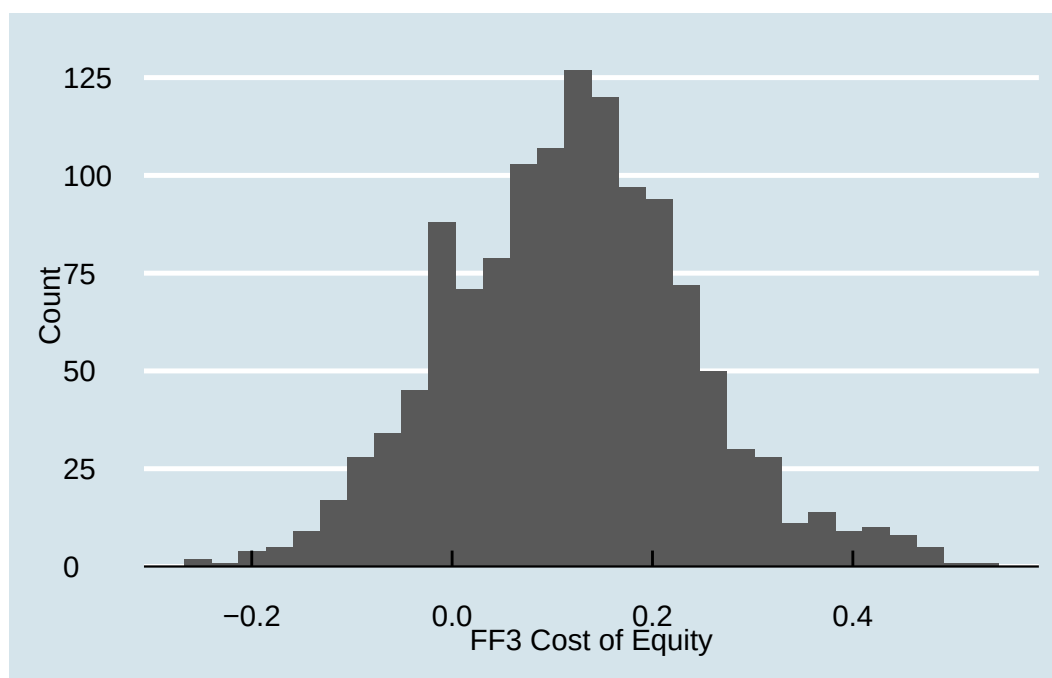
5.3 期待リターンと配当成長率

5.3.1 期待リターンの企業間分布

```
plot_cost_of_equity_distribution <- tbl_cost_of_equity |>
  ggplot(
    aes(x = cost_of_equity)
  ) +
  geom_histogram(bins = 30) +
  labs(
    x = "FF3 Cost of Equity",
    y = "Count"
  ) +
  scale_y_continuous(
    expand = expansion(mult = c(0, 0.05))
  )

plot_cost_of_equity_distribution <- add_lagrange_theme_LFL(
  plot_cost_of_equity_distribution
)

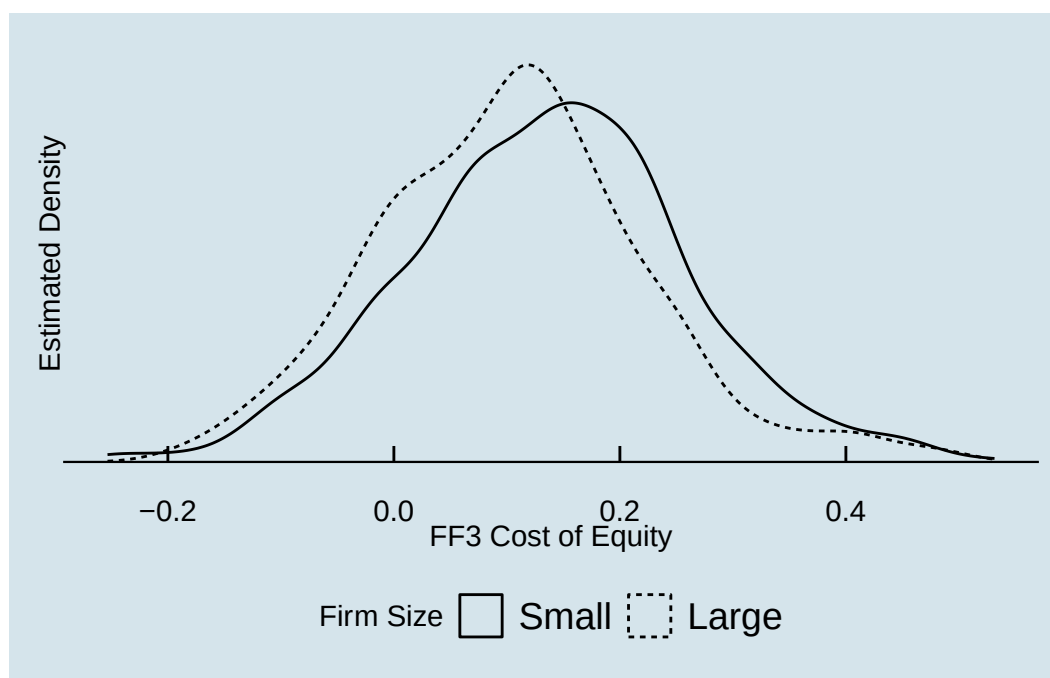
show_plot_LFL(plot_cost_of_equity_distribution)
```



5.3.2 企業規模と期待リターン

```
tbl_firm_characteristics_2020 <- tbl_annual |>
  arrange(
    firm_ID,
    year
  ) |>
  group_by(firm_ID) |>
  mutate(
    lagged_ME = lag(ME)
  ) |>
  ungroup() |>
  filter(year == 2020) |>
  mutate(
    size_group = ntile(lagged_ME, 2),
    size_group = factor(
      size_group,
      levels = c(1, 2),
      labels = c("Small", "Large")
    )
  ) |>
  inner_join(
    tbl_cost_of_equity,
    by = "firm_ID"
  ) |>
  drop_na()
```

```
    lagged_ME,  
    cost_of_equity,  
    size_group  
  )  
  
plot_cost_of_equity_by_size <- tbl_firm_characteristics_2020 |>  
  ggplot(  
    aes(  
      x = cost_of_equity,  
      linetype = size_group  
    )  
  ) +  
  geom_density() +  
  labs(  
    x = "FF3 Cost of Equity",  
    y = "Estimated Density",  
    linetype = "Firm Size"  
  ) +  
  scale_y_continuous(  
    expand = expansion(mult = c(0, 0.05)),  
    breaks = NULL  
  )  
  
plot_cost_of_equity_by_size <- add_lagrange_theme_LFL(  
  plot_cost_of_equity_by_size  
)  
  
show_plot_LFL(plot_cost_of_equity_by_size)
```



5.3.3 配当利回り

```
tbl_dividend_yield_2020 <- tbl_monthly |>
  filter(year == 2020) |>
  group_by(firm_ID) |>
  summarise(
    annual_dividend = sum(
      DPS * shares_outstanding,
      na.rm = TRUE
    ),
    latest_ME = ME[which.max(month_ID)],
    .groups = "drop"
  ) |>
  mutate(
    dividend_yield = annual_dividend / latest_ME
  ) |>
  filter(
    is.finite(dividend_yield),
    dividend_yield > 0
  )

show_table_head_LFL(tbl_dividend_yield_2020)
#> # A tibble: 6 x 4
#>   firm_ID annual_dividend latest_ME dividend_yield
#>   <int>         <dbl>         <dbl>         <dbl>
#> 1      1      325472000    9708946000    0.033523
```



```
#> 2      2      37536000  5194044000    0.0072267
#> 3      3      4660110000 214032195000    0.021773
#> 4      4      101352000   8652927000    0.011713
#> 5      5      745488000  235170402000    0.0031700
#> 6      7      64848000   15909376000    0.0040761
```

5.3.4 潜在配当成長率

株式資本コストを r_i 、配当利回りを y_i とすると、本章で用いる潜在配当成長率 g_i は次式から逆算される。

$$g_i = \frac{r_i - y_i}{1 + y_i}$$

```
tbl_implied_dividend_growth <- tbl_dividend_yield_2020 |>
  inner_join(
    tbl_cost_of_equity |>
      select(
        firm_ID,
        cost_of_equity
      ),
    by = "firm_ID"
  ) |>
  mutate(
    implied_dividend_growth =
      (cost_of_equity - dividend_yield) /
      (1 + dividend_yield)
  )

show_table_head_LFL(tbl_implied_dividend_growth)
#> # A tibble: 6 x 6
#>   firm_ID annual_dividend latest_ME dividend_yield cost_of_equity
#>   <int>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1     1    325472000    9708946000    0.033523    0.32563
#> 2     2    37536000    5194044000    0.0072267    0.056027
#> 3     3    4660110000 214032195000    0.021773    0.15155
#> 4     4    101352000   8652927000    0.011713    0.021505
#> 5     5    745488000  235170402000    0.0031700    0.24190
#> 6     7    64848000   15909376000    0.0040761    0.065702
#> # i 1 more variable: implied_dividend_growth <dbl>
```

5.4 ファクターモデルによるリスク推定

5.4.1 ファクターの分散共分散

```
tbl_factor_covariance <- compute_covariance_table_LFL(
  tbl_data = tbl_factor,
  vec_variables = vec_factor_names,
  annualization_factor = 12
)

matrix_factor_covariance <- convert_covariance_table_to_matrix_LFL(
  tbl_factor_covariance
)

show_table_LFL(tbl_factor_covariance)
#> # A tibble: 9 x 3
#>   var_row var_column covariance
#>   <chr>   <chr>         <dbl>
#> 1 R_Me   R_Me           0.020665
#> 2 R_Me   SMB            -0.0039063
#> 3 R_Me   HML            0.0015310
#> 4 SMB    R_Me            -0.0039063
#> 5 SMB    SMB             0.0062694
#> 6 SMB    HML            -0.0018855
#> 7 HML    R_Me            0.0015310
#> 8 HML    SMB            -0.0018855
#> 9 HML    HML            0.0039129
```

5.4.2 投資ユニバース

```
n_portfolio_firms <- 100L

tbl_investment_universe <- tbl_annual |>
  filter(year == 2020) |>
  inner_join(
    tbl_cost_of_equity |>
      select(
        firm_ID,
        cost_of_equity
      ),
    by = "firm_ID"
  ) |>
```

```

slice_max(
  order_by = ME,
  n = n_portfolio_firms,
  with_ties = FALSE
) |>
arrange(desc(ME)) |>
select(
  firm_ID,
  ME,
  cost_of_equity
)

vec_investment_firm_ids <- tbl_investment_universe |>
  pull(firm_ID)

n_portfolio_firms <- length(vec_investment_firm_ids)

show_table_head_LFL(tbl_investment_universe)
#> # A tibble: 6 x 3
#>   firm_ID      ME cost_of_equity
#>   <int>    <dbl>         <dbl>
#> 1     790 2.7339e13      0.061424
#> 2     987 5.4441e12     -0.071023
#> 3    1237 5.2764e12      0.091188
#> 4    1057 5.1050e12     -0.058108
#> 5     988 4.9127e12     -0.10059
#> 6    1400 3.7669e12      0.20979

```

5.4.3 企業固有リスク

```

tbl_idiosyncratic_variance <- tbl_monthly_factor |>
  filter(
    firm_ID %in% vec_investment_firm_ids
  ) |>
  inner_join(
    tbl_ff3_loadings,
    by = "firm_ID"
  ) |>
  mutate(
    fitted_return =
      R_F +
      beta_market * R_Me +
      beta_size * SMB +

```

```

    beta_value * HML,
    residual_return = R - fitted_return
  ) |>
  group_by(firm_ID) |>
  summarise(
    idiosyncratic_variance =
      12 * var(
        residual_return,
        na.rm = TRUE
      ),
    .groups = "drop"
  ) |>
  right_join(
    tbl_investment_universe |>
      select(firm_ID),
    by = "firm_ID"
  ) |>
  arrange(
    match(
      firm_ID,
      vec_investment_firm_ids
    )
  )

show_table_head_LFL(tbl_idiosyncratic_variance)
#> # A tibble: 6 x 2
#>   firm_ID idiosyncratic_variance
#>   <int>         <dbl>
#> 1     790         0.055478
#> 2     987         0.036155
#> 3    1237         0.053695
#> 4    1057         0.045481
#> 5     988         0.12455
#> 6    1400         0.094833

```

5.4.4 株式リターンの分散共分散

企業別ファクター・ローディングを並べた行列を B 、企業固有リスクの対角行列を Σ_ϵ とすると、株式リターンの分散共分散行列は次式で構築できる。

$$\Sigma = B\Sigma_F B^\top + \Sigma_\epsilon$$

```
tbl_portfolio_loadings <- tbl_investment_universe |>
  select(firm_ID) |>
  left_join(
    tbl_ff3_loadings,
    by = "firm_ID"
  )

matrix_factor_loadings <- tbl_portfolio_loadings |>
  select(
    beta_market,
    beta_size,
    beta_value
  ) |>
  as.matrix()

vec_idiosyncratic_variance <- tbl_idiosyncratic_variance |>
  pull(idiosyncratic_variance)

matrix_idiosyncratic_covariance <- diag(
  vec_idiosyncratic_variance
)

matrix_return_covariance <-
  matrix_factor_loadings %*%
  matrix_factor_covariance %*%
  t(matrix_factor_loadings) +
  matrix_idiosyncratic_covariance

rownames(matrix_return_covariance) <- vec_investment_firm_ids
colnames(matrix_return_covariance) <- vec_investment_firm_ids
```

5.4.5 期待リターン

```
vec_expected_returns <- tbl_investment_universe |>
  pull(cost_of_equity)

names(vec_expected_returns) <- vec_investment_firm_ids

tibble(
  firm_ID = vec_investment_firm_ids,
  expected_return = vec_expected_returns
) |>
  show_table_head_LFL()
```

```
#> # A tibble: 6 x 2
#>   firm_ID expected_return
#>   <int>         <dbl>
#> 1     790         0.061424
#> 2     987        -0.071023
#> 3    1237         0.091188
#> 4    1057        -0.058108
#> 5     988        -0.10059
#> 6    1400         0.20979
```

5.5 平均分散最適化

5.5.1 最適化問題

目標期待リターンを μ_p^* とすると、ポートフォリオ・ウェイト w は次の問題から求められる。

$$\min_w \frac{1}{2} w^T \Sigma w$$

$$w^T \mu = \mu_p^*$$

$$w^T \mathbf{1} = 1$$

本章の基本設定では、空売りを禁止する不等式制約は追加しない。

5.5.2 目標期待リターンの設定

```
target_return <- 0.10

list_mean_variance_portfolio <- compute_mean_variance_portfolio_LFL(
  matrix_covariance = matrix_return_covariance,
  vec_expected_returns = vec_expected_returns,
  target_return = target_return
)
```

5.5.3 最適ポートフォリオ

```
tbl_optimal_portfolio <- create_portfolio_weight_table_LFL(
  list_optimization = list_mean_variance_portfolio,
  vec_asset_ids = vec_investment_firm_ids,
  target_return = target_return
) |>
```

```

arrange(
  desc(abs(weight))
)

show_table_head_LFL(
  tbl_optimal_portfolio,
  n = 10
)
#> # A tibble: 10 x 3
#>   asset_ID    weight target_return
#>   <int>    <dbl>         <dbl>
#> 1     208 -0.16960           0.1
#> 2    1473  0.075792          0.1
#> 3     727  0.075617          0.1
#> 4    1253  0.073272          0.1
#> 5     987  0.068849          0.1
#> 6     829  0.062825          0.1
#> 7     409 -0.059685          0.1
#> 8      33  0.057845          0.1
#> 9     765  0.057677          0.1
#> 10    709  0.057313          0.1

```

5.5.4 最適化結果の検算

```

tbl_portfolio_summary <- compute_portfolio_summary_LFL(
  list_optimization = list_mean_variance_portfolio,
  target_return = target_return
) |>
mutate(
  realized_expected_return = sum(
    list_mean_variance_portfolio$solution *
    vec_expected_returns
  ),
  return_constraint_error =
    realized_expected_return -
    target_return,
  weight_constraint_error =
    weight_sum -
    1
)

show_table_LFL(tbl_portfolio_summary)
#> # A tibble: 1 x 8

```

```
#>   tar_retur por_retur por_varia por_risk wei_sum rea_retur   ret_error wei_error
#>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1      0.1      0.1 0.0069899 0.083606      1      0.1 -2.7756e-17      0
```

5.6 効率的フロンティア

5.6.1 目標期待リターンの範囲

```
vec_target_returns <- seq(
  from = -0.10,
  to = 0.40,
  length.out = 100
)
```

5.6.2 複数ポートフォリオの推定

```
tbl_efficient_frontier <- compute_efficient_frontier_LFL(
  matrix_covariance = matrix_return_covariance,
  vec_expected_returns = vec_expected_returns,
  vec_target_returns = vec_target_returns
)

show_table_head_LFL(
  tbl_efficient_frontier |>
    select(
      target_return,
      portfolio_risk
    )
)

#> # A tibble: 6 x 2
#>   target_return portfolio_risk
#>      <dbl>      <dbl>
#> 1    -0.1      0.079982
#> 2   -0.094949  0.078497
#> 3   -0.089899  0.077068
#> 4   -0.084848  0.075697
#> 5   -0.079798  0.074389
#> 6   -0.074747  0.073145
```


5.6.3 効率的フロンティアの描画

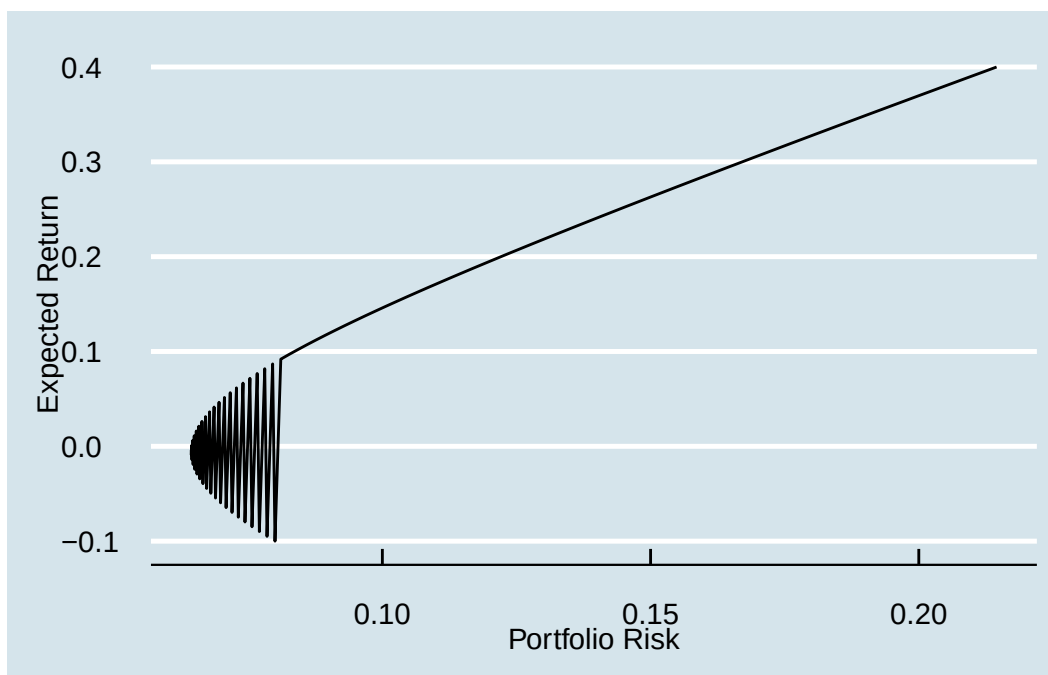
```

plot_efficient_frontier <- tbl_efficient_frontier |>
  ggplot(
    aes(
      x = portfolio_risk,
      y = target_return
    )
  ) +
  geom_line() +
  labs(
    x = "Portfolio Risk",
    y = "Expected Return"
  )

plot_efficient_frontier <- add_lagrange_theme_LFL(
  plot_efficient_frontier
)

show_plot_LFL(plot_efficient_frontier)

```



5.7 まとめ

本章では、Fama-French の 3 ファクター・モデルを企業別に推定し、市場、企業規模、簿価時価比率に対するファクター・ローディングを求めた。ファクター・リスクプレミアムとの積和から株式資本コストを計算し、

配当利回りと組み合わせることで潜在配当成長率を推定した。

さらに、共通ファクターの分散共分散行列と企業固有リスクから株式リターンの分散共分散行列を構築した。これを平均分散最適化へ接続し、単一の目標期待リターンに対応する最適ポートフォリオと、複数の目標期待リターンからなる効率的フロンティアを確認した。

第 I-6 章 イベントスタディ

本章では、企業の決算発表を対象としてイベントスタディを行う。イベントスタディは、企業イベントの前後に観察された株式リターンから、通常時に予想されるリターンを差し引き、イベント固有の株価反応を測定する方法である。

本章では、次の処理を順番に行う。

1. 個別株リターン、市場リターン、イベント情報を読み込む
2. 決算サプライズを計算し、イベントを 5 群に分類する
3. 発表日の翌取引日をイベント日 0 として設定する
4. 推定期間を使ってイベントごとのマーケット・モデルを推定する
5. 異常リターンと累積異常リターンを計算する
6. AAR、CAAR、複数のイベント・ウィンドウについて統計的検定を行う
7. 決算サプライズと短期 CAR の関係を確認する
8. イベント単位と日次集計の結果を保存する

6.1 イベントスタディの基本設計

6.1.1 マーケット・モデル

企業 i の日 t における実現リターンを $R_{i,t}$ 、市場リターンを $R_{M,t}$ とする。推定期間では、次のマーケット・モデルを推定する。

$$R_{i,t} = \alpha_i + \beta_i R_{M,t} + \varepsilon_{i,t}$$

推定された通常リターンは次式で表される。

$$\widehat{R}_{i,t} = \hat{\alpha}_i + \hat{\beta}_i R_{M,t}$$

異常リターン $AR_{i,t}$ は、実現リターンと通常リターンの差である。

$$AR_{i,t} = R_{i,t} - \widehat{R}_{i,t}$$

イベント期間の開始日から日 t までの累積異常リターン $CAR_{i,t}$ は、異常リターンを累積して計算する。

$$CAR_{i,t} = \sum_{\tau=T_1}^t AR_{i,\tau}$$

6.1.2 分析期間

本章では、イベント日 0 を基準として次の期間を使用する。

期間	相対取引日	用途
推定期間	-130 日から-31 日	マーケット・モデルの推定
事前イベント期間	-30 日から-1 日	発表前の株価反応
イベント日	0 日	発表後最初の取引日
事後イベント期間	1 日から 30 日	発表後の株価反応

推定期間は 100 取引日、イベント期間は 61 取引日である。マーケット・モデルの推定には、最低 80 観測を必要とする。

6.1.3 分析環境

```
estimation_start <- -130L
estimation_end <- -31L
event_start <- -30L
event_end <- 30L
minimum_estimation_observations <- 80L

tbl_window_parameter <- tribble(
  ~parameter, ~value,
  "estimation_start", estimation_start,
  "estimation_end", estimation_end,
  "event_start", event_start,
  "event_end", event_end,
  "minimum_estimation_observations", minimum_estimation_observations
)

tbl_window_parameter
#> # A tibble: 5 x 2
#>   parameter          value
#>   <chr>          <int>
#> 1 estimation_start    -130
#> 2 estimation_end      -31
#> 3 event_start        -30
#> 4 event_end           30
#> 5 minimum_estimation_observations    80
```

プロジェクトルートとデータディレクトリを確認する。

```
vec_project_candidate <- unique(c(
  Sys.getenv("QUARTO_PROJECT_DIR"),
  getwd(),
  dirname(getwd())
))

vec_project_candidate <- vec_project_candidate[
  nzchar(vec_project_candidate)
]

vec_project_match <- vec_project_candidate[
  file.exists(file.path(vec_project_candidate, "_quarto.yml"))
]

if (length(vec_project_match) == 0L) {
  stop("LagrangeFinanceのプロジェクトルートを確認できません。")
}

project_dir <- normalizePath(
  vec_project_match[[1]],
  winslash = "/",
  mustWork = TRUE
)

data_dir <- get_data_dir_LFL()

raw_event_data_dir <- file.path(
  data_dir,
  "raw",
  "events"
)

derived_event_data_dir <- file.path(
  data_dir,
  "derived",
  "events"
)

dir.create(
  derived_event_data_dir,
  recursive = TRUE,
  showWarnings = FALSE
)
```

```
tibble(
  directory = c(
    "project",
    "raw_events",
    "derived_events"
  ),
  path = base::c(
    ".",
    relative_path_LFL(raw_event_data_dir),
    relative_path_LFL(derived_event_data_dir)
  )
)
#> # A tibble: 3 x 2
#>   directory      path
#>   <chr>         <chr>
#> 1 project      .
#> 2 raw_events   data/raw/events
#> 3 derived_events data/derived/events
```

6.2 入力データの読み込みと検証

6.2.1 入力ファイル

本章では次の 3 ファイルを使用する。

```
data/raw/events/firm_returns.parquet
data/raw/events/market_returns.parquet
data/raw/events/earnings_events.parquet
```

ファイル名はデータの内容を表す名称に統一し、Parquet 形式で読み込む。

```
vec_input_file <- c(
  return = file.path(
    "raw",
    "events",
    "firm_returns.parquet"
  ),
  market = file.path(
    "raw",
    "events",
    "market_returns.parquet"
  ),
  event = file.path(
    "raw",
```

```
    "events",
    "earnings_events.parquet"
  )
)

vec_input_path <- purrr::map_chr(
  vec_input_file,
  find_data_file_LFL
)

tibble(
  data_name = names(vec_input_path),
  file_name = basename(vec_input_path),
  path = relative_path_LFL(base::unname(vec_input_path))
)

#> # A tibble: 3 x 3
#>   data_name file_name      path
#>   <chr>      <chr>      <chr>
#> 1 return    firm_returns.parquet data/raw/events/firm_returns.parquet
#> 2 market    market_returns.parquet data/raw/events/market_returns.parquet
#> 3 event     earnings_events.parquet data/raw/events/earnings_events.parquet
```

6.2.2 データの読み込み

```
tbl_return_raw <-
  read_parquet_file_LFL(
    vec_input_file[["return"]]
  )

tbl_market_raw <-
  read_parquet_file_LFL(
    vec_input_file[["market"]]
  )

tbl_event_raw <-
  read_parquet_file_LFL(
    vec_input_file[["event"]]
  )
```

6.2.3 必須列の確認

```
validate_required_columns_LFL(  
  tbl_return_raw,  
  c("date", "firm_ID", "R"),  
  "tbl_return_raw"  
)  
  
validate_required_columns_LFL(  
  tbl_market_raw,  
  c("date", "R_M"),  
  "tbl_market_raw"  
)  
  
validate_required_columns_LFL(  
  tbl_event_raw,  
  c(  
    "event_date",  
    "firm_ID",  
    "earnings_forecast",  
    "realized_earnings",  
    "lagged_ME"  
  ),  
  "tbl_event_raw"  
)
```

6.2.4 データ型の整理

```
tbl_return <- tbl_return_raw |>  
  transmute(  
    date = as.Date(date),  
    firm_ID = as.integer(firm_ID),  
    R = as.numeric(R)  
  ) |>  
  arrange(  
    firm_ID,  
    date  
  )  
  
tbl_market <- tbl_market_raw |>  
  transmute(  
    date = as.Date(date),  
    R_M = as.numeric(R_M)  
  ) |>
```



```
arrange(date)

tbl_event <- tbl_event_raw |>
  transmute(
    event_date = as.Date(event_date),
    firm_ID = as.integer(firm_ID),
    earnings_forecast = as.numeric(
      earnings_forecast
    ),
    realized_earnings = as.numeric(
      realized_earnings
    ),
    lagged_ME = as.numeric(lagged_ME)
  ) |>
  arrange(
    event_date,
    firm_ID
  )
```

6.2.5 キーと欠損値の検証

```
validate_unique_keys_LFL(
  tbl_return,
  c("firm_ID", "date"),
  "tbl_return"
)

validate_unique_keys_LFL(
  tbl_market,
  "date",
  "tbl_market"
)

validate_unique_keys_LFL(
  tbl_event,
  c("firm_ID", "event_date"),
  "tbl_event"
)
```

```
tbl_missing_summary <- bind_rows(
  summarize_missing_values_LFL(
    tbl_return,
    "return"
  )
)
```

```

),
summarize_missing_values_LFL(
  tbl_market,
  "market"
),
summarize_missing_values_LFL(
  tbl_event,
  "event"
)
)

tbl_missing_summary
#> # A tibble: 10 x 5
#>   data_name observations missing_ratio variable      missing_values
#>   <chr>          <int>          <dbl> <chr>          <int>
#> 1 return        1610400            0 date              0
#> 2 return        1610400            0 firm_ID           0
#> 3 return        1610400            0 R                 0
#> 4 market         1220            0 date              0
#> 5 market         1220            0 R_M               0
#> 6 event          3960            0 event_date        0
#> 7 event          3960            0 firm_ID           0
#> 8 event          3960            0 earnings_forecast 0
#> 9 event          3960            0 realized_earnings 0
#> 10 event         3960            0 lagged_ME         0

```

6.2.6 入力データの概要

```

tbl_input_overview <- tribble(
  ~data_name, ~observations, ~firms, ~first_date, ~last_date,
  "individual returns",
  nrow(tbl_return),
  n_distinct(tbl_return$firm_ID),
  min(tbl_return$date, na.rm = TRUE),
  max(tbl_return$date, na.rm = TRUE),
  "market returns",
  nrow(tbl_market),
  NA_integer_,
  min(tbl_market$date, na.rm = TRUE),
  max(tbl_market$date, na.rm = TRUE),
  "events",
  nrow(tbl_event),
  n_distinct(tbl_event$firm_ID),

```

```

min(tbl_event$event_date, na.rm = TRUE),
max(tbl_event$event_date, na.rm = TRUE)
)

tbl_input_overview
#> # A tibble: 3 x 5
#>   data_name      observations firms first_date last_date
#>   <chr>          <int> <int> <date>    <date>
#> 1 individual returns    1610400  1320 2016-01-04 2020-12-30
#> 2 market returns        1220    NA 2016-01-04 2020-12-30
#> 3 events                3960  1320 2017-01-04 2020-04-07

```

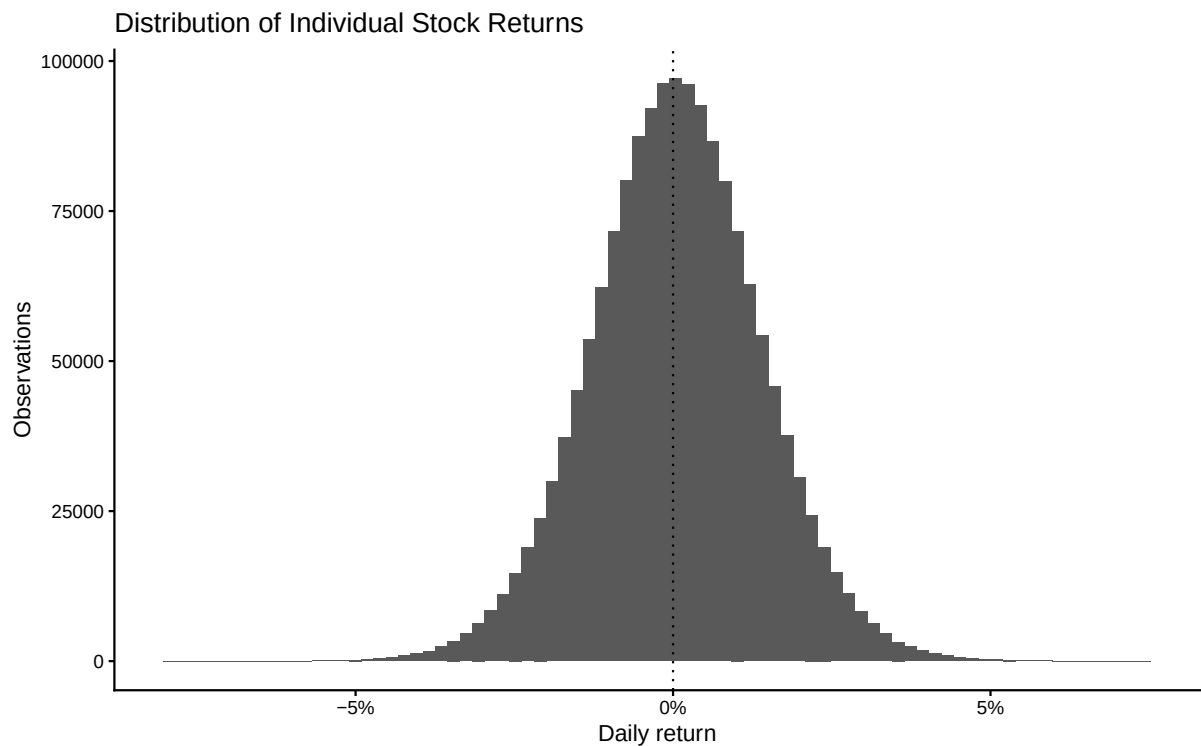
6.2.7 個別株リターンの分布

```

plot_return_distribution <- tbl_return |>
  filter(is.finite(R)) |>
  ggplot(aes(x = R)) +
  geom_histogram(bins = 80) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Daily return",
    y = "Observations",
    title = "Distribution of Individual Stock Returns"
  ) +
  theme_classic()

plot_return_distribution

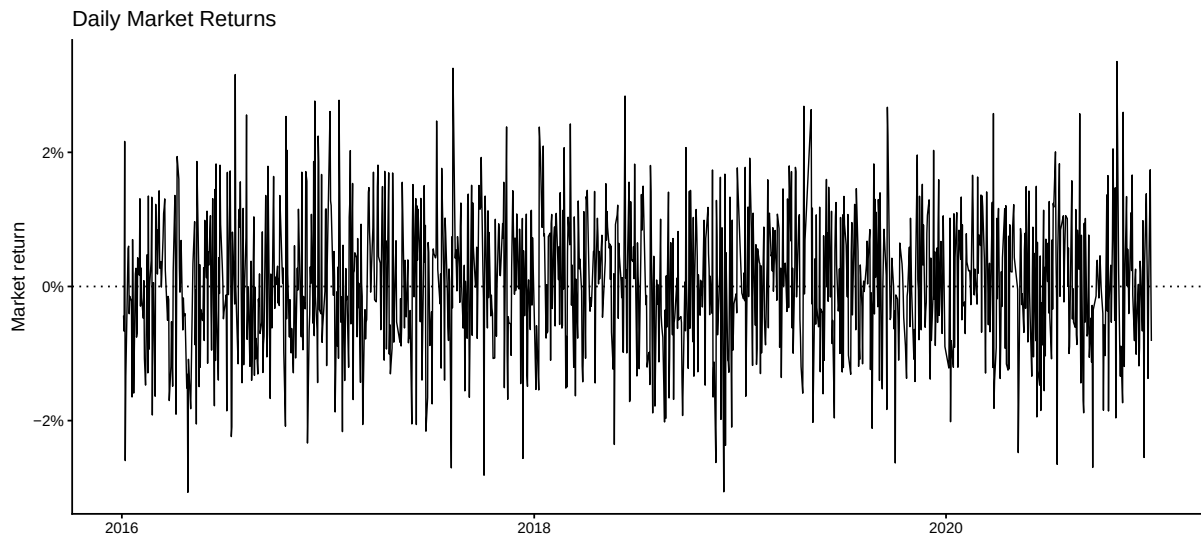
```



6.2.8 市場リターンの時系列

```
plot_market_return_series <- tbl_market |>
  ggplot(aes(x = date, y = R_M)) +
  geom_line(linewidth = 0.4) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = NULL,
    y = "Market return",
    title = "Daily Market Returns"
  ) +
  theme_classic()

plot_market_return_series
```



6.3 決算サプライズとイベント分類

6.3.1 決算サプライズ

決算サプライズは、実現利益と予想利益の差を期首時価総額で標準化して計算する。

$$Surprise_i = \frac{RealizedEarnings_i - ForecastEarnings_i}{ME_{i,-1}}$$

正の値は実現利益が予想を上回ったことを、負の値は予想を下回ったことを表す。

```
tbl_event <- compute_earnings_surprise_LFL(
  tbl_event = tbl_event,
  number_of_groups = 5L
)

tbl_event |>
  select(
    event_ID,
    firm_ID,
    event_date,
    event_year,
    earnings_surprise,
    event_strength
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 6
#>   event_ID firm_ID event_date event_year earnings_surprise event_strength
#>   <int>   <int> <date>      <dbl>          <dbl> <ord>
#> 1     1     1    39 2017-01-04    2017    -0.0081327 1
#> 2     2     2    57 2017-01-04    2017    -0.010049 1
```

```

#> 3      3      73 2017-01-04      2017      -0.0020232 4
#> 4      4     109 2017-01-04      2017      -0.0043961 3
#> 5      5     111 2017-01-04      2017      -0.0044873 3
#> 6      6     231 2017-01-04      2017      -0.014399  1
#> 7      7     944 2017-01-04      2017      -0.0068979 2
#> 8      8    1227 2017-01-04      2017      -0.0011248 5
#> 9      9     422 2017-01-05      2017      -0.0048408 3
#> 10     10    1203 2017-01-05      2017      -0.00086568 5
#> 11     11     346 2017-01-06      2017      -0.0058582 2
#> 12     12     590 2017-01-06      2017      -0.010344  1
#> 13     13     644 2017-01-06      2017      -0.0064889 2
#> 14     14    1153 2017-01-06      2017      -0.0021003 4
#> 15     15     191 2017-01-10      2017      -0.0034658 4
#> 16     16     353 2017-01-10      2017      -0.010249  1
#> 17     17     566 2017-01-10      2017      -0.0043876 3
#> 18     18     322 2017-01-11      2017      -0.0046897 3
#> 19     19    1050 2017-01-11      2017      -0.0024991 4
#> 20     20     426 2017-01-12      2017      -0.0010089 5

```

6.3.2 イベント強度別の概要

```

tbl_event_strength_summary <- tbl_event |>
  group_by(
    event_year,
    event_strength
  ) |>
  summarise(
    events = n(),
    mean_surprise = mean(
      earnings_surprise,
      na.rm = TRUE
    ),
    median_surprise = median(
      earnings_surprise,
      na.rm = TRUE
    ),
    .groups = "drop"
  ) |>
  arrange(
    event_year,
    event_strength
  )

```

```
tbl_event_strength_summary
#> # A tibble: 20 x 5
#>   event_year event_strength events mean_surprise median_surprise
#>   <dbl> <ord>         <int>         <dbl>         <dbl>
#> 1 2017 1             214 -0.010271 -0.0098319
#> 2 2017 2             214 -0.0064294 -0.0063866
#> 3 2017 3             214 -0.0044249 -0.0043693
#> 4 2017 4             213 -0.0028679 -0.0028917
#> 5 2017 5             213 -0.00062223 -0.00081805
#> 6 2018 1             263 -0.0099427 -0.0092589
#> 7 2018 2             263 -0.0066866 -0.0066374
#> 8 2018 3             263 -0.0048770 -0.0048670
#> 9 2018 4             263 -0.0031921 -0.0031796
#> 10 2018 5            263 -0.00098501 -0.0011702
#> 11 2019 1             262 -0.0098017 -0.0094002
#> 12 2019 2             262 -0.0064618 -0.0064241
#> 13 2019 3             261 -0.0045936 -0.0045890
#> 14 2019 4             261 -0.0029579 -0.0029594
#> 15 2019 5            261 -0.00088716 -0.0010823
#> 16 2020 1             54 -0.010749 -0.010333
#> 17 2020 2             54 -0.0065058 -0.0065211
#> 18 2020 3             54 -0.0043621 -0.0044412
#> 19 2020 4             54 -0.0026712 -0.0026349
#> 20 2020 5             54 -0.00051141 -0.00069626
```

6.3.3 決算サプライズの分布

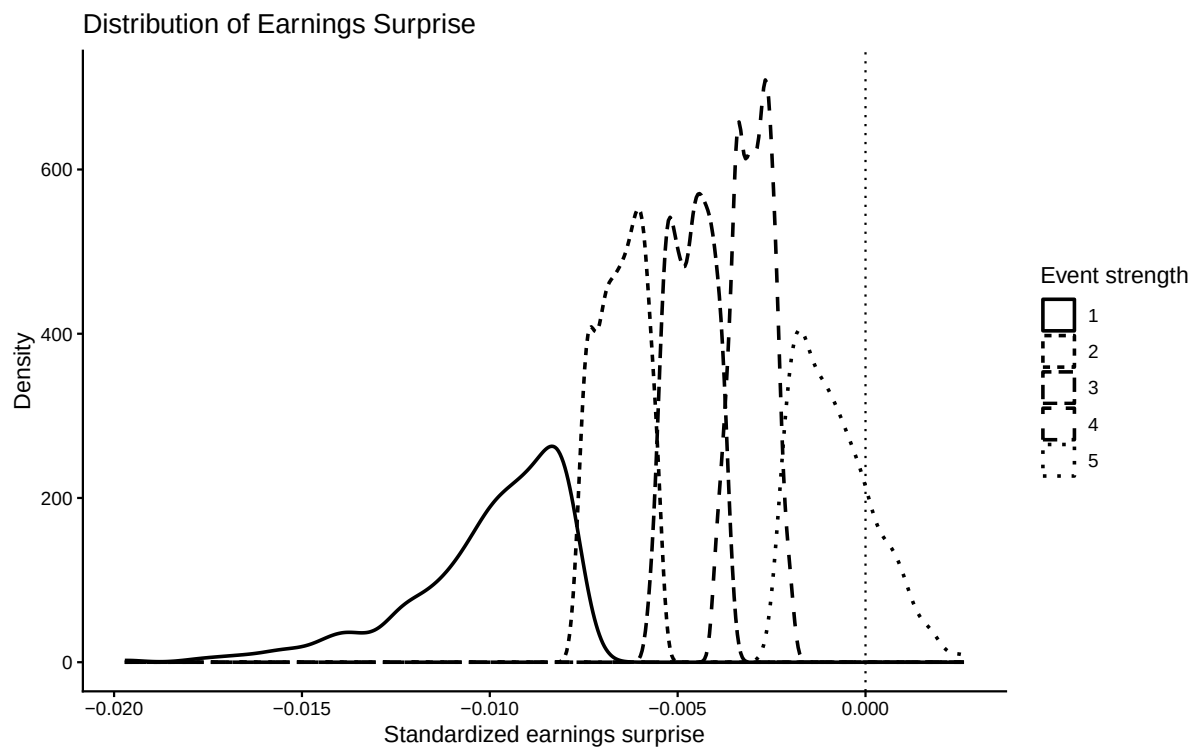
```
plot_earnings_surprise_distribution <- tbl_event |>
  filter(is.finite(earnings_surprise)) |>
  ggplot(
    aes(
      x = earnings_surprise,
      linetype = event_strength
    )
  ) +
  geom_density(linewidth = 0.8) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  labs(
    x = "Standardized earnings surprise",
    y = "Density",
```

```

    linetype = "Event strength",
    title = "Distribution of Earnings Surprise"
  ) +
  theme_classic()

plot_earnings_surprise_distribution

```



6.3.4 イベント強度別の箱ひげ図

```

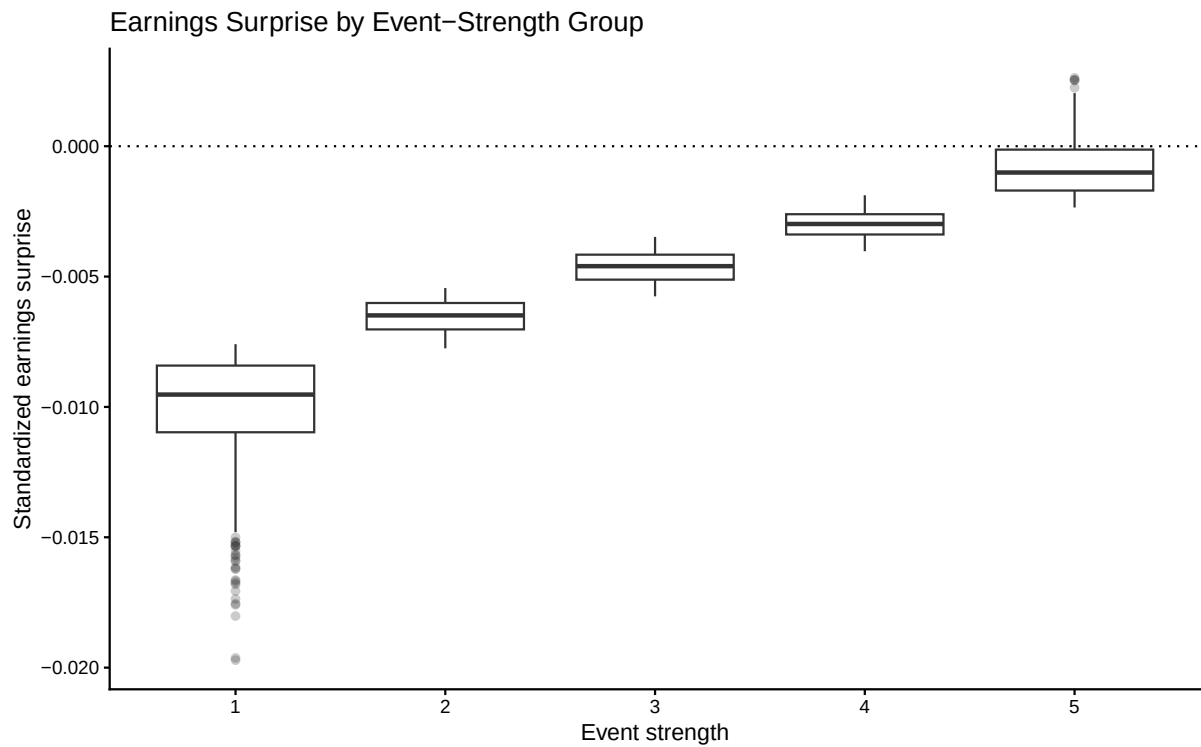
plot_surprise_by_strength <- tbl_event |>
  filter(is.finite(earnings_surprise)) |>
  ggplot(
    aes(
      x = event_strength,
      y = earnings_surprise
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +

```



```
labs(
  x = "Event strength",
  y = "Standardized earnings surprise",
  title = "Earnings Surprise by Event-Strength Group"
) +
theme_classic()

plot_surprise_by_strength
```

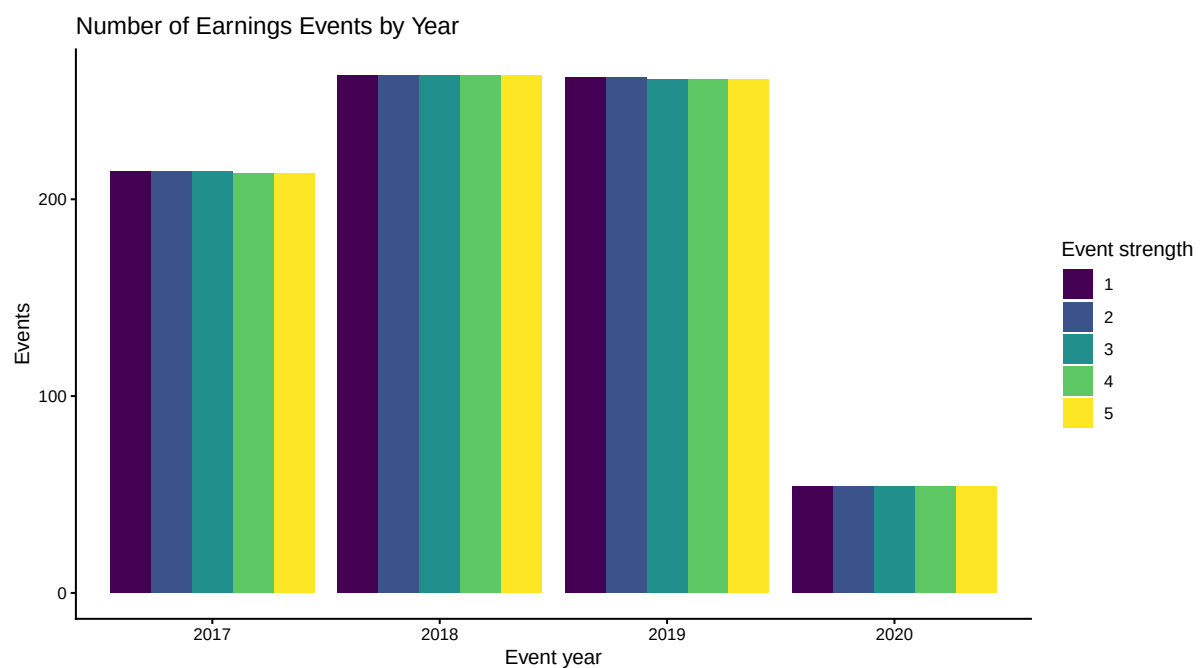


6.3.5 年度別イベント件数

```
plot_events_by_year <- tbl_event |>
  count(
    event_year,
    event_strength,
    name = "events"
  ) |>
  ggplot(
    aes(
      x = factor(event_year),
      y = events,
      fill = event_strength
    )
  ) +
```

```
geom_col(
  position = "dodge"
) +
labs(
  x = "Event year",
  y = "Events",
  fill = "Event strength",
  title = "Number of Earnings Events by Year"
) +
theme_classic()

plot_events_by_year
```



6.4 イベント日とイベント・パネル

6.4.1 取引日カレンダー

市場リターンの日付を取引日カレンダーとして使用する。

```
tbl_trading_calendar <- tbl_market |>
  distinct(date) |>
  arrange(date) |>
  mutate(
    date_ID = row_number()
  ) |>
  select(
    date_ID,
```

```
    date
  )

tbl_market_with_ID <- tbl_market |>
  inner_join(
    tbl_trading_calendar,
    by = "date"
  ) |>
  select(
    date_ID,
    date,
    R_M
  )

tbl_return_market <- tbl_return |>
  inner_join(
    tbl_market_with_ID,
    by = "date"
  ) |>
  select(
    firm_ID,
    date_ID,
    R,
    R_M
  ) |>
  arrange(
    firm_ID,
    date_ID
  )
```

6.4.2 イベント日 0

決算発表は取引終了後に行われると仮定する。そのため、発表日が取引日であっても、その次の取引日をイベント日 0 とする。

```
tbl_event <- assign_event_day_zero_LFL(
  tbl_event = tbl_event,
  tbl_trading_calendar = tbl_trading_calendar
)

tbl_event |>
  select(
    event_ID,
    firm_ID,
```

```

    event_date,
    event_trade_date,
    event_trade_date_ID,
    event_strength,
    earnings_surprise
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 7
#>   event_ID firm_ID event_date event_trade_date event_trade_date_ID
#>   <int>   <int> <date>      <date>                <int>
#> 1     1     39 2017-01-04 2017-01-05                247
#> 2     2     57 2017-01-04 2017-01-05                247
#> 3     3     73 2017-01-04 2017-01-05                247
#> 4     4    109 2017-01-04 2017-01-05                247
#> 5     5    111 2017-01-04 2017-01-05                247
#> 6     6    231 2017-01-04 2017-01-05                247
#> 7     7    944 2017-01-04 2017-01-05                247
#> 8     8   1227 2017-01-04 2017-01-05                247
#> 9     9    422 2017-01-05 2017-01-06                248
#> 10    10   1203 2017-01-05 2017-01-06                248
#> 11    11    346 2017-01-06 2017-01-10                249
#> 12    12    590 2017-01-06 2017-01-10                249
#> 13    13    644 2017-01-06 2017-01-10                249
#> 14    14   1153 2017-01-06 2017-01-10                249
#> 15    15    191 2017-01-10 2017-01-11                250
#> 16    16    353 2017-01-10 2017-01-11                250
#> 17    17    566 2017-01-10 2017-01-11                250
#> 18    18    322 2017-01-11 2017-01-12                251
#> 19    19   1050 2017-01-11 2017-01-12                251
#> 20    20    426 2017-01-12 2017-01-13                252
#> # i 2 more variables: event_strength <ord>, earnings_surprise <dbl>

```

6.4.3 発表日からイベント日 0 までの日数

```

tbl_event_gap <- tbl_event |>
  mutate(
    calendar_day_gap = as.integer(
      event_trade_date - event_date
    )
  )

plot_event_date_gap <- tbl_event_gap |>
  count(

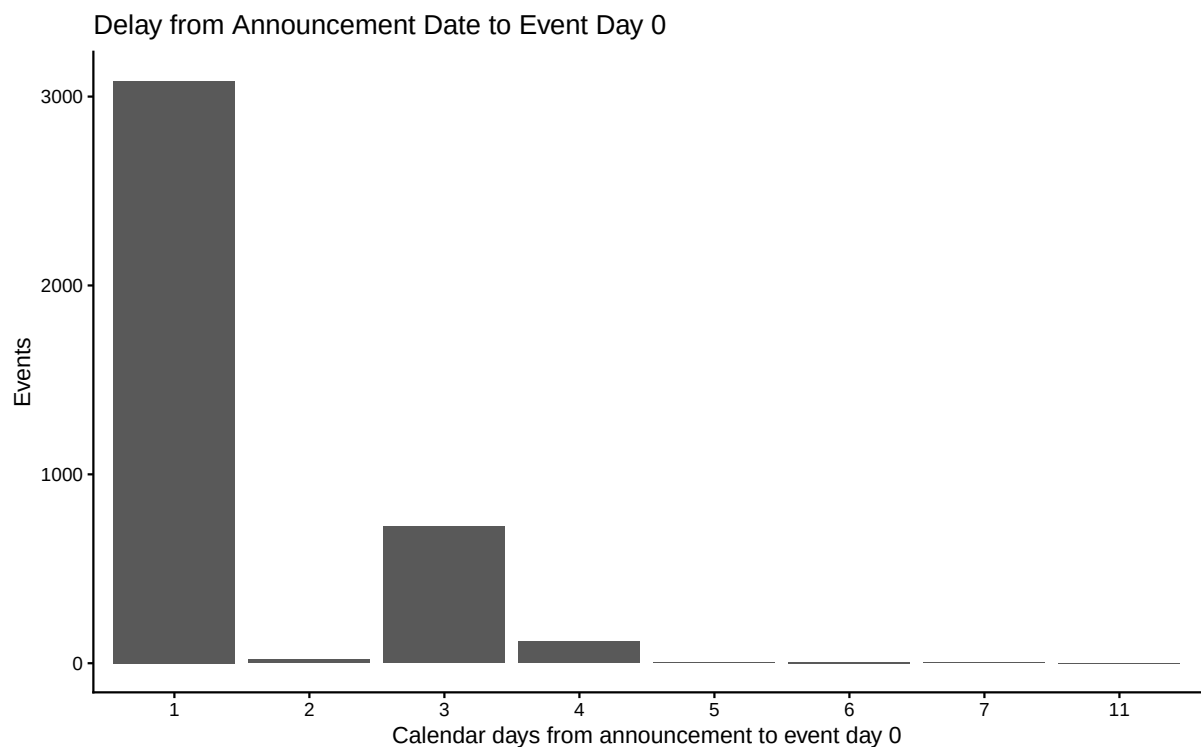
```

```

    calendar_day_gap,
    name = "events"
  ) |>
  ggplot(
    aes(
      x = factor(calendar_day_gap),
      y = events
    )
  ) +
  geom_col() +
  labs(
    x = "Calendar days from announcement to event day 0",
    y = "Events",
    title = "Delay from Announcement Date to Event Day 0"
  ) +
  theme_classic()

plot_event_date_gap

```



6.4.4 イベント・パネル

```

tbl_event_panel <- create_event_study_panel_LFL(
  tbl_event = tbl_event,
  tbl_trading_calendar = tbl_trading_calendar,

```

```
tbl_return_market = tbl_return_market,
estimation_start = estimation_start,
event_end = event_end
)

tbl_event_panel |>
  select(
    event_ID,
    firm_ID,
    relative_day,
    date,
    R,
    R_M
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 6
#>   event_ID firm_ID relative_day date          R          R_M
#>   <int>    <int>      <int> <date>      <dbl>      <dbl>
#> 1         1         39    -130 2016-06-23  0.0098506  0.018087
#> 2         1         39    -129 2016-06-24  0.012580  0.0092862
#> 3         1         39    -128 2016-06-27  0.015076  0.0031098
#> 4         1         39    -127 2016-06-28  0.015788  0.00067260
#> 5         1         39    -126 2016-06-29 -0.0033987 -0.0048162
#> 6         1         39    -125 2016-06-30 -0.0071048 -0.0038061
#> 7         1         39    -124 2016-07-01  0.014684 -0.0032365
#> 8         1         39    -123 2016-07-04 -0.0069849 -0.0012511
#> 9         1         39    -122 2016-07-05 -0.023656 -0.018547
#> 10        1         39    -121 2016-07-06  0.011629  0.017017
#> 11        1         39    -120 2016-07-07 -0.0064901 -0.0011435
#> 12        1         39    -119 2016-07-08 -0.0062219 -0.0012174
#> 13        1         39    -118 2016-07-11  0.0055502  0.017228
#> 14        1         39    -117 2016-07-12  0.0064255  0.0060624
#> 15        1         39    -116 2016-07-13 -0.027040 -0.022380
#> 16        1         39    -115 2016-07-14 -0.014094 -0.020934
#> 17        1         39    -114 2016-07-15  0.0064607  0.0081064
#> 18        1         39    -113 2016-07-19 -0.010573 -0.0026187
#> 19        1         39    -112 2016-07-20  0.024173  0.031602
#> 20        1         39    -111 2016-07-21 -0.00044361 -0.00024792
```

6.4.5 データ利用可能性

```
estimation_window_length <-
  estimation_end -
```

```
estimation_start + 1L

event_window_length <-
  event_end -
  event_start + 1L

tbl_event_coverage <- tbl_event_panel |>
  group_by(
    event_ID,
    firm_ID,
    event_strength
  ) |>
  summarise(
    valid_estimation_days = sum(
      between(
        relative_day,
        estimation_start,
        estimation_end
      ) &
      is.finite(R) &
      is.finite(R_M)
    ),
    valid_event_days = sum(
      between(
        relative_day,
        event_start,
        event_end
      ) &
      is.finite(R) &
      is.finite(R_M)
    ),
    .groups = "drop"
  )

tbl_event_coverage |>
  summarise(
    events = n(),
    complete_estimation_window = sum(
      valid_estimation_days ==
        estimation_window_length
    ),
    usable_estimation_window = sum(
      valid_estimation_days >=
```

```

    minimum_estimation_observations
  ),
  complete_event_window = sum(
    valid_event_days ==
      event_window_length
  )
)
#> # A tibble: 1 x 4
#>   events complete_estimation_window usable_estimation_wi~1 complete_event_window
#>   <int>                <int>                <int>                <int>
#> 1   3960                3960                3960                3960
#> # i abbreviated name: 1: usable_estimation_window

```

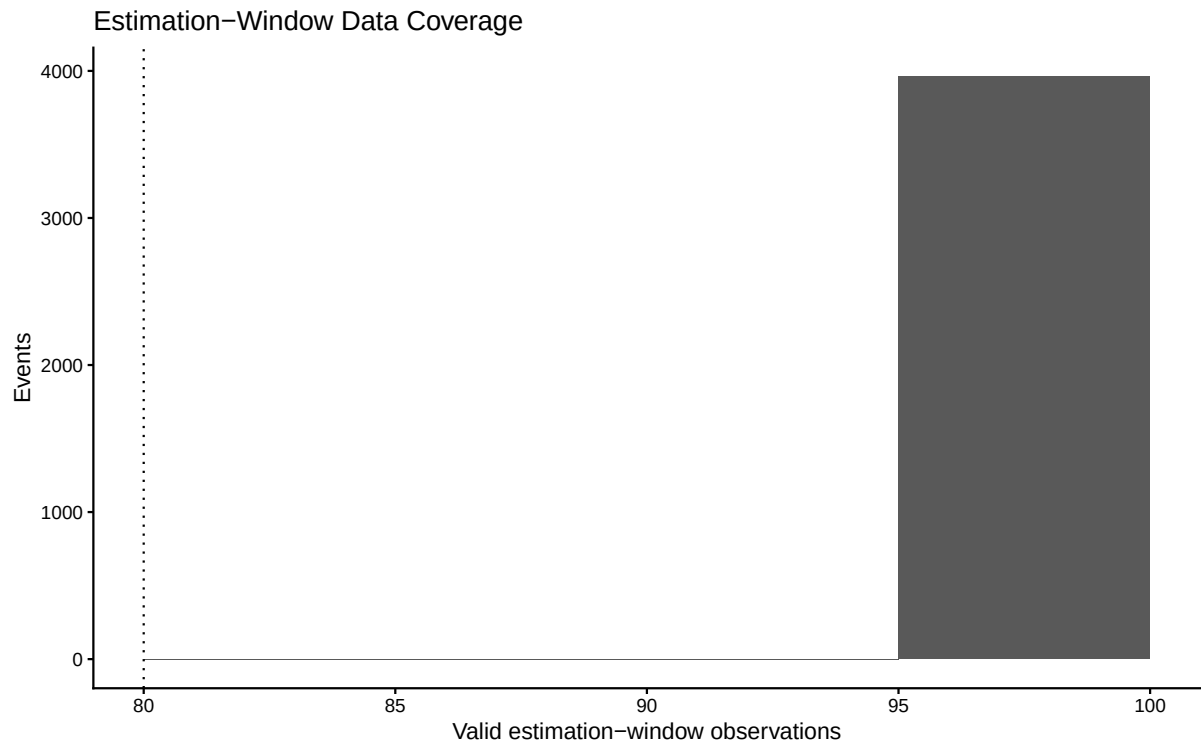
6.4.6 推定期間の利用可能日数

```

plot_estimation_coverage <- tbl_event_coverage |>
  ggplot(
    aes(
      x = valid_estimation_days
    )
  ) +
  geom_histogram(
    binwidth = 5,
    boundary = 0
  ) +
  geom_vline(
    xintercept =
      minimum_estimation_observations,
    linetype = "dotted"
  ) +
  labs(
    x = "Valid estimation-window observations",
    y = "Events",
    title = "Estimation-Window Data Coverage"
  ) +
  theme_classic()

plot_estimation_coverage

```

6.5 マーケット・モデルの推定

6.5.1 イベント別回帰

```
tbl_market_model <- estimate_market_models_LFL(
  tbl_event_panel = tbl_event_panel,
  estimation_start = estimation_start,
  estimation_end = estimation_end,
  minimum_observations =
    minimum_estimation_observations
)

tbl_market_model |>
  slice_head(n = 20)
```

#> # A tibble: 20 x 13

#>	event_ID	firm_ID	event_date	event_trade_date	event_year	event_strength
#>	<int>	<int>	<date>	<date>	<dbl>	<ord>
#> 1	1	39	2017-01-04	2017-01-05	2017	1
#> 2	2	57	2017-01-04	2017-01-05	2017	1
#> 3	3	73	2017-01-04	2017-01-05	2017	4
#> 4	4	109	2017-01-04	2017-01-05	2017	3
#> 5	5	111	2017-01-04	2017-01-05	2017	3
#> 6	6	231	2017-01-04	2017-01-05	2017	1
#> 7	7	944	2017-01-04	2017-01-05	2017	2

```
#> 8      8      1227 2017-01-04 2017-01-05      2017 5
#> 9      9      422 2017-01-05 2017-01-06      2017 3
#> 10     10     1203 2017-01-05 2017-01-06      2017 5
#> 11     11      346 2017-01-06 2017-01-10      2017 2
#> 12     12      590 2017-01-06 2017-01-10      2017 1
#> 13     13      644 2017-01-06 2017-01-10      2017 2
#> 14     14     1153 2017-01-06 2017-01-10      2017 4
#> 15     15      191 2017-01-10 2017-01-11      2017 4
#> 16     16      353 2017-01-10 2017-01-11      2017 1
#> 17     17      566 2017-01-10 2017-01-11      2017 3
#> 18     18      322 2017-01-11 2017-01-12      2017 3
#> 19     19     1050 2017-01-11 2017-01-12      2017 4
#> 20     20      426 2017-01-12 2017-01-13      2017 5
#> # i 7 more variables: earnings_surprise <dbl>, alpha <dbl>, beta <dbl>,
#> #   sigma_AR <dbl>, r_squared <dbl>, adjusted_r_squared <dbl>,
#> #   estimation_observations <int>
```

6.5.2 推定結果の概要

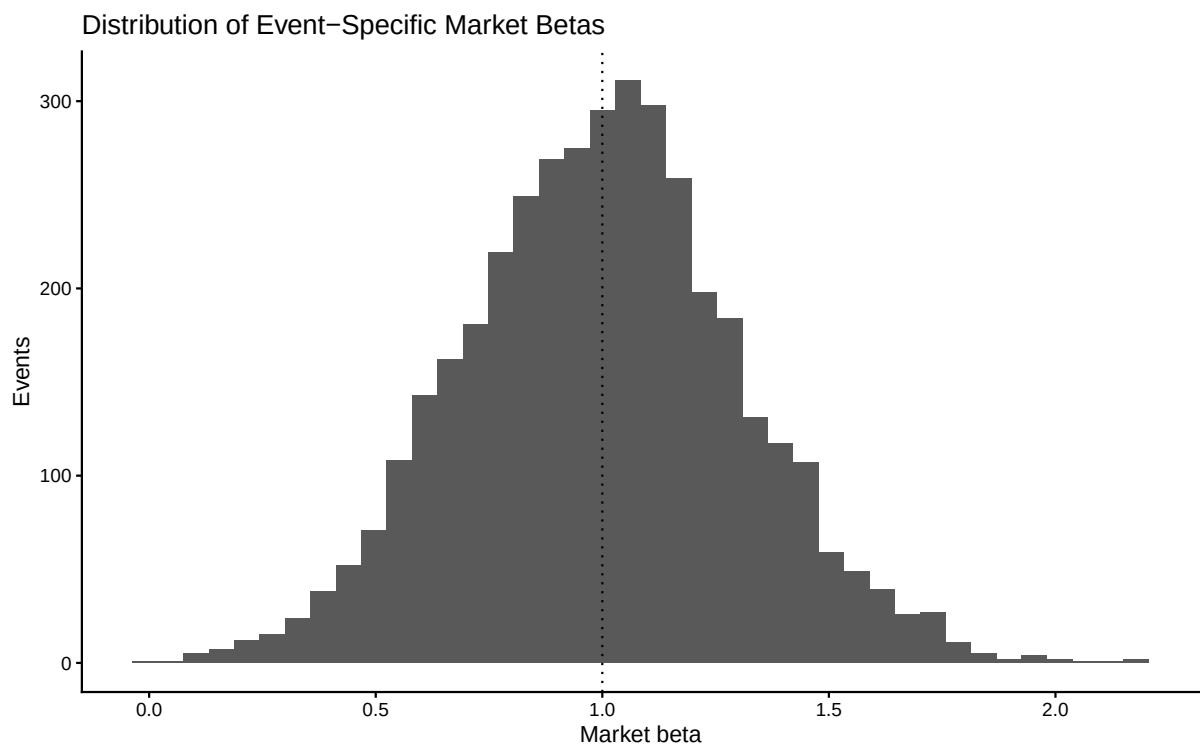
```
tbl_market_model_summary <- tbl_market_model |>
  summarise(
    estimated_events = n(),
    mean_alpha = mean(alpha),
    mean_beta = mean(beta),
    median_beta = median(beta),
    minimum_beta = min(beta),
    maximum_beta = max(beta),
    mean_r_squared = mean(r_squared),
    median_r_squared = median(r_squared),
    mean_sigma_AR = mean(sigma_AR),
    minimum_observations =
      min(estimation_observations),
    maximum_observations =
      max(estimation_observations)
  )

tbl_market_model_summary
#> # A tibble: 1 x 11
#>   estimated_events mean_alpha mean_beta median_beta minimum_beta maximum_beta
#>   <int>          <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1       3960 -0.000017343  0.99751    1.0048  -0.0083393    2.1776
#> # i 5 more variables: mean_r_squared <dbl>, median_r_squared <dbl>,
#> #   mean_sigma_AR <dbl>, minimum_observations <int>, maximum_observations <int>
```

6.5.3 ベータの分布

```
plot_market_beta <- tbl_market_model |>
  ggplot(aes(x = beta)) +
  geom_histogram(bins = 40) +
  geom_vline(
    xintercept = 1,
    linetype = "dotted"
  ) +
  labs(
    x = "Market beta",
    y = "Events",
    title = "Distribution of Event-Specific Market Betas"
  ) +
  theme_classic()

plot_market_beta
```

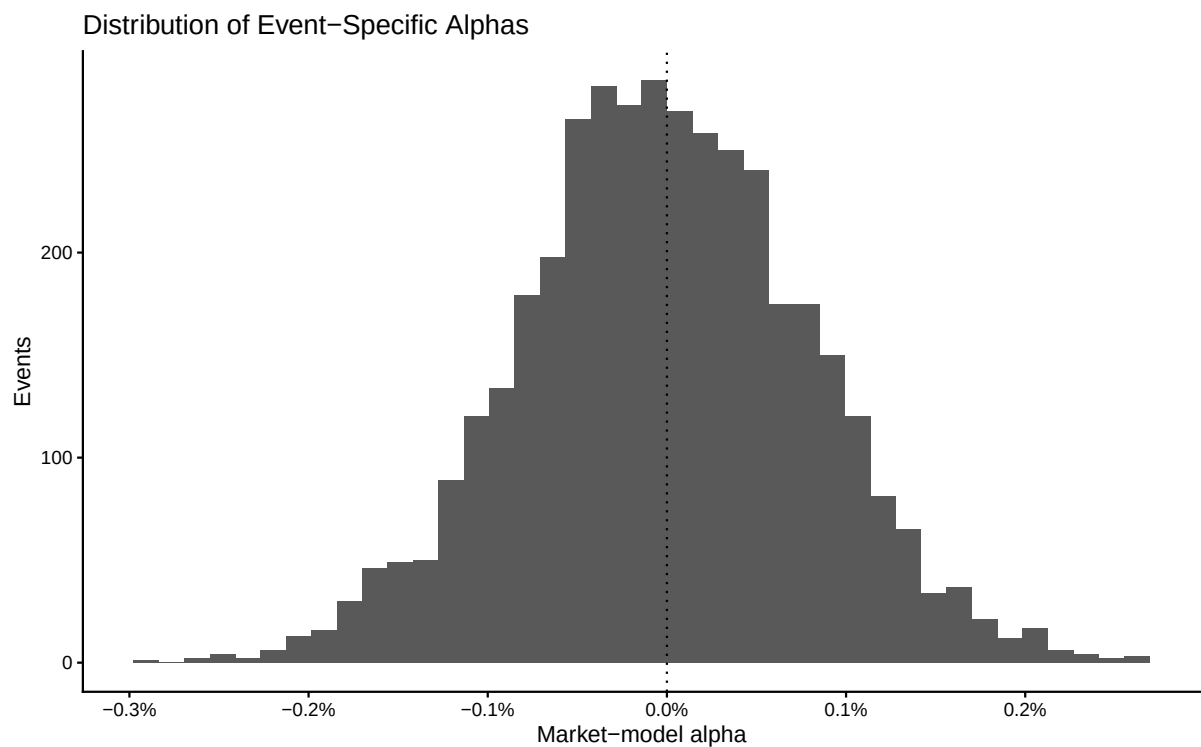


6.5.4 アルファの分布

```
plot_market_alpha <- tbl_market_model |>
  ggplot(aes(x = alpha)) +
  geom_histogram(bins = 40) +
```

```
geom_vline(
  xintercept = 0,
  linetype = "dotted"
) +
scale_x_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Market-model alpha",
  y = "Events",
  title = "Distribution of Event-Specific Alphas"
) +
theme_classic()

plot_market_alpha
```



6.5.5 決定係数の分布

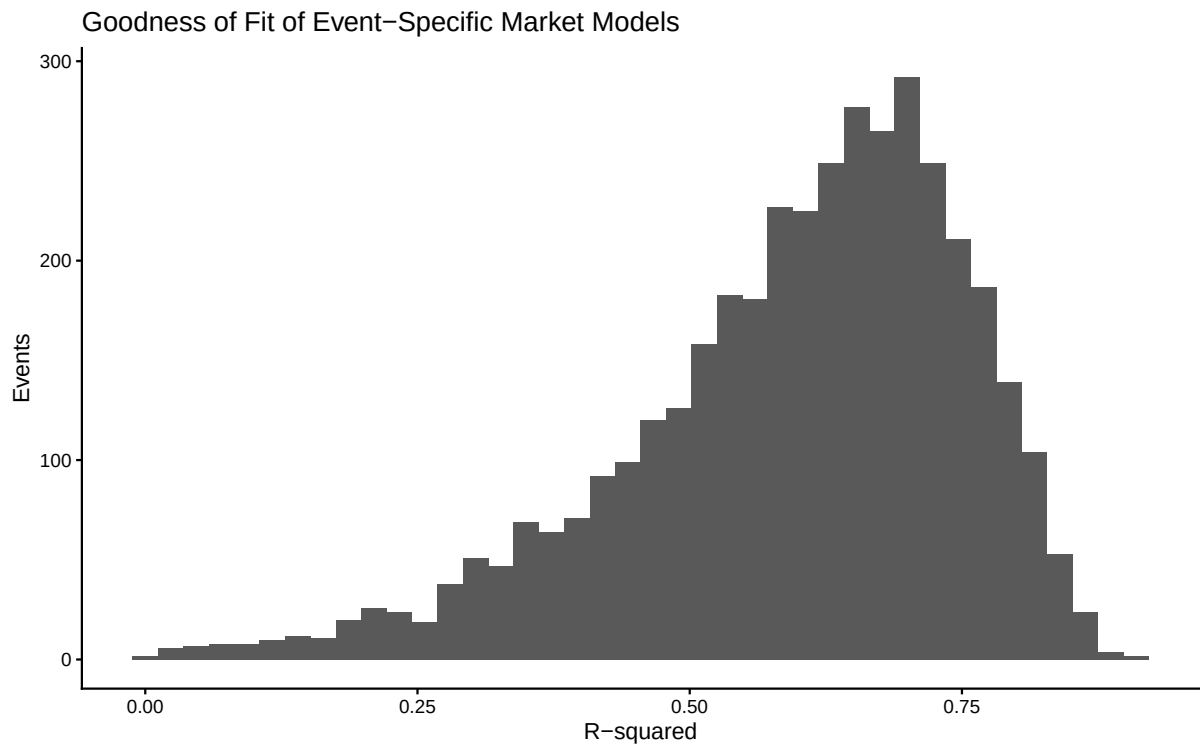
```
plot_market_model_r_squared <- tbl_market_model |>
ggplot(aes(x = r_squared)) +
geom_histogram(bins = 40) +
labs(
  x = "R-squared",
  y = "Events",
```

```

    title = "Goodness of Fit of Event-Specific Market Models"
  ) +
  theme_classic()

plot_market_model_r_squared

```



6.5.6 ベータと決定係数

```

plot_beta_r_squared <- tbl_market_model |>
  ggplot(
    aes(
      x = beta,
      y = r_squared,
      shape = event_strength
    )
  ) +
  geom_point(alpha = 0.55) +
  geom_vline(
    xintercept = 1,
    linetype = "dotted"
  ) +
  labs(
    x = "Market beta",
    y = "R-squared",

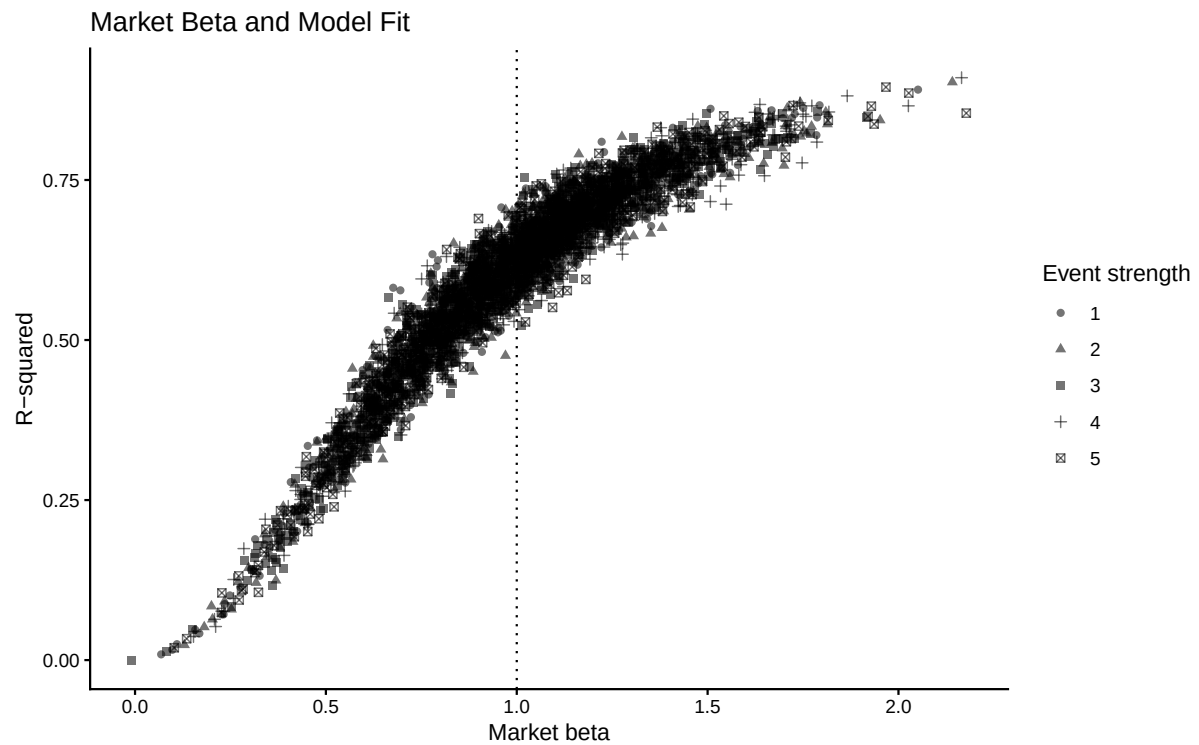
```

```

    shape = "Event strength",
    title = "Market Beta and Model Fit"
  ) +
  theme_classic()

plot_beta_r_squared

```



6.6 異常リターンと累積異常リターン

6.6.1 AR と CAR

```

tbl_abnormal_return <- compute_abnormal_returns_LFL(
  tbl_event_panel = tbl_event_panel,
  tbl_market_model = tbl_market_model,
  event_start = event_start,
  event_end = event_end,
  require_complete_window = TRUE
)

tbl_abnormal_return |>
  select(
    event_ID,
    firm_ID,
    event_strength,

```

```

    relative_day,
    date,
    R,
    R_M,
    normal_return,
    AR,
    CAR
  ) |>
  slice_head(n = 20)
#> # A tibble: 20 x 10
#>   event_ID firm_ID event_strength relative_day date          R          R_M
#>   <int>   <int> <ord>          <int> <date>        <dbl>        <dbl>
#> 1         1     39 1          -30 2016-11-18 -0.0013155 -1.4725e-3
#> 2         1     39 1          -29 2016-11-21  0.0037198 -3.3404e-3
#> 3         1     39 1          -28 2016-11-22  0.021021  1.7138e-2
#> 4         1     39 1          -27 2016-11-24  0.0030893  1.5701e-2
#> 5         1     39 1          -26 2016-11-25 -0.0062645 -2.3346e-2
#> 6         1     39 1          -25 2016-11-28 -0.0040561 -6.5350e-3
#> 7         1     39 1          -24 2016-11-29  0.012325  -4.1437e-3
#> 8         1     39 1          -23 2016-11-30 -0.0096488  2.9259e-3
#> 9         1     39 1          -22 2016-12-01  0.014048  4.7329e-4
#> 10        1     39 1          -21 2016-12-02  0.012192  1.0758e-2
#> 11        1     39 1          -20 2016-12-05  0.0097831 -5.4821e-3
#> 12        1     39 1          -19 2016-12-06  0.026133  1.8634e-2
#> 13        1     39 1          -18 2016-12-07 -0.0044072 -7.3350e-3
#> 14        1     39 1          -17 2016-12-08  0.015023  2.7649e-2
#> 15        1     39 1          -16 2016-12-09  0.0074484 -1.4362e-3
#> 16        1     39 1          -15 2016-12-12  0.011444 -6.4222e-3
#> 17        1     39 1          -14 2016-12-13 -0.011043 -1.4330e-2
#> 18        1     39 1          -13 2016-12-14  0.019488  2.2428e-2
#> 19        1     39 1          -12 2016-12-15  0.0054953  1.8578e-2
#> 20        1     39 1          -11 2016-12-16 -0.0066719 -3.4403e-3
#> # i 3 more variables: normal_return <dbl>, AR <dbl>, CAR <dbl>

```

6.6.2 AAR と CAAR

イベント強度 g に属するイベント数を N_g とすると、平均異常リターンは次式で表される。

$$AAR_{g,t} = \frac{1}{N_g} \sum_{i \in g} AR_{i,t}$$

平均累積異常リターンは次式で表される。

$$CAAR_{g,t} = \frac{1}{N_g} \sum_{i \in g} CAR_{i,t}$$

```
significance_stars <- function(p_value) {
  case_when(
    is.na(p_value) ~ "",
    p_value < 0.01 ~ "***",
    p_value < 0.05 ~ "**",
    p_value < 0.10 ~ "*",
    TRUE ~ ""
  )
}
```

```
tbl_daily_statistics <- tbl_abnormal_return |>
  group_by(
    relative_day,
    event_strength
  ) |>
  summarise(
    events = n_distinct(event_ID),
    AAR = mean(AR),
    CAAR = mean(CAR),
    standard_deviation_AR = sd(AR),
    standard_deviation_CAR = sd(CAR),
    .groups = "drop"
  ) |>
  mutate(
    standard_error_AAR =
      standard_deviation_AR /
      sqrt(events),
    standard_error_CAAR =
      standard_deviation_CAR /
      sqrt(events),
    t_AAR = if_else(
      is.finite(standard_error_AAR) &
        standard_error_AAR > 0,
      AAR / standard_error_AAR,
      NA_real_
    ),
    t_CAAR = if_else(
      is.finite(standard_error_CAAR) &
        standard_error_CAAR > 0,
      CAAR / standard_error_CAAR,
      NA_real_
    )
  )
```



```

),
p_AAR = if_else(
  is.finite(t_AAR),
  2 * pt(
    -abs(t_AAR),
    df = events - 1L
  ),
  NA_real_
),
p_CAAR = if_else(
  is.finite(t_CAAR),
  2 * pt(
    -abs(t_CAAR),
    df = events - 1L
  ),
  NA_real_
),
significance_AAR =
  significance_stars(p_AAR),
significance_CAAR =
  significance_stars(p_CAAR)
) |>
arrange(
  event_strength,
  relative_day
)

tbl_daily_statistics
#> # A tibble: 305 x 15
#>   relative_day event_strength events      AAR      CAAR
#>   <int> <ord>      <int>    <dbl>    <dbl>
#> 1      -30 1          793 -0.00033279 -0.00033279
#> 2      -29 1          793  0.00034601  0.000013216
#> 3      -28 1          793 -0.000030469 -0.000017253
#> 4      -27 1          793  0.00024231  0.00022506
#> 5      -26 1          793  0.00042018  0.00064524
#> 6      -25 1          793  0.00020818  0.00085342
#> 7      -24 1          793  0.00053366  0.0013871
#> 8      -23 1          793 -0.000028460  0.0013586
#> 9      -22 1          793 -0.00014238  0.0012162
#> 10     -21 1          793 -0.00019155  0.0010247
#> # i 295 more rows
#> # i 10 more variables: standard_deviation_AR <dbl>,

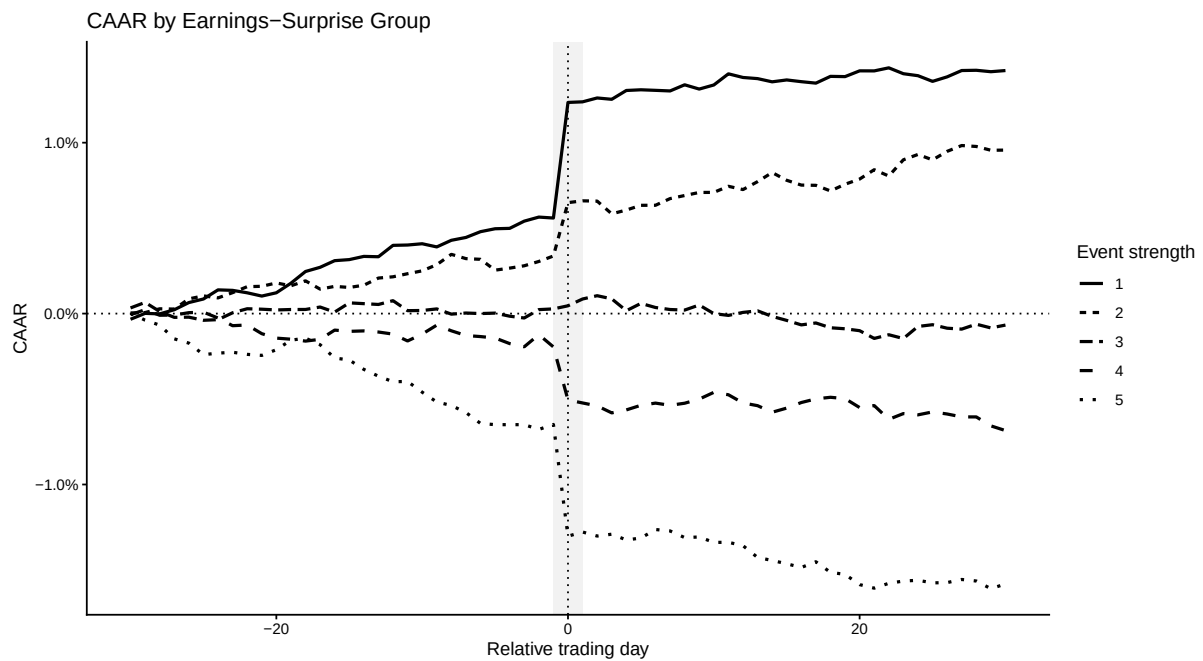
```

```
#> #   standard_deviation_CAR <dbl>, standard_error_AAR <dbl>,
#> #   standard_error_CAAR <dbl>, t_AAR <dbl>, t_CAAR <dbl>, p_AAR <dbl>,
#> #   p_CAAR <dbl>, significance_AAR <chr>, significance_CAAR <chr>
```

6.6.3 イベント強度別 CAAR

```
plot_caar_by_strength <- tbl_daily_statistics |>
  ggplot(
    aes(
      x = relative_day,
      y = CAAR,
      linetype = event_strength
    )
  ) +
  annotate(
    "rect",
    xmin = -1,
    xmax = 1,
    ymin = -Inf,
    ymax = Inf,
    alpha = 0.08
  ) +
  geom_line(linewidth = 0.9) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Relative trading day",
    y = "CAAR",
    linetype = "Event strength",
    title = "CAAR by Earnings-Surprise Group"
  ) +
  theme_classic()

plot_caar_by_strength
```



6.6.4 イベント日前後の AAR

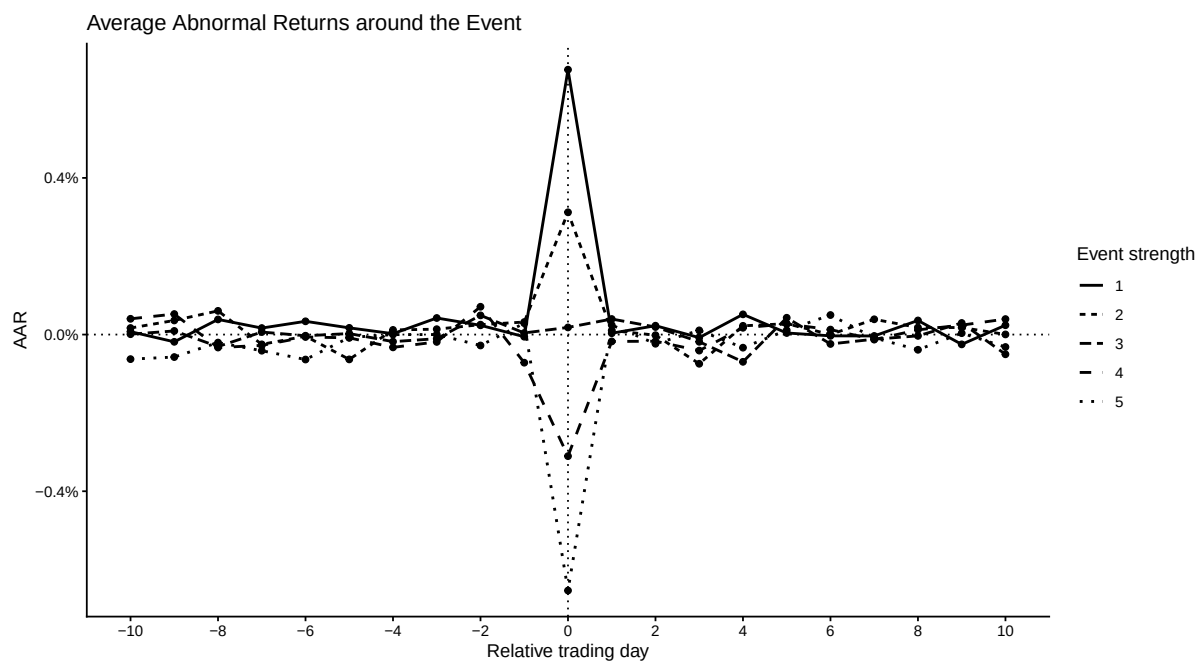
```
plot_aar_around_event <- tbl_daily_statistics |>
  filter(
    between(
      relative_day,
      -10L,
      10L
    )
  ) |>
  ggplot(
    aes(
      x = relative_day,
      y = AAR,
      linetype = event_strength
    )
  ) +
  geom_line(linewidth = 0.8) +
  geom_point() +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  geom_hline(
    yintercept = 0,
```

```

    linetype = "dotted"
  ) +
  scale_x_continuous(
    breaks = seq(-10, 10, 2)
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Relative trading day",
    y = "AAR",
    linetype = "Event strength",
    title = "Average Abnormal Returns around the Event"
  ) +
  theme_classic()

plot_aar_around_event

```



6.6.5 イベント日 0 の AAR

```

plot_event_day_aar <- tbl_daily_statistics |>
  filter(relative_day == 0L) |>
  ggplot(
    aes(
      x = event_strength,
      y = AAR
    )
  )

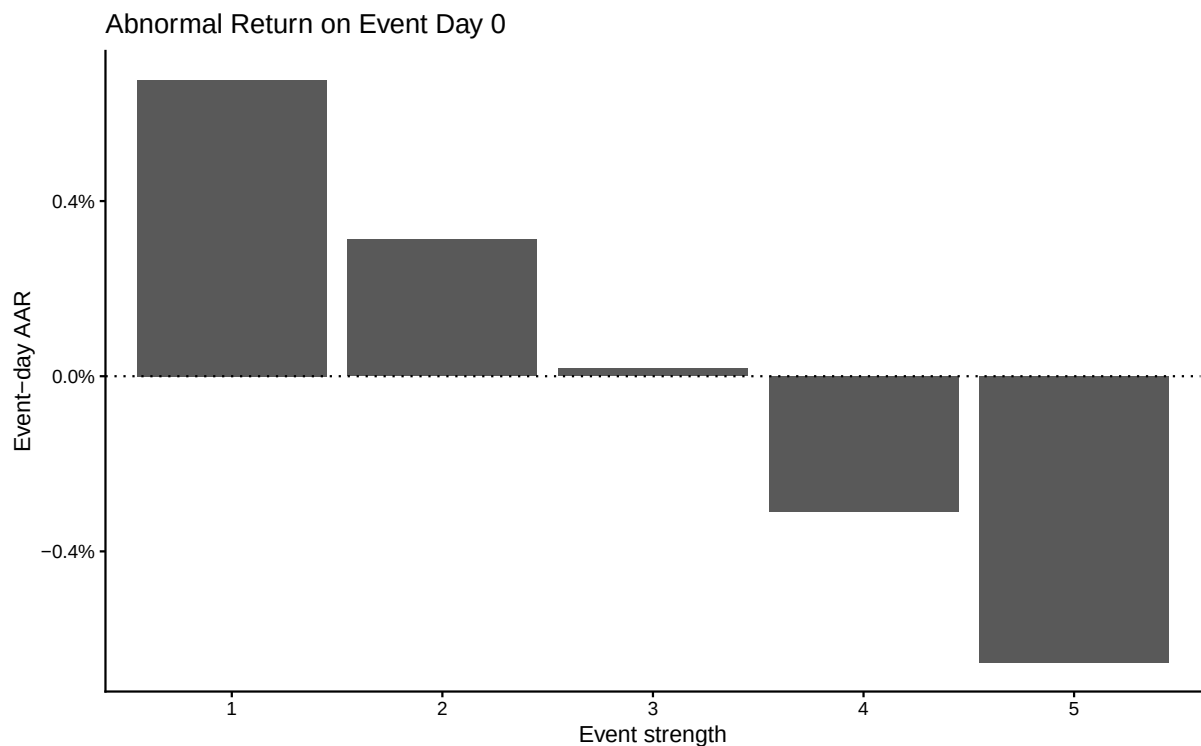
```

```

    )
  ) +
  geom_col() +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Event strength",
    y = "Event-day AAR",
    title = "Abnormal Return on Event Day 0"
  ) +
  theme_classic()

plot_event_day_aar

```



6.6.6 イベント日後の CAAR

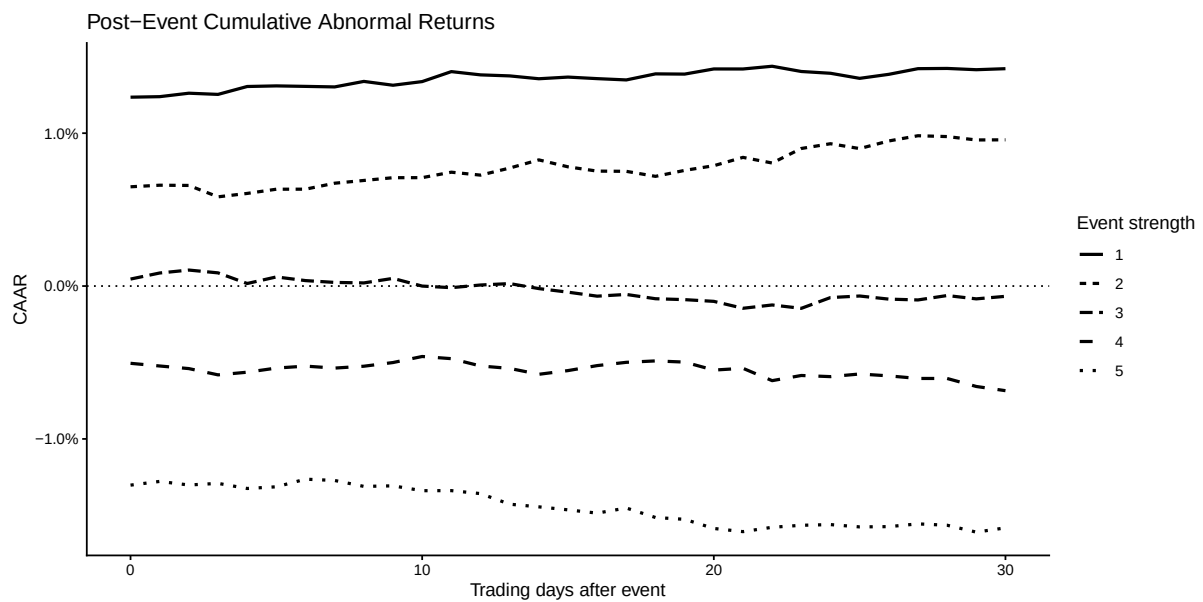
```

plot_post_event_caar <- tbl_daily_statistics |>
  filter(
    between(

```

```
    relative_day,
    0L,
    30L
  )
) |>
ggplot(
  aes(
    x = relative_day,
    y = CAAR,
    linetype = event_strength
  )
) +
geom_line(linewidth = 0.9) +
geom_hline(
  yintercept = 0,
  linetype = "dotted"
) +
scale_y_continuous(
  labels = scales::label_percent()
) +
labs(
  x = "Trading days after event",
  y = "CAAR",
  linetype = "Event strength",
  title = "Post-Event Cumulative Abnormal Returns"
) +
theme_classic()

plot_post_event_caar
```



6.6.7 イベント強度別 AR 分布

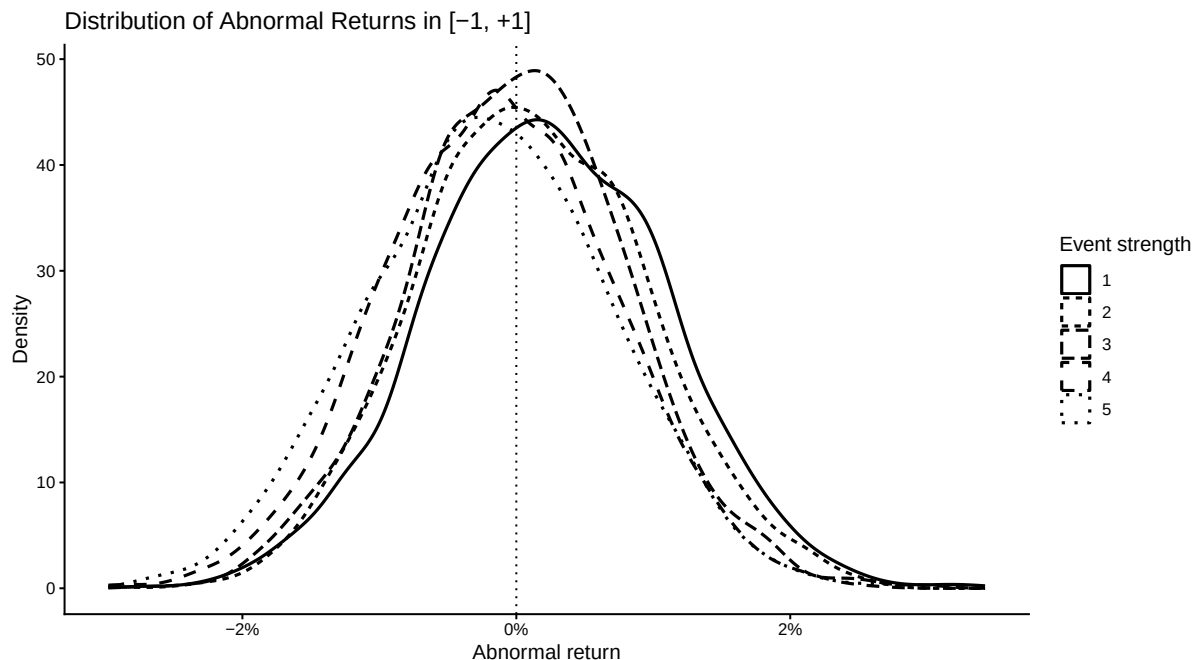
```
plot_ar_distribution_by_strength <- tbl_abnormal_return |>
  filter(
    between(
      relative_day,
      -1L,
      1L
    )
  ) |>
  ggplot(
    aes(
      x = AR,
      linetype = event_strength
    )
  ) +
  geom_density(linewidth = 0.8) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted"
  ) +
  scale_x_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Abnormal return",
    y = "Density",
    linetype = "Event strength",
  )
```

```

    title = "Distribution of Abnormal Returns in [-1, +1]"
  ) +
  theme_classic()

plot_ar_distribution_by_strength

```



6.7 イベント・ウィンドウ分析

6.7.1 ウィンドウ定義

```

tbl_summary_window <- tribble(
  ~window_ID, ~window_label, ~start_day, ~end_day,
  "m1_p1", "[-1, +1]", -1L, 1L,
  "0_p1", "[0, +1]", 0L, 1L,
  "m5_p5", "[-5, +5]", -5L, 5L,
  "m10_p10", "[-10, +10]", -10L, 10L,
  "m30_p30", "[-30, +30]", -30L, 30L
)

```

```

tbl_summary_window
#> # A tibble: 5 x 4
#>   window_ID window_label start_day end_day
#>   <chr>      <chr>         <int>  <int>
#> 1 m1_p1     [-1, +1]           -1      1
#> 2 0_p1      [0, +1]            0      1
#> 3 m5_p5     [-5, +5]          -5      5

```



```
#> 4 m10_p10 [-10, +10] -10 10
#> 5 m30_p30 [-30, +30] -30 30
```

6.7.2 イベント別 CAR

```
tbl_event_window_return <-
  compute_event_window_returns_LFL(
    tbl_abnormal_return = tbl_abnormal_return,
    tbl_window = tbl_summary_window
  )

tbl_event_window_return |>
  slice_head(n = 20)
#> # A tibble: 20 x 14
#>   event_ID firm_ID event_date event_trade_date event_year event_strength
#>   <int>   <int> <date>      <date>          <dbl> <ord>
#> 1     1     1     39 2017-01-04 2017-01-05      2017 1
#> 2     2     2     57 2017-01-04 2017-01-05      2017 1
#> 3     3     3     73 2017-01-04 2017-01-05      2017 4
#> 4     4     4    109 2017-01-04 2017-01-05      2017 3
#> 5     5     5    111 2017-01-04 2017-01-05      2017 3
#> 6     6     6    231 2017-01-04 2017-01-05      2017 1
#> 7     7     7    944 2017-01-04 2017-01-05      2017 2
#> 8     8     8   1227 2017-01-04 2017-01-05      2017 5
#> 9     9     9    422 2017-01-05 2017-01-06      2017 3
#> 10    10    1203 2017-01-05 2017-01-06      2017 5
#> 11    11    346 2017-01-06 2017-01-10      2017 2
#> 12    12    590 2017-01-06 2017-01-10      2017 1
#> 13    13    644 2017-01-06 2017-01-10      2017 2
#> 14    14   1153 2017-01-06 2017-01-10      2017 4
#> 15    15    191 2017-01-10 2017-01-11      2017 4
#> 16    16    353 2017-01-10 2017-01-11      2017 1
#> 17    17    566 2017-01-10 2017-01-11      2017 3
#> 18    18    322 2017-01-11 2017-01-12      2017 3
#> 19    19   1050 2017-01-11 2017-01-12      2017 4
#> 20    20    426 2017-01-12 2017-01-13      2017 5
#> # i 8 more variables: earnings_surprise <dbl>, window_ID <chr>,
#> #   window_label <chr>, start_day <int>, end_day <int>, expected_days <int>,
#> #   CAR_window <dbl>, observed_days <int>
```

6.7.3 イベント強度別の平均 CAR

```
tbl_event_window_summary <- tbl_event_window_return |>
  group_by(
    window_ID,
    window_label,
    start_day,
    end_day,
    event_strength
  ) |>
  summarise(
    events = n_distinct(event_ID),
    mean_CAR = mean(CAR_window),
    median_CAR = median(CAR_window),
    standard_deviation_CAR =
      sd(CAR_window),
    positive_CAR_ratio =
      mean(CAR_window > 0),
    .groups = "drop"
  ) |>
  mutate(
    standard_error_CAR =
      standard_deviation_CAR /
      sqrt(events),
    t_value = if_else(
      is.finite(standard_error_CAR) &
        standard_error_CAR > 0,
      mean_CAR /
        standard_error_CAR,
      NA_real_
    ),
    p_value = if_else(
      is.finite(t_value),
      2 * pt(
        -abs(t_value),
        df = events - 1L
      ),
      NA_real_
    ),
    significance =
      significance_stars(p_value)
  ) |>
```

```

arrange(
  window_ID,
  event_strength
)

tbl_event_window_summary
#> # A tibble: 25 x 14
#>   window_ID window_label start_day end_day event_strength events mean_CAR
#>   <chr>      <chr>          <int>  <int> <ord>          <int>    <dbl>
#> 1 0_p1      [0, +1]            0      1 1           793  0.0067977
#> 2 0_p1      [0, +1]            0      1 2           793  0.0032266
#> 3 0_p1      [0, +1]            0      1 3           792  0.00057929
#> 4 0_p1      [0, +1]            0      1 4           791 -0.0032795
#> 5 0_p1      [0, +1]            0      1 5           791 -0.0062931
#> 6 m10_p10  [-10, +10]          -10     10 1           793  0.0093638
#> 7 m10_p10  [-10, +10]          -10     10 2           793  0.0047579
#> 8 m10_p10  [-10, +10]          -10     10 3           792 -0.00017347
#> 9 m10_p10  [-10, +10]          -10     10 4           791 -0.0030068
#> 10 m10_p10 [-10, +10]          -10     10 5           791 -0.0094161
#> # i 15 more rows
#> # i 7 more variables: median_CAR <dbl>, standard_deviation_CAR <dbl>,
#> #   positive_CAR_ratio <dbl>, standard_error_CAR <dbl>, t_value <dbl>,
#> #   p_value <dbl>, significance <chr>

```

6.7.4 短期 CAR の分布

```

plot_short_window_car <- tbl_event_window_return |>
  filter(window_ID == "m1_p1") |>
  ggplot(
    aes(
      x = event_strength,
      y = CAR_window
    )
  ) +
  geom_boxplot(
    outlier.alpha = 0.25
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  )

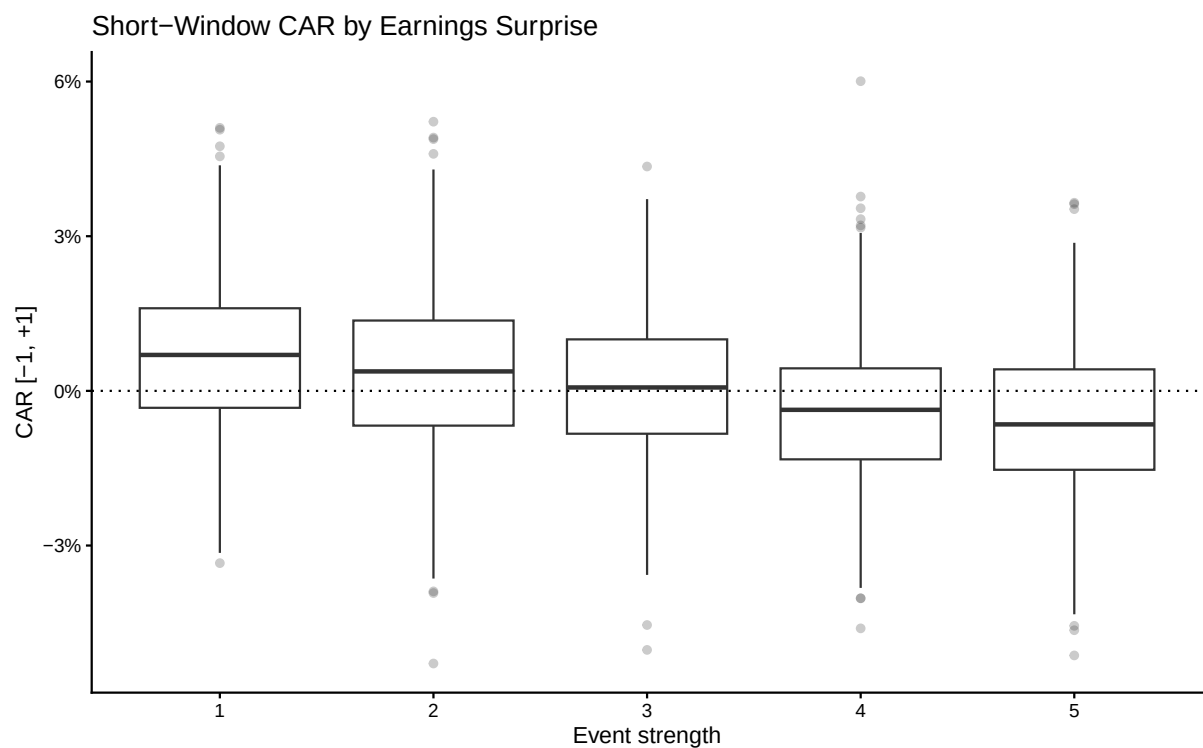
```

```

) +
labs(
  x = "Event strength",
  y = "CAR [-1, +1]",
  title = "Short-Window CAR by Earnings Surprise"
) +
theme_classic()

plot_short_window_car

```



6.7.5 ウィンドウ別平均 CAR

```

plot_window_mean_car <- tbl_event_window_summary |>
ggplot(
  aes(
    x = event_strength,
    y = mean_CAR,
    fill = window_label
  )
) +
geom_col(
  position = "dodge"
) +
geom_hline(

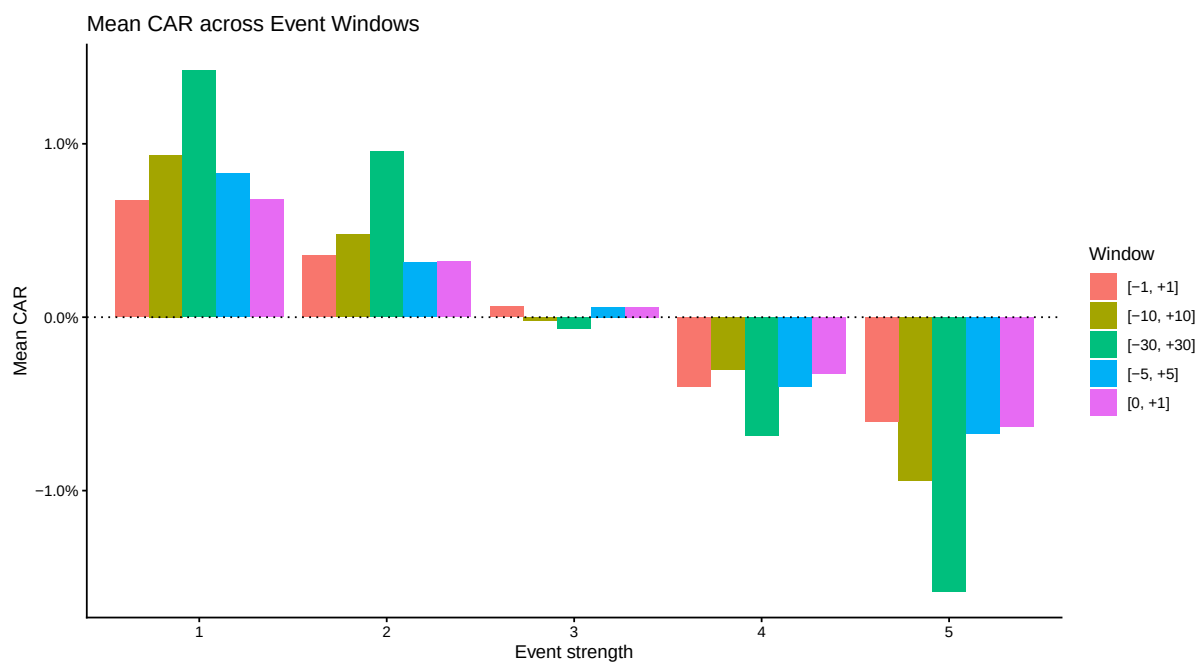
```

```

    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Event strength",
    y = "Mean CAR",
    fill = "Window",
    title = "Mean CAR across Event Windows"
  ) +
  theme_classic()

plot_window_mean_car

```



6.7.6 正の CAR となったイベントの割合

```

plot_positive_car_ratio <- tbl_event_window_summary |>
  ggplot(
    aes(
      x = event_strength,
      y = positive_CAR_ratio,
      fill = window_label
    )
  ) +

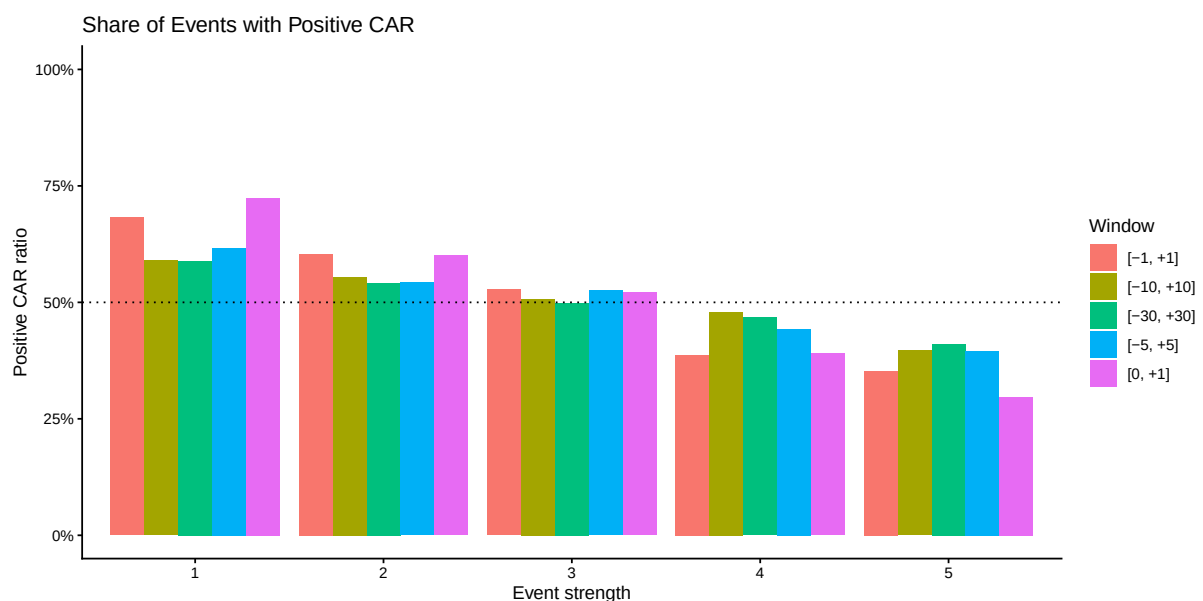
```

```

geom_col(
  position = "dodge"
) +
geom_hline(
  yintercept = 0.5,
  linetype = "dotted"
) +
scale_y_continuous(
  labels = scales::label_percent(),
  limits = c(0, 1)
) +
labs(
  x = "Event strength",
  y = "Positive CAR ratio",
  fill = "Window",
  title = "Share of Events with Positive CAR"
) +
theme_classic()

plot_positive_car_ratio

```



6.7.7 平均 CAR のヒートマップ

```

plot_window_car_heatmap <- tbl_event_window_summary |>
ggplot(
  aes(
    x = event_strength,
    y = window_label,

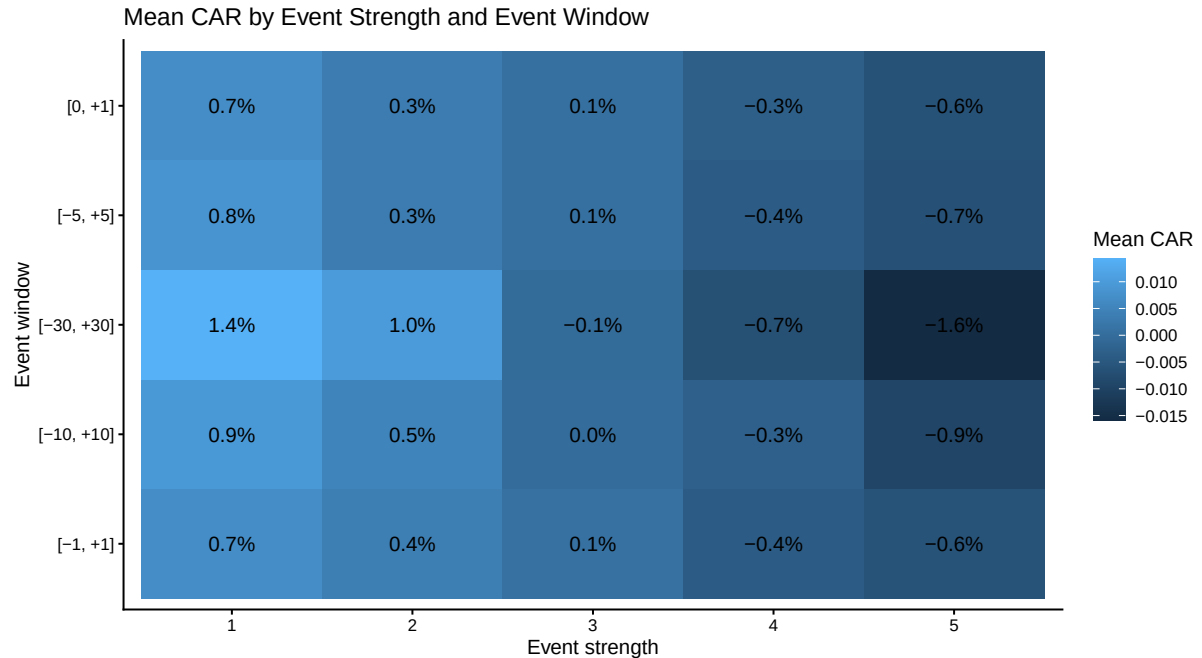
```

```

    fill = mean_CAR
  )
) +
geom_tile() +
geom_text(
  aes(
    label = scales::percent(
      mean_CAR,
      accuracy = 0.1
    )
  )
) +
labs(
  x = "Event strength",
  y = "Event window",
  fill = "Mean CAR",
  title = "Mean CAR by Event Strength and Event Window"
) +
theme_classic()

plot_window_car_heatmap

```



6.8 決算サプライズと短期 CAR

6.8.1 回帰分析

```
tbl_surprise_regression <- tbl_event_window_return |>
  filter(
    window_ID == "m1_p1",
    is.finite(earnings_surprise),
    is.finite(CAR_window)
  )

model_surprise_car <- lm(
  CAR_window ~ earnings_surprise,
  data = tbl_surprise_regression
)

broom::tidy(model_surprise_car)
#> # A tibble: 2 x 5
#>   term                estimate std.error statistic    p.value
#>   <chr>                <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept)        -0.0066774 0.00041568  -16.064 2.6646e-56
#> 2 earnings_surprise  -1.3714    0.069096   -19.848 1.1933e-83
broom::glance(model_surprise_car)
#> # A tibble: 1 x 12
#>   r.squared adj.r.squared  sigma statistic    p.value    df logLik    AIC
#>   <dbl>      <dbl>      <dbl>     <dbl>    <dbl> <dbl> <dbl> <dbl>
#> 1  0.090519    0.090289 0.014511   393.93 1.1933e-83     1 11144. -22282.
#> # i 4 more variables: BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>
```

6.8.2 決算サプライズと短期 CAR

```
plot_surprise_car <- tbl_surprise_regression |>
  ggplot(
    aes(
      x = earnings_surprise,
      y = CAR_window,
      shape = event_strength
    )
  ) +
  geom_point(alpha = 0.45) +
  geom_smooth(
    method = "lm",
```

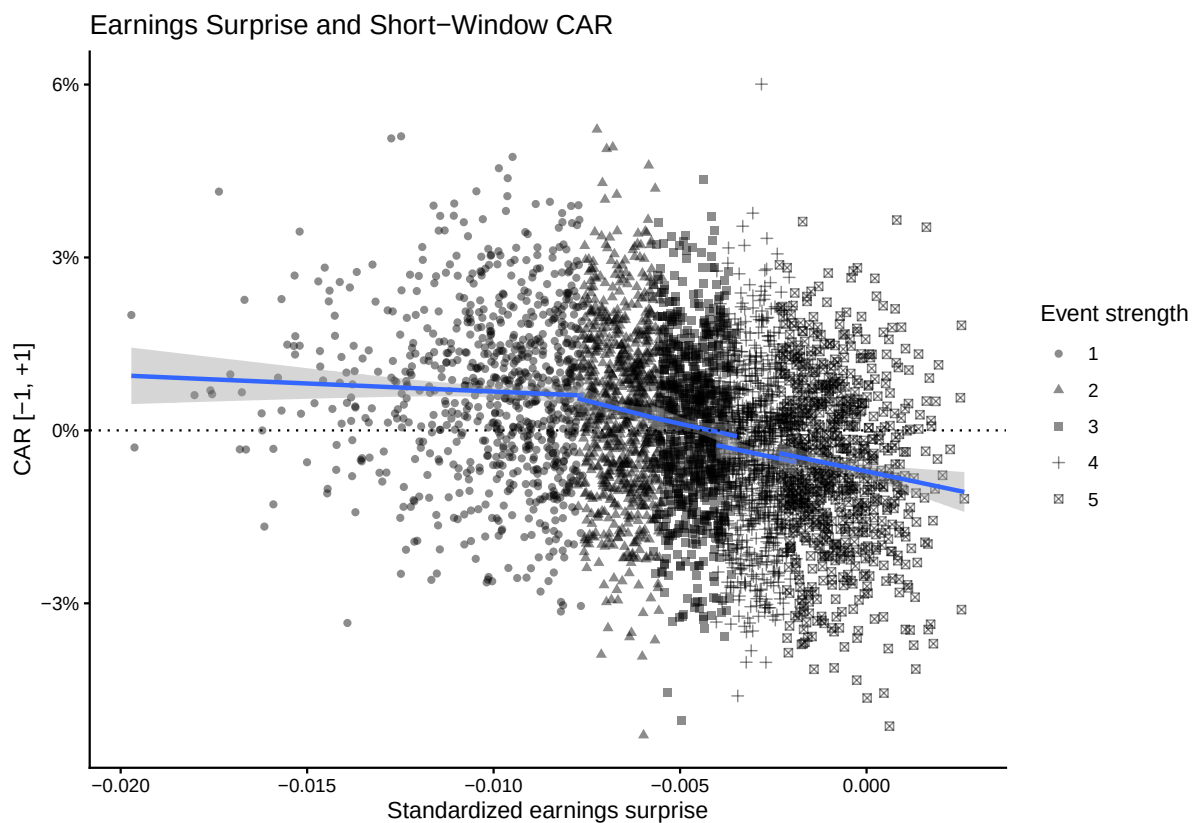


```

    se = TRUE
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Standardized earnings surprise",
    y = "CAR [-1, +1]",
    shape = "Event strength",
    title = "Earnings Surprise and Short-Window CAR"
  ) +
  theme_classic()

plot_surprise_car

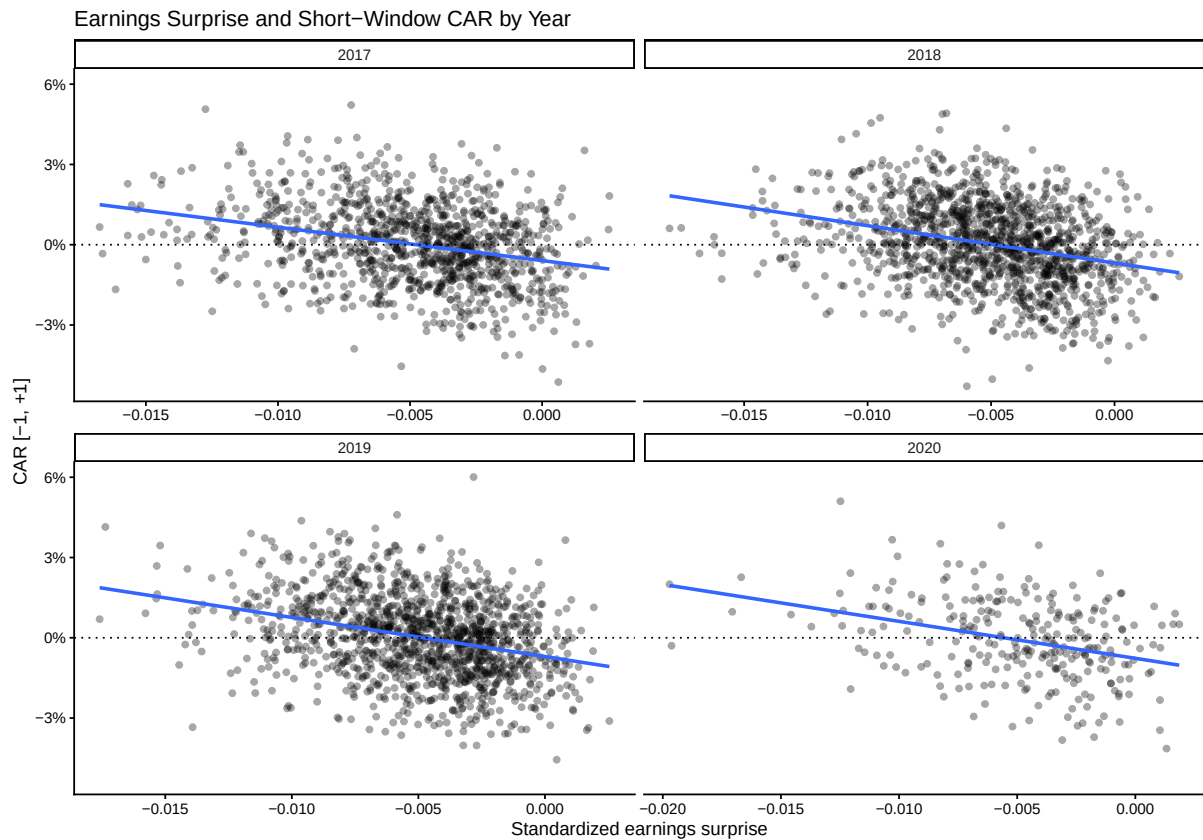
```



6.8.3 年度別の関係

```
plot_surprise_car_by_year <- tbl_surprise_regression |>
  ggplot(
    aes(
      x = earnings_surprise,
      y = CAR_window
    )
  ) +
  geom_point(alpha = 0.35) +
  geom_smooth(
    method = "lm",
    se = FALSE
  ) +
  facet_wrap(
    vars(event_year),
    scales = "free_x"
  ) +
  geom_hline(
    yintercept = 0,
    linetype = "dotted"
  ) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = "Standardized earnings surprise",
    y = "CAR [-1, +1]",
    title = "Earnings Surprise and Short-Window CAR by Year"
  ) +
  theme_classic()

plot_surprise_car_by_year
```



6.9 分析結果の保存

6.9.1 イベント単位の結果

```
tbl_event_window_wide <- tbl_event_window_return |>
  select(
    event_ID,
    window_ID,
    CAR_window
  ) |>
  pivot_wider(
    names_from = window_ID,
    values_from = CAR_window,
    names_prefix = "CAR_"
  )

tbl_event_output <- tbl_market_model |>
  semi_join(
    tbl_abnormal_return |>
      distinct(event_ID),
    by = "event_ID"
  ) |>
```

```
left_join(  
  tbl_event_window_wide,  
  by = "event_ID"  
) |>  
arrange(event_ID)
```

6.9.2 日次結果

```
tbl_event_daily_output <- tbl_abnormal_return |>  
  select(  
    event_ID,  
    firm_ID,  
    event_date,  
    event_trade_date,  
    event_year,  
    event_strength,  
    earnings_surprise,  
    relative_day,  
    date,  
    R,  
    R_M,  
    normal_return,  
    AR,  
    CAR,  
    alpha,  
    beta,  
    sigma_AR,  
    r_squared,  
    estimation_observations  
  )  
  
tbl_event_summary_output <- tbl_daily_statistics |>  
  arrange(  
    event_strength,  
    relative_day  
  )
```

6.9.3 Parquet 保存

```
list_output <- list(  
  event_results = tbl_event_output,  
  event_daily = tbl_event_daily_output,
```

```

    event_summary = tbl_event_summary_output
  )

vec_output_file <- c(
  event_results = file.path(
    "events",
    "event_study_results.parquet"
  ),
  event_daily = file.path(
    "events",
    "event_study_daily.parquet"
  ),
  event_summary = file.path(
    "events",
    "event_study_summary.parquet"
  )
)

vec_output_path <- purrr::imap_chr(
  list_output,
  \(tbl_output, output_name)
    write_derived_parquet_file_LFL(
      tbl_output,
      vec_output_file[[output_name]]
    )
)

tibble(
  output_name = names(vec_output_path),
  path = relative_path_LFL(base::unname(vec_output_path))
)

#> # A tibble: 3 x 2
#>   output_name  path
#>   <chr>      <chr>
#> 1 event_results data/derived/events/event_study_results.parquet
#> 2 event_daily  data/derived/events/event_study_daily.parquet
#> 3 event_summary data/derived/events/event_study_summary.parquet

```

6.9.4 保存結果の検証

```

tbl_output_verification <- purrr::imap_dfr(
  list_output,
  \(tbl_expected, output_name) {

```

```

output_file <- vec_output_file[[output_name]]
output_path <- vec_output_path[[output_name]]

tbl_check <-
  read_derived_parquet_file_LFL(
    output_file
  )

stopifnot(
  nrow(tbl_check) ==
    nrow(tbl_expected)
)

stopifnot(
  identical(
    names(tbl_check),
    names(tbl_expected)
  )
)

tibble(
  output_name = output_name,
  output_file = output_path,
  rows = nrow(tbl_check),
  columns = ncol(tbl_check),
  size_bytes =
    file.info(output_path)$size
)
}
)

tbl_output_verification
#> # A tibble: 3 x 5
#>   output_name  output_file      rows columns size_bytes
#>   <chr>        <chr>      <int>  <int>    <dbl>
#> 1 event_results C:/AnalyticFin/Projects/LagrangeFinan~ 3960    18    467503
#> 2 event_daily   C:/AnalyticFin/Projects/LagrangeFinan~ 241560   19    9222492
#> 3 event_summary C:/AnalyticFin/Projects/LagrangeFinan~ 305     15     33564

```

6.10 まとめ

本章では、決算発表の翌取引日をイベント日 0 として、マーケット・モデルによるイベントスタディを実施した。推定期間にはイベント日の 130 取引日前から 31 取引日前までの 100 日を使用し、イベント期間には 30 取

引日前から 30 取引日後までの 61 日を使用した。

決算サプライズは、実現利益と予想利益の差を期首時価総額で標準化し、年度ごとに 5 群へ分類した。イベント別にマーケット・モデルを推定し、通常リターン、異常リターン、累積異常リターンを計算した。さらに、日次の AAR と CAAR、複数イベント・ウィンドウの CAR、正の CAR となったイベントの割合、決算サプライズと短期 CAR の回帰関係を確認した。

反復処理には `purrr` を利用し、明示的なループは使用していない。データ探索、入力検証、イベント日 0 の設定、イベント・パネルの作成、マーケット・モデルの推定、AR・CAR の計算、イベント・ウィンドウ集計は、`utility_LFL.R` の共通関数へ分離した。

結果は次の 3 ファイルに保存される。

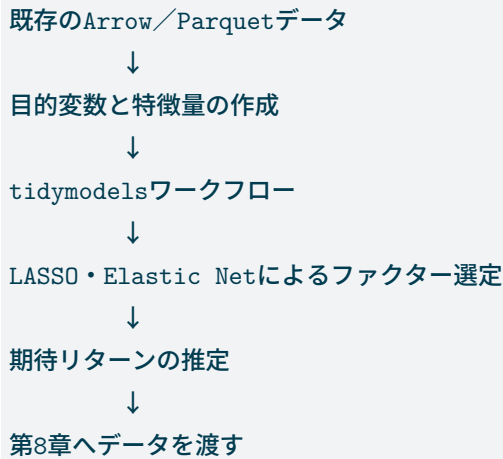
```
data/derived/events/event_study_results.parquet
data/derived/events/event_study_daily.parquet
data/derived/events/event_study_summary.parquet
```

第 1-7 章機械学習によるファクター選定

本章では、第 2 章から第 6 章までに作成した財務データ、株式リターン、ファクター・データおよびイベントスタディ結果を利用し、翌期リターンの予測に有効なファクターを機械学習によって選定する。

分析には `tidymodels` を用いる。目的は複雑なモデルを競わせることではなく、データ準備、前処理、学習、評価、予測という一連の流れを、再利用可能なワークフローとして理解することである。

本章では、次の順序で処理を進める。



7.1 機械学習によるファクター分析

第 4 章では、企業規模と簿価時価比率から SMB と HML を構築した。第 5 章では、それらのファクターを使って企業別の期待リターンを推定した。

本章では、あらかじめ少数の変数だけを選ぶのではなく、財務、価格、リスク、市場環境に関する複数の候補から、翌期リターンの予測に有効な変数を選定する。

LASSO は、回帰係数に罰則を加え、一部の係数を 0 にする。

$$\min_{\beta} \left[\frac{1}{N} \sum_{i=1}^N (y_i - x_i^{\top} \beta)^2 + \lambda \sum_{j=1}^p |\beta_j| \right]$$

係数が 0 にならなかった変数を、予測に利用されたファクターとして確認できる。

7.2 分析データの作成

既存データの読み込み

```
tbl_financial <- read_derived_parquet_file_LFL(  
  file.path(  
    "fundamentals",  
    "annual_firm_characteristics.parquet"  
  )  
)  
  
tbl_monthly <- read_derived_parquet_file_LFL(  
  file.path(  
    "returns",  
    "monthly_stock_returns.parquet"  
  )  
)  
  
tbl_factor <- read_derived_parquet_file_LFL(  
  file.path(  
    "factors",  
    "fama_french_factors.parquet"  
  )  
)  
  
tbl_input_overview <- tribble(  
  ~data, ~observations, ~variables,  
  "annual_firm_characteristics",  
  nrow(tbl_financial),  
  ncol(tbl_financial),  
  "monthly_stock_returns",  
  nrow(tbl_monthly),  
  ncol(tbl_monthly),  
  "fama_french_factors",  
  nrow(tbl_factor),  
  ncol(tbl_factor)  
)  
  
show_table_LFL(tbl_input_overview)  
#> # A tibble: 3 x 3  
#>   data                observations variables  
#>   <chr>                <int>      <int>  
#> 1 annual_firm_characteristics    7920      31
```

```
#> 2 monthly_stock_returns          95040      39
#> 3 fama_french_factors             60      16
```

目的変数

年度 t の情報から、年度 $t + 1$ の超過リターンを予測する。

$$y_{i,t} = R_{i,t+1}^e$$

```
tbl_target <- tbl_financial |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  arrange(firm_ID, year) |>
  group_by(firm_ID) |>
  mutate(
    forward_excess_return = lead(Re),
    prediction_year = year + 1L
  ) |>
  ungroup()
```

価格特徴量

```
tbl_price_features <- tbl_monthly |>
  mutate(
    firm_ID = as.integer(firm_ID),
    year = as.integer(year)
  ) |>
  filter(is.finite(R)) |>
  group_by(firm_ID, year) |>
  summarise(
    momentum = prod(1 + R) - 1,
    volatility = sd(R),
    downside_risk = sqrt(mean(pmin(R, 0)^2)),
    average_market_equity = mean(ME, na.rm = TRUE),
    .groups = "drop"
  )
```

市場環境特徴量

```
tbl_market_features <- tbl_factor |>
  group_by(year) |>
  summarise(
    market_premium = prod(1 + R_Me) - 1,
    smb_return = prod(1 + SMB) - 1,
    hml_return = prod(1 + HML) - 1,
    market_volatility = sd(R_Me),
    .groups = "drop"
  )
```

学習データ

```
vec_candidate_features <- c(
  "ROE",
  "BE",
  "ME",
  "lagged_BE",
  "lagged_ME",
  "lagged_BEME",
  "sales",
  "X",
  "NOA",
  "NFO"
)

vec_available_features <- intersect(
  vec_candidate_features,
  names(tbl_target)
)

tbl_ml <- tbl_target |>
  select(
    firm_ID,
    year,
    prediction_year,
    forward_excess_return,
    any_of("industry_ID"),
    all_of(vec_available_features)
  ) |>
  left_join(
```

```
tbl_price_features,
  by = c("firm_ID", "year")
) |>
left_join(
  tbl_market_features,
  by = "year"
) |>
mutate(
  across(
    where(is.numeric),
    \(x) replace(x, !is.finite(x), NA_real_)
  )
) |>
filter(!is.na(forward_excess_return)) |>
arrange(year, firm_ID)

show_table_LFL(tbl_ml, n = 20)
#> # A tibble: 6,405 x 23
#>   fir_ID year pre_year for_retur ind_ID ROE BE ME lag_BE
#>   <dbl> <dbl>   <dbl>      <dbl> <fct> <dbl> <dbl> <dbl> <dbl>
#> 1     1   2015   2016  0.99547     1    NA  9695.3 3577294000 NA
#> 2     2   2015   2016  0.37395     1    NA  1054.9 4086732000 NA
#> 3     3   2015   2016  0.37398     1    NA  93620. 70434234000 NA
#> 4     4   2015   2016 -0.18631     1    NA  12287. 14518674000 NA
#> 5     5   2015   2016  0.20702     1    NA 146587. 59856474000 NA
#> 6     6   2015   2016  0.11198     1    NA  7571.9 5428920000 NA
#> 7     7   2015   2016 -0.052504     1    NA  12970. 16125536000 NA
#> 8     9   2015   2016  0.43471     1    NA  12151. 3983774000 NA
#> 9    10   2015   2016  0.15483     1    NA  14306. 5485326000 NA
#> 10   11   2015   2016  0.29474     1    NA 159546. 195685659000 NA
#> 11   12   2015   2016  0.15874     1    NA  489.68 935820000 NA
#> 12   13   2015   2016  0.15519     1    NA  98895. 157028040000 NA
#> 13   14   2015   2016 -0.15663     1    NA  3888.3 12791475000 NA
#> 14   15   2015   2016 -0.20041     1    NA  5282.7 7087944000 NA
#> 15   16   2015   2016  0.19678     1    NA  25353 27627851000 NA
#> 16   17   2015   2016  0.63234     1    NA 212026. 111711138000 NA
#> 17   18   2015   2016  0.0057293     1    NA  14701. 42414944000 NA
#> 18   20   2015   2016  0.19451     1    NA  14702. 19779450000 NA
#> 19   21   2015   2016  1.0695     1    NA 145766. 74118912000 NA
#> 20   22   2015   2016  0.16118     1    NA  10149. 9317574000 NA
#> # i 6,385 more rows
#> # i 14 more variables: lag_ME <dbl>, lag_BEME <dbl>, sales <dbl>, X <dbl>,
#> # NOA <dbl>, NFO <dbl>, momentum <dbl>, volatility <dbl>, dow_risk <dbl>,
```

```
#> #   ave_equit <dbl>, mar_premi <dbl>, smb_retur <dbl>, hml_retur <dbl>,  
#> #   mar_volat <dbl>
```

7.3 tidymodels による学習

tidymodels では、前処理とモデルを別々に管理し、それらを `workflow()` でまとめる。

```
recipe  
  ↓  
model specification  
  ↓  
workflow  
  ↓  
resampling  
  ↓  
tuning  
  ↓  
prediction
```

前処理

```
factor_recipe <- recipes::recipe(  
  forward_excess_return ~ .,  
  data = tbl_ml  
) |>  
  recipes::update_role(  
    firm_ID,  
    year,  
    prediction_year,  
    new_role = "id"  
) |>  
  recipes::step_unknown(  
    recipes::all_nominal_predictors()  
) |>  
  recipes::step_novel(  
    recipes::all_nominal_predictors()  
) |>  
  recipes::step_dummy(  
    recipes::all_nominal_predictors()  
) |>  
  recipes::step_impute_median(  
    recipes::all_numeric_predictors()  
) |>
```

```
recipes::step_zv(
  recipes::all_predictors()
) |>
recipes::step_normalize(
  recipes::all_numeric_predictors()
)
```

時系列分割

ランダム分割ではなく、年度の順序を維持して学習期間と評価期間を分ける。

```
vec_analysis_years <- sort(unique(tbl_ml$year))

split_year <- vec_analysis_years[
  max(
    2L,
    floor(length(vec_analysis_years) * 0.7)
  )
]

tbl_training <- tbl_ml |>
  filter(year <= split_year)

tbl_testing <- tbl_ml |>
  filter(year > split_year)

tbl_split_overview <- tribble(
  ~sample, ~observations, ~first_year, ~last_year,
  "training",
  nrow(tbl_training),
  min(tbl_training$year),
  max(tbl_training$year),
  "testing",
  nrow(tbl_testing),
  min(tbl_testing$year),
  max(tbl_testing$year)
)

show_table_LFL(tbl_split_overview)
#> # A tibble: 2 x 4
#>   sample observations fir_year las_year
#>   <chr>          <int>    <dbl>    <dbl>
#> 1 training      3779     2015     2017
#> 2 testing      2626     2018     2019
```

7.4 ファクター選定

LASSO、Ridge、Elastic Net は、同じ `glmnet` エンジンで表現できる。

- `mixture = 1` : LASSO
- `mixture = 0` : Ridge
- `0 < mixture < 1` : Elastic Net

```
elastic_net_spec <- parsnip::linear_reg(  
  penalty = tune::tune(),  
  mixture = tune::tune()  
) |>  
  parsnip::set_engine("glmnet")  
  
elastic_net_workflow <- workflows::workflow() |>  
  workflows::add_recipe(factor_recipe) |>  
  workflows::add_model(elastic_net_spec)
```

本ドラフトでは、まず小さなグリッドを使って処理全体が動くことを確認する。

```
tbl_parameter_grid <- tidyr::crossing(  
  penalty = c(0.0001, 0.001, 0.01, 0.1),  
  mixture = c(0, 0.5, 1)  
)
```

```
set.seed(1234)
```

```
validation_folds <- rsample::vfold_cv(  
  tbl_training,  
  v = min(5L, nrow(tbl_training))  
)
```

```
factor_tuning <- tune::tune_grid(  
  elastic_net_workflow,  
  resamples = validation_folds,  
  grid = tbl_parameter_grid,  
  metrics = yardstick::metric_set(  
    yardstick::rmse,  
    yardstick::mae  
  )  
)
```

```
best_parameters <- tune::select_best(  
  factor_tuning,  
  metric = "rmse"  
)
```

```

show_table_LFL(best_parameters)
#> # A tibble: 1 x 3
#>   penalty mixture .config
#>   <dbl>    <dbl> <chr>
#> 1  0.001      0.5 pre0_mod05_post0

final_factor_workflow <- tune::finalize_workflow(
  elastic_net_workflow,
  best_parameters
)

final_factor_fit <- parsnip::fit(
  final_factor_workflow,
  data = tbl_training
)

```

選択された変数を係数から確認する。

```

tbl_selected_factors <- final_factor_fit |>
  workflows::extract_fit_parsnip() |>
  broom::tidy() |>
  filter(
    term != "(Intercept)",
    estimate != 0
  ) |>
  arrange(desc(abs(estimate)))

show_table_LFL(tbl_selected_factors, n = 30)
#> # A tibble: 22 x 3
#>   term                estimate penalty
#>   <chr>                <dbl>    <dbl>
#> 1 volatility          -0.052925  0.001
#> 2 lagged_BEME           0.040087  0.001
#> 3 momentum             0.035791  0.001
#> 4 market_premium      -0.027679  0.001
#> 5 smb_return           -0.026595  0.001
#> 6 hml_return           0.025744  0.001
#> 7 market_volatility  -0.025094  0.001
#> 8 industry_ID_X7       0.022350  0.001
#> 9 ME                   -0.019009  0.001
#> 10 industry_ID_X10     0.017595  0.001
#> 11 industry_ID_X5     -0.015146  0.001
#> 12 lagged_ME           0.015131  0.001
#> 13 X                   -0.011249  0.001

```



```
#> 14 ROE -0.0099735 0.001
#> 15 industry_ID_X2 0.0094625 0.001
#> 16 industry_ID_X6 -0.0092099 0.001
#> 17 industry_ID_X3 0.0088236 0.001
#> 18 industry_ID_X9 -0.0058770 0.001
#> 19 NOA 0.0049012 0.001
#> 20 lagged_BE -0.0034932 0.001
#> 21 BE 0.0023084 0.001
#> 22 industry_ID_X8 0.0015864 0.001
```

7.5 期待リターンの推定

テスト期間の予測

```
tbl_test_predictions <- predict(
  final_factor_fit,
  new_data = tbl_testing
) |>
  bind_cols(
    tbl_testing |>
      select(
        firm_ID,
        year,
        prediction_year,
        forward_excess_return
      )
  ) |>
  rename(
    predicted_return = .pred,
    realized_return = forward_excess_return
  )
```

```
tbl_prediction_metrics <- yardstick::metric_set(
  yardstick::rmse,
  yardstick::mae,
  yardstick::rsq
)(
  tbl_test_predictions,
  truth = realized_return,
  estimate = predicted_return
)

show_table_LFL(tbl_prediction_metrics)
```

```
#> # A tibble: 3 x 3
#>   .metric .estimator .estimate
#>   <chr>   <chr>       <dbl>
#> 1 rmse    standard     3.9625
#> 2 mae     standard     3.0002
#> 3 rsq     standard     0.25561
```

全データによる最終モデル

```
final_factor_fit_all <- parsnip::fit(
  final_factor_workflow,
  data = tbl_ml
)

tbl_security_scores <- predict(
  final_factor_fit_all,
  new_data = tbl_ml
) |>
  bind_cols(
    tbl_ml |>
      select(
        firm_ID,
        year,
        prediction_year,
        forward_excess_return
      )
  ) |>
  rename(
    expected_return = .pred,
    realized_return = forward_excess_return
  ) |>
  group_by(prediction_year) |>
  mutate(
    score_rank = percent_rank(expected_return),
    score_decile = ntile(expected_return, 10L)
  ) |>
  ungroup()
```

7.6 ポートフォリオ候補の作成

ここでは最適化を行わず、予測値によって銘柄を並べ、分位ポートフォリオを作成する。

```
tbl_decile_returns <- tbl_security_scores |>
  group_by(
    prediction_year,
    score_decile
  ) |>
  summarise(
    securities = n(),
    average_expected_return = mean(
      expected_return,
      na.rm = TRUE
    ),
    average_realized_return = mean(
      realized_return,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

show_table_LFL(tbl_decile_returns)
#> # A tibble: 50 x 5
#>   pre_year sco_decil securities ave_retur ave_retur
#>   <dbl>     <int>     <int>    <dbl>    <dbl>
#> 1   2016         1       125  0.040431 0.0093272
#> 2   2016         2       124  0.079922 0.10518
#> 3   2016         3       124  0.10226  0.018788
#> 4   2016         4       124  0.11709  0.098786
#> 5   2016         5       124  0.13332  0.11719
#> 6   2016         6       124  0.14901  0.14958
#> 7   2016         7       124  0.16426  0.21376
#> 8   2016         8       124  0.17963  0.22106
#> 9   2016         9       124  0.19541  0.26803
#> 10  2016        10       124  0.22457  0.21087
#> # i 40 more rows
```

最上位 10 分位と最下位 10 分位の差を確認する。

```
tbl_long_short <- tbl_decile_returns |>
  filter(score_decile %in% c(1L, 10L)) |>
  select(
    prediction_year,
    score_decile,
    average_realized_return
  ) |>
  pivot_wider(
```

```

    names_from = score_decile,
    values_from = average_realized_return,
    names_prefix = "decile_"
  ) |>
  mutate(
    long_short_return =
      decile_10 - decile_1
  )

show_table_LFL(tbl_long_short)
#> # A tibble: 5 x 4
#>   pre_year      dec_1    dec_10 lon_retur
#>   <dbl>      <dbl>    <dbl>    <dbl>
#> 1    2016  0.0093272  0.21087    0.20154
#> 2    2017  0.041990  0.30981    0.26782
#> 3    2018 -0.18027   0.11281    0.29308
#> 4    2019  0.17750   0.64451    0.46701
#> 5    2020 -0.17070   0.024575   0.19527

```

7.7 第 8 章へのデータ保存

第 8 章では、期待リターン、過去リターン、投資対象銘柄を使って制約付きポートフォリオを構築する。

```

tbl_expected_returns <- tbl_security_scores |>
  transmute(
    rebalance_year = prediction_year,
    firm_ID,
    expected_return,
    score_rank,
    score_decile
  )

tbl_asset_return_history <- tbl_monthly |>
  transmute(
    firm_ID = as.integer(firm_ID),
    month_ID = as.integer(month_ID),
    month_date = as.Date(month_date),
    return = R
  ) |>
  filter(is.finite(return))

tbl_investment_universe <- tbl_expected_returns |>
  left_join(
    tbl_financial |>

```

```
    transmute(
      rebalance_year = as.integer(year) + 1L,
      firm_ID = as.integer(firm_ID),
      market_equity = ME
    ),
    by = c(
      "rebalance_year", "firm_ID")
  ) |>
  filter(
    is.finite(expected_return),
    is.finite(market_equity),
    market_equity > 0
  )
```

```
tbl_chapter_07_outputs <- list(
  expected_returns = tbl_expected_returns,
  security_scores = tbl_security_scores,
  factor_selection = tbl_selected_factors,
  asset_return_history = tbl_asset_return_history,
  investment_universe = tbl_investment_universe
)
```

```
vec_chapter_07_files <- c(
  expected_returns = file.path(
    "portfolios",
    "expected_returns.parquet"
  ),
  security_scores = file.path(
    "factors",
    "security_scores.parquet"
  ),
  factor_selection = file.path(
    "factors",
    "factor_selection.parquet"
  ),
  asset_return_history = file.path(
    "portfolios",
    "asset_return_history.parquet"
  ),
  investment_universe = file.path(
    "portfolios",
    "investment_universe.parquet"
  )
)
```

```
vec_chapter_07_paths <- tbl_chapter_07_outputs |>
  imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,
        vec_chapter_07_files[[name]]
      )
    }
  )

tibble::tibble(
  output = base::names(vec_chapter_07_paths),
  path = relative_path_LFL(base::unname(vec_chapter_07_paths))
) |>
  show_table_LFL()
#> # A tibble: 5 x 2
#>   output          path
#>   <chr>          <chr>
#> 1 expected_returns data/derived/portfolios/expected_returns.parquet
#> 2 security_scores  data/derived/factors/security_scores.parquet
#> 3 factor_selection data/derived/factors/factor_selection.parquet
#> 4 asset_return_history data/derived/portfolios/asset_return_history.parquet
#> 5 investment_universe data/derived/portfolios/investment_universe.parquet
```

本章では、既存の財務データ、株式リターンおよびファクター・データから機械学習用データを作成し、`tidymodels` のワークフローによってファクター選定と期待リターン推定を行った。

第 8 章では、この期待リターンを制約付きポートフォリオ最適化へ接続する。

第 1-8 章制約付きポートフォリオ最適化とバックテスト

本章では、第 7 章で推定した銘柄別期待リターンと、これまでに作成した株式リターン履歴を利用し、実務的な制約条件を満たすポートフォリオを構築する。

第 5 章では平均分散最適化の基本を確認した。本章では、その考え方にロングオンリー、銘柄別ウェイト上限、VaR 上限および売買回転率を加え、アウト・オブ・サンプルのバックテストへ発展させる。

本章では、次の順序で処理を進める。

第7章の期待リターン



投資ユニバースと共分散行列



制約条件



最適ウェイト



ローリング・リバランス



バックテストと評価

8.1 ポートフォリオ構築の全体像

銘柄数を N 、ウェイトを w 、期待リターンを μ 、分散共分散行列を Σ とする。

ポートフォリオ期待リターンは、

$$\mu_p = w^\top \mu$$

ポートフォリオ分散は、

$$\sigma_p^2 = w^\top \Sigma w$$

である。

本章では、期待リターンを高めながらリスクを抑える目的関数を用いる。

$$\max_w \left[w^\top \mu - \frac{\gamma}{2} w^\top \Sigma w \right]$$

ここで γ はリスク回避度である。

8.2 投資ユニバースとリスク推定

第 7 章の出力

```
tbl_expected_returns <- read_derived_parquet_file_LFL(
  file.path(
    "portfolios",
    "expected_returns.parquet"
  )
)

tbl_return_history <- read_derived_parquet_file_LFL(
  file.path(
    "portfolios",
    "asset_return_history.parquet"
  )
)

tbl_investment_universe <- read_derived_parquet_file_LFL(
  file.path(
    "portfolios",
    "investment_universe.parquet"
  )
)

tbl_input_overview <- tribble(
  ~data, ~observations, ~variables,
  "expected_returns",
  nrow(tbl_expected_returns),
  ncol(tbl_expected_returns),
  "asset_return_history",
  nrow(tbl_return_history),
  ncol(tbl_return_history),
  "investment_universe",
  nrow(tbl_investment_universe),
  ncol(tbl_investment_universe)
)
```



```
show_table_LFL(tbl_input_overview)
#> # A tibble: 3 x 3
#>   data                observations variables
#>   <chr>                <int>      <int>
#> 1 expected_returns      6405         5
#> 2 asset_return_history  93525         4
#> 3 investment_universe   6405         6
```

最初のリバランス年度

本ドラフトでは、まず1年度分のポートフォリオを構築し、最適化が実行できることを確認する。

```
rebalance_year <- tbl_investment_universe |>
  summarise(
    rebalance_year =
      min(rebalance_year, na.rm = TRUE)
  ) |>
  pull(rebalance_year)

tbl_universe_one_year <- tbl_investment_universe |>
  filter(rebalance_year == !!rebalance_year) |>
  arrange(desc(market_equity)) |>
  slice_head(n = 20)
```

対象銘柄を20銘柄に限定することで、最初のドラフトではデータ確認と最適化の流れを見やすくする。

リターン行列

```
vec_firm_ids <- tbl_universe_one_year |>
  pull(firm_ID)

tbl_return_wide <- tbl_return_history |>
  filter(firm_ID %in% vec_firm_ids) |>
  select(
    month_ID,
    firm_ID,
    return
  ) |>
  pivot_wider(
    names_from = firm_ID,
    values_from = return
  ) |>
  arrange(month_ID)
```

```
mat_return <- tbl_return_wide |>
  select(-month_ID) |>
  drop_na() |>
  as.matrix()

mat_covariance <- cov(mat_return)

vec_expected_return <- tbl_universe_one_year |>
  arrange(firm_ID) |>
  pull(expected_return)
```

列順を一致させる。

```
vec_covariance_firm_ids <- colnames(mat_return) |>
  as.integer()

tbl_universe_one_year <- tbl_universe_one_year |>
  filter(firm_ID %in% vec_covariance_firm_ids) |>
  arrange(match(firm_ID, vec_covariance_firm_ids))

vec_expected_return <- tbl_universe_one_year |>
  pull(expected_return)

mat_covariance <- mat_covariance[
  as.character(tbl_universe_one_year$firm_ID),
  as.character(tbl_universe_one_year$firm_ID),
  drop = FALSE
]
```

8.3 制約条件

完全投資

$$\sum_{i=1}^N w_i = 1$$

ロングオンリー

$$w_i \geq 0$$

銘柄別ウェイト上限

$$w_i \leq w_{\max}$$

本ドラフトでは、1 銘柄の上限を 20% とする。

VaR 上限

正規分布近似による VaR を、

$$VaR_c = z_c \sigma_p - \mu_p$$

とする。

$$z_c \sqrt{w^\top \Sigma w} - w^\top \mu \leq VaR_{\max}$$

売買回転率

前回ウェイトを w_{t-1} とすると、

$$Turnover_t = \frac{1}{2} \sum_{i=1}^N |w_{i,t} - w_{i,t-1}|$$

である。

最初のドラフトでは、ロングオンリー、完全投資、ウェイト上限、VaR 上限までを実装し、売買回転率はバックテスト部分で計算する。

8.4 制約付きポートフォリオ最適化

計算関数

```
portfolio_return_LFL <- function(
  weight,
  expected_return
) {
  sum(weight * expected_return)
}

portfolio_variance_LFL <- function(
  weight,
```

```
      covariance_matrix
    ) {
      as.numeric(
        t(weight) %*%
          covariance_matrix %*%
            weight
      )
    }
  }

portfolio_volatility_LFL <- function(
  weight,
  covariance_matrix
) {
  sqrt(
    portfolio_variance_LFL(
      weight,
      covariance_matrix
    )
  )
}

portfolio_var_LFL <- function(
  weight,
  expected_return,
  covariance_matrix,
  confidence_level = 0.95
) {
  qnorm(confidence_level) *
    portfolio_volatility_LFL(
      weight,
      covariance_matrix
    ) -
    portfolio_return_LFL(
      weight,
      expected_return
    )
}
```

最適化条件

```
maximum_weight <- 0.20
maximum_var <- 0.10
confidence_level <- 0.95
```

```
risk_aversion <- 5
```

nloptr による最適化

```
assets <- length(vec_expected_return)

objective_function <- function(weight) {
  -portfolio_return_LFL(
    weight,
    vec_expected_return
  ) +
  risk_aversion / 2 *
  portfolio_variance_LFL(
    weight,
    mat_covariance
  )
}

objective_function <- function(weight) {
  objective_value <-
    -portfolio_return_LFL(
      weight,
      vec_expected_return
    ) +
    risk_aversion / 2 *
    portfolio_variance_LFL(
      weight,
      mat_covariance
    )

  objective_gradient <-
    -vec_expected_return +
    risk_aversion *
    as.numeric(
      mat_covariance %*% weight
    )

  list(
    objective = objective_value,
    gradient = objective_gradient
  )
}
```

```
equality_constraint <- function(weight) {  
  list(  
    constraints = sum(weight) - 1,  
    jacobian = matrix(  
      rep(1, length(weight)),  
      nrow = 1  
    )  
  )  
}  
  
inequality_constraint <- function(weight) {  
  portfolio_volatility <-  
    portfolio_volatility_LFL(  
      weight,  
      mat_covariance  
    )  
  
  constraint_value <-  
    qnorm(confidence_level) *  
    portfolio_volatility -  
    portfolio_return_LFL(  
      weight,  
      vec_expected_return  
    ) -  
    maximum_var  
  
  volatility_gradient <- if (  
    portfolio_volatility > 1e-12  
  ) {  
    as.numeric(  
      mat_covariance %*% weight  
    ) /  
    portfolio_volatility  
  } else {  
    rep(0, length(weight))  
  }  
  
  constraint_gradient <-  
    qnorm(confidence_level) *  
    volatility_gradient -  
    vec_expected_return  
  
  list(  

```

```

    constraints = constraint_value,
    jacobian = matrix(
      constraint_gradient,
      nrow = 1
    )
  )
}

equality_constraint <- function(weight) {
  sum(weight) - 1
}

inequality_constraint <- function(weight) {
  portfolio_var_LFL(
    weight,
    vec_expected_return,
    mat_covariance,
    confidence_level
  ) - maximum_var
}

objective_function <- function(weight) {
  objective_value <-
    -portfolio_return_LFL(
      weight,
      vec_expected_return
    ) +
    risk_aversion / 2 *
    portfolio_variance_LFL(
      weight,
      mat_covariance
    )

  objective_gradient <-
    -vec_expected_return +
    risk_aversion *
    as.numeric(
      mat_covariance %*% weight
    )

  list(
    objective = objective_value,
    gradient = objective_gradient
  )
}

```

```
}

equality_constraint <- function(weight) {
  list(
    constraints = sum(weight) - 1,
    jacobian = rep(1, length(weight))
  )
}

inequality_constraint <- function(weight) {
  portfolio_volatility <-
    portfolio_volatility_LFL(
      weight,
      mat_covariance
    )

  constraint_value <-
    qnorm(confidence_level) *
    portfolio_volatility -
    portfolio_return_LFL(
      weight,
      vec_expected_return
    ) -
    maximum_var

  volatility_gradient <- if (
    portfolio_volatility > 1e-12
  ) {
    as.numeric(
      mat_covariance %*% weight
    ) /
      portfolio_volatility
  } else {
    rep(0, length(weight))
  }

  constraint_gradient <-
    qnorm(confidence_level) *
    volatility_gradient -
    vec_expected_return

  list(
    constraints = constraint_value,
```



```

    jacobian = constraint_gradient
  )
}

assets <- length(vec_expected_return)

if (assets * maximum_weight < 1) {
  stop(
    "銘柄数とmaximum_weightの組合せでは完全投資制約を満たせません。",
    call. = FALSE
  )
}

```

```

optimization_result <- nloptr::nloptr(
  x0 = rep(1 / assets, assets),
  eval_f = objective_function,
  lb = rep(0, assets),
  ub = rep(maximum_weight, assets),
  eval_g_eq = equality_constraint,
  eval_g_ineq = inequality_constraint,
  opts = list(
    algorithm = "NLOPT_LD_SLSQP",
    xtol_rel = 1e-8,
    maxeval = 1000,
    print_level = 0
  )
)

tbl_optimized_weights <- tbl_universe_one_year |>
  select(
    rebalance_year,
    firm_ID,
    expected_return
  ) |>
  mutate(
    weight = optimization_result$solution
  ) |>
  arrange(desc(weight))

show_table_LFL(tbl_optimized_weights)
#> # A tibble: 20 x 4
#>   reb_year fir_ID exp_retur    weight
#>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1     2016   1253  0.20697 2     e- 1

```

```
#> 2      2016      472 0.15848 2      e- 1
#> 3      2016     1205 0.14965 2      e- 1
#> 4      2016     1400 0.18788 2.0000e- 1
#> 5      2016     1408 0.17726 2.0000e- 1
#> 6      2016      721 0.14213 8.8991e-16
#> 7      2016     1057 0.051368 1.3889e-16
#> 8      2016     1237 0.12760 1.1713e-16
#> 9      2016      456 0.015991 7.5341e-17
#> 10     2016      725 0.096705 2.5315e-17
#> # i 10 more rows
```

制約条件を確認する。

```
tbl_constraint_check <- tribble(
  ~constraint, ~value, ~limit,
  "weight_sum",
  sum(tbl_optimized_weights$weight),
  1,
  "minimum_weight",
  min(tbl_optimized_weights$weight),
  0,
  "maximum_weight",
  max(tbl_optimized_weights$weight),
  maximum_weight,
  "portfolio_var",
  portfolio_var_LFL(
    tbl_optimized_weights$weight,
    vec_expected_return,
    mat_covariance,
    confidence_level
  ),
  maximum_var
)

show_table_LFL(tbl_constraint_check)
#> # A tibble: 4 x 3
#>   constraint      value limit
#>   <chr>          <dbl> <dbl>
#> 1 weight_sum      1.0000      1
#> 2 minimum_weight  0          0
#> 3 maximum_weight  0.2         0.2
#> 4 portfolio_var   0.0052375 0.1
```

8.5 バックテスト

最適化したウェイトを、その後の実現リターンへ適用する。

本ドラフトでは、最初のリバランス年度の翌 12 か月を対象とする。

```
rebalance_month_ID <- tbl_return_history |>
  mutate(
    year = lubridate::year(month_date)
  ) |>
  filter(year == rebalance_year) |>
  summarise(
    first_month_ID =
      min(month_ID, na.rm = TRUE)
  ) |>
  pull(first_month_ID)
```

```
tbl_backtest_returns <- tbl_return_history |>
  filter(
    firm_ID %in% tbl_optimized_weights$firm_ID,
    month_ID >= rebalance_month_ID,
    month_ID < rebalance_month_ID + 12L
  ) |>
  left_join(
    tbl_optimized_weights |>
      select(
        firm_ID,
        weight
      ),
    by = "firm_ID"
  ) |>
  group_by(
    month_ID,
    month_date
  ) |>
  summarise(
    portfolio_return = sum(
      weight * return,
      na.rm = TRUE
    ),
    .groups = "drop"
  ) |>
  arrange(month_ID)
```

比較対象として等ウェイト・ポートフォリオを作成する。

```
tbl_equal_weight_returns <- tbl_return_history |>
  filter(
    firm_ID %in% tbl_optimized_weights$firm_ID,
    month_ID >= rebalance_month_ID,
    month_ID < rebalance_month_ID + 12L
  ) |>
  group_by(
    month_ID,
    month_date
  ) |>
  summarise(
    equal_weight_return =
      mean(return, na.rm = TRUE),
    .groups = "drop"
  )

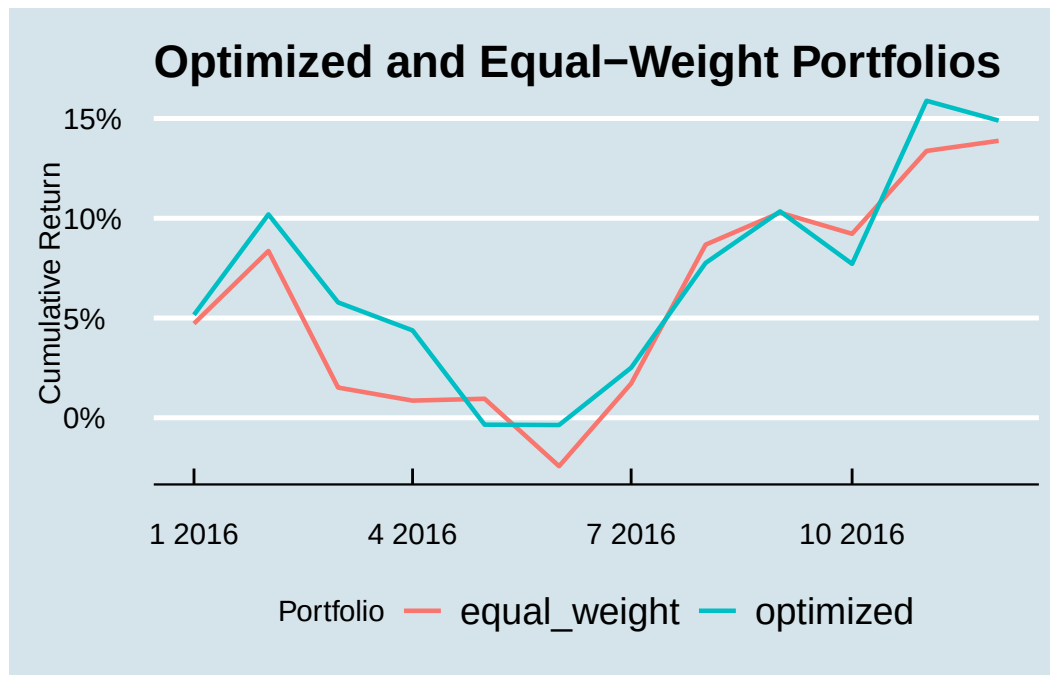
tbl_backtest_returns <- tbl_backtest_returns |>
  left_join(
    tbl_equal_weight_returns,
    by = c(
      "month_ID",
      "month_date"
    )
  )
```

```
tbl_backtest_plot <- tbl_backtest_returns |>
  mutate(
    optimized = cumprod(
      1 + portfolio_return
    ) - 1,
    equal_weight = cumprod(
      1 + equal_weight_return
    ) - 1
  ) |>
  select(
    month_date,
    optimized,
    equal_weight
  ) |>
  pivot_longer(
    cols = c(
      optimized,
      equal_weight
    )
  )
```

```
    ),
    names_to = "portfolio",
    values_to = "cumulative_return"
  )

plot_backtest <- tbl_backtest_plot |>
  ggplot(
    aes(
      x = month_date,
      y = cumulative_return,
      colour = portfolio
    )
  ) +
  geom_line(linewidth = 0.8) +
  scale_y_continuous(
    labels = scales::label_percent()
  ) +
  labs(
    x = NULL,
    y = "Cumulative Return",
    colour = "Portfolio",
    title = "Optimized and Equal-Weight Portfolios"
  )

show_plot_LFL(
  add_lagrange_theme_LFL(
    plot_backtest
  )
)
```



8.6 パフォーマンス評価

```
annualized_return_LFL <- function(
  x,
  periods = 12
) {
  prod(1 + x)^(periods / length(x)) - 1
}

annualized_volatility_LFL <- function(
  x,
  periods = 12
) {
  sd(x) * sqrt(periods)
}

maximum_drawdown_LFL <- function(x) {
  wealth <- cumprod(1 + x)
  drawdown <- wealth / cummax(wealth) - 1
  min(drawdown)
}

tbl_performance_summary <- tribble(
  ~portfolio, ~annualized_return,
  ~annualized_volatility, ~sharpe_ratio,
  ~maximum_drawdown,
```

```

"optimized",
annualized_return_LFL(
  tbl_backtest_returns$portfolio_return
),
annualized_volatility_LFL(
  tbl_backtest_returns$portfolio_return
),
annualized_return_LFL(
  tbl_backtest_returns$portfolio_return
) /
annualized_volatility_LFL(
  tbl_backtest_returns$portfolio_return
),
maximum_drawdown_LFL(
  tbl_backtest_returns$portfolio_return
),
"equal_weight",
annualized_return_LFL(
  tbl_backtest_returns$equal_weight_return
),
annualized_volatility_LFL(
  tbl_backtest_returns$equal_weight_return
),
annualized_return_LFL(
  tbl_backtest_returns$equal_weight_return
) /
annualized_volatility_LFL(
  tbl_backtest_returns$equal_weight_return
),
maximum_drawdown_LFL(
  tbl_backtest_returns$equal_weight_return
)
)

show_table_LFL(tbl_performance_summary)
#> # A tibble: 2 x 5
#>   portfolio    ann_retur ann_volat sha_ratio max_drawd
#>   <chr>         <dbl>    <dbl>    <dbl>    <dbl>
#> 1 optimized      0.14896  0.13773   1.0816 -0.095817
#> 2 equal_weight   0.13883  0.12901   1.0761 -0.099548

```

取引コストと売買回転率は、複数回のリバランスを実装した段階で追加する。本ドラフトでは、最適化、制約確認、実現リターンへの適用、等ウェイトとの比較までを一つの流れとして確認する。

8.7 結果の保存

```
tbl_chapter_08_outputs <- list(  
  optimized_weights =  
    tbl_optimized_weights,  
  backtest_returns =  
    tbl_backtest_returns,  
  performance_summary =  
    tbl_performance_summary  
)  
  
vec_chapter_08_files <- c(  
  optimized_weights = file.path(  
    "portfolios",  
    "optimized_weights.parquet"  
  ),  
  backtest_returns = file.path(  
    "portfolios",  
    "backtest_returns.parquet"  
  ),  
  performance_summary = file.path(  
    "portfolios",  
    "performance_summary.parquet"  
  )  
)  
  
vec_chapter_08_paths <- tbl_chapter_08_outputs |>  
  imap_chr(  
    \(data, name) {  
      write_derived_parquet_file_LFL(  
        data,  
        vec_chapter_08_files[[name]]  
      )  
    }  
  )  
  
tibble::tibble(  
  output = base::names(vec_chapter_08_paths),  
  path = relative_path_LFL(base::unname(vec_chapter_08_paths))  
) |>  
  show_table_LFL()  
#> # A tibble: 3 x 2
```



```
#>   output          path
#>   <chr>         <chr>
#> 1 optimized_weights data/derived/portfolios/optimized_weights.parquet
#> 2 backtest_returns  data/derived/portfolios/backtest_returns.parquet
#> 3 performance_summary data/derived/portfolios/performance_summary.parquet
```

本章では、第7章の期待リターンを使い、ロングオンリー、完全投資、銘柄別ウェイト上限および VaR 上限を満たすポートフォリオを構築した。

さらに、最適ウェイトを実現リターンへ適用し、等ウェイト・ポートフォリオとの比較を行った。次の段階では、複数年度のローリング・リバランス、売買回転率、取引コストを追加し、バックテストを拡張する。

第Ⅱ部 金融工学 無裁定モデル

第 II-1 章確定キャッシュフロー債券評価

金融では、異なる時点に発生する金額をそのまま比較することはできない。今日受け取る 100 円と 1 年後に受け取る 100 円では、運用可能期間や金利が異なるため、経済的な価値も異なるからである。そこで、時点の異なるキャッシュフローを評価する場合には、将来の金額を現在価値へ割り引くか、現在の金額を将来価値へ換算し、同一時点の価値にそろえる必要がある。金融商品の価格評価では、通常、将来のすべてのキャッシュフローを評価時点の現在価値へ換算する。

1.1 金利期間構造の確認

債券価格は、将来の各キャッシュフローを支払時点に対応する割引係数で現在価値へ換算し、その合計として求める。個別債券の評価に入る前に、金利期間構造をスポットレート、割引係数およびフォワードレートの三つの側面から確認する。

市場で直接観測される金利は、必ずしも各満期に対応するスポットレートではない。市場では、預金金利、OIS レート、スワップレート、国債の価格や複利回りなどが観測される。これらの市場情報と、日数計算規則、利払頻度および受渡日などのマーケットコンベンションを用いて、各満期のゼロクーポン・スポットレートとディスカウントファクター（以下、DF）を順次求める。この手続きをブートストラップという。

スポットレートは、評価時点から特定の満期までのゼロクーポン金利を表す。DF は、将来の 1 単位の金額を評価時点の価値へ換算する倍率であり、債券、スワップおよびデリバティブのキャッシュフロー評価に直接用いられる。さらに、異なる満期の DF を比較することで、将来の一定期間に適用されるフォワードレートを求めることができる。このように、市場で観測される複数の金利商品からスポットレート、DF およびフォワードレートからなる金利期間構造を構築することは、金融工学における価格評価の基礎となる。

以下では、EONIA の市場データから構築した金利曲線を用いて、スポットカーブ、DF カーブおよびフォワードカーブの関係を確認する。さらに、本章では、固定利付債を例として、クリーン価格、ダーティ価格、経過利子、最終利回り、デュレーションおよびコンベクシティを計算する。あわせて、日数計算規則、休日カレンダー、営業日調整、受渡日および利払スケジュールといったマーケットコンベンションを確認する。

```
eonia_benchmark <- GaussQuant::eonia_curve_benchmark_GQL()

eonia_curve_tbl <- eonia_benchmark$final_validation$quote_repricing |>
  dplyr::select(
    pillar_date,
    discount_factor,
    zero_rate,
    one_day_forward_rate
```

```

) |>
dplyr::filter(!is.na(.data$pillar_date)) |>
dplyr::distinct(.data$pillar_date, .keep_all = TRUE) |>
dplyr::arrange(.data$pillar_date)

spot_curve_plot <- eonia_curve_tbl |>
dplyr::filter(!is.na(.data$zero_rate)) |>
ggplot2::ggplot(
  ggplot2::aes(
    x = .data$pillar_date,
    y = 100 * .data$zero_rate
  )
) +
ggplot2::geom_line(linewidth = 0.7) +
ggplot2::geom_point(size = 1.5) +
ggplot2::scale_x_date(
  date_breaks = "5 years",
  date_labels = "%Y"
) +
ggplot2::labs(
  title = "EONIAスポットカーブ",
  x = "満期",
  y = "スポットレート (%)"
)

discount_curve_plot <- eonia_curve_tbl |>
dplyr::filter(!is.na(.data$discount_factor)) |>
ggplot2::ggplot(
  ggplot2::aes(
    x = .data$pillar_date,
    y = .data$discount_factor
  )
) +
ggplot2::geom_line(linewidth = 0.7) +
ggplot2::geom_point(size = 1.5) +
ggplot2::scale_x_date(
  date_breaks = "5 years",
  date_labels = "%Y"
) +
ggplot2::labs(
  title = "EONIA割引係数カーブ",
  x = "満期",
  y = "割引係数"
)

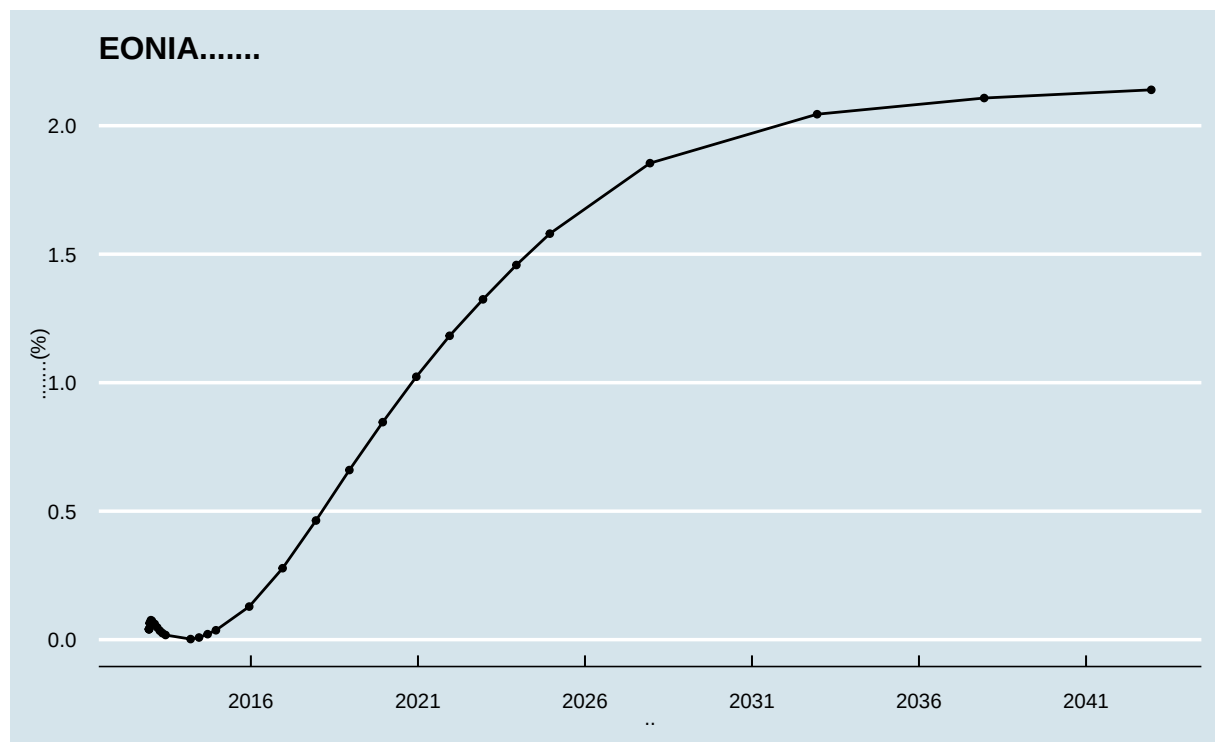
```

```

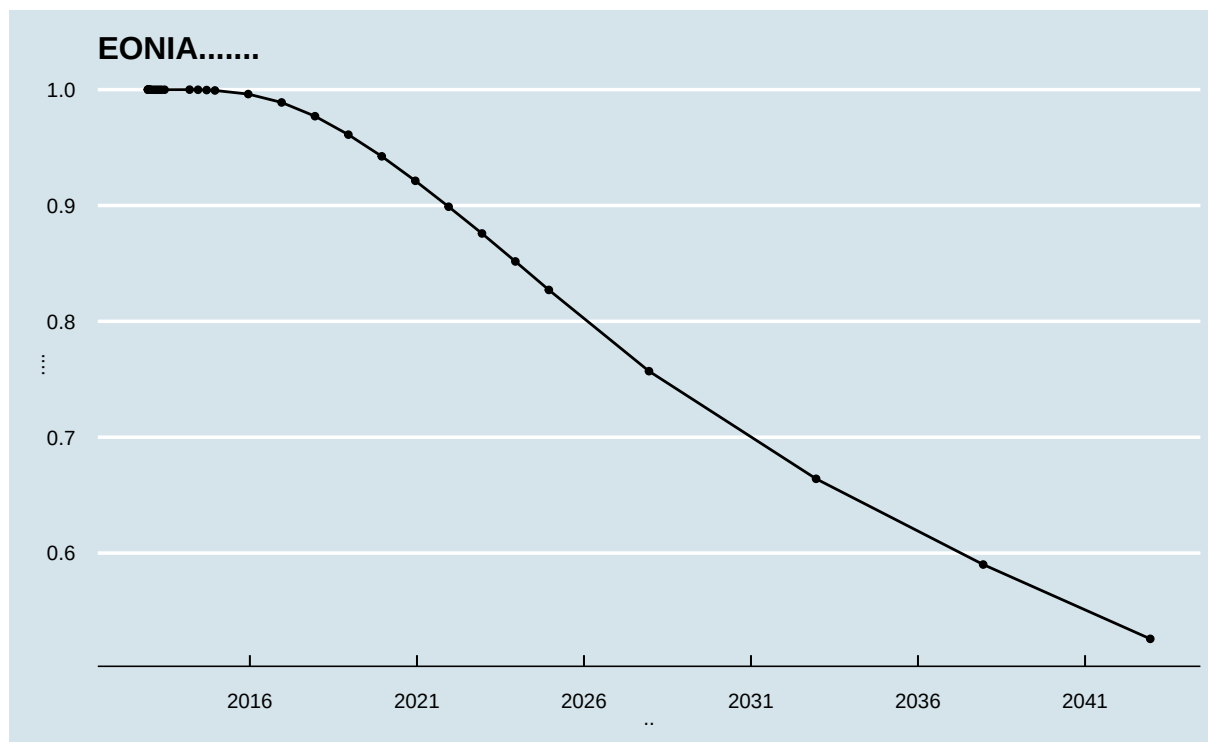
)

forward_curve_plot <- eonia_curve_tbl |>
  dplyr::filter(!is.na(.data$one_day_forward_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$pillar_date,
      y = 100 * .data$one_day_forward_rate
    )
  ) +
  ggplot2::geom_line(linewidth = 0.7) +
  ggplot2::geom_point(size = 1.5) +
  ggplot2::scale_x_date(
    date_breaks = "5 years",
    date_labels = "%Y"
  ) +
  ggplot2::labs(
    title = "EONIA 1営業日フォワードカーブ",
    x = "開始日",
    y = "フォワードレート (%)"
  )
show_plot_LFL(spot_curve_plot)

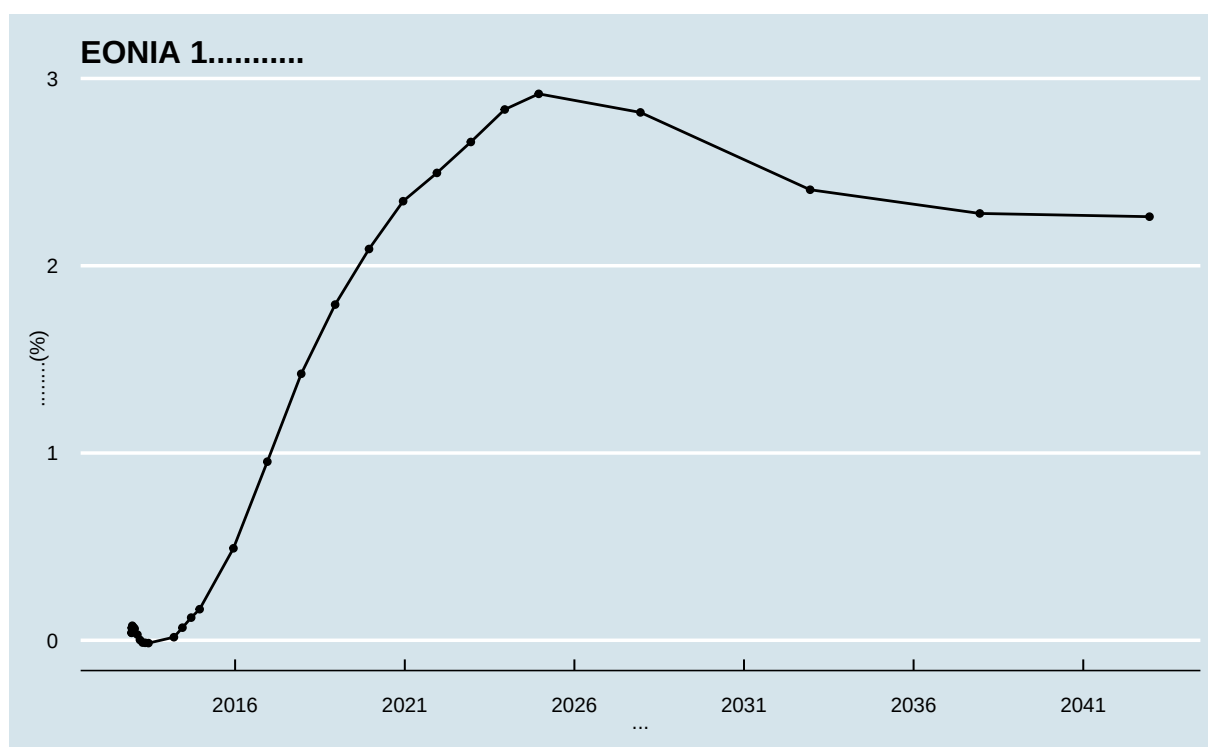
```



```
show_plot_LFL(discount_curve_plot)
```



```
show_plot_LFL(forward_curve_plot)
```



1.2 マーケットコンベンション

債券評価では、価格や利回りの計算方法だけでなく、日付、利払頻度、休日および端数処理に関する市場慣行を確認する必要がある。同じ債券であっても、採用する日数計算規則や利回り表示方式が異なれば、経過利子、残存年数および計算価格も異なる。

1.2.1 一般的な市場慣行

日数計算規則 (Day Count Convention) は、二つの日付の間の期間を年数へ換算する規則である。主な方式は次のとおりである。

- **Actual/365 Fixed** 二つの日付間の実日数を 365 で除する。
- **Actual/360** 二つの日付間の実日数を 360 で除する。短期金融市場や金利商品で広く用いられる。
- **Actual/Actual ICMA** 実日数を、利払期間の実日数と年間利払回数に基づいて年換算する。固定利付債で用いられる代表的な方式である。
- **Actual/Actual ISDA** 対象期間を平年部分と閏年部分に分け、それぞれ 365 または 366 で除する。
- **30/360 Bond Basis** 1 か月を 30 日、1 年を 360 日として計算し、月末の日付には所定の調整を行う。
- **30E/360** 開始日または終了日が 31 日の場合に 30 日へ調整し、1 か月を 30 日、1 年を 360 日として計算する。

30/360 方式は手計算を容易にするために考案された方式であり、半年、四半期および 1 か月をそれぞれ 180 日、90 日および 30 日として扱える。一方、Actual 方式では暦上の実日数を用いるため、同じ利率でも計算期間によって利息額が異なることがある。日数計算規則の違いは、経過利子だけでなく、割引期間や利回りにも影響する。詳細は [Day count convention](#) を参照されたい。ただし、日数計算規則はすべての計算に同じものを用いればよいわけではない。Actual/Actual ICMA や 30/360 Bond Basis は、規則的な利払期間を基準としてクーポン額や経過利子を求める用途に適している。一方、金利期間構造では、評価日から各将来日までの距離を一貫した時間軸へ変換する必要があるため、Actual/360 または Actual/365 Fixed のような単純な規則を用いることが多い。

とくに Actual/Actual ICMA は、基準となる利払期間 (reference period) を必要とする。不規則な初回または最終利払期間では、同じ二つの日付であっても、基準期間の与え方によって年数が変わり得る。利払スケジュールを持たない金利カーブにこの規則をそのまま使用すると、割引期間や割引係数が不自然になる場合がある。したがって、債券のクーポン計算に用いる日数計算規則と、割引カーブの時間軸に用いる日数計算規則は区別して設定する必要がある。

受渡日や利払日が非営業日に当たる場合には、対象市場の休日カレンダーと営業日調整規則を用いる。代表的な規則には、次営業日へ移す Following、翌月へ越える場合には前営業日へ戻す Modified Following、前営業日へ戻す Preceding、および日付を調整しない Unadjusted がある。

1.2.2 日本の債券市場における慣行

日本の公社債市場では、単価と複利利回りに加えて、単利利回りが広く用いられてきた。日本証券業協会の公社債店頭売買参考統計値でも、原則として報告された単利利回りから単価を求め、その単価から複利利回りを計算する手順が採用されている。

利付債の経過利子に用いる経過日数は、直前利払期日から受渡日までの実日数を片端入れて数える。一方、最終利回りに用いる残存日数は、受渡日の翌日から償還日までを片端入れて数えるため、経過日数と残存日数を区別する必要がある。

残存日数を N 、残存年数を T とすると、残存年数は次式で求める。

$$T = \frac{N}{365}$$

日本市場の実務では、残存期間が 1 年以上の場合には、原則として期間中の 2 月 29 日を残存日数から除外する。残存期間が 1 年未満の場合には 2 月 29 日を含めるが、分母は 365 のままとする。

さらに、計算された残存年数は小数点以下第 8 位で切り捨て、小数点以下第 7 位までの値を以後の計算に用いる。四捨五入ではなく切り捨てである点が重要である。

額面 100 円、年間クーポン額を C 、償還価格を R 、単価を P 、残存年数を T とすると、単利の最終利回り Y は次式で表される。

$$Y = \frac{C + \frac{R - P}{T}}{P} \times 100$$

この単利利回りは、将来の各キャッシュフローを支払時点ごとに割り引く複利利回りとは異なる。クーポンと償還差損益を残存年数で年換算する日本市場独自の表示方法であり、キャッシュフローごとの時間価値を直接には反映しない。

利回りから単価を逆算する場合には、小数点以下第 7 位までに切り捨てた残存年数を使用する。算出された単価も、原則として小数点以下第 4 位を切り捨て、小数点以下第 3 位までとする。

単価で約定する取引では、計算途中の端数が約定価格を直接変更する場面は限られる。しかし、単利利回りから単価を逆算する場合には、残存年数の切り捨てが償還差損益の年換算額を変化させ、さらに単価の切り捨てによって価格差が生じる。このため、日本の債券計算では、残存日数、残存年数および単価の端数処理を正確に再現する必要がある。

なお、以下のコード例では一般的な債券評価の確認を目的として、30/360 Bond Basis、半年複利および Unadjusted を採用している。したがって、この設定は日本国内債の単利利回り慣行をそのまま再現するものではない。

以下では、日本の休日カレンダー、取引日から 2 営業日後の受渡日、30/360 Bond Basis、半年複利および年 2 回払いを設定する。

```
jp_calendar <- QuantLib::Japan()

dc_a365 <- QuantLib::Actual365Fixed()
dc_a360 <- QuantLib::Actual360()
dc_30 <- GaussQuant::day_counter_GQL(
  "Thirty360",
  convention = "BondBasis"
)

compd_compounded <- QuantLib::Compounding_Compounded_get()
compd_simple <- QuantLib::Compounding_Simple_get()
```



```
freq_annual <- QuantLib::Frequency_Annual_get()
freq_semiannual <- QuantLib::Frequency_Semiannual_get()

trade_date_chr <- "2022-08-19"
issue_date_chr <- "2022-07-28"
maturity_date_chr <- "2025-07-28"

trade_date <- GaussQuant::date_GQL(trade_date_chr)
issue_date_ql <- GaussQuant::date_GQL(issue_date_chr)
maturity_date_ql <- GaussQuant::date_GQL(maturity_date_chr)

settlement_days <- 2L

GaussQuant::set_eval_date_GQL(trade_date)

settlement_date <- GaussQuant::advance_days_GQL(
  calendar_obj = jp_calendar,
  date_obj = trade_date,
  n_days = settlement_days
)

settlement_date_chr <- as.character(settlement_date)
settlement_date_ql <- GaussQuant::date_GQL(settlement_date)

setup_tbl <- tibble::tibble(
  item = c(
    "trade_date",
    "settlement_date",
    "issue_date",
    "maturity_date",
    "settlement_days",
    "day_counter",
    "compounding",
    "frequency"
  ),
  value = c(
    trade_date_chr,
    settlement_date_chr,
    issue_date_chr,
    maturity_date_chr,
    as.character(settlement_days),
    "Thirty360(BondBasis)",
    "Compounded",
```

```

    "Semiannual"
  )
)

GaussQuant::show_tbl_GQH(setup_tbl, "Bond setup dates and conventions")
#> # A tibble: 8 x 2
#>   item          value
#>   <chr>         <chr>
#> 1 trade_date    2022-08-19
#> 2 settlement_date 2022-08-23
#> 3 issue_date     2022-07-28
#> 4 maturity_date  2025-07-28
#> 5 settlement_days 2
#> 6 day_counter    Thirty360(BondBasis)
#> 7 compounding    Compounded
#> 8 frequency      Semiannual

```

表示された表から、取引日、受渡日、発行日、満期日および評価に用いる市場慣行を確認する。受渡日は日本の休日カレンダーを用いて取引日から 2 営業日後に計算される。

1.3 固定利付債計算

額面 100、年率 0.37%、半年払い、満期 2025 年 7 月 28 日の固定利付債を作成する。利払スケジュールは満期日から発行日へ向かって 6 か月ごとに生成し、各利払日は Unadjusted とする。

```

face_amount <- 100
coupon_rate <- 0.00370
clean_price_input <- 97.0

bond_obj <- GaussQuant::fixed_rate_bond_GQL(
  issue_date = issue_date_chr,
  maturity_date = maturity_date_chr,
  face_amount = face_amount,
  coupon_rate = coupon_rate,
  settlement_days = settlement_days,
  calendar = jp_calendar,
  accrual_day_counter = dc_30,
  payment_convention = "Unadjusted",
  schedule_convention = "Unadjusted",
  maturity_convention = "Unadjusted",
  date_generation = "Backward",
  end_of_month = FALSE,
  schedule_frequency = GaussQuant::period_months_GQL(6)
)

```

```

bond_schedule <- QuantLib::Schedule(
  issue_date_ql,
  maturity_date_ql,
  GaussQuant::period_months_GQL(6),
  jp_calendar,
  "Unadjusted",
  "Unadjusted",
  "Backward",
  FALSE
)

schedule_tbl <- GaussQuant::schedule_dates_GQL(bond_schedule)

bond_setup_tbl <- tibble::tibble(
  metric = c(
    "bond_class",
    "face_amount",
    "coupon_rate",
    "clean_price_input"
  ),
  value = list(
    paste(class(bond_obj), collapse = ", "),
    face_amount,
    coupon_rate,
    clean_price_input
  )
)

GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(bond_setup_tbl), "Fixed-rate bond object")
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 bond_class  _p_FixedRateBond
#> 2 face_amount 100.00000000
#> 3 coupon_rate 0.00370000
#> 4 clean_price_input 97.00000000
GaussQuant::show_tbl_GQH(schedule_tbl, "Bond schedule", n = 20)
#> # A tibble: 7 x 1
#>   schedule_date
#>   <date>
#> 1 2022-07-28
#> 2 2023-01-28
#> 3 2023-07-28

```

```
#> 4 2024-01-28
#> 5 2024-07-28
#> 6 2025-01-28
#> 7 2025-07-28
```

債券オブジェクトの条件と、発行日から満期日までの利払日を確認する。

1.3.1 クリーン価格から最終利回り

市場で提示されるクリーン価格 97 から、将来キャッシュフローの現在価値がこの価格と一致する最終利回りを逆算する。

```
price_to_yield <- GaussQuant::bond_yield_from_clean_safe_GQL(
  bond = bond_obj,
  clean_price = clean_price_input,
  day_counter = dc_30,
  compounding = compd_compounded,
  frequency = freq_semiannual
)

yield_tbl <- tibble::tibble(
  metric = c(
    "clean_price_input",
    "yield_from_clean_price",
    "yield_percent"
  ),
  value = list(
    clean_price_input,
    price_to_yield,
    100 * price_to_yield
  )
)

GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(yield_tbl), "Clean price -> yield")
#> # A tibble: 3 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 clean_price_input 97.00000000
#> 2 yield_from_clean_price 0.01418724
#> 3 yield_percent    1.41872370
```

債券価格が額面を下回っているため、算出される最終利回りはクーポンレートを上回る。

1.3.2 最終利回りから価格

前節で求めた最終利回りを使って、クリーン価格、ダーティ価格および経過利子を計算する。両価格の関係は次式で表される。

$$\text{Dirty Price} = \text{Clean Price} + \text{Accrued Interest}$$

```
price_tbl <- GaussQuant::bond_price_measures_GQL(
  bond = bond_obj,
  yield = price_to_yield,
  day_counter = dc_30,
  compounding = cmpd_compounded,
  frequency = freq_semiannual,
  settlement_date = settlement_date_chr
)

price_tbl <- GaussQuant::complete_accrual_metrics_GQL(
  price_tbl = price_tbl,
  bond_schedule = bond_schedule,
  settlement_date = settlement_date,
  day_counter = dc_30
)

price_tbl_display <- GaussQuant::metric_tbl_display_GQH(price_tbl)

GaussQuant::show_tbl_GQH(price_tbl_display, "Yield -> price summary")
#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 settlement_date 2022-08-23
#> 2 previous_coupon_date 2022-07-28
#> 3 accrued_days    25.00000000
#> 4 accrued_amount  0.02569444
#> 5 clean_price     97.00000012
#> 6 dirty_price     97.02569456

dirty_price_from_yield <- GaussQuant::get_metric_num_GQH(price_tbl, "dirty_price")
clean_price_from_yield <- GaussQuant::get_metric_num_GQH(price_tbl, "clean_price")
accrued_amount <- GaussQuant::get_metric_num_GQH(price_tbl, "accrued_amount")

cat(
  "clean price:",
  sprintf("%.8f", clean_price_from_yield),
```

```

"\n",
"dirty price:",
sprintf("%.8f", dirty_price_from_yield),
"\n",
"accrued:",
sprintf("%.8f", accrued_amount),
"\n"
)
#> clean price: 97.00000012
#> dirty price: 97.02569456
#> accrued: 0.02569444

```

再計算されたクリーン価格が入力価格 97 と一致することを確認する。ダーティ価格との差額は、前回利払日から受渡日までに発生した経過利子である。

1.3.3 金利リスク

修正デュレーションは、利回りの小さな変化に対する債券価格の一次感応度を表す。コンベクシティは価格と利回りの非線形な関係を補正する二次感応度である。

```

risk_tbl <- GaussQuant::bond_risk_measures_GQL(
  bond = bond_obj,
  yield = price_to_yield,
  day_counter = dc_30,
  compounding = cmpd_compounded,
  frequency = freq_semiannual
)

risk_tbl_display <- GaussQuant::metric_tbl_display_GQH(risk_tbl)

GaussQuant::show_tbl_GQH(risk_tbl_display, "Bond risk measures")
#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 modified_duration 2.89593215
#> 2 bpv              0.02809798
#> 3 dv01              0.02809798
#> 4 convexity         9.84952310

modified_duration <- GaussQuant::get_metric_num_GQH(risk_tbl, "modified_duration")
convexity_value <- GaussQuant::get_metric_num_GQH(risk_tbl, "convexity")

cat(
  "modified duration:",

```

```

    sprintf("%.8f", modified_duration),
    "\n",
    "convexity:",
    sprintf("%.8f", convexity_value),
    "\n"
)
#> modified duration: 2.89593215
#> convexity: 9.84952310

```

修正デュレーションが大きいほど、同じ利回り変化に対する価格変化も大きくなる。コンベクシティは、利回り変化が大きくなるほど重要になる。

1.3.4 デュレーションとコンベクシティによる価格変化の近似

利回り変化を Δy とすると、債券価格の相対変化は修正デュレーションとコンベクシティを使って近似できる。

$$\frac{\Delta P}{P} \approx -D_{\text{mod}} \Delta y + \frac{1}{2} C (\Delta y)^2$$

```

handcalc_tbl <- GaussQuant::bond_risk_handcalc_GQL(
  bond = bond_obj,
  yield = price_to_yield,
  dirty_price = dirty_price_from_yield,
  modified_duration = modified_duration,
  convexity = convexity_value,
  day_counter = dc_30,
  compounding = cmpd_compounded,
  frequency = freq_semiannual
)

handcalc_tbl_display <- GaussQuant::metric_tbl_display_GQH(handcalc_tbl)

GaussQuant::show_tbl_GQH(handcalc_tbl_display, "Risk hand calculations")
#> # A tibble: 5 x 2
#>   metric                value
#>   <chr>                <chr>
#> 1 bpv_hand            -0.02809798
#> 2 modified_duration_hand 0.02895932
#> 3 convexity_hand       0.00098495
#> 4 price_up_100bp       94.26306473
#> 5 price_approx_delta_gamma 94.26367912

```

一次近似と二次近似を比較し、コンベクシティを加えることで実際の価格変化に近づくことを確認する。

1.4 キャッシュフローによる検証

各利払日のクーポンと満期時の元本を最終利回りで割り引き、ダーティ価格を手計算する。そこから経過利子を差し引くことでクリーン価格を求める。

```

schedule_dates_chr <- schedule_tbl[[1]] |>
  as.character()

schedule_dates_chr <- schedule_dates_chr[!is.na(schedule_dates_chr)]

future_dates_chr <- schedule_dates_chr[schedule_dates_chr > settlement_date_chr]

coupon_per_period <- face_amount * coupon_rate / 2

manual_cf_tbl <- tibble::tibble(
  payment_date = future_dates_chr,
  period_no = seq_along(future_dates_chr),
  cashflow = coupon_per_period
) |>
  dplyr::mutate(
    cashflow = ifelse(
      .data$payment_date == maturity_date_chr,
      .data$cashflow + face_amount,
      .data$cashflow
    ),
    ql_date = purrr::map(payment_date, GaussQuant::date_GQL),
    time = purrr::map_dbl(
      ql_date,
      ~ dc_30$yearFraction(settlement_date_ql, .x)
    ),
    discount_factor = 1 / (1 + price_to_yield / 2)^(2 * time),
    pv = cashflow * discount_factor
  ) |>
  dplyr::select(-ql_date)

manual_dirty_price <- sum(manual_cf_tbl$pv, na.rm = TRUE)
manual_clean_price <- manual_dirty_price - accrued_amount

manual_summary_tbl <- tibble::tibble(
  metric = c(
    "manual_dirty_price",
    "manual_clean_price",
    "wrapper_dirty_price",
  )
)

```



```

    "wrapper_clean_price",
    "dirty_price_difference",
    "clean_price_difference"
  ),
  value = list(
    manual_dirty_price,
    manual_clean_price,
    dirty_price_from_yield,
    clean_price_from_yield,
    manual_dirty_price - dirty_price_from_yield,
    manual_clean_price - clean_price_from_yield
  )
)

GaussQuant::show_tbl_GQH(manual_cf_tbl, "Manual cashflow PV table")
#> # A tibble: 6 x 6
#>   payment_date period_no cashflow    time discount_factor      pv
#>   <chr>          <int>    <dbl>    <dbl>          <dbl>    <dbl>
#> 1 2023-01-28         1    0.185 0.43056          0.99393 0.18388
#> 2 2023-07-28         2    0.185 0.93056          0.98693 0.18258
#> 3 2024-01-28         3    0.185 1.4306          0.97998 0.18130
#> 4 2024-07-28         4    0.185 1.9306          0.97308 0.18002
#> 5 2025-01-28         5    0.185 2.4306          0.96622 0.17875
#> 6 2025-07-28         6  100.18 2.9306          0.95942 96.119

GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(manual_summary_tbl), "Manual PV check")
#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 manual_dirty_price 97.02569456
#> 2 manual_clean_price 97.00000012
#> 3 wrapper_dirty_price 97.02569456
#> 4 wrapper_clean_price 97.00000012
#> 5 dirty_price_difference 0.00000000
#> 6 clean_price_difference 0.00000000

```

手計算と関数による価格との差が十分に小さいことを確認する。これにより、キャッシュフロー、割引期間および経過利子の関係を検算できる。

1.4.1 フラットな利回りによる価格評価

次に、最終利回りを一定とするフラットな割引曲線を構築し、割引エンジンから債券価格を計算する。

```

flat_yield_handle <- GaussQuant::bond_flat_forward_handle_GQL(
  settlement_date = settlement_date_chr,

```

```

    rate = price_to_yield,
    day_counter = dc_30,
    compounding = compd_compounded,
    frequency = freq_semiannual
)

flat_engine <- QuantLib::DiscountingBondEngine(flat_yield_handle)

QuantLib::Instrument_setPricingEngine(
  bond_obj,
  flat_engine
)
#> NULL

flat_dirty_price <- QuantLib::Instrument_NPV(bond_obj)

flat_yield_tbl <- tibble::tibble(
  metric = c(
    "flat_yield",
    "dirty_price_from_flat_engine",
    "dirty_price_from_yield",
    "difference"
  ),
  value = list(
    price_to_yield,
    flat_dirty_price,
    dirty_price_from_yield,
    flat_dirty_price - dirty_price_from_yield
  )
)

GaussQuant::show_tbl_GQH(GaussQuant::metric_tbl_display_GQH(flat_yield_tbl), "Flat yield pricing")
#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <chr>
#> 1 flat_yield                          0.01418724
#> 2 dirty_price_from_flat_engine 97.02569456
#> 3 dirty_price_from_yield       97.02569456
#> 4 difference                      -0.00000000

```

フラットな利回りを使った割引価格と、最終利回りから直接求めたダーティ価格を比較する。### 1.4.2 Z-spread による価格調整

Z-spread は、基準となる割引カーブの各年限に一律に加えるスプレッドである。ここでは基準カーブに 50

ベースポイントを上乘せし、債券価格への影響を確認する。

```
z_spread <- 50 / 10000

zspread_result <- GaussQuant::bond_npv_with_zspread_GQL(
  bond = bond_obj,
  base_curve_handle = flat_yield_handle,
  z_spread = z_spread,
  compounding = compd_compounded,
  frequency = freq_semiannual,
  day_counter = dc_30
)

zspread_dirty_price <- zspread_result$npv

zspread_tbl <- tibble::tibble(
  metric = c(
    "base_yield",
    "z_spread_bp",
    "dirty_price_before_spread",
    "dirty_price_after_spread",
    "price_difference"
  ),
  value = list(
    price_to_yield,
    10000 * z_spread,
    flat_dirty_price,
    zspread_dirty_price,
    zspread_dirty_price - flat_dirty_price
  )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::metric_tbl_display_GQH(zspread_tbl),
  "Z-spread pricing"
)

#> # A tibble: 5 x 2
#>   metric                value
#>   <chr>                <chr>
#> 1 base_yield          0.01418724
#> 2 z_spread_bp         50.00000000
#> 3 dirty_price_before_spread 97.02569456
#> 4 dirty_price_after_spread 95.63266391
#> 5 price_difference     -1.39303065
```

正の Z-spread を加えると割引率が上昇するため、債券価格は低下する。Z-spread には発行体の信用リスクだけでなく、流動性、市場需給および基準カーブとの相違も含まれ得るため、純粋なデフォルト確率を直接表す指標ではない。ここでは全期間への一律なスプレッドだけを扱い、年限別の非平行シフトは金利カーブの章で扱う。### 1.4.3 債券キャッシュフロー

債券に含まれる将来のクーポンと元本償還を一覧で確認する。

```
bond_cf_tbl <- GaussQuant::bond_cashflow_table_GQL(
  bond = bond_obj
)

GaussQuant::show_tbl_GQH(bond_cf_tbl, "Bond cashflow table", n = 30)
#> # A tibble: 7 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2023-01-28    0.18500
#> 2 2023-07-28    0.18500
#> 3 2024-01-28    0.18500
#> 4 2024-07-28    0.18500
#> 5 2025-01-28    0.18500
#> 6 2025-07-28    0.18500
#> 7 2025-07-28   100
```

各行は支払日と支払額を表し、満期日のキャッシュフローには最終クーポンと元本償還が含まれる。

1.5 計算結果のまとめ

これまでに設定した日付、価格、利回りおよび金利リスク指標を一つの表にまとめる。

```
summary_tbl <- tibble::tibble(
  metric = c(
    "trade_date",
    "settlement_date",
    "issue_date",
    "maturity_date",
    "clean_price_input",
    "yield_from_clean_price",
    "clean_price_from_yield",
    "dirty_price_from_yield",
    "accrued_amount",
    "modified_duration",
    "convexity"
  ),
  value = c(
    GaussQuant::iso_GQL(trade_date),
```

```
    settlement_date_chr,  
    issue_date_chr,  
    maturity_date_chr,  
    sprintf("%.8f", clean_price_input),  
    sprintf("%.8f", price_to_yield),  
    sprintf("%.8f", clean_price_from_yield),  
    sprintf("%.8f", dirty_price_from_yield),  
    sprintf("%.8f", accrued_amount),  
    sprintf("%.8f", modified_duration),  
    sprintf("%.8f", convexity_value)  
  )  
)  
  
GaussQuant::show_tbl_GQH(summary_tbl, "Bond analysis summary", n = 20)  
#> # A tibble: 11 x 2  
#>   metric          value  
#>   <chr>          <chr>  
#> 1 trade_date      2022-08-19  
#> 2 settlement_date 2022-08-23  
#> 3 issue_date       2022-07-28  
#> 4 maturity_date    2025-07-28  
#> 5 clean_price_input 97.00000000  
#> 6 yield_from_clean_price 0.01418724  
#> 7 clean_price_from_yield 97.00000012  
#> 8 dirty_price_from_yield 97.02569456  
#> 9 accrued_amount    0.02569444  
#> 10 modified_duration 2.89593215  
#> 11 convexity        9.84952310  
  
cat("\nchapter01 bonds stable rewrite completed successfully.\n")  
#>  
#> chapter01 bonds stable rewrite completed successfully.
```

第 II-2 章金利カーブ構築と債券先物 CTD、BPV

債券、金利スワップおよび金利オプションの評価では、将来キャッシュフローを現在価値へ換算するための金利期間構造が必要となる。金利期間構造は、満期ごとの金利を並べた単なる曲線ではない。市場で観測される預金金利、債券価格、FRA およびスワップレートなどから、各将来日の割引係数を整合的に推定した結果である。

同じ期間構造は、割引係数、ゼロ金利またはフォワード金利として表現できる。これらは相互に変換可能であるが、表示方法によって曲線の特徴の見え方が異なる。割引係数は現在価値計算に直接用いられ、ゼロ金利は満期ごとの平均的な金利水準を示し、フォワード金利は将来の区間ごとの限界的な金利を示す。

本章では、最初にゼロ金利ノードから単純なカーブを構築し、割引係数、ゼロ金利およびフォワード金利の関係を確認する。次に、補間方式による曲線形状の違いを比較する。その後、預金と債券、ならびに預金とスワップを用いたブートストラップを行い、最後に Implied Term Structure と平行シフトを確認する。

2.1 金利カーブの基礎

評価日を 0、将来時点を t とし、時点 t に 1 を受け取るキャッシュフローの現在価値を $D(0, t)$ とする。 $D(0, t)$ は割引係数である。連続複利のゼロ金利を $z(0, t)$ とすると、両者の関係は次式で表される。

$$D(0, t) = \exp[-z(0, t)t]$$

したがって、割引係数からゼロ金利を求めると次式となる。

$$z(0, t) = -\frac{\log D(0, t)}{t}$$

二つの時点 t_1 と t_2 の間に適用される連続複利のフォワード金利を $f(t_1, t_2)$ とすると、次式で求められる。

$$f(t_1, t_2) = -\frac{\log [D(0, t_2)/D(0, t_1)]}{t_2 - t_1}$$

割引係数、ゼロ金利およびフォワード金利は同じ期間構造を異なる方法で表している。ただし、補間方法の違いは、とくにフォワード金利の形状に強く表れる。

2.1.1 ゼロ金利ノード

最初に、評価日と複数の将来日について連続複利のゼロ金利を設定する。ここで用いる数値は、カーブの構造を確認するための例示値である。

```
evaluation_date_chr <- "2022-08-19"

GaussQuant::set_eval_date_GQL(evaluation_date_chr)

zero_nodes <- tibble::tribble(
  ~tenor, ~date, ~zero_rate,
  "0D",   as.Date("2022-08-19"), 0.0005,
  "1M",   as.Date("2022-09-19"), 0.0006,
  "3M",   as.Date("2022-11-21"), 0.0008,
  "6M",   as.Date("2023-02-20"), 0.0011,
  "1Y",   as.Date("2023-08-21"), 0.0018,
  "2Y",   as.Date("2024-08-19"), 0.0032,
  "3Y",   as.Date("2025-08-19"), 0.0048,
  "5Y",   as.Date("2027-08-19"), 0.0075,
  "10Y",  as.Date("2032-08-19"), 0.0115
)

GaussQuant::show_tbl_GQH(
  zero_nodes,
  "Zero-rate nodes",
  n = 20
)

#> # A tibble: 9 x 3
#>   tenor date      zero_rate
#>   <chr> <date>      <dbl>
#> 1 0D    2022-08-19    0.0005
#> 2 1M    2022-09-19    0.0006
#> 3 3M    2022-11-21    0.0008
#> 4 6M    2023-02-20    0.0011
#> 5 1Y    2023-08-21    0.0018
#> 6 2Y    2024-08-19    0.0032
#> 7 3Y    2025-08-19    0.0048
#> 8 5Y    2027-08-19    0.0075
#> 9 10Y   2032-08-19    0.0115
```

2.1.2 ゼロカーブ

`build_zero_curve_GQL()` は、日付とゼロ金利の組から QuantLib の ZeroCurve を構築する。ここではカーブの時間軸に Actual/365 Fixed を用いる。

```
zero_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = as.character(zero_nodes$date),
  zero_rates = zero_nodes$zero_rate,
  day_counter = "Actual365Fixed"
)

zero_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  zero_curve
)

zero_curve_tbl <- GaussQuant::curve_tbl_GQH(
  curve = zero_curve,
  tenors = c(
    "1D", "1W", "1M", "3M", "6M",
    "1Y", "2Y", "3Y", "5Y", "7Y", "10Y"
  )
)

GaussQuant::show_tbl_GQH(
  zero_curve_tbl,
  "Zero curve",
  n = 20
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>      <dbl>
#> 1 1D    2022-08-20  1.00000  0.00050323
#> 2 1W    2022-08-26  0.99999  0.00052258
#> 3 1M    2022-09-19  0.99995  0.00060000
#> 4 3M    2022-11-19  0.99980  0.00079365
#> 5 6M    2023-02-19  0.99945  0.0010967
#> 6 1Y    2023-08-19  0.99821  0.0017923
#> 7 2Y    2024-08-19  0.99361  0.0032000
#> 8 3Y    2025-08-19  0.98569  0.0048000
#> 9 5Y    2027-08-19  0.96317  0.0075
#> 10 7Y    2029-08-19  0.93824  0.0091004
#> 11 10Y   2032-08-19  0.89128  0.0115
```

カーブから取得したゼロ金利は連続複利表示である。入力ノードの間では QuantLib の補間規則に従って金利

が補間される。

2.1.3 カーブの表現

カーブを細かい時間間隔へ展開し、割引係数、ゼロ金利および隣接区間のフォワード金利を計算する。

```
zero_curve_grid <- GaussQuant::curve_grid_tbl_GQL(
  curve = zero_curve,
  n = 240L,
  extrapolate = TRUE
) |>
dplyr::mutate(
  next_time = dplyr::lead(.data$time),
  next_discount = dplyr::lead(.data$discount_factor),
  forward_rate = dplyr::if_else(
    !is.na(.data$next_time) &
      .data$next_time > .data$time &
      .data$discount_factor > 0 &
      .data$next_discount > 0,
    -log(.data$next_discount / .data$discount_factor) /
      (.data$next_time - .data$time),
    NA_real_
  )
)

grid_show_index <- unique(
  round(
    seq(
      from = 1,
      to = nrow(zero_curve_grid),
      length.out = 12
    )
  )
)

GaussQuant::show_tbl_GQH(
  zero_curve_grid |>
  dplyr::slice(grid_show_index) |>
  dplyr::select(
    .data$curve_date,
    .data$time,
    .data$discount_factor,
    .data$zero_rate,
    .data$forward_rate
  )
)
```

```

    ),
    "Zero-curve grid",
    n = 20
)
#> # A tibble: 12 x 5
#>   curve_date      time discount_factor zero_rate forward_rate
#>   <date>         <dbl>          <dbl>    <dbl>      <dbl>
#> 1 2022-08-19     0            1          0      0.00054839
#> 2 2023-07-21  0.92055        0.99845 0.0016808  0.0030308
#> 3 2024-06-06  1.8            0.99477 0.0029154  0.0055000
#> 4 2025-05-08  2.7205        0.98824 0.0043485  0.0087671
#> 5 2026-04-09  3.6411        0.97960 0.0056618  0.010636
#> 6 2027-03-12  4.5644        0.96896 0.0069082  0.013126
#> 7 2028-01-26  5.4411        0.95819 0.0078503  0.012233
#> 8 2028-12-28  6.3644        0.94681 0.0085881  0.013707
#> 9 2029-11-29  7.2849        0.93433 0.0093238  0.015178
#> 10 2030-10-31  8.2055        0.92077 0.010059   0.016652
#> 11 2031-09-17  9.0849        0.90685 0.010762   0.018057
#> 12 2032-08-19 10.008        0.89128 0.0115     NA

```

```
discount_factor_plot <- zero_curve_grid |>
```

```

  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = .data$discount_factor
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Discount factor curve",
    x = NULL,
    y = "Discount factor"
  )

```

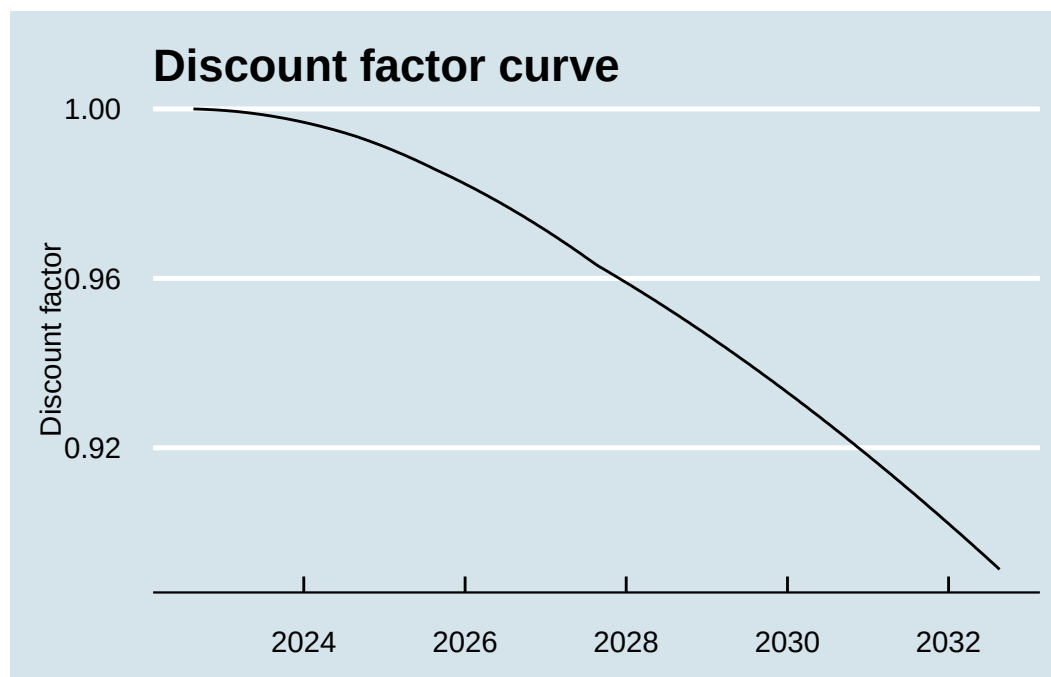
```
zero_rate_plot <- zero_curve_grid |>
```

```

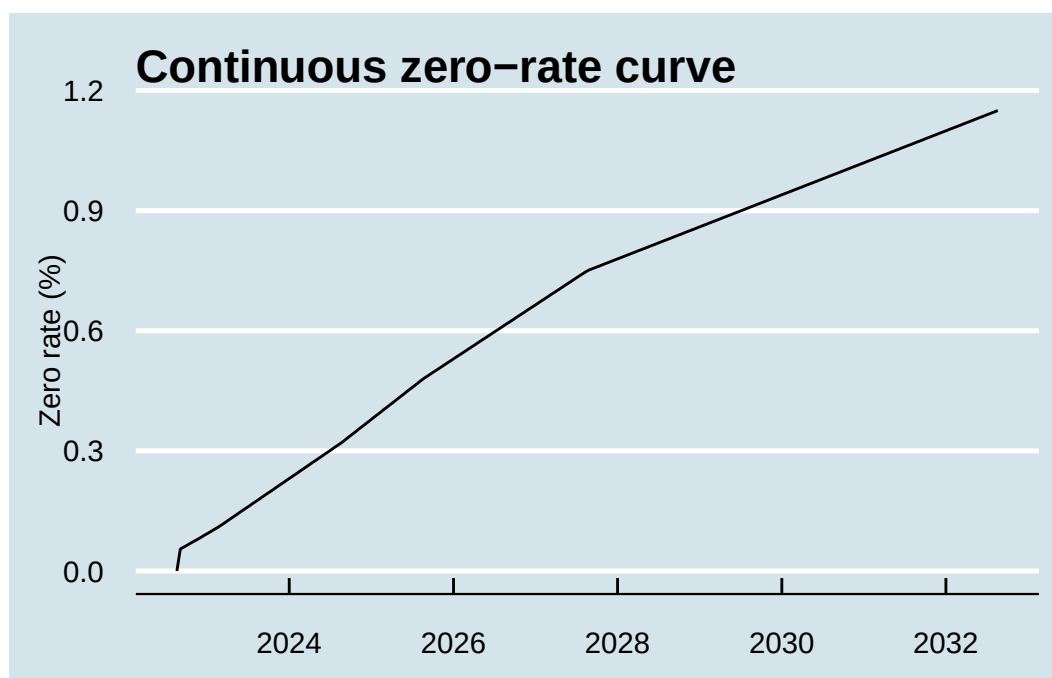
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = 100 * .data$zero_rate
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Continuous zero-rate curve",

```

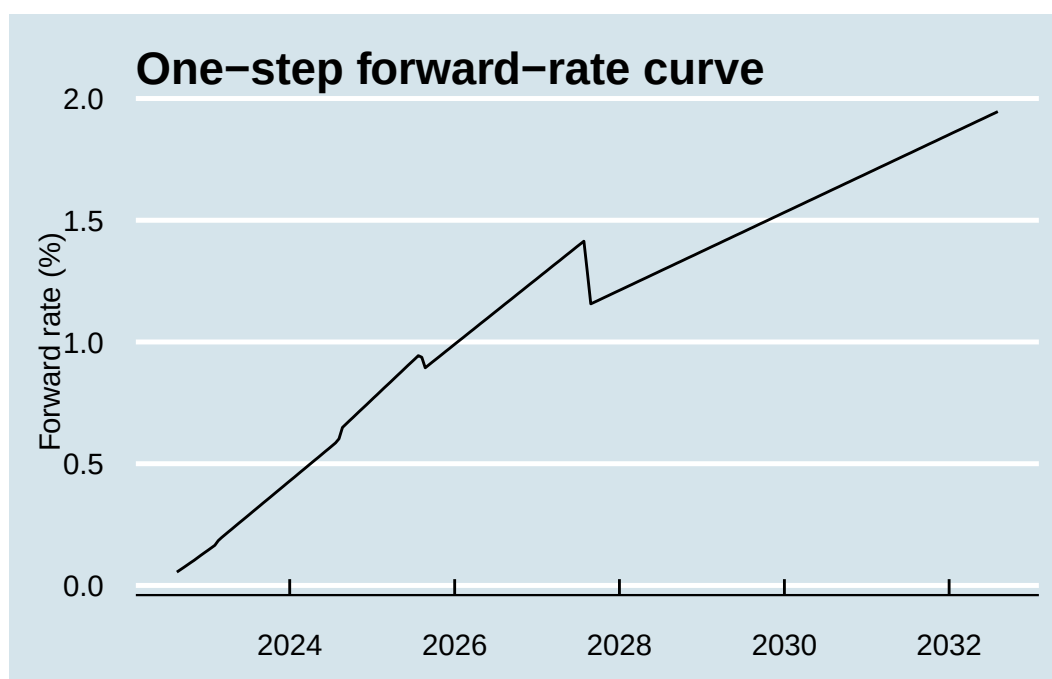
```
x = NULL,  
y = "Zero rate (%)"  
)  
  
forward_rate_plot <- zero_curve_grid |>  
  dplyr::filter(!is.na(.data$forward_rate)) |>  
  ggplot2::ggplot(  
    ggplot2::aes(  
      x = .data$curve_date,  
      y = 100 * .data$forward_rate  
    )  
  ) +  
  ggplot2::geom_line() +  
  ggplot2::labs(  
    title = "One-step forward-rate curve",  
    x = NULL,  
    y = "Forward rate (%)"  
  )  
show_plot_LFL(discount_factor_plot)
```



```
show_plot_LFL(zero_rate_plot)
```



```
show_plot_LFL(forward_rate_plot)
```



割引係数は将来の 1 単位を現在価値へ換算する倍率である。正の金利環境では、満期が長くなるほど一般に小さくなる。ゼロ金利は評価日から各満期までの平均的な金利水準を示す。一方、フォワード金利は隣接する二時点間の金利を表すため、補間方法やノード配置の影響を強く受ける。

2.2 補間と参照日

2.2.1 補間方式とフォワード金利

市場では限られた満期の金利または価格しか観測できないため、ノード間の値を補間する必要がある。補間対象には、ゼロ金利、割引係数、対数割引係数またはフォワード金利などがある。

ここでは、同じゼロ金利ノードから次の二つのカーブを構築する。

- ZeroCurve：ゼロ金利を基礎とするカーブ
- DiscountCurve：ノードのゼロ金利を割引係数へ変換し、割引係数を基礎とするカーブ

```
discount_curve <- GaussQuant::build_discount_curve_GQL(  
  nodes = zero_nodes |>  
    dplyr::select(  
      .data$date,  
      .data$zero_rate  
    ),  
  day_counter = "Actual365Fixed"  
)  
  
discount_curve_handle <- GaussQuant::yield_curve_handle_GQL(  
  discount_curve  
)  
  
comparison_max_time <- min(  
  as.numeric(zero_curve$maxTime()),  
  as.numeric(discount_curve$maxTime())  
)  
  
comparison_time_grid <- seq(  
  from = 0,  
  to = comparison_max_time,  
  length.out = 300  
)  
  
comparison_reference_date <- as.Date(  
  GaussQuant::iso_GQL(  
    zero_curve$referenceDate()  
  )  
)  
  
interpolation_tbl <- tibble::tibble(  
  time = comparison_time_grid,  
  date = comparison_reference_date +
```

```
round(comparison_time_grid * 365)
) |>
dplyr::mutate(
  ql_date = purrr::map(
    as.character(.data$date),
    GaussQuant::date_GQL
  ),
  zero_curve_discount = purrr::map_dbl(
    .data$ql_date,
    ~ GaussQuant::curve_discount_safe_GQL(
      zero_curve,
      .x
    )
  ),
  discount_curve_discount = purrr::map_dbl(
    .data$ql_date,
    ~ GaussQuant::curve_discount_safe_GQL(
      discount_curve,
      .x
    )
  )
) |>
dplyr::select(
  -.data$ql_date
) |>
tidyr::pivot_longer(
  cols = c(
    zero_curve_discount,
    discount_curve_discount
  ),
  names_to = "interpolation",
  values_to = "discount_factor"
) |>
dplyr::mutate(
  interpolation = dplyr::recode(
    .data$interpolation,
    zero_curve_discount = "ZeroCurve",
    discount_curve_discount = "DiscountCurve"
  )
) |>
dplyr::group_by(
  .data$interpolation
) |>
```

```
dplyr::arrange(
  .data$interpolation,
  .data$time
) |>
dplyr::mutate(
  zero_rate = dplyr::if_else(
    .data$time > 0 &
    .data$discount_factor > 0,
    -log(.data$discount_factor) /
    .data$time,
    zero_nodes$zero_rate[[1]]
  ),
  next_time = dplyr::lead(
    .data$time
  ),
  next_discount = dplyr::lead(
    .data$discount_factor
  ),
  forward_rate = dplyr::if_else(
    !is.na(.data$next_time) &
    .data$next_time > .data$time &
    .data$discount_factor > 0 &
    .data$next_discount > 0,
    -log(
      .data$next_discount /
      .data$discount_factor
    ) / (
      .data$next_time -
      .data$time
    ),
    NA_real_
  )
) |>
dplyr::ungroup()
```

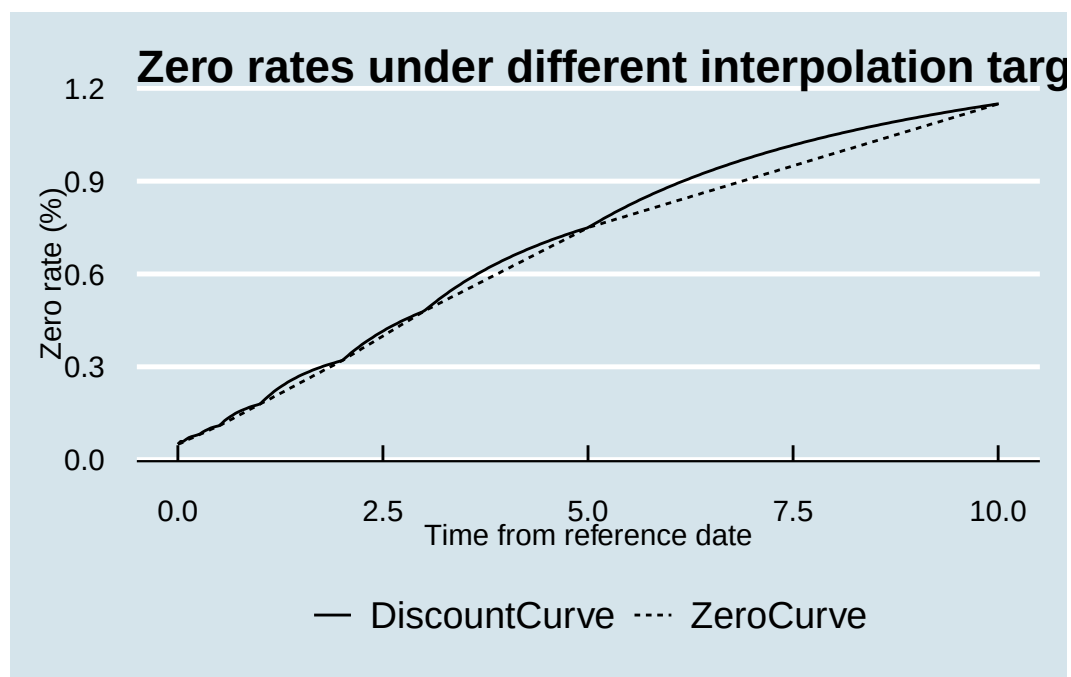
```
interpolation_zero_plot <- interpolation_tbl |>
ggplot2::ggplot(
  ggplot2::aes(
    x = .data$time,
    y = 100 * .data$zero_rate,
    linetype = .data$interpolation
  )
) +
ggplot2::geom_line() +
```

```

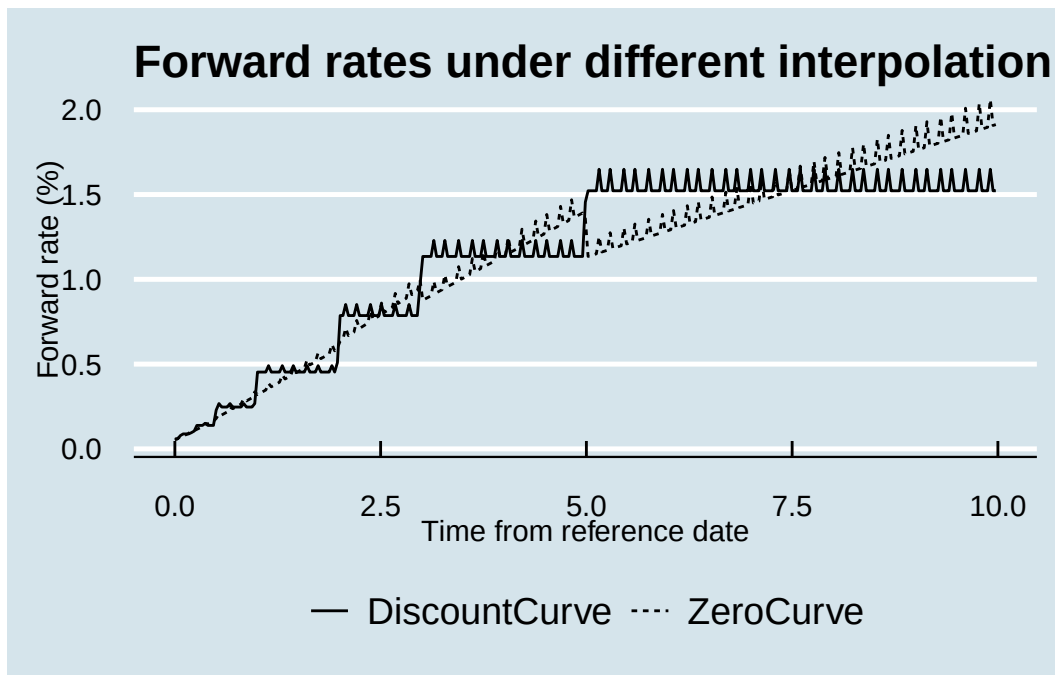
ggplot2::labs(
  title = "Zero rates under different interpolation targets",
  x = "Time from reference date",
  y = "Zero rate (%)",
  linetype = NULL
)

interpolation_forward_plot <- interpolation_tbl |>
  dplyr::filter(!is.na(.data$forward_rate)) |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$time,
      y = 100 * .data$forward_rate,
      linetype = .data$interpolation
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Forward rates under different interpolation targets",
    x = "Time from reference date",
    y = "Forward rate (%)",
    linetype = NULL
  )
show_plot_LFL(interpolation_zero_plot)

```



```
show_plot_LFL(interpolation_forward_plot)
```

両カーブは入力ノードではほぼ同じ情報を再現するが、ノード間では異なる形状を示す。ゼロ金利のグラフでは差が小さく見えても、フォワード金利へ変換すると差が拡大することがある。したがって、カーブの妥当性はゼロ金利だけでなく、フォワード金利も確認する必要がある。

2.2.2 参照日と評価日

カーブの参照日は、割引係数が 1 となる時点であり、時間軸の起点である。日付ノードから構築したカーブでは、通常、最初のノード日が固定参照日となる。

次の確認では、QuantLib の評価日を変更した後も、すでに構築済みの ZeroCurve の参照日が変わらないことを確認する。

```
reference_date_before <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)

GaussQuant::set_eval_date_GQL(
  "2022-09-19"
)

reference_date_after <- as.Date(
  GaussQuant::iso_GQL(
    zero_curve$referenceDate()
  )
)
```

```
reference_date_tbl <- tibble::tribble(
  ~item, ~value,
  "curve_reference_date_before", as.character(reference_date_before),
  "evaluation_date_after_change", "2022-09-19",
  "curve_reference_date_after", as.character(reference_date_after)
)

GaussQuant::show_tbl_GQH(
  reference_date_tbl,
  "Fixed reference date"
)

#> # A tibble: 3 x 2
#>   item                                value
#>   <chr>                                <chr>
#> 1 curve_reference_date_before 2022-08-19
#> 2 evaluation_date_after_change 2022-09-19
#> 3 curve_reference_date_after 2022-08-19

GaussQuant::set_eval_date_GQL(
  evaluation_date_chr
)
```

固定参照日のカーブは、評価日を変更しても参照日が変化しない。一方、決済日数と休日カレンダーから参照日を決定する Moving Reference Date 型のカーブでは、評価日の変更に応じて参照日も移動する。評価日を進めるシナリオ分析では、既存カーブを固定したまま使うのか、市場データから新しいカーブを再構築するのかを区別する必要がある。

2.3 ブートストラップ

2.3.1 預金と債券

市場ではゼロ金利を直接観測できる満期は限られている。そのため、短期預金、国債価格およびスワップレートなどから、割引係数を順次推定するブートストラップを行う。

短期部分では預金金利から最初の割引係数を求め、その先では固定利付債の市場価格と理論価格が一致するようにカーブを延長する。GaussQuant の `build_bond_discount_curve_GQL()` は、短期預金と複数の固定利付債を用いて PiecewiseFlatForward カーブを構築する。

```
GaussQuant::set_eval_date_GQL(
  "2008-09-15"
)

bond_curve <- GaussQuant::build_bond_discount_curve_GQL(
  settlement_date = "2008-09-18",
  fixing_days = 3,
```

```

    settlement_days = 3
  )

bond_curve_tbl <- GaussQuant::curve_tbl_GQH(
  curve = bond_curve,
  tenors = c(
    "1M", "3M", "6M", "1Y",
    "2Y", "3Y", "5Y", "7Y",
    "10Y", "15Y", "20Y"
  )
)

GaussQuant::show_tbl_GQH(
  bond_curve_tbl,
  "Deposit and bond bootstrap curve",
  n = 30
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1M    2008-10-18  0.99921 0.0096148
#> 2 3M    2008-12-18  0.99761 0.0096148
#> 3 6M    2009-03-18  0.99286 0.014471
#> 4 1Y    2009-09-18  0.98097 0.019229
#> 5 2Y    2010-09-18  0.95732 0.021818
#> 6 3Y    2011-09-18  0.92904 0.024541
#> 7 5Y    2013-09-18  0.85971 0.030237
#> 8 7Y    2015-09-18  0.78217 0.035102
#> 9 10Y   2018-09-18  0.67867 0.038764
#> 10 15Y  2023-09-18  0.53179 0.042103
#> 11 20Y  2028-09-18  0.41668 0.043772

bond_curve_grid <- GaussQuant::curve_grid_tbl_GQL(
  curve = bond_curve,
  n = 300L,
  extrapolate = TRUE
)

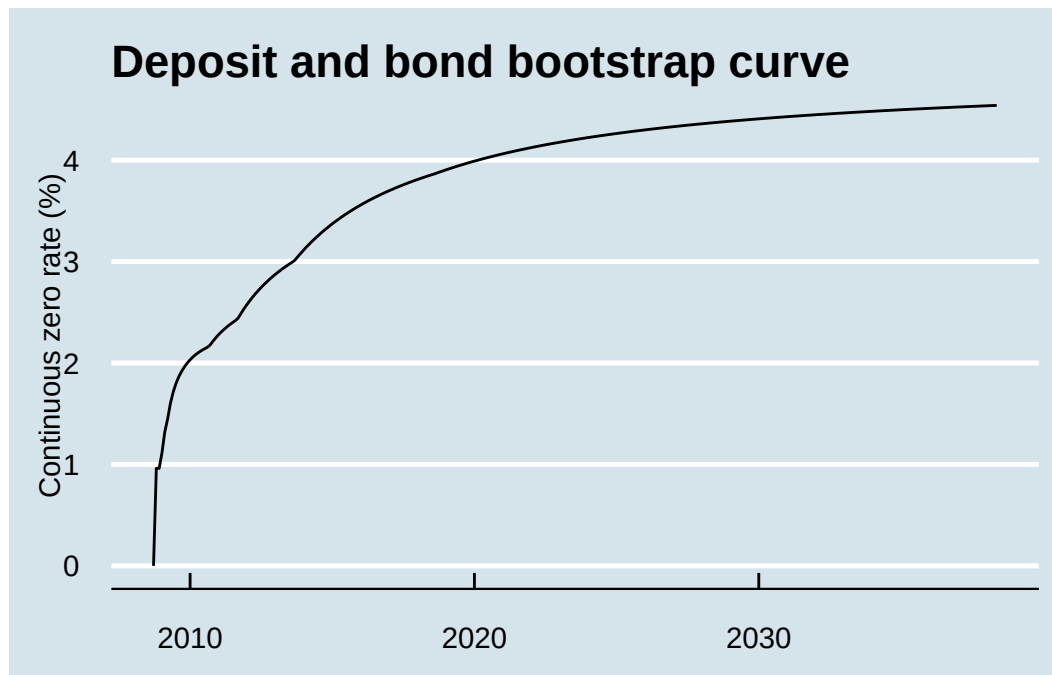
bond_curve_plot <- bond_curve_grid |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$curve_date,
      y = 100 * .data$zero_rate
    )
  )

```

```

) +
  ggplot2::geom_line() +
  ggplot2::labs(
    title = "Deposit and bond bootstrap curve",
    x = NULL,
    y = "Continuous zero rate (%)"
  )
show_plot_LFL(bond_curve_plot)

```



債券価格を用いるブートストラップでは、各債券のクーポン日、満期日、経過利子および日数計算規則がカーブ構築に影響する。債券のクーポン計算には商品固有の日数計算規則を用いる一方、期間構造の時間軸には単純で加法的な日数計算規則を用いる必要がある。

2.3.2 預金とスワップ

次に、短期部分を預金金利、中長期部分を固定対変動金利スワップの市場レートから構築する。スワップの市場レートは、固定レグと変動レグの現在価値を等しくするパー・スワップレートである。

GaussQuant の `build_swap_curve_GQL()` は、短期預金と複数満期のスワップレートを用いて `PiecewiseFlatForward` カーブを構築する。

```

swap_curve <- GaussQuant::build_swap_curve_GQL(
  settlement_date = "2008-09-18",
  fixing_days = 3
)

swap_curve_handle <- GaussQuant::yield_curve_handle_GQL(
  swap_curve

```

```
)

swap_curve_tbl <- GaussQuant::curve_tbl_GQH(
  curve = swap_curve,
  tenors = c(
    "1W", "1M", "3M", "6M",
    "1Y", "2Y", "3Y", "5Y",
    "7Y", "10Y", "15Y"
  )
)
```

```
GaussQuant::show_tbl_GQH(
  swap_curve_tbl,
  "Deposit and swap bootstrap curve",
  n = 30
)

#> # A tibble: 11 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1W    2008-09-25  0.99916  0.044079
#> 2 1M    2008-10-18  0.99733  0.032579
#> 3 3M    2008-12-18  0.99197  0.032440
#> 4 6M    2009-03-18  0.98327  0.034085
#> 5 1Y    2009-09-18  0.96714  0.033440
#> 6 2Y    2010-09-18  0.94376  0.028954
#> 7 3Y    2011-09-18  0.90902  0.031805
#> 8 5Y    2013-09-18  0.83739  0.035499
#> 9 7Y    2015-09-18  0.76276  0.038692
#> 10 10Y   2018-09-18  0.66310  0.041086
#> 11 15Y   2023-09-18  0.52138  0.043421
```

```
comparison_tenors <- c(
  "1M", "3M", "6M", "1Y",
  "2Y", "3Y", "5Y", "7Y",
  "10Y", "15Y"
)

bootstrap_comparison_tbl <- dplyr::bind_rows(
  GaussQuant::curve_tbl_GQH(
    curve = bond_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Deposit + bond"
  )
)
```

```

    ),
    GaussQuant::curve_tbl_GQH(
      curve = swap_curve,
      tenors = comparison_tenors
    ) |>
    dplyr::mutate(
      curve = "Deposit + swap"
    )
  )

GaussQuant::show_tbl_GQH(
  bootstrap_comparison_tbl,
  "Bootstrap curve comparison",
  n = 30
)

#> # A tibble: 20 x 5
#>   tenor date      discount zero_rate curve
#>   <chr> <date>      <dbl>      <dbl> <chr>
#> 1 1M     2008-10-18  0.99921 0.0096148 Deposit + bond
#> 2 3M     2008-12-18  0.99761 0.0096148 Deposit + bond
#> 3 6M     2009-03-18  0.99286 0.014471  Deposit + bond
#> 4 1Y     2009-09-18  0.98097 0.019229  Deposit + bond
#> 5 2Y     2010-09-18  0.95732 0.021818  Deposit + bond
#> 6 3Y     2011-09-18  0.92904 0.024541  Deposit + bond
#> 7 5Y     2013-09-18  0.85971 0.030237  Deposit + bond
#> 8 7Y     2015-09-18  0.78217 0.035102  Deposit + bond
#> 9 10Y    2018-09-18  0.67867 0.038764  Deposit + bond
#> 10 15Y   2023-09-18  0.53179 0.042103  Deposit + bond
#> 11 1M     2008-10-18  0.99733 0.032579  Deposit + swap
#> 12 3M     2008-12-18  0.99197 0.032440  Deposit + swap
#> 13 6M     2009-03-18  0.98327 0.034085  Deposit + swap
#> 14 1Y     2009-09-18  0.96714 0.033440  Deposit + swap
#> 15 2Y     2010-09-18  0.94376 0.028954  Deposit + swap
#> 16 3Y     2011-09-18  0.90902 0.031805  Deposit + swap
#> 17 5Y     2013-09-18  0.83739 0.035499  Deposit + swap
#> 18 7Y     2015-09-18  0.76276 0.038692  Deposit + swap
#> 19 10Y    2018-09-18  0.66310 0.041086  Deposit + swap
#> 20 15Y   2023-09-18  0.52138 0.043421  Deposit + swap

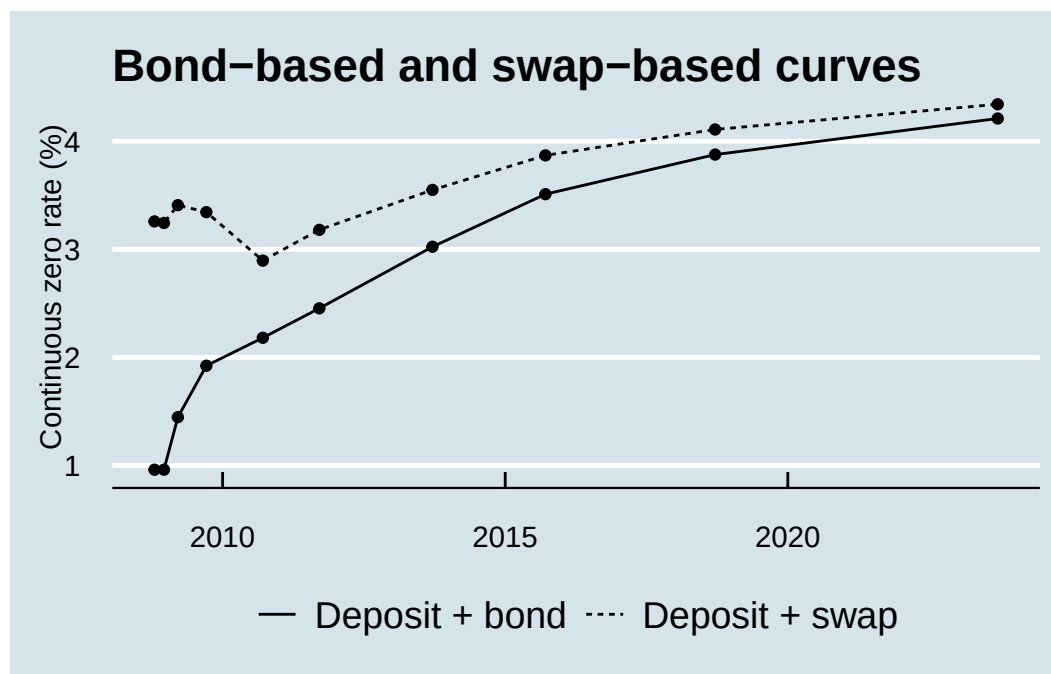
bootstrap_zero_plot <- bootstrap_comparison_tbl |>
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$date,

```

```

    y = 100 * .data$zero_rate,
    linetype = .data$curve
  )
) +
ggplot2::geom_line() +
ggplot2::geom_point() +
ggplot2::labs(
  title = "Bond-based and swap-based curves",
  x = NULL,
  y = "Continuous zero rate (%)",
  linetype = NULL
)
show_plot_LFL(bootstrap_zero_plot)

```



債券カーブとスワップカーブは、異なる市場商品から構築されるため一致するとは限らない。両者の差には、市場の流動性、商品固有の信用・担保条件、日数計算規則および観測時点の違いが含まれる。

FRA は、将来の一定期間に適用される金利を直接観測する商品として、預金とスワップの間を接続するために用いられる。本章の実装では預金とスワップを用いるが、FRA を追加する場合も、各市場商品の理論価格が市場価格と一致するように同じブートストラップの枠組みへ組み込む。

2.4 カーブ操作と検証

2.4.1 Implied Term Structure

Implied Term Structure は、既存カーブが示す将来の割引関係を保ったまま、参照日を将来へ移した期間構造である。元のカーブの参照日を t_0 、新しい参照日を t_1 、将来日を t_2 とすると、Implied Term Structure の割引係数は概念的に次式で表される。

$$D_{t_1}(t_2) = \frac{D_{t_0}(t_2)}{D_{t_0}(t_1)}$$

```

implied_reference_date <- GaussQuant::date_GQL(
  "2009-09-18"
)

implied_constructor <- get0(
  "ImpliedTermStructure",
  envir = asNamespace("QuantLib"),
  inherits = FALSE
)

implied_curve <- if (is.null(implied_constructor)) {
  NULL
} else {
  tryCatch(
    implied_constructor(
      swap_curve_handle,
      implied_reference_date
    ),
    error = function(e) NULL
  )
}

implied_curve_tbl <- if (is.null(implied_curve)) {
  tibble::tibble(
    status = "ImpliedTermStructure is not available in the installed QuantLib binding."
  )
} else {
  GaussQuant::curve_tbl_GQH(
    curve = implied_curve,
    tenors = c(
      "1M", "3M", "6M", "1Y",
      "2Y", "3Y", "5Y", "10Y"
    )
  )
}

GaussQuant::show_tbl_GQH(
  implied_curve_tbl,
  "Implied term structure",
  n = 20
)

```



```
#> # A tibble: 8 x 4
#>   tenor date      discount zero_rate
#>   <chr> <date>      <dbl>    <dbl>
#> 1 1M    2009-10-18  0.99799  0.024471
#> 2 3M    2009-12-18  0.99392  0.024471
#> 3 6M    2010-03-18  0.98794  0.024471
#> 4 1Y    2010-09-18  0.97583  0.024471
#> 5 2Y    2011-09-18  0.93990  0.030989
#> 6 3Y    2012-09-18  0.90209  0.034338
#> 7 5Y    2014-09-18  0.82636  0.038145
#> 8 10Y   2019-09-18  0.65344  0.042551
```

Implied Term Structure は、将来時点から見た割引係数を現在のカーブから機械的に導くものであり、将来の市場カーブを予測したものではない。評価日を将来へ進めたシナリオで市場リスクを評価する場合には、カーブの単純な時間経過と市場金利の変動を区別する必要がある。

2.4.2 カーブの平行シフト

金利リスク分析では、基準カーブに一定のスプレッドを加えたカーブを用いる。すべての満期へ同じスプレッドを加える変化を平行シフトという。

ここでは、預金・スワップカーブへ 50 ベーシスポイントを加え、ゼロ金利と割引係数の変化を確認する。

```
parallel_spread <- 50 / 10000

spreaded_curve_bundle <- GaussQuant::make_zero_spreaded_curve_GQL(
  base_curve_handle = swap_curve_handle,
  spread_rate = parallel_spread,
  day_counter = QuantLib::Actual365Fixed(),
  extrapolate = TRUE
)

spreaded_curve <- spreaded_curve_bundle$curve

parallel_shift_tbl <- dplyr::bind_rows(
  GaussQuant::curve_tbl_GQH(
    curve = swap_curve,
    tenors = comparison_tenors
  ) |>
  dplyr::mutate(
    curve = "Base curve"
  ),
  GaussQuant::curve_tbl_GQH(
    curve = spreaded_curve,
    tenors = comparison_tenors
  )
)
```

```

) |>
  dplyr::mutate(
    curve = "Base curve + 50bp"
  )
)

parallel_shift_wide_tbl <- parallel_shift_tbl |>
  dplyr::select(
    .data$tenor,
    .data$date,
    .data$curve,
    .data$discount,
    .data$zero_rate
  ) |>
  tidyr::pivot_wider(
    names_from = curve,
    values_from = c(
      discount,
      zero_rate
    )
  ) |>
  dplyr::mutate(
    observed_zero_shift_bp =
      10000 * (
        .data$`zero_rate_Base curve + 50bp` -
        .data$`zero_rate_Base curve`
      )
  )

GaussQuant::show_tbl_GQH(
  parallel_shift_wide_tbl,
  "Parallel curve shift",
  n = 30
)

#> # A tibble: 10 x 7
#>   tenor date      `discount_Base curve` `discount_Base curve + 50bp`
#>   <chr> <date>          <dbl>          <dbl>
#> 1 1M    2008-10-18      0.99733      0.99693
#> 2 3M    2008-12-18      0.99197      0.99074
#> 3 6M    2009-03-18      0.98327      0.98088
#> 4 1Y    2009-09-18      0.96714      0.96249
#> 5 2Y    2010-09-18      0.94376      0.93494
#> 6 3Y    2011-09-18      0.90902      0.89679

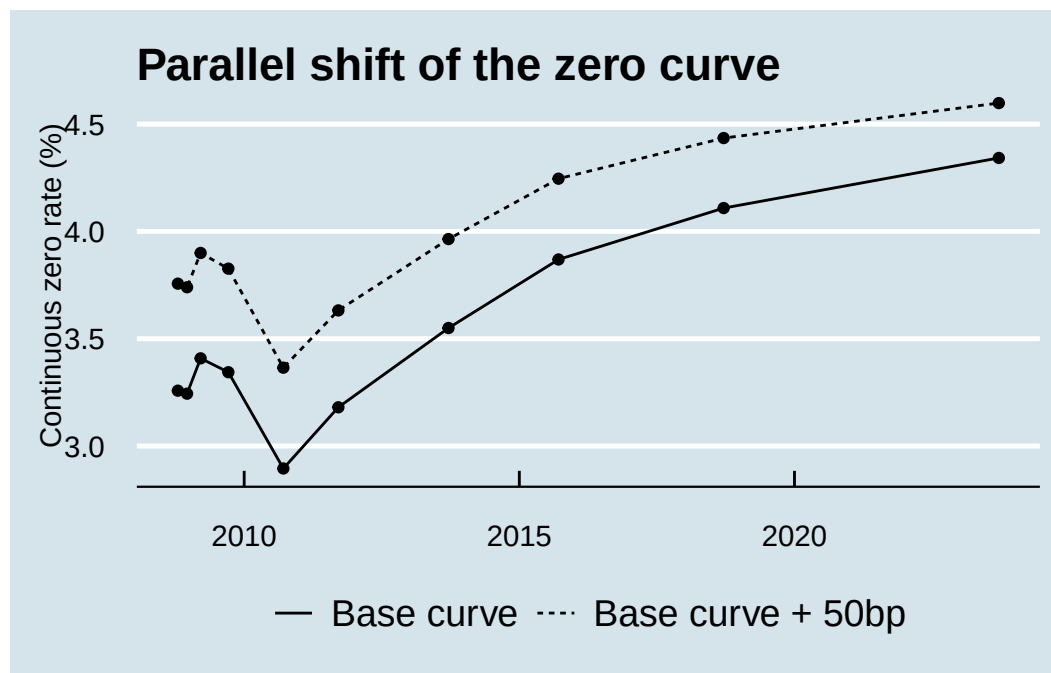
```

```
#> 7 5Y    2013-09-18      0.83739      0.82022
#> 8 7Y    2015-09-18      0.76276      0.74293
#> 9 10Y   2018-09-18      0.66310      0.64182
#> 10 15Y  2023-09-18      0.52138      0.50176
#> # i 3 more variables: `zero_rate_Base curve` <dbl>,
#> #   `zero_rate_Base curve + 50bp` <dbl>, observed_zero_shift_bp <dbl>
```

```
parallel_shift_plot <- parallel_shift_tbl |>
```

```
  ggplot2::ggplot(
    ggplot2::aes(
      x = .data$date,
      y = 100 * .data$zero_rate,
      linetype = .data$curve
    )
  ) +
  ggplot2::geom_line() +
  ggplot2::geom_point() +
  ggplot2::labs(
    title = "Parallel shift of the zero curve",
    x = NULL,
    y = "Continuous zero rate (%)",
    linetype = NULL
  )
```

```
show_plot_LFL(parallel_shift_plot)
```



正の平行シフトでは割引率が上昇し、将来キャッシュフローの割引係数は低下する。そのため、通常の固定キャッシュフロー商品の現在価値も低下する。

実際の市場変動は完全な平行移動とは限らない。短期だけが動く変化、長期だけが動く変化、曲線の傾きが変わるスティーピングまたはフラットニングも生じる。年限別の Key Rate Shift では、特定年限のノードを動かし、周辺年限へ補間しながら再評価する。これらは平行シフトより細かなリスク分解である。

2.4.3 カーブの検証

構築したカーブについて、少なくとも次の点を確認する必要がある。

- 参照日と最終日が想定どおりであること
- 割引係数が正であること
- 欠損値や計算不能な区間がないこと
- ゼロ金利とフォワード金利に不自然な跳びがないこと
- 入力商品の理論価格または理論レートが市場値へ再現されること

まず、例示的なゼロ金利ノードが二つのカーブでどの程度再現されるかを確認する。

```
node_recovery_tbl <- zero_nodes |>
  dplyr::mutate(
    ql_date = purrr::map(
      as.character(.data$date),
      GaussQuant::date_GQL
    ),
    time = as.numeric(
      .data$date -
      as.Date(evaluation_date_chr)
    ) / 365,
    zero_curve_discount = purrr::map_dbl(
      .data$ql_date,
      ~ GaussQuant::curve_discount_safe_GQL(
        zero_curve,
        .x
      )
    ),
    discount_curve_discount = purrr::map_dbl(
      .data$ql_date,
      ~ GaussQuant::curve_discount_safe_GQL(
        discount_curve,
        .x
      )
    ),
    zero_curve_implied_rate = dplyr::if_else(
      .data$time > 0,
      -log(.data$zero_curve_discount) /
        .data$time,
      .data$zero_rate
    )
  )
```

```

    ),
    discount_curve_implied_rate = dplyr::if_else(
      .data$time > 0,
      -log(.data$discount_curve_discount) /
        .data$time,
      .data$zero_rate
    ),
    zero_curve_error =
      .data$zero_curve_implied_rate -
      .data$zero_rate,
    discount_curve_error =
      .data$discount_curve_implied_rate -
      .data$zero_rate
  ) |>
  dplyr::select(
    .data$tenor,
    .data$date,
    .data$zero_rate,
    .data$zero_curve_implied_rate,
    .data$discount_curve_implied_rate,
    .data$zero_curve_error,
    .data$discount_curve_error
  )

GaussQuant::show_tbl_GQH(
  node_recovery_tbl,
  "Node recovery check",
  n = 20
)

#> # A tibble: 9 x 7
#>   tenor date      zero_rate zero_curve_implied_rate discount_curve_implied_rate
#>   <chr> <date>      <dbl>          <dbl>          <dbl>
#> 1 0D    2022-08-19    0.0005          0.0005          0.0005
#> 2 1M    2022-09-19    0.0006          0.00060000     0.00060000
#> 3 3M    2022-11-21    0.0008          0.00080000     0.00080000
#> 4 6M    2023-02-20    0.0011          0.0011000     0.0011000
#> 5 1Y    2023-08-21    0.0018          0.0018000     0.0018000
#> 6 2Y    2024-08-19    0.0032          0.0032000     0.0032000
#> 7 3Y    2025-08-19    0.0048          0.0048000     0.0048000
#> 8 5Y    2027-08-19    0.0075          0.0075         0.0075
#> 9 10Y   2032-08-19    0.0115          0.0115         0.0115
#> # i 2 more variables: zero_curve_error <dbl>, discount_curve_error <dbl>

```

続いて、各カーブを密なグリッドへ展開し、割引係数の範囲と欠損値を確認する。

```

curve_validation_tbl <- dplyr::bind_rows(
  GaussQuant::curve_grid_tbl_GQL(
    zero_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>
  dplyr::mutate(
    curve = "ZeroCurve"
  ),
  GaussQuant::curve_grid_tbl_GQL(
    discount_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>
  dplyr::mutate(
    curve = "DiscountCurve"
  ),
  GaussQuant::curve_grid_tbl_GQL(
    bond_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>
  dplyr::mutate(
    curve = "Deposit + bond"
  ),
  GaussQuant::curve_grid_tbl_GQL(
    swap_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>
  dplyr::mutate(
    curve = "Deposit + swap"
  ),
  GaussQuant::curve_grid_tbl_GQL(
    spreaded_curve,
    n = 200L,
    extrapolate = TRUE
  ) |>
  dplyr::mutate(
    curve = "Deposit + swap + 50bp"
  )
) |>
dplyr::group_by(.data$curve) |>

```

```

dplyr::summarise(
  first_date = min(
    .data$curve_date,
    na.rm = TRUE
  ),
  last_date = max(
    .data$curve_date,
    na.rm = TRUE
  ),
  min_discount = min(
    .data$discount_factor,
    na.rm = TRUE
  ),
  max_discount = max(
    .data$discount_factor,
    na.rm = TRUE
  ),
  nonpositive_discount_count = sum(
    .data$discount_factor <= 0,
    na.rm = TRUE
  ),
  missing_discount_count = sum(
    is.na(.data$discount_factor)
  ),
  missing_zero_rate_count = sum(
    is.na(.data$zero_rate)
  ),
  .groups = "drop"
)

GaussQuant::show_tbl_GQH(
  curve_validation_tbl,
  "Curve validation summary",
  n = 20
)

#> # A tibble: 5 x 8
#>   curve   first_date last_date  min_discount max_discount nonpositive_discount~1
#>   <chr>   <date>     <date>      <dbl>      <dbl>          <int>
#> 1 Deposi~ 2008-09-18 2038-05-15    0.26019      1            0
#> 2 Deposi~ 2008-09-18 2023-09-18    0.52138      1            0
#> 3 Deposi~ 2008-09-18 2023-09-18    0.50176      1            0
#> 4 Discou~ 2022-08-19 2032-08-19    0.89128      1            0
#> 5 ZeroCu~ 2022-08-19 2032-08-19    0.89128      1            0

```

```
#> # i abbreviated name: 1: nonpositive_discount_count
#> # i 2 more variables: missing_discount_count <int>,
#> #   missing_zero_rate_count <int>
```

割引係数が正であることは、裁定のない期間構造を確認するための基本条件である。ただし、正の割引係数だけでは十分ではない。入力商品の再評価、補間区間のフォワード金利、外挿方法、日数計算規則および参照日の設定も確認する必要がある。

2.5 国債先物の基礎

株価指数先物では、指数そのものを受け渡すことができないため、満期時には差金決済が行われる。これに対し、国債先物では、満期まで保有された建玉について、受渡適格銘柄に指定された国債を用いた現物受渡しが行われる。現物受渡しを通じて、先物市場と現物国債市場の価格連関が保たれている。

国債先物の取引対象は、実際に発行された特定の国債ではなく、クーポンや償還期限を標準化した標準物である。この標準物は実在しないため、最終決済では、取引所が定めた複数の受渡適格銘柄のいずれかを用いて受渡しを行う。銘柄ごとにクーポン、残存期間および価格が異なるため、その価値の差は変換係数（Conversion Factor: CF）によって調整される。

受渡銘柄を選択する権利は、国債を受け取る買方ではなく、国債を引き渡す売方にある。したがって売方は、受渡適格銘柄の中から、現物価格だけでなく、変換係数、経過利子、クーポン収入および受渡日までの資金調達費用を考慮し、最も低い経済的費用で受け渡すことのできる銘柄を選択する。この銘柄を最割安銘柄（Cheapest-to-Deliver: CTD）という。

CTD は、単純に現物単価が最も低い債券ではない。先物価格に変換係数を乗じて求める受渡価値と、現物債を調達して受渡日まで保有する費用とを比較して決定される。

本章では、受渡適格銘柄ごとに現物価格、グロスベースス、キャリー、ネットベーススおよびインプライド・レボを計算する。これらの比較を通じて、どのような経済計算によって CTD が選定されるのかを確認し、最後に CTD を基礎とする国債先物の BPV を求める。

```
knitr::opts_chunk$set(
  collapse = TRUE,
  comment = "#>",
  warning = FALSE,
  message = FALSE
)

suppressPackageStartupMessages({
  library(tidyverse)
  library(QuantLib)
  library(GaussQuant)
})
```


2.5.1 計算条件と市場慣行

本章では、米国国債先物を例として計算する。現物国債には米国政府債カレンダーと Actual/Actual (Bond) を使用し、レポ期間には Actual/360 を使用する。現物債の受渡日は取引日の 1 営業日後とする。

米国国債先物の価格は、通常、額面 100 に対するポイントと 32 分の 1 で表示される。たとえば、コード中の $109 + 10 / 32$ は、相場表示では 109-10 に相当する。

```
cal_us_gov <- QuantLib::UnitedStates("GovernmentBond")

dc_act_act_bond <- GaussQuant::day_counter_GQL("ActualActual_Bond")
dc_act_360 <- GaussQuant::day_counter_GQL("Actual360")

compd_compounded <- QuantLib::Compounding_Compounded_get()
freq_semiannual <- QuantLib::Frequency_Semiannual_get()

settlement_days_t1 <- 1L
trade_date_chr <- "2023-04-20"
repo_end_date_chr <- "2023-07-06"

trade_date <- GaussQuant::date_GQL(trade_date_chr)
repo_end_date_ql <- GaussQuant::date_GQL(repo_end_date_chr)

GaussQuant::set_eval_date_GQL(trade_date)

settle_date <- GaussQuant::advance_days_GQL(
  calendar_obj = cal_us_gov,
  date_obj = trade_date,
  n_days = settlement_days_t1
)

settle_date_chr <- as.character(settle_date)
settle_date_ql <- GaussQuant::date_GQL(settle_date_chr)

setup_tbl <- tibble::tribble(
  ~item, ~value,
  "trade_date", trade_date_chr,
  "spot_settlement_date", settle_date_chr,
  "repo_end_date", repo_end_date_chr,
  "settlement_days", as.character(settlement_days_t1),
  "calendar", "UnitedStates(GovernmentBond)",
  "bond_day_counter", "ActualActual(Bond)",
  "repo_day_counter", "Actual360",
  "compounding", "Compounded",
```

```

    "frequency", "Semiannual"
  )

GaussQuant::show_tbl_GQH(setup_tbl, "Treasury futures setup")
#> # A tibble: 9 x 2
#>   item                value
#>   <chr>              <chr>
#> 1 trade_date        2023-04-20
#> 2 spot_settlement_date 2023-04-21
#> 3 repo_end_date      2023-07-06
#> 4 settlement_days    1
#> 5 calendar           UnitedStates(GovernmentBond)
#> 6 bond_day_counter    ActualActual(Bond)
#> 7 repo_day_counter    Actual360
#> 8 compounding         Compounded
#> 9 frequency          Semiannual

```

2.5.2 受渡適格国債

最初に、受渡適格銘柄の例となる米国国債を作成する。発行日、償還日、クーポンおよび利払スケジュールは、現物価格、経過利子、キャリアおよび BPV を計算する基礎となる。

次のコードでは、年率 4.125%、年 2 回払いの米国国債を作成し、利払スケジュールを確認する。あわせて、利回り 6% を仮定した場合のクリーン価格を計算する。

```

bond_bundle <- GaussQuant::us_treasury_bond_GQL(
  effective_date = "2022-09-30",
  maturity_date = "2027-09-30",
  coupon_rate_pct = 4.125,
  face_amount = 100,
  settlement_days = settlement_days_t1,
  calendar = cal_us_gov,
  day_counter = dc_act_act_bond
)

bond_obj <- bond_bundle$bond

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_dates_GQL(bond_bundle$schedule),
  "US Treasury bond schedule",
  n = 20
)
#> # A tibble: 11 x 1
#>   schedule_date

```

```
#>    <date>
#>  1 2022-09-30
#>  2 2023-03-30
#>  3 2023-09-30
#>  4 2024-03-30
#>  5 2024-09-30
#>  6 2025-03-30
#>  7 2025-09-30
#>  8 2026-03-30
#>  9 2026-09-30
#> 10 2027-03-30
#> 11 2027-09-30

clean_price_at_six_percent <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = bond_obj,
  yield_rate = 6 / 100,
  day_counter = dc_act_act_bond,
  compounding = compd_compounded,
  frequency = freq_semiannual,
  settlement_date = GaussQuant::date_GQL("2023-07-06")
)

cat(
  "Yield 6%, settle 2023-07-06 clean price:",
  GaussQuant::fmt_num_GQH(clean_price_at_six_percent, 4),
  "\n"
)

#> Yield 6%, settle 2023-07-06 clean price: 93.07
```

2.5.3 現物債評価

CTD 分析では、現物債を取得するために必要な資金を把握する必要がある。市場利回りからクリーン価格を求め、受渡日までの経過利子を加えると、実際の決済金額に対応するダーティ価格が得られる。

$$\text{ダーティ価格} = \text{クリーン価格} + \text{経過利子}$$

先物価格には 109-10、現物債の市場利回りには 3.70% を用いる。

```
bond_yield <- 3.70 / 100
futures_price <- 109 + 10 / 32

accrued_amount <- bond_obj$accruedAmount(settle_date_ql)
clean_price <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = bond_obj,
```

```

yield_rate = bond_yield,
day_counter = dc_act_act_bond,
compounding = cmpd_compounded,
frequency = freq_semiannual,
settlement_date = settle_date_ql
)

dirty_price <- clean_price + accrued_amount

price_tbl <- tibble::tibble(
  trade_date = as.Date(trade_date_chr),
  settlement_date = as.Date(settle_date_chr),
  yield = bond_yield,
  accrued_amount = accrued_amount,
  clean_price = clean_price,
  dirty_price = dirty_price,
  futures_price = futures_price
)

GaussQuant::show_tbl_GQH(price_tbl, "Bond price from yield")
#> # A tibble: 1 x 7
#>   trade_date settlement_date yield accrued_amount clean_price dirty_price
#>   <date>      <date>      <dbl>         <dbl>         <dbl>      <dbl>
#> 1 2023-04-20 2023-04-21    0.037         0.24660         101.72    101.97
#> # i 1 more variable: futures_price <dbl>

cat(
  "Yield:",
  sprintf("%.8f", bond_yield),
  "\nClean price:",
  sprintf("%.8f", clean_price),
  "\nDirty price:",
  sprintf("%.8f", dirty_price),
  "\nAccrued amount:",
  sprintf("%.8f", accrued_amount),
  "\n"
)

#> Yield: 0.03700000
#> Clean price: 101.72260614
#> Dirty price: 101.96920940
#> Accrued amount: 0.24660326

```

2.5.4 キャッシュフロー検証

次の計算では、各キャッシュフローへ一般的な割引係数を適用し、現在価値の合計と債券価格を比較する。

ただし、Actual/Actual (Bond) は利払参照期間を用いるため、ここで作成する一般的な割引係数は、QuantLib による債券利回り計算を完全には再現しない。この表は、債券価格が将来キャッシュフローの現在価値から構成されることを確認するためのものであり、差額がゼロになることを目的とした検算ではない。

```
interest_rate_obj <- QuantLib::InterestRate(
  bond_yield,
  dc_act_act_bond,
  compd_compounded,
  freq_semiannual
)

settle_date_r <- as.Date(settle_date_chr)

bond_cf_tbl <- GaussQuant::bond_cashflow_table_GQL(bond_obj) |>
  dplyr::mutate(
    pay_date = as.Date(.data$date),
    discount_factor = purrr::map_dbl(
      as.character(.data$pay_date),
      function(x) {
        if (as.Date(x) < settle_date_r) {
          return(NA_real_)
        }

        tryCatch(
          interest_rate_obj$discountFactor(
            settle_date_ql,
            GaussQuant::date_GQL(x)
          ),
          error = function(e) NA_real_
        )
      }
    ),
    pv = .data$amount * .data$discount_factor
  ) |>
  dplyr::select(
    .data$pay_date,
    .data$amount,
    .data$discount_factor,
    .data$pvalue
  )
```

```

cashflow_pv <- sum(bond_cf_tbl$p_v, na.rm = TRUE)

cashflow_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "cashflow_present_value", cashflow_pv,
  "dirty_price_from_yield", dirty_price,
  "illustrative_difference", cashflow_pv - dirty_price
)

GaussQuant::show_tbl_GQH(bond_cf_tbl, "Bond cashflow present-value table", n = 20)
#> # A tibble: 11 x 4
#>   pay_date      amount discount_factor      pv
#>   <date>        <dbl>         <dbl>    <dbl>
#> 1 2023-03-30    2.0625          NA        NA
#> 2 2023-10-02    2.0625      0.98484    2.0312
#> 3 2024-04-01    2.0625      0.96695    1.9943
#> 4 2024-09-30    2.0625      0.94939    1.9581
#> 5 2025-03-31    2.0625      0.93214    1.9225
#> 6 2025-09-30    2.0625      0.91521    1.8876
#> 7 2026-03-30    2.0625      0.89859    1.8533
#> 8 2026-09-30    2.0625      0.88227    1.8197
#> 9 2027-03-30    2.0625      0.86624    1.7866
#> 10 2027-09-30   2.0625      0.85051    1.7542
#> 11 2027-09-30  100          0.85051   85.051
GaussQuant::show_tbl_GQH(
  GaussQuant::metric_tbl_display_GQH(cashflow_check_tbl),
  "Cashflow PV illustration"
)
#> # A tibble: 3 x 2
#>   metric          value
#>   <chr>          <chr>
#> 1 cashflow_present_value 102.05828711
#> 2 dirty_price_from_yield 101.96920940
#> 3 illustrative_difference 0.08907771

```

2.5.5 経過利子

経過利子は、直前利払日から受渡日まで発生したクーポンを日割りして求める。Actual/Actual (Bond) では、経過日数を利払期間の実日数で除してクーポンを按分する。

$$\text{経過利子} = 1 \text{ 回分のクーポン} \times \frac{\text{経過日数}}{\text{利払期間の日数}}$$

手計算結果と QuantLib の結果との差を確認する。

```
previous_coupon_date <- GaussQuant::date_GQL("2023-03-30")
next_coupon_date <- GaussQuant::date_GQL("2023-09-30")

accrued_days_hand <- dc_act_act_bond$dayCount(
  previous_coupon_date,
  settle_date_ql
)

coupon_period_days <- dc_act_act_bond$dayCount(
  previous_coupon_date,
  next_coupon_date
)

coupon_amount_semiannual <- 4.125 / 2

accrued_interest_hand <- coupon_amount_semiannual *
  accrued_days_hand /
  coupon_period_days

accrued_tbl <- tibble::tibble(
  previous_coupon_date = as.Date(GaussQuant::iso_GQL(previous_coupon_date)),
  settlement_date = as.Date(settle_date_chr),
  next_coupon_date = as.Date(GaussQuant::iso_GQL(next_coupon_date)),
  accrued_days = accrued_days_hand,
  coupon_period_days = coupon_period_days,
  coupon_amount = coupon_amount_semiannual,
  accrued_interest_hand = accrued_interest_hand,
  accrued_amount_quantlib = accrued_amount,
  difference = accrued_interest_hand - accrued_amount
)

GaussQuant::show_tbl_GQH(accrued_tbl, "Accrued interest hand calculation")
#> # A tibble: 1 x 9
#>   previous_coupon_date settlement_date next_coupon_date accrued_days
#>   <date>                <date>          <date>          <int>
#> 1 2023-03-30          2023-04-21      2023-09-30           22
#> # i 5 more variables: coupon_period_days <int>, coupon_amount <dbl>,
#> #   accrued_interest_hand <dbl>, accrued_amount_quantlib <dbl>,
#> #   difference <dbl>
```

2.6 CTD 分析と先物 BPV

2.6.1 受渡適格銘柄と変換係数

国債先物では、複数の受渡適格銘柄が存在する。各銘柄はクーポンや残存期間が異なるため、そのままでは同じ先物価格に対して比較できない。そこで、各銘柄へ変換係数を適用し、標準物に対する相対的な受渡価値を調整する。

受渡時に買方が売方へ支払う金額は、概念的には次式で表される。

$$\text{インボイス価格} = \text{先物価格} \times \text{変換係数} + \text{受渡時経過利子}$$

ここでいう CF は Conversion Factor の略であり、キャッシュフローを意味する CF とは区別する。

次の表では、3 銘柄の発行日、償還日、クーポン、変換係数および市場利回りを設定する。

```
deliverable_tbl <- tibble::tribble(
  ~issue_date, ~maturity_date, ~coupon_rate_pct, ~conversion_factor, ~market_yield_pct,
  "2022-09-30", "2027-09-30", 4.125, 0.9305, 3.70,
  "2022-08-31", "2027-08-31", 3.125, 0.8953, 3.69,
  "2023-01-31", "2028-01-31", 3.500, 0.9011, 3.65
)

GaussQuant::show_tbl_GQH(deliverable_tbl, "Treasury futures deliverable basket")
#> # A tibble: 3 x 5
#>   issue_date maturity_date coupon_rate_pct conversion_factor market_yield_pct
#>   <chr>         <chr>          <dbl>          <dbl>          <dbl>
#> 1 2022-09-30 2027-09-30          4.125          0.9305          3.7
#> 2 2022-08-31 2027-08-31          3.125          0.8953          3.69
#> 3 2023-01-31 2028-01-31          3.5           0.9011          3.65
```

2.6.2 グロスベースス

グロスベーススは、現物債のクリーン価格と、変換係数で調整した先物価格との差である。

$$\text{グロスベースス} = \text{現物クリーン価格} - \text{先物価格} \times \text{変換係数}$$

グロスベーススが小さい銘柄ほど受渡しに有利に見えるが、この段階では受渡日までの資金調達費用、クーポン収入および経過利子の変化を考慮していない。

2.6.3 キャリーとネットベースス

現物債を購入して受渡日まで保有する場合、レポによる資金調達費用が発生する一方、保有期間中にはクーポン収入や経過利子の増加が得られる。これらをまとめたものがキャリーである。

概念的には、ネットベーススは次式で表される。

$$\text{ネットベースス} = \text{グロスベースス} - \text{キャリー}$$

同一の前提条件で比較した場合、一般にネットベーススが最も小さい銘柄が CTD となる。

2.6.4 インプライド・レポ

インプライド・レポは、現物債を購入し、レポで資金調達して受渡日まで保有し、先物契約へ受け渡す取引から得られる収益率である。

ネットベーススが受渡コストの大きさを表すのに対し、インプライド・レポは同じ取引を収益率として表す。通常、ネットベーススが最小となる銘柄と、インプライド・レポが最大となる銘柄は一致する。

次のコードでは、各受渡適格銘柄について現物価格、BPV、グロスベースス、キャリー、ネットベーススおよびインプライド・レポを計算し、CTD を選定する。

```
repo_rate <- 5.10 / 100

repo_days <- dc_act_360$dayCount(
  settle_date_ql,
  repo_end_date_ql
)

repo_year_fraction <- dc_act_360$yearFraction(
  settle_date_ql,
  repo_end_date_ql
)

ctd_result <- GaussQuant::bond_futures_ctd_table_GQL(
  deliverable_tbl = deliverable_tbl,
  settlement_date = settle_date_chr,
  futures_price = futures_price,
  repo_end_date = repo_end_date_chr,
  repo_rate = repo_rate,
  settlement_days = settlement_days_t1,
  calendar = cal_us_gov,
  day_counter = dc_act_act_bond,
  compounding = cmpd_compounded,
  frequency = freq_semiannual,
  repo_day_counter = dc_act_360,
  carry_day_counter = dc_act_360
)

basket_tbl <- ctd_result$basket
```

```

gross_basis_tbl <- basket_tbl |>
  dplyr::select(
    .data$issue_date,
    .data$maturity,
    .data$coupon,
    .data$yield_pct,
    .data$bpv,
    .data$clean_price,
    .data$dirty_price,
    .data$conversion_factor,
    .data$gross_basis
  )

net_basis_tbl <- basket_tbl |>
  dplyr::select(
    .data$maturity,
    .data$conversion_factor,
    .data$gross_basis,
    .data$accrued_start,
    .data$accrued_end,
    .data$coupon_income,
    .data$repo_cost,
    .data$carry,
    .data$net_basis,
    .data$forward_price,
    .data$implied_repo,
    .data$ctd
  )

basket_display_tbl <- basket_tbl |>
  dplyr::select(-dplyr::any_of("bond_obj"))

ctd_tbl <- ctd_result$ctd
ctd_display_tbl <- ctd_tbl |>
  dplyr::select(-dplyr::any_of("bond_obj"))

stopifnot(
  nrow(ctd_tbl) == 1L,
  isTRUE(ctd_tbl$ctd[[1L]])
)

cat(
  "Futures price:",

```

```

    sprintf("%.6f", futures_price),
    "\nRepo rate:",
    sprintf("%.6f", repo_rate),
    "\nRepo days:",
    repo_days,
    "\nRepo year fraction:",
    sprintf("%.8f", repo_year_fraction),
    "\n"
)
#> Futures price: 109.312500
#> Repo rate: 0.051000
#> Repo days: 76
#> Repo year fraction: 0.21111111

GaussQuant::show_tbl_GQH(gross_basis_tbl, "Gross basis table", n = 20)
#> # A tibble: 3 x 9
#>   issue_date maturity    coupon yield_pct    bpv clean_price dirty_price
#>   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2022-09-30 2027-09-30 0.04125    3.7  0.041018    101.72    101.97
#> 2 2022-08-31 2027-08-31 0.03125    3.69 0.039410     97.740    98.181
#> 3 2023-01-31 2028-01-31 0.035      3.65 0.043340     99.343    100.12
#> # i 2 more variables: conversion_factor <dbl>, gross_basis <dbl>

GaussQuant::show_tbl_GQH(net_basis_tbl, "Net basis and implied repo table", n = 20)
#> # A tibble: 3 x 12
#>   maturity conversion_factor gross_basis accrued_start accrued_end coupon_income
#>   <chr>          <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1 2027-09~      0.9305    0.0073249    0.24660    1.0985    0.85190
#> 2 2027-08~      0.8953   -0.12767    0.44158    1.0870    0.64538
#> 3 2028-01~      0.9011    0.84146    0.77348    1.5083    0.73481
#> # i 6 more variables: repo_cost <dbl>, carry <dbl>, net_basis <dbl>,
#> #   forward_price <dbl>, implied_repo <dbl>, ctd <lgl>

GaussQuant::show_tbl_GQH(basket_display_tbl, "Deliverable basket summary", n = 20)
#> # A tibble: 3 x 18
#>   issue_date maturity    coupon yield_pct    bpv clean_price dirty_price
#>   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2022-09-30 2027-09-30 0.04125    3.7  0.041018    101.72    101.97
#> 2 2022-08-31 2027-08-31 0.03125    3.69 0.039410     97.740    98.181
#> 3 2023-01-31 2028-01-31 0.035      3.65 0.043340     99.343    100.12
#> # i 11 more variables: conversion_factor <dbl>, gross_basis <dbl>,
#> #   accrued_start <dbl>, accrued_end <dbl>, coupon_income <dbl>,
#> #   repo_cost <dbl>, carry <dbl>, net_basis <dbl>, forward_price <dbl>,
#> #   implied_repo <dbl>, ctd <lgl>

GaussQuant::show_tbl_GQH(ctd_display_tbl, "Cheapest-to-deliver bond", n = 20)

```

```
#> # A tibble: 1 x 18
#>   issue_date maturity    coupon yield_pct    bpv clean_price dirty_price
#>   <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1 2022-09-30 2027-09-30 0.04125    3.7 0.041018    101.72    101.97
#> # i 11 more variables: conversion_factor <dbl>, gross_basis <dbl>,
#> #   accrued_start <dbl>, accrued_end <dbl>, coupon_income <dbl>,
#> #   repo_cost <dbl>, carry <dbl>, net_basis <dbl>, forward_price <dbl>,
#> #   implied_repo <dbl>, ctd <lgl>
```

結果では、現物価格だけでなく、変換係数とキャリーを反映したネットベースを比較する。ctd が TRUE となった銘柄が、本章の条件における最割安銘柄である。インプライド・レポもあわせて確認し、CTD 選定と整合しているかを見る。

2.6.5 CTD 調整後の先物 BPV

国債先物の金利感応度は、CTD 債券の BPV を変換係数で調整して概算する。

$$\text{先物 BPV} \approx \frac{\text{CTD 債券の BPV}}{\text{変換係数}}$$

この計算は、金利が小幅に変化しても現在の CTD が変わらないことを前提とする。金利水準やイールドカーブの形状が大きく変化すると、別の受渡適格銘柄が CTD となる可能性がある。そのため、国債先物のリスクは、現在の CTD 債券の感応度だけでなく、CTD が切り替わる可能性にも依存する。

```
ctd_bond <- ctd_tbl$bond_obj[[1L]]
ctd_conversion_factor <- ctd_tbl$conversion_factor[[1L]]
ctd_yield <- ctd_tbl$yield_pct[[1L]] / 100

ctd_futures_bpv <- GaussQuant::bond_futures_bpv_GQL(
  bond = ctd_bond,
  conversion_factor = ctd_conversion_factor,
  contracts = 1,
  measure = "dv01",
  ytm = ctd_yield,
  day_counter = dc_act_act_bond,
  compounding = compd_compounded,
  frequency = freq_semiannual
)

ctd_summary_tbl <- tibble::tibble(
  metric = c(
    "ctd_maturity",
    "ctd_coupon",
    "ctd_yield_pct",
    "conversion_factor",
```

```

    "gross_basis",
    "net_basis",
    "implied_repo",
    "ctd_bond_bpv",
    "ctd_futures_bpv"
  ),
  value = list(
    as.character(ctd_tbl$maturity[[1L]]),
    ctd_tbl$coupon[[1L]],
    ctd_tbl$yield_pct[[1L]],
    ctd_conversion_factor,
    ctd_tbl$gross_basis[[1L]],
    ctd_tbl$net_basis[[1L]],
    ctd_tbl$implied_repo[[1L]],
    ctd_tbl$bpv[[1L]],
    ctd_futures_bpv
  )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::metric_tbl_display_GQH(ctd_summary_tbl),
  "CTD and futures BPV summary"
)

#> # A tibble: 9 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 ctd_maturity 2027-09-30
#> 2 ctd_coupon   0.04125000
#> 3 ctd_yield_pct 3.70000000
#> 4 conversion_factor 0.93050000
#> 5 gross_basis   0.00732489
#> 6 net_basis     0.25329120
#> 7 implied_repo  0.03923370
#> 8 ctd_bond_bpv  0.04101840
#> 9 ctd_futures_bpv 0.04408210

```

2.7 まとめ

本章では、金利期間構造を割引係数、ゼロ金利およびフォワード金利の三つの表現から確認した。これらは同じカーブを表すが、補間方式の影響はフォワード金利に強く現れる。

また、ゼロ金利ノードから直接構築する方法に加え、預金と債券、ならびに預金とスワップを用いたブートストラップを確認した。市場商品から構築したカーブでは、各商品の価格計算、市場慣行および日数計算規則が

期間構造へ組み込まれる。

さらに、固定参照日、Implied Term Structure および平行シフトを確認した。カーブを評価へ使用する際には、単にカーブオブジェクトを作成するだけでなく、参照日、補間方式、外挿、入力商品の再現性およびフォワード金利の形状を検証する必要がある。国債先物では、売方が複数の受渡適格銘柄から実際に引き渡す銘柄を選択する。銘柄間のクーポンや残存期間の差は変換係数によって調整されるが、変換係数だけで経済価値の差が完全に解消されるわけではない。

そのため、CTD の選定では、現物価格と変換係数から求めるグロスベースに加え、レボ費用、クーポン収入および経過利子の変化を反映したキャリーを考慮する。ネットベースが最小、またはインプライド・レボが最大となる銘柄が、売方にとって最も有利な受渡銘柄となる。

国債先物の BPV は、CTD 債券の BPV と変換係数に依存する。ただし、市場環境の変化によって CTD が切り替わる可能性があるため、国債先物のリスク管理では受渡銘柄選択権も考慮する必要がある。

第 II-3 章 OIS スワップとマルチカーブ評価

3.1 IBOR 改革と OIS の基礎

3.1.1 IBOR 改革と歴史的背景

2007 年から 2009 年にかけての世界金融危機、とりわけ 2008 年 9 月のリーマン・ブラザーズ破綻は、金利デリバティブの評価方法を大きく変化させた。それ以前の市場では、LIBOR は信用リスクをほとんど含まない基準金利として扱われ、同一の LIBOR カーブから将来の変動金利を予測すると同時に、将来キャッシュフローの現在価値を計算するシングルカーブ評価が広く用いられていた。

しかし、LIBOR は銀行間の無担保資金調達金利である。したがって、LIBOR には将来の短期金利に対する市場予想だけでなく、借り手銀行の信用リスク、資金流動性リスクおよび借入期間に応じたプレミアムが含まれる。金融危機時に LIBOR と OIS 金利の差が急拡大したことにより、LIBOR を信用リスクのない割引金利として扱うことができないことが明らかになった。

この認識を背景として、デリバティブ市場では中央清算および担保契約の利用が拡大した。担保付取引では、担保に付される翌日物金利に近い OIS カーブを用いて将来キャッシュフローを割り引くことが経済的に整合的である。一方、LIBOR、TIBOR または EURIBOR などを参照する契約の将来変動金利は、それぞれの参照金利に対応するフォワードカーブから予測する必要がある。その結果、将来金利を予測するカーブと現在価値を求めるディスカウントカーブを分離するマルチカーブ評価が定着した。

ここで、「担保付取引への移行」と「翌日物指標の性質」は区別する必要がある。米ドルの SOFR は米国債レポ取引に基づく有担保翌日物金利であるのに対し、日本円の TONA は無担保コール市場の翌日物金利である。いずれも翌日物のリスク・フリー・レートに近い指標として用いられるが、基礎となる市場の担保性は同一ではない。

さらに、LIBOR の不正操作および虚偽報告が明らかになり、銀行の判断に大きく依存する金利指標の信頼性が損なわれた。金融安定理事会は 2014 年、既存 IBOR の強化と、実取引に基づく代替的なリスク・フリー・レートの育成を提言した。2017 年には、英国の金融行為規制機構である FCA が、2021 年末以降も銀行へ LIBOR の呈示を求め続けることはないとの方針を示した。

代表性のある円、ポンド、ユーロおよびスイスフラン LIBOR の多くは 2021 年末に終了し、米ドル LIBOR の銀行パネルも 2023 年 6 月 30 日に終了した。経過措置として残された合成 LIBOR も 2024 年 9 月 30 日に最終公表され、すべての LIBOR 設定が恒久的に終了した。

LIBOR に代わる主要な指標として、日本円では TONA、米ドルでは SOFR が採用された。これらの翌日物金利を参照する OIS では、通常、計算期間中の日次金利を後決めて複利計算する。そのため、LIBOR のように期間開始時点で適用金利が確定する取引とは異なり、期間全体の金利と利息額が確定するのは期末近くとなる。日次金利の公表時刻、Fixing が利用可能になるまでの時間差、計算終了日から支払日までの事務処理期間

は、新たなオペレーショナル・リスクとして管理する必要がある。

以上の IBOR 改革によって、金利デリバティブ評価は、単一の LIBOR カーブですべてを処理する方法から、翌日物金利を基礎とする OIS カーブを割引に用い、参照金利ごとに将来金利を予測する方法へ移行した。

本章では、この歴史的変化を踏まえ、日本円の TONA および米ドルの SOFR を参照する OIS を取り上げる。まず市場レートから OIS カーブを構築し、固定レグと翌日物変動レグを評価する。次に、変動レグを日次金利へ分解し、Fixing の確定前後で評価額がどのように変化するかを確認する。最後に、将来金利を予測するカーブとキャッシュフローを割引くカーブを分離し、マルチカーブ評価の基本構造を示す。

3.1.2 OIS と翌日物金利

OIS (Overnight Index Swap) は、あらかじめ定めた固定金利と、計算期間中の翌日物金利を複利で積み上げた変動金利を交換する金利スワップである。元本そのものは交換せず、通常は固定レグと変動レグの利息差額を決済する。

日次金利を r_i 、その金利が適用される日数を d_i 、日数計算の年間基準を B とすると、計算期間全体の複利収益率は概念的に次式で表される。

$$R_{\text{comp}} = \prod_{i=1}^n \left(1 + r_i \frac{d_i}{B} \right) - 1$$

元本を N とすると、変動レグの利息額は次式となる。

$$I_{\text{overnight}} = NR_{\text{comp}}$$

固定金利を K 、固定レグの年率換算期間を α とすると、固定レグの利息額は次式となる。

$$I_{\text{fixed}} = NK\alpha$$

固定金利を支払う OIS の価値は、変動レグの現在価値から固定レグの現在価値を差し引いて求める。

$$PV_{\text{OIS}} = PV_{\text{overnight}} - PV_{\text{fixed}}$$

契約締結時に両レグの現在価値を等しくする固定金利がフェア OIS レートである。

以下では、カーブを表形式で確認するための補助関数を定義する。これは計算ロジックを新たに実装するものではなく、QuantLib のカーブから割引係数と連続複利ゼロ金利を取り出す表示用関数である。

```
curve_tbl_local <- function(curve_obj, n = 200, extrapolate = TRUE) {
  if (extrapolate) {
    QuantLib::TermStructure_enableExtrapolation(curve_obj)
  }

  reference_date <- as.Date(
    GaussQuant::iso_GQL(curve_obj$referenceDate())
```



```

)

curve_times <- seq(
  from = 0,
  to = curve_obj$maxTime(),
  length.out = n
)

tibble::tibble(time = curve_times) |>
  dplyr::mutate(
    discount_factor = purrr::map_dbl(
      time,
      function(x) curve_obj$discount(x)
    ),
    zero_rate = dplyr::if_else(
      time > 0,
      -log(discount_factor) / time,
      0
    ),
    date = reference_date + round(time * 365)
  ) |>
  dplyr::select(
    date,
    time,
    discount_factor,
    zero_rate
  )
}

```

3.2 TONA カーブと短期 OIS 評価

TONA は、日本の無担保コール市場における翌日物金利を基礎とする円のリスク・フリー・レートである。本節では、2022 年 8 月 19 日の例示的な TONA および OIS 市場レートから、円 OIS カーブを構築する。

短期部分には翌日物金利や短期 OIS を使用し、中長期部分には各満期の OIS スワップレートを使用する。ブートストラップでは、各市場商品の理論価格が市場価格と一致するように、満期ごとの割引係数を順次決定する。

3.2.1 TONA 市場レート

```

trade_date_tona <- "2022-08-19"

GaussQuant::set_eval_date_GQL(trade_date_tona)

```

```

tona_quotes <- tibble::tribble(
  ~as_of_date, ~currency, ~instrument, ~kind, ~tenor, ~rate,
  as.Date(trade_date_tona), "JPY", "TONA", "depo", "1D", -0.0000900,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "1W", -0.0001261,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "2W", -0.0001469,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "1M", -0.0001807,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "3M", -0.0001919,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "6M", -0.0001043,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "9M", 0.0000022,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "12M", 0.0001250,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "18M", 0.0003125,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "2Y", 0.0004875,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "3Y", 0.0007375,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "4Y", 0.0009479,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "5Y", 0.0011854,
  as.Date(trade_date_tona), "JPY", "TONA", "ois", "7Y", 0.0019146
)

GaussQuant::show_tbl_GQH(
  tona_quotes,
  "TONA market quotes",
  n = 20
)

#> # A tibble: 14 x 6
#>   as_of_date currency instrument kind tenor      rate
#>   <date>      <chr>      <chr>      <chr> <chr>    <dbl>
#> 1 2022-08-19 JPY      TONA      depo  1D     -0.00009
#> 2 2022-08-19 JPY      TONA      ois    1W     -0.0001261
#> 3 2022-08-19 JPY      TONA      ois    2W     -0.0001469
#> 4 2022-08-19 JPY      TONA      ois    1M     -0.0001807
#> 5 2022-08-19 JPY      TONA      ois    3M     -0.0001919
#> 6 2022-08-19 JPY      TONA      ois    6M     -0.0001043
#> 7 2022-08-19 JPY      TONA      ois    9M      0.0000022
#> 8 2022-08-19 JPY      TONA      ois   12M      0.000125
#> 9 2022-08-19 JPY      TONA      ois   18M      0.0003125
#> 10 2022-08-19 JPY      TONA      ois    2Y      0.0004875
#> 11 2022-08-19 JPY      TONA      ois    3Y      0.0007375
#> 12 2022-08-19 JPY      TONA      ois    4Y      0.0009479
#> 13 2022-08-19 JPY      TONA      ois    5Y      0.0011854
#> 14 2022-08-19 JPY      TONA      ois    7Y      0.0019146

```

3.2.2 TONA OIS カーブの構築

```

tona_curve_envs <- GaussQuant::ir_build_ois_curve_envs_GQH(
  quotes = tona_quotes,
  trade_date = trade_date_tona,
  verbose = TRUE
)

tona_curve_env <- tona_curve_envs[["JPY::TONA"]]
tona_curve <- tona_curve_env$curve

tona_curve_display_tbl <- curve_tbl_local(
  curve_obj = tona_curve,
  n = 200
)

GaussQuant::show_tbl_GQH(
  tona_curve_display_tbl,
  "TONA OIS curve",
  n = 12
)

#> # A tibble: 12 x 4
#>   date          time discount_factor zero_rate
#>   <date>        <dbl>          <dbl>      <dbl>
#> 1 2022-08-23 0          1          0
#> 2 2022-09-05 0.035203    1.0000 -0.00014504
#> 3 2022-09-18 0.070407    1.0000 -0.00017306
#> 4 2022-10-01 0.10561      1.0000 -0.00018278
#> 5 2022-10-13 0.14081      1.0000 -0.00018668
#> 6 2022-10-26 0.17602      1.0000 -0.00018902
#> 7 2022-11-08 0.21122      1.0000 -0.00019057
#> 8 2022-11-21 0.24642      1.0000 -0.00019169
#> 9 2022-12-04 0.28163      1.0000 -0.00017512
#> 10 2022-12-17 0.31683      1.0000 -0.00015741
#> 11 2022-12-29 0.35203      1.0001 -0.00014325
#> 12 2023-01-11 0.38724      1.0001 -0.00013166

tona_curve_check_tbl <- tibble::tibble(
  reference_date = GaussQuant::iso_GQL(
    tona_curve$referenceDate()
  ),
  discount_2022_08_23 = tona_curve$discount(

```

```

    GaussQuant::date_GQL("2022-08-23")
  )
)

GaussQuant::show_tbl_GQH(
  tona_curve_check_tbl,
  "TONA curve checks",
  n = 20
)
#> # A tibble: 1 x 2
#>   reference_date discount_2022_08_23
#>   <chr>                <dbl>
#> 1 2022-08-23                1

```

割引係数 $P(0, T)$ は、将来時点 T の 1 円を評価日時点の価値へ換算する係数である。満期が長くなるほど、通常は割引係数が低下する。ゼロ金利は、各満期の割引係数を単一の金利として表したものである。

3.2.3 JPY 短期 OIS の評価

次に、TONA カーブを用いて短期円 OIS を評価する。評価対象は、2022 年 8 月 23 日から 8 月 30 日までの 7 日間、元本 1,000 万円、固定金利マイナス 0.015% の固定金利支払い OIS である。

OIS の固定レグと翌日物レグを確認し、NPV とフェアレートを表示する。

```

jpy_ois_short <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = tona_curve_env,
  effective = "2022-08-23",
  maturity = "2022-08-30",
  notional = 10000000,
  fixed_rate = -0.00015,
  pay_receive = "pay",
  verbose = TRUE
)

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = jpy_ois_short,
  title = "JPY short OIS summary"
)

jpy_ois_fixed_cf_tbl <- GaussQuant::swap_fixed_leg_table_GQL(
  jpy_ois_short$swap
)

jpy_ois_overnight_cf_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  jpy_ois_short$swap
)

```

```

)

GaussQuant::show_tbl_GQH(
  jpy_ois_fixed_cf_tbl,
  "TONA fixed leg cashflows",
  n = 10
)

#> # A tibble: 1 x 2
#>   date      amount
#>   <chr>      <dbl>
#> 1 2022-08-30 -28.767

GaussQuant::show_tbl_GQH(
  jpy_ois_overnight_cf_tbl,
  "TONA overnight leg cashflows",
  n = 10
)

#> # A tibble: 1 x 2
#>   date      amount
#>   <chr>      <dbl>
#> 1 2022-08-30 -24.184

```

3.2.4 フェアレートの手計算

単一期間の OIS では、開始日の割引係数を $P(0, T_0)$ 、終了日の割引係数を $P(0, T_1)$ 、固定レグの支払日の割引係数を $P(0, T_p)$ 、固定レグの年率換算期間を α とすると、フェアレートは概念的に次式で求められる。

$$K_{\text{fair}} = \frac{P(0, T_0) - P(0, T_1)}{\alpha P(0, T_p)}$$

```

jpy_effective_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-08-23")
)

jpy_maturity_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-08-30")
)

jpy_payment_df <- tona_curve$discount(
  GaussQuant::date_GQL("2022-09-01")
)

jpy_annuity <- (7 / 365) * jpy_payment_df

```

```

jpy_swap_rate_hand <- (
  jpy_effective_df - jpy_maturity_df
) / jpy_annuity

jpy_fair_rate_check_tbl <- tibble::tibble(
  effective_discount_factor = jpy_effective_df,
  maturity_discount_factor = jpy_maturity_df,
  payment_discount_factor = jpy_payment_df,
  annuity = jpy_annuity,
  fair_swap_rate_hand = jpy_swap_rate_hand
)

GaussQuant::show_tbl_GQH(
  jpy_fair_rate_check_tbl,
  "TONA fair-rate hand check",
  n = 20
)

#> # A tibble: 1 x 5
#>   effective_discount_fa~1 maturity_discount_fa~2 payment_discount_fac~3 annuity
#>   <dbl> <dbl> <dbl> <dbl>
#> 1      1      1.0000      1.0000 0.019178
#> # i abbreviated names: 1: effective_discount_factor,
#> #   2: maturity_discount_factor, 3: payment_discount_factor
#> # i 1 more variable: fair_swap_rate_hand <dbl>

```

3.3 日次複利と Fixing

3.3.1 OIS 変動レグの日次複利

OIS の変動レグでは、計算期間中の翌日物金利を日ごとに複利で積み上げる。休日をまたぐ金利は、次の営業日まで複数日分に適用されるため、各行の適用日数を確認する必要がある。

次のコードでは、変動レグを日次単位に分解し、割引係数から日次フォワード金利を求める。

```

jpy_ois_daily <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = tona_curve_env,
  effective = "2022-08-23",
  maturity = "2022-08-30",
  notional = 10000000,
  fixed_rate = -0.00015,
  pay_receive = "pay",
  fixed_schedule_tenor = "1Y",
  floating_schedule_tenor = "1D",
  pay_lag = 0,

```

```

    verbose = TRUE
  )

GaussQuant::set_eval_date_GQL(trade_date_tona)

jpy_daily_float_cf_tbl <- GaussQuant::ois_daily_forward_table_GQH(
  swap = jpy_ois_daily$swap,
  curve = tona_curve,
  index = jpy_ois_daily$index,
  eval_date = trade_date_tona,
  notional = 10000000
)

GaussQuant::show_tbl_GQH(
  jpy_daily_float_cf_tbl,
  "TONA daily forward leg",
  n = 20
)

#> # A tibble: 5 x 9
#>   fixing_date next_date   days fixing_before_eval fixing_on_eval fixing_value
#>   <chr>         <chr>   <int> <lgl>                <lgl>                <dbl>
#> 1 2022-08-23 2022-08-24     1 FALSE                FALSE                -0.000090000
#> 2 2022-08-24 2022-08-25     1 FALSE                FALSE                -0.00013212
#> 3 2022-08-25 2022-08-26     1 FALSE                FALSE                -0.00013212
#> 4 2022-08-26 2022-08-29     3 FALSE                FALSE                -0.00013212
#> 5 2022-08-29 2022-08-30     1 FALSE                FALSE                -0.00013212
#> # i 3 more variables: forward_rate_from_df <dbl>, applied_rate <dbl>,
#> #   amount <dbl>

```

各日の単利フォワード金利は、開始日の割引係数を $P(0, t_i)$ 、終了日の割引係数を $P(0, t_{i+1})$ 、年率換算期間を δ_i とすると、次式で求められる。

$$f_i = \frac{P(0, t_i)/P(0, t_{i+1}) - 1}{\delta_i}$$

日次フォワード金利を用いて複利収益率と利息額を確認する。

```

daily_compounding_result <- if (
  nrow(jpy_daily_float_cf_tbl) >= 2 &&
  all(c("days", "rate") %in% names(jpy_daily_float_cf_tbl))
) {
  daily_compounded_return <- (
    1 +
    jpy_daily_float_cf_tbl$days[-1] /
    365 *

```

```

    jpy_daily_float_cf_tbl$rate[-1]
  ) |>
  prod() -
  1

  tibble::tibble(
    compounded_return = daily_compounded_return,
    floating_interest = daily_compounded_return * 10000000,
    equivalent_annual_rate = daily_compounded_return * 365 / 7
  )
} else {
  tibble::tibble(
    compounded_return = NA_real_,
    floating_interest = NA_real_,
    equivalent_annual_rate = NA_real_
  )
}

GaussQuant::show_tbl_GQH(
  daily_compounding_result,
  "TONA daily compounding result",
  n = 20
)

#> # A tibble: 1 x 3
#>   compounded_return floating_interest equivalent_annual_rate
#>   <dbl>             <dbl>             <dbl>
#> 1             NA             NA             NA

```

3.3.2 Fixing の確定と再評価

評価日より前の翌日物金利には、実際に公表された Fixing を使用する。評価日より後の金利は、OIS カーブから推定したフォワード金利を使用する。

したがって、評価途中の OIS 変動レグは、次の二つから構成される。

- 評価日までに確定した日次 Fixing の複利部分
- 評価日より後のフォワード金利による予測部分

後決め複利では、期間終了まで変動レグ全体の利息額が確定しない。Fixing の公表時刻と支払処理の間に必要な時間を確保するため、実務では Lookback、Observation Shift、Lockout または Payment Delay などの規則が用いられることがある。

次の例では、2022 年 8 月 23 日および 24 日の TONA Fixing を登録し、2022 年 8 月 25 日時点で再評価する。

```

jpy_fixings_tbl <- tibble::tribble(
  ~date, ~currency, ~instrument, ~fixing,

```



```

    "2022-08-23", "JPY", "TONA", -0.000090,
    "2022-08-24", "JPY", "TONA", -0.000085
  )

GaussQuant::set_eval_date_GQL(trade_date_tona)

jpy_overnight_before_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  jpy_ois_short$swap
)

GaussQuant::ir_apply_fixings_GQH(
  index_obj = jpy_ois_short$index,
  fixings_tbl = jpy_fixings_tbl,
  currency = "JPY",
  instrument = "TONA"
)

GaussQuant::set_eval_date_GQL("2022-08-25")

jpy_overnight_after_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  jpy_ois_short$swap
)

GaussQuant::ir_show_fixing_before_after_GQH(
  before_tbl = jpy_overnight_before_tbl,
  after_tbl = jpy_overnight_after_tbl,
  title = "TONA fixing before / after",
  n = 10
)

#> # A tibble: 1 x 3
#>   row_id amount_before amount_after
#>   <int>         <dbl>         <dbl>
#> 1     1           -24.184         -24.183

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = jpy_ois_short,
  title = "TONA short OIS summary after fixings"
)

jpy_fixing_diag_tbl <- GaussQuant::ir_fixing_table_GQH(
  swap = jpy_ois_daily$swap,
  curve = tona_curve,
  index = jpy_ois_short$index

```

```

)

GaussQuant::show_tbl_GQH(
  jpy_fixing_diag_tbl,
  "TONA daily fixing decomposition",
  n = 20
)

#> # A tibble: 1 x 7
#>       i fixing_date fixing_value pay_date   amount    df      pv
#>   <int> <chr>          <dbl> <chr>      <dbl> <dbl> <dbl>
#> 1     1 2022-08-29   -0.00014154 2022-08-30 -24.183 1.0000 -24.183

```

3.4 SOFR とマルチカーブ評価

SOFR は、米国債を担保とする翌日物レポ取引に基づく米ドルのリスク・フリー・レートである。TONA が無担保翌日物市場に基づくのに対し、SOFR は有担保取引に基づく点が異なる。

OIS 評価の基本構造は TONA と同じであり、通貨、休日カレンダー、日数計算規則、市場クォートおよび翌日物指数が異なる。

3.4.1 SOFR 市場レートと OIS カーブ

```

trade_date_sofr <- "2023-09-26"

GaussQuant::set_eval_date_GQL(trade_date_sofr)

sofr_quotes <- tibble::tribble(
  ~as_of_date, ~currency, ~instrument, ~kind, ~tenor, ~rate,
  as.Date(trade_date_sofr), "USD", "SOFR", "depo", "1D", 0.0531,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "1M", 0.0532,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "3M", 0.0538,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "6M", 0.0546,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "1Y", 0.0545,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "2Y", 0.0501,
  as.Date(trade_date_sofr), "USD", "SOFR", "ois", "3Y", 0.0467
)

sofr_curve_envs <- GaussQuant::ir_build_ois_curve_envs_GQH(
  quotes = sofr_quotes,
  trade_date = trade_date_sofr,
  verbose = TRUE
)

```

```
sofr_curve_env <- sofr_curve_envs[["USD::SOFR"]]
sofr_curve <- sofr_curve_env$curve

GaussQuant::show_tbl_GQH(
  curve_tbl_local(sofr_curve),
  "SOFR OIS curve",
  n = 12
)
#> # A tibble: 12 x 4
#>   date          time discount_factor zero_rate
#>   <date>        <dbl>          <dbl>     <dbl>
#> 1 2023-09-28 0          1          0
#> 2 2023-10-04 0.015299    0.99919 0.053075
#> 3 2023-10-09 0.030597    0.99838 0.053075
#> 4 2023-10-15 0.045896    0.99757 0.053075
#> 5 2023-10-20 0.061195    0.99676 0.053075
#> 6 2023-10-26 0.076494    0.99595 0.053075
#> 7 2023-11-01 0.091792    0.99514 0.053092
#> 8 2023-11-06 0.10709     0.99432 0.053170
#> 9 2023-11-12 0.12239     0.99351 0.053228
#> 10 2023-11-17 0.13769     0.99269 0.053273
#> 11 2023-11-23 0.15299     0.99188 0.053309
#> 12 2023-11-28 0.16829     0.99106 0.053339
```

3.4.2 SOFR OIS の評価

```
sofr_ois_trade <- GaussQuant::trade_ois_swap_py_GQH(
  curve_env = sofr_curve_env,
  effective = "2023-09-28",
  maturity = "2025-09-28",
  notional = 10000000,
  fixed_rate = 0.05,
  pay_receive = "pay",
  verbose = TRUE
)

GaussQuant::ir_show_swap_summary_GQH(
  trade_env = sofr_ois_trade,
  title = "SOFR OIS summary"
)

sofr_overnight_cf_tbl <- GaussQuant::ois_overnight_leg_table_GQL(
  sofr_ois_trade$swap
```

```
)

GaussQuant::show_tbl_GQH(
  sofr_overnight_cf_tbl,
  "SOFR overnight leg cashflows",
  n = 20
)

#> # A tibble: 2 x 2
#>   date          amount
#>   <chr>         <dbl>
#> 1 2024-09-30 557111.
#> 2 2025-09-29 459522.
```

3.4.3 マルチカーブ評価

金融危機以前のシングルカーブ評価では、同一の金利カーブを用いて、将来変動金利の予測とキャッシュフローの割引を行っていた。

マルチカーブ評価では、二つの役割を分離する。

- フォワードカーブ参照金利の将来 Fixing または期間フォワード金利を予測する。
- ディスカウントカーブ担保契約に対応する金利を用いて、将来キャッシュフローを現在価値へ割り引く。

将来変動金利を F_j 、年率換算期間を α_j 、割引係数を D_j 、元本を N とすると、変動レグの現在価値は概念的に次式で表される。

$$PV_{\text{floating}} = N \sum_{j=1}^m \alpha_j F_j D_j$$

ここで、 F_j はフォワードカーブから求め、 D_j はディスカウントカーブから求める。

以下では、マルチカーブの効果だけを明確にするため、EURIBOR 6 か月物を参照する一般的なバニラスワップを用いる。同じフォワードカーブを使用しながら、割引カーブを同一にした場合と分離した場合の NPV を比較する。

```
GaussQuant::set_eval_date_GQL("2023-09-26")

multi_curve_dates <- c(
  "2023-09-26",
  "2024-09-26",
  "2025-09-26",
  "2026-09-28",
  "2028-09-26"
)
```

```
forecast_zero_rates <- c(
  0.0450,
  0.0470,
  0.0460,
  0.0445,
  0.0420
)

discount_zero_rates <- c(
  0.0400,
  0.0415,
  0.0410,
  0.0400,
  0.0380
)

forecast_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = multi_curve_dates,
  zero_rates = forecast_zero_rates,
  day_counter = "Actual365Fixed"
)

discount_curve <- GaussQuant::build_zero_curve_GQL(
  date_chr = multi_curve_dates,
  zero_rates = discount_zero_rates,
  day_counter = "Actual365Fixed"
)

QuantLib::TermStructure_enableExtrapolation(forecast_curve)
#> NULL
QuantLib::TermStructure_enableExtrapolation(discount_curve)
#> NULL

forecast_handle <- GaussQuant::yield_curve_handle_GQL(
  forecast_curve
)

discount_handle <- GaussQuant::yield_curve_handle_GQL(
  discount_curve
)

multi_curve_comparison_tbl <- tibble::tibble(
  date = as.Date(multi_curve_dates)
```

```

) |>
dplyr::mutate(
  forecast_discount_factor = purrr::map_dbl(
    as.character(date),
    function(x) {
      forecast_curve$discount(
        GaussQuant::date_GQL(x)
      )
    }
  ),
  ois_discount_factor = purrr::map_dbl(
    as.character(date),
    function(x) {
      discount_curve$discount(
        GaussQuant::date_GQL(x)
      )
    }
  )
)

GaussQuant::show_tbl_GQH(
  multi_curve_comparison_tbl,
  "Forecast and discount curves",
  n = 20
)

#> # A tibble: 5 x 3
#>   date          forecast_discount_factor ois_discount_factor
#>   <date>                <dbl>                <dbl>
#> 1 2023-09-26                1                1
#> 2 2024-09-26            0.95396            0.95924
#> 3 2025-09-26            0.91199            0.92117
#> 4 2026-09-28            0.87471            0.88663
#> 5 2028-09-26            0.81040            0.82679

```

3.4.4 シングルカーブ評価との比較

```

single_curve_swap <- GaussQuant::make_vanilla_swap_GQL(
  effective_date = "2023-09-28",
  maturity_date = "2025-09-28",
  fixed_rate = 0.045,
  notional = 10000000,
  forecast_handle = forecast_handle,
  discount_handle = forecast_handle,

```

```

    swap_type = "payer",
    index = "Euribor6M",
    fixed_tenor = GaussQuant::period_years_GQL(1),
    floating_tenor = GaussQuant::period_months_GQL(6),
    calendar = QuantLib::TARGET()
  )

multi_curve_swap <- GaussQuant::make_vanilla_swap_GQL(
  effective_date = "2023-09-28",
  maturity_date = "2025-09-28",
  fixed_rate = 0.045,
  notional = 10000000,
  forecast_handle = forecast_handle,
  discount_handle = discount_handle,
  swap_type = "payer",
  index = "Euribor6M",
  fixed_tenor = GaussQuant::period_years_GQL(1),
  floating_tenor = GaussQuant::period_months_GQL(6),
  calendar = QuantLib::TARGET()
)

single_curve_summary <- GaussQuant::swap_summary_GQL(
  single_curve_swap
)

multi_curve_summary <- GaussQuant::swap_summary_GQL(
  multi_curve_swap
)

curve_valuation_comparison_tbl <- dplyr::bind_rows(
  single_curve_summary |>
    dplyr::mutate(valuation_method = "single_curve"),
  multi_curve_summary |>
    dplyr::mutate(valuation_method = "multi_curve")
) |>
  dplyr::relocate(valuation_method)

GaussQuant::show_tbl_GQH(
  curve_valuation_comparison_tbl,
  "Single-curve and multi-curve valuation",
  n = 20
)

#> # A tibble: 2 x 6

```

```
#>   valuation_method   npv fixed_leg_npv floating_leg_npv fair_rate fair_spread
#>   <chr>             <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
#> 1 single_curve      40292.        -840562.        880855.    0.047157    -0.0020999
#> 2 multi_curve       39436.        -847103.        886539.    0.047095    -0.0020420
```

両方のスワップは同じフォワードカーブから将来 EURIBOR を予測するが、シングルカーブ評価ではそのカーブを割引にも使用し、マルチカーブ評価では別のディスカウントカーブを使用する。割引係数が異なるため、同じ将来キャッシュフローであっても NPV およびフェアレートが変化する。

3.5 ケーススタディ 1：EONIA カーブと OIS 評価

本ケーススタディでは、2012 年 12 月 11 日の EONIA 市場データから OIS カーブを構築する。短期部分には翌日物預金と短期 OIS を用い、その先には日付指定 OIS と長期 OIS を用いる。

カーブ構築後、年末をまたぐ市場金利に含まれる流動性効果を分離し、年末ジャンプを組み込んだ最終的な EONIA カーブを作成する。さらに、カーブ構築に使用した 5 年 OIS の市場レートを契約固定金利として OIS を構築し、NPV とフェアレートを確認する。

3.5.1 評価日と EONIA 市場データ

評価日は 2012 年 12 月 11 日とする。市場データには、翌日物預金、短期 OIS、日付指定 OIS および長期 OIS を使用する。

```
case_evaluation_date <- "2012-12-11"

GaussQuant::set_eval_date_GQL(
  case_evaluation_date
)

case_eonia_quotes <- GaussQuant::ametrano_bianchetti_eonia_quotes_GQL()

GaussQuant::show_tbl_GQH(
  case_eonia_quotes,
  "EONIA market quotes",
  n = 40
)

#> # A tibble: 30 x 9
#>   quote_id instrument_type rate_pct   rate fixing_days tenor_n tenor_unit
#>   <chr>      <chr>         <dbl>   <dbl>    <int>   <int> <chr>
#> 1 DEP-ON-0D deposit         0.04   0.0004      0      1 Days
#> 2 DEP-ON-1D deposit         0.04   0.0004      1      1 Days
#> 3 DEP-ON-2D deposit         0.04   0.0004      2      1 Days
#> 4 OIS-1W   spot_ois         0.07   0.0007      2      1 Weeks
#> 5 OIS-2W   spot_ois         0.069  0.00069     2      2 Weeks
```



```
#> 6 OIS-3W spot_ois 0.078 0.00078 2 3 Weeks
#> 7 OIS-1M spot_ois 0.074 0.00074 2 1 Months
#> 8 OIS-ECB-1 dated_ois 0.046 0.00046 NA NA <NA>
#> 9 OIS-ECB-2 dated_ois 0.016 0.00016 NA NA <NA>
#> 10 OIS-ECB-3 dated_ois -0.007 -0.00007 NA NA <NA>
#> # i 20 more rows
#> # i 2 more variables: start_date <chr>, end_date <chr>
```

rate_pct はパーセント表示の市場レートであり、rate は QuantLib へ渡す小数表示の金利である。

預金には 1 日の期間を設定し、評価日からスポット日までを接続するため、Fixing 日数を 0 日、1 日および 2 日とした。短期 OIS と長期 OIS にはスポット日から満期までの期間を指定し、日付指定 OIS には開始日と終了日を直接指定する。

3.5.2 EONIA カーブの構築

次に、市場レートから EONIA カーブを構築する。

最初に対数割引係数を 3 次補間するカーブと、区分的に一定のフォワード金利を用いるカーブを構築する。後者を用いて年末部分のフォワード金利を確認し、年末ジャンプを推定する。

```
case_eonia_benchmark <- GaussQuant::eonia_curve_benchmark_GQL(
  quotes = case_eonia_quotes,
  evaluation_date = case_evaluation_date
)

case_eonia_curve <- case_eonia_benchmark$final$curve
case_eonia_handle <- case_eonia_benchmark$final$curve_handle

case_jump_summary <- case_eonia_benchmark$jump_summary

GaussQuant::show_tbl_GQH(
  case_jump_summary,
  "EONIA year-end jump",
  n = 20
)
#> # A tibble: 1 x 7
#>   original_forward clean_forward t12 jump_time jump_rate
#>   <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1 0.00081531 0.00067122 0.038356 0.0054795 0.0010086
#> # i 2 more variables: jump_discount_factor <dbl>, jump_date <date>
```

年末ジャンプの推定では、2012 年 12 月 24 日から 2013 年 1 月 7 日までのフォワード金利を確認する。次に、2013 年 1 月 3 日の区分フォワード金利を前後のノードの平均へ置き換え、年末要因を含まないフォワードカーブを作成する。

元のフォワード金利と調整後のフォワード金利の差を、2012 年 12 月 31 日から 2013 年 1 月 2 日までの期間へ換算し、年末ジャンプを求める。

3.5.3 ベンチマーク値の確認

年末ジャンプを推定する過程で得られたフォワード金利を確認する。

```
case_benchmark_check_tbl <- tibble::tribble(
  ~metric, ~reference_value_pct, ~r_value_pct,
  "Original two-week forward", 0.082,
  100 * case_jump_summary$original_forward[[1L]],
  "Adjusted two-week forward", 0.067,
  100 * case_jump_summary$clean_forward[[1L]],
  "Year-end jump", 0.101,
  100 * case_jump_summary$jump_rate[[1L]]
) |>
dplyr::mutate(
  difference_pct =
    r_value_pct -
    reference_value_pct
)

GaussQuant::show_tbl_GQH(
  case_benchmark_check_tbl,
  "EONIA benchmark comparison",
  n = 20
)

#> # A tibble: 3 x 4
#>   metric                reference_value_pct r_value_pct difference_pct
#>   <chr>                  <dbl>         <dbl>         <dbl>
#> 1 Original two-week forward      0.082      0.081531    -0.00046933
#> 2 Adjusted two-week forward      0.067      0.067122     0.00012164
#> 3 Year-end jump                  0.101      0.10086    -0.00013673

stopifnot(
  all(
    abs(case_benchmark_check_tbl$difference_pct) <= 0.0005
  )
)
```

基準値はパーセント単位で小数第 3 位まで表示されている。そのため、照合では表示桁の半分に相当する 0.0005 パーセントポイントを許容範囲とする。

3.5.4 市場レートの再現性

構築した最終カーブを用いて、各 Rate Helper のインプライド・レートを求める。

```
case_eonia_validation <- case_eonia_benchmark$final_validation

case_eonia_repricing_tbl <- case_eonia_validation$quote_repricing |>
  dplyr::mutate(
    quote_error_bp = 10000 * quote_error
  )

GaussQuant::show_tbl_GQH(
  case_eonia_repricing_tbl,
  "EONIA quote repricing",
  n = 40
)

#> # A tibble: 30 x 10
#>   quote_id instrument_type market_quote implied_quote quote_error pillar_date
#>   <chr>      <chr>           <dbl>         <dbl>         <dbl> <date>
#> 1 DEP-ON-0D deposit           0.0004      0.00040000    2.6205e-15 2012-12-12
#> 2 DEP-ON-1D deposit           0.0004      0.00040000    2.6205e-15 2012-12-13
#> 3 DEP-ON-2D deposit           0.0004      0.00040000    2.6205e-15 2012-12-14
#> 4 OIS-1W    spot_ois           0.0007      0.00070000    4.5565e-13 2012-12-20
#> 5 OIS-2W    spot_ois           0.00069     0.00069000    1.0801e-14 2012-12-27
#> 6 OIS-3W    spot_ois           0.00078     0.00078000   -1.3611e-15 2013-01-03
#> 7 OIS-1M    spot_ois           0.00074     0.00074000    1.8503e-15 2013-01-14
#> 8 OIS-ECB-1 dated_ois         0.00046     0.00046000    6.5201e-12 2013-02-13
#> 9 OIS-ECB-2 dated_ois         0.00016     0.00016000    8.4594e-12 2013-03-13
#> 10 OIS-ECB-3 dated_ois        -0.00007    -0.000070000   9.1291e-12 2013-04-10
#> # i 20 more rows
#> # i 4 more variables: discount_factor <dbl>, zero_rate <dbl>,
#> #   one_day_forward_rate <dbl>, quote_error_bp <dbl>

stopifnot(
  max(
    abs(case_eonia_repricing_tbl$quote_error),
    na.rm = TRUE
  ) < 1e-8
)
```

各商品のインプライド・レートと市場レートが一致することにより、預金、短期 OIS、日付指定 OIS および長期 OIS が同じ期間構造上で再現されていることを確認できる。

3.5.5 5 年 OIS の構築

市場データに含まれる 5 年 OIS を取り上げる。契約固定金利には、カーブ構築に用いた 5 年 OIS の市場レートを使用する。

```
case_ois_tenor <- QuantLib::Period(
  5,
  "Years"
)

case_ois_fixed_rate <- case_eonia_quotes |>
  dplyr::filter(
    quote_id == "OIS-5Y"
  ) |>
  dplyr::pull(
    rate
  )

case_eonia_index <- QuantLib::Eonia(
  case_eonia_handle
)

case_ois_contract_tbl <- tibble::tibble(
  evaluation_date = as.Date(case_evaluation_date),
  index = "EONIA",
  tenor = "5Y",
  settlement_days = 2L,
  contract_fixed_rate = case_ois_fixed_rate
)

GaussQuant::show_tbl_GQH(
  case_ois_contract_tbl,
  "Five-year EONIA OIS contract",
  n = 20
)

#> # A tibble: 1 x 5
#>   evaluation_date index tenor settlement_days contract_fixed_rate
#>   <date>         <chr> <chr>          <int>          <dbl>
#> 1 2012-12-11     EONIA 5Y              2           0.00456
```

3.5.6 OIS の構築と計算結果の照合

最初に、QuantLib の MakeOIS を直接使用して OIS を構築する。

```
case_ois_builder_quantlib <- QuantLib::MakeOIS(
  swapTenor = case_ois_tenor,
  overnightIndex = case_eonia_index,
  fixedRate = case_ois_fixed_rate
)

case_ois_quantlib <- QuantLib::MakeOIS_makeOIS(
  case_ois_builder_quantlib
)
```

次に、同じ期間、同じ EONIA インデックスおよび同じ固定金利を用いて OIS を構築する。

```
case_ois_gaussquant <- GaussQuant::build_ois_GQL(
  swap_tenor = case_ois_tenor,
  overnight_index = case_eonia_index,
  fixed_rate = case_ois_fixed_rate
)
```

両方の OIS について、NPV、固定レグの現在価値、翌日物レグの現在価値、フェアレートおよびフェアスプレッドを確認する。

```
case_ois_quantlib_summary <- GaussQuant::swap_summary_GQL(
  case_ois_quantlib
)

case_ois_gaussquant_summary <- GaussQuant::swap_summary_GQL(
  case_ois_gaussquant
)

case_ois_comparison_tbl <- dplyr::bind_rows(
  case_ois_quantlib_summary |>
    dplyr::mutate(
      calculation_route = "QuantLib"
    ),
  case_ois_gaussquant_summary |>
    dplyr::mutate(
      calculation_route = "GaussQuant"
    )
) |>
  dplyr::relocate(
    calculation_route
  )

GaussQuant::show_tbl_GQH(
  case_ois_comparison_tbl,
```

```

    "Five-year EONIA OIS valuation",
    n = 20
  )
#> # A tibble: 2 x 6
#>   calculation_route      npv fixed_leg_npv floating_leg_npv fair_rate
#>   <chr>              <dbl>      <dbl>          <dbl>      <dbl>
#> 1 QuantLib          -1.6546e-13    -0.022951      0.022951  0.0045600
#> 2 GaussQuant        -1.6546e-13    -0.022951      0.022951  0.0045600
#> # i 1 more variable: fair_spread <dbl>

case_ois_metrics <- c(
  "npv",
  "fixed_leg_npv",
  "floating_leg_npv",
  "fair_rate",
  "fair_spread"
)

case_quantlib_values <- unlist(
  case_ois_quantlib_summary |>
    dplyr::select(
      dplyr::all_of(case_ois_metrics)
    ),
  use.names = FALSE
)

case_gaussquant_values <- unlist(
  case_ois_gaussquant_summary |>
    dplyr::select(
      dplyr::all_of(case_ois_metrics)
    ),
  use.names = FALSE
)

stopifnot(
  isTRUE(
    all.equal(
      case_quantlib_values,
      case_gaussquant_values,
      tolerance = 1e-12
    )
  )
)

```

3.5.7 市場固定金利とフェアレート

カーブ構築に使用した 5 年 OIS の市場レートと、構築後のカーブから求めたフェアレートを比較する。

```
case_ois_rate_check_tbl <- tibble::tibble(
  market_fixed_rate = case_ois_fixed_rate,
  calculated_fair_rate =
    case_ois_gaussquant_summary$fair_rate[[1L]],
  difference_bp = 10000 * (
    case_ois_gaussquant_summary$fair_rate[[1L]] -
    case_ois_fixed_rate
  ),
  npv = case_ois_gaussquant_summary$npv[[1L]]
)

GaussQuant::show_tbl_GQH(
  case_ois_rate_check_tbl,
  "Five-year EONIA OIS rate check",
  n = 20
)

#> # A tibble: 1 x 4
#>   market_fixed_rate calculated_fair_rate difference_bp      npv
#>   <dbl>             <dbl>             <dbl>      <dbl>
#> 1         0.00456         0.0045600      -3.2876e-10 -1.6546e-13

stopifnot(
  abs(
    case_ois_gaussquant_summary$fair_rate[[1L]] -
    case_ois_fixed_rate
  ) < 1e-8
)
```

5 年 OIS はカーブ構築に使用した商品の一つであるため、市場固定金利とカーブから再計算したフェアレートは一致し、NPV はゼロに近い値となる。

3.5.8 固定レグと翌日物レグ

最後に、5 年 OIS の固定レグと翌日物レグのキャッシュフローを確認する。

```
case_ois_fixed_leg_tbl <- GaussQuant::leg_to_cashflow_tbl_GQL(
  case_ois_gaussquant$fixedLeg()
)

case_ois_overnight_leg_tbl <- GaussQuant::leg_to_cashflow_tbl_GQL(
```

```

    case_ois_gaussquant$overnightLeg()
)

GaussQuant::show_tbl_GQH(
  case_ois_fixed_leg_tbl,
  "Five-year EONIA OIS fixed leg",
  n = 20
)

#> # A tibble: 5 x 2
#>   date          amount
#>   <chr>          <dbl>
#> 1 2013-12-13 0.0046233
#> 2 2014-12-15 0.0046487
#> 3 2015-12-14 0.0046107
#> 4 2016-12-13 0.0046233
#> 5 2017-12-13 0.0046233

GaussQuant::show_tbl_GQH(
  case_ois_overnight_leg_tbl,
  "Five-year EONIA OIS overnight leg",
  n = 20
)

#> # A tibble: 5 x 2
#>   date          amount
#>   <chr>          <dbl>
#> 1 2013-12-13 0.000014593
#> 2 2014-12-15 0.00071766
#> 3 2015-12-14 0.0031409
#> 4 2016-12-13 0.0072966
#> 5 2017-12-13 0.012153

```

固定レグと翌日物レグは反対方向のキャッシュフローを持ち、市場固定金利がフェアレートと一致する場合には、両レグの現在価値が相殺される。

3.6 ケーススタディ 2：市場レート変動と評価損益

第 9 節では、2012 年 12 月 11 日の EONIA 市場データから期間構造を構築し、5 年 OIS の市場固定金利とフェアレートが一致することを確認した。

本ケーススタディでは、評価日と OIS の約定条件を変更せず、カーブ構築に使用する市場レートだけを上下へ 10 ベースポイント変化させる。各シナリオについて EONIA カーブを再構築し、第 9 節と同じ 5 年 OIS を再評価する。

比較するシナリオは、次の三つである。

- EONIA 市場レートを一律に 10 ベーシスポイント低下
- 第 9 節と同じ基準市場レート
- EONIA 市場レートを一律に 10 ベーシスポイント上昇

契約固定金利には、第 9 節で設定した 5 年 OIS の市場レート 0.456% を引き続き使用する。したがって、各シナリオの NPV の差は、市場レートの変化による評価損益を表す。

3.6.1 市場レートシナリオ

最初に、基準となる EONIA 市場レートへ、マイナス 10 ベーシスポイント、ゼロおよびプラス 10 ベーシスポイントのシフトを加える。

ここでは構築済みのゼロカーブを直接動かすのではなく、預金、短期 OIS、日付指定 OIS および長期 OIS の市場レートを変化させる。その後、それぞれの市場データから EONIA カーブを改めて構築する。

```
case_rate_scenario_tbl <- tibble::tribble(
  ~scenario, ~shift_bp,
  "10bp低下", -10,
  "基準",      0,
  "10bp上昇",  10
) |>
dplyr::mutate(
  quotes = purrr::map(
    shift_bp,
    function(shift_bp) {
      case_eonia_quotes |>
        dplyr::mutate(
          rate = rate + shift_bp / 10000,
          rate_pct = 100 * rate
        )
    }
  )
)
```

10 ベーシスポイントは、小数表示の金利では 0.001、パーセント表示では 0.1 パーセントポイントに相当する。

市場レートの変化を確認するため、翌日物預金、1 か月 OIS、5 年 OIS および 30 年 OIS を取り出す。

```
case_rate_scenario_quote_tbl <- purrr::map2_dfr(
  case_rate_scenario_tbl$scenario,
  case_rate_scenario_tbl$quotes,
  function(scenario, quotes) {
    quotes |>
      dplyr::filter(
        quote_id %in% c(
          "DEP-ON-OD",
          "OIS-1M",

```

```

      "OIS-5Y",
      "OIS-30Y"
    )
  ) |>
  dplyr::transmute(
    scenario = scenario,
    quote_id = quote_id,
    instrument_type = instrument_type,
    rate_pct = rate_pct,
    rate = rate
  )
}
)

GaussQuant::show_tbl_GQH(
  case_rate_scenario_quote_tbl,
  "EONIA market-rate scenarios",
  n = 20
)

#> # A tibble: 12 x 5
#>   scenario quote_id instrument_type rate_pct    rate
#>   <chr>    <chr>    <chr>          <dbl>  <dbl>
#> 1 10bp低下 DEP-ON-OD deposit        -0.06 -0.0006
#> 2 10bp低下 OIS-1M   spot_ois        -0.026 -0.00026
#> 3 10bp低下 OIS-5Y   spot_ois         0.356 0.00356
#> 4 10bp低下 OIS-30Y  spot_ois         1.938 0.01938
#> 5 基準     DEP-ON-OD deposit         0.04 0.0004
#> 6 基準     OIS-1M   spot_ois         0.074 0.00074
#> 7 基準     OIS-5Y   spot_ois         0.456 0.00456
#> 8 基準     OIS-30Y  spot_ois         2.038 0.02038
#> 9 10bp上昇 DEP-ON-OD deposit         0.14 0.0014
#> 10 10bp上昇 OIS-1M   spot_ois         0.174 0.00174
#> 11 10bp上昇 OIS-5Y   spot_ois         0.556 0.00556
#> 12 10bp上昇 OIS-30Y  spot_ois         2.138 0.02138

```

各商品のレートは、基準値から一律に 10 ベーシスポイント低下または上昇している。評価日、Fixing 日数、期間、開始日および終了日などの市場慣行は変更しない。

3.6.2 シナリオ別 EONIA カーブ

各シナリオの市場データから、年末ジャンプを含む EONIA カーブを再構築する。

```

case_rate_scenario_tbl <- case_rate_scenario_tbl |>
  dplyr::mutate(

```

```

benchmark = purrr::map(
  quotes,
  function(quotes) {
    GaussQuant::eonia_curve_benchmark_GQL(
      quotes = quotes,
      evaluation_date = case_evaluation_date
    )
  }
),
curve = purrr::map(
  benchmark,
  ~ .x$final$curve
),
curve_handle = purrr::map(
  benchmark,
  ~ .x$final$curve_handle
),
max_quote_error = purrr::map_dbl(
  benchmark,
  function(benchmark) {
    max(
      abs(
        benchmark$final_validation$
          quote_repricing$
          quote_error
      ),
      na.rm = TRUE
    )
  }
)

case_curve_validation_tbl <- case_rate_scenario_tbl |>
  dplyr::select(
    scenario,
    shift_bp,
    max_quote_error
  )

GaussQuant::show_tbl_GQH(
  case_curve_validation_tbl,
  "EONIA scenario-curve validation",
  n = 20

```

```

)
#> # A tibble: 3 x 3
#>   scenario shift_bp max_quote_error
#>   <chr>      <dbl>      <dbl>
#> 1 10bp低下    -10      9.8407e-12
#> 2 基準         0      9.8573e-12
#> 3 10bp上昇     10      9.8710e-12

stopifnot(
  all(
    case_curve_validation_tbl$max_quote_error < 1e-8
  )
)

```

すべてのシナリオについて、Rate Helper のインプライド・レートが入力した市場レートと一致することを確認する。これにより、評価損益の比較に使用する各カーブが、それぞれの市場データを再現していることが分かる。

3.6.3 同一 OIS の再評価

第 9 節と同じ 5 年 OIS を、各シナリオの EONIA カーブへ接続する。

契約期間は 5 年、契約固定金利は 0.456% のままとし、市場カーブだけを変更する。

```

case_rate_scenario_tbl <- case_rate_scenario_tbl |>
  dplyr::mutate(
    eonia_index = purrr::map(
      curve_handle,
      QuantLib::Eonia
    ),
    ois = purrr::map(
      eonia_index,
      function(eonia_index) {
        GaussQuant::build_ois_GQL(
          swap_tenor = case_ois_tenor,
          overnight_index = eonia_index,
          fixed_rate = case_ois_fixed_rate
        )
      }
    ),
    valuation = purrr::map(
      ois,
      GaussQuant::swap_summary_GQL
    )
  )

```

各シナリオの評価結果を一つの表へまとめる。

```
case_ois_scenario_result_tbl <- case_rate_scenario_tbl |>
  dplyr::select(
    scenario,
    shift_bp,
    valuation
  ) |>
  tidyr::unnest(
    cols = valuation
  )

case_base_fair_rate <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "基準"
  ) |>
  dplyr::pull(
    fair_rate
  )

case_base_npv <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "基準"
  ) |>
  dplyr::pull(
    npv
  )

case_ois_scenario_result_tbl <- case_ois_scenario_result_tbl |>
  dplyr::mutate(
    contract_fixed_rate = case_ois_fixed_rate,
    fair_rate_change_bp = 10000 * (
      fair_rate -
      case_base_fair_rate
    ),
    payer_valuation_change =
      npv -
      case_base_npv,
    receiver_npv = -npv,
    receiver_valuation_change =
      -(
        npv -
        case_base_npv
      )
  )
```

```

) |>
dplyr::select(
  scenario,
  shift_bp,
  contract_fixed_rate,
  fair_rate,
  fair_rate_change_bp,
  payer_npv = npv,
  payer_valuation_change,
  receiver_npv,
  receiver_valuation_change
)

GaussQuant::show_tbl_GQH(
  case_ois_scenario_result_tbl,
  "Five-year EONIA OIS scenario valuation",
  n = 20
)

#> # A tibble: 3 x 9
#>   scenario shift_bp contract_fixed_rate fair_rate fair_rate_change_bp
#>   <chr>      <dbl>          <dbl>      <dbl>          <dbl>
#> 1 10bp低下    -10            0.00456  0.0035600      -10.000
#> 2 基準         0             0.00456  0.0045600         0
#> 3 10bp上昇     10            0.00456  0.0055600      10.0000
#> # i 4 more variables: payer_npv <dbl>, payer_valuation_change <dbl>,
#> #   receiver_npv <dbl>, receiver_valuation_change <dbl>

```

`payer_npv` は固定金利を支払うポジションの NPV であり、`receiver_npv` は固定金利を受け取る反対ポジションの NPV である。

`payer_valuation_change` と `receiver_valuation_change` は、それぞれのシナリオの NPV から基準シナリオの NPV を差し引いた評価損益である。

3.6.4 基準シナリオの照合

基準シナリオは、第 9 節で使用した市場データと約定条件をそのまま使用している。そのため、基準シナリオの NPV とフェアレートは、第 9 節の計算結果と一致する。

```

case_base_scenario_tbl <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "基準"
  )

case_base_check_tbl <- tibble::tibble(
  metric = c(

```

```

    "NPV",
    "Fair rate"
  ),
  chapter9_value = c(
    case_ois_gaussquant_summary$npv[[1L]],
    case_ois_gaussquant_summary$fair_rate[[1L]]
  ),
  chapter10_value = c(
    case_base_scenario_tbl$payer_npv[[1L]],
    case_base_scenario_tbl$fair_rate[[1L]]
  )
) |>
dplyr::mutate(
  difference =
    chapter10_value -
    chapter9_value
)

GaussQuant::show_tbl_GQH(
  case_base_check_tbl,
  "Base-scenario consistency check",
  n = 20
)

#> # A tibble: 2 x 4
#>   metric      chapter9_value chapter10_value difference
#>   <chr>          <dbl>          <dbl>          <dbl>
#> 1 NPV          -1.6546e-13      -1.6546e-13          0
#> 2 Fair rate     4.5600e- 3       4.5600e- 3          0

stopifnot(
  isTRUE(
    all.equal(
      case_base_check_tbl$chapter9_value,
      case_base_check_tbl$chapter10_value,
      tolerance = 1e-12
    )
  )
)

```

この照合により、第 9 節の基準評価と、第 10 節の基準シナリオが同じ市場データ、評価日および約定条件に基づいていることを確認できる。

3.6.5 市場レート変動と評価損益

市場レートが低下すると、5 年 OIS のフェアレートも低下する。この場合、固定金利支払側は、低下後の市場水準より高い 0.456% を支払うため、既存契約の価値は低下する。

一方、市場レートが上昇すると、固定金利支払側は市場水準より低い 0.456% を支払う契約を保有することになるため、既存契約の価値は上昇する。

固定金利受取側では、この関係が逆になる。固定金利支払側の利益は固定金利受取側の損失となり、両者の NPV と評価損益は符号が反対になる。

3.6.6 1 ベーシスポイント当たりの評価変化

上下 10 ベーシスポイントの結果から、固定金利支払側について、1 ベーシスポイント当たりの平均的な NPV 変化を求める。

```
case_npv_down <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "10bp低下"
  ) |>
  dplyr::pull(
    payer_npv
  )

case_npv_up <- case_ois_scenario_result_tbl |>
  dplyr::filter(
    scenario == "10bp上昇"
  ) |>
  dplyr::pull(
    payer_npv
  )

case_ois_bpv_tbl <- tibble::tibble(
  position = "Fixed-rate payer",
  npv_10bp_down = case_npv_down,
  npv_base = case_base_npv,
  npv_10bp_up = case_npv_up,
  average_npv_change_per_bp =
    (
      case_npv_up -
      case_npv_down
    ) / 20
)
```



```
GaussQuant::show_tbl_GQH(
  case_ois_bpv_tbl,
  "Five-year EONIA OIS rate sensitivity",
  n = 20
)
#> # A tibble: 1 x 5
#>   position      npv_10bp_down  npv_base npv_10bp_up average_npv_change_per~1
#>   <chr>          <dbl>          <dbl>      <dbl>          <dbl>
#> 1 Fixed-rate payer    -0.0050484 -1.6546e-13  0.0050178          0.00050331
#> # i abbreviated name: 1: average_npv_change_per_bp
```

average_npv_change_per_bp は、マイナス 10 ベーシスポイントからプラス 10 ベーシスポイントまでの NPV 変化を 20 で除した値である。

本ケーススタディでは名目元本を 1 として OIS を構築しているため、この値も単位元本当たりの評価変化を表す。実際の取引では、これに取引元本を乗じることによって、ポジション全体の金利感応度を求めることができる。

3.7 まとめ

本章では、世界金融危機と IBOR 改革を背景として、金利デリバティブの評価がシングルカーブ方式からマルチカーブ方式へ移行した経緯を確認した。LIBOR などの IBOR には銀行の信用リスクや資金流動性リスクが含まれるため、担保付取引の割引金利としてそのまま使用することは適切ではない。そこで、担保契約に対応する OIS カーブを割引に用い、参照金利ごとのフォワードカーブから将来金利を予測する評価方法が定着した。

OIS では、固定金利と、計算期間中の日々の翌日物金利を複利で積み上げた変動金利を交換する。評価日以前の翌日物金利には確定した Fixing を使用し、評価日以後の金利には期間構造から求めた予測値を使用する。TONA と SOFR はいずれも翌日物金利であるが、その基礎となる市場や担保性は異なるため、対象通貨の市場慣行を正しく設定する必要がある。

マルチカーブ評価では、将来の変動金利を求めるフォワードカーブと、将来キャッシュフローを現在価値へ変換するディスカウントカーブを分離する。同じ将来キャッシュフローであっても、使用する割引カーブが異なれば NPV やフェアレートも変化する。したがって、金利スワップの評価では、参照インデックスがどのカーブへ接続され、価格計算エンジンがどのカーブを割引に使用しているかを明確にする必要がある。

ケーススタディでは、期間構造ハンドルを翌日物金利インデックスへ接続し、MakeOIS を用いて OIS を構築した。さらに、NPV、フェアレート、固定レグおよび翌日物レグのキャッシュフローを確認し、OIS の契約条件と市場カーブが評価結果へどのように反映されるかを示した。

また、契約固定金利を変更せずに市場カーブを上下へ平行シフトし、OIS の評価損益を比較した。契約締結後も固定金利は変化しないが、市場カーブの変動によって将来の翌日物金利予測と割引係数が変化するため、OIS の NPV は日々変動する。固定金利支払側と固定金利受取側では、同じ市場変動に対する評価損益の符号が反対になる。

実際のリスク管理では、カーブ全体の平行シフトだけでなく、短期、中期および長期の金利を個別に変化させ

る必要がある。また、Fixing の確定、休日カレンダー、日数計算規則、支払ラグ、予測カーブおよび割引カーブの設定を総合的に確認することが、OIS と金利スワップを正しく評価するための基本となる。

参考資料

- Financial Conduct Authority, [The future of LIBOR](#), 27 July 2017.
- Financial Conduct Authority, [The end of LIBOR](#), 1 October 2024.
- Financial Stability Board, [Reforming Major Interest Rate Benchmarks](#), 22 July 2014.
- Bank of Japan, [Uncollateralized Overnight Call Rate](#).

第 II-4 章 株価・為替と幾何ブラウン運動モデル

1973 年に公表された Black-Scholes-Merton モデルは、株式オプションの理論価格を求めるための基本モデルとして、金融工学の発展に大きな影響を与えた。このモデルでは、株価が幾何ブラウン運動に従い、将来の株価が対数正規分布に従うと仮定する。

一方、金利先物、債券先物、商品先物およびフォワード金利を原資産とするオプションでは、現物価格よりも満期に対応する先物価格またはフォワード価格を直接扱う方が自然である。Black は 1976 年、Black-Scholes-Merton モデルの考え方を先物オプションへ適用した。現在、この評価方法は Black-76 モデルまたは単に Black モデルと呼ばれている。

Black-Scholes-Merton モデルでは、現物価格、無リスク金利および配当利回りから将来のフォワード価格を求める。これに対し Black-76 モデルでは、観測されたフォワード価格または先物価格を直接入力し、満期時の Payoff を割り引いて現在価値を求める。

したがって、Black-76 と Black-Scholes-Merton は無関係な別のモデルではない。両者は、原資産価格が正の値を取り、その変化率が正規分布に従うという共通の仮定を持つ。現物価格を出発点とするか、フォワード価格を出発点とするかが異なる。

本章では、まず Black-76 による先物・フォワードオプションの解析評価を行い、オプション価格と Greeks を確認する。次に、同じ確率過程から Monte Carlo パスを生成し、満期 Payoff の割引期待値が解析解へ近づくことを確認する。

その後、Black-Scholes-Merton モデルによる株式オプション評価へ戻り、European オプションの解析解と二項 Tree を比較する。さらに、American オプションの早期行使価値を、近似解、二項 Tree および Longstaff-Schwartz 法によって評価する。

Black モデルは、原資産価格を正の値に限定する対数正規モデルである。この性質は株式や先物価格の評価には都合がよい一方、金利がゼロまたはマイナスとなる市場では制約となる。次章では、この制約を持たない Normal モデルを取り上げ、Black モデルとの違いを確認する。

4.1 Black-Scholes-Merton と Black-76

4.1.1 Black-Scholes-Merton モデル

リスク中立測度の下で、株価 S_t は次の確率微分方程式に従うと仮定する。

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t$$

ここで、 r は無リスク金利、 q は配当利回り、 σ はボラティリティ、 W_t はブラウン運動である。

満期 T のフォワード価格は次式となる。

$$F_0 = S_0 e^{(r-q)T}$$

したがって、Black-Scholes-Merton モデルは現物価格を出発点として、金利と配当を通じてフォワード価格を内生的に求めるモデルとみることができる。

4.1.2 Black-76 モデル

Black-76 では、フォワード価格 F_0 を直接入力する。European call の価格は次式で与えられる。

$$C = D(0, T) [F_0 N(d_1) - K N(d_2)]$$

European put の価格は次式となる。

$$P = D(0, T) [K N(-d_2) - F_0 N(-d_1)]$$

ここで、

$$d_1 = \frac{\log(F_0/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

であり、 K は行使価格、 $D(0, T)$ は満期までの割引係数、 $N(\cdot)$ は標準正規分布の累積分布関数である。

金利が決定論的である場合、同一満期の先物価格とフォワード価格は一致するため、Black-76 は先物オプションとフォワードオプションの双方に用いられる。

4.1.3 Black-76 の入力条件

本節では、債券先物オプションを意識した価格表示を使用する。フォワード価格は 107 と 32 分の 6.75、行使価格は 107.5、ボラティリティは 5.2%、無リスク金利は 5% とする。

Black-76 では、現物価格ではなく、オプション満期に対応するフォワード価格または先物価格を原資産として使用する。

```

black_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2023-07-21",
  "maturity_date", "2023-08-26",
  "forward_price", as.character(107 + 6.75 / 32),
  "strike_price",  as.character(107.5),
  "volatility",    as.character(0.052),
  "risk_free_rate", as.character(0.05)
)

trade_date <- GaussQuant::date_GQL("2023-07-21")
maturity_date <- GaussQuant::date_GQL("2023-08-26")
forward_price <- 107 + 6.75 / 32
strike_price <- 107.5
volatility <- 0.052
risk_free_rate <- 0.05
day_counter <- QuantLib::Actual365Fixed()

call_payoff <- QuantLib::PlainVanillaPayoff(
  QuantLib::Option_Call_get(),
  strike_price
)

GaussQuant::set_eval_date_GQL(trade_date)

maturity <- day_counter$yearFraction(
  trade_date,
  maturity_date
)

GaussQuant::show_tbl_GQH(
  black_input_tbl,
  "Black-76 input assumptions",
  n = 20
)

#> # A tibble: 6 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2023-07-21
#> 2 maturity_date 2023-08-26
#> 3 forward_price 107.2109375
#> 4 strike_price 107.5
#> 5 volatility 0.052

```

```
#> 6 risk_free_rate 0.05
```

4.2 Black-76 による解析評価

4.2.1 Black 公式による価格と Greeks

BlackCalculator は、フォワード価格、行使価格、総標準偏差および割引係数から、オプション価格と Greeks を直接計算する。

QuantLib へ渡される総標準偏差は、単なる年率ボラティリティではなく、

$$\sigma\sqrt{T}$$

である。

本節では、次の Greeks を確認する。

- **Delta**：フォワード価格の変化に対するオプション価格の感応度
- **Gamma**：フォワード価格の変化に対する Delta の感応度
- **Vega**：ボラティリティの変化に対するオプション価格の感応度
- **Theta**：時間経過に対するオプション価格の感応度

Black-76 の Delta はフォワード価格に対する Delta であり、株式オプションの Spot Delta とは区別する必要がある。

```
black_bundle <- GaussQuant::black_process_bundle_GQL(
  valuation_date = trade_date,
  forward = forward_price,
  risk_free_rate = risk_free_rate,
  volatility = volatility,
  day_counter = day_counter
)

discount_factor <- black_bundle$discount_curve$discount(
  maturity_date
)

black_calculator_tbl <- GaussQuant::black_calculator_table_GQL(
  payoff = call_payoff,
  forward = forward_price,
  volatility = volatility,
  maturity = maturity,
  discount_factor = discount_factor
)

GaussQuant::show_tbl_GQH(
```

```
black_calculator_tbl,
  "Black calculator summary",
  n = 20
)
#> # A tibble: 6 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        0.56160
#> 2 delta      0.43558
#> 3 gamma      0.22397
#> 4 vega       13.203
#> 5 theta     -3.4524
#> 6 theta_per_day -0.0094587
```

4.2.2 Black 過程による European option 評価

前節では Black 公式を直接使用した。本節では、同じ入力条件から Black process を構築し、European option へ解析的 Pricing Engine を設定する。

数式を直接評価する方法と、確率過程および Pricing Engine を通じて評価する方法が、同一の理論価格を与えることを確認する。

```
european_call <- QuantLib::VanillaOption(
  call_payoff,
  QuantLib::EuropeanExercise(maturity_date)
)

analytic_engine <- QuantLib::AnalyticEuropeanEngine(
  black_bundle$process
)

european_call$setPricingEngine(
  analytic_engine
)
#> NULL

analytic_european_tbl <- GaussQuant::option_metric_table_GQL(
  option = european_call,
  process = black_bundle$process
)

analytic_npv <- analytic_european_tbl |>
  dplyr::filter(.data$metric == "npv") |>
  dplyr::pull(.data$value)
```

```
GaussQuant::show_tbl_GQH(
  analytic_european_tbl,
  "Analytic European call",
  n = 20
)
#> # A tibble: 7 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        0.56160
#> 2 delta       0.43558
#> 3 gamma       0.22397
#> 4 vega       13.203
#> 5 theta      -3.4524
#> 6 theta_per_day -0.0094587
#> 7 implied_volatility 0.051993
```

4.3 Black–Scholes–Merton と European option

4.3.1 Black–Scholes–Merton モデルの入力条件

ここからは現物株価を原資産とする Black–Scholes–Merton モデルを扱う。現物価格 100、行使価格 100、満期 1 年、ボラティリティ 20%、無リスク金利 5%、配当利回り 0% とする。

Black–Scholes–Merton process では、現物価格に加えて、無リスク金利カーブ、配当利回りカーブおよびボラティリティカーブを設定する。

```
bsm_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2024-03-01",
  "maturity",   "1Y",
  "spot_price", as.character(100),
  "strike_price", as.character(100),
  "volatility",  as.character(0.20),
  "risk_free_rate", as.character(0.05),
  "dividend_rate", as.character(0)
)

trade_date_bsm <- GaussQuant::date_GQL("2024-03-01")
calendar_bsm <- QuantLib::TARGET()

maturity_date_bsm <- calendar_bsm$advance(
  trade_date_bsm,
  1L,
  "Years"
```



```

)

spot_price_bsm <- 100
strike_price_bsm <- 100
volatility_bsm <- 0.20
risk_free_rate_bsm <- 0.05
dividend_rate_bsm <- 0

GaussQuant::set_eval_date_GQL(
  trade_date_bsm
)

bsm_bundle <- GaussQuant::bsm_process_bundle_GQL(
  valuation_date = trade_date_bsm,
  spot = spot_price_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_rate = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm
)

GaussQuant::show_tbl_GQH(
  bsm_input_tbl,
  "Black-Scholes-Merton assumptions",
  n = 20
)

#> # A tibble: 7 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2024-03-01
#> 2 maturity   1Y
#> 3 spot_price 100
#> 4 strike_price 100
#> 5 volatility  0.2
#> 6 risk_free_rate 0.05
#> 7 dividend_rate 0

```

4.3.2 European put の解析解と二項 Tree

European オプションは満期時にのみ行使できるため、Black-Scholes-Merton モデルの解析解を利用できる。

二項 Tree では、連続時間の価格過程を有限個の時間ステップへ分割する。ステップ数を増やすほど、理論上は連続時間モデルの解析値へ近づく。

ここでは、25、100 および 500 ステップの Cox–Ross–Rubinstein Tree を解析解と比較する。

```

european_put <- GaussQuant::make_european_option_GQL(
  spot = spot_price_bsm,
  strike = strike_price_bsm,
  maturity_date = maturity_date_bsm,
  option_type = "put",
  valuation_date = trade_date_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_yield = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm
)

european_engines <- list(
  crr_25 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    25L
  ),
  analytic = QuantLib::AnalyticEuropeanEngine(
    bsm_bundle$process
  ),
  crr_100 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    100L
  ),
  crr_500 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    500L
  )
)

european_comparison_tbl <- purrr::imap_dfr(
  european_engines,
  function(engine, method) {
    european_put$setPricingEngine(engine)

    tibble::tibble(
      method = method,
      npv = european_put$NPV()
    )
  }
)

```

```

analytic_put_npv <- european_comparison_tbl |>
  dplyr::filter(.data$method == "analytic") |>
  dplyr::pull(.data$npv)

european_comparison_tbl <- european_comparison_tbl |>
  dplyr::mutate(
    difference_from_analytic = .data$npv - analytic_put_npv
  )

GaussQuant::show_tbl_GQH(
  european_comparison_tbl,
  "European put engine comparison",
  n = 20
)

#> # A tibble: 4 x 3
#>   method      npv difference_from_analytic
#>   <chr>      <dbl>                <dbl>
#> 1 crr_25    5.6546                0.071974
#> 2 analytic 5.5826                0
#> 3 crr_100   5.5629               -0.019666
#> 4 crr_500   5.5786               -0.0039372

```

4.4 Monte Carlo 評価

4.4.1 擬似乱数 Monte Carlo 評価

リスク中立評価では、オプション価格は満期 Payoff の割引期待値として表される。

$$V_0 = D(0, T)E^Q[\max(F_T - K, 0)]$$

Monte Carlo 法では、多数の将来パスを生成し、各パスの満期 Payoff を求め、その平均値を割り引く。

European call には解析解があるため、ここで Monte Carlo 法を使う目的は、解析解に代わる高度な方法を示すことではない。確率過程から生成したパスの期待値が Black 公式の価格へ近づくことを確認し、後に解析解を持たない商品へ応用する準備をすることにある。

```

mc_setting_tbl <- tibble::tribble(
  ~setting, ~value,
  "steps", 12,
  "paths", 20000,
  "seed", 1
)

mc_path_tbl <- GaussQuant::generate_path_table_GQL(

```

```

process = black_bundle$process,
maturity = maturity,
n_steps = 12L,
n_paths = 200L,
seed = 1L,
sequence = "pseudo"
)

mc_result <- GaussQuant::terminal_option_mc_GQL(
  path_tbl = mc_path_tbl,
  strike = strike_price,
  discount_factor = discount_factor,
  option_type = "call"
)

mc_comparison_tbl <- tibble::tribble(
  ~method, ~npv, ~standard_error,
  "Black analytic", analytic_npv, NA_real_,
  "Pseudo-random Monte Carlo", mc_result$summary$npv[[1]], mc_result$summary$standard_error[[1]]
) |>
  dplyr::mutate(
    difference_from_analytic = .data$npv - analytic_npv
  )

GaussQuant::show_tbl_GQH(
  mc_setting_tbl,
  "Monte Carlo settings",
  n = 20
)

#> # A tibble: 3 x 2
#>   setting value
#>   <chr>    <dbl>
#> 1 steps      12
#> 2 paths    20000
#> 3 seed        1

GaussQuant::show_tbl_GQH(
  mc_comparison_tbl,
  "Analytic and Monte Carlo comparison",
  n = 20
)

#> # A tibble: 2 x 4
#>   method npv standard_error difference_from_analytic

```

#>	<chr>	<dbl>	<dbl>	<dbl>
#> 1	Black analytic	0.56160	NA	0
#> 2	Pseudo-random Monte Carlo	0.47698	0.059721	-0.084615

標準誤差は、有限本数のパスを用いたことによる Monte Carlo 推定値の不確実性を表す。解析値との差が標準誤差と同程度であれば、シミュレーション結果は統計的に整合的と判断できる。

4.4.2 Monte Carlo パスの可視化

Black process では、フォワード価格は対数正規分布に従う。したがって、各パスの価格は常に正であり、満期価格の分布は左右対称にはならない。

次のグラフでは、前節と同じオプション満期までの 120 本のパスと、その満期価格分布を表示する。

```
path_plot_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = 12L,
  n_paths = 120L,
  seed = 1L,
  sequence = "pseudo"
)

path_plot <- ggplot2::ggplot(
  path_plot_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$price,
    group = .data$path_id
  )
) +
  ggplot2::geom_line(
    alpha = 0.45,
    linewidth = 0.4
  ) +
  ggplot2::labs(
    title = "Black model Monte Carlo paths",
    x = "Year",
    y = "Forward price"
  )

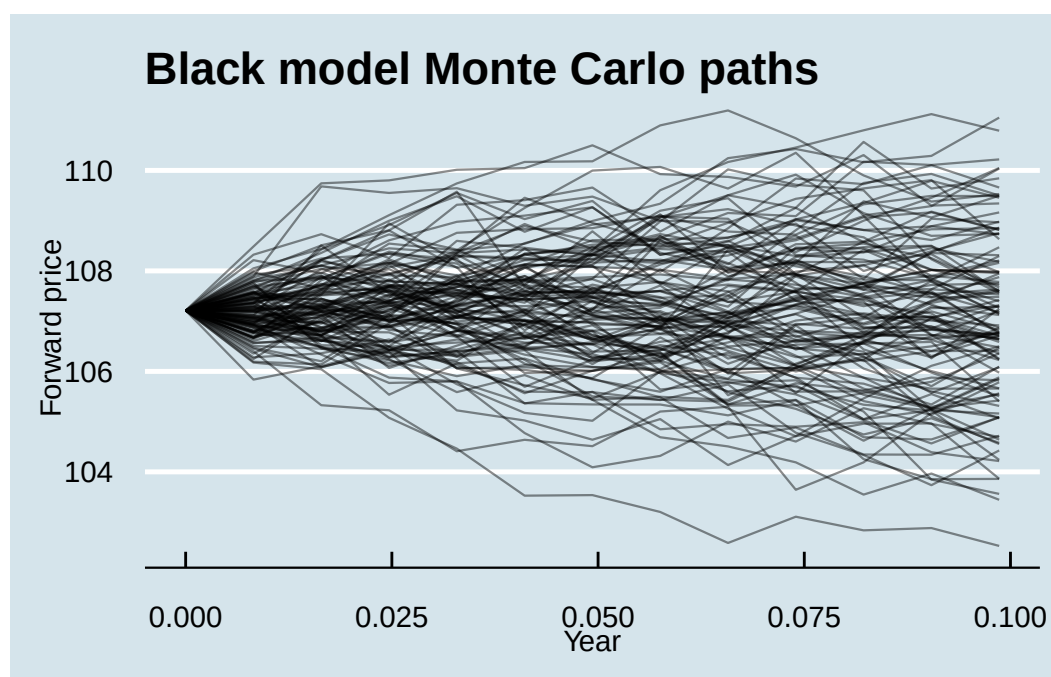
terminal_plot_tbl <- path_plot_tbl |>
  dplyr::group_by(.data$path_id) |>
  dplyr::slice_max(
    order_by = .data$step,
```

```

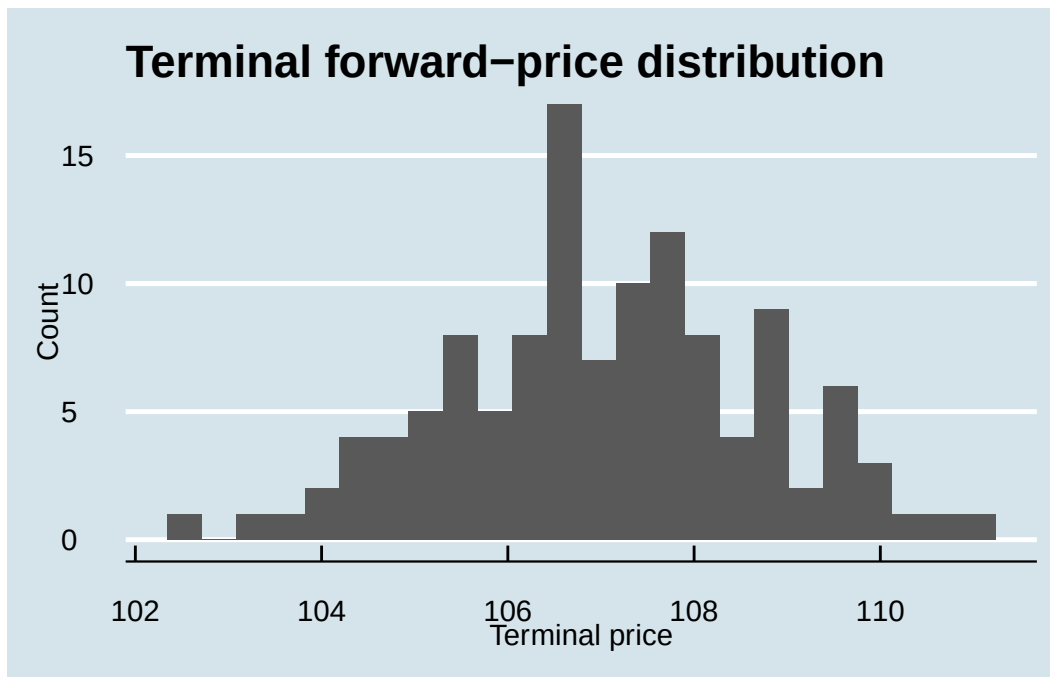
    n = 1L,
    with_ties = FALSE
  ) |>
  dplyr::ungroup()

terminal_plot <- ggplot2::ggplot(
  terminal_plot_tbl,
  ggplot2::aes(x = .data$price)
) +
  ggplot2::geom_histogram(
    bins = 24
  ) +
  ggplot2::labs(
    title = "Terminal forward-price distribution",
    x = "Terminal price",
    y = "Count"
  )
show_plot_LFL(path_plot)

```



```
show_plot_LFL(terminal_plot)
```



4.4.3 Black 過程の 1 パス

Black process のフォワード価格は、時間幅 Δt ごとに次式で更新できる。

$$F_{t+\Delta t} = F_t \exp \left[-\frac{1}{2} \sigma^2 \Delta t + \sigma \sqrt{\Delta t} Z \right]$$

ここで、 Z は標準正規乱数である。フォワード測度の下ではフォワード価格のドリフトがゼロであるため、指数部には Ito 補正項である $-\frac{1}{2}\sigma^2\Delta t$ が現れる。

QuantLib の PathGenerator が使用する Gaussian increment を取り出し、同じ乱数から手計算したパスと比較する。

```
n_steps_check <- 3L
seed_check <- 1L

generated_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = black_bundle$process,
  maturity = maturity,
  n_steps = n_steps_check,
  n_paths = 1L,
  seed = seed_check,
  sequence = "pseudo"
)

gaussian_rng_check <- QuantLib::GaussianRandomSequenceGenerator(
  QuantLib::UniformRandomSequenceGenerator(
    n_steps_check,
```

```

  QuantLib::UniformRandomGenerator(seed_check)
)
)

gaussian_increment <- gaussian_rng_check$nextSequence()$value() |>
  as.numeric()

time_grid_check <- generated_path_tbl |>
  dplyr::arrange(.data$step) |>
  dplyr::pull(.data$time)

dt_check <- diff(time_grid_check)

growth_factor <- exp(
  -0.5 * volatility^2 * dt_check +
  volatility * sqrt(dt_check) * gaussian_increment
)

hand_price <- forward_price * c(
  1,
  cumprod(growth_factor)
)

one_path_check_tbl <- generated_path_tbl |>
  dplyr::arrange(.data$step) |>
  dplyr::transmute(
    step = .data$step,
    time = .data$time,
    generated_price = .data$price,
    hand_price = hand_price,
    difference = .data$price - hand_price
  )

GaussQuant::show_tbl_GQH(
  one_path_check_tbl,
  "One-path hand calculation",
  n = 20
)

#> # A tibble: 4 x 5
#>   step    time generated_price hand_price difference
#>   <int>  <dbl>          <dbl>      <dbl>      <dbl>
#> 1     0  0             107.21      107.21      0
#> 2     1 0.032877        106.99      106.99      0

```



```
#> 3      2 0.065753      109.82      109.82  0
#> 4      3 0.098630      110.42      110.42 -1.4211e-14
```

差が数値誤差の範囲に収まれば、PathGenerator が Black 過程の離散化式に従っていることを確認できる。

4.5 American put の早期行使価値

American オプションは満期までの任意の時点で行使できる。したがって、その価値は同じ条件の European オプション以上となる。

$$V_{\text{American}} \geq V_{\text{European}}$$

両者の差は早期行使権の価値である。

American put には一般的な閉形式の解析解がないため、二項 Tree または近似法を用いる。本節では、次の方法を比較する。

- 25 ステップ CRR Tree
- Barone-Adesi-Whaley 近似
- Bjerkstrand-Stensland 近似
- 500 ステップ CRR Tree

500 ステップ CRR Tree を比較基準とする。

```
american_put <- GaussQuant::make_american_option_GQL(
  spot = spot_price_bsm,
  strike = strike_price_bsm,
  maturity_date = maturity_date_bsm,
  option_type = "put",
  valuation_date = trade_date_bsm,
  risk_free_rate = risk_free_rate_bsm,
  dividend_yield = dividend_rate_bsm,
  volatility = volatility_bsm,
  day_counter = day_counter,
  calendar = calendar_bsm,
  steps = 500L
)

american_engines <- list(
  crr_25 = QuantLib::BinomialCRRVanillaEngine(
    bsm_bundle$process,
    25L
  ),
  barone_adesi_whaley = QuantLib::BaroneAdesiWhaleyApproximationEngine(
    bsm_bundle$process
```

```

),
bjerksund_stensland = QuantLib::BjerksundStenslandApproximationEngine(
  bsm_bundle$process
),
crr_500 = QuantLib::BinomialCRRVanillaEngine(
  bsm_bundle$process,
  500L
)
)

```

```

american_comparison_tbl <- purrr::imap_dfr(
  american_engines,
  function(engine, method) {
    american_put$setPricingEngine(engine)

    tibble::tibble(
      method = method,
      npv = american_put$NPV()
    )
  }
)

```

```

crr_500_american_npv <- american_comparison_tbl |>
  dplyr::filter(.data$method == "crr_500") |>
  dplyr::pull(.data$npv)

```

```

american_comparison_tbl <- american_comparison_tbl |>
  dplyr::mutate(
    difference_from_crr_500 = .data$npv - crr_500_american_npv
  )

```

```

GaussQuant::show_tbl_GQH(
  american_comparison_tbl,
  "American put engine comparison",
  n = 20
)

```

```

#> # A tibble: 4 x 3
#>   method          npv difference_from_crr_500
#>   <chr>          <dbl>          <dbl>
#> 1 crr_25        6.1595          0.058390
#> 2 barone_adesi_whaley 6.1100          0.0089221
#> 3 bjerksund_stensland 5.9951         -0.10604
#> 4 crr_500        6.1011           0

```

4.6 準 Monte Carlo と Longstaff-Schwartz 法

4.6.1 Sobol 系列によるパス生成

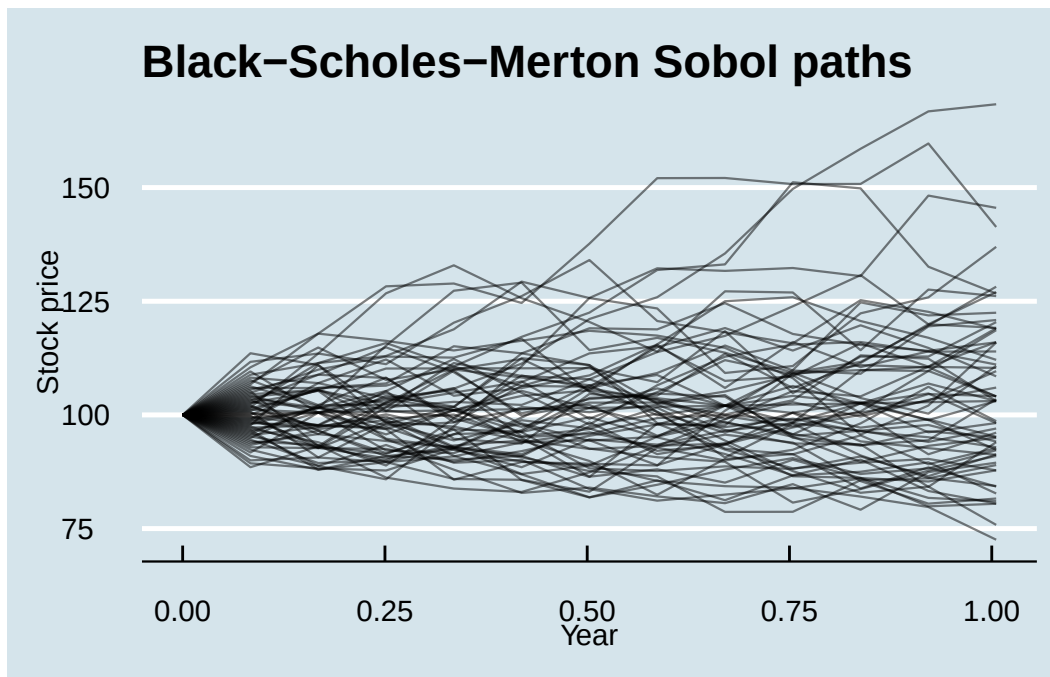
擬似乱数は乱数生成器によって確率的な点列を作る。これに対し Sobol 系列は、積分領域をできるだけ均等に覆うように構成された低乖離列である。

Sobol 系列を用いる準 Monte Carlo 法では、通常の擬似乱数より少ないパスで安定した数値積分が得られる場合がある。ただし、パス同士は独立な確率標本ではないため、通常の標準誤差の解釈をそのまま適用できない点に注意する。

次のコードでは、Black-Scholes-Merton process から 64 本の Sobol パスを生成する。

```
maturity_bsm <- day_counter$yearFraction(  
  trade_date_bsm,  
  maturity_date_bsm  
)  
  
sobol_display_tbl <- GaussQuant::generate_path_table_GQL(  
  process = bsm_bundle$process,  
  maturity = maturity_bsm,  
  n_steps = 12L,  
  n_paths = 64L,  
  seed = 1L,  
  sequence = "sobol"  
)  
  
sobol_plot <- ggplot2::ggplot(  
  sobol_display_tbl,  
  ggplot2::aes(  
    x = .data$time,  
    y = .data$price,  
    group = .data$path_id  
  )  
) +  
  ggplot2::geom_line(  
    alpha = 0.5,  
    linewidth = 0.4  
  ) +  
  ggplot2::labs(  
    title = "Black-Scholes-Merton Sobol paths",  
    x = "Year",  
    y = "Stock price"  
  )
```

```
GaussQuant::show_tbl_GQH(
  sobol_display_tbl,
  "Sobol path table",
  n = 20
)
#> # A tibble: 20 x 4
#>   path_id step    time price
#>   <int> <int>   <dbl> <dbl>
#> 1     1     1     0 0      100
#> 2     1     1     1 0.083790 100.25
#> 3     1     2     0.16758 100.50
#> 4     1     3     0.25137 100.76
#> 5     1     4     0.33516 101.01
#> 6     1     5     0.41895 101.26
#> 7     1     6     0.50274 101.52
#> 8     1     7     0.58653 101.78
#> 9     1     8     0.67032 102.03
#> 10    1     9     0.75411 102.29
#> 11    1    10     0.83790 102.55
#> 12    1    11     0.92169 102.80
#> 13    1    12     1.0055   103.06
#> 14     2     0     0      100
#> 15     2     1     0.083790 104.24
#> 16     2     2     0.16758 100.50
#> 17     2     3     0.25137 104.77
#> 18     2     4     0.33516 101.01
#> 19     2     5     0.41895 105.30
#> 20     2     6     0.50274 101.52
show_plot_LFL(sobol_plot)
```



4.6.2 Longstaff-Schwartz 法

American オプションでは、各行使可能時点において、即時行使価値と保有を継続する価値を比較する必要がある。

put の即時行使価値は次式である。

$$I(S_t) = \max(K - S_t, 0)$$

Longstaff-Schwartz 法では、将来受け取る割引 Cashflow を原資産価格の関数として回帰し、条件付き継続価値を推定する。本章では、次の 2 次多項式を使用する。

$$\widehat{C}(S_t) = \beta_0 + \beta_1 S_t + \beta_2 S_t^2$$

即時行使価値が推定継続価値を上回る場合、その時点で行使する。

本節では、2,048 本の Sobol パスから American put を評価し、500 ステップ CRR Tree と比較する。

```
lsm_path_tbl <- GaussQuant::generate_path_table_GQL(
  process = bsm_bundle$process,
  maturity = maturity_bsm,
  n_steps = 12L,
  n_paths = 2048L,
  seed = 1L,
  sequence = "sobol"
)

lsm_result <- GaussQuant::lsm_american_put_GQL(
```

```

path_tbl = lsm_path_tbl,
strike = strike_price_bsm,
risk_free_rate = risk_free_rate_bsm
)

lsm_npv <- lsm_result$summary |>
  dplyr::filter(.data$metric == "lsm_npv") |>
  dplyr::pull(.data$value)

lsm_comparison_tbl <- tibble::tribble(
  ~method, ~npv,
  "Longstaff-Schwartz Sobol", lsm_npv,
  "CRR Tree 500 steps", crr_500_american_npv
) |>
  dplyr::mutate(
    difference_from_crr_500 = .data$npv - crr_500_american_npv
  )

exercise_summary_tbl <- lsm_result$exercise |>
  dplyr::count(
    .data$exercise_step,
    .data$exercise_time,
    name = "paths"
  ) |>
  dplyr::arrange(.data$exercise_step)

GaussQuant::show_tbl_GQH(
  lsm_result$summary,
  "Longstaff-Schwartz summary",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 lsm_npv      6.2925
#> 2 standard_error 0.16653
#> 3 paths      2048
#> 4 steps       12

GaussQuant::show_tbl_GQH(
  lsm_comparison_tbl,
  "Longstaff-Schwartz and CRR comparison",
  n = 20
)

```

```

)
#> # A tibble: 2 x 3
#>   method                                npv difference_from_crr_500
#>   <chr>                                <dbl>          <dbl>
#> 1 Longstaff-Schwartz Sobol 6.2925          0.19139
#> 2 CRR Tree 500 steps      6.1011          0

GaussQuant::show_tbl_GQH(
  exercise_summary_tbl,
  "Longstaff-Schwartz exercise timing",
  n = 20
)
#> # A tibble: 11 x 3
#>   exercise_step exercise_time paths
#>   <int>          <dbl> <int>
#> 1         2      0.16758    28
#> 2         3      0.25137    52
#> 3         4      0.33516    57
#> 4         5      0.41895   109
#> 5         6      0.50274   107
#> 6         7      0.58653   100
#> 7         8      0.67032    87
#> 8         9      0.75411    38
#> 9        10      0.83790    78
#> 10        11      0.92169    96
#> 11        12      1.0055   1296

GaussQuant::show_tbl_GQH(
  lsm_result$diagnostics,
  "Longstaff-Schwartz diagnostics",
  n = 20
)
#> # A tibble: 11 x 6
#>   step      time in_the_money_paths exercised_paths mean_immediate_value
#>   <int>    <dbl>          <int>          <int>          <dbl>
#> 1    11 0.92169           931            575          5.6825
#> 2    10 0.83790           918            459          5.5258
#> 3     9 0.75411           932            355          5.2815
#> 4     8 0.67032           937            392          5.0842
#> 5     7 0.58653           945            338          4.8180
#> 6     6 0.50274           946            282          4.5454
#> 7     5 0.41895           939            193          4.2509
#> 8     4 0.33516           959            105          3.8461

```

```
#> 9      3 0.25137      970      69      3.4077
#> 10     2 0.16758      963      28      2.8755
#> 11     1 0.083790     989      0      2.1136
#> # i 1 more variable: mean_realized_continuation <dbl>
```

Longstaff-Schwartz 法の価格は、パス数、時間ステップ、基底関数および回帰誤差に依存する。したがって、二項 Tree との差が完全にゼロになることを目的とするのではなく、早期行使判断を Monte Carlo パス上でのように近似するかを確認する。

4.7 まとめ

Black-76 と Black-Scholes-Merton は、原資産が対数正規分布に従うという共通の仮定を持つ。Black-Scholes-Merton は現物価格から出発し、Black-76 はフォワード価格または先物価格を直接入力する。

Black-76 では、Black 公式による直接計算と、Black process を用いた解析的 Pricing Engine が同じ価格を与えることを確認した。さらに、Monte Carlo 法によって満期 Payoff の割引期待値を求め、解析値と比較した。

Black-Scholes-Merton モデルでは、European put の解析解と CRR Tree を比較した。American put では、早期行使権を評価するため、近似 Engine、CRR Tree および Longstaff-Schwartz 法を比較した。

解析解、Tree、Monte Carlo および回帰法は、互いに競合する単純な代替手法ではない。Payoff の構造、早期行使の有無、モデルの次元および必要な計算精度に応じて使い分ける必要がある。

Black モデルの対数正規仮定は、価格が正である市場では扱いやすい。一方、金利がゼロまたはマイナスとなる場合には直接適用できない。この問題が、Normal モデルや Shifted Black モデルを用いる理由となる。

```
chapter04_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Black-76",        "Black calculator and analytic European pricing",
  "Pseudo-random Monte Carlo", "Discounted terminal payoff and standard error",
  "European put",    "Analytic and CRR Tree comparison",
  "American put",    "Approximation engines and CRR Tree comparison",
  "Sobol",           "Low-discrepancy path generation",
  "Longstaff-Schwartz", "Regression-based early-exercise approximation"
)

GaussQuant::show_tbl_GQH(
  chapter04_summary_tbl,
  "Chapter II-4 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Black-76      Black calculator and analytic European pricing
#> 2 Pseudo-random Monte Carlo Discounted terminal payoff and standard error
```


#> 3 European put	Analytic and CRR Tree comparison
#> 4 American put	Approximation engines and CRR Tree comparison
#> 5 Sobol	Low-discrepancy path generation
#> 6 Longstaff-Schwartz	Regression-based early-exercise approximation

第 II-5 章金利モデル（期間構造なし）Normal モデル

前章では、株価や為替レートのように、価格の変化率をウィーナー過程によって表現する対象について、Black モデルを検討した。Black モデルでは、原資産の変動幅がその時点の価格水準に比例する幾何ブラウン運動を仮定するため、原資産価格は常に正の値を取る。

これに対して、本章では金利を原資産とするオプションを検討する。金利は株価や為替レートとは異なり、満期ごとに異なる期間構造を持ち、ゼロまたはマイナスの値を取る可能性もある。このため、金利を単一の幾何ブラウン運動によって表現することには限界がある。

本章では、金利期間構造全体を確率モデルとして記述するのではなく、特定の将来期間に対応するフォワード金利を直接モデル化する。適切なフォワード測度の下では、フォワード金利をマルチンゲールとして扱うことができる。

Black モデルでは、フォワード金利の変化幅がその金利水準に比例すると仮定する。

$$dF_t = \sigma_B F_t dW_t$$

この場合、フォワード金利は理論上常に正の値を取り、ゼロまたはマイナスの金利を直接扱うことができない。

そこで本章では、フォワード金利の変化額がウィーナー過程に従う Normal モデルを検討する。

$$dF_t = \sigma_N dW_t$$

このモデルでは、将来のフォワード金利は次のように表される。

$$F_T = F_0 + \sigma_N \sqrt{T} Z$$

ここで、 Z は標準正規分布に従う確率変数である。フォワード金利の変化幅は現在の金利水準に依存しないため、将来金利はゼロまたはマイナスの値も取り得る。

本章では、あえて金利期間構造全体の動学モデルを導入せず、各商品の評価に必要なフォワード金利を、それぞれに対応する測度の下でマルチンゲールとして直接モデル化する。この枠組みに基づき、金利先物オプション、Caplet、Cap および Swaption を Normal モデルによって評価する。

5.1 Normal モデルの基礎

5.1.1 金利モデルと測度

金利デリバティブでは、すべてのフォワード金利を一つの確率過程で表現するとは限らない。単一の Caplet を評価する場合、その Caplet が参照する期間のフォワード金利を、支払日を満期とするゼロクーポン債を Numeraire としたフォワード測度の下で扱う。

期間 $[T_1, T_2]$ のフォワード金利を $L_t(T_1, T_2)$ とすると、 T_2 -forward measure の下で、これをマルチンゲールとして次のように置く。

$$dL_t(T_1, T_2) = \sigma_N dW_t^{T_2}$$

一方、Swaption では、原資産は単一期間のフォワード金利ではなく、複数期間から構成されるフォワードスワップレートである。この場合は、固定レグのアニュイティを Numeraire とする swap measure の下で、フォワードスワップレートをマルチンゲールとして扱う。

したがって本章では、期間構造全体を確率的に生成する HJM、LMM または短期金利モデルへ進むのではなく、各商品の自然なフォワード変数に対して局所的な Normal モデルを適用する。

現在価値を計算するためには割引係数が必要であるが、割引カーブ自体の動学はモデル化しない。本章では、与えられた割引係数または簡単な決定論的割引率を使用する。

5.1.2 Bachelier 価格公式

Normal モデルの下で、満期時のフォワード金利は次の正規分布に従う。

$$F_T \sim N(F_0, \sigma_N^2 T)$$

European call の価格は次式で与えられる。

$$C_N = D(0, T) \left[(F_0 - K)N(d) + \sigma_N \sqrt{T} \phi(d) \right]$$

European put の価格は次式となる。

$$P_N = D(0, T) \left[(K - F_0)N(-d) + \sigma_N \sqrt{T} \phi(d) \right]$$

ここで、

$$d = \frac{F_0 - K}{\sigma_N \sqrt{T}}$$

であり、 $D(0, T)$ は割引係数、 $N(\cdot)$ は標準正規分布の累積分布関数、 $\phi(\cdot)$ は標準正規分布の密度関数である。

Black 公式には $\log(F_0/K)$ が含まれるが、Normal 公式では差額 $F_0 - K$ だけを使用する。このため、フォワード金利または行使金利がゼロ以下であっても計算できる。

5.1.3 Normal volatility の単位

Black volatility は原資産価格に対する相対的な変化率であり、通常は百分率で表示される。

これに対して Normal volatility は、原資産と同じ単位を持つ。金利を小数で表す場合、Normal volatility 50bp は次の値である。

$$50 \text{ bp} = \frac{50}{10000} = 0.0050$$

したがって、 $\text{vol} = 0.0050$ は「0.5% の Black volatility」ではなく、**年率 50bp の Normal volatility** を意味する。

Normal モデルでは、

$$F_T - F_0 = \sigma_N \sqrt{T} Z$$

であるため、 $\sigma_N \sqrt{T}$ は満期までの金利変化額の標準偏差となる。

```
normal_volatility_bp <- 50
normal_volatility <- normal_volatility_bp / 10000

normal_volatility_tbl <- tibble::tribble(
  ~metric,          ~value,
  "normal_volatility_bp",      normal_volatility_bp,
  "normal_volatility_decimal",  normal_volatility,
  "one_standard_deviation_1y_bp", normal_volatility * 10000
)

GaussQuant::show_tbl_GQH(
  normal_volatility_tbl,
  "Normal volatility units",
  n = 20
)

#> # A tibble: 3 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 normal_volatility_bp      50
#> 2 normal_volatility_decimal  0.005
#> 3 one_standard_deviation_1y_bp 50
```

5.2 SOFR 先物オプション

5.2.1 先物価格から金利への変換

短期金利先物は、一般に金利と反対方向に動く価格表示を使用する。本例では次の関係を用いる。

$$\text{Futures Price} = 100 - 100 \times \text{Rate}$$

したがって、先物価格 94.54 は金利 5.46% に対応する。

$$F_0 = \frac{100 - 94.54}{100} = 0.0546$$

行使価格 94.50 は行使金利 5.50% に対応する。

$$K = \frac{100 - 94.50}{100} = 0.0550$$

先物価格の call は、金利が低下して先物価格が上昇したときに価値を持つ。金利を原資産として表現すると、これは put に相当する。そのため、Normal 公式では `option_sign = -1` を使用する。

```
trade_date <- as.Date("2023-09-26")
maturity_date <- as.Date("2023-12-15")

futures_price <- 94.54
strike_futures_price <- 94.50

forward_rate <- (100 - futures_price) / 100
strike_rate <- (100 - strike_futures_price) / 100

normal_volatility <- 50 / 10000
risk_free_rate <- 5.4 / 100

maturity_year_fraction <- as.numeric(
  maturity_date - trade_date
) / 365

maturity_discount_factor <- exp(
  -risk_free_rate * maturity_year_fraction
)

option_sign <- -1

futures_option_input_tbl <- tibble::tribble(
```

```

~input,                                ~value,
"trade_date",                          as.character(trade_date),
"maturity_date",                      as.character(maturity_date),
"futures_price",                      as.character(futures_price),
"strike_futures_price",              as.character(strike_futures_price),
"forward_rate",                      as.character(forward_rate),
"strike_rate",                      as.character(strike_rate),
"normal_volatility",                as.character(normal_volatility),
"maturity_year_fraction",          as.character(maturity_year_fraction),
"discount_factor",                  as.character(maturity_discount_factor)
)

GaussQuant::show_tbl_GQH(
  futures_option_input_tbl,
  "Normal futures-option assumptions",
  n = 20
)

#> # A tibble: 9 x 2
#>   input                value
#>   <chr>                <chr>
#> 1 trade_date          2023-09-26
#> 2 maturity_date       2023-12-15
#> 3 futures_price       94.54
#> 4 strike_futures_price 94.5
#> 5 forward_rate        0.05459999999999999
#> 6 strike_rate         0.055
#> 7 normal_volatility   0.005
#> 8 maturity_year_fraction 0.219178082191781
#> 9 discount_factor     0.988234148959797

```

5.2.2 価格と Greeks

Normal モデルによる価格と Greeks を計算する。

```

normal_greeks_tbl <- GaussQuant::normal_option_greeks_GQL(
  option_sign = option_sign,
  forward = forward_rate,
  strike = strike_rate,
  vol = normal_volatility,
  maturity = maturity_year_fraction,
  discount_factor = maturity_discount_factor,
  risk_free_rate = risk_free_rate
)

```

```

normal_npv <- normal_greeks_tbl |>
  dplyr::filter(.data$metric == "npv") |>
  dplyr::pull(.data$value)

contract_value_multiplier <- 100 * 2500

normal_summary_tbl <- dplyr::bind_rows(
  tibble::tibble(
    metric = c(
      "option_npv_rate_unit",
      "contract_value_multiplier",
      "delivery_amount"
    ),
    value = c(
      normal_npv,
      contract_value_multiplier,
      normal_npv * contract_value_multiplier
    )
  ),
  normal_greeks_tbl |>
    dplyr::filter(.data$metric != "npv")
)

GaussQuant::show_tbl_GQH(
  normal_summary_tbl,
  "Normal futures-option summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 option_npv_rate_unit                1.1340e-3
#> 2 contract_value_multiplier          2.5    e+5
#> 3 delivery_amount                    2.8349e+2
#> 4 delta                             -5.6116e-1
#> 5 gamma                             1.6598e+2
#> 6 vega                              1.8190e-1
#> 7 theta                             -2.0135e-3

```

option_npv_rate_unit は金利を小数表示した単位でのオプション価値である。金額換算は商品ごとの契約乗数に依存する。本例では、原章と同じ換算係数 100 * 2500 を使用する。

5.3 マイナス金利とボラティリティ分析

5.3.1 ゼロ金利とマイナス金利

Normal モデルの重要な特徴は、フォワード金利がゼロまたはマイナスの場合でも価格公式を変更する必要がないことである。

次の例では、フォワード金利をマイナス 0.20%、行使金利を 0%、Normal volatility を 60bp とする。

```
negative_rate_example_tbl <- tibble::tribble(
  ~option_type, ~option_sign, ~forward, ~strike,
  "call",      1,             -0.0020,  0.0000,
  "put",       -1,             -0.0020,  0.0000
) |>
dplyr::mutate(
  normal_volatility = 60 / 10000,
  maturity = 1,
  discount_factor = exp(-0.01),
  npv = purrr::pmap_dbl(
    list(
      .data$option_sign,
      .data$forward,
      .data$strike,
      .data$normal_volatility,
      .data$maturity,
      .data$discount_factor
    ),
    function(
      option_sign,
      forward,
      strike,
      normal_volatility,
      maturity,
      discount_factor
    ) {
      GaussQuant::normal_option_price_GQL(
        option_sign = option_sign,
        forward = forward,
        strike = strike,
        vol = normal_volatility,
        maturity = maturity,
        discount_factor = discount_factor
      )
    }
  )
)
```



```

    )
  )

GaussQuant::show_tbl_GQH(
  negative_rate_example_tbl,
  "Normal pricing with a negative forward rate",
  n = 20
)

#> # A tibble: 2 x 8
#>   option_type option_sign forward strike normal_volatility maturity
#>   <chr>         <dbl>   <dbl>  <dbl>         <dbl>      <dbl>
#> 1 call          1 -0.002    0         0.006        1
#> 2 put          -1 -0.002    0         0.006        1
#> # i 2 more variables: discount_factor <dbl>, npv <dbl>

```

Normal 分布は下方に限界を持たないため、極端なマイナス金利も正の確率で許容する。これは計算上の柔軟性である一方、経済的には常に現実的とは限らない。

5.3.2 インプライド Normal ボラティリティ

市場で観測されるオプション価格から、Normal モデルと整合するボラティリティを逆算したものがインプライド Normal ボラティリティである。

市場価格を V_{mkt} とすると、次式を満たす σ_N を求める。

$$V_N(F_0, K, T, D, \sigma_N) = V_{\text{mkt}}$$

GaussQuant には、Normal 価格からボラティリティを直接逆算する関数が用意されている。

```

target_npv <- 0.1133 / 100

implied_normal_volatility <- GaussQuant::normal_vol_from_price_GQL(
  option_sign = option_sign,
  strike = strike_rate,
  forward = forward_rate,
  maturity = maturity_year_fraction,
  option_npv = target_npv,
  discount_factor = maturity_discount_factor
)

repriced_target <- GaussQuant::normal_option_price_GQL(
  option_sign = option_sign,
  forward = forward_rate,
  strike = strike_rate,

```

```

    vol = implied_normal_volatility,
    maturity = maturity_year_fraction,
    discount_factor = maturity_discount_factor
  )

  implied_volatility_tbl <- tibble::tribble(
    ~metric, ~value,
    "target_npv", target_npv,
    "implied_normal_volatility", implied_normal_volatility,
    "implied_normal_volatility_bp", implied_normal_volatility * 10000,
    "repriced_npv", repriced_target,
    "repricing_difference", repriced_target - target_npv
  )

  GaussQuant::show_tbl_GQH(
    implied_volatility_tbl,
    "Implied normal volatility",
    n = 20
  )
#> # A tibble: 5 x 2
#>   metric value
#>   <chr>    <dbl>
#> 1 target_npv 0.001133
#> 2 implied_normal_volatility 0.0049948
#> 3 implied_normal_volatility_bp 49.948
#> 4 repriced_npv 0.001133
#> 5 repricing_difference 0

```

再評価値と市場価格の差が数値誤差の範囲に収まれば、逆算結果が正しいことを確認できる。

5.3.3 金利・ボラティリティ・時間シナリオ

Normal モデルでは、フォワード金利と Normal volatility がいずれも絶対変化量で表される。このため、シナリオも 1bp 単位の加算として設定する方が分かりやすい。

```

normal_calculator_function <- GaussQuant::make_normal_calculator_GQL(
  option_sign = option_sign,
  strike = strike_rate,
  maturity = maturity_year_fraction,
  discount_factor = maturity_discount_factor,
  forward = forward_rate,
  vol = normal_volatility
)

```

```

normal_calculator_object <- GaussQuant::normal_calculator_GQL(
  option_sign = option_sign,
  strike = strike_rate,
  maturity = maturity_year_fraction,
  discount_factor = maturity_discount_factor,
  forward = forward_rate,
  vol = normal_volatility
)

forward_up_1bp <- forward_rate + 1 / 10000
volatility_up_1bp <- normal_volatility + 1 / 10000

maturity_after_one_day <- max(
  maturity_year_fraction - 1 / 365,
  0
)

discount_factor_after_one_day <- exp(
  -risk_free_rate * maturity_after_one_day
)

closure_scenario_tbl <- tibble::tribble(
  ~scenario,      ~npv,
  "base",         normal_calculator_function(),
  "forward_up_1bp", normal_calculator_function(
    forward_new = forward_up_1bp
  ),
  "normal_vol_up_1bp", normal_calculator_function(
    vol_new = volatility_up_1bp
  ),
  "one_day_passed",  normal_calculator_function(
    maturity_new = maturity_after_one_day,
    discount_factor_new = discount_factor_after_one_day
  )
) |>
  dplyr::mutate(
    difference_from_base = .data$npv -
      .data$npv[.data$scenario == "base"]
  )

class_style_tbl <- tibble::tribble(
  ~scenario,      ~npv,
  "base",         normal_calculator_object$npv(),

```

```

"forward_up_1bp",    normal_calculator_object$npv(
  forward_new = forward_up_1bp
),
"normal_vol_up_1bp", normal_calculator_object$npv(
  vol_new = volatility_up_1bp
),
"one_day_passed",    normal_calculator_object$npv(
  maturity_new = maturity_after_one_day,
  discount_factor_new = discount_factor_after_one_day
)
)

GaussQuant::show_tbl_GQH(
  closure_scenario_tbl,
  "Normal-model scenarios",
  n = 20
)

#> # A tibble: 4 x 3
#>   scenario                npv difference_from_base
#>   <chr>                <dbl>                <dbl>
#> 1 base                  0.0011340                  0
#> 2 forward_up_1bp        0.0010787             -0.000055284
#> 3 normal_vol_up_1bp     0.0011521              0.000018195
#> 4 one_day_passed        0.0011284             -0.0000055347

GaussQuant::show_tbl_GQH(
  class_style_tbl,
  "Class-like normal calculator",
  n = 20
)

#> # A tibble: 4 x 2
#>   scenario                npv
#>   <chr>                <dbl>
#> 1 base                  0.0011340
#> 2 forward_up_1bp        0.0010787
#> 3 normal_vol_up_1bp     0.0011521
#> 4 one_day_passed        0.0011284

```

先物価格の call を金利 put として評価しているため、金利が 1bp 上昇するとオプション価値は低下する。Normal volatility が上昇すると、選択権の時間価値が増加する。

5.4 Caplet と Cap

Caplet は、将来の単一期間のフォワード金利に対する call option である。期間 $[T_{i-1}, T_i]$ のフォワード金利を F_i 、行使金利を K 、年率換算期間を α_i 、支払日の割引係数を $P(0, T_i)$ とする。

Caplet の価格は次式となる。

$$V_i^{\text{caplet}} = N\alpha_i P(0, T_i) [(F_i - K)N(d_i) + \sigma_{N,i}\sqrt{\tau_i}\phi(d_i)]$$

ここで、

$$d_i = \frac{F_i - K}{\sigma_{N,i}\sqrt{\tau_i}}$$

である。Cap は複数の Caplet の合計として評価される。

$$V^{\text{cap}} = \sum_i V_i^{\text{caplet}}$$

本節では、期間構造モデルを構築せず、各 Caplet のフォワード金利、Fixing までの時間および割引係数を外生的に与える。

```
cap_notional <- 10000000
cap_strike <- 0.05
flat_discount_rate <- 0.045

caplet_input_tbl <- tibble::tribble(
  ~caplet, ~fixing_time, ~payment_time, ~accrual_fraction, ~forward_rate, ~normal_vol_bp,
  1L,      0.25,      0.50,      0.25,      0.0510,      80,
  2L,      0.50,      0.75,      0.25,      0.0505,      82,
  3L,      0.75,      1.00,      0.25,      0.0495,      84,
  4L,      1.00,      1.25,      0.25,      0.0485,      86
) |>
dplyr::mutate(
  normal_volatility = .data$normal_vol_bp / 10000,
  discount_factor = exp(
    -flat_discount_rate * .data$payment_time
  ),
  option_value_per_rate_unit = purrr::pmap_dbl(
    list(
      .data$forward_rate,
      .data$normal_volatility,
      .data$fixing_time,
      .data$discount_factor
```

```

    ),
    function(
      forward_rate,
      normal_volatility,
      fixing_time,
      discount_factor
    ) {
      GaussQuant::normal_option_price_GQL(
        option_sign = 1,
        forward = forward_rate,
        strike = cap_strike,
        vol = normal_volatility,
        maturity = fixing_time,
        discount_factor = discount_factor
      )
    }
  ),
  caplet_npv = cap_notional *
    .data$accrual_fraction *
    .data$option_value_per_rate_unit
)

cap_summary_tbl <- tibble::tribble(
  ~metric,      ~value,
  "cap_notional", cap_notional,
  "cap_strike",   cap_strike,
  "cap_npv",      sum(caplet_input_tbl$caplet_npv)
)

GaussQuant::show_tbl_GQH(
  caplet_input_tbl,
  "Bachelier caplet decomposition",
  n = 20
)

#> # A tibble: 4 x 10
#>   caplet fixing_time payment_time accrual_fraction forward_rate normal_vol_bp
#>   <int>      <dbl>      <dbl>          <dbl>          <dbl>          <dbl>
#> 1     1        0.25        0.5            0.25          0.051           80
#> 2     2        0.5         0.75            0.25          0.0505          82
#> 3     3        0.75        1              0.25          0.0495          84
#> 4     4        1          1.25            0.25          0.0485          86
#> # i 4 more variables: normal_volatility <dbl>, discount_factor <dbl>,
#> #   option_value_per_rate_unit <dbl>, caplet_npv <dbl>

```

```
GaussQuant::show_tbl_GQH(
  cap_summary_tbl,
  "Bachelier cap summary",
  n = 20
)
#> # A tibble: 3 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 cap_notional 10000000
#> 2 cap_strike    0.05
#> 3 cap_npv      24274.
```

各 Caplet は異なる Fixing 時点とフォワード金利を持つ。ここでは期間構造を動的に生成せず、各フォワード金利をそれぞれの支払日フォワード測度の下で直接評価している。

5.5 Swaption 評価

5.5.1 フォワードスワップレートとアニュイティ

Swaption の原資産は、将来開始する金利スワップのフォワードスワップレートである。

固定レグ支払日を $T_1, \dots, T_n$ 、年率換算期間を α_i とすると、スワップアニュイティは次式である。

$$A(0) = \sum_{i=1}^n \alpha_i P(0, T_i)$$

スワップ開始日を T_0 、満期を T_n とすると、単一カーブの簡略化されたフォワードスワップレートは次式となる。

$$S_0 = \frac{P(0, T_0) - P(0, T_n)}{A(0)}$$

本節では決定論的な割引係数を与え、フォワードスワップレートとアニュイティを計算する。

```
swap_exercise_time <- 1
swap_start_time <- 1
fixed_payment_times <- c(2, 3, 4)
fixed_accrual_fractions <- rep(
  1,
  length(fixed_payment_times)
)
swap_flat_discount_rate <- 0.035

swap_discount_tbl <- tibble::tibble(
  payment_time = fixed_payment_times,
```

```

accrual_fraction = fixed_accrual_fractions,
discount_factor = exp(
  -swap_flat_discount_rate * fixed_payment_times
)
)

swap_annuity <- sum(
  swap_discount_tbl$accrual_fraction *
  swap_discount_tbl$discount_factor
)

swap_start_discount_factor <- exp(
  -swap_flat_discount_rate * swap_start_time
)

swap_end_discount_factor <- exp(
  -swap_flat_discount_rate *
  max(fixed_payment_times)
)

forward_swap_rate <- (
  swap_start_discount_factor -
  swap_end_discount_factor
) / swap_annuity

forward_swap_tbl <- tibble::tribble(
  ~metric,          ~value,
  "swap_start_discount_factor", swap_start_discount_factor,
  "swap_end_discount_factor",   swap_end_discount_factor,
  "swap_annuity",               swap_annuity,
  "forward_swap_rate",          forward_swap_rate
)

GaussQuant::show_tbl_GQH(
  swap_discount_tbl,
  "Underlying swap fixed-leg schedule",
  n = 20
)

#> # A tibble: 3 x 3
#>   payment_time accrual_fraction discount_factor
#>   <dbl>         <dbl>         <dbl>
#> 1         2         1         0.93239
#> 2         3         1         0.90032

```



```
#> 3          4          1      0.86936

GaussQuant::show_tbl_GQH(
  forward_swap_tbl,
  "Forward swap rate and annuity",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 swap_start_discount_factor 0.96561
#> 2 swap_end_discount_factor   0.86936
#> 3 swap_annuity              2.7021
#> 4 forward_swap_rate         0.035620
```

5.5.2 Bachelier Swaption

payer swaption は、将来の固定金利支払いスワップへ入る権利である。フォワードスワップレートが行使金利を上回ると価値を持つため、フォワードスワップレートに対する call option として評価できる。

swap measure の下でフォワードスワップレートをマルチンゲールとし、Normal モデルを仮定する。

$$dS_t = \sigma_S dW_t^A$$

payer swaption の価格は次式となる。

$$V_{\text{payer}} = NA(0) \left[(S_0 - K)N(d) + \sigma_S \sqrt{T_e} \phi(d) \right]$$

ここで、

$$d = \frac{S_0 - K}{\sigma_S \sqrt{T_e}}$$

である。receiver swaption は put option として評価する。

```
swaption_notional <- 10000000
swaption_strike <- 0.034
swaption_normal_volatility <- 100 / 10000

payer_option_value_per_annuity <- GaussQuant::normal_option_price_GQL(
  option_sign = 1,
  forward = forward_swap_rate,
  strike = swaption_strike,
  vol = swaption_normal_volatility,
```

```

    maturity = swap_exercise_time,
    discount_factor = 1
  )

receiver_option_value_per_annuity <- GaussQuant::normal_option_price_GQL(
  option_sign = -1,
  forward = forward_swap_rate,
  strike = swaption_strike,
  vol = swaption_normal_volatility,
  maturity = swap_exercise_time,
  discount_factor = 1
)

swaption_tbl <- tibble::tribble(
  ~option_type, ~option_sign, ~npv,
  "payer",      1,             swaption_notional *
                                swap_annuity *
                                payer_option_value_per_annuity,
  "receiver",   -1,             swaption_notional *
                                swap_annuity *
                                receiver_option_value_per_annuity
) |>
  dplyr::mutate(
    forward_swap_rate = forward_swap_rate,
    strike = swaption_strike,
    annuity = swap_annuity,
    normal_volatility = swaption_normal_volatility
  ) |>
  dplyr::relocate(
    .data$option_type,
    .data$forward_swap_rate,
    .data$strike,
    .data$annuity,
    .data$normal_volatility,
    .data$npv
  )

GaussQuant::show_tbl_GQH(
  swaption_tbl,
  "Bachelier swaption summary",
  n = 20
)

#> # A tibble: 2 x 7

```

```
#>   option_type forward_swap_rate strike annuity normal_volatility      npv
#>   <chr>          <dbl>   <dbl>   <dbl>          <dbl>   <dbl>
#> 1 payer          0.035620  0.034   2.7021          0.01 131091.
#> 2 receiver       0.035620  0.034   2.7021          0.01  87325.
#> # i 1 more variable: option_sign <dbl>
```

アニュイティ $A(0)$ はすでに将来の固定レグ支払いを現在価値へ割引いた量である。そのため、Normal 公式側では割引係数を 1 とし、結果へ元本とアニュイティを掛けている。

5.6 Black volatility と Normal volatility

Black モデルでは、ATM 付近のオプション価値は概ね次式に比例する。

$$V_{\text{Black,ATM}} \approx D(0, T) F_0 \sigma_B \sqrt{T} \phi(0)$$

Normal モデルでは次式となる。

$$V_{\text{Normal,ATM}} \approx D(0, T) \sigma_N \sqrt{T} \phi(0)$$

したがって、ATM 付近では両モデルのボラティリティに次の近似関係がある。

$$\sigma_N \approx F_0 \sigma_B$$

Normal volatility は金利水準と同じ単位を持つため、同じ Black volatility であっても、金利水準が低下すると対応する Normal volatility は小さくなる。

```
black_volatility <- 0.20

black_normal_comparison_tbl <- tibble::tribble(
  ~forward_rate, ~black_volatility,
  0.0500,        black_volatility,
  0.0200,        black_volatility,
  0.0050,        black_volatility
) |>
dplyr::mutate(
  approximate_normal_volatility =
    .data$forward_rate *
    .data$black_volatility,
  approximate_normal_volatility_bp =
    .data$approximate_normal_volatility *
    10000
)
```

```
GaussQuant::show_tbl_GQH(
  black_normal_comparison_tbl,
  "Approximate ATM Black-to-Normal volatility conversion",
  n = 20
)
#> # A tibble: 3 x 4
#>   forward_rate black_volatility approximate_normal_vola~1 approximate_normal_v~2
#>   <dbl>         <dbl>         <dbl>         <dbl>
#> 1      0.05          0.2          0.01          100
#> 2      0.02          0.2          0.004          40
#> 3      0.005         0.2          0.001          10
#> # i abbreviated names: 1: approximate_normal_volatility,
#> #   2: approximate_normal_volatility_bp
```

この関係は ATM 近似であり、Strike が ATM から離れる場合やボラティリティスマイルが存在する場合には、単純な換算だけでは十分でない。

5.7 まとめ

本章では、金利期間構造全体の確率モデルを与えず、各商品の評価に必要なフォワード金利またはフォワードスワップレートを、適切な測度の下でマルチングールとして直接モデル化した。

Normal モデルでは、フォワード金利の変化額が正規分布に従う。

$$dF_t = \sigma_N dW_t$$

このため、フォワード金利がゼロまたはマイナスとなる場合でも、価格公式を変更せずに評価できる。

Normal volatility は相対変化率ではなく、金利と同じ単位を持つ。金利市場では通常、ベースポイント単位で表示される。Normal volatility 50bp は小数表示で 0.0050 である。

金利先物オプションでは、先物価格から金利へ変換すると Payoff の向きが反転する。先物価格の call は、金利を原資産とした場合には put として評価される。

Cap は複数の Caplet の合計であり、各 Caplet は対応するフォワード金利を原資産とする。Swaption はフォワードスワップレートを原資産とし、スワップアニュイティを価格倍率として使用する。

Normal モデルはマイナス金利を扱える一方、金利水準にかかわらず同じ絶対変動幅を仮定し、極端な負の金利も許容する。実際の市場では、Black、Shifted Black、Normal および SABR などを、金利水準とボラティリティ市場の慣行に応じて使い分ける。

```
chapter05_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Normal dynamics", "Additive forward-rate process",
  "Futures option",  "Rate conversion, price, and Greeks",
  "Implied normal volatility", "Volatility inversion and repricing",
```

```

    "Negative rates",      "Pricing without a logarithmic restriction",
    "Cap",                "Sum of Bachelier caplets",
    "Swaption",           "Forward swap rate times annuity"
  )

GaussQuant::show_tbl_GQH(
  chapter05_summary_tbl,
  "Chapter II-5 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section                main_result
#>   <chr>                 <chr>
#> 1 Normal dynamics      Additive forward-rate process
#> 2 Futures option       Rate conversion, price, and Greeks
#> 3 Implied normal volatility Volatility inversion and repricing
#> 4 Negative rates       Pricing without a logarithmic restriction
#> 5 Cap                  Sum of Bachelier caplets
#> 6 Swaption             Forward swap rate times annuity

cat(
  "\nChapter II-5 Normal Bachelier model completed successfully.\n"
)
#>
#> Chapter II-5 Normal Bachelier model completed successfully.

```

第 II-6 章 SABR モデル

前章までの Black モデルと Normal モデルでは、満期と Strike が同じオプションに対して、一つのボラティリティを与えた。しかし、実際のオプション市場では、同じ満期であっても Strike ごとに異なるインプライド・ボラティリティが観測される。この Strike 方向の形状をボラティリティ・スマイルまたはスキューと呼ぶ。

SABR モデルは、フォワード価格またはフォワード金利と、そのボラティリティを同時に確率変数として扱う。モデル名は、Stochastic Alpha Beta Rho の頭文字に由来する。

フォワード F_t とボラティリティ係数 α_t の動学を、概念的に次のように表す。

$$dF_t = \alpha_t F_t^\beta dW_t^{(1)}$$

$$d\alpha_t = \nu \alpha_t dW_t^{(2)}$$

$$dW_t^{(1)} dW_t^{(2)} = \rho dt$$

SABR モデルには、 α 、 β 、 ν 、 ρ の 4 パラメータがある。

- α はボラティリティ水準を主に決定する。
- β はフォワード水準に対する変動幅の依存度を表す。
- ν はボラティリティ自体の変動性であり、スマイルの曲率に強く影響する。
- ρ はフォワードとボラティリティの相関であり、スマイルの左右非対称性に影響する。

$\beta = 1$ は対数正規型に近く、 $\beta = 0$ は Normal 型に近い。実務では 4 パラメータを同時推定すると不安定になりやすいため、 β をあらかじめ固定し、 α 、 ν 、 ρ を市場ボラティリティへカリブレーションすることが多い。

本章では、まず Black volatility のスマイルへ SABR モデルを適合させる。次に各パラメータがスマイルの形状へ与える影響を確認する。最後に、負の Strike を含む Normal volatility の市場データに対して Shifted SABR を適用する。

6.1 SABR モデルとパラメータ

6.1.1 SABR パラメータの役割

SABR の各パラメータは完全に独立してスマイルを動かすわけではないが、主な役割は次のように整理できる。

6.1.2 Alpha

α は初期ボラティリティ水準である。他の条件を固定して α を上げると、スマイル全体が概ね上方へ移動する。

6.1.3 Beta

β はフォワード水準に対する弾性を表す。

$$\text{instantaneous volatility} = \alpha_t F_t^\beta$$

$\beta = 1$ では変動幅がフォワード水準に比例し、Black モデルに近い。 $\beta = 0$ では変動幅がフォワード水準に依存せず、Normal モデルに近い。

6.1.4 Volatility of volatility

ν は volatility of volatility、すなわち vol-of-vol である。 ν が大きいほど、ボラティリティが将来大きく変化する可能性が高まり、一般にスマイルの曲率が強くなる。

本章のコードでは、変数名として `volvol` を使用する。

6.1.5 Rho

ρ はフォワード変化とボラティリティ変化の相関である。負の ρ は、フォワード低下時にボラティリティが上昇しやすい構造を表し、低 Strike 側のボラティリティを相対的に高くする。

6.1.6 Hagan 近似とカリブレーション

SABR モデルの市場利用では、Hagan らによる近似式を用いて、SABR パラメータから Black volatility または Normal volatility を計算することが多い。

市場 Strike を K_i 、市場ボラティリティを σ_i^{mkt} 、SABR 近似値を σ_i^{SABR} とする。本章では、適合度を次の RMSE で測定する。

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\sigma_i^{\text{mkt}} - \sigma_i^{\text{SABR}})^2}$$

β を固定し、RMSE が最小となる α 、 ν 、 ρ を数値的に求める。

6.2 Black SABR のカリブレーション

6.2.1 市場データ

フォワードを 0.56%、満期を 2 年、 $\beta = 0.5$ とする。市場 Black volatility は Strike ごとに異なり、明確なスマイルを形成している。

```

black_market_data <- tibble::tribble(
  ~strike_pct, ~market_vol_pct,
  0.06,        90.78,
  0.31,        46.09,
  0.56,        45.30,
  0.81,        50.17,
  1.06,        53.85,
  1.56,        58.42,
  2.56,        62.72
) |>
  dplyr::mutate(
    strike = .data$strike_pct / 100,
    market_vol = .data$market_vol_pct / 100
  )

forward_black <- 0.0056
maturity_black <- 2
beta_black <- 0.5

black_model_inputs <- tibble::tribble(
  ~input,      ~value,
  "forward",    forward_black,
  "maturity",   maturity_black,
  "fixed_beta", beta_black
)

initial_parameters_black <- c(
  alpha = 0.1,
  volvol = 0.1,
  rho = 0.1
)

GaussQuant::show_tbl_GQH(
  black_market_data |>
    dplyr::select(
      .data$strike,
      .data$market_vol
    ),
  "Black SABR market data",
  n = 20
)

#> # A tibble: 7 x 2
#>   strike market_vol

```



```
#>      <dbl>      <dbl>
#> 1 0.0006      0.9078
#> 2 0.0031      0.4609
#> 3 0.0056      0.453
#> 4 0.0081      0.5017
#> 5 0.0106      0.5385
#> 6 0.0156      0.5842
#> 7 0.0256      0.6272

GaussQuant::show_tbl_GQH(
  black_model_inputs,
  "Black SABR model inputs",
  n = 20
)
#> # A tibble: 3 x 2
#>   input      value
#>   <chr>      <dbl>
#> 1 forward    0.0056
#> 2 maturity    2
#> 3 fixed_beta 0.5
```

ATM に近い Strike は 0.56% であり、フォワード 0.56% と一致する。ATM 付近の水準は主に α に影響し、両翼の形状が ν と ρ の推定に影響する。

6.2.2 パラメータ・カリブレーション

目的関数では、各 Strike における QuantLib の SABR Black volatility と市場 Black volatility の RMSE を計算する。

```
objective_sabr_black <- function(parameters) {
  alpha <- parameters[[1]]
  volvol <- parameters[[2]]
  rho <- parameters[[3]]

  if (
    alpha <= 0 ||
    volvol < 0 ||
    rho <= -0.9999 ||
    rho >= 0.9999
  ) {
    return(1e6)
  }

  calculated_volatility <- purrr::map_dbl(
```

```

black_market_data$strike,
function(strike_value) {
  GaussQuant::safe_sabr_vol_GQL(
    strike = strike_value,
    forward = forward_black,
    maturity = maturity_black,
    alpha = alpha,
    beta = beta_black,
    volvol = volvol,
    rho = rho
  )
}
)

if (
  any(is.na(calculated_volatility)) ||
  any(!is.finite(calculated_volatility))
) {
  return(1e6)
}

GaussQuant::rmse_GQH(
  actual = black_market_data$market_vol,
  fitted = calculated_volatility
)
}

black_fit <- stats::optim(
  par = initial_parameters_black,
  fn = objective_sabr_black,
  method = "L-BFGS-B",
  lower = c(
    0.0001,
    0,
    -0.9999
  ),
  upper = c(
    Inf,
    Inf,
    0.9999
  )
)

```

```

black_parameters <- black_fit$par

black_calculated_volatility <- purrr::map_dbl(
  black_market_data$strike,
  function(strike_value) {
    GaussQuant::safe_sabr_vol_GQL(
      strike = strike_value,
      forward = forward_black,
      maturity = maturity_black,
      alpha = black_parameters[[1]],
      beta = beta_black,
      volvol = black_parameters[[2]],
      rho = black_parameters[[3]]
    )
  }
)

black_calibration_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "alpha",         black_parameters[[1]],
  "beta",          beta_black,
  "volvol",        black_parameters[[2]],
  "rho",           black_parameters[[3]],
  "rmse",          black_fit$value,
  "convergence_code", black_fit$convergence
)

GaussQuant::show_tbl_GQH(
  black_calibration_tbl,
  "Black SABR calibration summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 alpha         0.030583
#> 2 beta          0.5
#> 3 volvol        0.72848
#> 4 rho           0.44933
#> 5 rmse          0.013881
#> 6 convergence_code 0

```

`convergence_code` が 0 であれば、`optim()` は通常の収束条件を満たして終了している。RMSE は市場ボラティリティとモデル値の平均的な誤差を表す。

6.2.3 ボラティリティ・スマイル

```
black_smile_tbl <- black_market_data |>
  dplyr::transmute(
    strike = .data$strike,
    market_vol = .data$market_vol,
    sabr_vol = black_calculated_volatility,
    residual = .data$market_vol -
      black_calculated_volatility
  )

black_smile_plot_tbl <- black_smile_tbl |>
  dplyr::select(
    .data$strike,
    .data$market_vol,
    .data$sabr_vol
  ) |>
  tidyr::pivot_longer(
    cols = c(
      market_vol,
      sabr_vol
    ),
    names_to = "series",
    values_to = "volatility"
  )

black_smile_plot <- ggplot2::ggplot(
  black_smile_plot_tbl,
  ggplot2::aes(
    x = .data$strike,
    y = .data$volatility,
    group = .data$series,
    linetype = .data$series,
    shape = .data$series
  )
) +
  ggplot2::geom_line(
    linewidth = 0.6
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
```

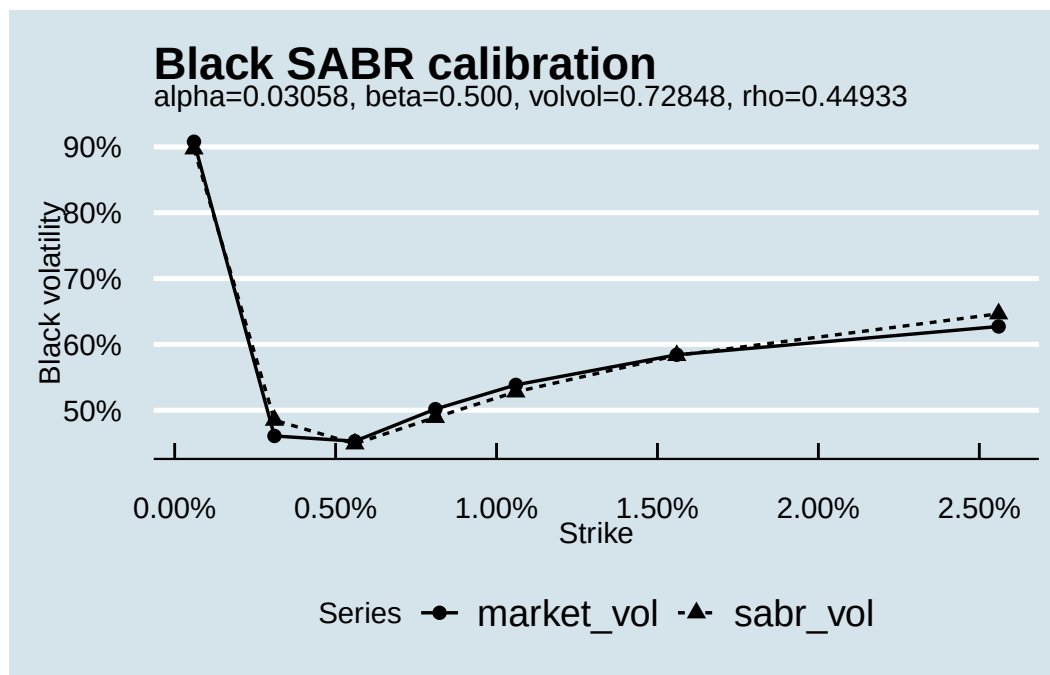
```

ggplot2::scale_x_continuous(
  labels = function(x) sprintf("%.2f%%", 100 * x)
) +
ggplot2::scale_y_continuous(
  labels = function(x) sprintf("%.0f%%", 100 * x)
) +
ggplot2::labs(
  title = "Black SABR calibration",
  subtitle = paste0(
    "alpha=",
    sprintf("%.5f", black_parameters[[1]]),
    ", beta=",
    sprintf("%.3f", beta_black),
    ", volvol=",
    sprintf("%.5f", black_parameters[[2]]),
    ", rho=",
    sprintf("%.5f", black_parameters[[3]])
  ),
  x = "Strike",
  y = "Black volatility",
  linetype = "Series",
  shape = "Series"
)

GaussQuant::show_tbl_GQH(
  black_smile_tbl,
  "Black SABR smile table",
  n = 20
)

#> # A tibble: 7 x 4
#>   strike market_vol sabr_vol   residual
#>   <dbl>      <dbl>    <dbl>     <dbl>
#> 1 0.0006      0.9078  0.89705  0.010753
#> 2 0.0031      0.4609  0.48489 -0.023985
#> 3 0.0056      0.453   0.44898  0.0040242
#> 4 0.0081      0.5017  0.48919  0.012510
#> 5 0.0106      0.5385  0.52789  0.010614
#> 6 0.0156      0.5842  0.58328  0.00092393
#> 7 0.0256      0.6272  0.64648 -0.019277
show_plot_LFL(black_smile_plot)

```



SABR はすべての市場点を完全に通過する補間法ではなく、少数のパラメータでスマイル全体の形状を表現するモデルである。残差が完全にゼロでなくても、Strike 方向に滑らかなボラティリティを与えられる点に意味がある。

6.3 SABR パラメータの感応度

推定結果を基準として、一つのパラメータだけを上下させた場合のスマイルを確認する。パラメータ間には相互作用があるため、これは厳密なリスク分解ではなく、形状を理解するための比較である。

```
base_parameter_tbl <- tibble::tribble(
  ~parameter, ~base_value,      ~shift,
  "alpha",    black_parameters[[1]], 0.02,
  "beta",     beta_black,           0.20,
  "volvol",   black_parameters[[2]], 0.40,
  "rho",      black_parameters[[3]], 0.44
)

base_parameters <- stats::setNames(
  base_parameter_tbl$base_value,
  base_parameter_tbl$parameter
)

parameter_shifts <- stats::setNames(
  base_parameter_tbl$shift,
  base_parameter_tbl$parameter
)
```

```
sabr_sensitivity_tbl <- purrr::map_dfr(
  names(base_parameters),
  function(parameter_name) {
    parameters_up <- base_parameters
    parameters_down <- base_parameters

    parameters_up[[parameter_name]] <-
      parameters_up[[parameter_name]] +
      parameter_shifts[[parameter_name]]

    parameters_down[[parameter_name]] <-
      parameters_down[[parameter_name]] -
      parameter_shifts[[parameter_name]]

    parameters_up[["alpha"]] <- max(
      parameters_up[["alpha"]],
      1e-6
    )
    parameters_down[["alpha"]] <- max(
      parameters_down[["alpha"]],
      1e-6
    )
    parameters_up[["beta"]] <- min(
      max(parameters_up[["beta"]], 0),
      1
    )
    parameters_down[["beta"]] <- min(
      max(parameters_down[["beta"]], 0),
      1
    )
    parameters_up[["volvol"]] <- max(
      parameters_up[["volvol"]],
      0
    )
    parameters_down[["volvol"]] <- max(
      parameters_down[["volvol"]],
      0
    )
    parameters_up[["rho"]] <- min(
      max(parameters_up[["rho"]], -0.95),
      0.95
    )
    parameters_down[["rho"]] <- min(
```

```

    max(parameters_down[["rho"]], -0.95),
    0.95
  )

scenario_parameter_tbl <- tibble::tribble(
  ~scenario, ~alpha, ~beta, ~volvol,
  "base",    base_parameters[["alpha"]], base_parameters[["beta"]], base_parameters[["volvol"]],
  "up",      parameters_up[["alpha"]],   parameters_up[["beta"]],   parameters_up[["volvol"]],
  "down",    parameters_down[["alpha"]], parameters_down[["beta"]], parameters_down[["volvol"]]
)

purrr::pmap_dfr(
  scenario_parameter_tbl,
  function(
    scenario,
    alpha,
    beta,
    volvol,
    rho
  ) {
    tibble::tibble(
      strike = black_market_data$strike,
      volatility = purrr::map_dbl(
        black_market_data$strike,
        function(strike_value) {
          GaussQuant::safe_sabr_vol_GQL(
            strike = strike_value,
            forward = forward_black,
            maturity = maturity_black,
            alpha = alpha,
            beta = beta,
            volvol = volvol,
            rho = rho
          )
        }
      ),
      scenario = scenario,
      parameter = parameter_name
    )
  }
)

```



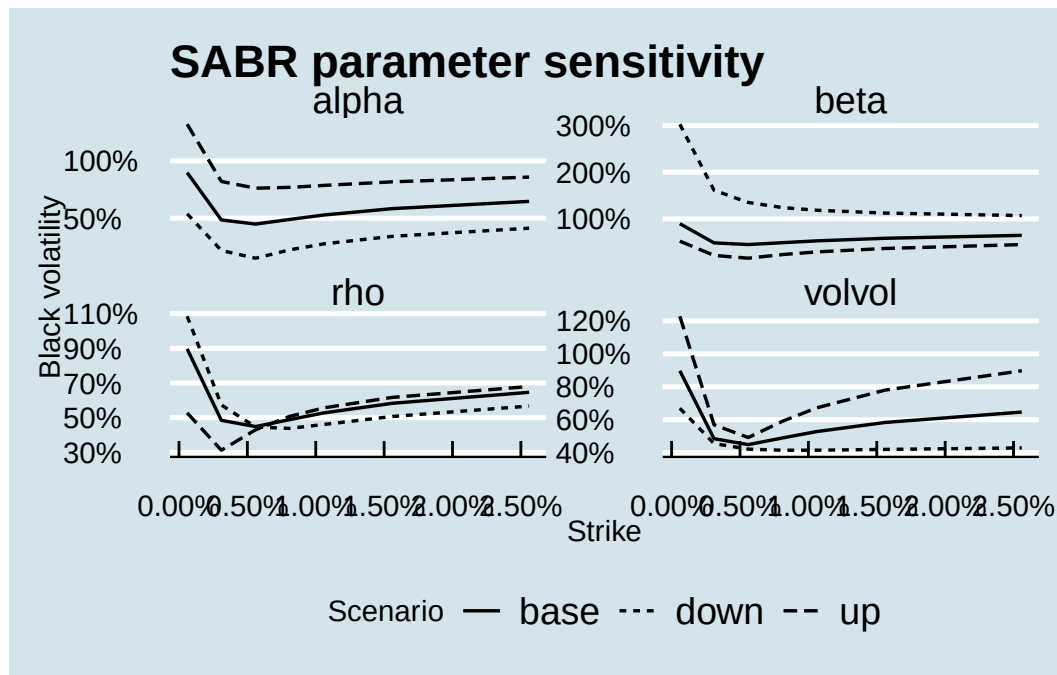
```

sabr_sensitivity_plot <- ggplot2::ggplot(
  sabr_sensitivity_tbl,
  ggplot2::aes(
    x = .data$strike,
    y = .data$volatility,
    group = .data$scenario,
    linetype = .data$scenario
  )
) +
  ggplot2::geom_line(
    linewidth = 0.6
  ) +
  ggplot2::facet_wrap(
    ~parameter,
    scales = "free_y"
  ) +
  ggplot2::scale_x_continuous(
    labels = function(x) sprintf("%.2f%%", 100 * x)
  ) +
  ggplot2::scale_y_continuous(
    labels = function(x) sprintf("%.0f%%", 100 * x)
  ) +
  ggplot2::labs(
    title = "SABR parameter sensitivity",
    x = "Strike",
    y = "Black volatility",
    linetype = "Scenario"
  )

GaussQuant::show_tbl_GQH(
  base_parameter_tbl,
  "SABR sensitivity settings",
  n = 20
)

#> # A tibble: 4 x 3
#>   parameter base_value shift
#>   <chr>         <dbl> <dbl>
#> 1 alpha         0.030583  0.02
#> 2 beta          0.5      0.2
#> 3 volvol        0.72848  0.4
#> 4 rho           0.44933  0.44
show_plot_LFL(sabr_sensitivity_plot)

```



一般に、 α はスマイルの水準、 ν は曲率、 ρ は傾きと左右非対称性に強く影響する。 β も水準と傾きの双方へ影響するため、他のパラメータとの識別が難しい。このため、 β を市場慣行や過去推定値に基づいて固定することが多い。

6.4 Shifted SABR と負の金利

通常の Black SABR 近似では、フォワードと Strike が正である必要がある。負の金利を含む市場では、フォワードと Strike へ同じ Shift s を加える。

$$\tilde{F} = F + s$$

$$\tilde{K} = K + s$$

Shift 後の値が正であれば、対数やべき乗を含む SABR 近似を適用できる。

本章では、Normal volatility の市場データに対して、Shift を加えた Hagan 近似を使用する。出力は Normal volatility であるが、内部では Shift 後のフォワードと Strike を使ってスマイルを表現する。

Shift は負の金利を消してしまう調整ではない。モデル計算上の座標を移動するだけであり、実際の Strike とフォワードは元の水準のまま解釈する。

6.5 Shifted SABR Normal のカリブレーション

6.5.1 市場データ

フォワードを 0.05%、満期を 2 年、 $\beta = 0.5$ 、Shift を 2.5% とする。市場データにはマイナス 0.95% の Strike が含まれる。

```

normal_market_data <- tibble::tribble(
  ~strike_pct, ~market_vol_bp,
  -0.95,      41.94,
  -0.45,      38.64,
  -0.20,      37.71,
  0.05,       37.80,
  0.30,       39.05,
  0.55,       41.37,
  1.05,       47.81
) |>
dplyr::mutate(
  strike = .data$strike_pct / 100,
  market_vol = .data$market_vol_bp / 10000
)

forward_normal <- 0.0005
maturity_normal <- 2
beta_normal <- 0.5
shift_normal <- 0.025

normal_model_inputs <- tibble::tribble(
  ~input,      ~value,
  "forward",    forward_normal,
  "maturity",   maturity_normal,
  "fixed_beta", beta_normal,
  "shift",      shift_normal
)

initial_parameters_normal <- c(
  alpha = 0.1,
  volvol = 0.1,
  rho = 0.1
)

GaussQuant::show_tbl_GQH(
  normal_market_data |>
  dplyr::select(
    .data$strike,
    .data$market_vol
  ),
  "Shifted SABR normal market data",
  n = 20
)

```

```

#> # A tibble: 7 x 2
#>   strike market_vol
#>   <dbl>         <dbl>
#> 1 -0.0095  0.004194
#> 2 -0.0045  0.003864
#> 3 -0.002   0.003771
#> 4  0.0005  0.00378
#> 5  0.003   0.003905
#> 6  0.0055  0.004137
#> 7  0.0105  0.004781

GaussQuant::show_tbl_GQH(
  normal_model_inputs,
  "Shifted SABR normal model inputs",
  n = 20
)
#> # A tibble: 4 x 2
#>   input      value
#>   <chr>      <dbl>
#> 1 forward  0.0005
#> 2 maturity  2
#> 3 fixed_beta 0.5
#> 4 shift    0.025

```

すべての市場 Strike について、

$$K + s > 0$$

が必要である。本例の最小 Strike はマイナス 0.95%、Shift は 2.5% であるため、Shift 後の Strike は正となる。

6.5.2 パラメータ・カリブレーション

Black SABR と同様に β を固定し、 α 、 ν 、 ρ を推定する。目的関数は Normal volatility 単位の RMSE である。

```

objective_sabr_normal <- function(parameters) {
  alpha <- parameters[[1]]
  volvol <- parameters[[2]]
  rho <- parameters[[3]]

  if (
    alpha <= 0.001 ||
    volvol < 0 ||
    rho <= -0.999 ||
    rho >= 0.999
  )

```

```

    ) {
      return(1e6)
    }

    calculated_volatility <- purrr::map_dbl(
      normal_market_data$strike,
      function(strike_value) {
        GaussQuant::shifted_normal_vol_hagan_GQL(
          strike = strike_value,
          forward = forward_normal,
          maturity = maturity_normal,
          beta = beta_normal,
          alpha = alpha,
          volvol = volvol,
          rho = rho,
          shift = shift_normal
        )
      }
    )

    if (
      any(is.na(calculated_volatility)) ||
      any(!is.finite(calculated_volatility))
    ) {
      return(1e6)
    }

    GaussQuant::rmse_GQH(
      actual = normal_market_data$market_vol,
      fitted = calculated_volatility
    )
  }

  normal_fit <- stats::optim(
    par = initial_parameters_normal,
    fn = objective_sabr_normal,
    method = "L-BFGS-B",
    lower = c(
      0.001,
      0,
      -0.999
    ),
    upper = c(

```

```

    Inf,
    Inf,
    0.999
  )
)

normal_parameters <- normal_fit$par

normal_calculated_volatility <- purrr::map_dbl(
  normal_market_data$strike,
  function(strike_value) {
    GaussQuant::shifted_normal_vol_hagan_GQL(
      strike = strike_value,
      forward = forward_normal,
      maturity = maturity_normal,
      beta = beta_normal,
      alpha = normal_parameters[[1]],
      volvol = normal_parameters[[2]],
      rho = normal_parameters[[3]],
      shift = shift_normal
    )
  }
)

normal_calibration_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "alpha",          normal_parameters[[1]],
  "beta",           beta_normal,
  "volvol",         normal_parameters[[2]],
  "rho",            normal_parameters[[3]],
  "shift",          shift_normal,
  "rmse",           normal_fit$value,
  "convergence_code", normal_fit$convergence
)

GaussQuant::show_tbl_GQH(
  normal_calibration_tbl,
  "Shifted SABR normal calibration summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   parameter      value
#>   <chr>          <dbl>

```

```
#> 1 alpha      0.025373
#> 2 beta       0.5
#> 3 volvol     0.099359
#> 4 rho        0.099797
#> 5 shift      0.025
#> 6 rmse       0.00029785
#> 7 convergence_code 0
```

6.5.3 Normal ボラティリティ・スマイル

```
normal_smile_tbl <- normal_market_data |>
  dplyr::transmute(
    strike = .data$strike,
    market_vol = .data$market_vol,
    sabr_vol = normal_calculated_volatility,
    residual = .data$market_vol -
      normal_calculated_volatility
  )

normal_smile_plot_tbl <- normal_smile_tbl |>
  dplyr::select(
    .data$strike,
    .data$market_vol,
    .data$sabr_vol
  ) |>
  tidyr::pivot_longer(
    cols = c(
      market_vol,
      sabr_vol
    ),
    names_to = "series",
    values_to = "volatility"
  )

normal_smile_plot <- ggplot2::ggplot(
  normal_smile_plot_tbl,
  ggplot2::aes(
    x = .data$strike,
    y = .data$volatility,
    group = .data$series,
    linetype = .data$series,
    shape = .data$series
  )
)
```

```

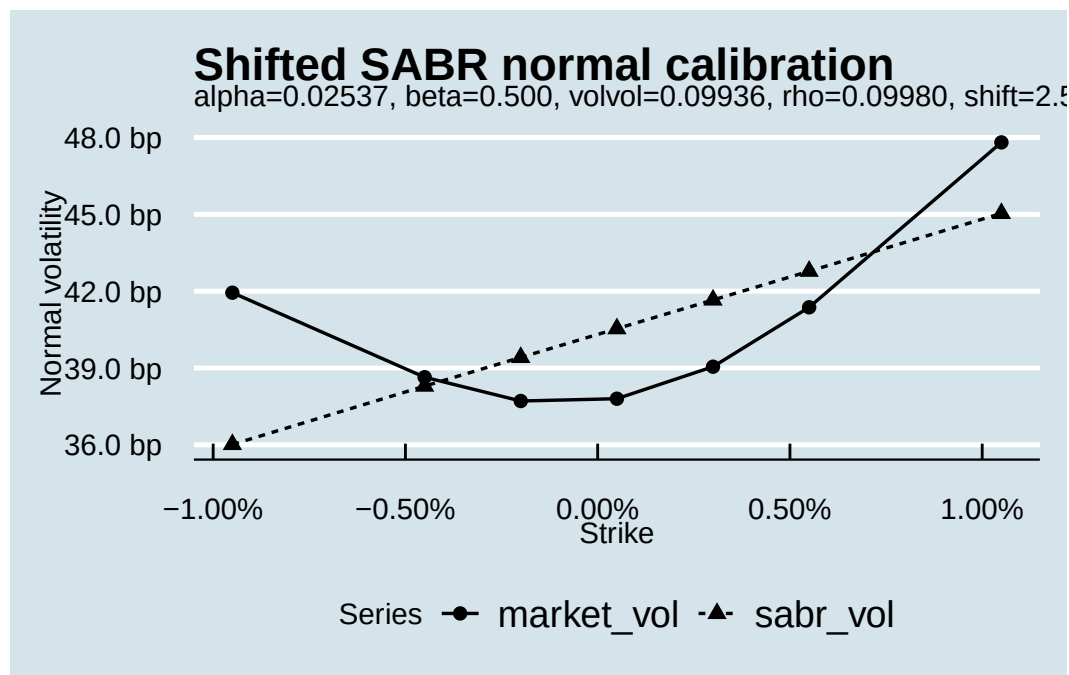
) +
  ggplot2::geom_line(
    linewidth = 0.6
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::scale_x_continuous(
    labels = function(x) sprintf("%.2f%%", 100 * x)
  ) +
  ggplot2::scale_y_continuous(
    labels = function(x) sprintf("%.1f bp", 10000 * x)
  ) +
  ggplot2::labs(
    title = "Shifted SABR normal calibration",
    subtitle = paste0(
      "alpha=",
      sprintf("%.5f", normal_parameters[[1]]),
      ", beta=",
      sprintf("%.3f", beta_normal),
      ", volvol=",
      sprintf("%.5f", normal_parameters[[2]]),
      ", rho=",
      sprintf("%.5f", normal_parameters[[3]]),
      ", shift=",
      sprintf("%.2f%%", 100 * shift_normal)
    ),
    x = "Strike",
    y = "Normal volatility",
    linetype = "Series",
    shape = "Series"
  )

GaussQuant::show_tbl_GQH(
  normal_smile_tbl |>
  dplyr::mutate(
    market_vol_bp = 10000 * .data$market_vol,
    sabr_vol_bp = 10000 * .data$sabr_vol,
    residual_bp = 10000 * .data$residual
  ),
  "Shifted SABR normal smile table",
  n = 20
)

```



```
#> # A tibble: 7 x 7
#>   strike market_vol  sabr_vol  residual market_vol_bp  sabr_vol_bp  residual_bp
#>   <dbl>      <dbl>    <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
#> 1 -0.0095  0.004194 0.0036013  5.9271e-4    41.94    36.013    5.9271
#> 2 -0.0045  0.003864 0.0038292  3.4815e-5    38.64    38.292    0.34815
#> 3 -0.002   0.003771 0.0039414 -1.7044e-4    37.71    39.414   -1.7044
#> 4  0.0005   0.00378  0.0040535 -2.7345e-4    37.8     40.535   -2.7345
#> 5  0.003     0.003905 0.0041656 -2.6056e-4    39.05    41.656   -2.6056
#> 6  0.0055   0.004137 0.0042779 -1.4094e-4    41.37    42.779   -1.4094
#> 7  0.0105   0.004781 0.0045037  2.7730e-4    47.81    45.037    2.7730
show_plot_LFL(normal_smile_plot)
```



Black volatility と Normal volatility は単位が異なるため、Black SABR の RMSE と Shifted SABR Normal の RMSE を数値の大きさだけで直接比較してはならない。Black 側は相対ボラティリティ、Normal 側は金利の絶対変化幅である。

6.6 外挿とモデル比較

6.6.1 カリブレーション範囲外の Strike

SABR モデルは、市場点の間を滑らかにつなぐだけでなく、市場点の外側にもボラティリティを与える。ただし、外挿結果は観測データによる制約が弱く、パラメータの影響を強く受ける。

次のコードでは、市場データの外側にある二つの Strike について Normal volatility を確認する。

```
outer_strike_input_tbl <- tibble::tribble(
  ~label,      ~strike,
  "lower_wing", -0.0195,
```

```

    "upper_wing", 0.0205
  )

outer_strike_tbl <- outer_strike_input_tbl |>
  dplyr::mutate(
    shifted_strike = .data$strike +
      shift_normal,
    normal_volatility = purrr::map_dbl(
      .data$strike,
      function(strike_value) {
        GaussQuant::shifted_normal_vol_hagan_GQL(
          strike = strike_value,
          forward = forward_normal,
          maturity = maturity_normal,
          beta = beta_normal,
          alpha = normal_parameters[[1]],
          volvol = normal_parameters[[2]],
          rho = normal_parameters[[3]],
          shift = shift_normal
        )
      }
    ),
    normal_volatility_bp = 10000 *
      .data$normal_volatility
  )

GaussQuant::show_tbl_GQH(
  outer_strike_tbl,
  "Shifted SABR outer-strike check",
  n = 20
)

#> # A tibble: 2 x 5
#>   label      strike shifted_strike normal_volatility normal_volatility_bp
#>   <chr>      <dbl>      <dbl>          <dbl>          <dbl>
#> 1 lower_wing -0.0195      0.0055      0.0030674      30.674
#> 2 upper_wing  0.0205      0.0455      0.0049584      49.584

```

Shift 後の Strike がゼロ以下となる領域では、この近似式を使用できない。また、数式上計算できる場合でも、カリブレーション範囲から遠い外挿値には注意が必要である。

6.6.2 Black SABR と Shifted SABR Normal

二つのカリブレーション結果を一覧にする。

```

calibration_comparison_tbl <- tibble::tribble(
  ~model,          ~alpha,          ~beta,      ~volvol,          ~rho,
  "Black SABR",    black_parameters[[1]],  beta_black,  black_parameters[[2]],  black_param
  "Shifted SABR Normal", normal_parameters[[1]], beta_normal, normal_parameters[[2]], normal_param
)

GaussQuant::show_tbl_GQH(
  calibration_comparison_tbl,
  "SABR calibration comparison",
  n = 20
)

#> # A tibble: 2 x 8
#>   model          alpha  beta  volvol      rho shift      rmse convergence_code
#>   <chr>         <dbl> <dbl>   <dbl>   <dbl> <dbl>   <dbl>          <int>
#> 1 Black SABR    0.030583  0.5 0.72848 0.44933 0      1.3881e-2          0
#> 2 Shifted SAB~ 0.025373  0.5 0.099359 0.099797 0.025 2.9785e-4          0

```

Black SABR では、正のフォワードと Strike から Black volatility を計算した。Shifted SABR Normal では、フォワードと Strike へ Shift を加え、負の Strike を含む市場データへ Normal volatility を適合させた。

どちらも同じ 4 つの SABR パラメータを用いるが、出力するインプライド・ボラティリティの定義と単位が異なる。したがって、パラメータ値や RMSE を異なるボラティリティ表現の間で単純比較するのではなく、それぞれの市場表示におけるスマイル適合度を確認する。

6.7 まとめ

SABR モデルは、フォワードとそのボラティリティを同時に確率変数として扱い、Strike ごとに異なるインプライド・ボラティリティを少数のパラメータで表現する。

α は主にボラティリティ水準、 ν はスマイルの曲率、 ρ は傾きと左右非対称性へ影響する。 β はフォワード水準に対する変動幅の弾性を表し、他のパラメータとの識別が難しいため、本章では固定してカリブレーションした。

Black SABR では、正のフォワードと Strike を前提として Black volatility のスマイルを評価した。負の金利を含む Normal volatility 市場では、フォワードと Strike へ共通の Shift を加える Shifted SABR を使用した。

SABR は市場ボラティリティを完全に補間することだけを目的とするものではない。Strike 方向に滑らかで、パラメータによる解釈が可能なスマイルを構築し、観測されていない Strike の評価やリスク管理へ利用する。

```

chapter06_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Black SABR",      "Calibration to a Black-volatility smile",
  "Parameter sensitivity", "Alpha, beta, vol-of-vol, and rho effects",
  "Shifted SABR Normal", "Normal-volatility smile with negative strikes",
  "Outer strikes",     "Extrapolation outside calibration points"
)

```

```
GaussQuant::show_tbl_GQH(
  chapter06_summary_tbl,
  "Chapter II-6 summary",
  n = 20
)
#> # A tibble: 4 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Black SABR      Calibration to a Black-volatility smile
#> 2 Parameter sensitivity Alpha, beta, vol-of-vol, and rho effects
#> 3 Shifted SABR Normal Normal-volatility smile with negative strikes
#> 4 Outer strikes   Extrapolation outside calibration points

cat(
  "\nChapter II-6 SABR model completed successfully.\n"
)
#>
#> Chapter II-6 SABR model completed successfully.
```

第 II-7 章 金利期間構造モデル Early Termination 債評価

Callable Bond は、発行体があらかじめ定められた日と価格で期限前償還できる債券である。市場金利が低下すると、発行体は既存の高いクーポンを支払い続けるよりも、債券を期限前償還して低い金利で資金を再調達する誘因を持つ。

したがって、コール条項は発行体にとって価値を持つ一方、投資家にとっては不利な権利である。同じクーポン、満期および信用条件を持つ場合、Callable Bond の価格はオプションを含まない Bullet Bond の価格以下となる。

概念的には、次の関係で表される。

$$V_{\text{callable}} = V_{\text{bullet}} - V_{\text{issuer call}}$$

発行体のコール権は、金利低下時に価値が上昇する。この性質は、固定金利を受け取り変動金利を支払う Receiver Swaption に近い。複数のコール可能日がある場合、その権利は Bermudan Receiver Swaption に対応する。

ただし、Callable Bond と Swaption では、元本交換、クーポンと固定レグ、決済、経過利子、行使価格および支払日の調整が必ずしも完全には一致しない。このため、本章では、

$$V_{\text{callable}} \approx V_{\text{bullet}} - V_{\text{Bermudan receiver swaption}}$$

をモデル構造の比較として確認する。

期限前償還権を評価するためには、将来の金利変動と各行使日における継続価値を扱う必要がある。本章では、初期金利期間構造へ適合できる Hull-White 1-factor モデルと金利 Tree を使用する。

```
knitr::opts_chunk$set(
  collapse = TRUE,
  comment = "#>",
  warning = FALSE,
  message = FALSE
)

suppressPackageStartupMessages({
```

```
library(tidyverse)
library(ggthemes)
library(QuantLib)
library(GaussQuant)
library(tidyverse)
library(broom)
})
```

7.1 Hull–White モデルと債券条件

7.1.1 Hull–White 1-factor モデル

Hull–White モデルでは、短期金利 r_t を次の Gaussian 過程で表す。

$$dr_t = [\theta(t) - ar_t] dt + \sigma dW_t$$

ここで、 a は平均回帰速度、 σ は短期金利のボラティリティ、 $\theta(t)$ は時間依存のドリフトである。

a が大きいほど、金利ショックの影響は速く減衰する。 σ が大きいほど将来金利の分布が広がり、金利オプションの価値は一般に高くなる。

Hull–White モデルの重要な特徴は、 $\theta(t)$ を調整することによって、評価時点で観測された初期イールドカーブを再現できることである。モデルの確率部分を状態変数 x_t として分離すると、次の Ornstein–Uhlenbeck 過程として表せる。

$$dx_t = -ax_t dt + \sigma dW_t$$

短期金利は、初期期間構造を表す決定論的な部分と状態変数の和として解釈できる。

$$r_t = \phi(t) + x_t$$

Hull–White モデルは Gaussian モデルであるため、短期金利が負になる可能性を持つ。前章までの Normal モデルと同様、ゼロ金利近傍を扱いやすい一方、極端な負の金利も理論上許容する。

7.1.2 債券条件

5 年満期、年率 3%、半年払いの固定利付債を考える。市場利回りも 3% とし、オプションを含まない Bullet Bond の価格がおおむね額面付近となる条件を設定する。

最初の 1 年をノンコール期間とし、その後の各クーポン日に額面 100 で期限前償還できるものとする。満期日は通常の元本償還日であるため、コール可能日から除外する。

```
trade_date <- GaussQuant::date_GQL("2024-04-15")
issue_date <- GaussQuant::date_GQL("2024-04-17")
maturity_date <- GaussQuant::date_GQL("2029-04-17")
```

```
GaussQuant::set_eval_date_GQL(trade_date)

coupon_rate <- 0.03
market_yield <- 0.03
settlement_days <- 2L
face_amount <- 100
call_price_clean <- 100
non_call_periods <- 2L

bond_calendar <- QuantLib::WeekendsOnly()
bond_day_counter <- QuantLib::ActualActual("Bond")
model_day_counter <- QuantLib::Actual365Fixed()
bond_compounding <- QuantLib::Compounding_Compounded_get()
bond_frequency <- QuantLib::Frequency_Semiannual_get()

bond_input_tbl <- tibble::tribble(
  ~input,          ~value,
  "trade_date",    "2024-04-15",
  "issue_date",    "2024-04-17",
  "maturity_date", "2029-04-17",
  "coupon_rate",   as.character(coupon_rate),
  "market_yield",  as.character(market_yield),
  "settlement_days", as.character(settlement_days),
  "face_amount",   as.character(face_amount),
  "call_price",    as.character(call_price_clean),
  "non_call_periods", as.character(non_call_periods)
)

GaussQuant::show_tbl_GQH(
  bond_input_tbl,
  "Callable bond input assumptions",
  n = 20
)

#> # A tibble: 9 x 2
#>   input          value
#>   <chr>         <chr>
#> 1 trade_date    2024-04-15
#> 2 issue_date    2024-04-17
#> 3 maturity_date 2029-04-17
#> 4 coupon_rate   0.03
#> 5 market_yield  0.03
#> 6 settlement_days 2
#> 7 face_amount   100
```

```
#> 8 call_price      100
#> 9 non_call_periods 2
```

7.1.3 Bullet Bond

クーポンスケジュールを作成し、コール条項を含まない固定利付債を評価する。

```
bond_schedule <- QuantLib::Schedule(
  issue_date,
  maturity_date,
  GaussQuant::period_months_GQL(6),
  bond_calendar,
  "Unadjusted",
  "Unadjusted",
  "Backward",
  FALSE
)

bond_schedule_tbl <- GaussQuant::schedule_table_GQL(
  bond_schedule
)

bullet_bond <- QuantLib::FixedRateBond(
  settlement_days,
  face_amount,
  bond_schedule,
  c(coupon_rate),
  bond_day_counter
)

bullet_settlement_date <- tryCatch(
  bullet_bond$settlementDate(),
  error = function(e) NULL
)

bullet_clean_price <- GaussQuant::bond_clean_price_from_yield_safe_GQL(
  bond = bullet_bond,
  yield_rate = market_yield,
  day_counter = bond_day_counter,
  compounding = bond_compounding,
  frequency = bond_frequency,
  settlement_date = bullet_settlement_date
)
```



```

bullet_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_rate",    coupon_rate,
  "market_yield",   market_yield,
  "clean_price_from_yield", bullet_clean_price,
  "face_amount",    face_amount
)

GaussQuant::show_tbl_GQH(
  bond_schedule_tbl,
  "Bullet bond schedule",
  n = 20
)

#> # A tibble: 11 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2024-04-17
#> 2     2 2024-10-17
#> 3     3 2025-04-17
#> 4     4 2025-10-17
#> 5     5 2026-04-17
#> 6     6 2026-10-17
#> 7     7 2027-04-17
#> 8     8 2027-10-17
#> 9     9 2028-04-17
#> 10    10 2028-10-17
#> 11    11 2029-04-17

GaussQuant::show_tbl_GQH(
  bullet_summary_tbl,
  "Bullet bond summary",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_rate      0.03
#> 2 market_yield      0.03
#> 3 clean_price_from_yield 99.999
#> 4 face_amount      100

```

7.1.4 初期カーブと Hull-White モデル

Callable Bond と Swaption を同じ初期カーブと Hull-White パラメータで評価する。

本節では、年率 3% のフラットカーブを使用する。Callable Bond の主目的はオプション構造と Tree 評価を確認することであり、複雑な市場カーブ構築は行わない。

```
hull_white_a <- 0.03
hull_white_sigma <- 0.01
tree_steps <- 120L

flat_curve <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date,
  rate = market_yield,
  day_counter = model_day_counter,
  compounding = "Continuous"
)

flat_curve_handle <- QuantLib::YieldTermStructureHandle(
  flat_curve
)

hull_white_model <- GaussQuant::hull_white_model_GQL(
  term_structure = flat_curve_handle,
  a = hull_white_a,
  sigma = hull_white_sigma
)

hull_white_input_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "mean_reversion_a", hull_white_a,
  "sigma",          hull_white_sigma,
  "tree_steps",     tree_steps,
  "flat_curve_rate", market_yield
)

GaussQuant::show_tbl_GQH(
  hull_white_input_tbl,
  "Hull-White model assumptions",
  n = 20
)

#> # A tibble: 4 x 2
#>   parameter      value
#>   <chr>         <dbl>
```

```
#> 1 mean_reversion_a    0.03
#> 2 sigma              0.01
#> 3 tree_steps         120
#> 4 flat_curve_rate    0.03
```

7.2 Callable Bond の構築と Tree 評価

7.2.1 コールスケジュール

スケジュールには発行日と満期日も含まれる。最初の 2 クーポン期間をノンコールとし、満期日を除く残りのクーポン日をコール可能日とする。

```
bond_schedule_dates <- GaussQuant::schedule_date_vector_GQL(
  bond_schedule
)

first_call_position <- non_call_periods + 1L

call_dates <- bond_schedule_dates[
  first_call_position:
    (length(bond_schedule_dates) - 1L)
]

call_schedule <- GaussQuant::make_callability_schedule_GQL(
  call_dates = call_dates,
  call_price_clean = call_price_clean
)

call_schedule_tbl <- tibble::tibble(
  call_number = seq_along(call_dates),
  call_date = call_dates,
  call_price = rep(
    call_price_clean,
    length(call_dates)
  )
)

GaussQuant::show_tbl_GQH(
  call_schedule_tbl,
  "Callable bond exercise schedule",
  n = 20
)

#> # A tibble: 8 x 3
#>   call_number call_date call_price
```

#>	<int>	<date>	<dbl>
#> 1	1	2025-04-17	100
#> 2	2	2025-10-17	100
#> 3	3	2026-04-17	100
#> 4	4	2026-10-17	100
#> 5	5	2027-04-17	100
#> 6	6	2027-10-17	100
#> 7	7	2028-04-17	100
#> 8	8	2028-10-17	100

7.2.2 Callable Bond の Tree 評価

Callable Bond を作成し、Hull-White モデルに基づく金利 Tree を Pricing Engine として設定する。

各 Tree node では、発行体はその時点でコールする価値と、債券を存続させる価値が比較される。発行体にとってコールが有利であれば期限前償還が選択される。

```
callable_bond <- QuantLib::CallableFixedRateBond(
  settlement_days,
  face_amount,
  bond_schedule,
  c(coupon_rate),
  bond_day_counter,
  "Unadjusted",
  call_price_clean,
  issue_date,
  call_schedule
)

callable_bond_engine <- QuantLib::TreeCallableFixedRateBondEngine(
  hull_white_model,
  tree_steps
)

callable_bond$setPricingEngine(
  callable_bond_engine
)

#> NULL

callable_clean_price <- GaussQuant::callable_bond_clean_price_safe_GQL(
  callable_bond
)

callable_dirty_price <- GaussQuant::callable_bond_dirty_price_safe_GQL(
```

```

    callable_bond
  )

callable_settlement_value <- GaussQuant::callable_bond_settlement_value_safe_GQL(
  callable_bond
)

embedded_call_value <- bullet_clean_price -
  callable_clean_price

callable_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "bullet_clean_price",    bullet_clean_price,
  "callable_clean_price",  callable_clean_price,
  "callable_dirty_price",  callable_dirty_price,
  "callable_settlement_value", callable_settlement_value,
  "embedded_call_value",   embedded_call_value
)

GaussQuant::show_tbl_GQH(
  callable_summary_tbl,
  "Callable bond summary",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 bullet_clean_price    99.999
#> 2 callable_clean_price  98.001
#> 3 callable_dirty_price  98.001
#> 4 callable_settlement_value 98.001
#> 5 embedded_call_value    1.9984

```

Callable Bond 価格が Bullet Bond 価格を下回る差額は、投資家が発行体へ与えている期限前償還権の価値と解釈できる。

7.3 Hull-White パラメータ感応度

短期金利のボラティリティが高いほど、将来金利が大きく低下する可能性も高くなる。したがって、発行体のコール権は一般に高くなり、Callable Bond 価格は低下する。

平均回帰速度 a は、金利ショックがどの程度の速さで消滅するかを決める。平均回帰が速い場合、長い期間にわたる金利ショックの影響は小さくなる。

```

callable_sensitivity_input_tbl <- tibble::tribble(
  ~scenario,      ~mean_reversion, ~sigma,
  "low_sigma",    0.03,            0.005,
  "base",         0.03,            0.010,
  "high_sigma",   0.03,            0.020,
  "slow_reversion", 0.01,            0.010,
  "fast_reversion", 0.10,            0.010
)

callable_sensitivity_tbl <- purrr::pmap_dfr(
  callable_sensitivity_input_tbl,
  function(
    scenario,
    mean_reversion,
    sigma
  ) {
    model_i <- GaussQuant::hull_white_model_GQL(
      term_structure = flat_curve_handle,
      a = mean_reversion,
      sigma = sigma
    )

    engine_i <- QuantLib::TreeCallableFixedRateBondEngine(
      model_i,
      tree_steps
    )

    callable_bond$setPricingEngine(
      engine_i
    )

    callable_price_i <- GaussQuant::callable_bond_clean_price_safe_GQL(
      callable_bond
    )

    tibble::tibble(
      scenario = scenario,
      mean_reversion = mean_reversion,
      sigma = sigma,
      callable_clean_price = callable_price_i,
      embedded_call_value = bullet_clean_price -
        callable_price_i
    )
  }
)

```

```

}
)

callable_bond$setPricingEngine(
  callable_bond_engine
)
#> NULL

GaussQuant::show_tbl_GQH(
  callable_sensitivity_tbl,
  "Callable bond Hull-White sensitivity",
  n = 20
)
#> # A tibble: 5 x 5
#>   scenario      mean_reversion sigma callable_clean_price embedded_call_value
#>   <chr>          <dbl> <dbl>          <dbl>          <dbl>
#> 1 low_sigma      0.03 0.005          98.962          1.0371
#> 2 base           0.03 0.01           98.001          1.9984
#> 3 high_sigma     0.03 0.02           96.090          3.9094
#> 4 slow_reversion 0.01 0.01           97.923          2.0766
#> 5 fast_reversion 0.1  0.01           98.243          1.7566

```

パラメータ感応度は他の条件を固定した比較である。市場カリブレーション後のパラメータは互いに独立ではないため、実務上のリスク分析ではカーブとボラティリティ市場を同時に考慮する必要がある。

7.4 Bermudan Receiver Swaption との比較

発行体が債券をコールする誘因は、市場金利がクーポン金利を下回ったときに強くなる。これは固定金利を受け取る Receiver Swap の価値が、金利低下時に上昇する性質と対応する。

そこで、債券のクーポンスケジュールを固定レグとする Receiver Swap を作成し、Callable Bond と同じ行使日を持つ Bermudan Receiver Swaption を評価する。

```

floating_index <- QuantLib::JPYLibor(
  GaussQuant::period_months_GQL(6),
  flat_curve_handle
)

tryCatch(
  floating_index$addFixing(
    trade_date,
    market_yield,
    TRUE
  ),

```

```

    error = function(e) NULL
  )
#> NULL

underlying_receiver_swap <- QuantLib::VanillaSwap(
  QuantLib::Swap_Receiver_get(),
  face_amount,
  bond_schedule,
  coupon_rate,
  bond_day_counter,
  bond_schedule,
  floating_index,
  0,
  bond_day_counter
)

bermudan_receiver_engine <- GaussQuant::hull_white_swaption_engine_GQL(
  term_structure = flat_curve_handle,
  a = hull_white_a,
  sigma = hull_white_sigma,
  method = "tree",
  time_steps = tree_steps
)

bermudan_receiver_swaption <- GaussQuant::make_bermudan_swaption_GQL(
  underlying_swap = underlying_receiver_swap,
  exercise_dates = call_dates,
  pricing_engine = bermudan_receiver_engine
)

bermudan_receiver_npv <- GaussQuant::swaption_npv_GQL(
  bermudan_receiver_swaption
)

receiver_swaption_value_per_100 <- bermudan_receiver_npv /
  face_amount *
  100

bullet_minus_receiver_swaption <- bullet_clean_price -
  receiver_swaption_value_per_100

callable_comparison_tbl <- tibble::tribble(
  ~valuation, ~value,

```



```

"bullet_bond_clean_price",      bullet_clean_price,
"callable_bond_clean_price",    callable_clean_price,
"receiver_swaption_value_per_100", receiver_swaption_value_per_100,
"bullet_minus_receiver_swaption", bullet_minus_receiver_swaption,
"pricing_difference",          callable_clean_price -
                                bullet_minus_receiver_swaption
)

GaussQuant::show_tbl_GQH(
  callable_comparison_tbl,
  "Callable bond and Bermudan receiver swaption comparison",
  n = 20
)

#> # A tibble: 5 x 2
#>   valuation      value
#>   <chr>         <dbl>
#> 1 bullet_bond_clean_price    99.999
#> 2 callable_bond_clean_price  98.001
#> 3 receiver_swaption_value_per_100 1.8868
#> 4 bullet_minus_receiver_swaption 98.113
#> 5 pricing_difference      -0.11163

```

Callable Bond と Bermudan Receiver Swaption は、同じ Hull-White モデルと行使日を使用している。しかし、次の条件は完全には一致していない。

- 債券の元本償還と Swap の元本交換
- Clean price と Swaption NPV の決済単位
- 経過利子
- コール価格
- 固定クーポンと Swap 固定レグ
- 浮動レグの Fixing および支払条件

したがって、pricing_difference が厳密にゼロとなることは要求しない。この比較の目的は、Callable Bond に埋め込まれた金利低下時のオプション価値を、Receiver Swaption との関係から理解することである。

7.5 Vasicek モデルと Hull-White モデル

7.5.1 Vasicek モデルとの違い

Vasicek モデルは、短期金利が一定の長期平均へ回帰する Gaussian モデルである。

$$dr_t = a(b - r_t)dt + \sigma dW_t$$

ここで、 b は長期平均金利である。

Hull–White モデルも Gaussian 平均回帰モデルであるが、時間依存ドリフト $\theta(t)$ を導入することによって、評価時点のイールドカーブへ適合できる。

項目	Vasicek	Hull–White
平均回帰先	一定の長期平均 b	時間依存ドリフトで調整
初期カーブ	一般には完全適合しない	初期カーブへ適合可能
主な入力	a, b, σ, r_0	初期カーブ、 a, σ
金利分布	Gaussian	Gaussian
負の金利	許容	許容

Vasicek モデルは金利期間構造の長期的な形状をモデルパラメータから生成する。Hull–White モデルは、観測カーブを出発点として、その周りの確率的変動を表現する。

7.5.2 Vasicek と Hull–White のゼロ金利曲線

Vasicek では初期短期金利を変えると、モデルが生成する期間構造全体が変化する。Hull–White では状態変数がゼロであれば初期カーブを再現し、状態変数のショックが平均回帰しながら期間構造へ影響する。

```
maturity_grid <- seq(
  0.25,
  10,
  by = 0.25
)

vasicek_scenario_tbl <- tibble::tribble(
  ~scenario,      ~short_rate,
  "r0 = 1%",      0.01,
  "r0 = 3%",      0.03,
  "r0 = 5%",      0.05
)

vasicek_curve_tbl <- purrr::pmap_dfr(
  vasicek_scenario_tbl,
  function(
    scenario,
    short_rate
  ) {
    tibble::tibble(
      maturity = maturity_grid,
      zero_rate = purrr::map_dbl(
        maturity_grid,
        function(maturity_value) {
          GaussQuant::vasicek_zero_rate_GQL(
            time = 0,
```

```

        maturity = maturity_value,
        mean_reversion = 0.30,
        sigma = 0.01,
        long_run_rate = 0.03,
        short_rate = short_rate
      )
    }
  ),
  model = "Vasicek",
  scenario = scenario
)
}
)

hull_white_scenario_tbl <- tibble::tribble(
  ~scenario,      ~state_variable,
  "x0 = -0.3%",   -0.003,
  "base curve",   0.000,
  "x0 = 0.3%",    0.003
)

hull_white_curve_tbl <- purrr::pmap_dfr(
  hull_white_scenario_tbl,
  function(
    scenario,
    state_variable
  ) {
    tibble::tibble(
      maturity = maturity_grid,
      zero_rate = purrr::map_dbl(
        maturity_grid,
        function(maturity_value) {
          GaussQuant::hull_white_zero_rate_GQL(
            time = 0,
            maturity = maturity_value,
            curve = flat_curve,
            mean_reversion = 0.30,
            sigma = 0.01,
            state_variable = state_variable
          )
        }
      ),
    ),
    model = "Hull-White",
  )
}

```

```

    scenario = scenario
  )
}
)

short_rate_curve_tbl <- dplyr::bind_rows(
  vasicek_curve_tbl,
  hull_white_curve_tbl
)

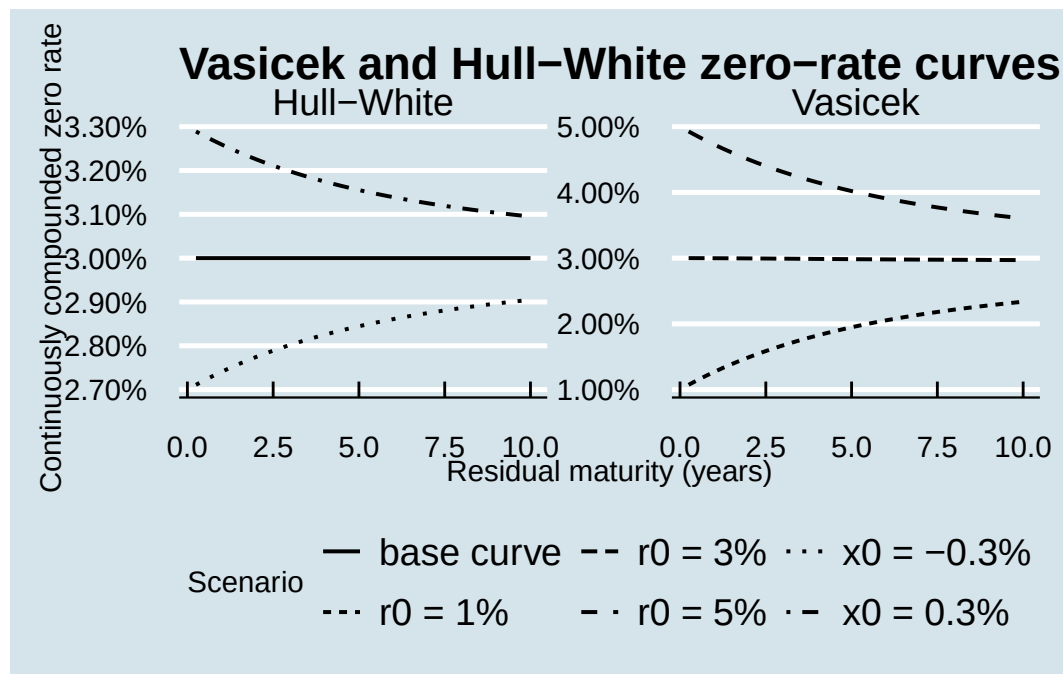
short_rate_curve_plot <- ggplot2::ggplot(
  short_rate_curve_tbl,
  ggplot2::aes(
    x = .data$maturity,
    y = .data$zero_rate,
    group = .data$scenario,
    linetype = .data$scenario
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::facet_wrap(
    ~model,
    scales = "free_y"
  ) +
  ggplot2::scale_y_continuous(
    labels = function(x) sprintf("%.2f%%", 100 * x)
  ) +
  ggplot2::labs(
    title = "Vasicek and Hull-White zero-rate curves",
    x = "Residual maturity (years)",
    y = "Continuously compounded zero rate",
    linetype = "Scenario"
  )

GaussQuant::show_tbl_GQH(
  short_rate_curve_tbl,
  "Short-rate model zero-rate curves",
  n = 30
)

#> # A tibble: 30 x 4
#>   maturity zero_rate model    scenario

```

```
#>      <dbl>      <dbl> <chr>    <chr>
#>  1      0.25      0.010731 Vasicek r0 = 1%
#>  2      0.5      0.011424 Vasicek r0 = 1%
#>  3      0.75      0.012082 Vasicek r0 = 1%
#>  4      1      0.012708 Vasicek r0 = 1%
#>  5      1.25      0.013302 Vasicek r0 = 1%
#>  6      1.5      0.013867 Vasicek r0 = 1%
#>  7      1.75      0.014405 Vasicek r0 = 1%
#>  8      2      0.014917 Vasicek r0 = 1%
#>  9      2.25      0.015404 Vasicek r0 = 1%
#> 10      2.5      0.015868 Vasicek r0 = 1%
#> # i 20 more rows
show_plot_LFL(short_rate_curve_plot)
```



Hull-White の **base curve** が初期フラットカーブ付近に位置することが、初期期間構造へ適合する性質を表している。状態変数のショックは短期側で大きく、満期が長くなるほど平均回帰によって影響が小さくなる。

7.6 カリブレーションと数値検証

7.6.1 Hull-White モデルのカリブレーション

Hull-White モデルのパラメータ a と σ は、Swaption 市場の価格またはインプライド・ボラティリティへ適合させる。

本節では、1 年後に開始して 5 年間継続する 1Y×5Y ATM Swaption を使用する。市場 Normal volatility を 36.06bp とし、平均回帰速度 $a = 0.03$ を固定して、短期金利ボラティリティ σ だけを推定する。

単一の市場商品から a と σ の双方を安定して識別することはできない。複数満期の Swaption を利用する場合

には、複数の Calibration Helper を同時に用いて両パラメータを推定する。

7.6.2 1Y×5Y ATM Swap

```
GaussQuant::set_eval_date_GQL(trade_date)

calibration_calendar <- QuantLib::Japan()
calibration_day_counter <- QuantLib::Actual365Fixed()

swaption_expiry_date <- calibration_calendar$advance(
  trade_date,
  1L,
  "Years"
)

swap_effective_date <- calibration_calendar$advance(
  swaption_expiry_date,
  2L,
  "Days"
)

swap_maturity_date <- calibration_calendar$advance(
  swap_effective_date,
  5L,
  "Years"
)

calibration_schedule <- QuantLib::Schedule(
  swap_effective_date,
  swap_maturity_date,
  GaussQuant::period_months_GQL(6),
  calibration_calendar,
  "ModifiedFollowing",
  "ModifiedFollowing",
  "Backward",
  FALSE
)

calibration_index <- QuantLib::JPYLibor(
  GaussQuant::period_months_GQL(6),
  flat_curve_handle
)
```

```

calibration_annuity <- GaussQuant::swap_annuity_from_schedule_GQL(
  schedule = calibration_schedule,
  curve = flat_curve,
  day_counter = calibration_day_counter
)

calibration_forward_swap_rate <- GaussQuant::forward_swap_rate_from_schedule_GQL(
  schedule = calibration_schedule,
  curve = flat_curve,
  day_counter = calibration_day_counter
)

calibration_swap_tbl <- tibble::tribble(
  ~metric,          ~value,
  "expiry_date",    GaussQuant::iso_GQL(swaption_expiry_date),
  "swap_effective_date", GaussQuant::iso_GQL(swap_effective_date),
  "swap_maturity_date", GaussQuant::iso_GQL(swap_maturity_date),
  "annuity",        as.character(calibration_annuity),
  "forward_swap_rate", as.character(calibration_forward_swap_rate)
)

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    calibration_schedule
  ),
  "1Yx5Y fixed-leg schedule",
  n = 20
)

#> # A tibble: 11 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2025-04-17
#> 2     2 2025-10-17
#> 3     3 2026-04-17
#> 4     4 2026-10-19
#> 5     5 2027-04-19
#> 6     6 2027-10-18
#> 7     7 2028-04-17
#> 8     8 2028-10-17
#> 9     9 2029-04-17
#> 10    10 2029-10-17
#> 11    11 2030-04-17

GaussQuant::show_tbl_GQH(

```

```

calibration_swap_tbl,
"1Yx5Y forward swap summary",
n = 20
)
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <chr>
#> 1 expiry_date 2025-04-15
#> 2 swap_effective_date 2025-04-17
#> 3 swap_maturity_date 2030-04-17
#> 4 annuity      4.47364922408499
#> 5 forward_swap_rate 0.0302262727429665

```

7.6.3 One-helper calibration

```

market_normal_volatility <- 36.06 / 10000
fixed_mean_reversion <- 0.03
initial_sigma <- 0.0001

calibration_model <- GaussQuant::hull_white_model_GQL(
  term_structure = flat_curve_handle,
  a = fixed_mean_reversion,
  sigma = initial_sigma
)

calibration_jamshidian_engine <- QuantLib::JamshidianSwaptionEngine(
  calibration_model
)

swaption_helper <- QuantLib::SwaptionHelper(
  swaption_expiry_date,
  GaussQuant::period_years_GQL(5),
  QuantLib::QuoteHandle(
    QuantLib::SimpleQuote(
      market_normal_volatility
    )
  ),
  calibration_index,
  GaussQuant::period_months_GQL(6),
  calibration_day_counter,
  calibration_day_counter,
  flat_curve_handle,
  QuantLib::BlackCalibrationHelper_RelativePriceError_get(),

```



```
calibration_forward_swap_rate,
1,
QuantLib::VolatilityType_Normal_get()
)

swaption_helper$setPricingEngine(
  calibration_jamshidian_engine
)
#> NULL

calibration_helper_vector <- GaussQuant::push_calibration_helpers_GQL(
  list(swaption_helper)
)

calibration_end_criteria <- QuantLib::EndCriteria(
  10000L,
  100L,
  1e-6,
  1e-8,
  1e-8
)

calibration_model$calibrate(
  calibration_helper_vector,
  QuantLib::LevenbergMarquardt(),
  calibration_end_criteria,
  QuantLib::NoConstraint(),
  numeric(),
  c(
    TRUE,
    FALSE
  )
)
#> NULL

calibration_parameter_array <- calibration_model$params()

calibrated_parameters <- c(
  as.numeric(calibration_parameter_array[1]),
  as.numeric(calibration_parameter_array[2])
)

calibration_result_tbl <- tibble::tribble(
```

```

~parameter,      ~value,
"a_fixed",        calibrated_parameters[[1]],
"sigma_calibrated", calibrated_parameters[[2]],
"market_normal_vol", market_normal_volatility
)

GaussQuant::show_tbl_GQH(
  calibration_result_tbl,
  "Hull-White one-helper calibration",
  n = 20
)

#> # A tibble: 3 x 2
#>   parameter      value
#>   <chr>          <dbl>
#> 1 a_fixed        0.03
#> 2 sigma_calibrated 0.0033441
#> 3 market_normal_vol 0.003606
calibration_parameter_array <- calibration_model$params()

calibration_result_tbl <- tibble::tribble(
  ~parameter,      ~value,
  "a_fixed",        calibrated_parameters[[1]],
  "sigma_calibrated", calibrated_parameters[[2]],
  "market_normal_vol", market_normal_volatility
)

GaussQuant::show_tbl_GQH(
  calibration_result_tbl,
  "Hull-White one-helper calibration",
  n = 20
)

#> # A tibble: 3 x 2
#>   parameter      value
#>   <chr>          <dbl>
#> 1 a_fixed        0.03
#> 2 sigma_calibrated 0.0033441
#> 3 market_normal_vol 0.003606

```

c(TRUE, FALSE) は、最初のパラメータ a を固定し、2 番目のパラメータ σ だけを最適化する指定である。

7.6.4 European Swaption の Engine 比較

カリブレーションに用いた 1Y×5Y ATM Swap を原資産とする European Payer Swaption を作成し、次の 3 つの方法で評価する。

- Jamshidian decomposition
- Hull-White Tree
- Bachelier analytic engine

Jamshidian と Tree は同じカリブレーション後の Hull-White モデルを使用する。Bachelier Engine は市場 Normal volatility を直接使用する。

```
calibration_underlying_swap <- QuantLib::VanillaSwap(
  QuantLib::Swap_Payer_get(),
  1,
  calibration_schedule,
  calibration_forward_swap_rate,
  calibration_day_counter,
  calibration_schedule,
  calibration_index,
  0,
  calibration_day_counter
)

calibration_european_swaption <- QuantLib::Swaption(
  calibration_underlying_swap,
  QuantLib::EuropeanExercise(
    swaption_expiry_date
  )
)

jamshidian_engine <- QuantLib::JamshidianSwaptionEngine(
  calibration_model
)

calibration_european_swaption$setPricingEngine(
  jamshidian_engine
)

#> NULL

npv_jamshidian <- GaussQuant::instrument_npv_safe_GQL(
  calibration_european_swaption
)

tree_engine_500 <- QuantLib::TreeSwaptionEngine(
  calibration_model,
  500L
)

calibration_european_swaption$setPricingEngine(
```

```

    tree_engine_500
  )
#> NULL

npv_tree_500 <- GaussQuant::instrument_npv_safe_GQL(
  calibration_european_swaption
)

bachelier_engine <- QuantLib::BachelierSwaptionEngine(
  flat_curve_handle,
  QuantLib::QuoteHandle(
    QuantLib::SimpleQuote(
      market_normal_volatility
    )
  )
)

calibration_european_swaption$setPricingEngine(
  bachelier_engine
)
#> NULL

npv_bachelier <- GaussQuant::instrument_npv_safe_GQL(
  calibration_european_swaption
)

engine_comparison_tbl <- tibble::tribble(
  ~engine,          ~npv,
  "Jamshidian",     npv_jamshidian,
  "Hull-White tree 500", npv_tree_500,
  "Bachelier",      npv_bachelier
) |>
  dplyr::mutate(
    difference_vs_bachelier = .data$npv -
      npv_bachelier,
    difference_vs_jamshidian = .data$npv -
      npv_jamshidian
  )

GaussQuant::show_tbl_GQH(
  engine_comparison_tbl,
  "1Yx5Y swaption engine comparison",
  n = 20

```

```
)
#> # A tibble: 3 x 4
#>   engine      npv difference_vs_bachelier difference_vs_jamshidian
#>   <chr>      <dbl>          <dbl>          <dbl>
#> 1 Jamshidian      0.0055519      -5.8042e-8          0
#> 2 Hull-White tree 500 0.0055480      -3.9537e-6     -3.8956e-6
#> 3 Bachelier      0.0055519          0          5.8042e-8
```

カリブレーション対象と同じ ATM Swaption であるため、Hull-White 価格と Bachelier 市場価格は近い値になることが期待される。Jamshidian と十分に細かい Tree の差は、主に Tree 離散化誤差を表す。

7.6.5 Tree step の収束

Hull-White Tree では、時間軸と金利状態を有限個の node へ離散化する。Tree step が少ない場合、金利分布とオプション Payoff の非線形性を粗くしか表現できない。

同じ European Swaption を異なる Tree step で評価し、Jamshidian 価格への収束を確認する。

```
tree_step_grid <- c(
  6L,
  12L,
  24L,
  60L,
  120L,
  240L,
  500L
)

tree_convergence_tbl <- purrr::map_dfr(
  tree_step_grid,
  function(tree_step) {
    tree_engine_i <- QuantLib::TreeSwaptionEngine(
      calibration_model,
      tree_step
    )

    calibration_european_swaption$setPricingEngine(
      tree_engine_i
    )

    tree_npv_i <- GaussQuant::instrument_npv_safe_GQL(
      calibration_european_swaption
    )

    tibble::tibble(
```

```

    tree_steps = tree_step,
    npv = tree_npv_i,
    difference_vs_jamshidian = tree_npv_i -
      npv_jamshidian,
    difference_vs_bachelier = tree_npv_i -
      npv_bachelier
  )
}
)

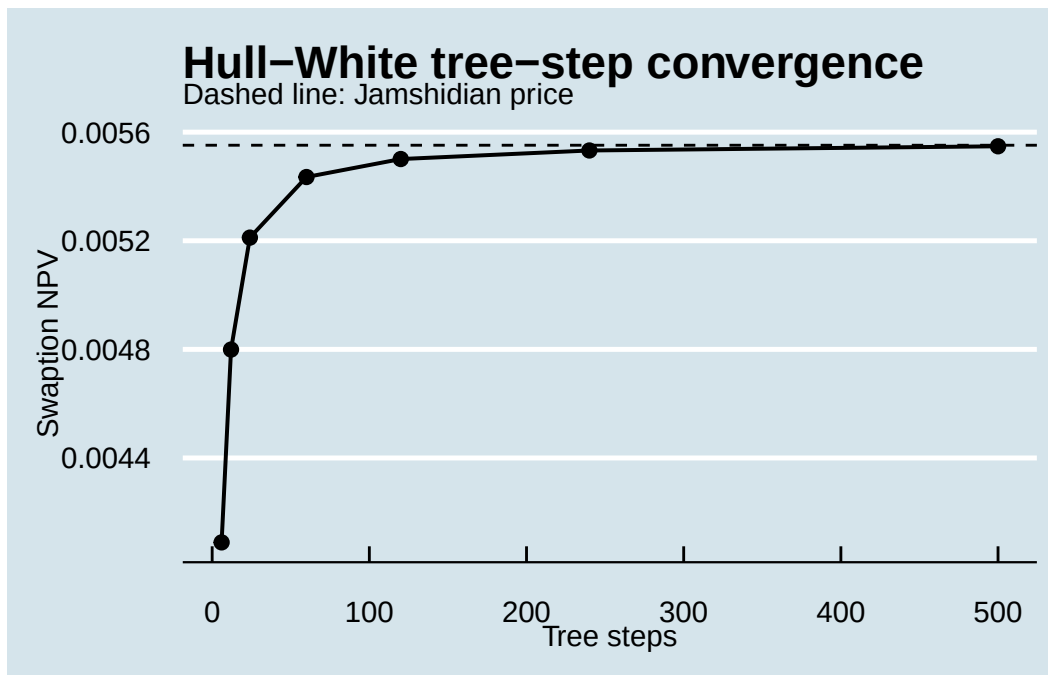
tree_convergence_plot <- ggplot2::ggplot(
  tree_convergence_tbl,
  ggplot2::aes(
    x = .data$tree_steps,
    y = .data$npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::geom_hline(
    yintercept = npv_jamshidian,
    linetype = "dashed"
  ) +
  ggplot2::labs(
    title = "Hull-White tree-step convergence",
    subtitle = "Dashed line: Jamshidian price",
    x = "Tree steps",
    y = "Swaption NPV"
  )

GaussQuant::show_tbl_GQH(
  tree_convergence_tbl,
  "Hull-White tree convergence",
  n = 20
)

#> # A tibble: 7 x 4
#>   tree_steps      npv difference_vs_jamshidian difference_vs_bachelier
#>   <int>      <dbl>                <dbl>                <dbl>
#> 1         6 0.0040890                -0.0014628                -0.0014629

```

```
#> 2      12 0.0047992      -0.00075265      -0.00075270
#> 3      24 0.0052116      -0.00034028      -0.00034034
#> 4      60 0.0054347      -0.00011714      -0.00011720
#> 5     120 0.0055010      -0.000050834     -0.000050892
#> 6     240 0.0055325      -0.000019365     -0.000019423
#> 7     500 0.0055480      -0.0000038956    -0.0000039537
show_plot_LFL(tree_convergence_plot)
```



Tree step を増やすほど、Tree 価格が Jamshidian 価格へ近づくことを確認する。Bermudan Swaption や Callable Bond には一般的な Jamshidian 解析解を利用できないため、European Swaption で Tree 精度を検証してから、早期行使商品へ同じ Tree 設定を適用することが重要である。

7.7 まとめ

Callable Bond は、Bullet Bond から発行体の期限前償還権を差し引いた商品として理解できる。金利が低下すると発行体のコール権は価値を持ち、投資家が保有する Callable Bond の価格上昇は抑制される。

発行体のコール権は、金利低下時に価値が上昇する Receiver Swaption と同じ方向の金利オプション性を持つ。複数のコール日がある場合には Bermudan Receiver Swaption に近い。ただし、債券と Swap では元本、決済およびキャッシュフロー条件が完全には一致しないため、価格関係は近似として解釈する必要がある。

Hull-White モデルは Gaussian 平均回帰型の短期金利モデルであり、時間依存ドリフトによって初期イールドカーブへ適合できる。Vasicek モデルがパラメータから期間構造を生成するのに対し、Hull-White モデルは観測カーブを出発点として、その周りの確率変動を表現する。

Hull-White の平均回帰速度 a は金利ショックの持続性を、 σ は短期金利変動の大きさを表す。本章では、1Y×5Y ATM Swaption の Normal volatility を使い、 a を固定して σ をカリブレーションした。

European Swaption では、Jamshidian decomposition、Hull-White Tree および Bachelier Engine を比較し

た。Tree step を増やすと Tree 価格は Jamshidian 価格へ近づく。Callable Bond や Bermudan Swaption のような早期行使商品では、この Tree 精度を確認したうえで評価する必要がある。

```
chapter07_summary_tbl <- tibble::tribble(
  ~section,          ~main_result,
  "Callable Bond",    "Bullet bond minus issuer call option",
  "Receiver Swaption", "Interest-rate direction of the issuer call",
  "Vasicek and Hull-White", "Initial-curve fit and mean reversion",
  "Calibration",      "Fixed a and calibrated sigma",
  "Tree convergence",  "Convergence toward Jamshidian pricing"
)

GaussQuant::show_tbl_GQH(
  chapter07_summary_tbl,
  "Chapter II-7 summary",
  n = 20
)

#> # A tibble: 5 x 2
#>   section          main_result
#>   <chr>           <chr>
#> 1 Callable Bond    Bullet bond minus issuer call option
#> 2 Receiver Swaption Interest-rate direction of the issuer call
#> 3 Vasicek and Hull-White Initial-curve fit and mean reversion
#> 4 Calibration      Fixed a and calibrated sigma
#> 5 Tree convergence Convergence toward Jamshidian pricing

cat(
  "\nChapter II-7 Callable Bond and Hull-White model completed successfully.\n"
)

#>
#> Chapter II-7 Callable Bond and Hull-White model completed successfully.
```


第 II-8 章信用リスクと CDS

前章までの評価では、将来の市場価格、金利またはボラティリティの変動を扱った。本章では、契約相手または参照企業が債務を履行できなくなる信用リスクを検討する。

Credit Default Swap (CDS) は、参照企業などに信用事由が発生した場合に、プロテクションの売り手が買い手へ損失補償を支払う契約である。プロテクションの買い手は、その対価として契約期間中に定期的な Premium を支払う。

したがって、CDS は二つのレグから構成される。

- Premium Leg：プロテクション買い手が定期的な Spread を支払う。
- Protection Leg：信用事由が発生した場合に、プロテクション売り手が損失を補償する。

プロテクション買い手から見た CDS 価値は、次のように表される。

$$V_{\text{CDS}} = V_{\text{protection}} + V_{\text{premium}}$$

Premium Leg は支払いであるため通常は負、Protection Leg は受取りであるため正となる。

CDS 評価には、割引カーブだけでなく、将来のデフォルト確率を表す Default Probability Curve が必要である。本章では、Hazard Rate から生存確率とデフォルト確率を求め、Premium Leg と Protection Leg を評価する。

まず、フラット割引率とフラット Hazard Rate を使用して、MidPoint 法による CDS 評価と手計算を比較する。次に、標準 CDS2015 契約を ISDA CDS Engine で評価し、Fair spread、Fair upfront および Accrual rebateを確認する。

```
knitr::opts_chunk$set(
  collapse = TRUE,
  comment = "#>",
  warning = FALSE,
  message = FALSE
)

suppressPackageStartupMessages({
  library(tidyverse)
  library(ggthemes)
  library(QuantLib)
```

```
library(GaussQuant)
library(tidyverse)
library(broom)
})
```

8.1 信用リスクと CDS の基礎

8.1.1 デフォルト時刻と生存確率

デフォルトが発生する確率時刻を τ とする。時点 t まで企業がデフォルトせずに存続する確率を、生存確率と呼ぶ。

$$Q(t) = P(\tau > t)$$

時点 0 から t までの累積デフォルト確率は、

$$P(\tau \leq t) = 1 - Q(t)$$

である。

Hazard Rate $\lambda(t)$ は、時点 t まで存続しているという条件の下で、その直後にデフォルトする瞬間的な強度を表す。

$$\lambda(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t < \tau \leq t + \Delta t \mid \tau > t)}{\Delta t}$$

Hazard Rate が一定のフラットモデルでは、生存確率は次式となる。

$$Q(t) = \exp(-\lambda t)$$

累積デフォルト確率は、

$$1 - Q(t) = 1 - \exp(-\lambda t)$$

である。

Hazard Rate は、通常の意味での「一年間のデフォルト確率」そのものではない。たとえば Hazard Rate が 2% であっても、1 年累積デフォルト確率は、

$$1 - e^{-0.02}$$

であり、厳密には 2% と一致しない。

8.1.2 Recovery Rate と Loss Given Default

デフォルトが発生した場合でも、債権価値のすべてが失われるとは限らない。デフォルト後に回収できる元本比率を Recovery Rate R とする。

損失率 Loss Given Default は、

$$LGD = 1 - R$$

である。

Notional を N とすると、Protection Leg が補償する損失額は概ね、

$$N(1 - R)$$

となる。

Recovery Rate が高いほどデフォルト時の損失は小さくなり、同じデフォルト確率であれば CDS Spread は低くなる。

8.1.3 CDS Spread と Hazard Rate の近似

連続的な Premium 支払い、一定 Hazard Rate、一定 Recovery Rate および単純な割引を仮定すると、CDS Spread s と Hazard Rate λ には次の近似関係がある。

$$s \approx (1 - R)\lambda$$

したがって、

$$\lambda \approx \frac{s}{1 - R}$$

となる。

この近似は CDS の信用水準を理解するために有用である。ただし、実際の評価では、四半期ごとの支払い、Accrual on default、割引、標準日付およびカーブ補間が影響するため、厳密な Hazard Rate とは一致しない。

8.2 フラットカーブによる CDS 評価

8.2.1 フラットカーブによる 1 年物 CDS

1 年物 CDS を考える。

- 評価日：2022 年 9 月 19 日
- 満期：2023 年 9 月 20 日

- Notional : 10,000,000
- 割引率 : 10%
- 市場 Spread : 130bp
- 契約 Coupon : 100bp
- Recovery Rate : 40%

市場 Spread は 130bp であるが、契約 Coupon は 100bp である。このため、契約時点の CDS 価値はゼロにならない。

```
trade_date_flat <- GaussQuant::date_GQL("2022-09-19")
maturity_date_flat <- GaussQuant::date_GQL("2023-09-20")

GaussQuant::set_eval_date_GQL(trade_date_flat)

notional_flat <- 10000000
risk_free_rate_flat <- 0.10
market_spread_flat <- 130 / 10000
contract_coupon_flat <- 100 / 10000
recovery_rate_flat <- 0.40

actual_365 <- QuantLib::Actual365Fixed()
actual_360 <- QuantLib::Actual360()
weekend_calendar <- QuantLib::WeekendsOnly()

flat_cds_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2022-09-19",
  "maturity_date", "2023-09-20",
  "notional",    as.character(notional_flat),
  "risk_free_rate", as.character(risk_free_rate_flat),
  "market_spread", as.character(market_spread_flat),
  "contract_coupon", as.character(contract_coupon_flat),
  "recovery_rate", as.character(recovery_rate_flat)
)

GaussQuant::show_tbl_GQH(
  flat_cds_input_tbl,
  "Flat CDS input assumptions",
  n = 20
)

#> # A tibble: 7 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2022-09-19
#> 2 maturity_date 2023-09-20
```

```
#> 3 notional      10000000
#> 4 risk_free_rate 0.1
#> 5 market_spread  0.013
#> 6 contract_coupon 0.01
#> 7 recovery_rate   0.4
```

8.2.2 Hazard Rate 近似と確率曲線

市場 Spread 130bp と Recovery Rate 40% から、フラット Hazard Rate を近似する。

```
hazard_rate_approximation <- market_spread_flat /
  (1 - recovery_rate_flat)

hazard_approximation_tbl <- tibble::tribble(
  ~metric,          ~value,
  "market_spread",  market_spread_flat,
  "recovery_rate",  recovery_rate_flat,
  "loss_given_default", 1 - recovery_rate_flat,
  "hazard_rate_approx", hazard_rate_approximation
)

probability_time_grid <- seq(
  0,
  5,
  by = 0.25
)

flat_probability_tbl <- tibble::tibble(
  time = probability_time_grid
) |>
  dplyr::mutate(
    survival_probability = exp(
      -hazard_rate_approximation * .data$time
    ),
    cumulative_default_probability = 1 -
      .data$survival_probability
  )

flat_probability_plot_tbl <- flat_probability_tbl |>
  tidyr::pivot_longer(
    cols = c(
      survival_probability,
      cumulative_default_probability
    ),
```

```

    names_to = "probability_type",
    values_to = "probability"
  )

flat_probability_plot <- ggplot2::ggplot(
  flat_probability_plot_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$probability,
    linetype = .data$probability_type
  )
) +
ggplot2::geom_line(
  linewidth = 0.7
) +
ggplot2::scale_y_continuous(
  labels = scales::percent_format(
    accuracy = 0.1
  )
) +
ggplot2::labs(
  title = "Flat-hazard survival and default probabilities",
  x = "Time (years)",
  y = "Probability",
  linetype = "Probability"
)

GaussQuant::show_tbl_GQH(
  hazard_approximation_tbl,
  "Flat hazard-rate approximation",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 market_spread    0.013
#> 2 recovery_rate    0.4
#> 3 loss_given_default 0.6
#> 4 hazard_rate_approx 0.021667

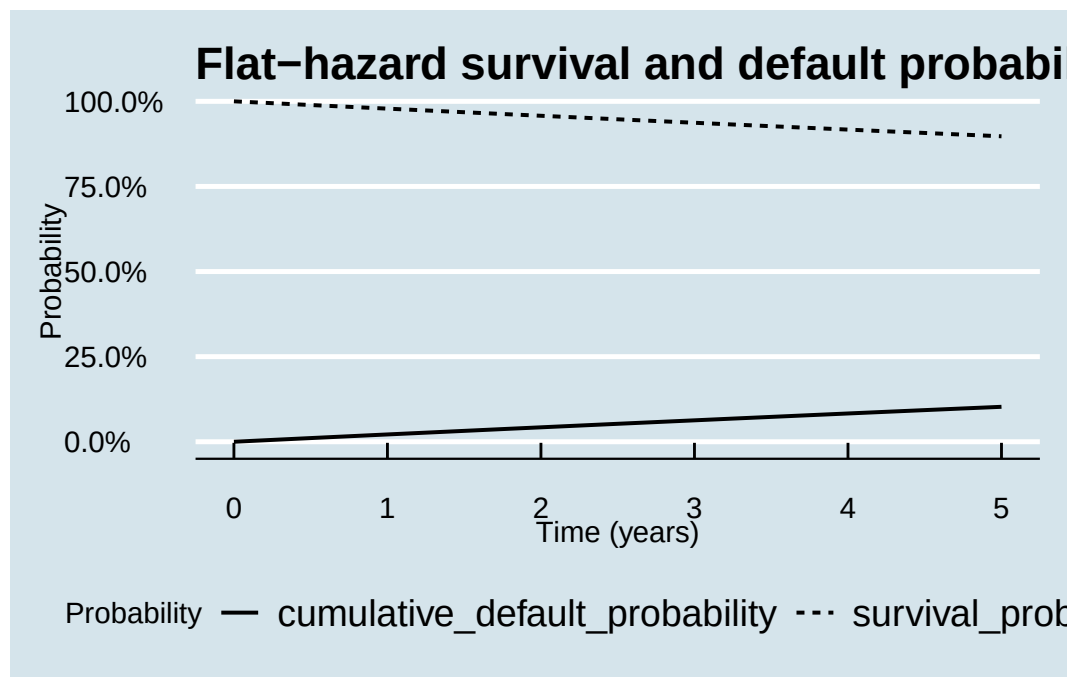
GaussQuant::show_tbl_GQH(
  flat_probability_tbl,
  "Flat-hazard probability curve",

```

```

n = 30
)
#> # A tibble: 21 x 3
#>   time survival_probability cumulative_default_probability
#>   <dbl>          <dbl>          <dbl>
#> 1  0          1          0
#> 2  0.25      0.99460      0.0054020
#> 3  0.5       0.98923      0.010775
#> 4  0.75      0.98388      0.016119
#> 5  1         0.97857      0.021434
#> 6  1.25      0.97328      0.026720
#> 7  1.5       0.96802      0.031978
#> 8  1.75      0.96279      0.037207
#> 9  2         0.95759      0.042408
#> 10 2.25      0.95242      0.047581
#> # i 11 more rows
show_plot_LFL(flat_probability_plot)

```



8.2.3 CDS Schedule とカーブ

Premium は四半期ごとに支払われるものとする。単純な教育用例として、評価日の翌日から満期までの Forward Schedule を作成する。

```

cds_schedule_flat <- QuantLib::Schedule(
  trade_date_flat + 1L,
  maturity_date_flat,
  GaussQuant::period_months_GQL(3),

```

```

weekend_calendar,
"Following",
"Unadjusted",
"Forward",
FALSE
)

discount_curve_flat <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date_flat,
  rate = risk_free_rate_flat,
  day_counter = actual_365,
  compounding = "Continuous"
)

discount_curve_handle_flat <- QuantLib::YieldTermStructureHandle(
  discount_curve_flat
)

hazard_curve_flat <- QuantLib::FlatHazardRate(
  trade_date_flat,
  GaussQuant::quote_handle_GQL(
    hazard_rate_approximation
  ),
  actual_365
)

hazard_curve_handle_flat <- QuantLib::DefaultProbabilityTermStructureHandle(
  hazard_curve_flat
)

flat_curve_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "discount_factor_at_maturity", discount_curve_flat$discount(
    maturity_date_flat
  ),
  "survival_probability_at_maturity", hazard_curve_flat$survivalProbability(
    maturity_date_flat
  ),
  "default_probability_to_maturity", 1 -
    hazard_curve_flat$survivalProbability(
    maturity_date_flat
  ),
  "hazard_rate_at_maturity", hazard_curve_flat$hazardRate(

```



```

                                maturity_date_flat
                                )
)

GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    cds_schedule_flat
  ),
  "CDS schedule: flat-curve example",
  n = 20
)

#> # A tibble: 5 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2022-09-20
#> 2     2 2022-12-20
#> 3     3 2023-03-20
#> 4     4 2023-06-20
#> 5     5 2023-09-20

GaussQuant::show_tbl_GQH(
  flat_curve_check_tbl,
  "Flat discount and hazard-curve check",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 discount_factor_at_maturity          0.90459
#> 2 survival_probability_at_maturity 0.97851
#> 3 default_probability_to_maturity    0.021492
#> 4 hazard_rate_at_maturity             0.021667

```

8.2.4 MidPoint CDS Engine

MidPoint 法では、各 Premium 期間中に発生するデフォルトを、その期間の midpoint で発生したものとして近似する。

100bp Coupon の CDS を作成し、GaussQuant の MidPoint CDS Engine で評価する。

```

cds_flat <- QuantLib::CreditDefaultSwap(
  GaussQuant::cds_buyer_side_GQL(),
  notional_flat,
  contract_coupon_flat,

```

```

    cds_schedule_flat,
    "Following",
    actual_360
)

midpoint_engine_flat <- GaussQuant::cds_midpoint_engine_GQL(
  probability_handle = hazard_curve_handle_flat,
  recovery_rate = recovery_rate_flat,
  discount_handle = discount_curve_handle_flat
)

cds_flat$setPricingEngine(
  midpoint_engine_flat
)
#> NULL

cds_flat_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_flat
)

cds_flat_leg_check_tbl <- tibble::tribble(
  ~metric,      ~value,
  "coupon_leg_npv",    cds_flat$couponLegNPV(),
  "default_leg_npv",   cds_flat$defaultLegNPV(),
  "sum_of_two_legs",   cds_flat$couponLegNPV() +
                        cds_flat$defaultLegNPV(),
  "reported_npv",     GaussQuant::cds_npv_GQL(
                        cds_flat
                        ),
  "difference",        GaussQuant::cds_npv_GQL(
                        cds_flat
                        ) -
                        (
                          cds_flat$couponLegNPV() +
                          cds_flat$defaultLegNPV()
                        )
)

GaussQuant::show_tbl_GQH(
  cds_flat_summary_tbl,
  "CDS summary: MidPoint engine with flat curves",
  n = 20
)

```

```
#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 npv            28116.
#> 2 fair_spread      0.012983
#> 3 coupon_leg_npv  -94246.
#> 4 default_leg_npv 122362.
```

```
GaussQuant::show_tbl_GQH(
  cds_flat_leg_check_tbl,
  "CDS two-leg NPV check",
  n = 20
)
```

```
#> # A tibble: 5 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_npv  -94246.
#> 2 default_leg_npv 122362.
#> 3 sum_of_two_legs 28116.
#> 4 reported_npv    28116.
#> 5 difference        0
```

プロテクション買い手から見て、Premium Leg は負、Protection Leg は正となる。契約 Coupon が Fair Spread より低ければ、プロテクション買い手に有利となり、CDS NPV は一般に正となる。

8.2.5 CDS キャッシュフローと確率表

GaussQuant の `cds_cashflow_table_GQL()` を利用し、各 Premium 期間の割引係数、生存確率、デフォルト確率および期間中点の情報を確認する。

```
cds_cashflow_flat_tbl <- GaussQuant::cds_cashflow_table_GQL(
  cds_schedule = cds_schedule_flat,
  protection_start_date = cds_flat$protectionStartDate(),
  coupon_rate = contract_coupon_flat,
  hazard_curve = hazard_curve_flat,
  discount_curve = discount_curve_flat,
  trade_date = trade_date_flat,
  notional = notional_flat
)
```

```
GaussQuant::show_tbl_GQH(
  cds_cashflow_flat_tbl,
  "CDS cash-flow and probability table",
  n = 20
)
```

```

)
#> # A tibble: 5 x 13
#>   payment_date coupon_rate accrual_start accrual_end accrual_days
#>   <chr>          <dbl> <chr>          <chr>          <dbl>
#> 1 2022-09-20      NA    <NA>          2022-09-20      NA
#> 2 2022-12-20      0.01 2022-09-20    2022-12-20      91
#> 3 2023-03-20      0.01 2022-12-20    2023-03-20      90
#> 4 2023-06-20      0.01 2023-03-20    2023-06-20      92
#> 5 2023-09-20      0.01 2023-06-20    2023-09-20      92
#> # i 8 more variables: accrual_year_fraction_365 <dbl>, coupon_amount <dbl>,
#> #   discount_factor <dbl>, survival_probability <dbl>,
#> #   default_probability <dbl>, midpoint_date <chr>,
#> #   midpoint_discount_factor <dbl>, midpoint_survival_probability <dbl>

```

8.3 Premium Leg と Protection Leg

8.3.1 Premium Leg の手計算

Premium Leg は、参照企業が各支払日まで存続した場合に支払われる。

単純化した現在価値は次式で表される。

$$V_{\text{premium}} \approx -Ns \sum_i \alpha_i D(0, t_i) Q(t_i)$$

ここで、 N は Notional、 s は契約 Coupon、 α_i は利息計算期間、 $D(0, t_i)$ は割引係数、 $Q(t_i)$ は生存確率である。

```

premium_leg_hand_flat <- -sum(
  cds_cashflow_flat_tbl$coupon_amount *
  cds_cashflow_flat_tbl$discount_factor *
  cds_cashflow_flat_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

```

この近似では期間中点の生存確率を使用している。実際の CDS では、期間中にデフォルトした場合の経過 Premium である Accrual on default も考慮される。

8.3.2 Protection Leg の手計算

期間 $(t_{i-1}, t_i]$ にデフォルトする確率を、

$$\Delta PD_i = Q(t_{i-1}) - Q(t_i)$$

とする。

Protection Leg は、各期間の中点 m_i で損失補償が支払われると近似する。

$$V_{\text{protection}} \approx N(1 - R) \sum_i D(0, m_i) \Delta PD_i$$

```
protection_leg_per_unit_hand_flat <- (
  1 - recovery_rate_flat
) *
sum(
  cds_cashflow_flat_tbl$midpoint_discount_factor *
  cds_cashflow_flat_tbl$default_probability,
  na.rm = TRUE
)

protection_leg_hand_flat <- notional_flat *
  protection_leg_per_unit_hand_flat
```

8.3.3 Risky PV01 と Fair Spread

Risky PV01 は、CDS Spread を 1 単位支払う Premium Leg の現在価値に相当する。

$$RPV01 \approx \sum_i \alpha_i D(0, t_i) Q(t_i)$$

Fair Spread は、Premium Leg と Protection Leg の価値を等しくし、Upfront なしの CDS NPV をゼロにする Running Spread である。

$$s_{\text{fair}} = \frac{V_{\text{protection}}/N}{RPV01}$$

```
risky_pv01_hand_flat <- sum(
  (
    cds_cashflow_flat_tbl$accrual_days /
    360
  ) *
  cds_cashflow_flat_tbl$discount_factor *
  cds_cashflow_flat_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

fair_spread_hand_flat <- protection_leg_per_unit_hand_flat /
  risky_pv01_hand_flat

cds_npv_hand_flat <- premium_leg_hand_flat +
  protection_leg_hand_flat
```

```

flat_hand_check_tbl <- tibble::tribble(
  ~metric,          ~hand_value,          ~quantlib_value,
  "coupon_leg_npv", premium_leg_hand_flat,    cds_flat$couponLegNPV(),
  "default_leg_npv", protection_leg_hand_flat, cds_flat$defaultLegNPV(),
  "cds_npv",        cds_npv_hand_flat,      GaussQuant::cds_npv_GQL(
                                cds_flat
                                ),
  "fair_spread",    fair_spread_hand_flat,    GaussQuant::cds_fair_spread_GQL(
                                cds_flat
                                )
) |>
dplyr::mutate(
  difference = .data$hand_value -
    .data$quantlib_value
)

flat_hand_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "risky_pv01",     risky_pv01_hand_flat,
  "protection_leg_per_unit", protection_leg_per_unit_hand_flat,
  "hand_fair_spread", fair_spread_hand_flat,
  "quantlib_fair_spread", GaussQuant::cds_fair_spread_GQL(
                                cds_flat
                                ),
  "fair_spread_difference", fair_spread_hand_flat -
                                GaussQuant::cds_fair_spread_GQL(
                                    cds_flat
                                )
)

GaussQuant::show_tbl_GQH(
  flat_hand_check_tbl,
  "CDS hand calculation and MidPoint engine comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric          hand_value quantlib_value    difference
#>   <chr>          <dbl>         <dbl>         <dbl>
#> 1 coupon_leg_npv -94244.         -94246.         2.0964
#> 2 default_leg_npv 122362.         122362.          0
#> 3 cds_npv        28118.          28116.         2.0964
#> 4 fair_spread     0.012984         0.012983 0.00000028881

```

```
GaussQuant::show_tbl_GQH(
  flat_hand_summary_tbl,
  "CDS hand-calculation summary",
  n = 20
)
#> # A tibble: 5 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 risky_pv01                          0.94244
#> 2 protection_leg_per_unit 0.012236
#> 3 hand_fair_spread              0.012984
#> 4 quantlib_fair_spread          0.012983
#> 5 fair_spread_difference 0.00000028881
```

手計算と MidPoint Engine は近い値となるが、厳密には一致しない。差には Accrual on default、日付規則、Protection 開始日および QuantLib 内部の計算規約が含まれる。

8.4 Hazard Rate 推定と感応度

8.4.1 Zero-NPV CDS から Hazard Rate を推定する

市場 Spread を契約 Coupon として設定した CDS は、契約開始時点で NPV がゼロとなるように Hazard Rate を調整できる。

ここでは、市場 Spread 130bp の CDS について、MidPoint Engine の NPV がゼロとなるフラット Hazard Rate を `uniroot()` で推定する。

```
cds_market_flat <- QuantLib::CreditDefaultSwap(
  GaussQuant::cds_buyer_side_GQL(),
  notional_flat,
  market_spread_flat,
  cds_schedule_flat,
  "Following",
  actual_360
)

cds_market_npv_at_hazard <- function(hazard_rate) {
  engine <- GaussQuant::cds_midpoint_engine_GQL(
    recovery_rate = recovery_rate_flat,
    hazard_rate = hazard_rate,
    discount_rate = risk_free_rate_flat,
    reference_date = trade_date_flat,
    day_counter = "Actual365Fixed"
  )
}
```

```

    cds_market_flat$setPricingEngine(
      engine
    )

    GaussQuant::cds_npv_GQL(
      cds_market_flat
    )
  }

hazard_calibration_result <- stats::uniroot(
  cds_market_npv_at_hazard,
  interval = c(
    1e-8,
    0.50
  ),
  tol = 1e-10
)

hazard_rate_calibrated <- hazard_calibration_result$root

calibrated_midpoint_engine <- GaussQuant::cds_midpoint_engine_GQL(
  recovery_rate = recovery_rate_flat,
  hazard_rate = hazard_rate_calibrated,
  discount_rate = risk_free_rate_flat,
  reference_date = trade_date_flat,
  day_counter = "Actual365Fixed"
)

cds_market_flat$setPricingEngine(
  calibrated_midpoint_engine
)

#> NULL

hazard_calibration_tbl <- tibble::tribble(
  ~metric,                ~value,
  "market_spread",        market_spread_flat,
  "simple_hazard_approximation", hazard_rate_approximation,
  "calibrated_hazard_rate", hazard_rate_calibrated,
  "zero_npv_check",       GaussQuant::cds_npv_GQL(
                                cds_market_flat
                              ),
  "fair_spread_check",    GaussQuant::cds_fair_spread_GQL(
                                cds_market_flat
                              )
)

```



```

    )
)

GaussQuant::show_tbl_GQH(
  hazard_calibration_tbl,
  "Flat hazard calibration from a zero-NPV CDS",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 market_spread                        0.013
#> 2 simple_hazard_approximation 0.021667
#> 3 calibrated_hazard_rate            0.021695
#> 4 zero_npv_check                    0.00000036661
#> 5 fair_spread_check                  0.013000

```

単純近似 $\lambda \approx s/(1 - R)$ とカリブレーション結果は近いが、支払頻度と割引の影響により差が生じる。

8.4.2 Spread と Recovery Rate の感応度

単純近似の下では、Spread が高いほど Hazard Rate は高くなる。また、同じ Spread で Recovery Rate を高く仮定すると、デフォルト時損失が小さくなるため、その Spread を説明するために必要な Hazard Rate は高くなる。

```

credit_sensitivity_input_tbl <- tidyr::crossing(
  market_spread_bp = c(
    50,
    100,
    200
  ),
  recovery_rate = c(
    0.20,
    0.40,
    0.60
  )
)

credit_sensitivity_tbl <- credit_sensitivity_input_tbl |>
  dplyr::mutate(
    market_spread = .data$market_spread_bp /
      10000,
    loss_given_default = 1 -
      .data$recovery_rate,

```

```

hazard_rate_approx = .data$market_spread /
  .data$loss_given_default,
survival_probability_5y = exp(
  -5 * .data$hazard_rate_approx
),
cumulative_default_probability_5y = 1 -
  .data$survival_probability_5y
)

GaussQuant::show_tbl_GQH(
  credit_sensitivity_tbl,
  "Spread and recovery-rate sensitivity",
  n = 30
)

#> # A tibble: 9 x 7
#>   market_spread_bp recovery_rate market_spread loss_given_default
#>   <dbl>           <dbl>           <dbl>           <dbl>
#> 1           50         0.2           0.005           0.8
#> 2           50         0.4           0.005           0.6
#> 3           50         0.6           0.005           0.4
#> 4          100         0.2           0.01            0.8
#> 5          100         0.4           0.01            0.6
#> 6          100         0.6           0.01            0.4
#> 7          200         0.2           0.02            0.8
#> 8          200         0.4           0.02            0.6
#> 9          200         0.6           0.02            0.4
#> # i 3 more variables: hazard_rate_approx <dbl>, survival_probability_5y <dbl>,
#> #   cumulative_default_probability_5y <dbl>

```

Recovery Rate と Hazard Rate は、CDS Spread だけから完全に独立して識別できない。実務では Recovery Rate を外生的に仮定し、その仮定の下で Default Probability Curve を構築することが多い。

8.5 標準 CDS2015 と ISDA カーブ

8.5.1 標準 CDS と CDS2015 満期

標準 CDS では、満期を単純に取引日から 5 年後とするのではなく、CDS IMM 日と CDS2015 ルールに従って決定する。

次の 5 年物 CDS を考える。

- 取引日：2022 年 8 月 19 日
- 市場 Spread：30.426bp
- 契約 Coupon：100bp
- Notional：10,000,000

- Recovery Rate : 40%
- 外生的な OIS 割引率 : 0.25%

第 II-3 章では OIS カーブの構築を扱った。本章の目的は信用カーブと CDS 評価であるため、割引カーブの動学やブートストラップを繰り返さず、フラット OIS 割引率を外生的に与える。

```
trade_date_isda <- GaussQuant::date_GQL(
  "2022-08-19"
)

GaussQuant::set_eval_date_GQL(
  trade_date_isda
)

cds_tenor_isda <- GaussQuant::period_years_GQL(
  5L
)

maturity_date_isda <- QuantLib::cdsMaturity(
  trade_date_isda,
  cds_tenor_isda,
  "CDS2015"
)

market_spread_isda <- 30.426 / 10000
contract_coupon_isda <- 100 / 10000
notional_isda <- 10000000
recovery_rate_isda <- 0.40
ois_discount_rate_isda <- 0.0025

isda_cds_input_tbl <- tibble::tribble(
  ~input,      ~value,
  "trade_date", "2022-08-19",
  "cds_tenor",  "5Y",
  "maturity_date", GaussQuant::iso_GQL(
    maturity_date_isda
  ),
  "market_spread_bp", as.character(
    market_spread_isda *
    10000
  ),
  "contract_coupon_bp", as.character(
    contract_coupon_isda *
    10000
  ),
)
```

```

    "notional",          as.character(
                          notional_isda
                        ),
    "recovery_rate",     as.character(
                          recovery_rate_isda
                        ),
    "ois_discount_rate", as.character(
                          ois_discount_rate_isda
                        )
  )

GaussQuant::show_tbl_GQH(
  isda_cds_input_tbl,
  "ISDA CDS input assumptions",
  n = 20
)

#> # A tibble: 8 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 trade_date 2022-08-19
#> 2 cds_tenor  5Y
#> 3 maturity_date 2027-06-20
#> 4 market_spread_bp 30.426
#> 5 contract_coupon_bp 100
#> 6 notional 10000000
#> 7 recovery_rate 0.4
#> 8 ois_discount_rate 0.0025

```

8.5.2 標準 CDS2015 Schedule

CDS2015 ルールにより、Premium 支払日は標準化された四半期日へ調整される。

```

cds_calendar_isda <- QuantLib::WeekendsOnly()

cds_schedule_isda <- QuantLib::Schedule(
  trade_date_isda,
  maturity_date_isda,
  GaussQuant::period_months_GQL(3),
  cds_calendar_isda,
  "Following",
  "Unadjusted",
  "CDS2015",
  FALSE
)

```

```
GaussQuant::show_tbl_GQH(
  GaussQuant::schedule_table_GQL(
    cds_schedule_isda
  ),
  "CDS schedule: 5Y CDS2015 example",
  n = 30
)
#> # A tibble: 21 x 2
#>   period schedule_date
#>   <int> <date>
#> 1     1 2022-06-20
#> 2     2 2022-09-20
#> 3     3 2022-12-20
#> 4     4 2023-03-20
#> 5     5 2023-06-20
#> 6     6 2023-09-20
#> 7     7 2023-12-20
#> 8     8 2024-03-20
#> 9     9 2024-06-20
#> 10    10 2024-09-20
#> # i 11 more rows
```

8.5.3 ISDA 用割引カーブ

```
discount_curve_isda <- GaussQuant::flat_curve_GQL(
  reference_date = trade_date_isda,
  rate = ois_discount_rate_isda,
  day_counter = actual_365,
  compounding = "Continuous"
)

discount_curve_handle_isda <- QuantLib::YieldTermStructureHandle(
  discount_curve_isda
)

isda_discount_check_tbl <- tibble::tribble(
  ~metric, ~value,
  "ois_discount_rate", ois_discount_rate_isda,
  "discount_factor_at_maturity", discount_curve_isda$discount(
    maturity_date_isda
  )
)
```

```
GaussQuant::show_tbl_GQH(
  isda_discount_check_tbl,
  "ISDA discount-curve check",
  n = 20
)
#> # A tibble: 2 x 2
#>   metric                value
#>   <chr>                <dbl>
#> 1 ois_discount_rate      0.0025
#> 2 discount_factor_at_maturity 0.98798
```

8.5.4 ISDA 整合 Hazard Rate の推定

市場 Spread 30.426bp を Running Spread とする標準 CDS を作成し、ISDA Engine の NPV がゼロとなるフラット Hazard Rate を推定する。

```
market_cds_isda <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = market_spread_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,
  coupon_tenor = "3M",
  day_counter = "Actual360",
  date_generation_rule = "CDS2015"
)

isda_market_npv_at_hazard <- function(hazard_rate) {
  probability_handle <- GaussQuant::flat_hazard_rate_GQL(
    hazard_rate = hazard_rate,
    reference_date = trade_date_isda,
    day_counter = "Actual365Fixed"
  )

  engine <- GaussQuant::cds_isda_engine_GQL(
    hazard_curve_handle = probability_handle,
    recovery_rate = recovery_rate_isda,
    discount_curve_handle = discount_curve_handle_isda
  )

  market_cds_isda$setPricingEngine(
    engine
  )
}
```

```

GaussQuant::cds_npv_GQL(
  market_cds_isda
)
}

isda_hazard_calibration_result <- stats::uniroot(
  isda_market_npv_at_hazard,
  interval = c(
    1e-8,
    0.50
  ),
  tol = 1e-10
)

hazard_rate_isda <- isda_hazard_calibration_result$root

hazard_curve_isda <- QuantLib::FlatHazardRate(
  trade_date_isda,
  GaussQuant::quote_handle_GQL(
    hazard_rate_isda
  ),
  actual_365
)

hazard_curve_handle_isda <- QuantLib::DefaultProbabilityTermStructureHandle(
  hazard_curve_isda
)

isda_hazard_check_tbl <- tibble::tribble(
  ~metric,          ~value,
  "market_spread",  market_spread_isda,
  "recovery_rate",  recovery_rate_isda,
  "implied_hazard_rate", hazard_rate_isda,
  "simple_hazard_approximation", market_spread_isda /
    (
      1 -
      recovery_rate_isda
    ),
  "survival_probability_at_maturity",
  hazard_curve_isda$survivalProbability(
    maturity_date_isda
  ),
  "default_probability_to_maturity",

```

```

1 -
  hazard_curve_isda$survivalProbability(
    maturity_date_isda
  )
)

GaussQuant::show_tbl_GQH(
  isda_hazard_check_tbl,
  "ISDA implied hazard-rate check",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 market_spread                        0.0030426
#> 2 recovery_rate                        0.4
#> 3 implied_hazard_rate                  0.0051398
#> 4 simple_hazard_approximation          0.005071
#> 5 survival_probability_at_maturity    0.97544
#> 6 default_probability_to_maturity     0.024562

```

市場 Spread と Recovery Rate から得られる単純近似と、ISDA Engine でゼロ NPV となる Hazard Rate には差が生じる。標準 CDS の日付、Accrual on default、割引および数値積分が評価へ反映されるためである。

8.6 ISDA Engine と評価比較

8.6.1 ISDA CDS Engine

契約 Coupon を標準 Coupon 100bp へ固定した 5 年物 CDS を評価する。

市場 Spread は 30.426bp であり、契約 Coupon 100bp より低い。プロテクション買い手は市場水準より高い Premium を支払うため、Running Coupon だけで評価すると一般に負の NPV となる。

```

isda_engine <- GaussQuant::cds_isda_engine_GQL(
  hazard_curve_handle = hazard_curve_handle_isda,
  recovery_rate = recovery_rate_isda,
  discount_curve_handle = discount_curve_handle_isda
)

cds_isda <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,
  running_spread = contract_coupon_isda,
  notional = notional_isda,
  side = "buyer",
  trade_date = trade_date_isda,

```



```

    coupon_tenor = "3M",
    day_counter = "Actual360",
    date_generation_rule = "CDS2015",
    pricing_engine = isda_engine
)

cds_isda_summary_base_tbl <- GaussQuant::cds_summary_GQL(
  cds_isda
)

GaussQuant::show_tbl_GQH(
  cds_isda_summary_base_tbl,
  "ISDA CDS base summary",
  n = 20
)

#> # A tibble: 4 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 npv          -3.3496e+5
#> 2 fair_spread    3.0426e-3
#> 3 coupon_leg_npv -4.9839e+5
#> 4 default_leg_npv 1.4649e+5

```

8.6.2 Accrual rebate と NPV 分解

標準 CDS では、取引日が前回の標準 Coupon 日と次回 Coupon 日の間にある場合、契約開始時に Accrual rebate が発生することがある。

この場合、プロテクション買い手から見た NPV は、

$$V_{\text{CDS}} = V_{\text{coupon}} + V_{\text{default}} + V_{\text{accrual rebate}}$$

となる。

```

coupon_leg_npv_isda <- cds_isda$couponLegNPV()
default_leg_npv_isda <- cds_isda$defaultLegNPV()

accrual_rebate_npv_isda <- tryCatch(
  as.numeric(
    cds_isda$accrualRebateNPV()
  ),
  error = function(e) NA_real_
)

```

```

three_component_sum_isda <- coupon_leg_npv_isda +
  default_leg_npv_isda +
  accrual_rebate_npv_isda

isda_leg_check_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_leg_npv", coupon_leg_npv_isda,
  "default_leg_npv", default_leg_npv_isda,
  "accrual_rebate_npv", accrual_rebate_npv_isda,
  "three_component_sum", three_component_sum_isda,
  "reported_npv",      GaussQuant::cds_npv_GQL(
                        cds_isda
                        ),
  "difference",        GaussQuant::cds_npv_GQL(
                        cds_isda
                        ) -
                        three_component_sum_isda
)

GaussQuant::show_tbl_GQH(
  isda_leg_check_tbl,
  "ISDA CDS NPV decomposition",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_npv -498392.
#> 2 default_leg_npv 146485.
#> 3 accrual_rebate_npv 16944.
#> 4 three_component_sum -334963.
#> 5 reported_npv -334963.
#> 6 difference      0

```

8.6.3 Fair Spread と Fair Upfront

Fair Spread は、Upfront を授受せずに CDS NPV をゼロとする Running Spread である。

Fair Upfront は、契約 Coupon を標準 Coupon へ固定したまま、契約価値を調整するために開始時点で授受する Notional 比率である。

```

fair_spread_isda <- GaussQuant::cds_fair_spread_GQL(
  cds_isda
)

```

```

fair_upfront_isda <- tryCatch(
  as.numeric(
    cds_isda$fairUpfront()
  ),
  error = function(e) NA_real_
)

fair_upfront_amount_isda <- fair_upfront_isda *
  notional_isda

isda_cds_summary_tbl <- tibble::tribble(
  ~metric,          ~value,
  "coupon_leg_npv", coupon_leg_npv_isda,
  "default_leg_npv", default_leg_npv_isda,
  "accrual_rebate_npv", accrual_rebate_npv_isda,
  "cds_npv",        GaussQuant::cds_npv_GQL(
    cds_isda
  ),
  "market_spread",  market_spread_isda,
  "contract_coupon", contract_coupon_isda,
  "fair_spread",     fair_spread_isda,
  "fair_upfront",    fair_upfront_isda,
  "fair_upfront_amount", fair_upfront_amount_isda,
  "hazard_rate",     hazard_rate_isda
)

GaussQuant::show_tbl_GQH(
  isda_cds_summary_tbl,
  "CDS summary: ISDA engine",
  n = 30
)

#> # A tibble: 10 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_npv -4.9839e+5
#> 2 default_leg_npv  1.4649e+5
#> 3 accrual_rebate_npv 1.6944e+4
#> 4 cds_npv        -3.3496e+5
#> 5 market_spread    3.0426e-3
#> 6 contract_coupon    1     e-2
#> 7 fair_spread       3.0426e-3
#> 8 fair_upfront     -3.3497e-2
#> 9 fair_upfront_amount -3.3497e+5

```

```
#> 10 hazard_rate          5.1398e-3
```

この例では、契約 Coupon が市場 Spread より高いため、プロテクション買い手は市場より多くの Premium を支払う。したがって、契約開始時の価値を調整する Upfront が必要となる。

Fair Upfront の正負は、QuantLib における Buyer / Seller の符号規約と契約設定に依存する。符号だけで判断せず、CDS NPV、契約 Coupon、Fair Spread および Upfront 金額を一緒に確認する必要がある。

8.6.4 ISDA 用キャッシュフロー表

標準 CDS2015 Schedule、ISDA 整合 Hazard Curve および割引カーブから、Premium 期間ごとの確率情報を作成する。

```
cds_cashflow_isda_tbl <- GaussQuant::cds_cashflow_table_GQL(
  cds_schedule = cds_schedule_isda,
  protection_start_date = cds_isda$protectionStartDate(),
  coupon_rate = contract_coupon_isda,
  hazard_curve = hazard_curve_isda,
  discount_curve = discount_curve_isda,
  trade_date = trade_date_isda,
  notional = notional_isda
)

GaussQuant::show_tbl_GQH(
  cds_cashflow_isda_tbl,
  "CDS cash-flow table: CDS2015 example",
  n = 30
)

#> # A tibble: 21 x 13
#>   payment_date coupon_rate accrual_start accrual_end accrual_days
#>   <chr>          <dbl> <chr>          <chr>          <dbl>
#> 1 2022-08-19      NA    <NA>          2022-08-19      NA
#> 2 2022-09-20      0.01 2022-06-20    2022-09-20      92
#> 3 2022-12-20      0.01 2022-09-20    2022-12-20      91
#> 4 2023-03-20      0.01 2022-12-20    2023-03-20      90
#> 5 2023-06-20      0.01 2023-03-20    2023-06-20      92
#> 6 2023-09-20      0.01 2023-06-20    2023-09-20      92
#> 7 2023-12-20      0.01 2023-09-20    2023-12-20      91
#> 8 2024-03-20      0.01 2023-12-20    2024-03-20      91
#> 9 2024-06-20      0.01 2024-03-20    2024-06-20      92
#> 10 2024-09-20     0.01 2024-06-20    2024-09-20      92
#> # i 11 more rows
#> # i 8 more variables: accrual_year_fraction_365 <dbl>, coupon_amount <dbl>,
#> #   discount_factor <dbl>, survival_probability <dbl>,
#> #   default_probability <dbl>, midpoint_date <chr>,
```

```
#> # midpoint_discount_factor <dbl>, midpoint_survival_probability <dbl>
```

この表は CDS の経済構造を理解するための MidPoint 型分解である。ISDA Engine は、標準日付、Accrual on default、数値積分および曲線補間をより厳密に扱うため、この表から得られる単純近似と完全には一致しない。

8.6.5 MidPoint 型近似と ISDA Engine

ISDA 用キャッシュフロー表から、Premium Leg、Protection Leg、RPV01 および Fair Spread を簡易計算する。

```
premium_leg_hand_isda <- -sum(
  cds_cashflow_isda_tbl$coupon_amount *
  cds_cashflow_isda_tbl$discount_factor *
  cds_cashflow_isda_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

protection_leg_per_unit_hand_isda <- (
  1 -
  recovery_rate_isda
) *
sum(
  cds_cashflow_isda_tbl$midpoint_discount_factor *
  cds_cashflow_isda_tbl$default_probability,
  na.rm = TRUE
)

protection_leg_hand_isda <- notional_isda *
  protection_leg_per_unit_hand_isda

risky_pv01_hand_isda <- sum(
  (
    cds_cashflow_isda_tbl$accrual_days /
    360
  ) *
  cds_cashflow_isda_tbl$discount_factor *
  cds_cashflow_isda_tbl$midpoint_survival_probability,
  na.rm = TRUE
)

fair_spread_hand_isda <- protection_leg_per_unit_hand_isda /
  risky_pv01_hand_isda
```

```

cds_npv_hand_isda <- premium_leg_hand_isda +
  protection_leg_hand_isda +
  accrual_rebate_npv_isda

isda_hand_check_tbl <- tibble::tribble(
  ~metric,          ~hand_value,          ~isda_value,
  "coupon_leg_npv", premium_leg_hand_isda, coupon_leg_npv_isda,
  "default_leg_npv", protection_leg_hand_isda, default_leg_npv_isda,
  "cds_npv",        cds_npv_hand_isda,    GaussQuant::cds_npv_GQL(
                                     cds_isda
                                     ),
  "fair_spread",    fair_spread_hand_isda, fair_spread_isda
) |>
  dplyr::mutate(
    difference = .data$hand_value -
      .data$isda_value
  )

GaussQuant::show_tbl_GQH(
  isda_hand_check_tbl,
  "MidPoint-style approximation and ISDA comparison",
  n = 20
)

#> # A tibble: 4 x 4
#>   metric          hand_value isda_value  difference
#>   <chr>          <dbl>      <dbl>      <dbl>
#> 1 coupon_leg_npv -4.9811e+5 -4.9839e+5  280.16
#> 2 default_leg_npv 1.4649e+5  1.4649e+5   0.19448
#> 3 cds_npv        -3.3468e+5 -3.3496e+5  280.35
#> 4 fair_spread     2.9408e-3  3.0426e-3 -0.00010178

```

8.6.6 MidPoint Engine と ISDA Engine の比較

同じ Hazard Curve、Recovery Rate、割引カーブおよび標準 Coupon を使用し、MidPoint Engine と ISDA Engine の評価差を確認する。

```

midpoint_engine_isda_inputs <- GaussQuant::cds_midpoint_engine_GQL(
  probability_handle = hazard_curve_handle_isda,
  recovery_rate = recovery_rate_isda,
  discount_handle = discount_curve_handle_isda
)

cds_midpoint_standard <- GaussQuant::make_cds_GQL(
  maturity_date = maturity_date_isda,

```

```

    running_spread = contract_coupon_isda,
    notional = notional_isda,
    side = "buyer",
    trade_date = trade_date_isda,
    coupon_tenor = "3M",
    day_counter = "Actual360",
    date_generation_rule = "CDS2015",
    pricing_engine = midpoint_engine_isda_inputs
  )

midpoint_standard_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_midpoint_standard
) |>
  dplyr::mutate(
    engine = "MidPoint"
  )

isda_standard_summary_tbl <- GaussQuant::cds_summary_GQL(
  cds_isda
) |>
  dplyr::mutate(
    engine = "ISDA"
  )

engine_comparison_tbl <- dplyr::bind_rows(
  midpoint_standard_summary_tbl,
  isda_standard_summary_tbl
) |>
  dplyr::relocate(
    .data$engine
  ) |>
  tidyr::pivot_wider(
    names_from = engine,
    values_from = value
  ) |>
  dplyr::mutate(
    difference_isda_minus_midpoint = .data$ISDA -
      .data$MidPoint
  )

GaussQuant::show_tbl_GQH(
  engine_comparison_tbl,
  "MidPoint and ISDA CDS engine comparison",

```

```

n = 20
)
#> # A tibble: 4 x 4
#>   metric      MidPoint      ISDA difference_isda_minus_midpoint
#>   <chr>      <dbl>      <dbl>      <dbl>
#> 1 npv        -3.3495e+5 -3.3496e+5      -1.1669e+1
#> 2 fair_spread  3.0427e-3  3.0426e-3      -7.6555e-8
#> 3 coupon_leg_npv -4.9838e+5 -4.9839e+5      -1.1474e+1
#> 4 default_leg_npv 1.4649e+5  1.4649e+5      -1.9448e-1

```

MidPoint 法は、各 Premium 期間内のデフォルト時点を期間中点へ集約する。ISDA Engine は標準市場規約に合わせて、より詳細な日付処理と積分を行う。そのため、同じカーブを用いても Leg NPV と Fair Spread に差が生じる。

8.7 まとめ

CDS は、Premium Leg と Protection Leg から構成される。プロテクション買い手は Premium を支払い、信用事由が発生した場合に損失補償を受け取る。

Hazard Rate は、企業がその時点まで存続しているという条件の下での瞬時的なデフォルト強度である。フラット Hazard Rate では、生存確率は次式で表される。

$$Q(t) = e^{-\lambda t}$$

Recovery Rate R を仮定すると、CDS Spread と Hazard Rate には、

$$\lambda \approx \frac{s}{1-R}$$

という近似関係がある。ただし、厳密な Hazard Rate は、Premium 支払日、割引、Accrual on default および標準 CDS 規約を反映して推定する。

Premium Leg の単位 Spread 当たり現在価値が Risky PV01 であり、Fair Spread は Protection Leg の単位 Notional 価値を Risky PV01 で割って求められる。

MidPoint Engine は、デフォルト時点を各期間の中点で近似するため、CDS 評価の構造を理解しやすい。ISDA Engine は、標準 CDS2015 の日付、Accrual rebate、Accrual on default および市場規約をより厳密に扱う。

標準 Coupon と市場 Spread が異なる場合、Running Coupon だけでは契約価値がゼロにならない。この差を契約開始時に調整するものが Fair Upfront である。

```

chapter08_summary_tbl <- tibble::tribble(
  ~section,      ~main_result,
  "Hazard Rate", "Survival and cumulative default probabilities",
  "MidPoint CDS", "Premium and protection leg decomposition",
  "Hazard calibration", "Zero-NPV market CDS",

```



```
"CDS2015",                "Standard maturity and coupon schedule",
"ISDA CDS",                "Fair spread, upfront, and accrual rebate",
"Engine comparison",       "MidPoint approximation versus ISDA pricing"
)

GaussQuant::show_tbl_GQH(
  chapter08_summary_tbl,
  "Chapter II-8 summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   section          main_result
#>   <chr>          <chr>
#> 1 Hazard Rate    Survival and cumulative default probabilities
#> 2 MidPoint CDS   Premium and protection leg decomposition
#> 3 Hazard calibration Zero-NPV market CDS
#> 4 CDS2015        Standard maturity and coupon schedule
#> 5 ISDA CDS       Fair spread, upfront, and accrual rebate
#> 6 Engine comparison MidPoint approximation versus ISDA pricing

cat(
  "\nChapter II-8 Credit risk and CDS completed successfully.\n"
)
#>
#> Chapter II-8 Credit risk and CDS completed successfully.
```

第 II-9 章エキゾチック・デリバティブと PRDC

これまでの章では、債券、金利先物、Swap、Option、Callable Bond および CDS を、それぞれの主要なリスク要因に基づいて評価した。本章では、第 II 部の締めくくりとして、金利と為替を組み合わせたエキゾチック・デリバティブである PRDC を検討する。

PRDC (Power Reverse Dual Currency Note) は、為替レートに連動する Coupon を支払う長期の仕組債である。典型的な Coupon rate は、次の形で表される。

$$C(S_t) = \min \left[C_{\max}, \max \left(a \frac{S_t}{S_0} - b, C_{\min} \right) \right]$$

ここで、

- S_t は Coupon 決定日における為替レート
- S_0 は Coupon 計算上の基準為替レート
- a は為替連動係数
- b は固定控除率
- $C_{\min}$ は Coupon Floor
- $C_{\max}$ は Coupon Cap

である。

為替レートが上昇すると Coupon rate は増加するが、Floor と Cap によって下限と上限が制限される。したがって、PRDC Coupon は FX Spot に対して区分線形である一方、将来価値は為替分布に依存する非線形商品となる。

本章では、Domestic 金利、Foreign 金利および FX volatility を一定とする Garman–Kohlhagen モデルを用いて、将来の Coupon 日ごとに FX パスを生成する。各パスで Coupon を計算し、その平均値から Coupon Leg と Redemption Leg の現在価値を求める。

実際の PRDC には、発行体の期限前償還権を含むことが多い。本章では商品の基本構造を明確にするため、**発行体コール権を含まない非 Callable PRDC** を評価する。Callable PRDC には、金利・為替の複数因子モデル、相関構造および Least-Squares Monte Carlo などが追加が必要となる。

9.1 PRDC の商品構造と市場条件

9.1.1 商品条件

2015 年 9 月 3 日から 2041 年 9 月 3 日まで、半年ごとに Coupon を支払う PRDC を考える。最初の 1 年間は固定 Coupon とし、その後は FX 連動 Coupon へ移行する。

評価日は 2023 年 4 月 11 日であるため、固定 Intro Coupon 期間はすでに終了している。ただし、GaussQuant の PRDC 評価関数が Intro Coupon を扱えることを確認するため、契約条件として残す。

本章では、計算時間を抑えながら Monte Carlo 評価を確認するため、基準評価のパス数を 3,000 本、感応度評価を 1,000 本とする。

```
valuation_date_prdc <- as.Date("2023-04-11")
effective_date_prdc <- as.Date("2015-09-03")
termination_date_prdc <- as.Date("2041-09-03")
intro_coupon_end_date_prdc <- as.Date("2016-09-03")
```

```
notional_prdc <- 300000000
intro_coupon_rate_prdc <- 0.022
coupon_frequency_months_prdc <- 6L
```

```
domestic_rate_prdc <- 0.01
foreign_rate_prdc <- 0.03
discount_rate_prdc <- 0.005
```

```
fx_spot_prdc <- 133.2681
fx_volatility_prdc <- 0.10
```

```
coupon_fx_base_prdc <- 120
coupon_leverage_prdc <- 0.122
coupon_subtraction_prdc <- 0.10
coupon_floor_prdc <- 0
coupon_cap_prdc <- 0.022
```

```
number_of_paths_prdc <- 300L
sensitivity_paths_prdc <- 100L
random_seed_prdc <- 123L
```

```
GaussQuant::set_eval_date_GQL(
  valuation_date_prdc
)
```

```
prdc_input_tbl <- tibble::tribble(
```

```

~input, ~value,
"valuation_date", as.character(valuation_date_prdc),
"effective_date", as.character(effective_date_prdc),
"termination_date", as.character(termination_date_prdc),
"intro_coupon_end_date", as.character(intro_coupon_end_date_prdc),
"notional_jpy", as.character(notional_prdc),
"intro_coupon_rate", as.character(intro_coupon_rate_prdc),
"coupon_frequency_months", as.character(coupon_frequency_months_prdc),
"domestic_rate", as.character(domestic_rate_prdc),
"foreign_rate", as.character(foreign_rate_prdc),
"discount_rate", as.character(discount_rate_prdc),
"fx_spot", as.character(fx_spot_prdc),
"fx_volatility", as.character(fx_volatility_prdc),
"coupon_fx_base", as.character(coupon_fx_base_prdc),
"coupon_leverage", as.character(coupon_leverage_prdc),
"coupon_subtraction", as.character(coupon_subtraction_prdc),
"coupon_floor", as.character(coupon_floor_prdc),
"coupon_cap", as.character(coupon_cap_prdc),
"valuation_paths", as.character(number_of_paths_prdc),
"sensitivity_paths", as.character(sensitivity_paths_prdc),
"random_seed", as.character(random_seed_prdc)
)

```

```

GaussQuant::show_tbl_GQH(
  prdc_input_tbl,
  "PRDC product and model assumptions",
  n = 30
)

```

```

#> # A tibble: 20 x 2
#>   input      value
#>   <chr>      <chr>
#> 1 valuation_date 2023-04-11
#> 2 effective_date 2015-09-03
#> 3 termination_date 2041-09-03
#> 4 intro_coupon_end_date 2016-09-03
#> 5 notional_jpy 300000000
#> 6 intro_coupon_rate 0.022
#> 7 coupon_frequency_months 6
#> 8 domestic_rate 0.01
#> 9 foreign_rate 0.03
#> 10 discount_rate 0.005
#> 11 fx_spot 133.2681
#> 12 fx_volatility 0.1

```

```
#> 13 coupon_fx_base      120
#> 14 coupon_leverage     0.122
#> 15 coupon_subtraction   0.1
#> 16 coupon_floor        0
#> 17 coupon_cap          0.022
#> 18 valuation_paths     300
#> 19 sensitivity_paths   100
#> 20 random_seed         123
```

9.1.2 Coupon Schedule

契約開始日から満期までの全 Coupon Schedule と、固定 Intro Coupon 期間の Schedule を作成する。

```
valuation_date_ql_prdc <- GaussQuant::date_GQL(
  valuation_date_prdc
)

effective_date_ql_prdc <- GaussQuant::date_GQL(
  effective_date_prdc
)

termination_date_ql_prdc <- GaussQuant::date_GQL(
  termination_date_prdc
)

intro_coupon_end_date_ql_prdc <- GaussQuant::date_GQL(
  intro_coupon_end_date_prdc
)

coupon_schedule_prdc <- QuantLib::Schedule(
  effective_date_ql_prdc,
  termination_date_ql_prdc,
  GaussQuant::period_months_GQL(
    coupon_frequency_months_prdc
  ),
  QuantLib::TARGET(),
  "ModifiedFollowing",
  "ModifiedFollowing",
  "Backward",
  FALSE
)

intro_coupon_schedule_prdc <- QuantLib::Schedule(
  effective_date_ql_prdc,
```

```

intro_coupon_end_date_ql_prdc,
GaussQuant::period_months_GQL(
  coupon_frequency_months_prdc
),
QuantLib::TARGET(),
"ModifiedFollowing",
"ModifiedFollowing",
"Backward",
FALSE
)

coupon_schedule_tbl <- GaussQuant::schedule_table_GQL(
  coupon_schedule_prdc
)

intro_coupon_dates_prdc <- GaussQuant::schedule_date_vector_GQL(
  intro_coupon_schedule_prdc
)

remaining_intro_coupon_dates_prdc <- intro_coupon_dates_prdc[
  intro_coupon_dates_prdc >
  valuation_date_prdc
]

intro_coupon_status_tbl <- tibble::tribble(
  ~metric, ~value,
  "intro_coupon_rate", intro_coupon_rate_prdc,
  "total_intro_schedule_dates", length(intro_coupon_dates_prdc),
  "remaining_intro_coupon_dates", length(remaining_intro_coupon_dates_prdc),
  "has_remaining_intro_coupon", as.numeric(
    length(
      remaining_intro_coupon_dates_prdc
    ) > 0
  )
)

GaussQuant::show_tbl_GQH(
  coupon_schedule_tbl,
  "PRDC full coupon schedule",
  n = 60
)

#> # A tibble: 53 x 2
#>   period schedule_date

```

```
#>      <int> <date>
#> 1      1 2015-09-03
#> 2      2 2016-03-03
#> 3      3 2016-09-05
#> 4      4 2017-03-03
#> 5      5 2017-09-04
#> 6      6 2018-03-05
#> 7      7 2018-09-03
#> 8      8 2019-03-04
#> 9      9 2019-09-03
#> 10     10 2020-03-03
#> # i 43 more rows

GaussQuant::show_tbl_GQH(
  intro_coupon_status_tbl,
  "PRDC introductory coupon status",
  n = 20
)
#> # A tibble: 4 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 intro_coupon_rate                    0.022
#> 2 total_intro_schedule_dates            3
#> 3 remaining_intro_coupon_dates          0
#> 4 has_remaining_intro_coupon            0
```

9.1.3 Domestic、Foreign および Discount Curve

FX Quote を、外国通貨 1 単位当たりの円貨額として表す。

$$S_t = \text{JPY per unit of foreign currency}$$

このとき、

- Domestic Curve は JPY 金利
- Foreign Curve は外国通貨金利

である。

Garman–Kohlhagen モデルのリスク中立ドリフトには、Domestic 金利と Foreign 金利の差を使用する。

$$\frac{dS_t}{S_t} = (r_d - r_f)dt + \sigma_{\text{FX}}dW_t$$

本例では、Domestic 金利を 1%、Foreign 金利を 3% とする。そのため、リスク中立測度の下での FX ドリフトは年率マイナス 2% となる。

Coupon と元本の現在価値には、担保またはファンディング条件を表す外生的な割引率 0.5% を使用する。これは簡略化した設定であり、本格的な評価では Domestic OIS Curve と Collateral 条件を整合させる必要がある。

```
fx_day_counter_prdc <- QuantLib::Actual360()

domestic_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = domestic_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

foreign_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = foreign_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

discount_curve_prdc <- GaussQuant::flat_curve_GQL(
  reference_date = valuation_date_ql_prdc,
  rate = discount_rate_prdc,
  day_counter = fx_day_counter_prdc,
  compounding = "Continuous"
)

domestic_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  domestic_curve_prdc
)

foreign_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  foreign_curve_prdc
)

discount_curve_handle_prdc <- QuantLib::YieldTermStructureHandle(
  discount_curve_prdc
)

curve_summary_tbl <- tibble::tribble(
  ~curve, ~flat_rate,
  "Domestic FX curve", domestic_rate_prdc,
```



```

"Foreign FX curve", foreign_rate_prdc,
"PRDC discount curve", discount_rate_prdc,
"FX risk-neutral drift", domestic_rate_prdc -
  foreign_rate_prdc
)

GaussQuant::show_tbl_GQH(
  curve_summary_tbl,
  "PRDC flat-curve assumptions",
  n = 20
)

#> # A tibble: 4 x 2
#>   curve          flat_rate
#>   <chr>          <dbl>
#> 1 Domestic FX curve      0.01
#> 2 Foreign FX curve      0.03
#> 3 PRDC discount curve   0.005
#> 4 FX risk-neutral drift -0.02

```

9.2 FX プロセスと Coupon Payoff

9.2.1 FX Process

FX volatility を 10% とし、Garman–Kohlhagen Process を作成する。

```

fx_volatility_curve_prdc <- QuantLib::BlackConstantVol(
  valuation_date_ql_prdc,
  QuantLib::NullCalendar(),
  GaussQuant::quote_handle_GQL(
    fx_volatility_prdc
  ),
  fx_day_counter_prdc
)

fx_volatility_handle_prdc <- QuantLib::BlackVolTermStructureHandle(
  fx_volatility_curve_prdc
)

fx_process_prdc <- GaussQuant::fx_process_GQL(
  spot = fx_spot_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  volatility_handle = fx_volatility_handle_prdc
)

```

```

fx_market_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "fx_spot", fx_spot_prdc,
  "fx_volatility", fx_volatility_prdc,
  "domestic_rate", domestic_rate_prdc,
  "foreign_rate", foreign_rate_prdc,
  "risk_neutral_drift", domestic_rate_prdc -
    foreign_rate_prdc
)

GaussQuant::show_tbl_GQH(
  fx_market_summary_tbl,
  "PRDC FX market assumptions",
  n = 20
)

#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 fx_spot    133.27
#> 2 fx_volatility  0.1
#> 3 domestic_rate  0.01
#> 4 foreign_rate  0.03
#> 5 risk_neutral_drift -0.02

```

9.2.2 Coupon Payoff

本例の FX 連動 Coupon は次式である。

$$C(S) = \min \left[2.2\%, \max \left(12.2\% \frac{S}{120} - 10\%, 0 \right) \right]$$

Floor に到達する FX 水準は、

$$S_{\text{floor}} = S_0 \frac{b + C_{\min}}{a}$$

Cap に到達する FX 水準は、

$$S_{\text{cap}} = S_0 \frac{b + C_{\max}}{a}$$

である。

```

fx_floor_threshold_prdc <- coupon_fx_base_prdc *
(
  coupon_subtraction_prdc +
  coupon_floor_prdc
) /
coupon_leverage_prdc

fx_cap_threshold_prdc <- coupon_fx_base_prdc *
(
  coupon_subtraction_prdc +
  coupon_cap_prdc
) /
coupon_leverage_prdc

fx_payoff_grid_tbl <- tibble::tibble(
  fx_rate = seq(
    70,
    180,
    by = 2
  )
) |>
dplyr::mutate(
  uncapped_coupon = coupon_leverage_prdc *
    .data$fx_rate /
    coupon_fx_base_prdc -
    coupon_subtraction_prdc,
  coupon_rate = GaussQuant::prdc_coupon_GQL(
    fx_rate = .data$fx_rate,
    fx_base = coupon_fx_base_prdc,
    leverage = coupon_leverage_prdc,
    subtraction = coupon_subtraction_prdc,
    floor_rate = coupon_floor_prdc,
    cap_rate = coupon_cap_prdc
  )
)

coupon_threshold_tbl <- tibble::tribble(
  ~threshold, ~fx_rate,
  "floor_threshold", fx_floor_threshold_prdc,
  "cap_threshold", fx_cap_threshold_prdc,
  "current_spot", fx_spot_prdc
)

```

```

prdc_payoff_plot <- ggplot2::ggplot(
  fx_payoff_grid_tbl,
  ggplot2::aes(
    x = .data$fx_rate,
    y = .data$coupon_rate
  )
) +
  ggplot2::geom_line(
    linewidth = 0.8
  ) +
  ggplot2::geom_vline(
    xintercept = c(
      fx_floor_threshold_prdc,
      fx_cap_threshold_prdc
    ),
    linetype = "dashed"
  ) +
  ggplot2::scale_y_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "PRDC coupon payoff",
    subtitle = "Floored and capped FX-linked coupon",
    x = "FX rate",
    y = "Coupon rate"
  )

GaussQuant::show_tbl_GQH(
  coupon_threshold_tbl,
  "PRDC coupon FX thresholds",
  n = 20
)

#> # A tibble: 3 x 2
#>   threshold      fx_rate
#>   <chr>          <dbl>
#> 1 floor_threshold  98.361
#> 2 cap_threshold   120
#> 3 current_spot    133.27

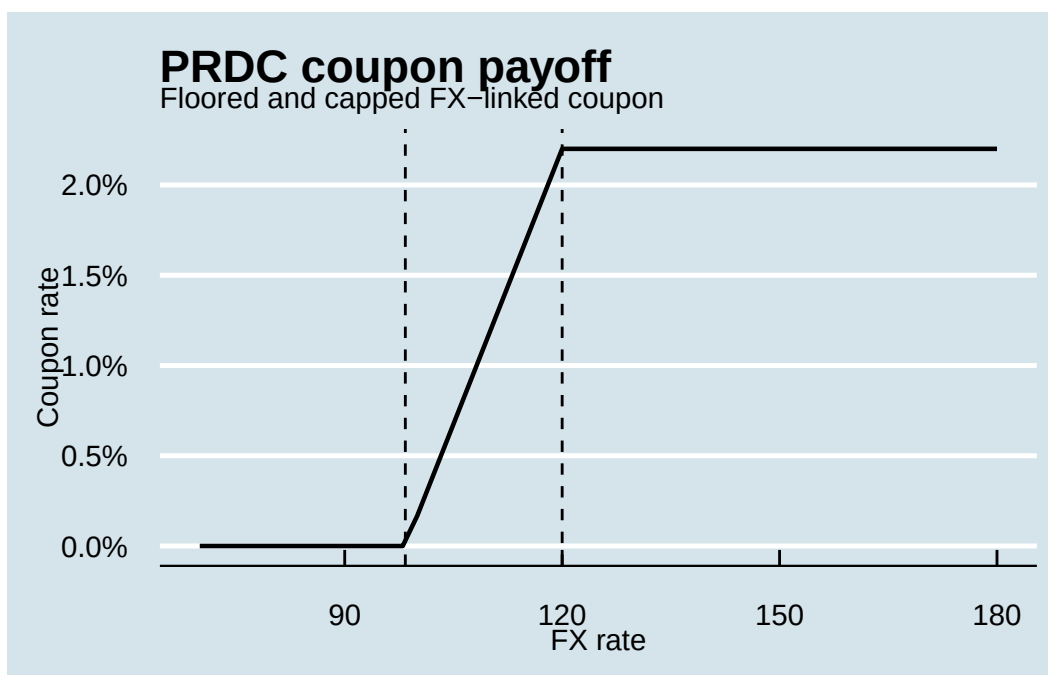
GaussQuant::show_tbl_GQH(
  fx_payoff_grid_tbl,

```

```

"PRDC coupon payoff table",
  n = 60
)
#> # A tibble: 56 x 3
#>   fx_rate uncapped_coupon coupon_rate
#>   <dbl>      <dbl>      <dbl>
#> 1     70    -0.028833         0
#> 2     72    -0.026800         0
#> 3     74    -0.024767         0
#> 4     76    -0.022733         0
#> 5     78    -0.0207         0
#> 6     80    -0.018667         0
#> 7     82    -0.016633         0
#> 8     84    -0.014600         0
#> 9     86    -0.012567         0
#> 10    88    -0.010533         0
#> # i 46 more rows
show_plot_LFL(prdc_payoff_plot)

```



現在の FX Spot は Cap 到達水準を上回っているため、評価時点の Coupon 式だけを見れば Cap に張り付いている。しかし、満期までの FX 分布には Floor、Linear 領域および Cap 領域のすべてが含まれ得る。

9.3 Monte Carlo による PRDC 評価

9.3.1 Coupon 日ベースの FX 時間グリッド

PRDC 評価に必要なのは、評価日より後に到来する Coupon 日である。GaussQuant の `fx_path_generator_GQL()` は、実際の Coupon 日を用いて不均一な時間間隔を保持する。

```
fx_path_generator_prdc <- GaussQuant::fx_path_generator_GQL(
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  day_counter = fx_day_counter_prdc,
  process = fx_process_prdc
)

time_grid_tbl <- tibble::tibble(
  coupon_number = seq_along(
    fx_path_generator_prdc$remaining_coupon_dates
  ),
  coupon_date = fx_path_generator_prdc$remaining_coupon_dates,
  time_from_valuation = fx_path_generator_prdc$coupon_times,
  time_step = fx_path_generator_prdc$time_steps
)

path_generator_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "remaining_coupon_dates", length(
    fx_path_generator_prdc$remaining_coupon_dates
  ),
  "number_of_steps", fx_path_generator_prdc$number_of_steps,
  "first_coupon_time", min(
    fx_path_generator_prdc$coupon_times
  ),
  "final_coupon_time", max(
    fx_path_generator_prdc$coupon_times
  )
)

GaussQuant::show_tbl_GQH(
  path_generator_summary_tbl,
  "PRDC FX path-generator summary",
  n = 20
)

#> # A tibble: 4 x 2
```

```
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 remaining_coupon_dates 37
#> 2 number_of_steps        37
#> 3 first_coupon_time      0.40556
#> 4 final_coupon_time      18.667

GaussQuant::show_tbl_GQH(
  time_grid_tbl,
  "PRDC remaining coupon-date time grid",
  n = 60
)
#> # A tibble: 37 x 4
#>   coupon_number coupon_date time_from_valuation time_step
#>   <int> <date>           <dbl>         <dbl>
#> 1         1 2023-09-04         0.40556      0.40556
#> 2         2 2024-03-04         0.91111      0.50556
#> 3         3 2024-09-03         1.4194       0.50833
#> 4         4 2025-03-03         1.9222       0.50278
#> 5         5 2025-09-03         2.4333       0.51111
#> 6         6 2026-03-03         2.9361       0.50278
#> 7         7 2026-09-03         3.4472       0.51111
#> 8         8 2027-03-03         3.95         0.50278
#> 9         9 2027-09-03         4.4611       0.51111
#> 10        10 2028-03-03         4.9667       0.50556
#> # i 27 more rows
```

QuantLib の `StochasticProcess1D$evolve()` へは標準正規変量を渡す。Brownian increment を外側で二重に時間調整してはならない。GaussQuant の Path Generator では、標準正規乱数と各 Coupon 間の実際の時間差を分けて処理している。

9.3.2 サンプル FX パス

Monte Carlo 評価へ進む前に、20 本の FX パスを生成し、将来の FX 分布と Coupon の動きを確認する。

```
sample_path_count_prdc <- 20L

sample_fx_path_matrix_prdc <- GaussQuant::prdc_path_matrix_GQL(
  path_generator = fx_path_generator_prdc,
  n_paths = sample_path_count_prdc,
  seed = random_seed_prdc
)

sample_fx_path_tbl <- purrr::map_dfr(
```

```

seq_len(
  ncol(sample_fx_path_matrix_prdc)
),
function(path_id) {
  path_values <- sample_fx_path_matrix_prdc[
    ,
    path_id
  ]

  tibble::tibble(
    path_id = path_id,
    coupon_number = seq_along(
      path_values
    ),
    coupon_date = fx_path_generator_prdc$remaining_coupon_dates,
    time = fx_path_generator_prdc$coupon_times,
    fx_rate = path_values,
    coupon_rate = GaussQuant::prdc_coupon_GQL(
      fx_rate = path_values,
      fx_base = coupon_fx_base_prdc,
      leverage = coupon_leverage_prdc,
      subtraction = coupon_subtraction_prdc,
      floor_rate = coupon_floor_prdc,
      cap_rate = coupon_cap_prdc
    )
  )
}
)

sample_fx_path_plot <- ggplot2::ggplot(
  sample_fx_path_tbl,
  ggplot2::aes(
    x = .data$time,
    y = .data$fx_rate,
    group = .data$path_id
  )
) +
  ggplot2::geom_line(
    linewidth = 0.5,
    alpha = 0.7
  ) +
  ggplot2::labs(
    title = "Sample FX paths for PRDC valuation",

```



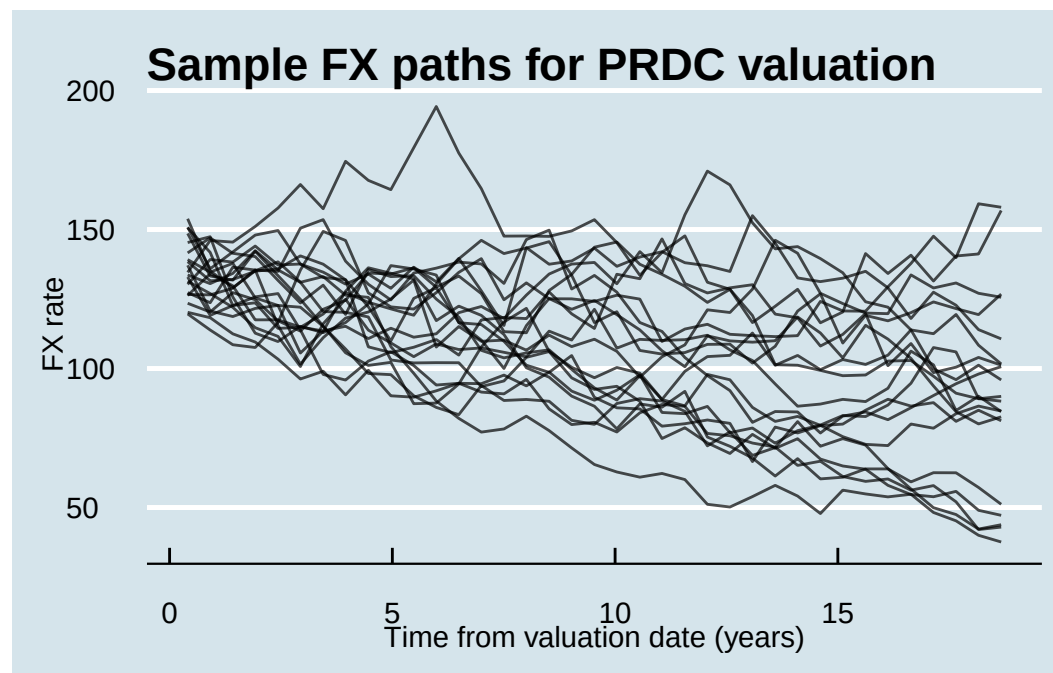
```

    x = "Time from valuation date (years)",
    y = "FX rate"
  )

GaussQuant::show_tbl_GQH(
  sample_fx_path_tbl,
  "Sample PRDC FX and coupon paths",
  n = 40
)

#> # A tibble: 40 x 6
#>   path_id coupon_number coupon_date    time fx_rate coupon_rate
#>   <int>      <int> <date>      <dbl>   <dbl>      <dbl>
#> 1         1          1 2023-09-04  0.40556 127.30      0.022
#> 2         1          2 2024-03-04  0.91111 123.66      0.022
#> 3         1          3 2024-09-03  1.4194  136.45      0.022
#> 4         1          4 2025-03-03  1.9222  135.42      0.022
#> 5         1          5 2025-09-03  2.4333  134.94      0.022
#> 6         1          6 2026-03-03  2.9361  150.49      0.022
#> 7         1          7 2026-09-03  3.4472  153.56      0.022
#> 8         1          8 2027-03-03  3.95    138.63      0.022
#> 9         1          9 2027-09-03  4.4611  130.31      0.022
#> 10        1         10 2028-03-03  4.9667  124.66      0.022
#> # i 30 more rows
show_plot_LFL(sample_fx_path_plot)

```



9.3.3 PRDC の現在価値

各 Coupon 日の期待 Coupon rate は、Monte Carlo によって次のように近似する。

$$E[C(S_{t_i})] \approx \frac{1}{M} \sum_{j=1}^M C(S_{t_i}^{(j)})$$

ここで、 M はパス数、 $S_{t_i}^{(j)}$ はパス j における Coupon 日 t_i の FX Rate である。

各 Coupon cash flow は、

$$CF_i = N_0 \alpha_i E[C(S_{t_i})]$$

であり、PRDC 現在価値は、

$$PV_{\text{PRDC}} = \sum_i D(0, t_i) CF_i + D(0, T) N_0$$

となる。

```
prdc_result <- GaussQuant::prdc_npv_GQL(
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  process = fx_process_prdc,
  discount_curve = discount_curve_prdc,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  day_counter = fx_day_counter_prdc,
  intro_coupon_dates = intro_coupon_dates_prdc,
  intro_coupon_rate = intro_coupon_rate_prdc,
  n_paths = number_of_paths_prdc,
  seed = random_seed_prdc,
  redemption_amount = notional_prdc,
  return_details = TRUE
)

fx_path_matrix_prdc <- prdc_result$path_matrix
expected_coupon_tbl <- prdc_result$expected_coupon
prdc_cashflow_tbl <- prdc_result$cashflows
```

```

coupon_leg_present_value_prdc <- prdc_result$summary |>
  dplyr::filter(
    .data$metric == "coupon_leg_npv"
  ) |>
  dplyr::pull(
    value
  )

redemption_present_value_prdc <- prdc_result$summary |>
  dplyr::filter(
    .data$metric == "redemption_leg_npv"
  ) |>
  dplyr::pull(
    value
  )

prdc_present_value_jpy <- prdc_result$npv

prdc_valuation_summary_tbl <- tibble::tribble(
  ~metric, ~value,
  "coupon_leg_pv_jpy", coupon_leg_present_value_prdc,
  "redemption_leg_pv_jpy", redemption_present_value_prdc,
  "total_pv_jpy", prdc_present_value_jpy,
  "pv_as_percent_of_notional", prdc_present_value_jpy /
    notional_prdc,
  "number_of_paths", number_of_paths_prdc,
  "number_of_coupon_steps", nrow(
    fx_path_matrix_prdc
  )
)

GaussQuant::show_tbl_GQH(
  prdc_valuation_summary_tbl,
  "PRDC Monte Carlo valuation summary",
  n = 20
)

#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 coupon_leg_pv_jpy    6.2094e7
#> 2 redemption_leg_pv_jpy 2.7327e8
#> 3 total_pv_jpy        3.3536e8
#> 4 pv_as_percent_of_notional 1.1179e0

```

```
#> 5 number_of_paths      3      e2
#> 6 number_of_coupon_steps 3.7    e1
```

本章の元本償還額は固定されているため、Redemption Leg は割引率に依存するが FX には依存しない。FX Spot と FX volatility の影響は、主に Coupon Leg を通じて現れる。

9.3.4 期待 Coupon とキャッシュフロー

```
expected_coupon_display_tbl <- expected_coupon_tbl |>
  dplyr::select(
    coupon_number,
    coupon_date,
    time,
    expected_fx,
    expected_coupon_rate,
    is_intro_coupon,
    final_coupon_rate
  )

prdc_cashflow_display_tbl <- prdc_cashflow_tbl |>
  dplyr::select(
    coupon_number,
    accrual_start,
    accrual_end,
    final_coupon_rate,
    accrual_fraction,
    coupon_amount,
    redemption_amount,
    total_amount,
    discount_factor,
    total_present_value
  )

GaussQuant::show_tbl_GQH(
  expected_coupon_display_tbl,
  "PRDC expected FX and coupon rates",
  n = 60
)

#> # A tibble: 37 x 7
#>   coupon_number coupon_date      time expected_fx expected_coupon_rate
#>       <int> <date>         <dbl>      <dbl>          <dbl>
#> 1           1 2023-09-04  0.40556      132.60         0.021746
#> 2           2 2024-03-04  0.91111      131.83         0.021017
```

```
#> 3          3 2024-09-03 1.4194      130.23      0.020090
#> 4          4 2025-03-03 1.9222      128.77      0.018675
#> 5          5 2025-09-03 2.4333      126.86      0.017577
#> 6          6 2026-03-03 2.9361      124.74      0.016621
#> 7          7 2026-09-03 3.4472      123.82      0.016350
#> 8          8 2027-03-03 3.95       122.57      0.015578
#> 9          9 2027-09-03 4.4611      121.51      0.014453
#> 10         10 2028-03-03 4.9667      120.22      0.013940
#> # i 27 more rows
#> # i 2 more variables: is_intro_coupon <lgl>, final_coupon_rate <dbl>

GaussQuant::show_tbl_GQH(
  prdc_cashflow_display_tbl,
  "PRDC expected cash-flow table",
  n = 60
)
#> # A tibble: 37 x 10
#>   coupon_number accrual_start accrual_end final_coupon_rate accrual_fraction
#>   <int> <date>      <date>      <dbl>      <dbl>
#> 1         1 2023-03-03 2023-09-04      0.021746      0.51389
#> 2         2 2023-09-04 2024-03-04      0.021017      0.50556
#> 3         3 2024-03-04 2024-09-03      0.020090      0.50833
#> 4         4 2024-09-03 2025-03-03      0.018675      0.50278
#> 5         5 2025-03-03 2025-09-03      0.017577      0.51111
#> 6         6 2025-09-03 2026-03-03      0.016621      0.50278
#> 7         7 2026-03-03 2026-09-03      0.016350      0.51111
#> 8         8 2026-09-03 2027-03-03      0.015578      0.50278
#> 9         9 2027-03-03 2027-09-03      0.014453      0.51111
#> 10        10 2027-09-03 2028-03-03      0.013940      0.50556
#> # i 27 more rows
#> # i 5 more variables: coupon_amount <dbl>, redemption_amount <dbl>,
#> #   total_amount <dbl>, discount_factor <dbl>, total_present_value <dbl>
```

長期の FX 期待値は Domestic 金利と Foreign 金利の差の影響を受ける。本例では Foreign 金利が Domestic 金利より高いため、リスク中立測度の下では FX Rate に下向きのドリフトが働く。その結果、満期が長くなるほど Cap 領域から Linear 領域または Floor 領域へ移る確率が高くなる。

9.4 Coupon の期間構造と分布

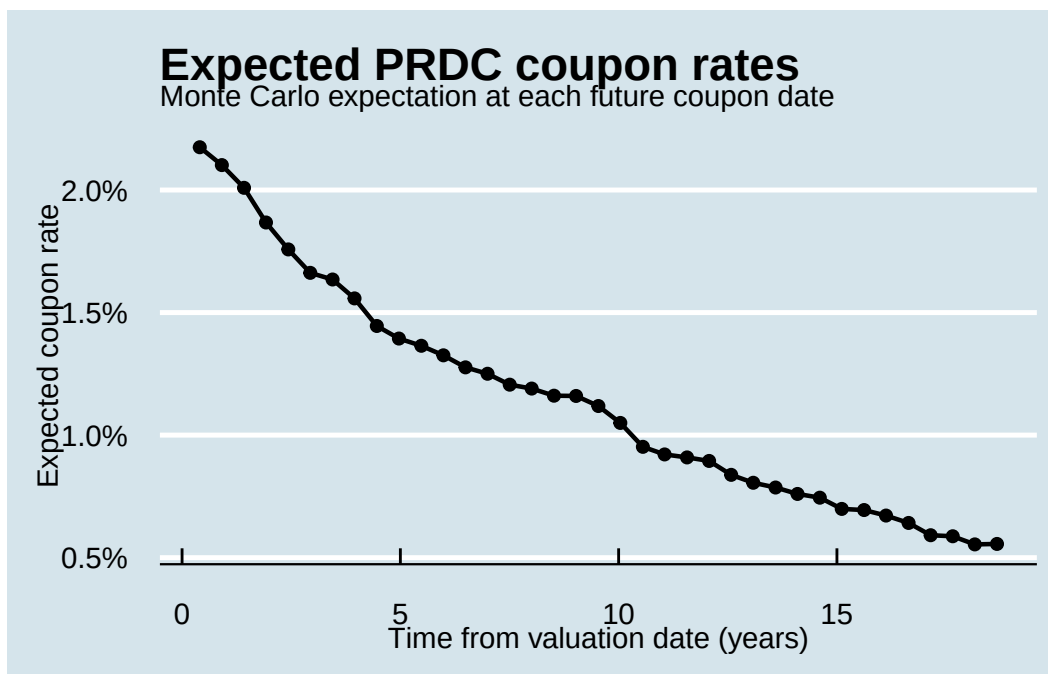
9.4.1 期待 Coupon の期間構造

```
expected_coupon_plot <- ggplot2::ggplot(
  expected_coupon_tbl,
```

```

ggplot2::aes(
  x = .data$time,
  y = .data$final_coupon_rate
) +
ggplot2::geom_line(
  linewidth = 0.8
) +
ggplot2::geom_point(
  size = 1.8
) +
ggplot2::scale_y_continuous(
  labels = scales::percent_format(
    accuracy = 0.1
  )
) +
ggplot2::labs(
  title = "Expected PRDC coupon rates",
  subtitle = "Monte Carlo expectation at each future coupon date",
  x = "Time from valuation date (years)",
  y = "Expected coupon rate"
)
show_plot_LFL(expected_coupon_plot)

```



期待 Coupon は、単に期待 FX を Coupon 式へ代入した値とは一般に一致しない。

$$E[C(S_t)] \neq C(E[S_t])$$

Cap と Floor を含む非線形 Payoff では、FX 分布全体を考慮する必要がある。

9.4.2 Coupon Cap / Floor 到達確率

各 Coupon 日について、シミュレーションされた Coupon が Floor または Cap に到達する確率を計算する。

```
coupon_cap_floor_probability_tbl <- GaussQuant::prdc_cap_floor_probability_GQL(
  path_matrix = fx_path_matrix_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  coupon_dates = fx_path_generator_prdc$remaining_coupon_dates,
  coupon_times = fx_path_generator_prdc$coupon_times
)

GaussQuant::show_tbl_GQH(
  coupon_cap_floor_probability_tbl,
  "PRDC coupon floor and cap probabilities",
  n = 60
)

#> # A tibble: 37 x 6
#>   coupon_number floor_probability cap_probability interior_probability
#>   <int>          <dbl>          <dbl>          <dbl>
#> 1         1         0         0.93333         0.06667
#> 2         2         0         0.82          0.18
#> 3         3    0.006667         0.76333         0.23
#> 4         4    0.023333         0.67          0.30667
#> 5         5    0.056667         0.61667         0.32667
#> 6         6      0.1         0.58667         0.31333
#> 7         7    0.10667         0.54333         0.35
#> 8         8    0.14667         0.49          0.36333
#> 9         9    0.16333         0.47333         0.36333
#> 10        10    0.19333         0.48          0.32667
#> # i 27 more rows
#> # i 2 more variables: coupon_date <date>, time <dbl>
```

Cap 確率、Linear 領域の確率および Floor 確率は、長期的な Coupon の性質を直接示す。現在の Spot が Cap 水準を上回っていても、長期 Coupon が常に Cap へ固定されとは限らない。

9.4.3 Coupon 分布

短期、中期、長期の代表的な Coupon 日について、Coupon rate の分布を比較する。

```

coupon_path_matrix_prdc <- GaussQuant::prdc_coupon_GQL(
  fx_rate = fx_path_matrix_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc
)

number_of_coupon_steps_prdc <- nrow(
  coupon_path_matrix_prdc
)

distribution_coupon_ids <- unique(
  pmax(
    1L,
    pmin(
      number_of_coupon_steps_prdc,
      c(
        1L,
        round(
          number_of_coupon_steps_prdc /
            2
        ),
        number_of_coupon_steps_prdc
      )
    )
  )
)

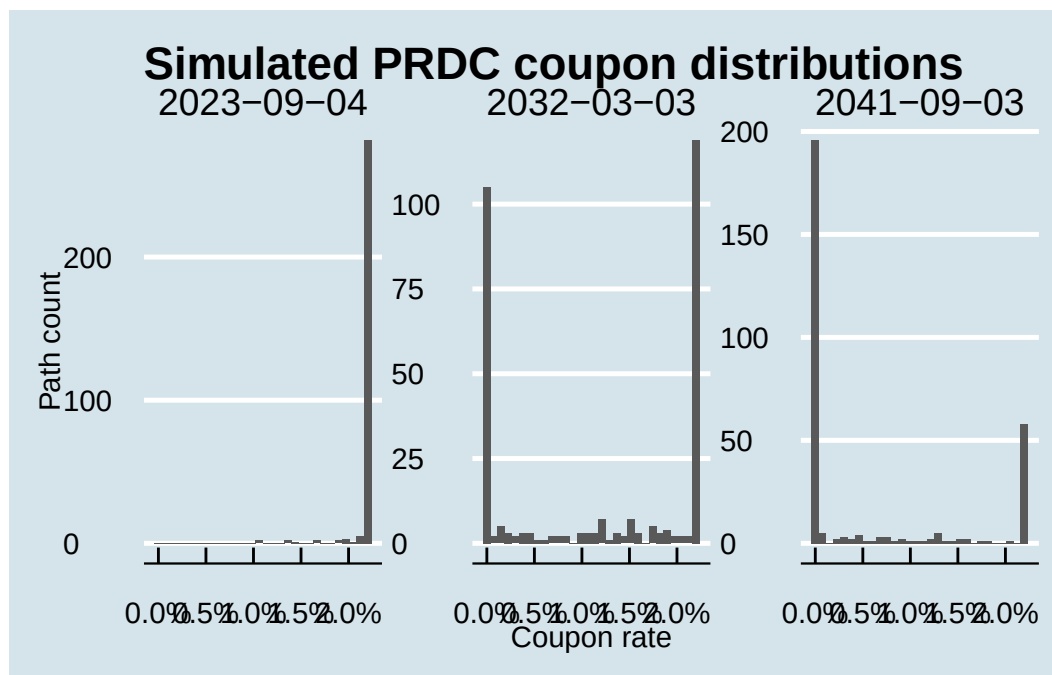
coupon_distribution_tbl <- purrr::map_dfr(
  distribution_coupon_ids,
  function(coupon_id) {
    tibble::tibble(
      coupon_number = coupon_id,
      coupon_date = as.character(
        fx_path_generator_prdc$remaining_coupon_dates[
          coupon_id
        ]
      ),
      coupon_rate = coupon_path_matrix_prdc[
        coupon_id,
      ]
    )
  }
)

```



```
)
}
)

coupon_distribution_plot <- ggplot2::ggplot(
  coupon_distribution_tbl,
  ggplot2::aes(
    x = .data$coupon_rate
  )
) +
  ggplot2::geom_histogram(
    bins = 30,
    linewidth = 0.3
  ) +
  ggplot2::facet_wrap(
    ~coupon_date,
    scales = "free_y"
  ) +
  ggplot2::scale_x_continuous(
    labels = scales::percent_format(
      accuracy = 0.1
    )
  ) +
  ggplot2::labs(
    title = "Simulated PRDC coupon distributions",
    x = "Coupon rate",
    y = "Path count"
  )
show_plot_LFL(coupon_distribution_plot)
```



Coupon 分布は連続的な正規分布または対数正規分布ではない。Floor と Cap に確率質量が集まり、その間に Linear 領域の連続分布が存在する。

9.5 FX 感応度と Monte Carlo 誤差

9.5.1 FX Spot 感応度

PRDC Coupon は FX Spot へ直接依存する。Spot を上下させたシナリオを比較する。

各シナリオで同じ乱数 Seed を使用することにより、Monte Carlo ノイズを抑えた Common Random Numbers 型の比較となる。

```
spot_scenario_input_tbl <- tibble::tribble(
  ~scenario, ~fx_spot,
  "spot_minus_10", fx_spot_prdc * 0.90,
  "spot_minus_5", fx_spot_prdc * 0.95,
  "base", fx_spot_prdc,
  "spot_plus_5", fx_spot_prdc * 1.05,
  "spot_plus_10", fx_spot_prdc * 1.10
)

spot_sensitivity_tbl <- GaussQuant::prdc_spot_sensitivity_GQL(
  spot_scenarios = spot_scenario_input_tbl,
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
  foreign_curve_handle = foreign_curve_handle_prdc,
  domestic_curve_handle = domestic_curve_handle_prdc,
  volatility_handle = fx_volatility_handle_prdc,
```

```

discount_curve = discount_curve_prdc,
notional = notional_prdc,
fx_base = coupon_fx_base_prdc,
leverage = coupon_leverage_prdc,
subtraction = coupon_subtraction_prdc,
floor_rate = coupon_floor_prdc,
cap_rate = coupon_cap_prdc,
day_counter = fx_day_counter_prdc,
intro_coupon_dates = intro_coupon_dates_prdc,
intro_coupon_rate = intro_coupon_rate_prdc,
n_paths = sensitivity_paths_prdc,
seed = random_seed_prdc,
redemption_amount = notional_prdc,
base_scenario = "base"
)

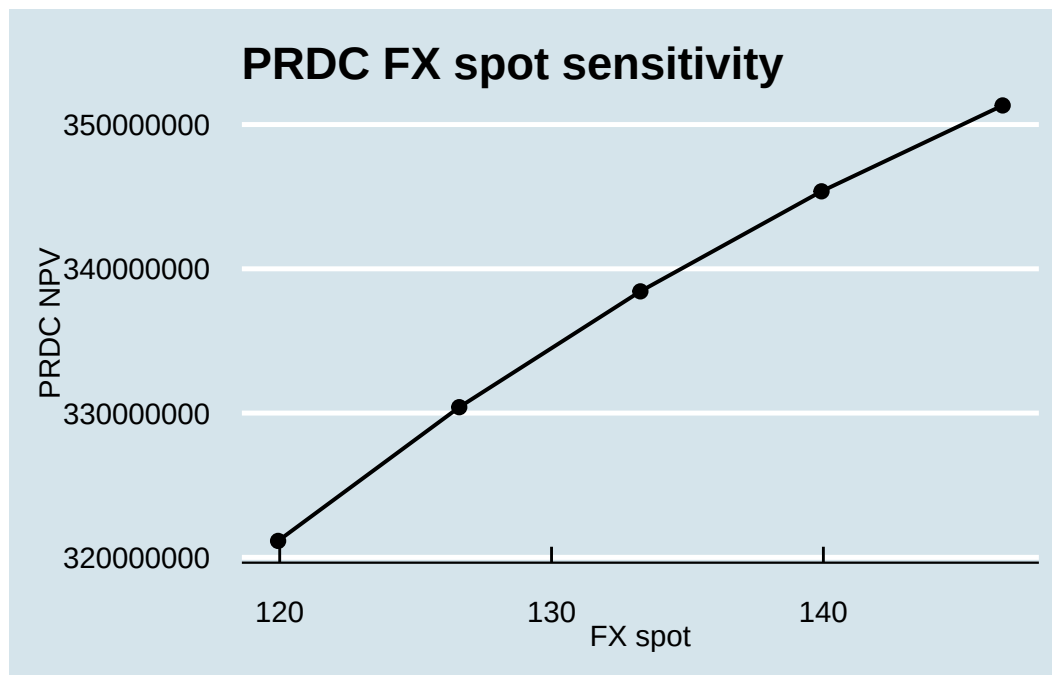
spot_sensitivity_plot <- ggplot2::ggplot(
  spot_sensitivity_tbl,
  ggplot2::aes(
    x = .data$fx_spot,
    y = .data$prdc_npv
  )
) +
  ggplot2::geom_line(
    linewidth = 0.7
  ) +
  ggplot2::geom_point(
    size = 2.2
  ) +
  ggplot2::labs(
    title = "PRDC FX spot sensitivity",
    x = "FX spot",
    y = "PRDC NPV"
  )

GaussQuant::show_tbl_GQH(
  spot_sensitivity_tbl,
  "PRDC FX spot sensitivity",
  n = 20
)

#> # A tibble: 5 x 4
#>   scenario      fx_spot prdc_npv npv_difference_from_base
#>   <chr>         <dbl>    <dbl>          <dbl>

```

```
#> 1 spot_minus_10 119.94 321139116. -17297149.
#> 2 spot_minus_5 126.60 330408227. -8028038.
#> 3 base 133.27 338436265. 0
#> 4 spot_plus_5 139.93 345370836. 6934571.
#> 5 spot_plus_10 146.59 351322971. 12886706.
show_plot_LFL(spot_sensitivity_plot)
```



Spot 上昇は Coupon を上昇させる方向に働くが、Cap へ到達した後の限界効果は小さくなる。Spot 低下時も、Floor へ到達すると Coupon の低下は止まる。このため、PRDC 価値の Spot 感応度は全域で一定ではない。

9.5.2 FX Volatility 感応度

PRDC Coupon は、Floor と Cap を持つ非線形 Payoff である。そのため、FX volatility の変化は将来の FX 分布だけでなく、Floor、Linear 領域および Cap へ入る確率を変化させる。

```
volatility_scenario_input_tbl <- tibble::tribble(
  ~scenario, ~fx_volatility,
  "vol_5pct", 0.05,
  "vol_7_5pct", 0.075,
  "base", fx_volatility_prdc,
  "vol_12_5pct", 0.125,
  "vol_15pct", 0.15
)

volatility_sensitivity_tbl <- GaussQuant::prdc_volatility_sensitivity_GQL(
  volatility_scenarios = volatility_scenario_input_tbl,
  valuation_date = valuation_date_prdc,
  coupon_schedule = coupon_schedule_prdc,
```

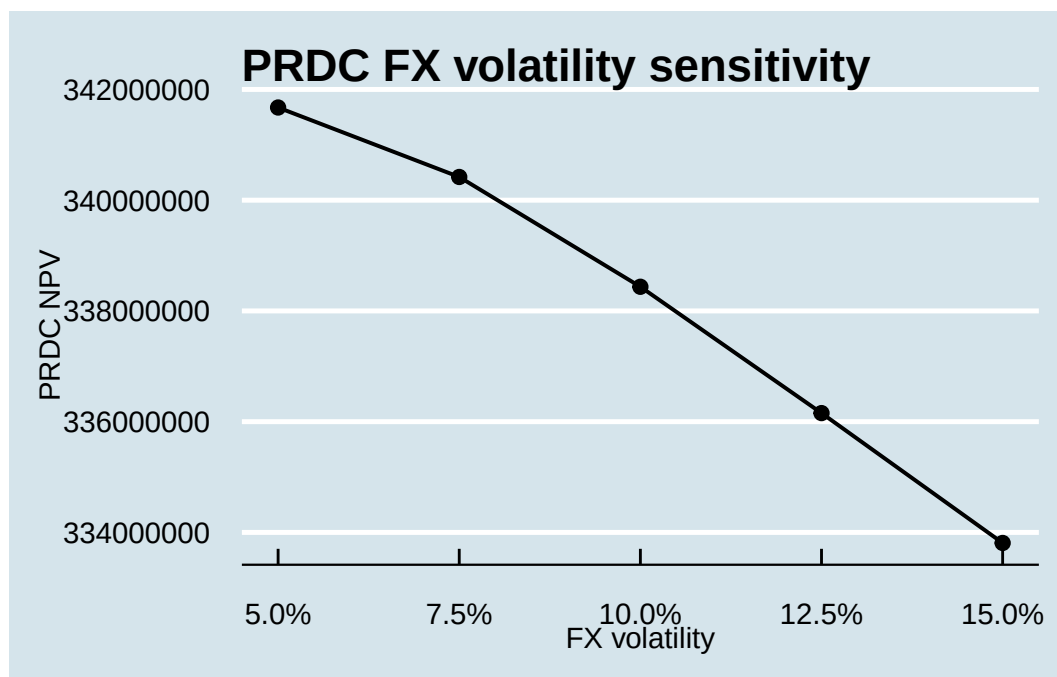
```

spot = fx_spot_prdc,
foreign_curve_handle = foreign_curve_handle_prdc,
domestic_curve_handle = domestic_curve_handle_prdc,
discount_curve = discount_curve_prdc,
notional = notional_prdc,
fx_base = coupon_fx_base_prdc,
leverage = coupon_leverage_prdc,
subtraction = coupon_subtraction_prdc,
floor_rate = coupon_floor_prdc,
cap_rate = coupon_cap_prdc,
day_counter = fx_day_counter_prdc,
calendar = QuantLib::NullCalendar(),
intro_coupon_dates = intro_coupon_dates_prdc,
intro_coupon_rate = intro_coupon_rate_prdc,
n_paths = sensitivity_paths_prdc,
seed = random_seed_prdc,
redemption_amount = notional_prdc,
base_scenario = "base"
)

volatility_sensitivity_plot <- ggplot2::ggplot(
  volatility_sensitivity_tbl,
  ggplot2::aes(
    x = .data$fx_volatility,
    y = .data$prdc_npv
  )
) +
ggplot2::geom_line(
  linewidth = 0.7
) +
ggplot2::geom_point(
  size = 2.2
) +
ggplot2::scale_x_continuous(
  labels = scales::percent_format(
    accuracy = 0.1
  )
) +
ggplot2::labs(
  title = "PRDC FX volatility sensitivity",
  x = "FX volatility",
  y = "PRDC NPV"
)

```

```
GaussQuant::show_tbl_GQH(
  volatility_sensitivity_tbl,
  "PRDC FX volatility sensitivity",
  n = 20
)
#> # A tibble: 5 x 4
#>   scenario    fx_volatility  prdc_npv npv_difference_from_base
#>   <chr>          <dbl>      <dbl>          <dbl>
#> 1 vol_5pct        0.05  341673690.        3237426.
#> 2 vol_7_5pct      0.075 340419233.        1982968.
#> 3 base           0.1    338436265.           0
#> 4 vol_12_5pct     0.125 336154662.       -2281603.
#> 5 vol_15pct       0.15  333808877.       -4627388.
show_plot_LFL(volatility_sensitivity_plot)
```



通常の Option のように「Volatility 上昇は必ず価値上昇」と単純には言えない。PRDC Coupon は Cap 付きの上昇 Payoff であると同時に Floor を持ち、現在の Spot、金利差、満期および各境界への到達確率によって Volatility 感応度が変わる。

9.5.3 Monte Carlo 標準誤差

Monte Carlo 推定値には統計誤差が存在する。各パスについて全 Coupon と Redemption の現在価値を計算し、標本平均、標準偏差、標準誤差および信頼区間を求める。

標準誤差は、パス現在価値の標準偏差を $\hat{\sigma}_{PV}$ 、パス数を M とすると、

$$SE = \frac{\hat{\sigma}_{PV}}{\sqrt{M}}$$

である。

```
monte_carlo_error_result <- GaussQuant::prdc_monte_carlo_error_GQL(
  path_matrix = fx_path_matrix_prdc,
  cashflow_tbl = prdc_cashflow_tbl,
  notional = notional_prdc,
  fx_base = coupon_fx_base_prdc,
  leverage = coupon_leverage_prdc,
  subtraction = coupon_subtraction_prdc,
  floor_rate = coupon_floor_prdc,
  cap_rate = coupon_cap_prdc,
  intro_coupon_dates = intro_coupon_dates_prdc,
  intro_coupon_rate = intro_coupon_rate_prdc,
  confidence_level = 0.95
)

path_present_value_tbl <- monte_carlo_error_result$path_present_values
monte_carlo_error_tbl <- monte_carlo_error_result$summary

GaussQuant::show_tbl_GQH(
  monte_carlo_error_tbl,
  "PRDC Monte Carlo error estimate",
  n = 20
)

#> # A tibble: 7 x 2
#>   metric                value
#>   <chr>                <dbl>
#> 1 number_of_paths        300
#> 2 mean_present_value 335361061.
#> 3 present_value_sd    35421784.
#> 4 standard_error      2045078.
#> 5 lower_confidence_bound 331352782.
#> 6 upper_confidence_bound 339369339.
#> 7 confidence_level      0.95
```

パス数を 4 倍にすると、他の条件が同じであれば標準誤差は概ね半分になる。ただし、計算量も増加する。実務では、パス数を増やすだけでなく、Antithetic Variates、Control Variates、Quasi-Monte Carlo などの分散削減法を併用する。

9.6 モデルの限界と Part II 総括

9.6.1 PRDC 評価の限界

本章のモデルは、PRDC の基本的な非線形 Coupon と Monte Carlo 評価を理解するための簡略モデルである。主な制約は次のとおりである。

9.6.2 金利を決定論的とした

Domestic、Foreign および Discount Curve をフラットかつ決定論的とした。実際の長期 PRDC では、Domestic 金利と Foreign 金利の変動が、FX Forward、割引係数および Coupon 価値へ影響する。

9.6.3 FX volatility を一定とした

一定の Black volatility を使用した。実際の FX Option 市場には、満期方向と Strike 方向の Volatility Surface が存在する。長期 PRDC では、Smile、Skew および Volatility term structure の影響が重要となる。

9.6.4 相関を明示的に扱っていない

本格的な PRDC モデルでは、Domestic 金利、Foreign 金利および FX の間の相関を扱う必要がある。金利と FX を別々に評価して単純に合成するだけでは、長期の Cross-currency リスクを十分に表現できない。

9.6.5 発行体コール権を含めていない

実際の PRDC には、発行体が Coupon 日に商品を期限前償還できる Callable 条項が含まれることが多い。

発行体は各行使日で、

Call value と Continuation value

を比較する。将来の Continuation value は Closed-form では求めにくいため、Least-Squares Monte Carlo または Backward Induction を使用する。

本章の評価結果は、非 Callable PRDC の Coupon Leg と Redemption Leg の価値である。Callable PRDC の投資家価値は、概念的には次式となる。

$$V_{\text{callable PRDC}} = V_{\text{noncallable PRDC}} - V_{\text{issuer call}}$$

9.6.6 Part II のまとめ

第 II 部では、金融商品のキャッシュフロー、マーケットコンベンション、確率モデルおよび数値計算を段階的に扱った。


```

part2_chapter_summary_tbl <- tibble::tribble(
  ~chapter, ~subject, ~main_method,
  "II-1", "Bonds and market conventions", "Yield, price, duration, and convexity",
  "II-2", "Treasury futures and CTD", "Basis, carry, implied repo, and futures BPV",
  "II-3", "OIS and multi-curve valuation", "Discount and forecast curve separation",
  "II-4", "Black model", "Lognormal forward-option pricing",
  "II-5", "Normal model", "Bachelier pricing for zero and negative rates",
  "II-6", "SABR model", "Volatility-smile calibration",
  "II-7", "Callable Bond and Hull-White", "Short-rate tree and early exercise",
  "II-8", "Credit risk and CDS", "Hazard curve, MidPoint, and ISDA valuation",
  "II-9", "Exotic derivatives and PRDC", "FX-linked coupon Monte Carlo valuation"
)

GaussQuant::show_tbl_GQH(
  part2_chapter_summary_tbl,
  "Part II chapter summary",
  n = 20
)

#> # A tibble: 9 x 3
#>   chapter subject                main_method
#>   <chr>   <chr>                <chr>
#> 1 II-1    Bonds and market conventions Yield, price, duration, and convexity
#> 2 II-2    Treasury futures and CTD      Basis, carry, implied repo, and futures~
#> 3 II-3    OIS and multi-curve valuation Discount and forecast curve separation
#> 4 II-4    Black model                    Lognormal forward-option pricing
#> 5 II-5    Normal model                   Bachelier pricing for zero and negative~
#> 6 II-6    SABR model                    Volatility-smile calibration
#> 7 II-7    Callable Bond and Hull-White Short-rate tree and early exercise
#> 8 II-8    Credit risk and CDS            Hazard curve, MidPoint, and ISDA valuat~
#> 9 II-9    Exotic derivatives and PRDC     FX-linked coupon Monte Carlo valuation

```

債券評価から始まり、金利先物、OIS、Option、Volatility Smile、短期金利モデル、信用リスク、そして複数の市場要因を持つエキゾチック商品へ進んだ。

Part II を通じた共通原則は、商品ごとに異なる数式を暗記することではない。次の要素を分離し、再利用可能な計算層へ落とすことである。

- 契約条件と Schedule
- 市場データと Curve
- リスク中立測度と確率過程
- Cash flow と Payoff
- Pricing Engine または数値計算法
- NPV、Risk および Calibration
- 表示と検証

GaussQuant は、QuantLib SWIG の低水準オブジェクトを金融商品の計算層へまとめる。LagrangeFinance の章側は、商品構造、数式、入力条件、表、グラフおよび金融的解釈に集中する。

9.7 まとめ

PRDC は、将来の FX Rate に連動する Coupon を持つ長期の仕組債である。

Coupon rate は、

$$C(S) = \min \left[C_{\max}, \max \left(a \frac{S}{S_0} - b, C_{\min} \right) \right]$$

という Floor 付き・Cap 付きの非線形 Payoff である。

Garman-Kohlhagen モデルでは、FX Rate のリスク中立ドリフトは Domestic 金利と Foreign 金利の差によって決まる。

$$\frac{dS_t}{S_t} = (r_d - r_f)dt + \sigma_{FX}dW_t$$

Coupon 日の FX 分布を Monte Carlo で生成し、各 Coupon 日の期待 Coupon、Coupon Leg、Redemption Leg および PRDC NPV を計算した。

FX Spot と FX volatility の感応度は、Floor と Cap によって非線形となる。Monte Carlo 評価には統計誤差があるため、NPV だけでなく標準誤差と信頼区間を確認する必要がある。

本章は非 Callable PRDC を対象とした。実際の Callable PRDC には、Domestic 金利、Foreign 金利、FX、Volatility Surface、相関および発行体の早期行使判断を同時に扱う、より高度なモデルが必要となる。

```
chapter09_summary_tbl <- tibble::tribble(
  ~section, ~main_result,
  "Coupon payoff", "Floored and capped FX-linked coupon",
  "FX process", "Garman-Kohlhagen risk-neutral dynamics",
  "Monte Carlo valuation", "Coupon leg and redemption leg NPV",
  "Spot sensitivity", "Nonlinear response caused by cap and floor",
  "Volatility sensitivity", "Distributional effect on coupon regions",
  "Monte Carlo error", "Standard error and confidence interval",
  "Model boundary", "Non-callable deterministic-rate PRDC"
)

GaussQuant::show_tbl_GQH(
  chapter09_summary_tbl,
  "Chapter II-9 summary",
  n = 20
)

#> # A tibble: 7 x 2
#>   section                               main_result
```

```
#>   <chr>                <chr>
#> 1 Coupon payoff      Floored and capped FX-linked coupon
#> 2 FX process          Garman-Kohlhagen risk-neutral dynamics
#> 3 Monte Carlo valuation Coupon leg and redemption leg NPV
#> 4 Spot sensitivity    Nonlinear response caused by cap and floor
#> 5 Volatility sensitivity Distributional effect on coupon regions
#> 6 Monte Carlo error    Standard error and confidence interval
#> 7 Model boundary      Non-callable deterministic-rate PRDC

cat(
  "\nChapter II-9 Exotic derivatives and PRDC completed successfully.\n"
)
#>
#> Chapter II-9 Exotic derivatives and PRDC completed successfully.
```

第 III 部 エンタープライズ金融計算 — 並 列・分散処理から自動微分へ

第 III-1 章 クオন্ツ計算の進化 — 並列化から自動微分へ

金融工学は、数学理論だけで発展してきたのではない。1960 年代以降、金融理論の発展とコンピュータの発展は互いに影響しながら進んできた。理論上は表現できても計算できなかった問題が、計算機能力の向上によって実務へ取り込まれ、逆に金融商品の複雑化が新しい計算技術が必要としてきた。

本章では、まずこの約 60 年の流れを大きく振り返る。次に、2020 年代に Cloud、HPC、GPU、Open Source Software、Tensor 計算 Library がそろったことで、金融計算がどこまで変わったのかを整理する。そのうえで、第 III 部前半の出発点として、一台の PC の複数 Core を利用する並列計算を R の `future` / `furrr` で実装する。

第 III 部全体の中心にあるのは、金融計算を高速化する二つの方向である。一つは、CPU、GPU、Grid、Cloud などの計算資源を拡張する **Hardware / Infrastructure Acceleration** であり、もう一つは、Automatic Differentiation によって計算方法そのものを変える **Algorithmic Acceleration** である。

金融工学とコンピュータの 60 年

1960 年代から 1970 年代にかけて、Portfolio Theory、CAPM、Black-Scholes / Merton モデルなど、現代金融工学の基礎となる理論が整備された。当時の計算資源は現在と比べて極めて限られており、実務へ取り込みやすいモデルは解析解を持つものが中心であった。

1973 年の Black-Scholes / Merton モデルは、その象徴的な例である。European Option の価格を

$$C = C(S, K, r, \sigma, T)$$

とすると、Delta、Gamma、Vega などの Greeks も解析的に表すことができる。

しかし 1980 年代以降、早期行使、経路依存性、複雑な Payoff、複数のリスクファクターを持つ商品が増えるにつれて、閉形式の解析解だけでは十分でなくなった。PDE、Finite Difference、Binomial / Trinomial Tree などの数値計算が実務へ広がった。

1990 年代には Monte Carlo Simulation が、複雑なデリバティブ評価や VaR をはじめとするリスク管理の重要な計算基盤となった。価格は多数の Path または Scenario の平均として、概念的に

$$\hat{V} = \frac{1}{M} \sum_{m=1}^M V^{(m)}$$

と評価される。Path 数を増やせば統計的精度を高められる一方、計算量も増加する。さらに Portfolio、Scenario、Risk Factor が増えると、必要となる Pricing 回数は急速に膨らむ。

2000 年代には、大規模 Portfolio や複雑な Exotic / Hybrid 商品の評価を支えるため、金融機関で On-premise Grid Computing が広く利用されるようになった。Portfolio や Scenario を多数の Job へ分割し、Scheduler が複数の Server へ配布するという構造である。

同じ時期、CPU は単純な Clock Speed の向上から Multi-core 化へ進んだ。一台の Server 内部でも複数の Task を同時に実行できるようになり、金融計算は「一つずつ順番に計算する」構造から「独立した計算を分割して同時に処理する」構造へ移っていった。

2010 年代には GPU による Massive Parallel Computing と Cloud Computing が急速に普及した。Monte Carlo のように、同種の計算を多数の Path へ適用する処理は GPU と相性がよく、Tensor 演算によって大規模な計算をまとめて処理できるようになった。

この流れを大きく整理すると、次のようになる。

1960s-1970s

Portfolio Theory / CAPM / Analytical Pricing

|

v

1980s

PDE / Finite Difference / Tree

|

v

1990s

Monte Carlo / VaR / Bump & Revalue

|

v

2000s

On-premise Grid / Job Scheduler / Multi-core

|

v

2010s

GPU / Cloud / Open Source / Tensor Computing

|

v

2020s

Flexible HPC / Massive Monte Carlo / Automatic Differentiation

2020 年代に変わった金融計算の前提

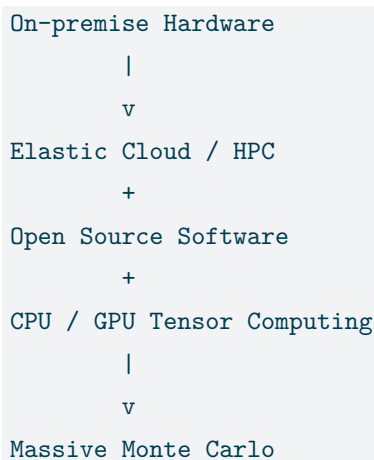
2020 年代の変化は、単にコンピュータが高速になったことではない。大きいのは、計算資源を「所有する」ことから、必要ときに必要な規模だけ「利用する」方向へ移ったことである。

従来の On-premise 環境では、大規模な計算を行うには Server、Storage、Network、Schedulerなどを自ら用

意し、最大負荷を想定した設備投資を行う必要があった。現在は Cloud や HPC Service を利用することで、計算負荷に応じて CPU や GPU を柔軟に追加できる。SaaS 型・Managed Service 型の計算基盤も増え、インフラの構築と保守に必要な負担は大きく低下した。

同時に、R や Python を中心とする Open Source Software が成熟した。数値計算、統計、最適化、Machine Learning、Distributed Computing、GPU Tensor 計算を、商用の専用計算環境だけに依存せず組み合わせることができる。

この二つが重なることで、以前は大規模金融機関の専用 Grid でなければ現実的でなかった計算を、より安価かつ柔軟な環境で実行できるようになった。問題設定と計算環境によって必要時間は大きく異なるものの、数百万から数千万、さらに十分な GPU / HPC 資源を用いる場合には数億 Path 規模を視野に入れた Monte Carlo Simulation も現実的な設計対象となっている。



Hardware Acceleration だけでは残る問題

計算資源を増やすことで、一回の Pricing は速くできる。しかし Risk Sensitivity を Bump & Revalue で求める場合、Risk Factor ごとに Pricing を繰り返す構造は残る。

例えば中心差分では、

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x-h)}{2h}$$

となる。Risk Factor が 100 個あれば、多数の Bump を作り、そのたびに大規模 Monte Carlo を再実行する必要がある。

したがって、

More Cores / More GPUs $\neq$ Fewer Revaluations

である。Hardware を増やすだけでは、Bump & Revalue という Algorithm そのものが要求する再計算回数は減らない。

Automatic Differentiation が実用技術になった理由

Automatic Differentiation 自体は新しい数学ではない。しかし近年、TensorFlow、Keras、PyTorch、R の torch など、Deep Learning の発展によって成熟した Tensor 計算 Library が、安価で安定した Automatic Differentiation 環境を提供するようになった。

これらの Library では、Pricing を Tensor 演算として記述すると、その計算過程が Computational Graph として保持される。最終的な価格から Reverse Mode で Graph を逆向きにたどることで、多数の入力 Risk Factor に対する Gradient を効率的に取得できる。

$$(x_1, x_2, \dots, x_n) \longrightarrow \text{Monte Carlo Pricing} \longrightarrow V$$

に対して、

$$\nabla V = \left(\frac{\partial V}{\partial x_1}, \dots, \frac{\partial V}{\partial x_n} \right)$$

を求める。厳密には Forward Pricing に続いて Backward Pass を行うが、Risk Factor ごとに独立した Monte Carlo Pricing を何度もやり直す必要はない。

ここで金融計算は、

$$\text{Hardware Acceleration} \longrightarrow \text{Hardware} + \text{Algorithmic Acceleration}$$

へ進む。

第 III 部で扱う二つの計算技術

第 III 部では、この二つの方向を R で実装する。

前半の第 III-2 章から第 III-4 章では、future / furrr を出発点として、PC 内の並列処理、非同期実行、Job / Task Control、分散計算、そしてエンタープライズ金融計算システムの構造を扱う。計算をどのように分割し、どこで実行するかが中心となる。

後半の第 III-5 章から第 III-7 章では、R の torch を用いて Automatic Differentiation による Risk Sensitivity を実装する。対象は Option から Yield Curve、さらに PRDC の多因子 Risk へ段階的に拡張する。

$$\text{Option Greeks} \longrightarrow \text{Interest Rate Curve Risk} \longrightarrow \text{PRDC Multi-factor Risk}$$

本部全体の到達点は、次の形で表すことができる。

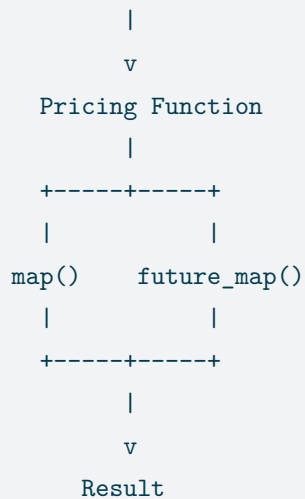
$$\text{Scale Up / Scale Out} + \text{Automatic Differentiation} = \text{Enterprise Differentiable Risk Engine}$$

まず本章では、その第一歩として一台の PC の中で計算を分割し、複数 Worker へ配布する並列計算を実装する。

R における並列コンピューティング

R では `purrr` による逐次処理と、`future` / `furrr` による並列処理をよく似た形で記述できる。金融計算では、入力データを `tibble` として保持し、Pricing Function を各行へ適用する構造にすると、逐次計算から並列計算へ移行しやすい。

Trade / Market / Scenario Data



小さな現在価値計算

まず、単純な現在価値計算を Task として定義する。

```
calculate_present_value_LFL <- function(cashflow, discount_rate, maturity) {
  tibble(
    cashflow = cashflow,
    discount_rate = discount_rate,
    maturity = maturity,
    present_value = cashflow / (1 + discount_rate)^maturity
  )
}
```

計算対象を `tibble` として用意する。

```
tbl_calculation <- tribble(
  ~scenario_ID, ~cashflow, ~discount_rate, ~maturity,
  1L, 100, 0.010, 1,
  2L, 100, 0.015, 2,
  3L, 100, 0.020, 3,
  4L, 100, 0.025, 4,
  5L, 100, 0.030, 5,
```

```

6L, 100, 0.035, 6
)

show_table_LFL(tbl_calculation)
#> # A tibble: 6 x 4
#>   sce_ID cashflow dis_rate maturity
#>   <int>    <dbl>    <dbl>    <dbl>
#> 1     1      100     0.01      1
#> 2     2      100     0.015     2
#> 3     3      100     0.02      3
#> 4     4      100     0.025     4
#> 5     5      100     0.03      5
#> 6     6      100     0.035     6

```

逐次計算

最初に `purrr::pmap()` で逐次実行する。

```

tbl_sequential_result <- tbl_calculation |>
  mutate(
    result = purrr::pmap(
      list(cashflow, discount_rate, maturity),
      calculate_present_value_LFL
    )
  ) |>
  select(scenario_ID, result) |>
  tidyr::unnest(result)

show_table_LFL(tbl_sequential_result)
#> # A tibble: 6 x 5
#>   sce_ID cashflow dis_rate maturity pre_value
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1     1      100     0.01      1    99.010
#> 2     2      100     0.015     2    97.066
#> 3     3      100     0.02      3    94.232
#> 4     4      100     0.025     4    90.595
#> 5     5      100     0.03      5    86.261
#> 6     6      100     0.035     6    81.350

```

Multi-core による並列計算

次に `future::multisession` を利用して Worker を起動する。利用可能な Core をすべて占有せず、この例では最大 2 Worker に制限する。

```

available_cores <- future::availableCores()
workers <- max(1L, min(2L, available_cores - 1L))

future::plan(
  future::multisession,
  workers = workers
)

tbl_future_setting <- tibble(
  available_cores = available_cores,
  workers = workers
)

show_table_LFL(tbl_future_setting)
#> # A tibble: 1 x 2
#>   ava_cores workers
#>   <int>    <int>
#> 1      8      2

```

計算ロジックを変えず、`purrr::pmap()` を `furrr::future_pmap()` へ置き換える。

```

tbl_parallel_result <- tbl_calculation |>
  mutate(
    result = furrr::future_pmap(
      list(cashflow, discount_rate, maturity),
      calculate_present_value_LFL,
      .options = furrr::furrr_options(seed = TRUE)
    )
  ) |>
  select(scenario_ID, result) |>
  tidyr::unnest(result)

show_table_LFL(tbl_parallel_result)
#> # A tibble: 6 x 5
#>   sce_ID cashflow dis_rate maturity pre_value
#>   <int>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1      1      100    0.01      1    99.010
#> 2      2      100    0.015     2    97.066
#> 3      3      100    0.02      3    94.232
#> 4      4      100    0.025     4    90.595
#> 5      5      100    0.03      5    86.261
#> 6      6      100    0.035     6    81.350

```

逐次計算と並列計算が同じ結果を返すことを確認する。

```
tbl_result_comparison <- tbl_sequential_result |>
  select(
    scenario_ID,
    sequential_present_value = present_value
  ) |>
  left_join(
    tbl_parallel_result |>
      select(
        scenario_ID,
        parallel_present_value = present_value
      ),
    by = "scenario_ID"
  ) |>
  mutate(
    difference = parallel_present_value - sequential_present_value
  )

show_table_LFL(tbl_result_comparison)
#> # A tibble: 6 x 4
#>   sce_ID seq_value par_value difference
#>   <int>   <dbl>   <dbl>   <dbl>
#> 1     1    99.010    99.010         0
#> 2     2    97.066    97.066         0
#> 3     3    94.232    94.232         0
#> 4     4    90.595    90.595         0
#> 5     5    86.261    86.261         0
#> 6     6    81.350    81.350         0

stopifnot(
  isTRUE(
    all.equal(
      tbl_result_comparison$sequential_present_value,
      tbl_result_comparison$parallel_present_value,
      tolerance = 1e-12
    )
  )
)
```

計算終了後は Plan を逐次処理へ戻す。

```
future::plan(future::sequential)
```

並列化による処理時間の短縮

小さすぎる Task では Worker 生成やデータ転送の Overhead が支配的になり、並列化が必ずしも高速になるとは限らない。そこで、一定時間を要する独立 Task を使って逐次処理と並列処理を比較する。

```
calculate_task_LFL <- function(task_ID) {
  Sys.sleep(0.10)

  tibble(
    task_ID = task_ID,
    result = sqrt(task_ID)
  )
}

vec_task_ID <- seq_len(12L)

sequential_time <- system.time(
  tbl_sequential_benchmark <- purrr::map(
    vec_task_ID,
    calculate_task_LFL
  )
)[["elapsed"]]

future::plan(
  future::multisession,
  workers = workers
)

parallel_time <- system.time(
  tbl_parallel_benchmark <- furrr::future_map(
    vec_task_ID,
    calculate_task_LFL,
    .options = furrr::furrr_options(seed = TRUE)
  )
)[["elapsed"]]

future::plan(future::sequential)

tbl_benchmark <- tribble(
  ~method, ~elapsed_seconds,
  "purrr::map", sequential_time,
  "furrr::future_map", parallel_time
) |>
mutate(
```

```

    relative_to_sequential = elapsed_seconds / sequential_time
  )

show_table_LFL(tbl_benchmark)
#> # A tibble: 2 x 3
#>   method          ela_secon rel_seque
#>   <chr>          <dbl>     <dbl>
#> 1 purrr::map      1.4       1
#> 2 furrr::future_map 1       0.71429

```

この例の目的は特定の高速化率を保証することではない。重要なのは、同じ Pricing Function を多数の独立 Task へ適用する場合、Task を複数 Worker へ配分することで実行時間を短縮できるという構造である。

一台の PC における並列化の限界

Multi-core による並列化には明確な限界がある。一台のコンピュータで利用できる CPU Core、Memory、GPU などの計算資源には上限があるためである。

さらに実務の Risk Calculation では、数件の Task ではなく、

- 数十万件の Trade
- 多数の Market Scenario
- Monte Carlo Path
- Stress Scenario
- Sensitivity Calculation

を同時に処理することがある。

この規模になると、一台の PC の Core を増やすだけでは対応できない。必要になるのは、Task を**複数のコンピュータへ配布する仕組み**である。

Single Machine Parallelism → Distributed Computing

また、計算資源を増やして Pricing を高速化しても、Sensitivity を Bump & Revalue で求める場合には、リスクファクターごとに Pricing を繰り返す構造そのものは残る。この問題は、本部後半の自動微分で改めて扱う。

まとめ

本章では、金融工学の計算が解析解から数値計算、Monte Carlo へ発展するにつれて計算量が増大し、その対応として Multi-core CPU や並列処理が重要になったことを確認した。

R では `future` / `furrr` を利用することで、逐次計算の構造を大きく変えずに一台の PC 上で複数 Worker へ Task を配分できる。

しかし、エンタープライズ金融計算では一台の PC だけでは計算資源が不足する。次章では、Task を複数の計算機へ配布する**分散コンピューティング**と **Job / Task Control** へ進む。

第 III-2 章 非同期・並列計算 — future / furrr による Task 実行

第 III-1 章では、金融工学と計算技術の発展を振り返り、一台の PC 上で `future` / `furrr` を利用する並列計算を実装した。そこで使える計算資源は、あくまで一台の Machine に搭載された CPU Core と Memory の範囲に限られる。

実務の Risk Calculation では、Trade、Scenario、Monte Carlo Path を多数の独立 Task へ分割し、それらを複数の計算資源へ配布する必要がある。本章では、PC 内の Parallel Computing から一段進み、**Task 生成 → Job Queue → Scheduler → Worker → Result Aggregation** という分散コンピューティングの基本構造を R で表現する。

```

Input Data
  |
  v
Task Generation
  |
  v
Job Queue
  |
  v
Scheduler
  |
  +---+---+---+
  |   |   |   |
W1 W2 W3 W4
  |   |   |   |
  +---+---+---+
  |
  v
Result Aggregation

```

分散計算の基本構造

```

tbl_task <- tribble(
  ~task_ID, ~input_value, ~worker_group,

```

```

1L, 10, "A",
2L, 20, "A",
3L, 30, "B",
4L, 40, "B",
5L, 50, "C",
6L, 60, "C"
)

show_table_LFL(tbl_task)
#> # A tibble: 6 x 3
#>   tas_ID inp_value wor_group
#>   <int>     <dbl> <chr>
#> 1     1         10 A
#> 2     2         20 A
#> 3     3         30 B
#> 4     4         40 B
#> 5     5         50 C
#> 6     6         60 C

```

```

calculate_task_result_LFL <- function(task_ID, input_value, worker_group) {
  tibble(
    task_ID = task_ID,
    worker_group = worker_group,
    input_value = input_value,
    output_value = sqrt(input_value)
  )
}

```

Remote Function と Worker 実行

```

tbl_remote_request <- tbl_task |>
  transmute(
    request_ID = paste0(
      "REQ-",
      stringr::str_pad(task_ID, width = 3, pad = "0")
    ),
    task_ID,
    payload = purrr::pmap(
      list(task_ID, input_value, worker_group),
      \(task_ID, input_value, worker_group) {
        list(
          task_ID = task_ID,
          input_value = input_value,

```



```

        worker_group = worker_group
      )
    }
  )
)

```

```

show_table_LFL(
  tbl_remote_request |>
    select(request_ID, task_ID)
)

```

```

#> # A tibble: 6 x 2
#>   req_ID tas_ID
#>   <chr>   <int>
#> 1 REQ-001     1
#> 2 REQ-002     2
#> 3 REQ-003     3
#> 4 REQ-004     4
#> 5 REQ-005     5
#> 6 REQ-006     6

```

```

remote_calculate_task_LFL <- function(payload) {
  calculate_task_result_LFL(
    task_ID = payload$task_ID,
    input_value = payload$input_value,
    worker_group = payload$worker_group
  )
}

```

```

tbl_remote_result <- tbl_remote_request |>
  mutate(
    response = purrr::map(
      payload,
      remote_calculate_task_LFL
    )
  ) |>
  select(request_ID, response) |>
  tidyr::unnest(response)

```

```

show_table_LFL(tbl_remote_result)
#> # A tibble: 6 x 5
#>   req_ID tas_ID wor_group inp_value out_value
#>   <chr>   <int> <chr>         <dbl>   <dbl>
#> 1 REQ-001     1 A             10    3.1623
#> 2 REQ-002     2 A             20    4.4721

```

```
#> 3 REQ-003      3 B      30    5.4772
#> 4 REQ-004      4 B      40    6.3246
#> 5 REQ-005      5 C      50    7.0711
#> 6 REQ-006      6 C      60    7.7460
```

Job Queue と Scheduler

```
tbl_job <- tbl_task |>
  transmute(
    job_ID = paste0(
      "JOB-",
      stringr::str_pad(task_ID, width = 3, pad = "0")
    ),
    task_ID,
    status = "queued",
    worker_group,
    input_value
  )
```

```
show_table_LFL(tbl_job)
#> # A tibble: 6 x 5
#>   job_ID  tas_ID status wor_group inp_value
#>   <chr>    <int> <chr>  <chr>      <dbl>
#> 1 JOB-001      1 queued A          10
#> 2 JOB-002      2 queued A          20
#> 3 JOB-003      3 queued B          30
#> 4 JOB-004      4 queued B          40
#> 5 JOB-005      5 queued C          50
#> 6 JOB-006      6 queued C          60
```

```
tbl_job_status <- tribble(
  ~job_ID, ~status, ~sequence,
  "JOB-001", "queued", 1L,
  "JOB-001", "running", 2L,
  "JOB-001", "completed", 3L,
  "JOB-002", "queued", 1L,
  "JOB-002", "running", 2L,
  "JOB-002", "failed", 3L
)
```

```
show_table_LFL(tbl_job_status)
#> # A tibble: 6 x 3
#>   job_ID  status  sequence
#>   <chr>    <chr>    <dbl>
```

```
#>   <chr>   <chr>         <int>
#> 1 JOB-001 queued           1
#> 2 JOB-001 running          2
#> 3 JOB-001 completed         3
#> 4 JOB-002 queued           1
#> 5 JOB-002 running          2
#> 6 JOB-002 failed           3
```

Slurm のような Scheduler では、Task を Job として登録し、利用可能な Worker へ割り当てる。R 側の Pricing Function は可能な限り変更せず、「どこで実行するか」を実行基盤側へ分離することが重要である。

分散システムのデータ設計

```
tbl_market_data <- tribble(
  ~market_data_ID, ~discount_rate, ~fx_spot,
  "MKT-001", 0.020, 150,
  "MKT-002", 0.025, 155
)

tbl_calibration_data <- tribble(
  ~calibration_ID, ~volatility, ~correlation,
  "CAL-001", 0.15, 0.20,
  "CAL-002", 0.20, 0.40
)

tbl_trade_data <- tribble(
  ~trade_ID, ~notional, ~maturity,
  "TRD-001", 1000000, 3,
  "TRD-002", 2000000, 5
)

tbl_scenario_data <- tidyr::crossing(
  trade_ID = tbl_trade_data$trade_ID,
  market_data_ID = tbl_market_data$market_data_ID,
  calibration_ID = tbl_calibration_data$calibration_ID
) |>
  mutate(
    scenario_ID = row_number(),
    .before = 1
  )

show_table_LFL(tbl_scenario_data)
#> # A tibble: 8 x 4
```

```
#>   sce_ID tra_ID  mar_ID  cal_ID
#>   <int> <chr>   <chr>   <chr>
#> 1     1 TRD-001 MKT-001 CAL-001
#> 2     2 TRD-001 MKT-001 CAL-002
#> 3     3 TRD-001 MKT-002 CAL-001
#> 4     4 TRD-001 MKT-002 CAL-002
#> 5     5 TRD-002 MKT-001 CAL-001
#> 6     6 TRD-002 MKT-001 CAL-002
#> 7     7 TRD-002 MKT-002 CAL-001
#> 8     8 TRD-002 MKT-002 CAL-002
```

```
tbl_distributed_input <- tbl_scenario_data |>
  left_join(tbl_trade_data, by = "trade_ID") |>
  left_join(tbl_market_data, by = "market_data_ID") |>
  left_join(tbl_calibration_data, by = "calibration_ID")

show_table_LFL(tbl_distributed_input)
#> # A tibble: 8 x 10
#>   sce_ID tra_ID  mar_ID  cal_ID  notional maturity dis_rate fx_spot volatility
#>   <int> <chr>   <chr>   <chr>      <dbl>      <dbl>      <dbl>  <dbl>      <dbl>
#> 1     1 TRD-001 MKT-001 CAL-001 1000000         3    0.02    150      0.15
#> 2     2 TRD-001 MKT-001 CAL-002 1000000         3    0.02    150      0.2
#> 3     3 TRD-001 MKT-002 CAL-001 1000000         3    0.025   155      0.15
#> 4     4 TRD-001 MKT-002 CAL-002 1000000         3    0.025   155      0.2
#> 5     5 TRD-002 MKT-001 CAL-001 2000000         5    0.02    150      0.15
#> 6     6 TRD-002 MKT-001 CAL-002 2000000         5    0.02    150      0.2
#> 7     7 TRD-002 MKT-002 CAL-001 2000000         5    0.025   155      0.15
#> 8     8 TRD-002 MKT-002 CAL-002 2000000         5    0.025   155      0.2
#> # i 1 more variable: correlation <dbl>
```

```
tbl_distributed_files <- list(
  trade_data = tbl_trade_data,
  market_data = tbl_market_data,
  calibration_data = tbl_calibration_data,
  scenario_data = tbl_scenario_data
)

vec_distributed_paths <- tbl_distributed_files |>
  purrr::imap_chr(
    \(data, name) {
      write_derived_parquet_file_LFL(
        data,
        file.path(
          "distributed",
```

```
        paste0(name, ".parquet")
      )
    )
  }
)

tbl_distributed_paths <- tibble(
  data_name = names(vec_distributed_paths),
  file_path = file.path(
    "data",
    "derived",
    "distributed",
    paste0(names(vec_distributed_paths), ".parquet")
  )
)

show_table_LFL(tbl_distributed_paths)
#> # A tibble: 4 x 2
#>   dat_name      fil_path
#>   <chr>        <chr>
#> 1 trade_data    data/derived/distributed/trade_data.parquet
#> 2 market_data   data/derived/distributed/market_data.parquet
#> 3 calibration_data data/derived/distributed/calibration_data.parquet
#> 4 scenario_data data/derived/distributed/scenario_data.parquet
```

まとめ

本章では、一台の Machine 内部の並列化から、複数の計算資源を前提とする分散処理へ考え方を拡張した。重要なのは、計算関数そのものよりも、Task を Job として管理し、Queue へ投入し、Scheduler が Worker へ配布し、結果を再び集約するという Control Plane である。

この構造があれば、Worker の実体は Local Process、別 Server、Container、Cloud Batch、HPC Cluster へ置き換えることができる。次章では、この計算基盤の上に Trade Data、Market Data、Calibration、Scenario、Pricing、Risk Aggregation を統合し、エンタープライズ金融計算システムとして組み立てる。

第 III-3 章 分散計算と Job / Task Control

第 III-1 章では Local Parallel Computing、第 III-2 章では Job / Task Control を含む Distributed Computing を扱った。本章では、それらを実際の金融リスク計算システムの形へ統合する。

エンタープライズ金融計算では、Pricing Function だけがシステムなのではない。Trade Data、Market Data、Calibration Data、Scenario、Calculation Configuration を分離して管理し、それらを Task へ変換し、並列・分散実行し、最終的に Risk Table へ集約する必要がある。

本章では PRDC を例に、**Data → Scenario → Task → Pricing → Risk → Output** という一連の Pipeline を R で構築する。

```
Trade Data
Market Data
Calibration Data
      |
      v
Scenario Grid
      |
      v
Job / Task
      |
      v
Pricing Function
      |
      v
Risk Table
      |
      v
Aggregation / Report
```

PRDC 取引・市場・計算条件

```
tbl_prdc_trade <- tribble(
  ~trade_ID, ~notional, ~maturity_years,
  "PRDC-001", 300000000, 15
)
```

```

show_table_LFL(tbl_prdc_trade)
#> # A tibble: 1 x 3
#>   tra_ID   notional mat_years
#>   <chr>     <dbl>     <dbl>
#> 1 PRDC-001 300000000      15

tbl_prdc_market <- tribble(
  ~market_data_ID, ~fx_spot, ~domestic_rate, ~foreign_rate,
  "MKT-BASE", 150, 0.010, 0.030
)

tbl_prdc_calibration <- tribble(
  ~calibration_ID, ~fx_volatility, ~correlation,
  "CAL-BASE", 0.10, 0.20
)

tbl_prdc_calculation <- tribble(
  ~calculation_ID, ~number_of_paths, ~random_seed,
  "CALC-BASE", 500L, 123L
)

show_table_LFL(tbl_prdc_market)
#> # A tibble: 1 x 4
#>   mar_ID   fx_spot dom_rate for_rate
#>   <chr>     <dbl>     <dbl>   <dbl>
#> 1 MKT-BASE    150      0.01    0.03

show_table_LFL(tbl_prdc_calibration)
#> # A tibble: 1 x 3
#>   cal_ID   fx_volat correlation
#>   <chr>     <dbl>     <dbl>
#> 1 CAL-BASE    0.1      0.2

show_table_LFL(tbl_prdc_calculation)
#> # A tibble: 1 x 3
#>   cal_ID   num_paths ran_seed
#>   <chr>     <int>     <int>
#> 1 CALC-BASE    500      123

```

Scenario Pricing Function

本章では計算基盤に焦点を当てるため、Scenario Pricing Function は軽量な代理モデルとして表現する。第 II 部で構築した PRDC Pricing Function へ置き換えても、Task 生成と並列実行の構造は変わらない。

```

price_prdc_scenario_LFL <- function(
  trade,
  fx_spot,
  fx_volatility,
  domestic_rate,
  foreign_rate
) {
  horizon <- trade$maturity_years[[1]]

  fx_forward <- fx_spot *
    exp((domestic_rate - foreign_rate) * horizon)

  volatility_adjustment <- exp(
    0.5 * fx_volatility^2 * horizon
  )

  tibble(
    trade_ID = trade$trade_ID[[1]],
    fx_spot = fx_spot,
    fx_volatility = fx_volatility,
    domestic_rate = domestic_rate,
    foreign_rate = foreign_rate,
    prdc_proxy_npv = trade$notional[[1]] *
      fx_forward / fx_spot *
      volatility_adjustment
  )
}

```

Scenario Grid

```

base_fx_spot <- tbl_prdc_market$fx_spot[[1]]

tbl_prdc_scenario <- tidyr::crossing(
  fx_spot = base_fx_spot * c(0.90, 0.95, 1.00, 1.05, 1.10),
  fx_volatility = c(0.075, 0.10, 0.125),
  domestic_rate = c(0.005, 0.010, 0.015),
  foreign_rate = c(0.025, 0.030, 0.035)
) |>
mutate(
  scenario_ID = row_number(),
  .before = 1
)

```



```
show_table_LFL(
  tbl_prdc_scenario,
  n = 10
)
#> # A tibble: 135 x 5
#>   sce_ID fx_spot fx_volat dom_rate for_rate
#>   <int>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1     1     135    0.075    0.005    0.025
#> 2     2     135    0.075    0.005    0.03
#> 3     3     135    0.075    0.005    0.035
#> 4     4     135    0.075    0.01     0.025
#> 5     5     135    0.075    0.01     0.03
#> 6     6     135    0.075    0.01     0.035
#> 7     7     135    0.075    0.015    0.025
#> 8     8     135    0.075    0.015    0.03
#> 9     9     135    0.075    0.015    0.035
#> 10    10     135    0.1      0.005    0.025
#> # i 125 more rows
```

並列シナリオ計算

```
available_cores <- future::availableCores()
workers <- max(1L, min(2L, available_cores - 1L))

future::plan(
  future::multisession,
  workers = workers
)

#| warning: false
tbl_prdc_scenario_result <- tbl_prdc_scenario |>
  mutate(
    result = furrr::future_pmap(
      list(
        fx_spot,
        fx_volatility,
        domestic_rate,
        foreign_rate
      ),
      \(fx_spot, fx_volatility, domestic_rate, foreign_rate) {
        price_prdc_scenario_LFL(
          trade = tbl_prdc_trade,
```

```

    fx_spot = fx_spot,
    fx_volatility = fx_volatility,
    domestic_rate = domestic_rate,
    foreign_rate = foreign_rate
  )
},
.options = furrr::furrr_options(seed = TRUE)
) |>
select(scenario_ID, result) |>
tidyr::unnest(result)

future::plan(future::sequential)

show_table_LFL(
  tbl_prdc_scenario_result,
  n = 10
)
#> # A tibble: 135 x 7
#>   sce_ID tra_ID   fx_spot fx_volat dom_rate for_rate   prd_npv
#>   <int> <chr>     <dbl>   <dbl>   <dbl>   <dbl>     <dbl>
#> 1     1 1 PRDC-001    135    0.075    0.005    0.025 231822032.
#> 2     2 2 PRDC-001    135    0.075    0.005    0.03 215071380.
#> 3     3 3 PRDC-001    135    0.075    0.005    0.035 199531072.
#> 4     4 4 PRDC-001    135    0.075    0.01     0.025 249877294.
#> 5     5 5 PRDC-001    135    0.075    0.01     0.03 231822032.
#> 6     6 6 PRDC-001    135    0.075    0.01     0.035 215071380.
#> 7     7 7 PRDC-001    135    0.075    0.015    0.025 269338775.
#> 8     8 8 PRDC-001    135    0.075    0.015    0.03 249877294.
#> 9     9 9 PRDC-001    135    0.075    0.015    0.035 231822032.
#> 10    10 10 PRDC-001    135    0.1      0.005    0.025 239554866.
#> # i 125 more rows

```

Risk Table

```

tbl_prdc_risk <- tbl_prdc_scenario_result |>
  arrange(
    fx_spot,
    fx_volatility,
    domestic_rate,
    foreign_rate
  ) |>

```

```
mutate(
  npv_change = prdc_proxy_npv -
    median(prdc_proxy_npv)
)

show_table_LFL(
  tbl_prdc_risk,
  n = 10
)

#> # A tibble: 135 x 8
#>   sce_ID tra_ID   fx_spot fx_volat dom_rate for_rate   prd_npv npv_chang
#>   <int> <chr>     <dbl>   <dbl>   <dbl>   <dbl>     <dbl>   <dbl>
#> 1     1    PRDC-001    135    0.075    0.005    0.025 231822032. -7732833.
#> 2     2    PRDC-001    135    0.075    0.005    0.03 215071380. -24483485.
#> 3     3    PRDC-001    135    0.075    0.005    0.035 199531072. -40023793.
#> 4     4    PRDC-001    135    0.075    0.01    0.025 249877294. 10322429.
#> 5     5    PRDC-001    135    0.075    0.01    0.03 231822032. -7732833.
#> 6     6    PRDC-001    135    0.075    0.01    0.035 215071380. -24483485.
#> 7     7    PRDC-001    135    0.075    0.015    0.025 269338775. 29783910.
#> 8     8    PRDC-001    135    0.075    0.015    0.03 249877294. 10322429.
#> 9     9    PRDC-001    135    0.075    0.015    0.035 231822032. -7732833.
#> 10    10    PRDC-001    135    0.1      0.005    0.025 239554866.      0
#> # i 125 more rows
```

```
tbl_prdc_spot_risk <- tbl_prdc_risk |>
  group_by(fx_spot) |>
  summarise(
    mean_npv = mean(prdc_proxy_npv),
    .groups = "drop"
  )
```

```
show_table_LFL(tbl_prdc_spot_risk)

#> # A tibble: 5 x 2
#>   fx_spot   mea_npv
#>   <dbl>     <dbl>
#> 1    135 241320900.
#> 2   142.5 241320900.
#> 3    150 241320900.
#> 4   157.5 241320900.
#> 5    165 241320900.
```

```
prdc_risk_output_path <- write_derived_parquet_file_LFL(
  tbl_prdc_risk,
  file.path(
```

```

    "risk_system",
    "prdc_risk_table.parquet"
  )
)

tbl_prdc_risk_output <- tibble(
  file_path = file.path(
    "data",
    "derived",
    "risk_system",
    "prdc_risk_table.parquet"
  )
)

show_table_LFL(tbl_prdc_risk_output)
#> # A tibble: 1 x 1
#>   fil_path
#>   <chr>
#> 1 data/derived/risk_system/prdc_risk_table.parquet

```

計算資源の拡張と残る問題

ここまでの並列・分散計算は、Pricing を多数回実行することでリスクを求める設計である。計算資源を増やせば処理時間を短縮できるが、Risk Factor が増えるほど Bump & Revalue の Pricing 回数が増えるという構造そのものは残る。

```

Hardware Acceleration
  |
  v
More Workers / More Cores
  |
  v
Faster Revaluation
  |
  v
But still many Revaluations

```

ここまでの、第 III 部前半のテーマである「計算を分割し、より多くの計算資源で処理する」構造が完成した。しかし、計算資源を増やしても、Sensitivity のために Pricing そのものを繰り返す構造は残る。

次章では発想を変え、Pricing Program そのものを微分可能な Computational Graph として構築する Automatic Differentiation へ進む。ここが、**Hardware / Infrastructure Acceleration** から **Algorithmic Acceleration** への転換点である。

第 III-4 章エンタープライズ・クオンツ計算基盤

第 III-1 章から第 III-3 章では、金融計算を Task へ分割し、Parallel / Distributed Computing によって大量の Pricing を処理する仕組みを構築した。これは計算資源を増やすことで処理時間を短縮する方法である。

しかし Greeks や Curve Risk を有限差分で求める場合、Risk Factor ごとに価格を再計算する必要がある。Worker や GPU を増やせば一回ごとの再評価は速くなるが、Bump & Revalue が必要とする Pricing 回数そのものは減らない。

そこで本章から、計算資源の拡張ではなく**計算方法そのものを変える**方向へ進む。

Automatic Differentiation（自動微分）は、この問題に対して、価格計算そのものを微分可能な計算グラフとして扱う。

本章では、金融リスク感応度を偏微分として整理したうえで、有限差分、連鎖律、Computational Graph、Forward Mode、Reverse Mode を順に確認する。最後に R の `torch` を用いて、自動微分の最小例を実装する。

金融リスク感応度と偏微分

金融商品の価格を、複数の市場変数に依存する関数として

$$V = V(x_1, x_2, \dots, x_n)$$

と表す。

ここで、 x_i は株価、為替レート、金利、Volatility などのリスクファクターである。

各リスクファクターに対する一次感応度は、

$$\frac{\partial V}{\partial x_i}$$

で表される。

したがって、すべての一次感応度をまとめると、

$$\nabla V = \left(\frac{\partial V}{\partial x_1}, \frac{\partial V}{\partial x_2}, \dots, \frac{\partial V}{\partial x_n} \right)$$

となる。

Option の Delta や Vega、金利商品の DV01、為替商品の FX Delta は、対象となるリスクファクターは異なるものの、数学的にはいずれも価格関数の偏微分として統一的に捉えることができる。

二次感応度も同様である。

例えば Option の Gamma は、

$$\Gamma = \frac{\partial^2 V}{\partial S^2}$$

であり、異なる二つのリスクファクターに対する Cross Gamma は、

$$\frac{\partial^2 V}{\partial x_i \partial x_j}$$

として表される。

このように、金融リスク計測の多くは、価格関数の一次または二次の微分を求める問題として整理できる。

有限差分と Bump & Revalue

最も直接的な感応度計算の一つが有限差分である。

リスクファクター x を微小量 h だけ変化させる Forward Difference では、

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x)}{h}$$

と近似する。

Central Difference では、

$$\boxed{\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x-h)}{2h}}$$

となる。

金融実務では、市場データを一定量 Bump し、商品を再評価することから、この方法は Bump & Revalue と呼ばれる。

実装が比較的容易であり、既存の Pricing Engine をそのまま利用できるという利点がある。

一方で、 n 個のリスクファクターについて Central Difference を計算する場合、単純には各リスクファクターについて Up と Down の再評価が必要になる。

$$\text{Number of Revaluations} \approx 2n$$

Monte Carlo による価格評価自体が重い場合、この再評価回数は大きな負荷となる。

また、有限差分には Bump 幅 h の選択という問題もある。

h が大きすぎれば近似誤差が大きくなり、小さすぎれば浮動小数点演算による丸め誤差の影響を受けやすくなる。

したがって、

$$\text{Finite Difference} = \text{Approximation} + \text{Repeated Pricing}$$

という性質を持つ。

自動微分は数値微分でも記号微分でもない

自動微分は有限差分による数値微分とは異なる。

また、数式全体を記号的に変形して導関数を求める Symbolic Differentiation と異なる。

例えば、

$$y = \exp(x^2)$$

を考える。

記号微分では、

$$\frac{dy}{dx} = 2x \exp(x^2)$$

という新しい数式を導出する。

一方、自動微分では、元の計算を

$$x \rightarrow x^2 \rightarrow \exp(x^2) \rightarrow y$$

という基本演算の列として捉える。

各演算の局所的な微分を既知として、連鎖律によって全体の微分を構成する。

したがって、自動微分の中心にあるのは、

$$\text{Elementary Operations} + \text{Chain Rule}$$

である。

連鎖律

自動微分を理解するうえで最も重要な数学が連鎖律である。

例えば、

$$y = f(x), \quad z = g(y), \quad V = h(z)$$

とする。

x に対する V の感応度は、

$$\frac{\partial V}{\partial x} = \frac{\partial V}{\partial z} \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}$$

となる。

金融商品の価格計算も、多数の小さな計算を順番に実行することで構成される。

例えば金利商品では、

$$\text{Market Quote} \rightarrow \text{Yield Curve} \rightarrow \text{Discount Factor} \rightarrow \text{Cash Flow} \rightarrow PV$$

という計算を考えることができる。

市場クォート q から金利 r が構築され、その金利から割引係数 D が計算され、最終的に価格 V が得られるなら、

$$q \rightarrow r \rightarrow D \rightarrow V$$

である。

したがって、

$$\frac{\partial V}{\partial q} = \frac{\partial V}{\partial D} \frac{\partial D}{\partial r} \frac{\partial r}{\partial q}$$

となる。

この連鎖律をプログラムの計算過程全体に適用することが、Automatic Differentiation の基本的な考え方である。

Computational Graph

プログラムによる計算を、変数と演算の関係として表したものを Computational Graph と呼ぶ。

簡単な例として、

$$a = x^2, \quad b = 3a, \quad V = b + 1$$

を考える。

計算グラフは、

```
x
|
v
x^2
|
v
a
|
v
* 3
|
v
b
|
v
+ 1
|
v
V
```

と表すことができる。

x から V へ向かう方向が通常の価格計算である。

一方、最終結果 V から x へ向かって局所微分を掛け合わせれば、

$$\frac{\partial V}{\partial x}$$

を求めることができる。

この「計算を行った経路を逆向きにたどる」という考え方が、Reverse Mode の直感的な理解につながる。

Forward Mode Automatic Differentiation

Forward Mode では、入力側から出力側へ向かって、値と同時に微分値を伝播させる。

例えば、

$$x \rightarrow a = f(x) \rightarrow b = g(a) \rightarrow V = h(b)$$

という計算では、

$$\frac{\partial a}{\partial x}, \quad \frac{\partial b}{\partial x}, \quad \frac{\partial V}{\partial x}$$

を順方向に計算する。

概念的には、

$$(x, \dot{x}) \rightarrow (a, \dot{a}) \rightarrow (b, \dot{b}) \rightarrow (V, \dot{V})$$

と、値と微分値を同時に運ぶ。

Forward Mode は、入力変数が少なく出力変数が多い問題に適している。

Reverse Mode Automatic Differentiation

Reverse Mode では、まず通常どおり Forward 計算を行い、最終出力を求める。

その後、出力側から入力側へ向かって微分値を逆伝播する。

金融商品の価格を、

$$V = V(x_1, x_2, \dots, x_n)$$

とする。

通常の Pricing では、

$$(x_1, x_2, \dots, x_n) \longrightarrow V$$

と計算する。

Reverse Mode では、得られた V から逆方向に、

$$V \longrightarrow \left(\frac{\partial V}{\partial x_1}, \frac{\partial V}{\partial x_2}, \dots, \frac{\partial V}{\partial x_n} \right)$$

と感応度を伝播する。

金融リスク計測では、一つの価格やポートフォリオ価値に対して、多数の市場リスクファクターの感応度を求めることが多い。

したがって、

Many Inputs → Few Outputs

という構造を持つ金融計算では、Reverse Mode が特に重要となる。

AAD と金融工学

金融分野では、Reverse Mode Automatic Differentiation を Adjoint Algorithmic Differentiation (AAD) という名称で扱うことが多い。

従来の Bump & Revalue では、

$$x_1 \rightarrow V_1, \quad x_2 \rightarrow V_2, \quad \dots, \quad x_n \rightarrow V_n$$

のように、多数の再評価を行う。

これに対して AAD では、Pricing 計算の過程を利用して、最終価格から各入力への感応度を逆向きに伝播させる。

概念的には、

```
Market Risk Factors
x1  x2  x3  ...  xn
 \   |   |      /
 \   |   |      /
    Computational Graph
      |
      v
    Price V
      |
      | backward
      v
dV/dx1 dV/dx2 ... dV/dxn
```

となる。

重要なのは、AAD がより高速な CPU を用いるという話ではなく、リスク計算のアルゴリズムそのものを変更するという点である。

R torch による最小の自動微分

ここから、R の torch を使って実際に自動微分を確認する。

次の関数を考える。

$$y = x^2 + 3x + 1$$

解析的には、

$$\frac{dy}{dx} = 2x + 3$$

である。

$x = 2$ なら、

$$\frac{dy}{dx} = 2 \times 2 + 3 = 7$$

となる。

torch では、微分対象の Tensor に `requires_grad = TRUE` を指定する。

```
x <- torch_tensor(  
  2,  
  requires_grad = TRUE  
)  
  
y <- x^2 + 3 * x + 1  
  
y  
#> torch_tensor  
#> 11  
#> [ CPUFloatType{1} ][ grad_fn = <AddBackward1> ]
```

`backward()` を実行すると、計算グラフを逆向きにたどって勾配が計算される。

```
y$backward()  
  
x$grad  
#> torch_tensor  
#> 7  
#> [ CPUFloatType{1} ]
```

理論値は、

```
analytic_grad <- 2 * 2 + 3

analytic_grad
#> [1] 7
```

である。

torch による自動微分と解析解が一致することを確認できる。

ここで重要なのは、torch が

$$y = x^2 + 3x + 1$$

という数式を記号微分したのではないことである。

実際に実行された Tensor 演算から Computational Graph を構築し、そのグラフに対して連鎖律を逆方向に適用している。

二変数関数の Gradient

次に、

$$V(x, y) = x^2 + xy + y^2$$

を考える。

偏微分は、

$$\frac{\partial V}{\partial x} = 2x + y$$

$$\frac{\partial V}{\partial y} = x + 2y$$

である。

$x = 2$, $y = 3$ なら、

$$\frac{\partial V}{\partial x} = 7, \quad \frac{\partial V}{\partial y} = 8$$

となる。

```
x <- torch_tensor(
  2,
  requires_grad = TRUE
)
```

```

y <- torch_tensor(
  3,
  requires_grad = TRUE
)

value <- x^2 + x * y + y^2

value$backward()

tbl_gradient_result <- tribble(
  ~variable, ~autograd,
  "x", x$grad$item(),
  "y", y$grad$item()
)

tbl_gradient_result
#> # A tibble: 2 x 2
#>   variable autograd
#>   <chr>      <dbl>
#> 1 x          7
#> 2 y          8

```

この例では、一つの Scalar 出力 `value` から、二つの入力 x と y に対する Gradient を一度の `backward()` で取得している。

これは金融商品の価格

$$V = V(S, r, \sigma, \dots)$$

から、

$$\Delta, \quad \rho, \quad Vega, \quad \dots$$

を求める構造と同じである。

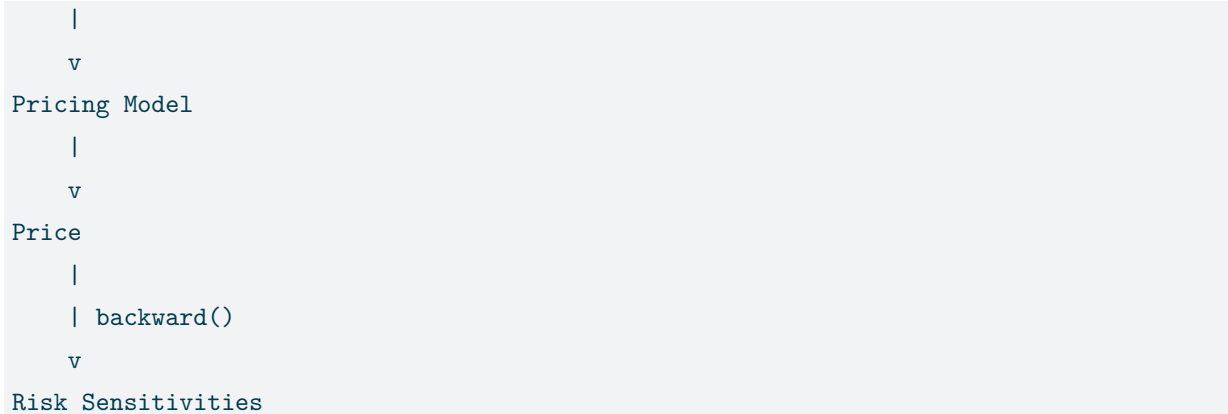
金融リスク計測への接続

ここまでの内容を金融工学へ置き換えると、

```

Market Data
|
v
Model Parameters

```



となる。

すなわち、

$$\text{Pricing} + \text{Computational Graph} + \text{Reverse Mode} = \text{Risk Sensitivities}$$

という構造である。

第 III 部では、この考え方を段階的に金融商品へ適用する。

まず Option について、

$$\Delta, \Gamma, Vega, Theta$$

を自動微分で求める。

次に金利商品について、

$$DV01$$

および Yield Curve 全体のリスク感応度へ拡張する。

最後に PRDC について、

$$FX + JPY \text{ Rates} + Foreign \text{ Rates}$$

という複数のリスクファクターを持つ商品へ適用する。

まとめ

本章では、自動微分の数学的な基礎を整理した。

有限差分は、

Bump → Revalue → Approximate Sensitivity

という方法である。

これに対して自動微分は、

Pricing Operations → Computational Graph → Chain Rule → Exact Derivative of the Program

という考え方をとる。

特に金融工学では、

$$(x_1, x_2, \dots, x_n) \longrightarrow V$$

という多数入力・少数出力の問題が多いため、Reverse Mode Automatic Differentiation、すなわち AAD が重要となる。

R の `torch` では、この Reverse Mode を

```
value$backward()
```

によって実行し、

```
x$grad
```

によって入力変数に対する Gradient を取得できる。

次章では European Option を題材として、GaussQuant による基準値と Monte Carlo + `torch` を比較し、

$$\Delta, \Gamma, Vega, Theta$$

を実際に計算する。

第 III-5 章 Option Greeks — Monte Carlo × Automatic Differentiation

前章では、Bump & Revalue から Automatic Differentiation へ発想を切り替え、偏微分、連鎖律、Computational Graph、Forward Mode、Reverse Mode を整理した。本章からは、その仕組みを実際の金融商品へ適用する。

本章では、解析的な Black-Scholes 式を `torch` で書き直すのではなく、既存の金融工学ライブラリである GaussQuant を基準値として利用し、同じ European Option を Monte Carlo Simulation と R の `torch` によって評価する。

比較する量は、

$$NPV, \Delta, \Gamma, Vega, Theta$$

である。

本章の中心は、

$$\text{GaussQuant} \leftrightarrow \text{Monte Carlo} + \text{torch} + \text{Automatic Differentiation}$$

という比較である。

GaussQuant は QuantLib を利用した既存の Pricing Engine として使い、`torch` では Monte Carlo Pricing そのものを Differentiable Computational Graph として構築する。

計算条件

同じ European Call Option を GaussQuant と Monte Carlo で評価する。

```
valuation_date <- "2026-01-02"
maturity_date <- "2027-01-02"

spot_value <- 100
strike_value <- 100
risk_free_rate_value <- 0.03
dividend_yield_value <- 0
volatility_value <- 0.20
```

```

maturity_value <- as.numeric(
  as.Date(maturity_date) -
  as.Date(valuation_date)
) / 365

n_paths <- 200000L
mc_seed <- 1234L

tbl_option_input <- tribble(
  ~parameter, ~value,
  "spot", spot_value,
  "strike", strike_value,
  "risk_free_rate", risk_free_rate_value,
  "dividend_yield", dividend_yield_value,
  "volatility", volatility_value,
  "maturity", maturity_value,
  "n_paths", n_paths
)

tbl_option_input
#> # A tibble: 7 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 spot          100
#> 2 strike         100
#> 3 risk_free_rate 0.03
#> 4 dividend_yield 0
#> 5 volatility     0.2
#> 6 maturity        1
#> 7 n_paths      200000

```

ここでは比較を明確にするため、フラットな金利、配当利回り、Volatility を用いる。

GaussQuant による基準価格と Greeks

GaussQuant では、European Option を QuantLib オブジェクトとして構築できる。

```

option_gql <- GaussQuant::make_european_option_GQL(
  spot = spot_value,
  strike = strike_value,
  maturity_date = maturity_date,
  option_type = "call",
  valuation_date = valuation_date,

```

```

    risk_free_rate = risk_free_rate_value,
    dividend_yield = dividend_yield_value,
    volatility = volatility_value
)

```

価格と Greeks をまとめて取得する。

```

tbl_gaussquant_greeks <- GaussQuant::option_summary_GQL(
  option_gql
) |>
  filter(
    metric %in% c(
      "npv",
      "delta",
      "gamma",
      "vega",
      "theta"
    )
  )

tbl_gaussquant_greeks
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        9.4134
#> 2 delta      0.59871
#> 3 gamma      0.019333
#> 4 vega      38.667
#> 5 theta     -5.3804

```

ここで得られる値を、本章における Benchmark とする。

Monte Carlo による Option Pricing

Risk-Neutral Measure の下で、配当利回り q を持つ株価過程を、

$$\frac{dS_t}{S_t} = (r - q)dt + \sigma dW_t$$

とする。

満期 T の株価は、

$$S_T = S_0 \exp \left[\left(r - q - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right]$$

である。

ここで、

$$Z \sim N(0, 1)$$

である。

European Call の Monte Carlo 価格は、

$$\hat{V} = e^{-rT} \frac{1}{M} \sum_{m=1}^M \max(S_T^{(m)} - K, 0)$$

となる。

Common Random Numbers

GaussQuant との比較や感応度の検算では、Monte Carlo Noise をできるだけ抑える必要がある。

本章では乱数を一度だけ生成し、価格計算と感応度計算で同じ乱数を使用する。

```
torch_manual_seed(
  mc_seed
)

z_t <- torch_randn(
  n_paths
)
```

乱数 Z は市場リスクファクターではないため、`requires_grad = TRUE` は指定しない。

微分可能な Monte Carlo Pricing

Monte Carlo Pricing を Tensor 演算だけで構成する。

```
european_call_mc_torch <- function(
  spot,
  strike,
  rate,
  dividend_yield,
  volatility,
  maturity,
  z
) {
  terminal_spot <- spot *
  torch_exp(
```

```

    (
      rate -
        dividend_yield -
        0.5 * volatility^2
    ) * maturity +
    volatility *
      torch_sqrt(maturity) *
      z
  )

  payoff <- torch_relu(
    terminal_spot -
      strike
  )

  torch_exp(
    -rate * maturity
  ) *
  torch_mean(
    payoff
  )
}

```

この関数では、

$$S_0, \quad r, \quad q, \quad \sigma, \quad T$$

から Terminal Spot、Payoff、Discounting、Monte Carlo Average までが一つの Computational Graph として接続される。

torch Tensor の設定

微分対象となる市場変数を Tensor として定義する。

```

spot_t <- torch_tensor(
  spot_value,
  requires_grad = TRUE
)

strike_t <- torch_tensor(
  strike_value
)

rate_t <- torch_tensor(

```

```

    risk_free_rate_value,
    requires_grad = TRUE
  )

  dividend_yield_t <- torch_tensor(
    dividend_yield_value
  )

  volatility_t <- torch_tensor(
    volatility_value,
    requires_grad = TRUE
  )

  maturity_t <- torch_tensor(
    maturity_value,
    requires_grad = TRUE
  )

```

Monte Carlo NPV

同じ乱数を用いて Monte Carlo 価格を計算する。

```

option_npv_mc_t <- european_call_mc_torch(
  spot = spot_t,
  strike = strike_t,
  rate = rate_t,
  dividend_yield = dividend_yield_t,
  volatility = volatility_t,
  maturity = maturity_t,
  z = z_t
)

option_npv_mc_t
#> torch_tensor
#>  9.4665
#> [ CPUFloatType{1} ][ grad_fn = <MulBackward0> ]

```

Delta・Vega・Theta の自動微分

Monte Carlo 価格を最終出力として Reverse Mode Automatic Differentiation を行う。

```

first_grad <- autograd_grad(
  outputs = option_npv_mc_t,
  inputs = list(

```

```

    spot_t,
    volatility_t,
    maturity_t
  )
)

delta_mc <- first_grad[[1]]$item()
vega_mc <- first_grad[[2]]$item()

## maturity は残存期間 T なので、
## Calendar-Time Theta は -dV/dT とする。
theta_mc <- -first_grad[[3]]$item()

tbl_mc_first_order_greeks <- tribble(
  ~metric, ~value,
  "delta", delta_mc,
  "vega", vega_mc,
  "theta", theta_mc
)

tbl_mc_first_order_greeks
#> # A tibble: 3 x 2
#>   metric    value
#>   <chr>     <dbl>
#> 1 delta    0.60086
#> 2 vega    38.900
#> 3 theta   -5.4086

```

ここで、

$$\Delta = \frac{\partial V}{\partial S_0}, \quad Vega = \frac{\partial V}{\partial \sigma}, \quad Theta = -\frac{\partial V}{\partial T}$$

を Monte Carlo Pricing Graph から直接取得している。

Gamma と Payoff の Kink

Gamma には注意が必要である。

European Call の Payoff、

$$\max(S_T - K, 0)$$

は Strike で Kink を持つ。

Pathwise Automatic Differentiation でこの Payoff をそのまま二階微分すると、各 Path ではほとんど至るところで二階微分がゼロとなるため、Monte Carlo Gamma を単純な二階 AD だけで安定して推定することはできない。

これは自動微分の失敗ではなく、Payoff の非滑らかさに由来する。

そこで本章では Gamma について、Payoff を滑らかに近似した Smooth Payoff を用いて二階自動微分を確認する。

Smooth Payoff による Gamma

正の部分、

$$x^+ = \max(x, 0)$$

を、

$$x^+ \approx \frac{1}{2} \left(x + \sqrt{x^2 + \varepsilon^2} \right)$$

で滑らかに近似する。

ε を小さくすれば通常の Call Payoff へ近づく。

```
smooth_positive_part_torch <- function(
  x,
  epsilon
) {
  0.5 * (
    x +
    torch_sqrt(
      x^2 +
      epsilon^2
    )
  )
}
```

Smooth Payoff を使った Monte Carlo Pricing Function を定義する。

```
european_call_mc_smooth_torch <- function(
  spot,
  strike,
  rate,
  dividend_yield,
  volatility,
  maturity,
```



```
z,
  epsilon = 0.25
) {
  terminal_spot <- spot *
    torch_exp(
      (
        rate -
          dividend_yield -
            0.5 * volatility^2
      ) * maturity +
      volatility *
        torch_sqrt(maturity) *
        z
    )

  payoff <- smooth_positive_part_torch(
    terminal_spot -
      strike,
    epsilon = epsilon
  )

  torch_exp(
    -rate * maturity
  ) *
    torch_mean(
      payoff
    )
}
```

Gamma 用に新しい計算グラフを構築する。

```
spot_gamma_t <- torch_tensor(
  spot_value,
  requires_grad = TRUE
)

volatility_gamma_t <- torch_tensor(
  volatility_value
)

maturity_gamma_t <- torch_tensor(
  maturity_value
)
```

```
option_npv_gamma_t <- european_call_mc_smooth_torch(
  spot = spot_gamma_t,
  strike = strike_t,
  rate = torch_tensor(
    risk_free_rate_value
  ),
  dividend_yield = dividend_yield_t,
  volatility = volatility_gamma_t,
  maturity = maturity_gamma_t,
  z = z_t,
  epsilon = 0.25
)
```

まず Delta を計算し、Gradient Graph を保持する。

```
delta_gamma_t <- autograd_grad(
  outputs = option_npv_gamma_t,
  inputs = list(
    spot_gamma_t
  ),
  create_graph = TRUE
)[[1]]

delta_gamma_t
#> torch_tensor
#> 0.6008
#> [ CPUFloatType{1} ][ grad_fn = <SumBackward1> ]
```

さらに Delta を Spot で微分する。

```
gamma_mc <- autograd_grad(
  outputs = delta_gamma_t,
  inputs = list(
    spot_gamma_t
  )
)[[1]]$item()

gamma_mc
#> [1] 0.01912059
```

これは、

$$V \xrightarrow{\text{AD}} \Delta \xrightarrow{\text{AD}} \Gamma$$

という二階 Automatic Differentiation である。

ただし、この Gamma は Smooth Payoff による近似値である点に注意する。

GaussQuant と Monte Carlo + torch の比較

Monte Carlo の結果をまとめる。

```
tbl_mc_greeks <- tribble(
  ~metric, ~value,
  "npv", option_npv_mc_t$item(),
  "delta", delta_mc,
  "gamma", gamma_mc,
  "vega", vega_mc,
  "theta", theta_mc
)

tbl_mc_greeks
#> # A tibble: 5 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv        9.4665
#> 2 delta      0.60086
#> 3 gamma      0.019121
#> 4 vega      38.900
#> 5 theta     -5.4086
```

GaussQuant と比較する。

```
tbl_option_greeks_comparison <- tbl_gaussquant_greeks |>
  rename(
    gaussquant = value
  ) |>
  left_join(
    tbl_mc_greeks |>
      rename(
        monte_carlo_torch = value
      ),
    by = "metric"
  ) |>
  mutate(
    difference = monte_carlo_torch -
      gaussquant,
    relative_difference = if_else(
      gaussquant == 0,
      NA_real_,
      difference /
```

```

    abs(
      gaussquant
    )
  )
)

tbl_option_greeks_comparison
#> # A tibble: 5 x 5
#>   metric gaussquant monte_carlo_torch difference relative_difference
#>   <chr>      <dbl>          <dbl>      <dbl>          <dbl>
#> 1 npv      9.4134          9.4665    0.053099      0.0056408
#> 2 delta    0.59871          0.60086   0.0021528     0.0035957
#> 3 gamma    0.019333          0.019121 -0.00021282    -0.011008
#> 4 vega     38.667           38.900    0.23292       0.0060238
#> 5 theta    -5.3804          -5.4086   -0.028157     -0.0052333

```

比較の構造は、

$$\boxed{\text{GaussQuant} / \text{QuantLib} \quad \longleftrightarrow \quad \text{Monte Carlo} + \text{torch AD}}$$

となる。

NPV、Delta、Vega、Theta は、Monte Carlo Path 上の微分をそのまま利用する Pathwise Derivative である。

Gamma については Call Payoff の Kink があるため、Smooth Payoff による二階 AD を使用している。

Monte Carlo 標準誤差

Monte Carlo 価格には統計誤差が存在する。

価格の差を解釈するため、標準誤差も確認する。

```

with_no_grad({
  terminal_spot_se_t <- torch_tensor(
    spot_value
  ) *
  torch_exp(
    (
      risk_free_rate_value -
      dividend_yield_value -
      0.5 * volatility_value^2
    ) * maturity_value +
    volatility_value *
    sqrt(
      maturity_value
    )
  )
})

```

```

    ) *
    z_t
  )

discounted_payoff_se_t <- exp(
  -risk_free_rate_value *
  maturity_value
) *
  torch_relu(
    terminal_spot_se_t -
    strike_value
  )

mc_standard_error <- torch_std(
  discounted_payoff_se_t
)$item() /
  sqrt(
    n_paths
  )
})

tbl_mc_error <- tribble(
  ~metric, ~value,
  "Monte Carlo NPV", option_npv_mc_t$item(),
  "Monte Carlo standard error", mc_standard_error,
  "GaussQuant NPV",
  tbl_gaussquant_greeks |>
    filter(
      metric == "npv"
    ) |>
    pull(
      value
    )
)

tbl_mc_error
#> # A tibble: 3 x 2
#>   metric                                value
#>   <chr>                                <dbl>
#> 1 Monte Carlo NPV                      9.4665
#> 2 Monte Carlo standard error          0.031533
#> 3 GaussQuant NPV                      9.4134

```

GaussQuant と Monte Carlo 価格の差が Monte Carlo Standard Error と同程度であれば、差の多くは

Simulation Noise として説明できる。

Automatic Differentiation の意味

本章で重要なのは、Black-Scholes の解析式を自動微分したことではない。

Monte Carlo Pricing を、

$$\text{Risk Factors} \rightarrow \text{Simulation} \rightarrow \text{Payoff} \rightarrow \text{Discounting} \rightarrow \text{NPV}$$

という Differentiable Computational Graph として構築したことである。

Forward 計算では、

$$S_0, \sigma, T \rightarrow V$$

を計算する。

Reverse 計算では、

$$V \rightarrow \Delta, \text{Vega}, \text{Theta}$$

へ Gradient を伝播する。

したがって、

$$\text{Monte Carlo Pricing Engine} + \text{Reverse Mode AD} = \text{Differentiable Risk Engine}$$

となる。

PRDC への接続

European Option では、Risk Factor は少数であった。

しかし同じ考え方は、

$$V = V(S_{FX}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX})$$

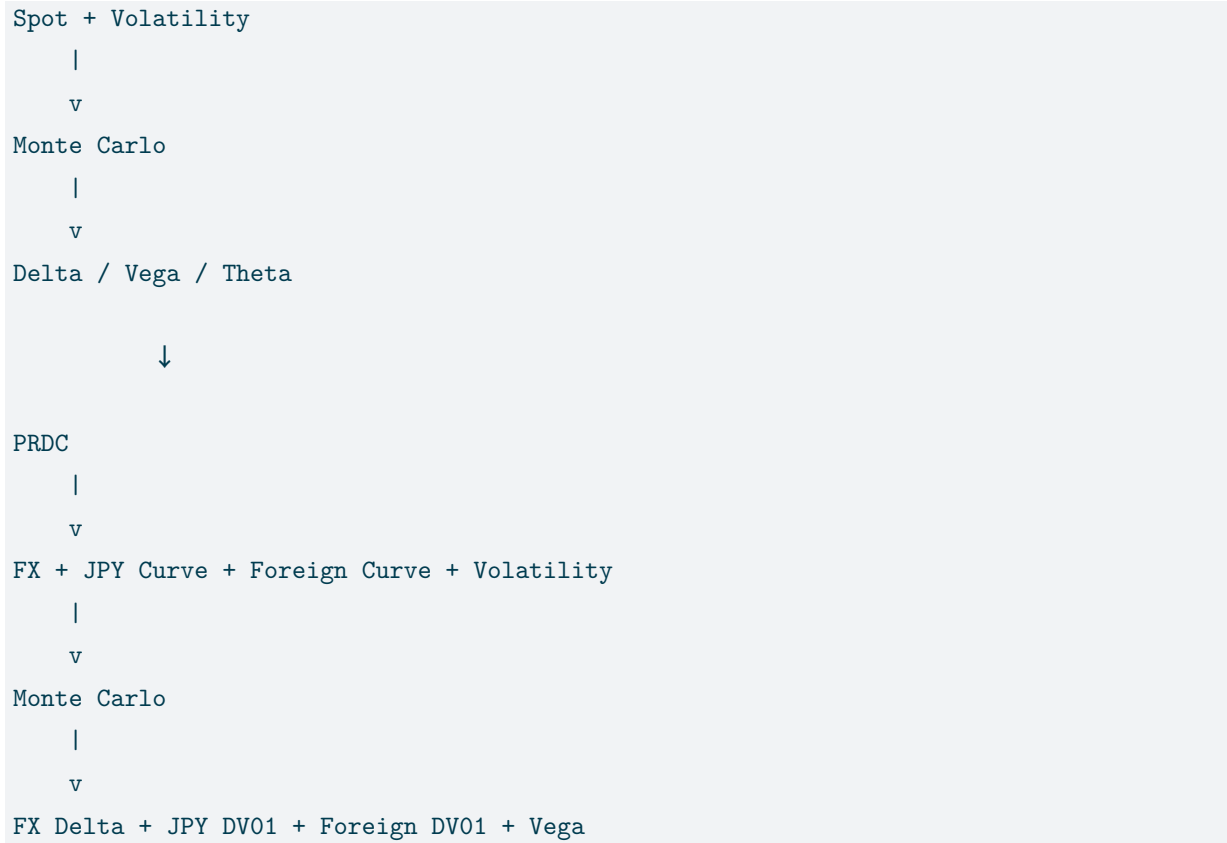
という PRDC にも適用できる。

違いは Computational Graph の規模だけである。

European Option

|

v



本章の Monte Carlo + torch 実装は、第 III-7 章で扱う PRDC Risk への直接の準備となる。

まとめ

本章では、European Option について、

GaussQuant vs Monte Carlo + torch Automatic Differentiation

を比較した。

GaussQuant では、QuantLib を利用して、

$$NPV, \Delta, \Gamma, Vega, Theta$$

を取得した。

一方、torch では Monte Carlo Pricing を Tensor 演算として構築し、Reverse Mode Automatic Differentiation によって、

$$\Delta, Vega, Theta$$

を Pricing Graph から直接取得した。

Gamma については Payoff の Kink による Pathwise 二階微分の問題を確認し、Smooth Payoff を用いた二階 Automatic Differentiation を適用した。

この比較によって、自動微分の目的は解析公式を置き換えることなく、

複雑な Pricing Algorithm そのものを微分する

ことにあと分かる。

次章では、少数の Risk Factor を持つ Option から、多数の Curve Node を持つ金利商品へ問題の次元を拡張し、

$DV01 \rightarrow \textit{Bucketed DV01} \rightarrow \textit{GPS}$

を Yield Curve 全体の Gradient として捉える。

第 III-6 章 金利リスク — DV01 から Curve Gradient へ

前章では、European Option を GaussQuant と Monte Carlo + torch で評価し、Delta、Gamma、Vega、Theta を Automatic Differentiation から取得した。Option では Risk Factor の数が比較的少なかったが、金利商品では Yield Curve 上の多数の Node が同時に価格へ影響する。

本章では、自動微分を金利商品へ拡張する。

Option では、

$$V = V(S, \sigma, T, \dots)$$

という少数のリスクファクターを扱った。

一方、金利商品では、価格は一つの金利だけでなく、Yield Curve 上の多数の点に依存する。

$$V = V(r_1, r_2, \dots, r_n)$$

したがって、金利リスク計測では、

$$\frac{\partial V}{\partial r}$$

という一つの感応度から、

$$\nabla_{\mathbf{r}} V = \left(\frac{\partial V}{\partial r_1}, \frac{\partial V}{\partial r_2}, \dots, \frac{\partial V}{\partial r_n} \right)$$

という Curve 全体の Gradient へ進む必要がある。

本章では、まず単一のフラット金利を用いた Swap の DV01 を計算し、その後、複数ノードを持つ Yield Curve へ拡張する。

最終的に、

$$\boxed{\text{DV01} \longrightarrow \text{Bucketed DV01} \longrightarrow \text{GPS} \longrightarrow \text{Curve Gradient}}$$

という流れを、自動微分によって統一的に捉える。

金利商品の現在価値

将来キャッシュフロー CF_i の現在価値は、

$$V = \sum_{i=1}^n CF_i D(0, t_i)$$

である。

単純化して、連続複利のフラット金利 r を用いると、

$$D(0, t_i) = e^{-rt_i}$$

なので、

$$V(r) = \sum_{i=1}^n CF_i(r) e^{-rt_i}$$

となる。

ここで重要なのは、金利商品では割引係数だけでなく、キャッシュフロー自身も金利に依存する可能性があることである。

したがって、

$$\frac{\partial V}{\partial r} = \sum_i \left[\frac{\partial CF_i}{\partial r} D_i + CF_i \frac{\partial D_i}{\partial r} \right]$$

となる。

これは、金利リスクが単なる Discounting Risk ではなく、Projected Cash Flow の変化も含むことを意味する。

Fixed Leg

元本 N 、固定金利 K 、Accrual Factor α_i とすると、固定レッグのキャッシュフローは、

$$CF_i^{\text{fix}} = NK\alpha_i$$

である。

したがって、

$$PV_{\text{fix}} = \sum_i NK\alpha_i e^{-rt_i}$$

となる。

r で微分すると、

$$\frac{\partial PV_{\text{fix}}}{\partial r} = - \sum_i t_i N K \alpha_i e^{-rt_i}$$

である。

金利が上昇すると割引係数が低下するため、Fixed Leg の現在価値は低下する。

Floating Leg

本章では、自動微分の仕組みを明確にするため、Floating Coupon を単純化して、

$$CF_i^{\text{float}}(r) = N(e^{r\alpha_i} - 1)$$

とする。

このとき、

$$PV_{\text{float}} = \sum_i N(e^{r\alpha_i} - 1)e^{-rt_i}$$

である。

微分すると、

$$\frac{\partial PV_{\text{float}}}{\partial r} = \sum_i N e^{-rt_i} [\alpha_i e^{r\alpha_i} - t_i (e^{r\alpha_i} - 1)]$$

となる。

Floating Leg では、

$$\text{Projected Cash Flow Risk} + \text{Discounting Risk}$$

の両方が金利感応度に含まれる。

Swap NPV

Receive Float / Pay Fixed Swap を考える。

$$V_{\text{swap}} = PV_{\text{float}} - PV_{\text{fixed}}$$

したがって、

$$\frac{\partial V_{\text{swap}}}{\partial r} = \frac{\partial PV_{\text{float}}}{\partial r} - \frac{\partial PV_{\text{fixed}}}{\partial r}$$

となる。

DV01

DV01 は、金利が 1bp 変化したときの価格変化を表す。

1bp は、

$$1\text{bp} = 10^{-4}$$

なので、微分による局所近似では、

$$DV01 \approx \frac{\partial V}{\partial r} \times 10^{-4}$$

となる。

符号については、業務システムやポジション表示の慣行により異なる場合があるため、本章では

$$\frac{\partial V}{\partial r} \times 10^{-4}$$

を signed DV01 として扱う。

torch による Swap 評価

まず計算条件を設定する。

```
tbl_swap_input <- tribble(
  ~parameter, ~value,
  "notional", 100000000,
  "fixed_rate", 0.0200,
  "rate", 0.0250,
  "years", 5
)

tbl_swap_input
#> # A tibble: 4 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 notional 100000000
#> 2 fixed_rate    0.02
```

```
#> 3 rate          0.025
#> 4 years          5
```

半年ごとの支払いを想定する。

```
notional <- 100000000
fixed_rate <- 0.0200
rate_value <- 0.0250

payment_time <- seq(
  0.5,
  5.0,
  by = 0.5
)

accrual <- rep(
  0.5,
  length(payment_time)
)
```

torch Tensor へ変換する。

```
rate <- torch_tensor(
  rate_value,
  requires_grad = TRUE
)

time_t <- torch_tensor(
  payment_time
)

accrual_t <- torch_tensor(
  accrual
)

notional_t <- torch_tensor(
  notional
)

fixed_rate_t <- torch_tensor(
  fixed_rate
)
```

割引係数を計算する。

```
df <- torch_exp(
  -rate * time_t
)
```

Fixed Leg を計算する。

```
fixed_cf <- notional_t *
  fixed_rate_t *
  accrual_t

pv_fixed <- torch_sum(
  fixed_cf * df
)

pv_fixed
#> torch_tensor
#> 9341618
#> [ CPUFloatType{} ][ grad_fn = <SumBackward0> ]
```

Floating Leg を計算する。

```
float_cf <- notional_t *
  (
    torch_exp(
      rate * accrual_t
    ) -
    1
  )

pv_float <- torch_sum(
  float_cf * df
)

pv_float
#> torch_tensor
#> 11750344
#> [ CPUFloatType{} ][ grad_fn = <SumBackward0> ]
```

Receive Float / Pay Fixed の Swap NPV は、

```
swap_npv <- pv_float - pv_fixed

swap_npv
#> torch_tensor
#> 2408726
#> [ CPUFloatType{} ][ grad_fn = <SubBackward0> ]
```

となる。

自動微分による DV01

価格に対して `backward()` を実行する。

```
swap_npv$backward()

rate$grad
#> torch_tensor
#> 4.66456e+08
#> [ CPUFloatType{1} ]
```

`rate$grad` は、

$$\frac{\partial V}{\partial r}$$

を表す。

1bp へ換算する。

```
tbl_swap_result <- tribble(
  ~metric, ~value,
  "npv",    swap_npv$item(),
  "dV_dr",  rate$grad$item(),
  "dv01",   rate$grad$item() * 1e-4
)

tbl_swap_result
#> # A tibble: 3 x 2
#>   metric      value
#>   <chr>      <dbl>
#> 1 npv      2408726
#> 2 dV_dr    466456256
#> 3 dv01      46646.
```

ここで実行していることは、

```
rate
|
+-----> floating cash flow
|
+-----> discount factor
          |
          v
        cash-flow PV
```

```

      |
      v
    swap NPV
      |
      | backward()
      v
    dV/dr

```

という Computational Graph を逆向きにたどることである。

有限差分との比較

同じ Swap 価格を通常の R 関数として定義する。

```

swap_value_flat <- function(
  rate,
  notional,
  fixed_rate,
  payment_time,
  accrual
) {
  df <- exp(
    -rate * payment_time
  )

  fixed_cf <- notional *
    fixed_rate *
    accrual

  float_cf <- notional *
    (
      exp(
        rate * accrual
      ) -
      1
    )

  sum(
    float_cf * df
  ) -
  sum(
    fixed_cf * df
  )
}

```


1bp Bump による Forward Difference を計算する。

```
bump <- 1e-4

base_npv <- swap_value_flat(
  rate = rate_value,
  notional = notional,
  fixed_rate = fixed_rate,
  payment_time = payment_time,
  accrual = accrual
)

up_npv <- swap_value_flat(
  rate = rate_value + bump,
  notional = notional,
  fixed_rate = fixed_rate,
  payment_time = payment_time,
  accrual = accrual
)

fd_dv01 <- up_npv - base_npv

fd_dv01
#> [1] 46634.17
```

Central Difference による一次微分も確認する。

```
down_npv <- swap_value_flat(
  rate = rate_value - bump,
  notional = notional,
  fixed_rate = fixed_rate,
  payment_time = payment_time,
  accrual = accrual
)

fd_dv_dr <- (
  up_npv -
  down_npv
) / (
  2 * bump
)

fd_dv01_central <- fd_dv_dr *
  1e-4
```

```
tbl_dv01_comparison <- tbl_dv01_comparison <- tribble(
  ~method, ~dv01,
  "Forward bump", fd_dv01,
  "Central difference", fd_dv01_central,
  "Automatic differentiation", rate$grad$item() * 1e-4
)

tbl_dv01_comparison
#> # A tibble: 3 x 2
#>   method          dv01
#>   <chr>          <dbl>
#> 1 Forward bump    46634.
#> 2 Central difference 46646.
#> 3 Automatic differentiation 46646.

tbl_dv01_comparison
#> # A tibble: 3 x 2
#>   method          dv01
#>   <chr>          <dbl>
#> 1 Forward bump    46634.
#> 2 Central difference 46646.
#> 3 Automatic differentiation 46646.
```

有限差分では価格を複数回評価する必要があるのに対し、自動微分では構築された計算グラフを利用して感応度を得る。

フラット金利から Yield Curve へ

実際の金利市場では、単一の金利 r ではなく、満期ごとに異なる金利を持つ。

例えば、

$$\mathbf{r} = (r_1, r_2, \dots, r_n)$$

とする。

価格は、

$$V = V(r_1, r_2, \dots, r_n)$$

となる。

したがって必要なのは Scalar Derivative ではなく、

$$\nabla_{\mathbf{r}} V = \left(\frac{\partial V}{\partial r_1}, \frac{\partial V}{\partial r_2}, \dots, \frac{\partial V}{\partial r_n} \right)$$

という Gradient である。

各成分を 1bp へ換算すれば、

$$DV01_i = \frac{\partial V}{\partial r_i} \times 10^{-4}$$

となる。

これが Bucketed DV01 や Point Sensitivity の基本になる。

Curve Node を Tensor として持つ

単純化した Zero Curve を考える。

```
tbl_curve <- tribble(
  ~tenor, ~rate,
  0.5, 0.0180,
  1.0, 0.0190,
  2.0, 0.0210,
  3.0, 0.0225,
  5.0, 0.0250
)

tbl_curve
#> # A tibble: 5 x 2
#>   tenor  rate
#>   <dbl> <dbl>
#> 1   0.5 0.018
#> 2   1.0 0.019
#> 3   2.0 0.021
#> 4   3.0 0.0225
#> 5   5.0 0.025
```

Curve Rate 全体を一つの Tensor として定義する。

```
curve_rate <- torch_tensor(
  tbl_curve$rate,
  requires_grad = TRUE
)

curve_time <- torch_tensor(
```

```
tbl_curve$tenor
)
```

簡単な Zero Coupon Cash Flow を考える。

```
cash_flow <- torch_tensor(
  c(
    500000,
    500000,
    500000,
    500000,
    102500000
  )
)
```

各 Node Rate をその満期の Zero Rate として使用する。

```
curve_df <- torch_exp(
  -curve_rate * curve_time
)

curve_pv <- torch_sum(
  cash_flow * curve_df
)

curve_pv
#> torch_tensor
#> 92388832
#> [ CPUFloatType{} ][ grad_fn = <SumBackward0> ]
```

自動微分を実行する。

```
curve_pv$backward()

curve_rate$grad
#> torch_tensor
#> 100000000 *
#> -0.0025
#> -0.0049
#> -0.0096
#> -0.0140
#> -4.5228
#> [ CPUFloatType{5} ]
```

これにより、

$$\left(\frac{\partial V}{\partial r_{0.5}}, \frac{\partial V}{\partial r_1}, \frac{\partial V}{\partial r_2}, \frac{\partial V}{\partial r_3}, \frac{\partial V}{\partial r_5} \right)$$

が一度に得られる。

1bp 換算する。

```
tbl_bucketed_dv01 <- tibble(
  tenor = tbl_curve$tenor,
  rate = tbl_curve$rate,
  sensitivity = as.numeric(
    curve_rate$grad
  ),
  dv01 = sensitivity * 1e-4
)
```

```
tbl_bucketed_dv01
#> # A tibble: 5 x 4
#>   tenor   rate sensitivity    dv01
#>   <dbl> <dbl>      <dbl>   <dbl>
#> 1  0.5 0.018   -247760.   -24.776
#> 2  1    0.019   -490590.   -49.059
#> 3  2    0.021   -958870.   -95.887
#> 4  3    0.0225  -1402092.  -140.21
#> 5  5    0.025  -452279648 -45228.
```

GPS としての Point Sensitivity

本章では、Yield Curve 上の各 Grid Point に対する価格感応度を GPS (Grid Point Sensitivity) として扱う。

各 Grid Point r_i に対して、

$$GPS_i = \frac{\partial V}{\partial r_i}$$

である。

1bp 単位で表示する場合は、

$$GPS01_i = GPS_i \times 10^{-4}$$

となる。

したがって、

$$\mathbf{GPS} = \left(\frac{\partial V}{\partial r_1}, \dots, \frac{\partial V}{\partial r_n} \right)$$

は、数学的にはそのまま Curve Gradient である。

```
tbl_gps <- tbl_bucketed_dv01 |>
  transmute(
    tenor,
    grid_rate = rate,
    gps = sensitivity,
    gps01 = dv01
  )
```

```
tbl_gps
```

```
#> # A tibble: 5 x 4
```

```
#>   tenor grid_rate      gps      gps01
#>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1   0.5   0.018 -247760. -24.776
#> 2    1    0.019 -490590. -49.059
#> 3    2    0.021 -958870. -95.887
#> 4    3    0.0225 -1402092. -140.21
#> 5    5    0.025 -452279648 -45228.
```

この考え方によって、

$$\text{DV01} = \text{Scalar Risk}$$

から、

$$\mathbf{GPS} = \text{Vector Risk}$$

へ拡張できる。

Total DV01 と Bucketed Risk

単純化された設定では、Bucketed DV01 を合計することで、Curve 全体を同時に 1bp 動かしたときの感応度に対応する量を得ることができる。

```
tbl_total_bucketed_dv01 <- tbl_bucketed_dv01 |>
  summarise(
    total_dv01 = sum(
      dv01
    )
  )
```

```
tbl_total_bucketed_dv01
#> # A tibble: 1 x 1
#>   total_dv01
#>       <dbl>
#> 1    -45538.
```

ただし、実務では Curve Construction、Interpolation、複数 Curve、Quote Mapping などが存在するため、

$$\text{Parallel DV01} = \sum_i \text{Bucketed DV01}_i$$

が常に厳密に成立するとは限らない。

重要なのは、各リスクファクターに対する局所感応度を Gradient として保持することである。

Market Quote から Curve Risk へ

実務の金利リスクでは、直接 Zero Rate を Bump するだけでは不十分である。

実際の計算は、

$$\text{Market Quotes} \rightarrow \text{Bootstrap} \rightarrow \text{Yield Curve} \rightarrow \text{Discount Factors} \rightarrow \text{Cash Flows} \rightarrow V$$

という計算グラフを持つ。

Market Quote を、

$$\mathbf{q} = (q_1, q_2, \dots, q_m)$$

Yield Curve Node を、

$$\mathbf{r} = (r_1, r_2, \dots, r_n)$$

とすると、

$$V = V(\mathbf{r}(\mathbf{q}))$$

である。

連鎖律より、

$$\frac{\partial V}{\partial q_j} = \sum_i \frac{\partial V}{\partial r_i} \frac{\partial r_i}{\partial q_j}$$

となる。

行列表現では、

$$\nabla_{\mathbf{q}} V = J_{\mathbf{r}, \mathbf{q}}^T \nabla_{\mathbf{r}} V$$

である。

ここで、

$$J_{\mathbf{r}, \mathbf{q}}$$

は Market Quote から Curve Node への Jacobian である。

この式は、金利リスク計測と Automatic Differentiation をつなぐ重要な関係である。

Multi-Curve への拡張

実際の Swap では、Discount Curve と Forward Curve を分離する Multi-Curve Framework が一般的である。

この場合、価格は、

$$V = V(\mathbf{r}^{disc}, \mathbf{r}^{fwd})$$

と表される。

したがって、

$$\nabla V = (\nabla_{\mathbf{r}^{disc}} V, \nabla_{\mathbf{r}^{fwd}} V)$$

となる。

すなわち、

$$\text{Discount Risk} + \text{Forward Risk}$$

を別々の Gradient として取得できる。

これは次章で扱う PRDC に直接つながる。

PRDC では、

$$\text{JPY Curve} + \text{Foreign Curve} + \text{FX}$$

という複数のリスクファクターを同時に扱うためである。

自動微分による金利リスク計測の意味

有限差分による Bucketed Risk では、Curve Node の数だけ Bump & Revalue を繰り返す必要がある。

例えば 100 個の Curve Node に対して Central Difference を使えば、

$$2 \times 100 = 200$$

回の再評価が必要になる。

一方、自動微分では、

$$V = V(r_1, r_2, \dots, r_{100})$$

という計算グラフを構築し、

$$V \longrightarrow \nabla_{\mathbf{r}} V$$

と逆伝播する。

したがって、金利リスク計測は、

```
Yield Curve
r1 r2 r3 ... rn
 \ | |   /
  \| |   /
Pricing Graph
  |
  v
NPV
  |
  | backward()
  v
dV/dr1 ... dV/drn
```

として捉えることができる。

これは、

Scalar Pricing $\longrightarrow$ Vector Risk
--

への変換である。

まとめ

本章では、Option Greeks で確認した自動微分を 金利商品へ拡張した。

まず単一のフラット金利について、

$$V = V(r)$$

から、

$$DV01 = \frac{\partial V}{\partial r} \times 10^{-4}$$

を求めた。

次に Yield Curve を、

$$\mathbf{r} = (r_1, \dots, r_n)$$

という Tensor として扱い、

$$\nabla_{\mathbf{r}} V = \left(\frac{\partial V}{\partial r_1}, \dots, \frac{\partial V}{\partial r_n} \right)$$

を一度の Reverse Mode Automatic Differentiation で取得した。

これにより、

$$DV01 \rightarrow \text{Bucketed DV01} \rightarrow \text{GPS} \rightarrow \text{Curve Gradient}$$

という金利リスク計測を統一的に表現できる。

さらに、

$$\text{Market Quote} \rightarrow \text{Curve} \rightarrow \text{PV}$$

という計算グラフに対して連鎖律を適用すれば、Market Quote 単位の Risk へ拡張できる。

次章では、この考え方を PRDC へ適用する。

PRDC 価格を、

$$V = V(S_{FX}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX}, \dots)$$

と表し、

$$\Delta_{FX}, \quad \mathbf{DV01}_{JPY}, \quad \mathbf{DV01}_{Foreign}, \quad Vega_{FX}$$

を同じ Computational Graph から計算する。

これにより、第 III 部の最終段階として、為替と複数金利カーブを持つ多因子商品の Differentiable Risk Engine を構築する。

第 III-7 章 PRDC — 多因子 Monte Carlo と Automatic Differentiation

第 III 部では、二つの方向からクオンツ計算の高度化を進めてきた。前半では Parallel / Distributed Computing によって大量の Pricing を処理する計算基盤を構築し、後半では Pricing そのものを Differentiable Computational Graph として設計することで、Automatic Differentiation から Risk Sensitivity を直接取得した。

最終章では、この二つの考え方が同時に必要となる代表例として PRDC を扱う。PRDC は FX、複数通貨の Yield Curve、Volatility など多数の Risk Factor を持ち、価格評価には Monte Carlo Simulation が必要となる。したがって、**大規模計算をどう実行するかと、多数の感応度をどう効率的に生成するか**という第 III 部の二つの課題が一つの商品に集約される。

第 III-5 章では Option Greeks、第 III-6 章では DV01 から Curve Gradient へと、価格関数の入力次元を段階的に増やしてきた。本章ではさらに、

$$V = V(S_{FX}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX})$$

という多因子 Monte Carlo Pricing を一つの計算 Graph として構成し、

$$\Delta_{FX}, \quad \mathbf{DV01}_{JPY}, \quad \mathbf{DV01}_{Foreign}, \quad Vega_{FX}$$

を一度の Reverse-mode Automatic Differentiation から取得する。

本章の目的は、単に PRDC の Greeks を計算することではない。**Monte Carlo Pricing と Risk Calculation を別々の処理として繰り返すのではなく、Pricing そのものから Risk を生成する**という Differentiable Risk Engine の設計を示すことである。

PRDC と複数のリスクファクター

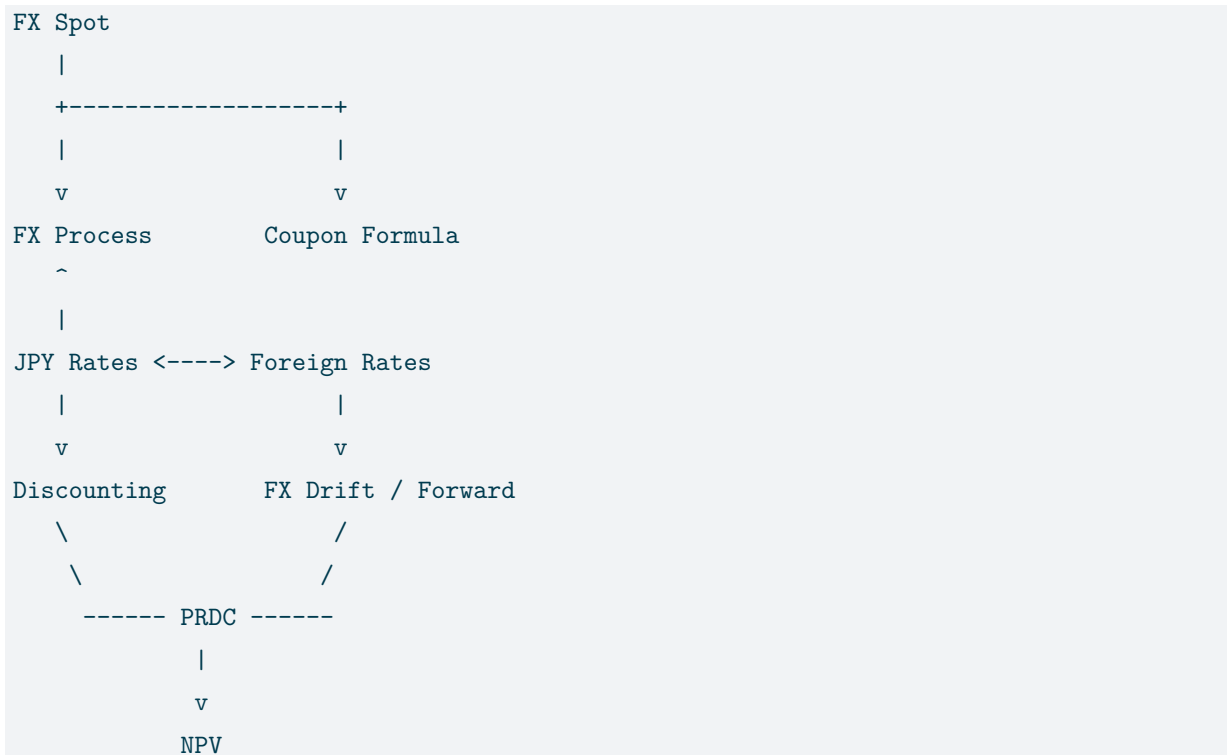
PRDC は、為替と複数通貨の金利を同時に扱うハイブリッド商品である。

単純な Option では、

$$V = V(S, \sigma, r, T)$$

と少数の入力変数を考えればよかった。

PRDC では、



という複数の経路から価格が構成される。

したがって、一つの市場変数を変更した場合でも、その影響は複数の計算経路を通して最終価格へ伝わる。

例えば JPY 金利は、

- JPY Discount Factor
- FX の Risk-Neutral Drift
- 将来 Coupon の期待値

へ影響する。

Foreign Rate も、

- FX Forward
- FX の Risk-Neutral Drift
- 将来 Coupon の期待値

へ影響する。

このような商品では、連鎖律をプログラム全体へ適用する Automatic Differentiation の考え方が特に重要になる。

本章で用いる簡略 PRDC

PRDC には多様な Coupon 構造、Callable 条件、Notional Exchange、Target Redemption、Barrier などが存在する。

本章では自動微分の構造を明確にするため、第 II 部・第 III 部で扱った PRDC の考え方を簡略化し、為替連動 Coupon を、

$$c_i = \min \left[c_{\max}, \max \left(0, c_0 + L \left(\frac{S_i}{S_{\text{ref}}} - 1 \right) \right) \right]$$

とする。

ここで、

- c_0 : Base Coupon
- L : FX Leverage
- S_i : Coupon Date の FX Spot
- S_{ref} : Reference FX
- $c_{\max}$: Coupon Cap

である。

Coupon Cash Flow は、

$$CF_i = N\alpha_i c_i$$

とする。

この Payoff は Automatic Differentiation の構造を明確にするための教育用モデルである。実際の PRDC では Coupon Formula、Callable 条件、Notional Exchangeなどを追加する必要があるが、重要なのは、それらと同じ Differentiable Computational Graph の中へ組み込めることである。

重要なのは Payoff の具体的な形ではなく、

$$\boxed{\text{FX} + \text{JPY Curve} + \text{Foreign Curve} + \text{Volatility} \longrightarrow \text{PRDC NPV}}$$

という計算グラフを微分可能な演算として構成することである。

FX の Risk-Neutral Dynamics

JPY を Domestic Currency、Foreign Currency を外貨とする。

簡略化した Lognormal FX Process として、

$$\frac{dS_t}{S_t} = (r_{JPY}(t) - r_{Foreign}(t)) dt + \sigma_{FX} dW_t$$

を用いる。

離散化すると、

$$S_{t+\Delta t} = S_t \exp \left[\left(r_{JPY} - r_{Foreign} - \frac{1}{2} \sigma_{FX}^2 \right) \Delta t + \sigma_{FX} \sqrt{\Delta t} Z \right]$$

となる。

ここで、

$$Z \sim N(0, 1)$$

である。

この式から、FX Path 自身が、

$$S_{t_i} = S_{t_i}(S_0, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX}, Z)$$

という関数になっていることが分かる。

したがって PRDC 価格を微分すると、Coupon だけではなく FX Path 生成まで連鎖律が伝播する。

Monte Carlo Pricing

PRDC の Monte Carlo 価格を、

$$\hat{V} = \frac{1}{M} \sum_{m=1}^M \left[\sum_{i=1}^n D_{JPY}(0, t_i) CF_i^{(m)} + D_{JPY}(0, T) N \right]$$

とする。

ここで、

$$M$$

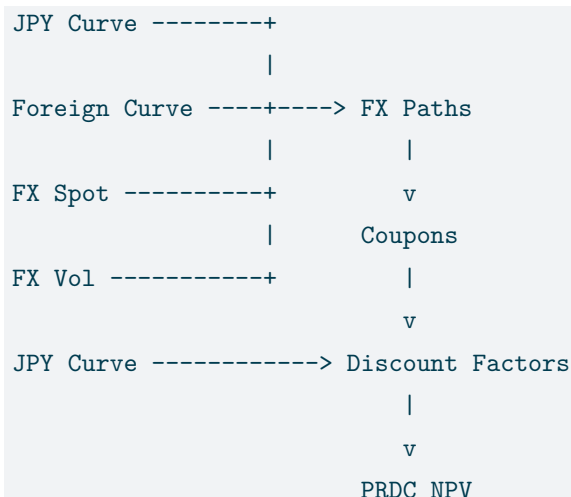
は Path 数である。

JPY Discount Factor は、期間ごとの短期金利を用いて、

$$D_{JPY}(0, t_i) = \exp \left[- \sum_{j=1}^i r_{JPY,j} \Delta t_j \right]$$

とする。

したがって計算全体は、



となる。

計算条件

本章では 5 年間、年 1 回 Coupon を支払う簡略 PRDC を考える。

```
tbl_prdc_input <- tribble(
  ~parameter, ~value,
  "notional", 100000000,
  "fx_spot", 150.00,
  "reference_fx", 150.00,
  "fixed_coupon", 0.0200,
  "leverage", 0.80,
  "coupon_cap", 0.0800,
  "fx_vol", 0.1200,
  "n_paths", 20000
)

tbl_prdc_input
#> # A tibble: 8 x 2
#>   parameter      value
#>   <chr>         <dbl>
#> 1 notional    100000000
#> 2 fx_spot      150
#> 3 reference_fx 150
#> 4 fixed_coupon 0.02
#> 5 leverage    0.8
#> 6 coupon_cap  0.08
#> 7 fx_vol      0.12
#> 8 n_paths     20000
```


JPY Curve と Foreign Curve を設定する。

```
tbl_prdc_curve <- tribble(
  ~year, ~jpy_rate, ~foreign_rate,
  1, 0.0100, 0.0400,
  2, 0.0120, 0.0380,
  3, 0.0140, 0.0360,
  4, 0.0160, 0.0340,
  5, 0.0180, 0.0320
)

tbl_prdc_curve
#> # A tibble: 5 x 3
#>   year jpy_rate foreign_rate
#>   <dbl>   <dbl>         <dbl>
#> 1     1    0.01          0.04
#> 2     2    0.012         0.038
#> 3     3    0.014         0.036
#> 4     4    0.016         0.034
#> 5     5    0.018         0.032
```

torch による Monte Carlo Path 生成

Monte Carlo で使用する乱数を固定する。

```
torch_manual_seed(1234)

n_paths <- 20000
n_steps <- nrow(tbl_prdc_curve)

z <- torch_randn(
  c(
    n_paths,
    n_steps
  )
)
```

乱数 Z はリスクファクターではないため、`requires_grad = TRUE` は指定しない。

一方、微分対象となる市場変数には、`requires_grad = TRUE` を指定する。

```
fx_spot_t <- torch_tensor(
  150.00,
  requires_grad = TRUE
)
```

```
jpy_curve_t <- torch_tensor(  
  tbl_prdc_curve$jpy_rate,  
  requires_grad = TRUE  
)  
  
foreign_curve_t <- torch_tensor(  
  tbl_prdc_curve$foreign_rate,  
  requires_grad = TRUE  
)  
  
fx_vol_t <- torch_tensor(  
  0.1200,  
  requires_grad = TRUE  
)
```

その他の契約条件を Tensor として準備する。

```
notional_t <- torch_tensor(  
  100000000  
)  
  
reference_fx_t <- torch_tensor(  
  150.00  
)  
  
fixed_coupon_t <- torch_tensor(  
  0.0200  
)  
  
leverage_t <- torch_tensor(  
  0.80  
)  
  
coupon_cap_t <- torch_tensor(  
  0.0800  
)  
  
dt_t <- torch_ones(  
  n_steps  
)
```

微分可能な PRDC Pricing Function

PRDC の Monte Carlo Pricing を、Tensor 演算だけで構成する。

```
prdc_price_torch <- function(
  fx_spot,
  jpy_curve,
  foreign_curve,
  fx_vol,
  z,
  notional,
  reference_fx,
  fixed_coupon,
  leverage,
  coupon_cap,
  dt
) {
  drift_step <- (
    jpy_curve -
    foreign_curve -
    0.5 * fx_vol^2
  ) * dt

  diffusion_step <- fx_vol *
    torch_sqrt(dt) *
    z

  log_fx_path <- torch_log(
    fx_spot
  ) +
    torch_cumsum(
      drift_step +
      diffusion_step,
      dim = 2
    )

  fx_path <- torch_exp(
    log_fx_path
  )

  raw_coupon_rate <- fixed_coupon +
    leverage * (
      fx_path / reference_fx -
      1
    )

  coupon_rate <- torch_minimum(
```

```
    torch_maximum(  
      raw_coupon_rate,  
      torch_zeros_like(raw_coupon_rate)  
    ),  
    coupon_cap  
  )  
  
  coupon_cf <- notional *  
    coupon_rate *  
    dt  
  
  discount_factor <- torch_exp(  
    -torch_cumsum(  
      jpy_curve * dt,  
      dim = 1  
    )  
  )  
  
  pv_coupon <- torch_sum(  
    coupon_cf *  
    discount_factor,  
    dim = 2  
  )  
  
  pv_redemption <- notional *  
    discount_factor[n_steps]  
  
  torch_mean(  
    pv_coupon +  
    pv_redemption  
  )  
}
```

この関数の中では、

- FX Path 生成
- Coupon 計算
- JPY Discounting
- Monte Carlo Average

のすべてが Tensor 演算として接続されている。

したがって、最終的な `prdc_npv` から、入力された市場リスクファクターまで計算グラフが維持される。

PRDC NPV

価格を計算する。

```
prdc_npv <- prdc_price_torch(
  fx_spot = fx_spot_t,
  jpy_curve = jpy_curve_t,
  foreign_curve = foreign_curve_t,
  fx_vol = fx_vol_t,
  z = z,
  notional = notional_t,
  reference_fx = reference_fx_t,
  fixed_coupon = fixed_coupon_t,
  leverage = leverage_t,
  coupon_cap = coupon_cap_t,
  dt = dt_t
)

prdc_npv
#> torch_tensor
#> 104242624
#> [ CPUFloatType{} ][ grad_fn = <MeanBackward0> ]
```

この一つの Scalar が、

$$V = V(S_{FX}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX})$$

という多変数関数になっている。

一度の Reverse Mode で複数リスクを取得する

Forward 計算で得られた Scalar NPV を出発点として、複数の市場入力へ Gradient を逆伝播させる。PRDC のように「多数入力・少数出力」となる Pricing では、Reverse Mode の利点が最も現れやすい。

```
prdc_grad <- autograd_grad(
  outputs = prdc_npv,
  inputs = list(
    fx_spot_t,
    jpy_curve_t,
    foreign_curve_t,
    fx_vol_t
  )
)
```

```
fx_delta_t <- prdc_grad[[1]]
jpy_rate_grad_t <- prdc_grad[[2]]
foreign_rate_grad_t <- prdc_grad[[3]]
fx_vega_t <- prdc_grad[[4]]
```

ここで、

$$fx_delta = \frac{\partial V}{\partial S_{FX}}$$

$$jpy_rate_grad_i = \frac{\partial V}{\partial r_{JPY,i}}$$

$$foreign_rate_grad_i = \frac{\partial V}{\partial r_{Foreign,i}}$$

$$fx_vega = \frac{\partial V}{\partial \sigma_{FX}}$$

である。

FX Delta

FX Delta は、

$$\Delta_{FX} = \frac{\partial V}{\partial S_{FX}}$$

である。

```
fx_delta <- fx_delta_t$item()
```

```
fx_delta
#> [1] 473469.3
```

PRDC の Coupon 構造によって、FX Delta の符号や大きさは大きく変化する。

本章の簡略 Payoff では FX 上昇によって Coupon が増加する形を用いているが、実際の PRDC では Currency Pair の定義や Coupon Formula によって感応度の符号は異なる。

JPY Curve Risk

JPY Curve の各 Node に対する感応度は、

$$\frac{\partial V}{\partial r_{JPY,i}}$$

である。

1bp 単位へ換算すると、

$$DV01_{JPY,i} = \frac{\partial V}{\partial r_{JPY,i}} \times 10^{-4}$$

となる。

```
tbl_jpy_risk <- tibble(
  year = tbl_prdc_curve$year,
  rate = tbl_prdc_curve$jpy_rate,
  sensitivity = as.numeric(
    jpy_rate_grad_t
  )
) |>
mutate(
  dv01 = sensitivity * 1e-4
)
```

```
tbl_jpy_risk
#> # A tibble: 5 x 4
#>   year  rate sensitivity    dv01
#>   <dbl> <dbl>         <dbl> <dbl>
#> 1     1  0.01    -33222176 -3322.2
#> 2     2  0.012   -53479396 -5347.9
#> 3     3  0.014   -67464704 -6746.5
#> 4     4  0.016   -77671080 -7767.1
#> 5     5  0.018   -86089920 -8609.0
```

JPY 金利は Discounting だけでなく FX Drift にも入っている。

したがって、

$$\frac{\partial V}{\partial r_{JPY,i}}$$

には、

$$\text{JPY Discount Risk} + \text{FX Drift Risk} + \text{Coupon Risk}$$

が連鎖律を通じて含まれる。

Foreign Curve Risk

Foreign Curve についても同様に、

$$DV01_{Foreign,i} = \frac{\partial V}{\partial r_{Foreign,i}} \times 10^{-4}$$

である。

```
tbl_foreign_risk <- tibble(
  year = tbl_prdc_curve$year,
  rate = tbl_prdc_curve$foreign_rate,
  sensitivity = as.numeric(
    foreign_rate_grad_t
  )
) |>
mutate(
  dv01 = sensitivity * 1e-4
)

tbl_foreign_risk
#> # A tibble: 5 x 4
#>   year  rate sensitivity    dv01
#>   <dbl> <dbl>      <dbl>   <dbl>
#> 1     1  0.04   -71020400 -7102.0
#> 2     2  0.038  -48264380 -4826.4
#> 3     3  0.036  -31967060 -3196.7
#> 4     4  0.034  -19592504 -1959.3
#> 5     5  0.032   -9121757  -912.18
```

Foreign Rate は本章の簡略モデルでは直接 Discounting には使用していない。

しかし、

$$r_{JPY} - r_{Foreign}$$

を通じて FX Drift へ入り、FX Path から Coupon へ感応度が伝播する。

つまり、

$$r_{Foreign} \rightarrow FX\ Path \rightarrow Coupon \rightarrow PRDC\ NPV$$

という経路が存在する。

FX Vega

FX Volatility に対する感応度は、

$$Vega_{FX} = \frac{\partial V}{\partial \sigma_{FX}}$$

である。

```
fx_vega <- fx_vega_t$item()

fx_vega
#> [1] 40120812
```

Volatility を 1 percentage point、すなわち 0.01 だけ変化させた場合の近似価格変化として、

$$Vega01_{FX} = \frac{\partial V}{\partial \sigma_{FX}} \times 0.01$$

を定義することもできる。

```
fx_vega_01 <- fx_vega * 0.01

fx_vega_01
#> [1] 401208.1
```

PRDC の Coupon には Floor や Cap が存在するため、Volatility の変化は単純な線形効果ではない。

Monte Carlo Path の分布が変化し、Floor や Cap へ到達する Path の割合も変化する。

自動微分では、この Path 生成から Payoff までの連鎖を同じ計算グラフの中で扱う。

PRDC Risk Summary

主要な Scalar Risk をまとめる。

```
tbl_prdc_risk_summary <- tribble(
  ~metric, ~value,
  "NPV", prdc_npv$item(),
  "FX Delta", fx_delta,
  "FX Vega", fx_vega,
  "FX Vega 1pct", fx_vega_01,
  "JPY Total DV01", sum(
    tbl_jpy_risk$dv01
  ),
  "Foreign Total DV01", sum(
```

```
tbl_foreign_risk$dv01
)
)

tbl_prdc_risk_summary
#> # A tibble: 6 x 2
#>   metric          value
#>   <chr>          <dbl>
#> 1 NPV            104242624
#> 2 FX Delta        473469.
#> 3 FX Vega         40120812
#> 4 FX Vega 1pct     401208.
#> 5 JPY Total DV01   -31793.
#> 6 Foreign Total DV01 -17997.
```

さらに Curve Risk は Vector として保持する。

```
tbl_curve_risk <- bind_rows(
  tbl_jpy_risk |>
    mutate(
      curve = "JPY"
    ),
  tbl_foreign_risk |>
    mutate(
      curve = "Foreign"
    )
) |>
  select(
    curve,
    year,
    rate,
    sensitivity,
    dv01
  )

tbl_curve_risk
#> # A tibble: 10 x 5
#>   curve  year rate sensitivity    dv01
#>   <chr> <dbl> <dbl>         <dbl>   <dbl>
#> 1 JPY     1 0.01  -33222176 -3322.2
#> 2 JPY     2 0.012 -53479396 -5347.9
#> 3 JPY     3 0.014 -67464704 -6746.5
#> 4 JPY     4 0.016 -77671080 -7767.1
#> 5 JPY     5 0.018 -86089920 -8609.0
```

```
#> 6 Foreign      1 0.04      -71020400 -7102.0
#> 7 Foreign      2 0.038     -48264380 -4826.4
#> 8 Foreign      3 0.036     -31967060 -3196.7
#> 9 Foreign      4 0.034     -19592504 -1959.3
#> 10 Foreign     5 0.032     -9121757  -912.18
```

これにより、一つの PRDC 価格から、

$$\nabla V = (\Delta_{FX}, \mathbf{DV01}_{JPY}, \mathbf{DV01}_{Foreign}, Vega_{FX})$$

を取得したことになる。

有限差分との検算

自動微分の結果を確認するため、FX Delta を Finite Difference で検算する。

Monte Carlo で有限差分を比較する場合には、乱数の違いによる Noise を避けるため、同じ乱数を使う Common Random Numbers が重要である。

本章では同じ z を使用して、FX Spot だけを Bump する。

```
fx_bump <- 0.01

fx_up <- torch_tensor(
  150.00 + fx_bump
)

fx_down <- torch_tensor(
  150.00 - fx_bump
)

jpy_curve_fd <- torch_tensor(
  tbl_prdc_curve$jpy_rate
)

foreign_curve_fd <- torch_tensor(
  tbl_prdc_curve$foreign_rate
)

fx_vol_fd <- torch_tensor(
  0.1200
)

price_up <- prdc_price_torch(
  fx_spot = fx_up,
```

```

    jpy_curve = jpy_curve_fd,
    foreign_curve = foreign_curve_fd,
    fx_vol = fx_vol_fd,
    z = z,
    notional = notional_t,
    reference_fx = reference_fx_t,
    fixed_coupon = fixed_coupon_t,
    leverage = leverage_t,
    coupon_cap = coupon_cap_t,
    dt = dt_t
)

price_down <- prdc_price_torch(
  fx_spot = fx_down,
  jpy_curve = jpy_curve_fd,
  foreign_curve = foreign_curve_fd,
  fx_vol = fx_vol_fd,
  z = z,
  notional = notional_t,
  reference_fx = reference_fx_t,
  fixed_coupon = fixed_coupon_t,
  leverage = leverage_t,
  coupon_cap = coupon_cap_t,
  dt = dt_t
)

fd_fx_delta <- (
  price_up$item() -
    price_down$item()
) / (
  2 * fx_bump
)

tbl_fx_delta_check <- tribble(
  ~method, ~fx_delta,
  "Finite difference", fd_fx_delta,
  "Automatic differentiation", fx_delta
)

tbl_fx_delta_check
#> # A tibble: 2 x 2
#>   method                fx_delta
#>   <chr>                <dbl>

```

```
#> 1 Finite difference          474000
#> 2 Automatic differentiation 473469.
```

同じ乱数を用いることで、差分結果に Monte Carlo Noise が混入することを抑えている。

Floor・Cap と微分可能性

本章の Coupon では、

$$\max(0, x)$$

および、

$$\min(c_{\max}, x)$$

を使用している。

これらの関数は Kink を持ち、境界点では通常の意味で微分不可能である。

`torch_minimum()` や `torch_maximum()` などの Tensor 演算は、こうした演算について実装上定義された局所 Gradient を使用して Reverse Mode を実行する。

Monte Carlo では、連続分布のもとで Path が Kink へ厳密に一致する確率は通常ゼロであるため、Pathwise Derivative を利用できる場合が多い。

ただし、Digital Payoff、Barrier Hit Indicator、Exercise Decision など不連続性の強い商品では、単純な Pathwise Automatic Differentiation だけでは 安定した感応度が得られない場合がある。

その場合には、

- Smoothing
- Likelihood Ratio Method
- Conditional Expectation
- Pathwise / LRM Hybrid
- Differentiable Approximation

などを検討する必要がある。

これは、Automatic Differentiation が「あらゆる金融商品をそのまま微分すればよい」という意味ではないことを示している。

Monte Carlo と Automatic Differentiation

PRDC のような商品では、

$$\hat{V} = \frac{1}{M} \sum_{m=1}^M V^{(m)}$$

という Monte Carlo Average を微分する。

条件を整えば、

$$\frac{\partial}{\partial x} E[V(x, Z)] = E \left[\frac{\partial V(x, Z)}{\partial x} \right]$$

と考えることができる。

つまり各 Path の Pricing Calculation を微分可能に構成すれば、その Average である Monte Carlo Price についても Automatic Differentiation を適用できる。

この考え方が、

$$\boxed{\text{Monte Carlo} + \text{Automatic Differentiation}}$$

を結び付ける。

GPU への拡張

第 III-1 章で確認したように、Monte Carlo は GPU と相性が良い。

本章の実装でも、

z

は、

$$M \times n$$

の Tensor であり、FX Path 全体を Loop ではなく Tensor 演算として計算している。

したがって計算デバイスを GPU へ移せる環境では、

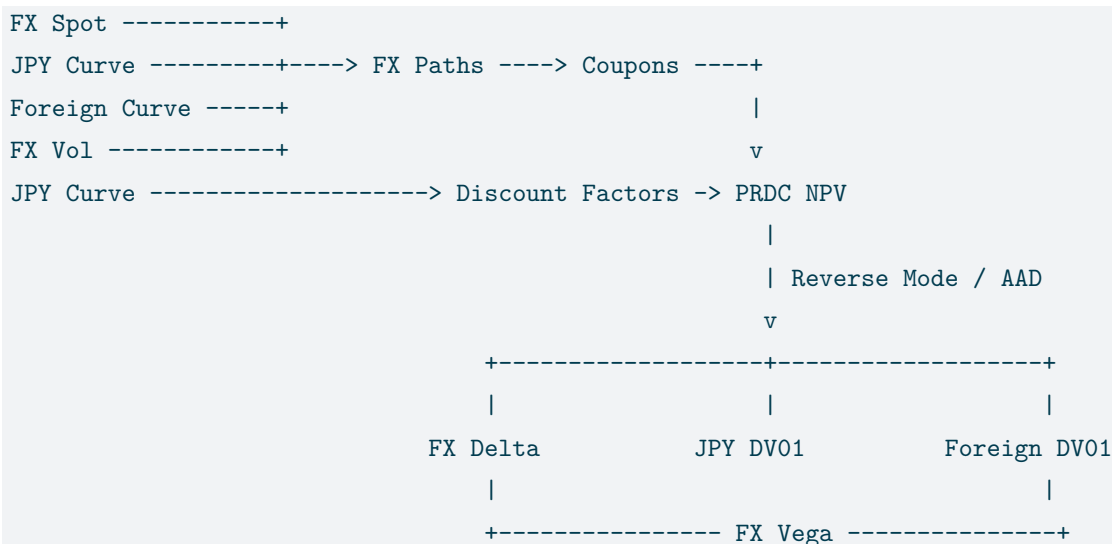
$$\boxed{\text{Monte Carlo} + \text{GPU} + \text{Automatic Differentiation}}$$

という組み合わせが可能になる。

ここで重要なのは、GPU による Hardware Acceleration と、Automatic Differentiation による Algorithmic Acceleration が同じ Tensor 計算基盤の上で共存できることである。

AAD として見る PRDC Risk

PRDC の Pricing Graph をもう一度整理する。



Forward 計算では、

$$\text{Risk Factors} \longrightarrow \text{PRDC NPV}$$

を求める。

Reverse 計算では、

$$\text{PRDC NPV} \longrightarrow \text{Risk Factors}$$

へ勾配を伝播する。

この Forward と Reverse の組み合わせが、PRDC を Differentiable Pricing Model として扱うという意味である。

第 III 部を通したリスク計測の拡張

第 III 部で扱った商品を並べると、自動微分の役割が明確になる。

Option

$$V = V(S, \sigma, T)$$

↓

$$\Delta, \Gamma, Vega, Theta$$

Interest Rate

$$V = V(r_1, r_2, \dots, r_n)$$

$$\Downarrow$$

$$DV01, \text{ Bucketed } DV01, \text{ GPS}$$

PRDC

$$V = V(S_{FX}, \mathbf{r}_{JPY}, \mathbf{r}_{Foreign}, \sigma_{FX})$$

$$\Downarrow$$

$$\Delta_{FX}, \mathbf{DV01}_{JPY}, \mathbf{DV01}_{Foreign}, Vega_{FX}$$

つまり、商品が複雑になっても基本的な数学は変わらない。

$$\text{Risk} = \text{Derivative of Price}$$

である。

変わるのは、価格関数を構成する Computational Graph の規模である。

まとめ

本章では、PRDC を題材として、FX Spot、JPY Curve、Foreign Curve、FX Volatility を同時に持つ多因子 Monte Carlo Pricing を R の `torch` で構成した。Pricing Function を Tensor 演算だけで接続することで、Monte Carlo Path 生成、Coupon 計算、Discounting、NPV までを一つの Differentiable Computational Graph として扱うことができた。

その結果、一度の Forward Pricing と Reverse Pass から、

$$\nabla V = (\Delta_{FX}, \mathbf{DV01}_{JPY}, \mathbf{DV01}_{Foreign}, Vega_{FX})$$

を取得した。ここで重要なのは、Risk Factor ごとに Pricing Engine を何度も呼び出す Bump & Revalue から、**Pricing Graph そのものが Risk を生成する構造**へ移行したことである。

第 III 部全体を振り返ると、前半では `future` / `furrr` を中心に Task を分割し、複数 Core・複数 Worker・分散環境へ計算を配布する方法を扱った。これは計算資源を増やすことによる **Hardware / Infrastructure Acceleration** である。後半では `torch` を用いて、価格計算に Reverse-mode Automatic Differentiation を組み込み、感応度計算の方法そのものを変えた。これは **Algorithmic Acceleration** である。

したがって、第 III 部の到達点は単なる Parallel Computing でも、Automatic Differentiation の数式例でもない。大量の Pricing を柔軟な計算資源へ配布し、その Pricing Graph から多数の Risk Sensitivity を同時に生成できる計算基盤である。

Scale Up / Scale Out + Automatic Differentiation = Enterprise Differentiable Risk Engine

これが、非同期並列化と自動微分を組み合わせたクオন্ツのエンタープライズ実装である。